LLD Handbook — Vol 4: Case Studies II

OOP · UML · SOLID · Clean Code · Refactoring · Design Patterns · Concurrency · 45+ case studies in Java

4 parts · one place to study LLD: DesignGurus courses + classic books + GitHub problem sets, combined

what it is | problem | how it works | trade-offs | real world | interview | self-check | TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 17 — Case Studies: Games & Rules**
   - Design Blackjack and a Deck of Cards
   - Design Chess
   - Design Tic-Tac-Toe
   - Design Snake and Ladder
   - Design 2048
   - Design a Jigsaw Puzzle
   - Chapter summary
2. **Chapter 18 — Case Studies: Platforms & Feeds (Part 1)**
   - 18.0 What platform problems test
   - 18.1 Design Stack Overflow
   - 18.2 Design Cricinfo
   - 18.3 Design LinkedIn
   - 18.4 Design Facebook
   - 18.5 Chapter summary
3. **Chapter 19 — Case Studies: Commerce & Marketplaces (Part 2 of Platforms)**
   - Design Amazon — Online Shopping System
   - Design an Online Stock Brokerage System
   - Design an Online Auction System
   - Design a Music Streaming Service (Spotify)
   - Design an Online Food Delivery Service (Swiggy) + FoodKart
   - Design a Grocery Store (POS & Checkout)
   - Chapter summary
4. **Chapter 20 — Case Studies: Pattern-Driven Components**
   - 20.1 Design Splitwise
   - 20.2 Design a Notification Service
   - 20.3 Design an In-Memory Cache (and an LRU Cache from scratch)
   - 20.4 Design a Ride Sharing Service (Uber)
   - 20.5 Design a Logging Framework
   - 20.6 Design a Pub-Sub System
   - 20.7 Design a Rate Limiter
   - 20.8 Chapter summary
   - 20.9 Chapter self-check
   - 20.10 Chapter TL;DR

Chapter 17 — Case Studies: Games & Rules

> **Where this fits.** Chapters 14–16 designed things that model the real world (an ATM, a parking lot, booking systems). Games are different: the "world" is a set of **rules**, and the rules are exact. That makes games a favourite interview topic. There is nowhere to hide a vague class, because every rule must live *somewhere*, and the interviewer will ask where.

What games test in an LLD interview

Skill	What it means	Where you see it in this chapter
Rule encapsulation (hiding a rule inside the class that owns it)	Each rule lives in one obvious class, not spread across `if` statements everywhere.	Blackjack scoring in `BlackjackScoring`; chess legality in `Rules`; 2048 merging in `LineMerger`.
Turn management	Whose turn is it, what may they do now, and what happens when the turn ends?	A turn index (tic-tac-toe), a queue of players (snake and ladder), `turn` colour (chess), rounds (blackjack).
Polymorphic pieces	Different kinds of objects answer the same question in their own way (*polymorphism* = "many shapes": one method call, many behaviours).	Chess `Piece.targets(...)` overridden by King, Knight, Pawn and the sliding pieces.
Pluggable strategies	Behaviour that varies (how a bot plays, how dice roll, when the game ends) sits behind an interface (a list of methods with no fixed implementation; the **Strategy** pattern, Chapter 10).	`PlayerStrategy`, `DealerPolicy`, `MoveStrategy`, `Dice`, `OverflowRule`, `WinStrategy`, `TileSpawner`.
Deterministic testing (same input, same result every time)	Randomness is *injected* (handed in from outside). Pass `new Random(seed)` and the "random" sequence is the same on every run, so tests and demos repeat exactly. Or pass a scripted source (stacked shoe, loaded dice).	Every `Main` in this chapter prints the same output every time.
State and status	A game moves through states (in progress, won, draw, over). Illegal actions in the wrong state are rejected.	`GameStatus`, `GameState` enums (types with a fixed list of named values) in every game.

> **Key idea: seeded randomness.** `java.util.Random` is *pseudo-random*: its numbers are computed from a starting **seed**, and the same seed always gives the same sequence (the algorithm is fixed by the Java specification); `Collections.shuffle(list, rnd)` is deterministic for a given `Random` too. Rule: **never call `new Random()` or `Math.random()` inside the game — accept a `Random` (or a `Dice` / `Spawner` interface) from outside.** Production passes an unseeded source (or `SecureRandom`, slower but unpredictable, for security); tests pass a seed. This is plain Dependency Injection (Chapter 5).

Problems in this chapter

#	Problem	Main ideas
17.1	Blackjack and a Deck of Cards	generic deck, shoe, soft/hard hand values, dealer rule, betting, split, double, insurance
17.2	Chess	board, polymorphic pieces, legal moves, check/checkmate/stalemate, castling, en passant, promotion, undo
17.3	Tic-Tac-Toe	N×N board, O(1) win detection with counters, pluggable players
17.4	Snake and Ladder	board config and validation, seeded dice, rules engine, pluggable win rules
17.5	2048	slide/merge logic, seeded tile spawns, game-over detection
17.6	Jigsaw Puzzle	pieces with flat/tab/blank sides, rotation, one placement check, validation, swappable solvers (brute force vs edge-first)

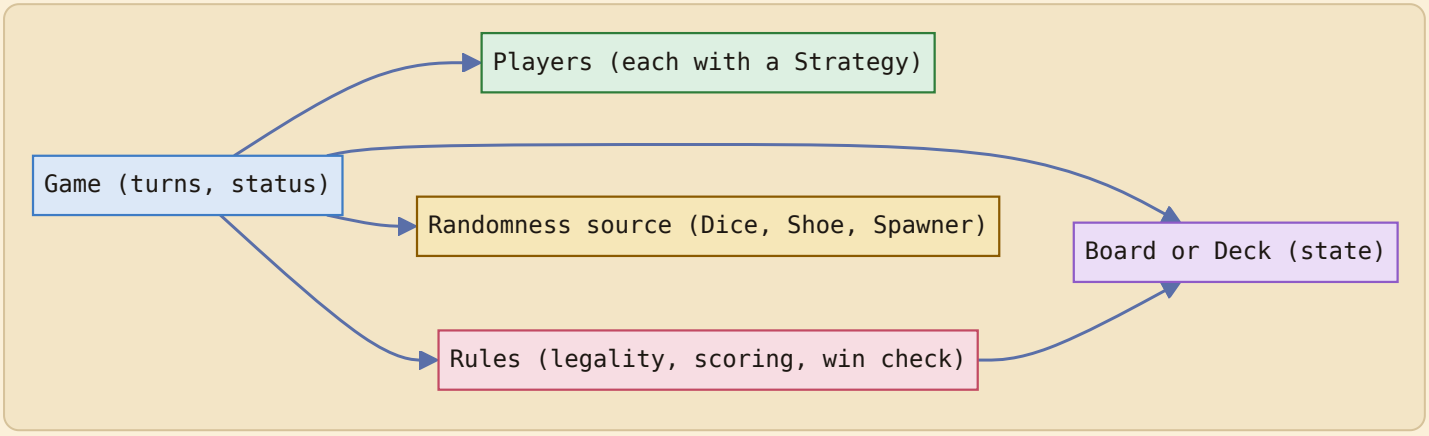
> **Key term — Big-O notation** describes how work grows with input size: **O(1)** (constant) stays the same, **O(n)** grows in step with n, **O(N²)** quadruples when N doubles.

How to run every example in this chapter

```
cd chapters/17-case-games/code
javac --release 21 -d /tmp/lld_17 $(find . -name '*.java')
java -cp /tmp/lld_17 lld.ch17.blackjack.Main    # or chess, tictactoe, snakeladder, game2048, jigsaw
```

`javac` compiles, and `-cp` (the classpath) tells `java` where the compiled classes are. All six `Main` classes use only the Java 21 standard library, and all randomness is seeded or scripted, so the output under each program is exactly what you will see. Listings show only the essential lines; the complete files are in `code/`.

A reusable skeleton for any board/card game. Almost every game design in an interview has the same five parts. Name them early and the rest of the interview becomes filling in details.



Part	Question it answers	Typical class names
State	What does the table look like right now?	`Board`, `Deck`, `Shoe`, `Hand`, `Cell`
Actors	Who acts, and how do they decide?	`Player` + `PlayerStrategy` / `MoveStrategy`
Rules	Is this action allowed? What does it do? Who won?	`Rules`, `DealerPolicy`, `OverflowRule`, `WinStrategy`
Controller	Whose turn? What state is the game in?	`Game`, `GameStatus`
Randomness	Where does chance come from?	`Dice`, `Shoe`, `TileSpawner` — always injected

Design Blackjack and a Deck of Cards



Figure — A blackjack table: players each compete against the dealer (course diagram)

Problem & clarifying questions

Blackjack is the casino card game played more than any other. Nobody plays against the other people at the table: every player's cards are measured only against the cards of the **dealer** (the dealer is the casino employee who deals and plays for the house). Everybody tries to get as close to **21 points** as possible without going over. Going over 21 is called **busting** and loses at once.

The interviewer often phrases it as "Design a deck of cards, then use it to build blackjack". The first half checks that you keep the deck **generic**; the second half checks rule placement.

Questions worth asking (with the answers this design assumes):

Question	Assumed answer
One deck or several?	A **shoe** holding several standard 52-card decks (casinos use 6–8). One deck is just a shoe of 1.
How many players?	One or more players, each against the dealer.
Which player options?	Hit, stand, double down, split (once), insurance. Surrender and side bets are out of scope.
Dealer rule?	Hit on 16 or less, stand on 17 or more. Some casinos also hit a "soft 17" — make it pluggable.
Payouts?	Win 1:1, blackjack 3:2, tie ("push") returns the bet, insurance 2:1 (odds like 3:2 mean you win 3 chips for every 2 bet).
Money?	Bets are counted in whole **chips** (an `int`, a whole-number type), and the ante (the starting bet) must be even. Never use `double` (an inexact binary floating-point number) for money.
Real UI (user interface — the screens), network, accounts?	Out of scope. The course model has `Account`/`Person` for sign-in; we keep `Player` minimal and say where it would attach.

Requirements

Background rules

- A standard deck has **13 ranks** (A, 2–10, J, Q, K) in **4 suits** (clubs, diamonds, hearts, spades). Blackjack uses one or more such decks.
- At the start, each player and the dealer receive **two cards**. Both player cards are face up. The dealer has one card face up (the **up card**) and one face down (the **hole card**), so players decide with incomplete information.
- **Card points:** 2–10 are worth their number. J, Q, K are worth 10. An **ace is worth 1 or 11**, whichever helps.
- **Soft vs hard total:** when an ace is counted as 11 the total is **soft** (it can still "soften" to 1 later, so one more card can never bust it). When every ace counts as 1 the total is **hard**. Example: A+5 is "soft 16" (or hard 6).
- **Blackjack** (also called a *natural*): exactly two cards totalling 21, i.e. an ace plus a 10-point card.
- **Dealer rule:** the dealer has no choices. Hit (take a card) while the hand is 16 or less; stand at 17 or more.

Functional requirements

1. A player must place an initial bet (the **ante**) to play a round.
2. Deal two cards to each player and to the dealer (one up, one hole).
3. If the dealer's up card is an **ace**, offer **insurance**: a side bet of half the ante that pays **2:1** if the dealer has blackjack. After that, the dealer checks ("peeks at") the hole card. If it is blackjack, the round ends at once: insurance wins, main bets lose, except a player blackjack, which ties.
4. A player whose hand is below 21 chooses:
    - **Hit** — take one more card, and choose again.
    - **Stand** ("stand pat") — take no more cards.
    - **Double down** — double the bet (the extra must equal the ante), take exactly **one** more card, and stop.
    - **Split** — only when the first two cards have the same rank: pay a second bet equal to the ante, split the pair into two hands, deal one card to each, and play each hand separately. No re-splitting.
5. A hand over 21 is **resolved immediately as a loss**.
6. The dealer then plays by the fixed rule.
7. **Resolution:** if the dealer busts, every non-bust hand wins. Otherwise the higher total wins; equal totals are a **push** (bet returned). A winning blackjack pays **3:2**; every other win pays **1:1**.
8. **Even money** (course requirement): if a player holds blackjack and the dealer shows an ace, the player may accept an immediate 1:1 payout instead of risking a push. (It is mathematically identical to buying insurance with a blackjack, so we explain it rather than build it — see Extensions.)

Non-functional requirements

- **Generic deck:** the `Card` / `Deck` classes must work for any card game, not just blackjack.
- **Deterministic tests:** shuffling uses an injected `Random`; a *stacked* shoe (fixed card order) makes exact scenarios reproducible.
- **Extensible rules:** dealer rule and player decision-making are pluggable.
- **Correct money:** bets and balances are whole chips in an `int`, never a `double`. This is the one place in the handbook where money is not a `BigDecimal`, deliberately: a table only accepts whole chips, and an **even ante** makes every payout whole (3:2 on 10 is 15; insurance is half the ante, 5, and pays 2:1 = 10). If fractions ever appear (3:2 on an odd ante, converting chips to currency), switch to `BigDecimal` with an explicit `RoundingMode` (e.g. round half up).

Use cases

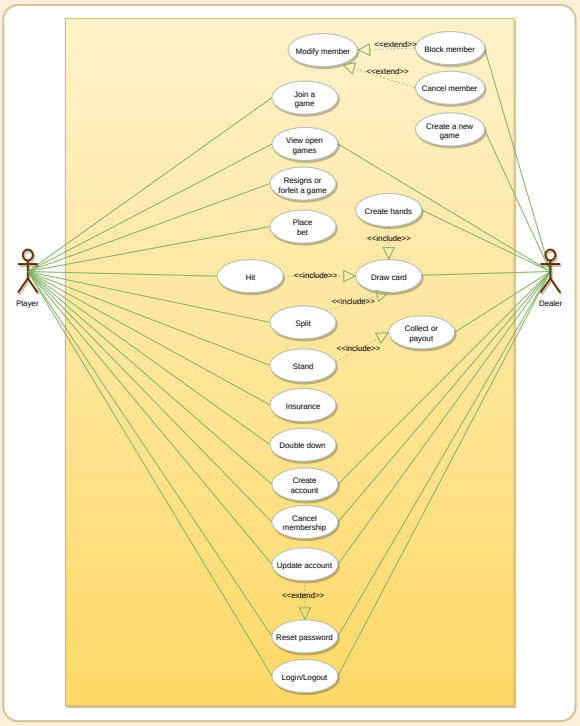


Figure — Blackjack use cases for the Player and the Dealer, including account management (course diagram)

The figure above is a **use case diagram** (UML — Unified Modeling Language — notation: stick figures are *actors*, the people or systems using the software; ovals are *use cases*, the goals they can achieve). In the figure above there are two actors:

- **Player** — places the initial bet; accepts or declines extra bets (insurance, split, double); accepts or declines an offered resolution such as even money; chooses hit, double or stand. The player also has account-level use cases (create account, log in, join or view games), which belong to a lobby service (the separate part of the app where players sign in and pick a table), not the game engine (the code that enforces the game's rules).
- **Dealer** — deals cards, plays the house hand by the fixed rule, collects losing bets and pays winners.

The main use cases — create hands, place bet, player plays the hand, dealer plays the hand, insurance / even money / examine hole card, split, game resolution — are exactly the functional requirements above. The `<<include>>` arrows mean one use case always runs another as part of itself: dealing and payout are part of every round. When insurance is offered and the hole card is not a 10-value card, insurance loses and the hole card stays hidden.

Activity: hit or stand

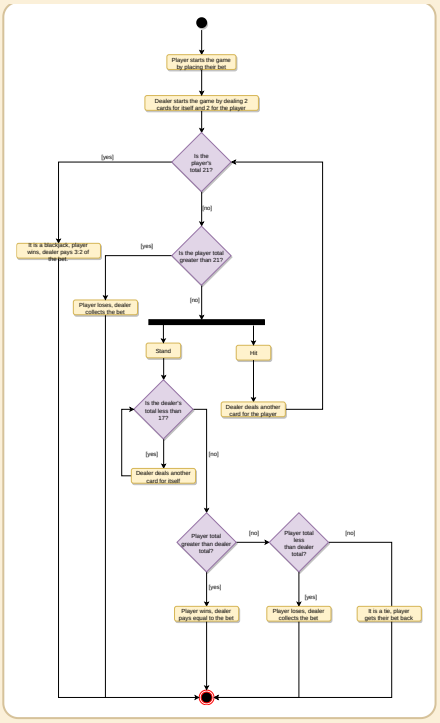


Figure — Activity diagram for one player's hit-or-stand round against the dealer (course diagram)

An **activity diagram** is a flowchart of one task, with diamonds for decisions. The figure above walks through one round: bet, deal, check for 21 (blackjack pays 3:2), check for bust, hit or stand, then the dealer draws while below 17, and finally the three outcomes — player wins 1:1, player loses, or a tie returns the bet.

Core entities

Class	Responsibility
`Suit`, `Rank` (enums)	The fixed values of a French deck (the standard 52-card deck with four suits). `Rank` stores the *face value* 1–13 (ace = 1, J = 11, Q = 12, K = 13).
`Card` (record — a compact class that just holds fixed values)	Rank + suit. Knows **nothing** about blackjack. Immutable (cannot change after creation).
`Deck<T>`	A generic (works for any type `T` you plug in), shuffleable stack of any card type `T`; `standard52()` factory (a method that builds objects) builds a French deck.
`Shoe`	Several decks shuffled together; deals one card at a time; reshuffles at the **cut card** (a marker card placed near the end of the shoe). `Shoe.stacked(...)` gives a fixed order for tests.
`BlackjackScoring`	The blackjack value of a rank (K = 10, ace = 1 hard). The *game* rule, kept off `Card`.
`Hand`	Cards + bet; computes hard/soft/best totals, bust, blackjack, can-split; splits and doubles.
`Action` (enum)	HIT, STAND, DOUBLE, SPLIT.
`PlayerStrategy` (interface)	How a player decides. `BasicStrategy` and `MimicDealerStrategy` are two bots (computer-controlled players); a UI-backed human would be a third.
`DealerPolicy` (interface)	When the dealer hits. `standOnAll17()` and `hitSoft17()`.
`Player`	Name, chip balance, strategy.
The dealer	Not a class of its own here: during a round the dealer is a `Hand` inside `BlackjackGame` plus the `DealerPolicy` that decides when it draws. (The course has a `Dealer` class next to `Player`, both under `BasePlayer`.)
`BlackjackGame`	Runs a round: bets, deal, insurance + peek, player turns, dealer turn, settlement.

Class diagram

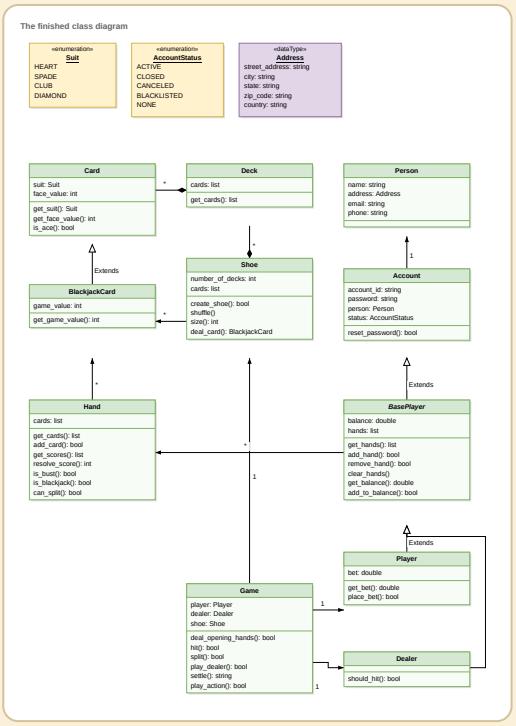
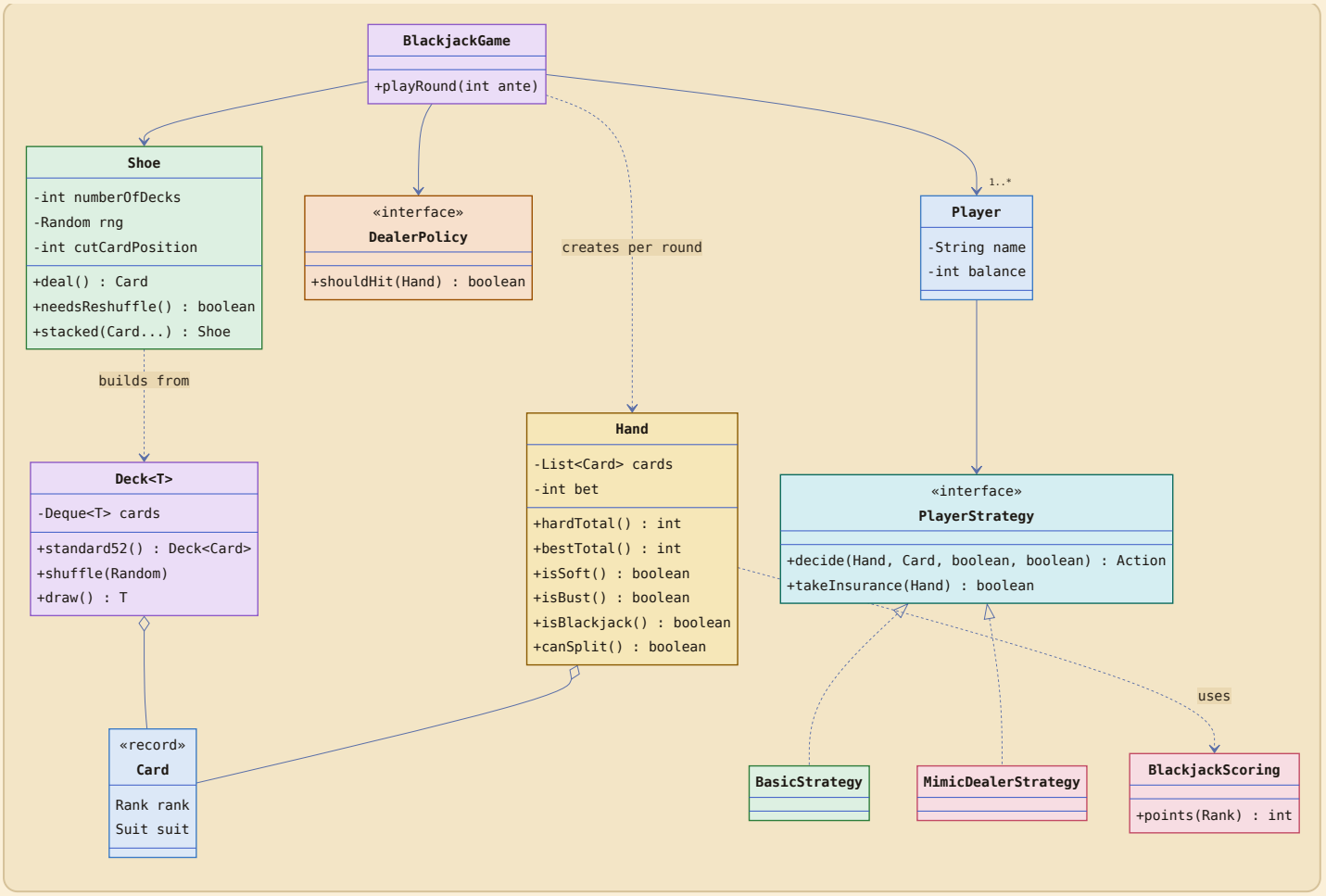


Figure — Course class diagram: Card, BlackjackCard, Deck, Shoe, Hand, BasePlayer, Player, Dealer, Game, Account (course diagram)

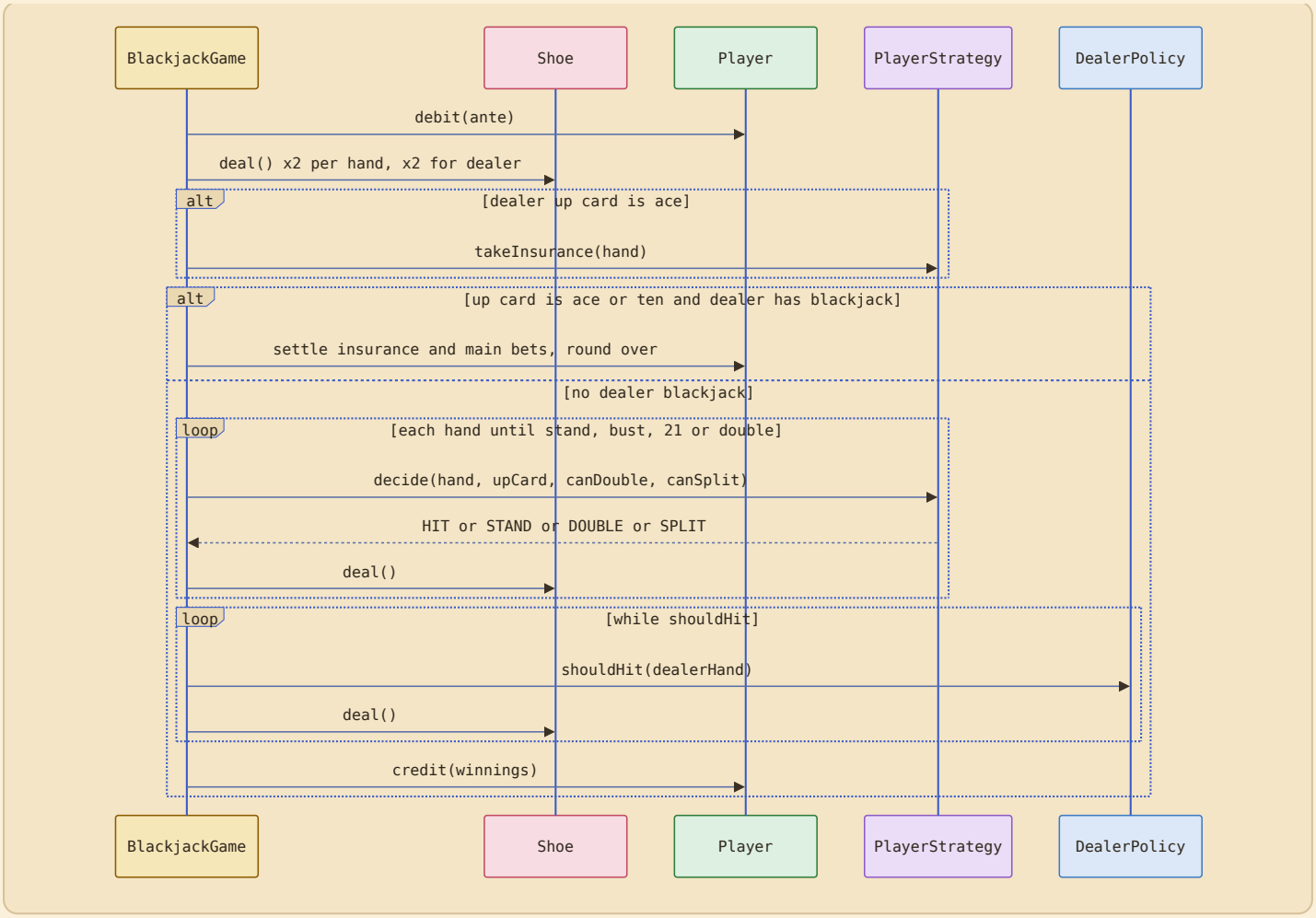
In the course model, `Card` holds a suit and a face value and a subclass `BlackjackCard` adds the *game value* (a king: face value 13, worth 10). A `Shoe` is composed of several `Deck`s (filled diamond = composition; plain arrow = association; open triangle = inheritance). `Hand` scores itself. `Player` (adds the bet) and `Dealer` (adds the "should I hit?" rule) share a `BasePlayer` (balance, hands) that extends `Account`; the `*` from `BasePlayer` to `Hand` is because splitting gives one player several hands. `Game` points at a player, a dealer and a shoe and holds the order of play.

Our Java design keeps the same core idea — **the game value must not live on the generic card** — but implements it with a separate scoring function instead of a subclass, and turns the two things that vary (player decisions, dealer rule) into strategies:

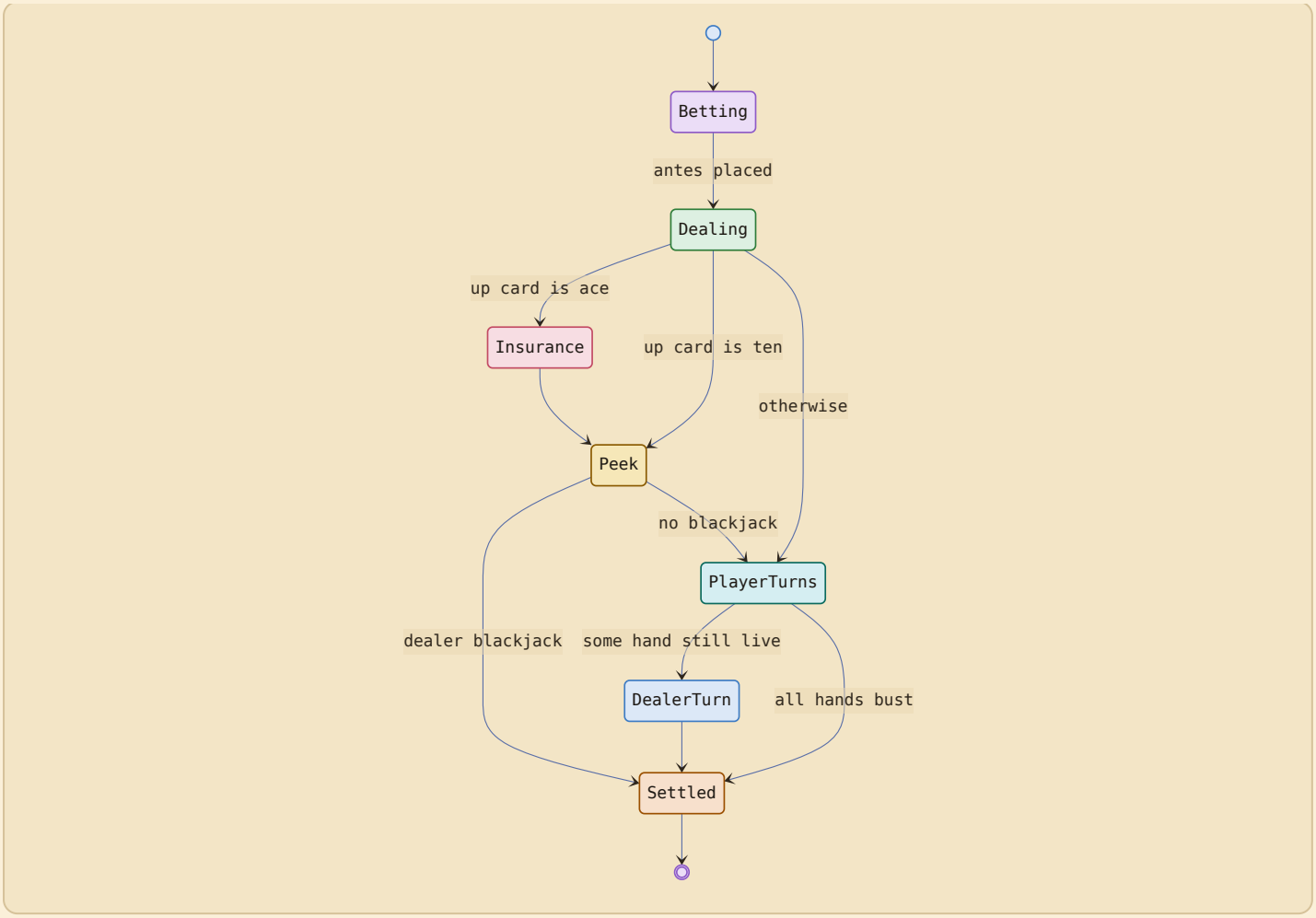


Key flows

One round (sequence)



States of one round



Scoring a hand (the ace problem). The trick is that **at most one ace can ever count as 11** (two elevens would be 22, a bust). So:

1. Add up every card with the ace as 1. This is the **hard total**.
2. If the hand has at least one ace and `hard + 10 ≤ 21`, one ace can be 11: the hand is **soft** and the best total is `hard + 10`.
3. Otherwise the best total is the hard total.

This is O(n) in the number of cards, needs no search over combinations, and gives the soft/hard flag for free (the dealer's "hit soft 17" rule needs it).

Design decisions & patterns used

Decision	Why	Alternative rejected
`Card` is a game-agnostic **record** (rank + suit).	A deck of cards is reused by poker, bridge, solitaire. A king's blackjack value (10) is a *rule of this game*, not a fact about the card.	Putting `getValue()` returning 10 for K on `Card` — breaks every other game.
Blackjack value in `BlackjackScoring`, not a `BlackjackCard` subclass.	Records cannot be extended, and the value depends only on the rank. A function is simpler and keeps one card type in the shoe.	The course's `BlackjackCard extends Card` — also valid, and also keeps `Card` clean. Say both in the interview; the important point is the same.
`Deck<T>` is **generic**.	"Design a deck of cards" follow-up: Uno or tarot cards reuse the same shuffle/draw code.	A deck hard-wired to `Card`.
Strategy for players (`PlayerStrategy`) and dealer (`DealerPolicy`).	Human UI, bots and test scripts plug in; casino variants (S17 vs H17: dealer stands on soft 17 vs hits soft 17) are one line. Open/Closed principle (add new behaviour with new classes, without editing working code).	`if (isHuman) ... else ...` and `if (casino == X)` branches inside the game.
Factory methods `Deck.standard52()`, `Shoe.stacked(...)`, `DealerPolicy.standOnAll17()`.	Named constructors make intent obvious.	Big constructors with flags.
Randomness injected (`Random` passed into `Shoe`).	Seeded shuffles make demos and tests exact.	`Collections.shuffle(list)` with a hidden random source.
`Hand` answers with `hardTotal()`, `bestTotal()` and `isSoft()`.	The course's `Hand` returns *every* possible total (A+6 is both 7 and 17) and lets the caller choose. Because at most one ace can ever count as 11, the hard total plus the soft flag already carries all of that information, and most callers just want `bestTotal()`.	Returning a list of totals that each caller must filter.
Hand stores its **own bet**.	Split and double change the bet *per hand*, not per player.	One bet per player — breaks as soon as a player splits.
The game **validates** a strategy's choice (`canDouble`, `canSplit` passed in, and re-checked).	A buggy or malicious client (the program sending the player's choices) cannot double without money.	Trusting the client.

Honest weaknesses (and what a longer design would do). `BlackjackGame.playRound` owns the whole sequence of play *and* the payout rules. Each new option (surrender, side bets) adds another branch. A larger design would model the round as a **State** machine (a fixed set of states plus the allowed moves between them; the State pattern gives each state its own class — one class per state in the diagram above, like the vending machine in Chapter 14) and move payouts into a `PayoutTable`. Say this out loud; interviewers reward knowing where your design will strain.

Java implementation

Package `lld.ch17.blackjack`; full runnable code (one file per class in Core entities, plus `Main`) is in `code/lld/ch17/blackjack/`. The essential lines (`/* ... */` and `//` summary comments stand for code left out):

```java
/** A generic deck: any card type T. Shuffling takes a Random so tests can pass a seeded one. */
public final class Deck<T> {
    /** Factory for the standard 52-card French deck (13 ranks x 4 suits). */
    public static Deck<Card> standard52() { /* every Suit x every Rank */ }
    public void shuffle(Random rng) { /* Collections.shuffle(list, rng) */ }
    public T draw() { /* NoSuchElementException when empty */ }
}

/** Several decks shuffled together. A "cut card" marks how deep we deal before reshuffling (penetration). */
public final class Shoe {
    public Shoe(int numberOfDecks, Random rng, double penetration) {
        if (penetration <= 0 || penetration >= 1) throw new IllegalArgumentException("penetration must be in
(0,1)");
        this.cutCardPosition = (int) Math.round(numberOfDecks * 52 * (1 - penetration));
        refill();
    }
    /** Cards come out in exactly the order given. Perfect for unit tests. */
    public static Shoe stacked(Card... order) { return new Shoe(List.of(order)); }
    public boolean needsReshuffle() { return rng != null && cards.size() < cutCardPosition; }
}

/** The blackjack value of a card lives here, NOT on Card: a king has face value 13 but is worth 10 here. */
public final class BlackjackScoring {
    public static int points(Rank rank) {
        return switch (rank) { case ACE -> 1; case JACK, QUEEN, KING -> 10; default -> rank.faceValue(); };
    }
}
```

```java
/** A blackjack hand: its cards, its own bet, and how to score itself (aces = 1 or 11). */
public final class Hand {
    public int hardTotal() { /* every ace counted as 1 */ }
    /** Soft when one ace can count as 11 without busting. At most one ace can ever do so. */
    public boolean isSoft() { return hasAce() && hardTotal() + 10 <= 21; }
    public int bestTotal() { return isSoft() ? hardTotal() + 10 : hardTotal(); }
    public boolean isBust() { return bestTotal() > 21; }
    /** A natural: exactly two cards worth 21, and not the result of a split. */
    public boolean isBlackjack() { return !fromSplit && cards.size() == 2 && bestTotal() == 21; }
    /** Two cards of the same rank, and no re-splitting (one split per round). */
    public boolean canSplit() { return !fromSplit && cards.size() == 2 && cards.get(0).rank() ==
cards.get(1).rank(); }
    Hand split()      { /* move 2nd card into a new Hand(bet); both marked fromSplit */ }
    void doubleDown() { bet *= 2; doubled = true; }
}

/** Strategy: HOW a player decides is pluggable. The game passes what is allowed. */
public interface PlayerStrategy {
    Action decide(Hand hand, Card dealerUpCard, boolean canDouble, boolean canSplit);
    /** Offered only when the dealer shows an ace. Mathematically a bad bet for most players. */
    default boolean takeInsurance(Hand hand) { return false; }
}
// BasicStrategy: a simplified basic-strategy table (always split aces and eights; double on 11, or 10 vs 2-9;
...).
// MimicDealerStrategy: a naive player who hits below 17 like the dealer and always buys insurance.

/** The house rule for when the dealer draws. Casinos differ, so it is a pluggable strategy. */
@FunctionalInterface
public interface DealerPolicy {
    boolean shouldHit(Hand dealerHand);
    static DealerPolicy standOnAll17() { return h -> h.bestTotal() <= 16; }                              // "S17"
    static DealerPolicy hitSoft17()    { return h -> h.bestTotal() <= 16 || (h.bestTotal() == 17 && h.isSoft()); }
// "H17"
}
```

```
public final class BlackjackGame {
    public void playRound(int ante) {
        if (ante <= 0 || ante % 2 != 0) throw new IllegalArgumentException("ante must be a positive even number");
        if (shoe.needsReshuffle()) shoe.reshuffle();
        // 1. Bets: each player who can afford the ante gets one hand (others sit out).
        // 2. Deal two rounds: one card to each player, then one to the dealer (2nd dealer card = hole card).
        // 3. Insurance (only when the up card is an ace, side bet = ante / 2); the dealer peeks when the up card
        //    is an ace or a ten-value. Dealer blackjack: insurance pays 2:1, player blackjacks push, the rest
lose.
        // 4. Player turns: playHand(...) for each hand; splitting inserts a new hand right after the current one.
        // 5. Dealer turn, skipped if every player hand is already bust:
        //        while (dealerPolicy.shouldHit(dealer)) dealer.add(shoe.deal());
        // 6. Settle every hand against the dealer.
    }

    private void playHand(Player p, List<Hand> hs, int index, Card up) {
        Hand hand = hs.get(index);
        String label = p.name() + (hs.size() > 1 ? " hand " + (index + 1) : "");
        while (!hand.isBlackjack() && hand.bestTotal() < 21) {
            boolean canDouble = hand.size() == 2 && p.canAfford(hand.bet());
            boolean canSplit = hand.canSplit() && p.canAfford(hand.bet());
            Action a = p.strategy().decide(hand, up, canDouble, canSplit);
            if ((a == Action.DOUBLE && !canDouble) || (a == Action.SPLIT && !canSplit))
                throw new IllegalStateException(label + " chose a move that is not allowed: " + a);
            switch (a) {
                case STAND -> { return; }
                case HIT -> hand.add(shoe.deal());
                case DOUBLE -> { p.debit(hand.bet()); hand.doubleDown(); hand.add(shoe.deal()); return; } //
exactly one card
                case SPLIT -> {
                    p.debit(hand.bet());
                    Hand second = hand.split();
                    hand.add(shoe.deal());
                    second.add(shoe.deal());
                    hs.add(index + 1, second);
                }
            }
        }
    }

    private void settle(Player p, Hand h, Hand dealer) {
        if (h.isBust()) return;                                  // LOSE the bet (already debited)
        if (h.isBlackjack()) { p.credit(h.bet() + h.bet() * 3 / 2); return; }   // pays 3:2
        int mine = h.bestTotal(), theirs = dealer.bestTotal();
        if (dealer.isBust() || mine > theirs) p.credit(h.bet() * 2);            // WIN
        else if (mine == theirs) p.credit(h.bet());                             // PUSH
    }
}
```

`Main.java` prints five sample hands, then plays a stacked shoe where Alice (`BasicStrategy`) splits 8s and doubles both hands, a stacked ace-up round where Bob (`MimicDealerStrategy`) buys insurance, and five rounds from a seeded 6-deck shoe (`new Random(42)`, penetration 0.75, `hitSoft17`) for Carol and Dave.

Output:

```
== 1. Hand values (aces are 1 or 11) ==
  [AS 6H] = soft 17   (hard total 7)
  [AS 6H 10C] = 17   (hard total 17)
  [AS AH 9C] = soft 21   (hard total 11)
  [KS QH 5C] = bust 25   (hard total 25)
  [AD KC] = BLACKJACK   (hard total 11)

== 2. Stacked shoe: split 8s, then double both hands ==
  Dealer shows 6S (hole card hidden)
  Alice [8H 8D] = 16
  Alice splits -> [8H 3C] = 11 and [8D 2S] = 10
  Alice hand 1 doubles (bet now 20): [8H 3C 9H] = 20
  Alice hand 2 doubles (bet now 20): [8D 2S 5D] = 15
  Dealer reveals and plays: [6S 10C 7S] = bust 23
  Alice [8H 3C 9H] = 20 vs dealer 23: WIN +20
  Alice [8D 2S 5D] = 15 vs dealer 23: WIN +20
  Balances: Alice=140

== 3. Stacked shoe: dealer shows an ace, Bob buys insurance ==
  Dealer shows AS (hole card hidden)
  Bob [10H 9C] = 19
  Bob buys insurance for 5
  Dealer peeks: [AS KD] = BLACKJACK -> round over
  Bob insurance pays 2:1 (+10)
  Bob: LOSE 10
  Balances: Bob=100

== 4. Seeded 6-deck shoe (seed 42), two players, 5 rounds ==
Round 1:
  Dealer shows AS (hole card hidden)
  Carol [8S 3S] = 11
  Dave [5D 9C] = 14
  Dave buys insurance for 5
  Dealer peeks: [AS QS] = BLACKJACK -> round over
  Dave insurance pays 2:1 (+10)
  Carol: LOSE 10
  Dave: LOSE 10
  Balances: Carol=90 Dave=100
Round 2:
  Dealer shows 6D (hole card hidden)
  Carol [2S 8H] = 10
  Dave [QS 10H] = 20
  Carol doubles (bet now 20): [2S 8H QD] = 20
  Dave stands on 20
  Dealer reveals and plays: [6D 6D 7C] = 19
  Carol [2S 8H QD] = 20 vs dealer 19: WIN +20
  Dave [QS 10H] = 20 vs dealer 19: WIN +10
  Balances: Carol=110 Dave=110
Round 3:
  Dealer shows 5C (hole card hidden)
  Carol [AD KD] = BLACKJACK
  Dave [8D AH] = soft 19
  Dave stands on 19
  Dealer reveals and plays: [5C 7D 7S] = 19
  Carol [AD KD] = BLACKJACK vs dealer 19: BLACKJACK pays 3:2 (+15)
  Dave [8D AH] = soft 19 vs dealer 19: PUSH
  Balances: Carol=125 Dave=110
Round 4:
  Dealer shows 10S (hole card hidden)
  Carol [8D 8S] = 16
  Dave [5C 9S] = 14
  Carol splits -> [8D 4S] = 12 and [8S JD] = 18
  Carol hand 1 hits: [8D 4S 7D] = 19
  Carol hand 1 stands on 19
  Carol hand 2 stands on 18
  Dave hits: [5C 9S KS] = bust 24
  Dave busts
  Dealer reveals and plays: [10S 9H] = 19
  Carol [8D 4S 7D] = 19 vs dealer 19: PUSH
  Carol [8S JD] = 18 vs dealer 19: LOSE 10
```

```
  Dave [5C 9S KS] = bust 24 vs dealer 19: LOSE 10
  Balances: Carol=115 Dave=100
Round 5:
  Dealer shows KD (hole card hidden)
  Carol [5S 8C] = 13
  Dave [8S 2C] = 10
  Carol hits: [5S 8C JS] = bust 23
  Carol busts
  Dave hits: [8S 2C 7C] = 17
  Dave stands on 17
  Dealer reveals and plays: [KD 10H] = 20
  Carol [5S 8C JS] = bust 23 vs dealer 20: LOSE 10
  Dave [8S 2C 7C] = 17 vs dealer 20: LOSE 10
  Balances: Carol=105 Dave=90
```

Reading the output. Part 1 shows the scoring: A+6 is soft 17, adding a 10 makes it a *hard* 17 (the ace dropped to 1), A+A+9 is soft 21. Part 2 uses a stacked shoe to force an exact scenario: Alice splits eights against a dealer 6, doubles both hands, and the dealer busts — she staked 40 chips and won 40. Part 3 shows insurance working as a hedge: Bob loses his main bet of 10 but the 5-chip insurance pays 10, so he breaks even. Part 4 is a seeded six-deck game: run it again and you get the same cards.

Concurrency & edge cases

- **One table = one thread.** A round is a strict sequence, so the simplest safe design runs each table's rounds on a single **thread** (one independent line of execution) or an actor/event loop (one worker processing messages from its own queue) and funnels player commands into it. Tables run in parallel and share nothing except the wallet service.
- **Balances are shared across tables** in a real casino app. Move `debit`/`credit` into a wallet service with **atomic** (indivisible) operations — a database transaction, or `AtomicLong`/`synchronized` in memory — and debit *before* dealing.
- **Odd antes and 3:2.** 3:2 on 5 chips is 7.5. We require even antes. Real casinos either round down or use smaller chip units — decide and state it.
- **Split aces.** Many casinos deal only one card to each split ace, and 21 after a split is *not* a blackjack (our `isBlackjack()` checks `fromSplit`).
- **Shoe exhaustion.** The cut card triggers a reshuffle *between* rounds; the shoe also refills if emptied mid-round. A stacked test shoe throws instead, so a wrong test fails loudly.
- **Player cannot afford a double/split.** The game passes `canDouble=false` and rejects the action if chosen anyway.
- **Dealer peek only on ace or ten.** If the up card is anything else the dealer cannot have blackjack, so there is no peek.

Extensions & follow-up questions

- **"Where would even money go?"** — It is the same as insurance for a player holding blackjack: insurance of half the ante plus a blackjack always nets exactly +1× the ante whatever the dealer has. Add `PlayerStrategy.takeEvenMoney(hand)` and, if accepted, settle that hand at 1:1 before the peek.
- **"Add surrender."** — New `Action.SURRENDER`, allowed only as the first decision; refund half the bet and end the hand.
- **"Several players at the table?"** — Already supported: the game iterates players in seat order, then the dealer plays once.
- **"Card counting detection / security?"** — Card counting is tracking which cards have gone to guess what is left. Shoe penetration (how deep into the shoe the cut card sits) already limits counting. Log every dealt card for audit (a later independent check of what happened); never send the hole card to clients until it is revealed.
- **"How would you test the dealer rule?"** — Stacked shoe + `DealerPolicy.hitSoft17()`: deal the dealer A+6 and assert it draws; with `standOnAll17()` assert it stands.
- **"Make the deck work for Uno."** — `Deck<UnoCard>` with its own factory; nothing else changes.
- **"Replace the round logic with a state machine."** — One class per state (Betting, Dealing, PlayerTurns, DealerTurn, Settled), each handling only the commands valid in that state; invalid commands are rejected by the current state.

Real-world example

Online casino tables (e.g. live-dealer blackjack) separate exactly these concerns: a certified, audited **shuffle/RNG service** (regulators such as the UK Gambling Commission require the random number generator to be independently tested), a **game engine** enforcing the table rules (number of decks, S17 or H17, payout 3:2 or 6:5 — printed on the table felt), and a **wallet** service that debits bets atomically. Rule variants are configuration, not code — exactly why `DealerPolicy` is a strategy.

In the interview

30-second answer: "A generic `Card` (rank, suit) and a generic `Deck<T>`. A `Shoe` combines several decks and deals with an injected `Random`, plus a stacked constructor for tests. The blackjack value of a card lives in the game's scoring, not on `Card`. `Hand` scores itself: hard total with aces as 1, and it's soft if one ace can be 11 without busting. Player decisions and the dealer's hit rule are strategies. `BlackjackGame` runs the round: bets, deal, insurance and peek, player turns with hit/stand/double/split, dealer turn, settle 3:2, 1:1 or push."

How to spend 30–45 minutes: 5 min questions and rules → 5 min entities → 10 min class diagram (defend `Card` vs game value, hand-owned bet) → 15 min code `Hand` scoring and `playRound` → 5 min edge cases (split aces, insurance, money) and extensions.

Likely follow-ups with answers

- *"Why not an `int value` on `Card`?"* — Because value is game-specific; poker ranks aces high, bridge uses suits, blackjack makes K = 10.
- *"How do you compute the best hand value?"* — Hard total, then add 10 if there is an ace and it fits. At most one ace can be 11.
- *"How do you make the game testable?"* — Inject the randomness: seeded `Random` for repeatability, stacked shoe for exact scenarios.

Self-check

1. **Q:** Why does the `Hand` store the bet instead of the `Player`? **A:** After a split, one player has two hands with separate bets; after a double, only that hand's bet changes.
2. **Q:** A+A+9 — soft or hard, and what total? **A:** Hard 11 with aces as 1; one ace can be 11 → soft 21.
3. **Q:** When does the dealer peek? **A:** Only when the up card is an ace or a 10-value card — otherwise blackjack is impossible.
4. **Q:** What does insurance pay, and how much is it? **A:** 2:1. In this design (and the course) it is exactly half the ante; real casinos let you buy *up to* half.
5. **Q:** Is 21 after splitting a blackjack? **A:** No, a blackjack must be the original two-card hand.

TL;DR

- Keep `Card` / `Deck` generic; the game's value rule lives in the game (`BlackjackScoring` , or a `BlackjackCard` subclass in the course version).
- Hand value: hard total + 10 if an ace fits → soft. Blackjack = original two cards totalling 21.
- Strategies for player decisions and dealer rule; inject randomness; stacked shoe for tests.
- Round = bet → deal → (insurance + peek) → player turns → dealer turn → settle (3:2, 1:1, push).

Design Chess



Figure — A chess board in play (course diagram)

Problem & clarifying questions

Chess is a two-player strategy game on an 8×8 board of 64 squares. Each side starts with 16 pieces: **8 pawns, 2 rooks, 2 knights, 2 bishops, 1 queen, 1 king**. The goal is **checkmate**: attack the enemy king so that it has no legal escape. We design an **online** two-player chess game.

Question	Assumed answer
Full international rules?	Yes: every piece's moves, captures, check, checkmate, stalemate, castling, en passant, promotion.
How are sides chosen?	Randomly assigned white or black (`Game.withRandomSides`, with an injected `Random`); **white moves first**; players alternate.
Can players take moves back?	Not in a rated game (one that counts toward the players' official ratings; course requirement). We still support `undo()` for analysis mode and tests.
How can a game end?	Checkmate, stalemate (a draw), resignation, or forfeit.
Move log?	Yes — every move by both players is recorded.
Timers, rating, spectators, draw by repetition?	Out of scope; discussed in Extensions.
AI opponent?	Out of scope; a bot would be a `MoveStrategy` that picks from `legalMoves()`.

Requirements

Functional

1. Two players play one game; each is assigned a colour, white moves first, turns alternate.
2. A player moves one of **their own** pieces; the system rejects moves out of turn, moves a piece cannot make, and moves that would leave the mover's own king in check.
3. All international rules, including the special moves:
    - **Castling** — the king moves two squares toward a rook and the rook jumps over to the square the king crossed. Allowed only if neither piece has moved, the squares between are empty, the king is not in check, and the king does not pass through or land on an attacked square.

    - **En passant** ("in passing") — if a pawn advances two squares and lands beside an enemy pawn, that enemy pawn may capture it *as if it had moved only one square*, but only on the very next move.
    - **Promotion** — a pawn reaching the last rank becomes a queen, rook, bishop or knight (the player's choice).
4. Detect **check** (the king is attacked), **checkmate** (in check and no legal move — the attacker wins) and **stalemate** (not in check but no legal move — a draw).
5. Support **resignation** (the player gives up) and **forfeit** (the player loses without choosing to, for example by leaving or timing out); record the result.
6. Keep a **log** of every move.

Non-functional

- Adding a new piece type (for a chess variant) should not touch existing pieces (Open/Closed).
- Legality checks must be correct first; speed is a secondary concern for a two-human game.
- Online play: the game object must be safe when two player connections call it concurrently (at the same time, from different threads).

Use cases

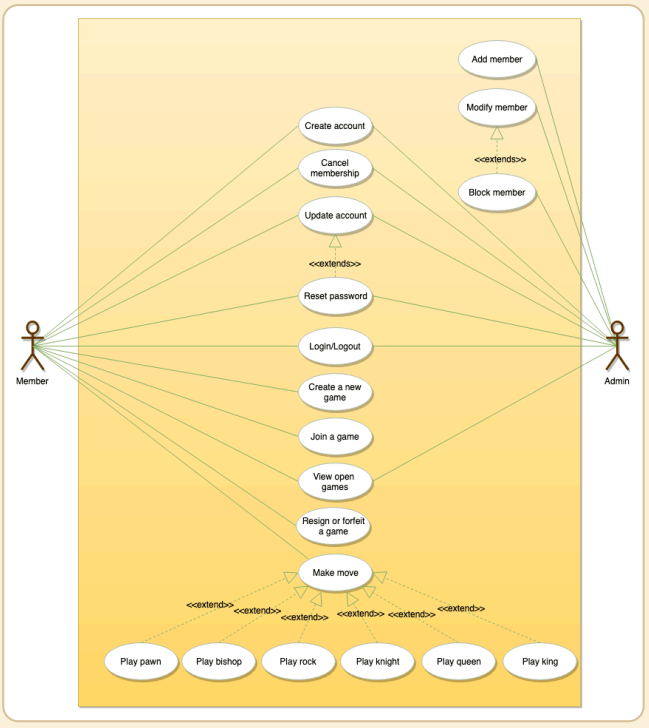


Figure — Chess use cases for Member (player) and Admin, with "Make move" extended by each piece type (course diagram)

In the figure above there are two actors:

- **Player** (a registered member) — creates or joins a game, views open games, **makes moves**, resigns or forfeits, and manages their account.
- **Admin** — adds, modifies, or blocks (bans) members.

The key use cases are **Player moves a piece** (extended per piece type — "Play pawn", "Play knight"... — which is exactly the polymorphism in our `Piece` hierarchy), **Resign or forfeit**, **Register/cancel membership**, and **Update game log** (every move is appended).

Activity: make a move

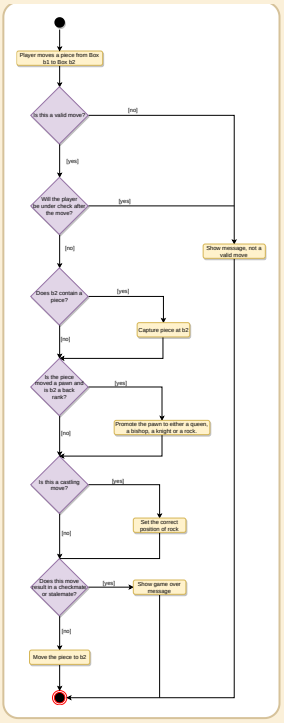


Figure — Activity diagram for making a move: validate, check self-check, capture, promotion, castling, and checkmate/stalemate (course diagram)

`Game.move` and `Game.play` follow the figure's order: reject a move invalid for that piece or one that leaves the mover's king in check; capture; promote a pawn reaching the back rank; move the rook too when castling; finally detect checkmate or stalemate.

Core entities

Class	Responsibility
`Color` (enum)	WHITE/BLACK and `opposite()`.
`Position` (record)	A square (file a–h, rank 1–8). Validates bounds; parses "e4". The course calls this a *Box*.
`Piece` (abstract — cannot be created directly, only through subclasses; sealed — only listed subclasses allowed)	Colour + `targets(board, from)`: where it can move by its own rules. `attacks(...)` for check detection.
`King`, `Knight`, `Pawn`	Step pieces and the pawn's special forward/diagonal rule.
`SlidingPiece` → `Rook`, `Bishop`, `Queen`	Share one "slide until blocked" loop; each subclass only lists its directions.
`Board`	8×8 grid of pieces; `copy()`, `kingOf(color)`, `isAttacked(square, byColor)`, `render()`. No turn or history knowledge.
`Move` (record) + `MoveType` (enum)	A move as data: from, to, type (NORMAL, CASTLE_KINGSIDE, CASTLE_QUEENSIDE, EN_PASSANT, PROMOTION), promotion piece.
`CastlingRights` (record)	The four castling permissions; how they change after a move.
`Rules`	Stateless (keeps no data between calls) rules engine: pseudo-legal moves (allowed by piece movement, before the king-safety check), legal moves (king-safety filter), special moves, `apply(board, move)`, `isInCheck`.
`Game`	Turn, status, winner, en passant square, castling rights, move log, undo (Command + Memento — patterns explained below), resign, forfeit, random side assignment. Thread-safe (correct even when several threads call it at once).
`Player` (record), `GameStatus` (enum), `IllegalMoveException`	Supporting types.

Class diagram

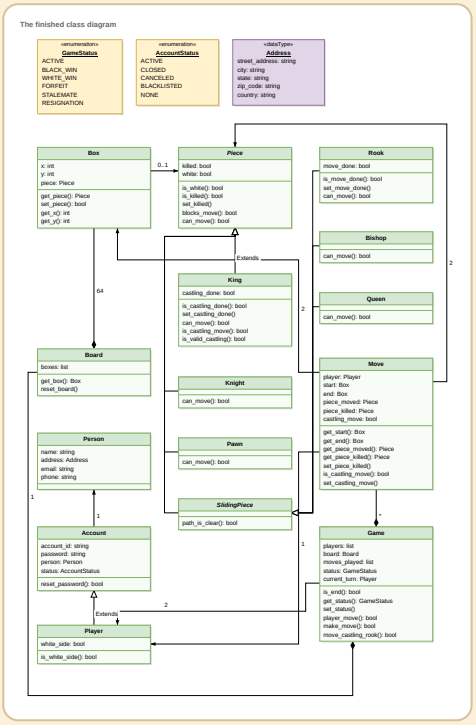
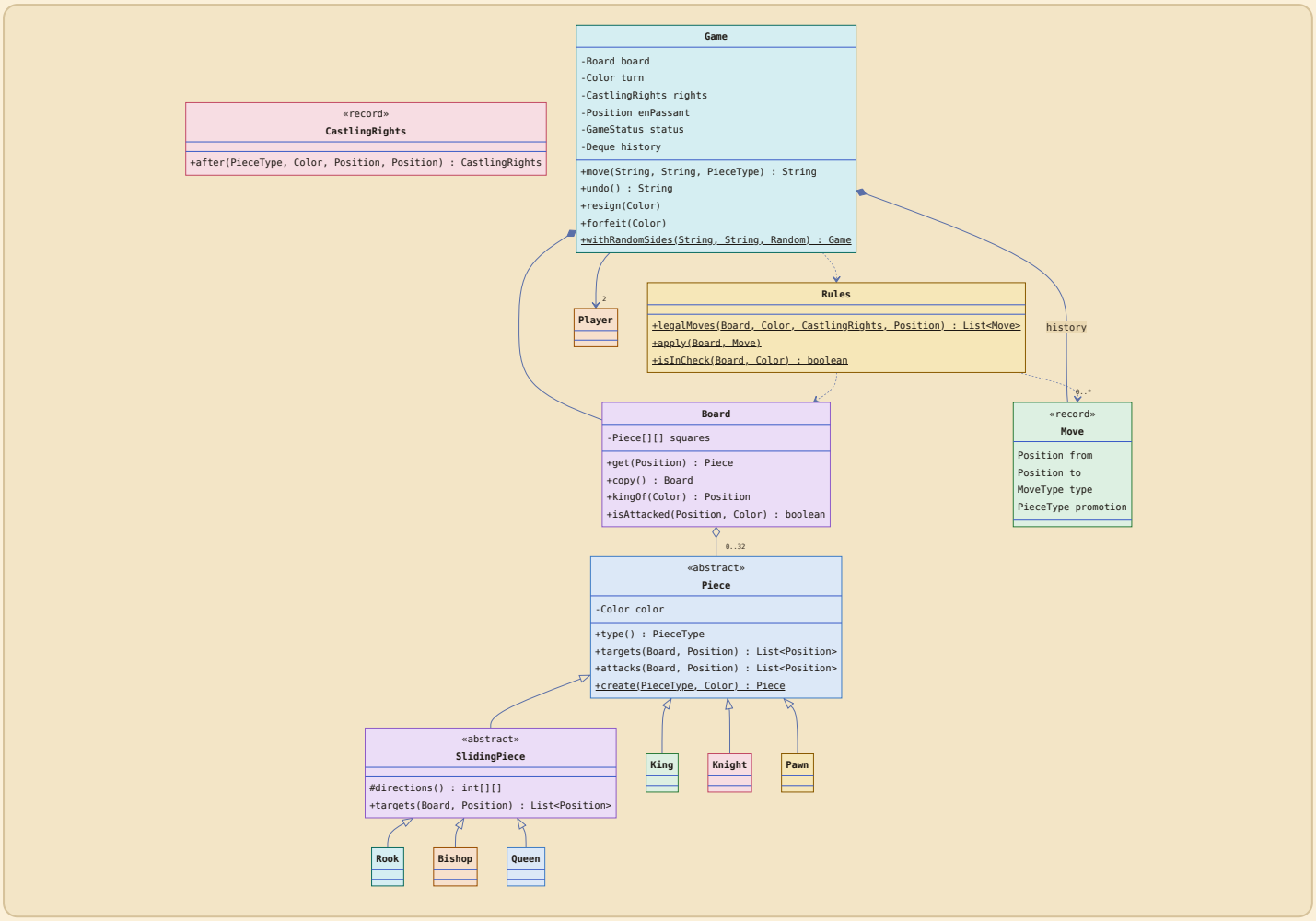


Figure — Course class diagram: Box, Board, Piece hierarchy with SlidingPiece, Move, Player, Account, Game, GameStatus (course diagram)

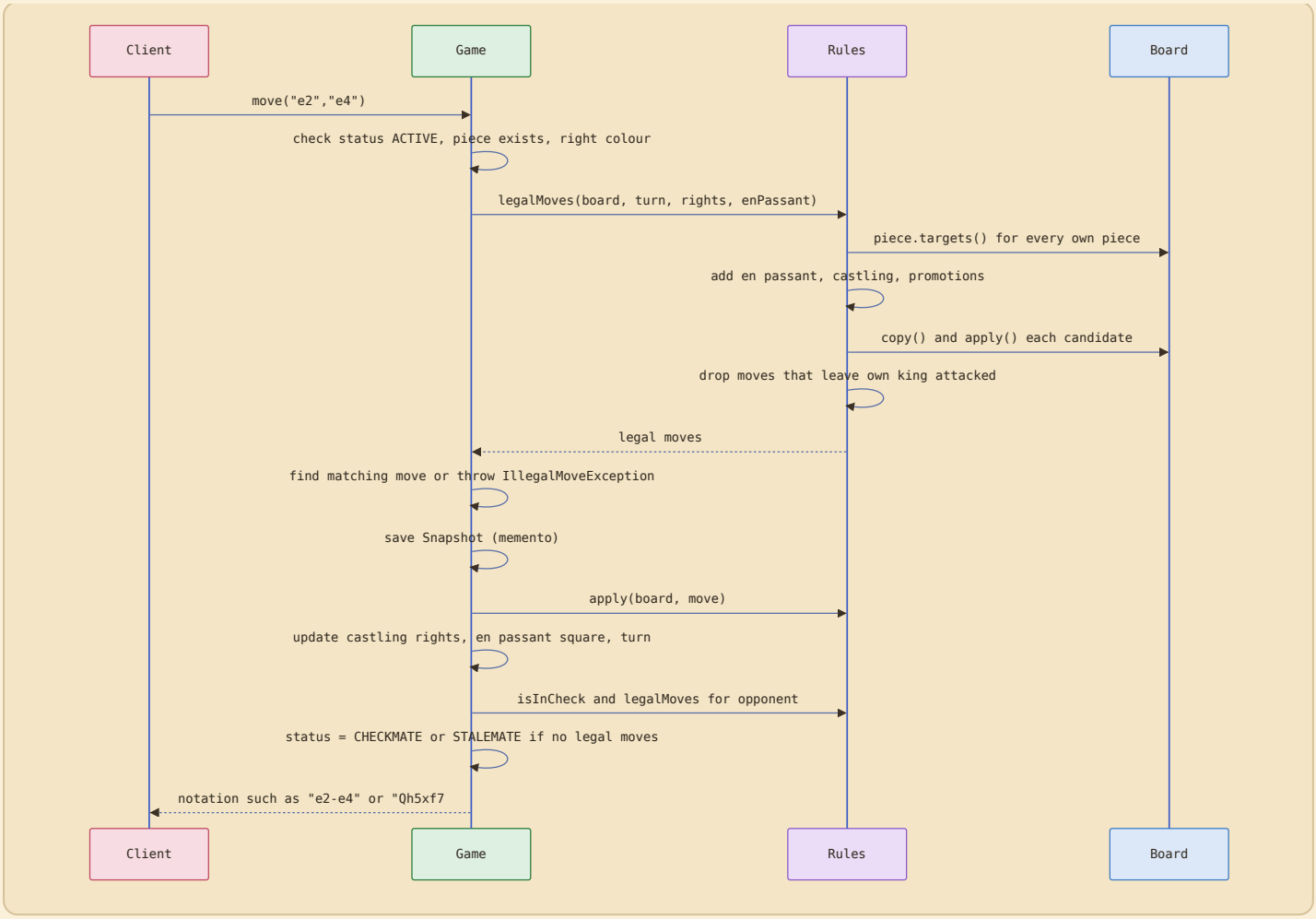
In the course model, a `Board` is **composed** of 64 `Box`es; a `Box` refers to at most one `Piece` by plain association (not composition), because a captured piece leaves the board without being destroyed — which keeps the move log replayable. `Piece` is abstract with `can_move(board, start, end)`; `SlidingPiece` sits between `Piece` and rook/bishop/queen so the "path is clear" check is written once. A `Move` records player, start and end box, piece moved, piece captured and a castling flag. `Game` owns the board and the moves and refers to the two players (who outlive the game). The course's `GameStatus`: ACTIVE, BLACK_WIN, WHITE_WIN, FORFEIT, STALEMATE, RESIGNATION.

Our Java design keeps that shape and fixes the two limits the course itself points out: (1) its pieces decide legality from the board alone, so history-based rules (castling rights, en passant) have nowhere to live, and (2) checkmate/stalemate are not detected. We add `Rules` (which sees history through `CastlingRights` and the en passant square) and make `Move` carry a `MoveType`, so a single move can move two pieces (castling) or remove a piece that is not on the target square (en passant).

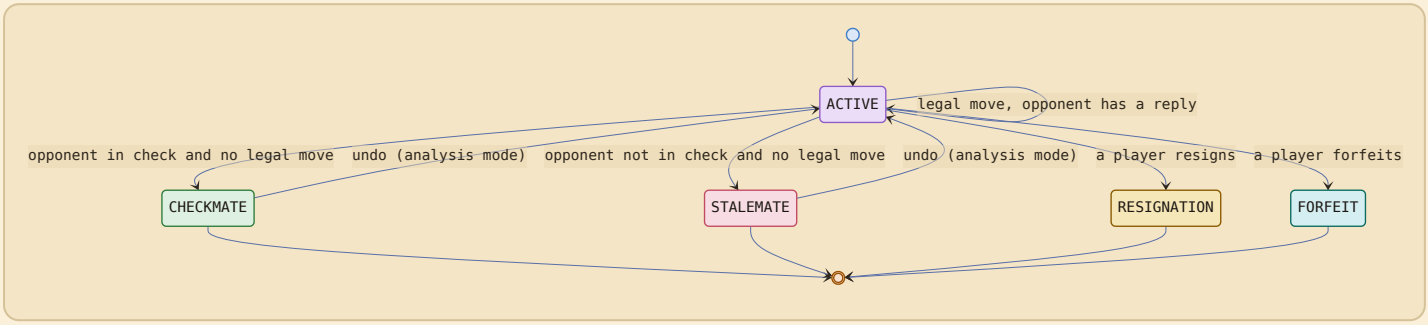


Key flows

Making a move



Game status



How check, checkmate and stalemate are detected (the "reasonable depth" answer)

1. **Pseudo-legal moves** — every move allowed by the pieces' movement rules, plus castling, en passant and promotions.
2. **Legal moves** — for each pseudo-legal move, apply it to a *copy* of the board and keep it only if the mover's king is **not attacked** afterwards. This one filter handles pins, moving into check, and ignoring a check, with no special cases.
3. **Is the king attacked?** Ask every enemy piece for the squares it attacks (`attacks(...)`); pawns attack diagonally even onto empty squares, which is why `Pawn` overrides `attacks`.
4. After a move, look at the **opponent**: if it has **zero legal moves**, it is **checkmate** when its king is in check, otherwise **stalemate**.

This is a one-ply (one move deep) search, which is all the *rules* need. Searching deeper is what a chess *engine* does to choose good moves — not required for enforcing rules.

Cost: at most ~200 pseudo-legal moves, each checked with an attack scan over ≤16 pieces — trivial for a human game. A production engine uses bitboards (each piece type's squares packed into one 64-bit number) and incremental attack maps (updated per move, not recomputed); mention only if asked.

Design decisions & patterns used

Decision	Why	Alternative rejected
Polymorphic pieces (`targets` overridden per piece), **Template-like** `SlidingPiece`.	Each piece's rule lives in its own class; the blocked-path loop is written once for three pieces.	One giant `switch (pieceType)` in the board.
Sealed hierarchy (`sealed ... permits`).	The set of chess pieces is closed; the compiler knows all subclasses, so `switch` on type can be exhaustive (the compiler checks every case is handled).	Open hierarchy — fine too for variants; say so.
Pieces are **immutable**; `Board.copy()` copies references (pointers to the same piece objects) only.	Trial moves and snapshots are cheap and safe.	Mutable pieces with `hasMoved` flags — then undo must restore every flag.
History-based rules in `Rules` / `Game`, not in pieces.	Castling rights and en passant depend on earlier moves. The course notes that a piece seeing only the board cannot know this.	Letting `King` look up history — couples a piece to the game log.
`Move` carries a `MoveType`.	One move can move two pieces (castling) or capture off-square (en passant).	Special-casing in `Game` (the course's `move_castling_rook`), which breaks "rules live in one place".
Command + Memento for undo. `Move` is the command (an action stored as an object); `Snapshot` is the memento (a saved copy of an object's state that can be restored later) of the state before it.	Restoring a snapshot is always correct, even for promotion/castling/en passant, with no inverse logic.	Pure Command with `undo()` per move type — works, but every special move needs a hand-written inverse, a common bug source.
Factory method `Piece.create(type, color)`.	Used by board set-up and promotion.	`new` calls scattered around.
`Game` methods are `synchronized`.	Two player connections may call `move` / `resign` at the same time.	No locking — race (result depends on which thread runs first) between a move and a resignation.

Memento vs incremental undo — trade-off. A snapshot of an 8×8 reference array is 64 references (~0.5 KB): nothing for a human game (a few hundred moves). An engine searching millions of positions would instead store only what the move changed (captured piece, old castling rights, old en passant square) and reverse it. Know both.

Java implementation

Package `lld.ch17.chess`; full runnable code (one file per class, plus `StepPieces`, `IllegalMoveException` and `Main`) is in `code/lld/ch17/chess/`. `Position` is an immutable square (file 0..7 = a..h, rank 0..7 = 1..8) parsed from "e4"; `GameStatus` is `ACTIVE`, `CHECKMATE`, `STALEMATE`, `RESIGNATION` or `FORFEIT`. The essential lines (`/* ... */` and `//` summary comments stand for code left out):

```java
/** Each piece answers "which squares can I reach from here?" its own way. Pieces are immutable. */
public abstract sealed class Piece permits King, Knight, Pawn, SlidingPiece {
    public abstract PieceType type();
    /** Squares reachable by ordinary rules (captures included, own-king safety NOT checked). */
    public abstract List<Position> targets(Board board, Position from);
    /** Squares this piece attacks. Same as targets for every piece except the pawn. */
    public List<Position> attacks(Board board, Position from) { return targets(board, from); }
    /** Simple factory used for set-up and promotion. */
    public static Piece create(PieceType type, Color color) { /* switch over every type */ }
}
// King and Knight jump fixed offsets via StepPieces.step(...). Castling is a two-piece move, so Rules
// generates it, not King. The knight is the only piece that ignores pieces in between.

/** Rook, bishop and queen slide until blocked. The "is the path clear" loop is written once, here. */
public abstract sealed class SlidingPiece extends Piece permits Rook, Bishop, Queen {
    protected abstract int[][] directions();          // Rook: STRAIGHT, Bishop: DIAGONAL, Queen: all eight
    @Override public final List<Position> targets(Board board, Position from) {
        List<Position> out = new ArrayList<>();
        for (int[] d : directions()) {
            int f = from.file() + d[0], r = from.rank() + d[1];
            while (Position.onBoard(f, r)) {
                Position p = new Position(f, r);
                Piece occupant = board.get(p);
                if (occupant == null) out.add(p);
                else { if (occupant.color() != color()) out.add(p); break; }   // capture, then stop
                f += d[0]; r += d[1];
            }
        }
        return out;
    }
}
// Pawn: one square forward (two from its start rank) onto empty squares; captures diagonally.
// Its attacks() are both forward diagonals whether or not anything stands there. En passant and promotion: Rules.

/** 8x8 grid of (immutable) pieces. Knows geometry and attacks; knows nothing about turns or history. */
public final class Board {
    /** Pieces are immutable, so copying the references is a full, independent copy of the position. */
    public Board copy() { /* clone each file's array */ }
    public boolean isAttacked(Position target, Color by) { /* any piece of `by` whose attacks() contain target */
}
}

/** A move as data (the Command's payload). promotion is null unless type == PROMOTION. */
public record Move(Position from, Position to, MoveType type, PieceType promotion) {}
// MoveType: NORMAL, CASTLE_KINGSIDE, CASTLE_QUEENSIDE, EN_PASSANT, PROMOTION
// CastlingRights: four booleans, lost for good once the king or that rook moves (or the rook is captured on its
corner).
```

```java
/** Stateless rules engine. "Legal" = pseudo-legal AND does not leave your own king attacked. */
public final class Rules {
    public static boolean isInCheck(Board board, Color side) { return board.isAttacked(board.kingOf(side),
side.opposite()); }

    public static List<Move> legalMoves(Board board, Color side, CastlingRights rights, Position enPassant) {
        List<Move> legal = new ArrayList<>();
        for (Move m : pseudoLegalMoves(board, side, rights, enPassant)) {   // pieces' targets + 4 promotion
choices
            Board trial = board.copy();                                      // + en passant + castling
            apply(trial, m);
            if (!isInCheck(trial, side)) legal.add(m);
        }
        return legal;
    }

    private static void addCastling(Board board, Color side, CastlingRights rights, List<Move> moves) {
        // ... king on e1/e8; cannot castle out of check
        if (rights.kingSide(side) && isOwnRook(board, new Position(7, r), side)
                && empty(board, r, 5, 6)
                && !board.isAttacked(new Position(5, r), enemy)             // cannot pass through check
                && !board.isAttacked(new Position(6, r), enemy))
            moves.add(new Move(kingSq, new Position(6, r), MoveType.CASTLE_KINGSIDE, null));
        // queen side: files 1-3 empty, d and c squares not attacked
    }
    // whyCannotCastle(...) explains a rejected two-square king move ("the king or that rook has already moved",
    // "the squares between king and rook are not empty", "the king is in check", "pass through the attacked
square").

    /** Mutates the board. Handles the extra piece movements of the special moves. */
    public static void apply(Board board, Move m) {
        Piece piece = board.get(m.from());
        board.set(m.from(), null);
        board.set(m.to(), piece);
        switch (m.type()) {
            case EN_PASSANT -> board.set(new Position(m.to().file(), m.from().rank()), null);   // the passed pawn
            case PROMOTION -> board.set(m.to(), Piece.create(m.promotion(), piece.color()));
            case CASTLE_KINGSIDE -> moveRook(board, m.from().rank(), 7, 5);
            case CASTLE_QUEENSIDE -> moveRook(board, m.from().rank(), 0, 3);
            case NORMAL -> { }
        }
    }
}
```

```java
/** Controls one game. Undo = Command (the move) + Memento (a snapshot of the state before it). */
public final class Game {
    /** Memento: everything needed to restore the position exactly. */
    private record Snapshot(Board board, Color turn, CastlingRights rights, Position enPassant,
                            GameStatus status, Color winner) {}
    private record HistoryEntry(Player player, Move move, String notation, Snapshot before) {}

    /** The course's rule: each player is randomly assigned a side. Pass a seeded Random for repeatable tests. */
    public static Game withRandomSides(String nameA, String nameB, Random rng) { /* rng.nextBoolean() */ }

    public synchronized String move(String fromSq, String toSq, PieceType promotion) {
        // rejects: game over, empty square, wrong turn; then finds the chosen move in Rules.legalMoves
        // (promotion defaults to QUEEN). If absent, explains why: castling reason, "would leave the king in
check",
        // or "<PIECE> on <sq> cannot move to <sq>".
    }

    private String play(Move m, Piece piece) {
        Player mover = turn == Color.WHITE ? white : black;
        Snapshot before = new Snapshot(board.copy(), turn, rights, enPassant, status, winner);
        boolean capture = board.get(m.to()) != null || m.type() == MoveType.EN_PASSANT;
        Rules.apply(board, m);
        rights = rights.after(piece.type(), turn, m.from(), m.to());
        enPassant = (piece.type() == PieceType.PAWN && Math.abs(m.to().rank() - m.from().rank()) == 2)
                ? new Position(m.from().file(), (m.from().rank() + m.to().rank()) / 2) : null;
        turn = turn.opposite();
        // End-of-game detection: the side to move has no legal move -> checkmate if in check, else stalemate.
        boolean check = Rules.isInCheck(board, turn);
        boolean noMoves = Rules.legalMoves(board, turn, rights, enPassant).isEmpty();
        if (noMoves) { status = check ? GameStatus.CHECKMATE : GameStatus.STALEMATE; winner = check ?
mover.color() : null; }
        String notation = notation(m, piece, capture) + (check ? (noMoves ? "#" : "+") : "");   // O-O, O-O-O, =Q,
e.p.
        history.push(new HistoryEntry(mover, m, notation, before));
        return notation;
    }

    public synchronized String undo() {
        HistoryEntry last = history.pop();
        Snapshot s = last.before();
        board = s.board(); turn = s.turn(); rights = s.rights(); enPassant = s.enPassant();
        status = s.status(); winner = s.winner();
        return last.notation();
    }
    // resign(who) and forfeit(who) (left the game, ran out of time, broke site rules): winner = who.opposite().
}
```

`Main.java` plays five scripted games between Ann and Ben: validation + Scholar's mate + undo; en passant, castling, promotion and a move that ignores check; Sam Loyd's 10-move stalemate; castling through an attacked square and out of check; random sides (`new Random(7)`), resignation and forfeit.

Output:

```
== 1. Validation, Scholar's mate, undo ==
  e7-e5 rejected: it is WHITE's turn
  e1-e3 rejected: KING on e1 cannot move to e3
  a1-a4 rejected: ROOK on a1 cannot move to a4
  h5-f7 played as Qh5xf7#
  Log: 1. e2-e4 e7-e5 2. Bf1-c4 Nb8-c6 3. Qd1-h5 Ng8-f6 4. Qh5xf7#
  Status: CHECKMATE, winner WHITE
    8  r . b q k b . r
    7  p p p p . Q p p
    6  . . n . . n . .
    5  . . . . p . . .
    4  . . B . P . . .
    3  . . . . . . . .
    2  P P P P . P P P
    1  R N B . K . N R
       a b c d e f g h
  undo Qh5xf7# -> status ACTIVE, WHITE to move, 43 legal moves

== 2. En passant, castling, promotion, and a move that ignores check ==
  e5-d6 played as e5xd6 e.p.
  e1-g1 played as O-O
  c7-d8 played as c7xd8=Q+
  Black in check? true
  b2-a1 rejected: b2-a1 would leave the BLACK king in check
  e8-d8 played as Ke8xd8
  Log: 1. e2-e4 Ng8-f6 2. e4-e5 d7-d5 3. e5xd6 e.p. Nb8-c6 4. Ng1-f3 a7-a5 5. Bf1-c4 a5-a4 6. O-O a4-a3 7. d6xc7
a3xb2 8. c7xd8=Q+ Ke8xd8
    8  r . b k . b . r
    7  . p . . p p p p
    6  . . n . . n . .
    5  . . . . . . . .
    4  . . B . . . . .
    3  . . . . . N . .
    2  P p P P . P P P
    1  R N B Q . R K .
       a b c d e f g h

== 3. Stalemate (Sam Loyd's 10-move stalemate) ==
  Log: 1. e2-e3 a7-a5 2. Qd1-h5 Ra8-a6 3. Qh5xa5 h7-h5 4. h2-h4 Ra6-h6 5. Qa5xc7 f7-f6 6. Qc7xd7+ Ke8-f7 7. Qd7xb7
Qd8-d3 8. Qb7xb8 Qd3-h7 9. Qb8xc8 Kf7-g6 10. Qc8-e6
  Status: STALEMATE, black in check? false, black legal moves: 0
    8  . . . . . b n r
    7  . . . . p . p q
    6  . . . . Q p k r
    5  . . . . . . . p
    4  . . . . . . . P
    3  . . . . P . . .
    2  P P P P . P P .
    1  R N B . K B N R
       a b c d e f g h

== 4. Castling rules: through an attacked square, and out of check ==
  e1-g1 rejected: cannot castle e1-g1: the king would pass through the attacked square f1
  e8-g8 rejected: cannot castle e8-g8: the king is in check
  c7-c6 played as c7-c6
  e1-g1 played as O-O
  e8-g8 played as O-O

== 5. Random sides, resignation and forfeit ==
  white = Ann, black = Ben
  Status: RESIGNATION, winner BLACK
  g2-g4 rejected: game is over: RESIGNATION
  Status: FORFEIT, winner WHITE
```

Reading the output. Part 1 shows validation (out of turn, illegal king jump, blocked rook), then Scholar's mate (`Qh5xf7#`), and `undo()` returning the game to ACTIVE with white to move. Part 2 plays en passant (`e5xd6 e.p.` removes the black pawn from d5), kingside castling (`O-O`), and a capturing promotion that gives check (`c7xd8=Q+`); black's attempt to ignore the check is rejected, and the king captures the new queen. Part 3 is the famous 10-move stalemate composed by Sam Loyd: black is not in

check but has no legal move, so the game is a draw. Part 4 shows the castling guards: white may not castle through f1 while the a6 bishop attacks it, black may not castle out of check, and once the check is blocked both sides castle. Part 5 assigns sides at random (seeded) and shows resignation and forfeit.

Concurrency & edge cases

- **Two players, one game.** `Game`'s public methods are `synchronized`, so a move and a resignation cannot interleave. With thousands of games, each game's own monitor (the built-in lock of every Java object) is its lock, so games never block each other.
- **Moving out of turn / game over / empty square** — rejected with a clear message before any rule work.
- **Pins and discovered checks** (a *pin* is a piece that cannot move because it shields its king; a *discovered check* is a check revealed when a blocking piece moves away) — handled automatically by the "apply to a copy and test the king" filter.
- **Castling through check** — explicitly tested for the squares the king crosses and lands on; the rook's crossing square on the queenside (b1/b8) may be attacked — that is legal. Section 4 of the output shows both refusals with their reason (`Rules.whyCannotCastle`): passing through an attacked f1, and castling while in check; once the check is blocked the same castle is accepted.
- **Castling rights** are lost when the king moves, when that rook moves, or when that rook is **captured** on its corner (we check both `from` and `to`).
- **En passant expires** after one move: the square is reset on every move unless a pawn just advanced two.
- **Promotion without a choice** defaults to a queen; underpromotion (knight, rook, bishop) is supported.
- **Not implemented, but should be named:** draw by threefold repetition (the same position occurs three times), the 50-move rule (50 moves each with no capture or pawn move), insufficient material (e.g. king vs king), clocks/timeouts, draw offers.

Extensions & follow-up questions

- **"Add threefold repetition."** — Keep a `Map<PositionKey, Integer>` where the key is (board, side to move, castling rights, en passant square). Production engines hash this (turn it into one compact number for quick comparison) with Zobrist hashing, which combines a random number per piece-and-square. Three occurrences → draw.
- **"Add a 50-move rule."** — A half-move (one player's move) clock reset on every pawn move or capture; at 100 half-moves either player may claim a draw.
- **"Add chess clocks."** — A `Clock` per player; the server stamps time on each move; a flag-fall (time running out) ends the game, and an Observer notifies both clients.
- **"Add an AI player."** — A `MoveStrategy` that picks from `legalMoves()`; a real one runs minimax (search ahead, assuming you pick your best move and the opponent picks the move worst for you) with alpha-beta pruning (skipping branches that provably cannot change the answer).
- **"Support Chess960 (a variant with a shuffled back row) or a new piece."** — New `Piece` subclass (e.g. an Archbishop = bishop + knight) and a different starting-position factory; existing pieces are untouched.
- **"Spectators and UI updates."** — Observer: `GameListener.onMove(notation)`. The course's `GameController` (takes input, calls `Game`) and `GameView` (redraws) are the MVC (Model–View–Controller) split from Chapter 11.
- **"Why is `Position` a record?"** — It is a value: two `e4`s are equal. Records generate `equals`/`hashCode`, which `List.contains` relies on.

Real-world example

Online chess sites such as Lichess and Chess.com treat the **server** as the source of truth (the one copy whose data wins whenever copies disagree): the board in your browser is only a view, and a move counts only after the server has accepted it as legal. Games are exported in PGN (Portable Game Notation — a move log like ours) and positions in FEN (Forsyth–Edwards Notation — one line describing the board, side to move, castling rights and en passant square, i.e. exactly the fields of our `Snapshot`).

"Takeback" is offered only when the opponent agrees, and then the server undoes the move — the same `undo()` our `Game` exposes.

In the interview

> **30-second answer:** "`Board` is an 8×8 grid of immutable `Piece`s. `Piece` is abstract with `targets(board, from)`; each piece overrides it, and rook/bishop/queen share a `SlidingPiece` loop. A stateless `Rules` class generates pseudo-legal moves, adds castling, en passant and promotion using history kept in `Game` (castling rights, en passant square), and filters out moves that leave your king attacked. After each move, if the opponent has no legal move, it's checkmate if they're in check, otherwise stalemate. `Game` handles turns, status, a move log, and undo via Command plus Memento."

How to spend 30–45 minutes: 5 min requirements → 5 min entities → 10 min class diagram (piece hierarchy, why history lives outside pieces) → 15 min code `Piece`/`SlidingPiece`/`Pawn` and the legal-move filter → 5–10 min checkmate/stalemate, special moves, concurrency.

Likely follow-ups with answers

- *"How do you detect checkmate?"* — No legal moves and in check. Legal = pseudo-legal filtered by "king not attacked after the move".
- *"Where does castling live?"* — In `Rules`, because it moves two pieces and depends on history; `MoveType.CASTLE_*` tells `apply` to move the rook too.
- *"Why is `Box -> Piece` not composition?"* — Removing a piece from a square doesn't destroy the piece; it goes to the captured list or the log.
- *"How is undo safe with promotion?"* — The memento stores the whole prior board, so undo simply puts the old pawn back.

Self-check

1. **Q:** Why does `Pawn` override `attacks()`? **A:** A pawn moves straight but attacks diagonally, even onto empty squares; check detection needs the attack squares.
2. **Q:** What is the difference between checkmate and stalemate? **A:** Both mean no legal move; checkmate is while in check (loss), stalemate is while not in check (draw).
3. **Q:** When is en passant allowed? **A:** Only on the move immediately after an enemy pawn advanced two squares past your pawn's attack square.
4. **Q:** Name three conditions for castling. **A:** King and that rook have not moved; squares between empty; king not in check, not passing through or landing on an attacked square.
5. **Q:** Why are pieces immutable in this design? **A:** Board copies for trial moves and undo snapshots can share pieces safely.

TL;DR

- Polymorphic `Piece.targets()`; `SlidingPiece` shares the path loop; pawn overrides `attacks()`.
- `Rules`: pseudo-legal → add castling / en passant / promotion → drop moves leaving own king attacked.
- No legal moves: in check = checkmate, else stalemate.
- History-dependent rules live in `Game` / `Rules`, not in pieces. Undo = Command + Memento. `synchronized` game for online play.

Design Tic-Tac-Toe

Problem & clarifying questions

Tic-tac-toe is played on a grid. Players take turns placing their mark (X or O). The first to fill a whole **row, column or diagonal** wins; if the board fills with no winner, it is a **draw**. The classic game is 3×3 with two players; interviewers love to generalise it to **N×N** and ask for **fast win detection**.

Question	Assumed answer
Board size?	Any N ≥ 3 (classic N = 3).
Win condition?	A full line of N marks: any row, column, or one of the two main diagonals.
Players?	Two by default; the design allows more (each with its own mark).
Human or computer?	Both: each player has a pluggable **move strategy** (human input, random bot, rule-based bot, scripted test).
Invalid moves?	Rejected (occupied or off-board); the same player tries again.
UI?	Console rendering only; the engine exposes state for any UI.

Requirements

Functional

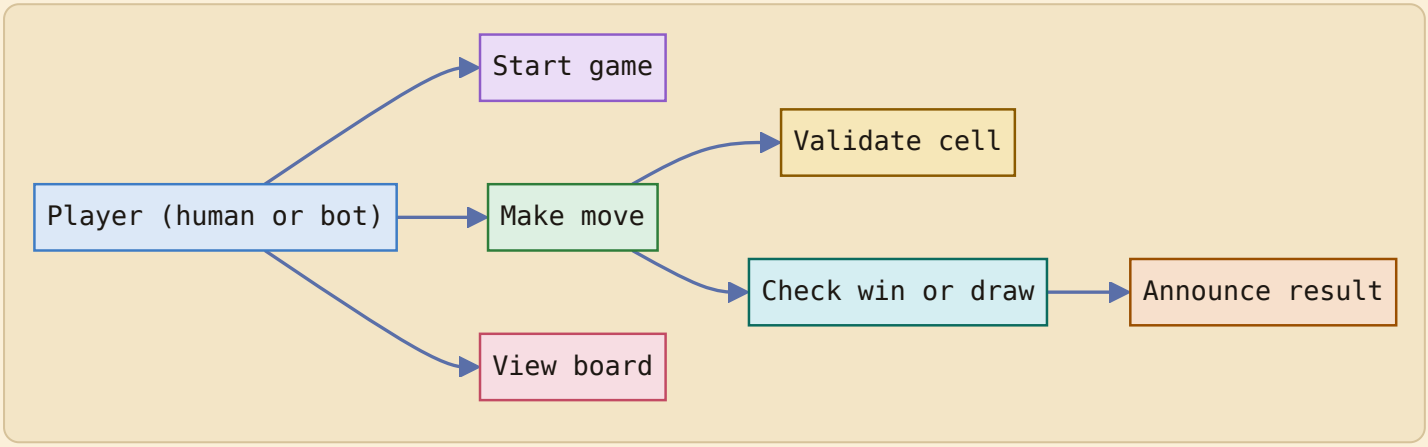
1. Create an N×N board and a game with two (or more) players, each with a unique mark.
2. Players take turns; the current player chooses an empty cell.
3. Validate moves: inside the board and empty.
4. After each move, detect a **win** (the mover completed a line) or a **draw** (board full).
5. Announce the result and show the board.

Non-functional

- Win check in **O(1)** per move (not a full O(N²) board scan).
- New player types (AI levels, network player) without touching `Game` (Open/Closed).
- Deterministic tests: random bots take a seeded `Random`; scripted players replay fixed moves.
- Separate games must not share state (many games can run at once on a server).

Use cases

Actors: **Player** (human or bot). Use cases: *start game* (choose N and players), *make move*, *view board*, *see result*. A system actor (the `Game`) validates moves, switches turns and declares the result.

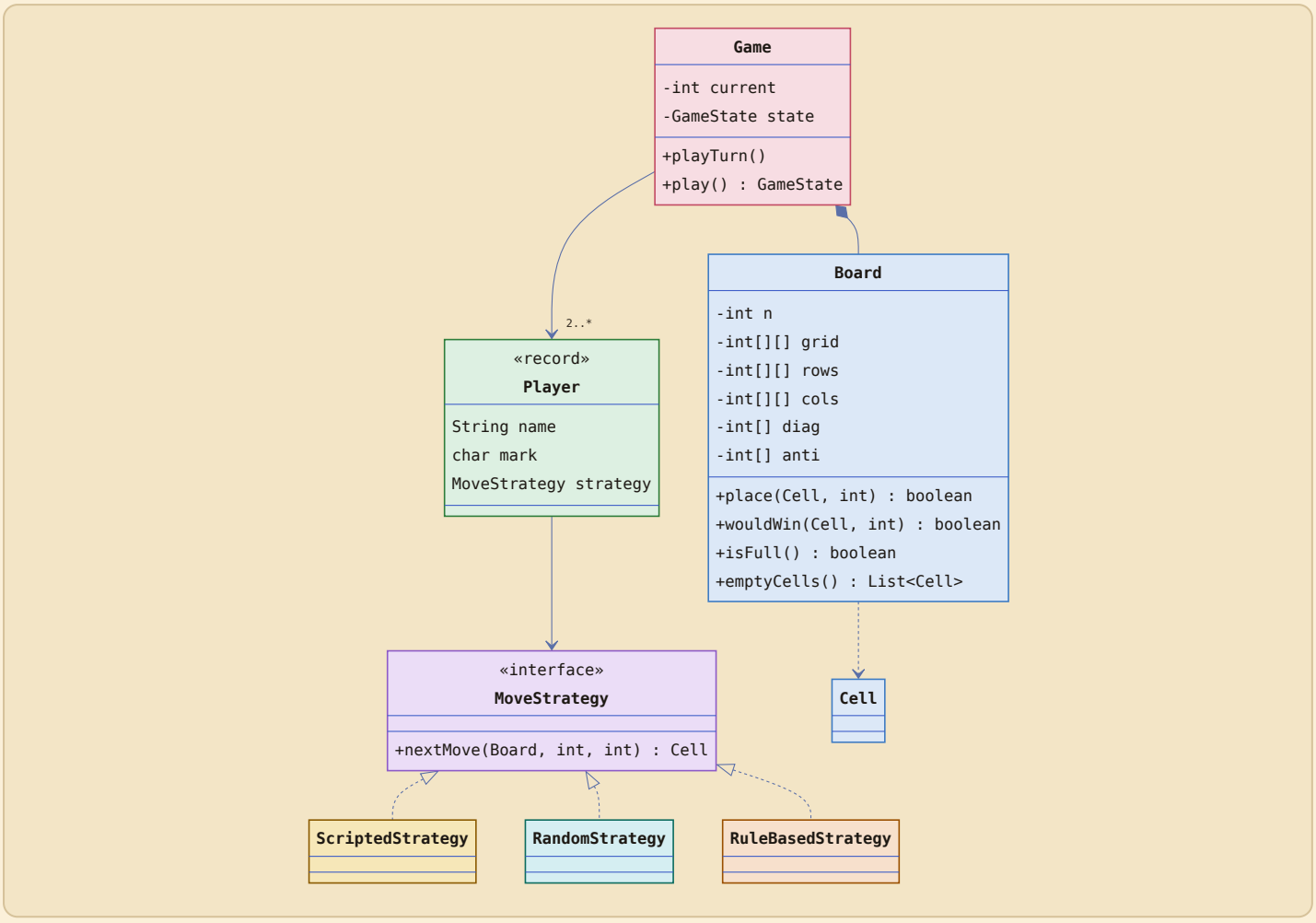


Core entities

Class	Responsibility
`Cell` (record)	Row + column of a square.
`Board`	The grid plus **per-player line counters**; `place` returns "did this win?" in O(1); `wouldWin` for bots; `isFull`, `emptyCells`, `render`.
`MoveStrategy` (interface)	Chooses the next cell. Implementations: `ScriptedStrategy`, `RandomStrategy`, `RuleBasedStrategy`.
`Player` (record)	Name, mark, strategy.
`GameState` (enum)	IN_PROGRESS, WON, DRAW.
`Game`	Turn order, move validation, state, winner, result.

Class diagram

There is no course lesson for this problem; it comes from the *awesome-low-level-design* problem set (3×3 grid, X and O turns, line wins, draw on a full board, display, validation, result). All of it is covered, generalised to N×N.



Key flows

O(1) win detection — how it works

A naive win check scans every row, column and diagonal after each move: O(N²). But a move at `(r, c)` can only complete **row r, column c, and (maybe) the two diagonals**. So we keep counters:

1. `rows[p][r]` = how many marks player `p` has in row `r`; `cols[p][c]` likewise.
2. `diag[p]` counts marks on the main diagonal (`r == c`); `anti[p]` on the anti-diagonal (`r + c == N-1`).
3. When player `p` places at `(r, c)`, increment the counters that cell belongs to.

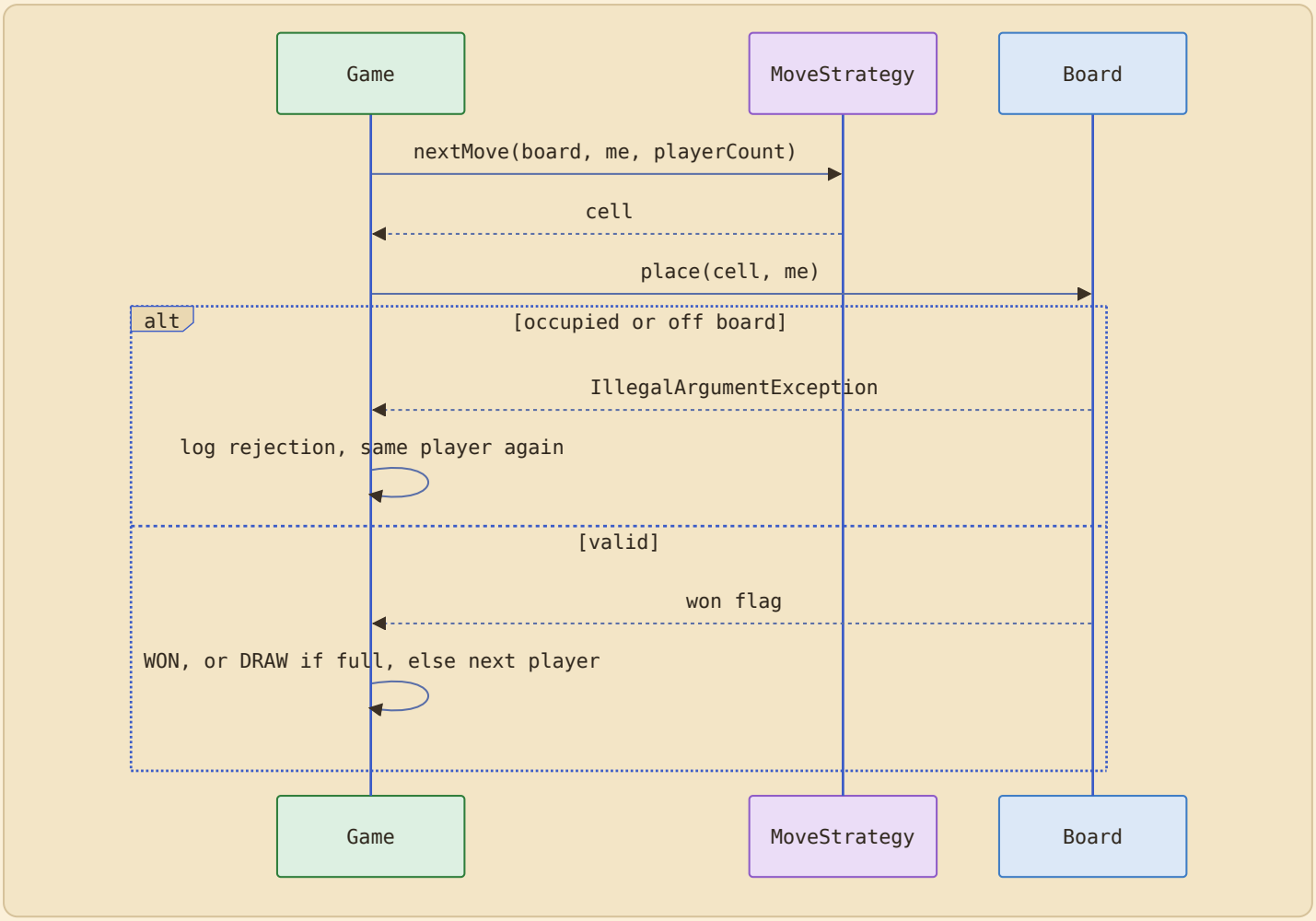
4. If any of them reaches **N**, player `p` owns the whole line — a win. Because only `p`'s marks increment `p`'s counters, no check for opponents' marks is needed.

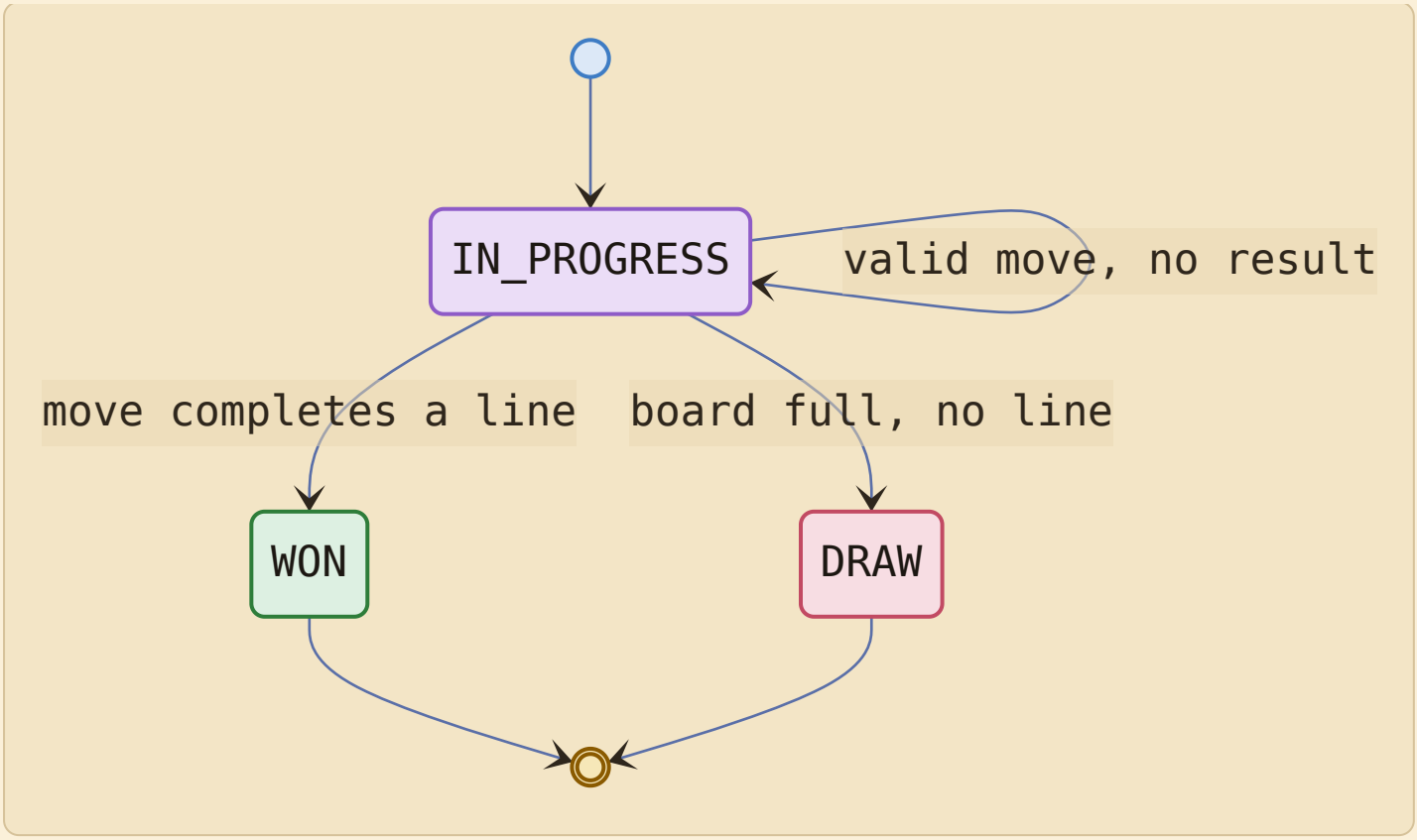
Four increments and four comparisons: **O(1) time**, **O(P·N) memory** for P players.

> **The 2-player trick you may see online:** one shared counter per line, +1 for X and −1 for O; `|count| == N` means a win. It uses half the memory but only works for exactly two players. Our per-player counters work for any number of players.

The same counters give a free **"would this move win?"** check: an empty cell wins for `p` if any of its lines already has `N-1` of `p`'s marks. The rule-based bot uses it to *win* if possible and *block* the opponent otherwise.

One turn





Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for player moves.	Human, random, rule-based, network or minimax players plug in; `Game` never changes.	`if (player.isComputer())` branches in `Game`.
Counters inside `Board` (not a separate "WinChecker" that scans).	O(1) check, kept next to the data it summarises.	Scanning all lines each move — O(N²) and duplicated for "would win".
Board returns "won?" from `place`.	The only moment a win can appear is right after a move.	Separate `checkWinner()` that the caller might forget to call.
Invalid move → exception from `Board`, caught by `Game`.	The board protects its invariant (a fact that must always stay true, e.g. one mark per cell); the game decides the policy (retry).	Silently ignoring a bad move.
Seeded `Random` injected into `RandomStrategy`.	Repeatable demos/tests.	`Math.random()` inside the bot.
Players as a list + index `current = (current+1) % size` (`%` is remainder, so the index wraps back to 0).	Works for any number of players.	Hard-coded X/O toggling.

Java implementation

Package `lld.ch17.tictactoe`; full runnable code (one file per class in Core entities, plus `ScriptedStrategy`, `RandomStrategy`, `RuleBasedStrategy` and `Main`) is in `code/lld/ch17/tictactoe/`. The essential lines (`/* ... */` and `//` summary comments stand for code left out):

```java
/**
 * N x N grid plus per-player line counters, so "did that move win?" is O(1) instead of O(N^2).
 * rows[p][r] = how many marks player p has in row r. Same idea for columns and the two diagonals.
 * A line is won when one player's count reaches N (other players' marks make that impossible).
 */
public final class Board {
    private final int[][] grid;       // -1 empty, otherwise player index
    private final int[][] rows, cols;
    private final int[] diag, anti;

    /** Places a mark and returns true if this move completed a line. O(1). */
    boolean place(Cell c, int player) {
        if (!inside(c)) throw new IllegalArgumentException("cell " + c + " is off the board");
        if (grid[c.row()][c.col()] != -1) throw new IllegalArgumentException("cell " + c + " is already taken");
        grid[c.row()][c.col()] = player;
        filled++;
        int r = c.row(), k = c.col();
        boolean win = ++rows[player][r] == n;
        win |= ++cols[player][k] == n;
        if (r == k) win |= ++diag[player] == n;
        if (r + k == n - 1) win |= ++anti[player] == n;
        return win;
    }

    /** Would placing player's mark on this empty cell win? Also O(1): a line needs exactly one more mark. */
    public boolean wouldWin(Cell c, int player) {
        int r = c.row(), k = c.col();
        return rows[player][r] == n - 1 || cols[player][k] == n - 1
                || (r == k && diag[player] == n - 1) || (r + k == n - 1 && anti[player] == n - 1);
    }
    public boolean isFull() { return filled == n * n; }
}

/** Strategy pattern: human input, scripted test moves, random bot, rule-based bot... all plug in here. */
public interface MoveStrategy { Cell nextMove(Board board, int me, int playerCount); }

/** Win if you can, else block an opponent's win, else take the centre, else the first corner, else anything. */
public final class RuleBasedStrategy implements MoveStrategy { /* uses board.wouldWin for me, then each opponent
*/ }
```

```java
/** Turn management and game state. Supports 2+ players on an N x N board (n >= 3). */
public final class Game {
    /** Plays one turn for the current player. An invalid cell is rejected and the same player goes again. */
    public void playTurn() {
        if (state != GameState.IN_PROGRESS) throw new IllegalStateException("game is over");
        Player p = players.get(current);
        Cell cell = p.strategy().nextMove(board, current, players.size());
        boolean won;
        try {
            won = board.place(cell, current);
        } catch (IllegalArgumentException e) {
            log.accept("  " + p.name() + " tried " + cell + ": rejected (" + e.getMessage() + "), try again");
            return;
        }
        if (won) { state = GameState.WON; winner = p; }
        else if (board.isFull()) state = GameState.DRAW;
        else current = (current + 1) % players.size();
    }
}
```

`Main.java` runs a scripted 3×3 game with one invalid move, rule-based bot vs seeded random bot (seed 7), a 4×4 rule-based mirror match, and a 5×5 three-player game (random seeds 1 and 2, plus one rule-based).

Output:

```
== 1. 3x3 scripted game (with one invalid move) ==
  Xena (X) -> (0,0)
  Omar tried (0,0): rejected (cell (0,0) is already taken), try again
  Omar (O) -> (0,1)
  Xena (X) -> (1,1)
  Omar (O) -> (0,2)
  Xena (X) -> (2,2)
    X O O
    . X .
    . . X
  Result: Xena wins

== 2. 3x3 rule-based bot vs seeded random bot (seed 7) ==
  Bot (X) -> (1,1)
  Rand (O) -> (2,0)
  Bot (X) -> (0,0)
  Rand (O) -> (1,0)
  Bot (X) -> (2,2)
    X . .
    O X .
    O . X
  Result: Bot wins

== 3. 4x4 rule-based vs rule-based ==
    O O X X
    O O X X
    X X X O
    O O O X
  Result: draw

== 4. 5x5 with three players (two random, seeds 1 and 2; one rule-based) ==
    Z Z Z O O
    X O X Z X
    X X Z O O
    Z O O X O
    Z X X X Z
  Result: draw
```

Reading the output. Game 1 shows the retry after an invalid move and a diagonal win found by the O(1) counters. Game 2: the rule-based bot takes the centre, then a corner, and completes the diagonal against a seeded random bot (no block was needed: the random bot's two O's in column 0 were already blocked by the X in the corner). Game 3: two rule-based bots on 4×4 draw (the move lines are filtered out of the log for brevity). Game 4 runs the same counters with three players on 5×5 (a draw).

Concurrency & edge cases

- **Many games on one server:** each `Game` owns its own `Board`, so games share nothing. Keep them in a thread-safe `ConcurrentHashMap<gameId, Game>` and process each game's moves one at a time (per-game lock or single-threaded actor).
- **Two clicks at once from the same game:** make `playTurn`/a `makeMove(player, cell)` method `synchronized` and check that the caller is the current player.
- **Edge cases:** move after game over (`IllegalStateException`); off-board or occupied cell (rejected, same player retries); a scripted strategy running out of moves (returns `null` → fail fast in tests); N = 3 minimum.
- **A cell on both diagonals** (the centre of an odd board) updates both diagonal counters — handled by two independent `if`s.

Extensions & follow-up questions

- **"Win with K in a row on an N×N board (Gomoku, K = 5)?"** — Counters no longer work because lines are shorter than the board. After each move, walk out from the new cell in the 4 directions (horizontal, vertical, two diagonals) and count consecutive own marks: O(K) per move.
- **"Unbeatable AI?"** — Minimax over the game tree (3×3 has at most 9! = 362,880 move orders, easily searched); add alpha-beta pruning or memoisation (caching each evaluated position) for larger boards. Plug it in as another `MoveStrategy`.

- **"Undo?"** — Store moves in a stack (last in, first out); undo = clear the cell and **decrement** the counters (counters are reversible, unlike a boolean "won" flag).
- **"Online multiplayer?"** — Server holds the `Game`; clients send `makeMove(gameId, playerId, cell)`; the server broadcasts the new state (Observer).
- **"Replay a game?"** — The move list plus `ScriptedStrategy` replays it exactly.

Real-world example

Google Search's built-in tic-tac-toe (search "tic tac toe") has difficulty levels — different computer players behind the same rules, i.e. this design's Strategy split. Tic-tac-toe is also the classic first minimax example in AI courses, because its full game tree is small enough to search completely.

In the interview

30-second answer: "`Board` is N×N plus per-player counters for each row, column and the two diagonals. Placing a mark bumps at most four counters; if one hits N, that player wins — O(1) per move. `Game` holds a list of players and a turn index, validates moves, and ends on a win or a full board. Each `Player` has a `MoveStrategy` — human, random with a seeded `Random`, or rule-based that wins if it can and blocks otherwise."

Time plan: 5 min requirements and N×N/players → 5 min entities → 10 min the counter idea (draw it!) → 15 min code → 5 min extensions (K-in-a-row, minimax, undo).

Self-check

1. **Q:** Why does a move only need to check four lines? **A:** A cell belongs to exactly one row, one column, and at most the two diagonals.
2. **Q:** How do you know a cell is on the anti-diagonal? **A:** `row + col == N - 1`.
3. **Q:** Why per-player counters instead of +1/−1? **A:** They work for more than two players and give "would win" for any player.
4. **Q:** How is a draw detected? **A:** No win and the number of filled cells equals N^2.
5. **Q:** How do you test a bot that plays randomly? **A:** Inject `new Random(seed)`; the game is then identical every run.

TL;DR

- Per-player row/column/diagonal counters → O(1) win check; `N-1` → O(1) "would win".
- `Game` = players list + turn index + state enum; invalid moves rejected, same player retries.
- Players via `MoveStrategy` (human, random-seeded, rule-based, minimax later).
- K-in-a-row needs a directional walk from the last move instead.

Design Snake and Ladder

Problem & clarifying questions

Snake and Ladder is a race on a numbered board (usually 1–100). Players take turns rolling a die and move forward by the number rolled. Landing on the **foot of a ladder** takes you up to its top; landing on the **head of a snake** sends you down to its tail. The first to reach the last square wins. There are no decisions — everything is dice and rules — so the design is about a **clean rules engine**, **turn management** and **testable randomness**.

Question	Assumed answer
Board size and snakes/ladders?	Configurable (default 100 squares, a given list of snakes and ladders). Validate the config.
How many dice?	Configurable: count and faces (default one six-sided die).
How many players?	Two or more; all start off the board (square 0).
Overshooting the last square?	Configurable: **exact finish** (stay put) or **bounce back**.
Rolling a six?	Common house rule (an optional rule some players add): roll again; three sixes in a row cancels the whole turn. Configurable on/off.
When does the game end?	Configurable: **first to finish** wins, or keep playing to **rank everyone**.
Several games at once?	Yes, independent sessions (e.g. many groups online).

Requirements

Functional

1. A numbered board (typically 100 cells) with predefined snakes and ladders; several players, one token each, starting off the board.
2. Players take turns rolling the dice and move forward by the total; a snake head slides the token to its tail, a ladder foot climbs to its top.
3. Overshooting the last square follows the exact-finish or bounce-back rule; optional bonus roll on a six, with the three-sixes penalty.
4. The game continues until the win strategy says it is over (first to finish, or everyone ranked).
5. Many independent game sessions run at once.

Non-functional

- **Deterministic** dice for tests (seeded or loaded).
- Rules are **pluggable** without editing the game loop.
- Invalid boards are rejected **at construction**, not discovered mid-game.

Use cases

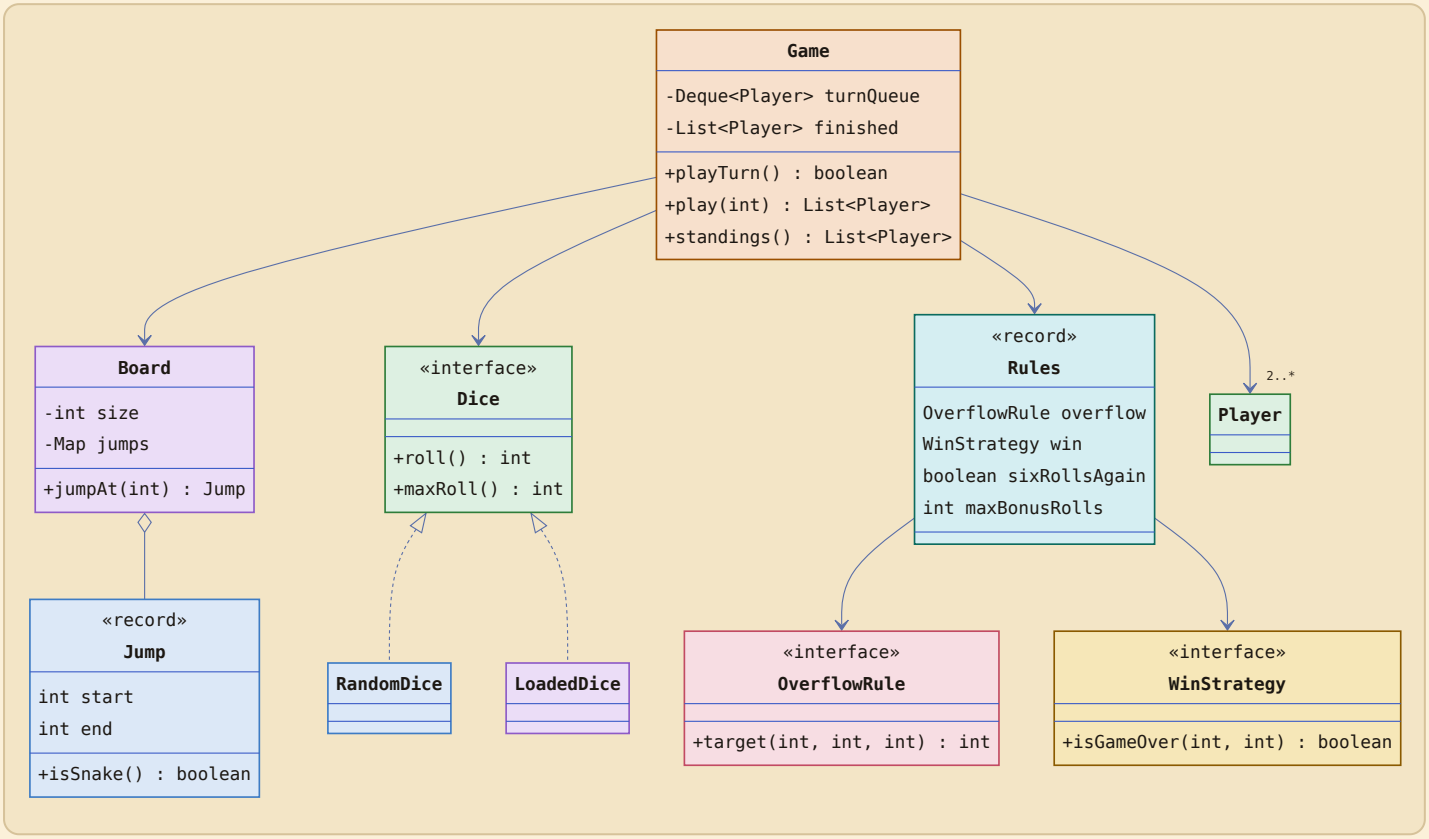
Actors: **Player** (rolls the dice — or the system rolls for them), **Game admin/host** (configures board, dice and rules, starts a game). Use cases: *configure board*, *add players*, *start game*, *take turn* (roll → move → apply snake/ladder → maybe bonus roll), *declare winner / standings*.

Core entities

Class	Responsibility
`Jump` (record)	A snake or a ladder: `start → end`. One class, because the behaviour is identical ("land on start, move to end"); `isSnake()` is just `end < start`.
`Board`	Size + `Map<start, Jump>` for O(1) lookup; validates configuration.
`Dice` (interface)	`roll()`, `maxRoll()`. `RandomDice` (count, faces, seeded `Random`), `LoadedDice` (scripted rolls for tests).
`OverflowRule` (functional interface — exactly one method, so a lambda, a short unnamed function like `x -> x + 1`, can implement it)	Where a roll past the end lands: `exactFinish()` or `bounceBack()`.
`WinStrategy` (functional interface)	When the game ends: `firstToFinish()` or `rankAll()`.
`Rules` (record)	Bundles a variant: overflow rule, win strategy, six-rolls-again flag, max bonus rolls.
`Player`	Name and position.
`Game`	Turn queue, `playTurn()`, `play(maxTurns)`, standings, event log.

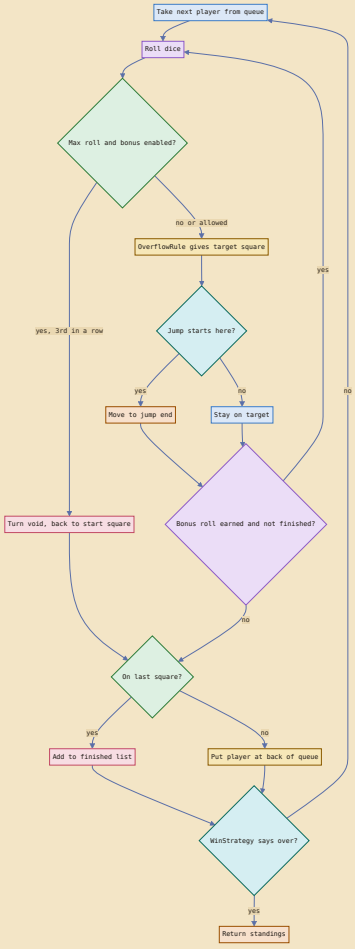
Class diagram

There is no course lesson for this problem; it comes from the *awesome-low-level-design* problem set, whose requirements are all covered above — including many independent game sessions at once (its reference solution uses a singleton `GameManager` with one thread per session); part 6 of the output runs eight sessions in parallel.



Key flows

One turn



Board validation (done once, in the constructor)

1. A jump must not start and end on the same square.
2. No jump starts on square 1 or the last square (you could never win if the last square were a snake).
3. No two jumps share a start square.
4. A jump may not end on another jump's start. This forbids chains, and therefore also forbids infinite loops (snake A → ladder B → snake A...).

Design decisions & patterns used

Decision	Why	Alternative rejected
One `Jump` type for snakes and ladders.	Identical behaviour; the direction is data.	`Snake` and `Ladder` classes with duplicate code — only worth it if they behave differently.
`Map<Integer, Jump>` lookup.	O(1) per move instead of scanning a list.	Array of size 101 — also fine; a map scales to sparse or huge boards.
Strategy for `Dice`, `OverflowRule`, `WinStrategy`.	House rules vary a lot; each is swappable. Lambdas keep them tiny.	Flags and `if`s in the game loop.
`Rules` record groups a variant.	One object = one rule set ("classic", "kids", "tournament"); easy to pass around.	Many constructor parameters on `Game`.
Queue of players.	Natural turn order; finished players simply are not re-queued.	Index arithmetic that must skip finished players.
Event log via `Consumer<String>` (a built-in "takes a value, returns nothing" function type).	The engine does not print; the caller decides (console, UI, nothing). A full design would use **Observer** (`GameListener.onMove/onSnake/onLadder/onWin`).	`System.out.println` inside the engine.
Validation in the `Board` constructor (fail fast).	The loop can trust the board.	Detecting a loop mid-game.
`play(maxTurns)` guard.	A misconfigured rule set can never hang (loop forever and never finish) a server thread.	An unbounded `while (true)`.

Java implementation

Package `lld.ch17.snakeladder`; full runnable code (one file per class in Core entities, plus `RandomDice`, `LoadedDice` and `Main`) is in `code/lld/ch17/snakeladder/`. The essential lines (`/* ... */` and `//` summary comments stand for code left out):

```java
/** A snake (goes down) or a ladder (goes up). Both are just "land on start -> move to end". */
public record Jump(int start, int end) {
    public boolean isSnake() { return end < start; }
}

/** Board configuration: size plus a map start -> Jump for O(1) lookup. Validated once, up front. */
public final class Board {
    public Board(int size, Collection<Jump> all) {
        if (size < 10) throw new IllegalArgumentException("board too small");
        for (Jump j : all) {
            if (j.start() == j.end()) throw new IllegalArgumentException("jump to itself: " + j);
            if (j.start() <= 1 || j.start() >= size) throw new IllegalArgumentException("jump cannot start on
first/last square: " + j);
            if (j.end() < 1 || j.end() > size) throw new IllegalArgumentException("jump ends off board: " + j);
            if (jumps.putIfAbsent(j.start(), j) != null) throw new IllegalArgumentException("two jumps start at "
+ j.start());
        }
        for (Jump j : jumps.values())   // no chains: an end square may not be another jump's start (also rules
out loops)
            if (jumps.containsKey(j.end())) throw new IllegalArgumentException("jump " + j + " ends on another
jump's start");
    }
    public Jump jumpAt(int square) { return jumps.get(square); }   // or null
}

/** Strategy for rolling. Real dice use a seeded Random; tests use a fixed sequence. */
public interface Dice { int roll(); int maxRoll(); }

/** What happens when a roll would carry a token past the last square. */
@FunctionalInterface
public interface OverflowRule {
    int target(int from, int roll, int size);
    /** Must land exactly on the last square, otherwise stay put. */
    static OverflowRule exactFinish() { return (from, roll, size) -> from + roll > size ? from : from + roll; }
    /** Bounce back by the extra amount (at 98 on a 100 board, a 5 goes to 100 then back to 97). */
    static OverflowRule bounceBack() { return (from, roll, size) -> from + roll > size ? size - (from + roll -
size) : from + roll; }
}

/** When is the whole game over? */
@FunctionalInterface
public interface WinStrategy {
    boolean isGameOver(int finishedPlayers, int totalPlayers);
    static WinStrategy firstToFinish() { return (finished, total) -> finished >= 1; }       // classic
    static WinStrategy rankAll()       { return (finished, total) -> finished >= total - 1; } // play on to rank
everybody
}

/** One game variant as one object. sixRollsAgain: max roll earns another; maxBonusRolls in a row voids the turn.
*/
public record Rules(OverflowRule overflow, WinStrategy win, boolean sixRollsAgain, int maxBonusRolls) {
    public static Rules classic() { return new Rules(OverflowRule.exactFinish(), WinStrategy.firstToFinish(),
true, 3); }
}
```

```java
/** Turn loop: a queue of players, a dice strategy, a board and a rule set. */
public final class Game {
    public boolean playTurn() {
        Player p = turnQueue.poll();
        takeTurn(p);
        if (p.position() == board.size()) finished.add(p);   // finished players are simply not re-queued
        else turnQueue.offer(p);
        return isOver();                                      // rules.win().isGameOver(finished, total)
    }
    /** Plays until the win strategy says stop; maxTurns guards against a misconfigured rule set. */
    public List<Player> play(int maxTurns) { /* throws "no result after N turns" */ }

    private void takeTurn(Player p) {
        int start = p.position(), bonus = 0;
        while (true) {
            int roll = dice.roll();
            boolean max = rules.sixRollsAgain() && roll == dice.maxRoll();
            if (max && ++bonus == rules.maxBonusRolls()) { p.moveTo(start); break; }   // too many sixes: turn
void
            move(p, roll);
            if (!max || p.position() == board.size()) break;
        }
    }
    private void move(Player p, int roll) {
        int target = rules.overflow().target(p.position(), roll, board.size());
        if (target == p.position()) return;                   // needs exact roll, stays put
        Jump j = board.jumpAt(target);
        if (j != null) target = j.end();
        p.moveTo(target);
    }
}
```

`Main.java` uses a 100-square board with ladders 4→14, 9→31, 21→42, 28→84, 51→67, 71→91, 80→99 and snakes 17→7, 54→34, 62→19, 64→60, 87→24, 93→73, 95→75, 98→79. Step 6 runs eight independent sessions on eight threads; each owns its players, turn queue and seeded dice and shares only the read-only `Board`, so each must give the same winner as when run alone.

Output:

```
== 1. Loaded dice: ladder, bonus roll, snake, three sixes ==
  Ann @0: rolls 4 -> 4 (ladder to 14)
  Bob @0: rolls 6 -> 6, bonus roll: rolls 3 -> 9 (ladder to 31)
  Ann @14: rolls 3 -> 17 (snake to 7)
  Bob @31: rolls 6 -> 37, bonus roll: rolls 6 -> 43, bonus roll: rolls 6 -> 3 sixes in a row, turn void, back to
31

== 2. Overflow rules: token on 97 of 100, rolls a 5 ==
  Exact @97: rolls 5 -> needs exact roll, stays at 97
  Bounce @97: rolls 5 -> 98

== 3. Bad configurations are rejected up front ==
  rejected: jump cannot start on first/last square: Jump[start=100, end=5]
  rejected: jump Jump[start=10, end=30] ends on another jump's start

== 4. Seeded game (seed 2024), 3 players, first to finish wins ==
  Ann @0: rolls 1 -> 1
  Bob @0: rolls 3 -> 3
  Cid @0: rolls 1 -> 1
  Ann @1: rolls 5 -> 6
  Bob @3: rolls 6 -> 9 (ladder to 31), bonus roll: rolls 6 -> 37, bonus roll: rolls 6 -> 3 sixes in a row, turn
void, back to 3
  Cid @1: rolls 6 -> 7, bonus roll: rolls 1 -> 8
  Ann @6: rolls 6 -> 12, bonus roll: rolls 4 -> 16
  Bob @3: rolls 4 -> 7
  Cid @8: rolls 3 -> 11
  Ann @16: rolls 4 -> 20
  Bob @7: rolls 2 -> 9 (ladder to 31)
  Cid @11: rolls 3 -> 14
  Ann @20: rolls 4 -> 24
  Bob @31: rolls 5 -> 36
  Cid @14: rolls 6 -> 20, bonus roll: rolls 1 -> 21 (ladder to 42)
  Ann @24: rolls 4 -> 28 (ladder to 84)
  Bob @36: rolls 3 -> 39
  Cid @42: rolls 5 -> 47
  Ann @84: rolls 4 -> 88
  Bob @39: rolls 5 -> 44
  Cid @47: rolls 3 -> 50
  Ann @88: rolls 1 -> 89
  Bob @44: rolls 6 -> 50, bonus roll: rolls 4 -> 54 (snake to 34)
  Cid @50: rolls 1 -> 51 (ladder to 67)
  Ann @89: rolls 4 -> 93 (snake to 73)
  Bob @34: rolls 2 -> 36
  Cid @67: rolls 5 -> 72
  Ann @73: rolls 6 -> 79, bonus roll: rolls 2 -> 81
  Bob @36: rolls 3 -> 39
  Cid @72: rolls 3 -> 75
  Ann @81: rolls 4 -> 85
  Bob @39: rolls 5 -> 44
  Cid @75: rolls 2 -> 77
  Ann @85: rolls 5 -> 90
  Bob @44: rolls 2 -> 46
  Cid @77: rolls 2 -> 79
  Ann @90: rolls 2 -> 92
  Bob @46: rolls 6 -> 52, bonus roll: rolls 5 -> 57
  Cid @79: rolls 1 -> 80 (ladder to 99)
  Ann @92: rolls 2 -> 94
  Bob @57: rolls 4 -> 61
  Cid @99: rolls 1 -> 100
  *** Cid finishes in place 1 ***
  Winner: Cid

== 5. Same seed, rank-everyone rule, bounce-back overflow (log hidden) ==
  Standings: Cid > Ann > Bob

== 6. Eight independent sessions on eight threads at once ==
  winners by seed: {1=Ann, 2=Bob, 3=Cid, 4=Bob, 5=Cid, 6=Cid, 7=Ann, 8=Bob}
  same as running each session alone? yes
```

Reading the output. Part 1 uses loaded dice to force every rule: Ann climbs 4→14, Bob rolls a 6 and gets a bonus roll that lands on the 9→31 ladder, Ann hits the 17→7 snake, and Bob's three sixes void his turn (he goes back to 31). Part 2 compares overflow rules from square 97 with a roll of 5: exact-finish stays, bounce-back goes to 100 and back 2 to 98. Part 3 shows bad boards rejected. Part 4 is a full seeded game; Part 5 replays with different rules and ranks every player. Part 6 runs eight games with different seeds on eight threads at the same time and checks that each winner is exactly what the same seed gives when that game runs alone — proof that sessions share no mutable state (data that can be changed after creation).

Concurrency & edge cases

- **Many sessions:** each `Game` owns its players, queue and dice. Give **each game its own** `Random` (sharing one `java.util.Random` across threads is thread-safe but makes threads contend for it and breaks per-game reproducibility). Keep games in a `ConcurrentHashMap` and run each game's turns on one thread at a time.
- **Rolling after the game ends** → `IllegalStateException`.
- **A player on the last square** is removed from the queue, so they are never asked to roll again.
- **Bonus roll that finishes the game** stops immediately — no roll after reaching 100.
- **Three sixes** → the turn is voided, and the token returns to where the turn started (even if a ladder was climbed during the turn).
- **Bounce-back landing on a snake** (e.g. bouncing to 98 on the standard board) is applied — the rule output goes through the same jump lookup.
- **Endless games** are impossible with a valid board and fair dice (probability of never finishing is 0), but `maxTurns` guards against bad custom rules.

Extensions & follow-up questions

- **"Add special cells (skip a turn, extra move, teleport)."** — Generalise `Jump` into a `CellEffect` interface (`apply(player, game)`); snakes and ladders become two implementations.
- **"Two dice, or a 1–12 die?"** — `new RandomDice(2, 6, rng)`; `maxRoll()` is 12, so the bonus rule triggers on double-max. Note that 2d6 (two six-sided dice added) is not uniform — 7 is the most likely total — which changes game balance.
- **"Players must roll a 6 to enter the board."** — A new rule strategy: `EntryRule` checked when position is 0.
- **"Pause and resume."** — Serialize (save to bytes or text that can be loaded back) the positions, queue order and dice state (or a seed plus the number of rolls so far).
- **"Real-time UI / spectators?"** — Observer: publish `PlayerMoved`, `SnakeBitten`, `LadderClimbed`, `PlayerFinished` events.
- **"Random board generation?"** — A `BoardGenerator` with a seed, calling the same validating constructor.

Real-world example

Online multiplayer board-game apps (for example Ludo King, which also offers a Snakes & Ladders mode) have to solve the same problems as this design: every match is an isolated session with its own state, the dice must be rolled where players cannot tamper with them (on the server, not in the client app), and each move is pushed to all players as an event. That is the "one game = isolated state + injected dice + event stream" split used here.

In the interview

> **30-second answer:** "`Board` holds size and a map from square to `Jump` (snake or ladder — same behaviour, just direction), validated on construction. `Dice` is a strategy: seeded random for play, loaded for tests. `Rules` bundles pluggable policies: overflow (exact or bounce), win strategy (first finish or rank all), and the bonus-six rule. `Game` keeps a queue of players; each turn rolls, applies the overflow rule, then any jump, and re-queues the player unless they finished."

Time plan: 5 min rules questions (overshoot, sixes, win) → 5 min entities → 10 min diagram + validation rules → 15 min code turn loop → 5 min extensions and concurrency.

Self-check

1. **Q:** Why is one `Jump` class enough for snakes and ladders? **A:** Both mean "land on start, go to end"; only the direction differs, which is data.
2. **Q:** How do you guarantee there is no infinite snake/ladder loop? **A:** Forbid any jump from ending on another jump's start.
3. **Q:** How do you test "three sixes voids the turn"? **A:** `LoadedDice(6, 6, 6)` and assert the position is unchanged.
4. **Q:** What changes to support "bounce back"? **A:** Pass `OverflowRule.bounceBack()` — no game-loop edits.
5. **Q:** Why a queue for turns? **A:** Players who finish simply are not put back; turn order stays correct.

TL;DR

- Board = size + `Map<start, Jump>`; validate once (no jump on last square, no chains/loops).
- Dice, overflow, win condition, bonus rule are all strategies; seeded or loaded dice for tests.
- Queue-based turn loop with a `maxTurns` guard; events go to a listener, not `System.out`.

Design 2048

Problem & clarifying questions

2048 is a single-player sliding puzzle on a 4×4 grid. Each move (left, right, up, down) slides every tile as far as possible in that direction. Two tiles with the **same value** that collide **merge** into one tile with double the value, and the merged value is added to the **score**. After every move that changes the board, a new tile (a 2 most of the time, sometimes a 4) appears in a random empty cell. You "win" by making a 2048 tile; the game is **over** when no move can change the board.

There is no course lesson for this problem. The statement is the common machine-coding version (a *machine-coding* round is one where you write working code for a small system in 1–2 hours; this exercise appears in LLD practice lists such as Tharun's). Its specific rules, all covered below: start with **two tiles numbered 2**; if **three** equal tiles line up, the pair **nearest the wall you move toward** merges; if **four** are equal, the first two and the last two merge; read moves as **0 = left, 1 = right, 2 = up, 3 = down**.

Question	Assumed answer
Board size?	Configurable N (default 4).
New tile?	One per successful move: value 2 with probability 0.9, else 4, in a uniformly random empty cell. The game starts with two 2s.
Merge rule?	Each tile merges **at most once per move**; merges happen from the wall you move toward.
Move that changes nothing?	Not a turn: no new tile, no score.
Win?	Reaching the target tile (2048, configurable). The player may continue afterwards.
Game over?	No empty cell **and** no two equal neighbours (horizontally or vertically).
Undo, high scores, animations?	Out of scope; see Extensions.

Requirements

Functional

1. Start an N×N board with two tiles numbered 2 at random positions.
2. Move in four directions: slide, merge equal neighbours once, add merged values to the score.
3. If the move changed the board, spawn one tile (2 or 4) in a random empty cell.

4. Detect reaching the target (WON, then allow continuing).
5. Detect game over (LOST) when no move is possible.
6. Accept the command-line codes 0/1/2/3 for left/right/up/down (`Direction.fromKey`).

Non-functional

- **Deterministic** spawns via an injected, seeded `Random` for tests.
- The merge logic must be written **once**, not four times (one per direction).
- The spawning rule is pluggable (e.g. an "evil" spawner for testing hard cases).

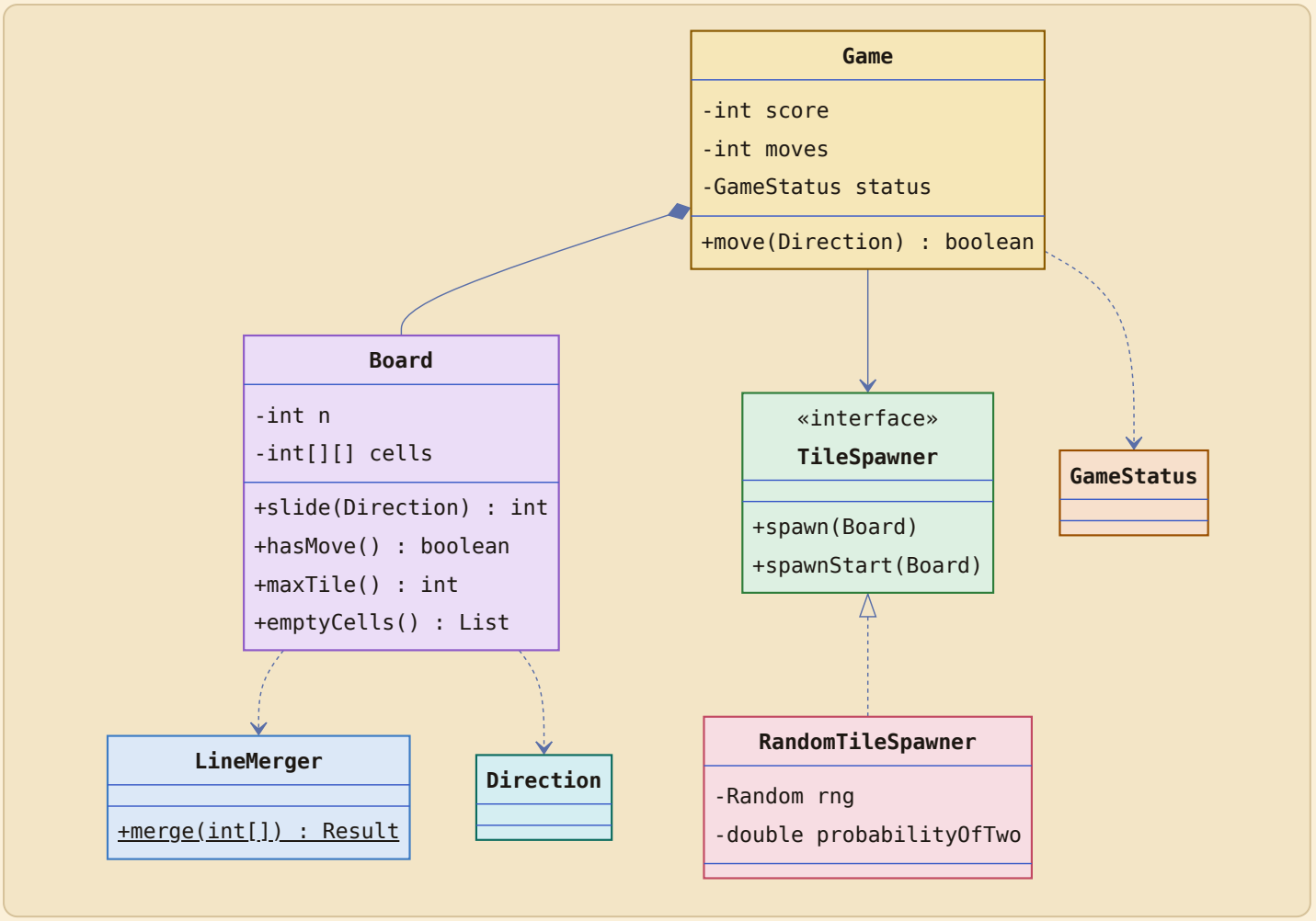
Use cases

Actor: **Player**. Use cases: *start new game*, *make a move* (includes *slide and merge*, *spawn tile*, *update score*), *see win*, *see game over*, *restart*.

Core entities

Class	Responsibility
`Direction` (enum)	LEFT, RIGHT, UP, DOWN; `fromKey(0..3)` maps the command-line codes.
`LineMerger`	Pure function (output depends only on input; changes nothing else): slide and merge **one line toward index 0**; returns the new line and points.
`Board`	N×N values; maps each direction onto lines for `LineMerger`; `slide`, `hasMove`, `maxTile`, `emptyCells`, `render`.
`TileSpawner` (interface) / `RandomTileSpawner`	Adds a new tile; seeded `Random`, 90% twos. `spawnStart` places the two starting tiles (always 2s).
`GameStatus` (enum)	PLAYING, WON, LOST.
`Game`	Score, move count, status; ties slide + spawn + status together.

Class diagram



Key flows

Merging one line (toward index 0) — the whole game in one function:

1. Walk the line from the wall outward, skipping zeros.
2. Keep one **pending** tile. If the next non-zero tile equals it, write the doubled value, add it to the points, and clear pending — so the new tile **cannot merge again this move**.
3. If it differs, write the pending tile and make the new tile pending.
4. At the end, write any pending tile; the rest of the line is zeros.

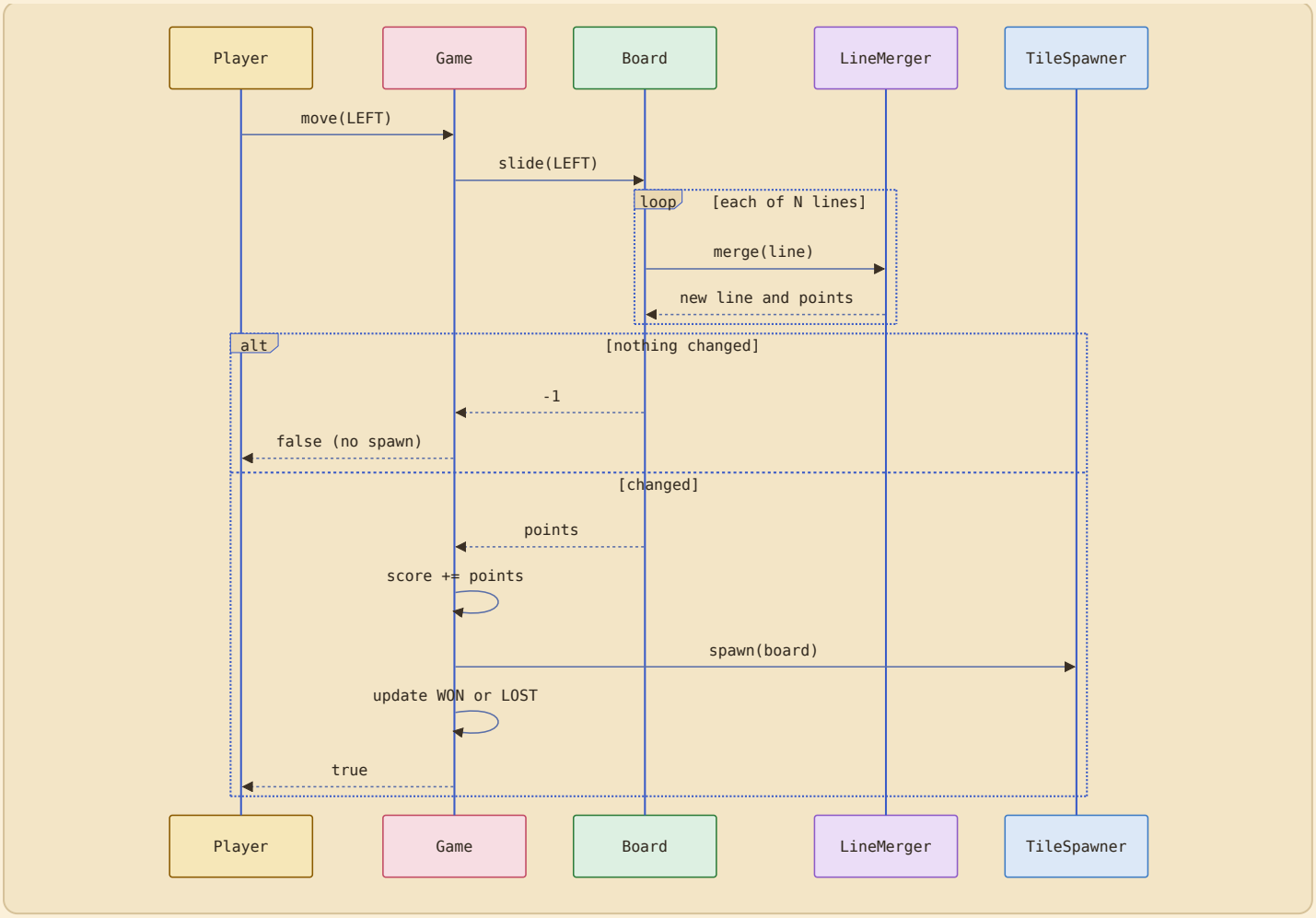
Examples (moving left): `[2,2,2,2] → [4,4,0,0]` (two merges, not one 8), `[2,2,2,0] → [4,2,0,0]` (of three equal tiles, the pair nearest the wall merges), `[2,2,4,0] → [4,4,0,0]` (the new 4 does not merge with the old 4), `[4,0,4,8] → [8,8,0,0]`.

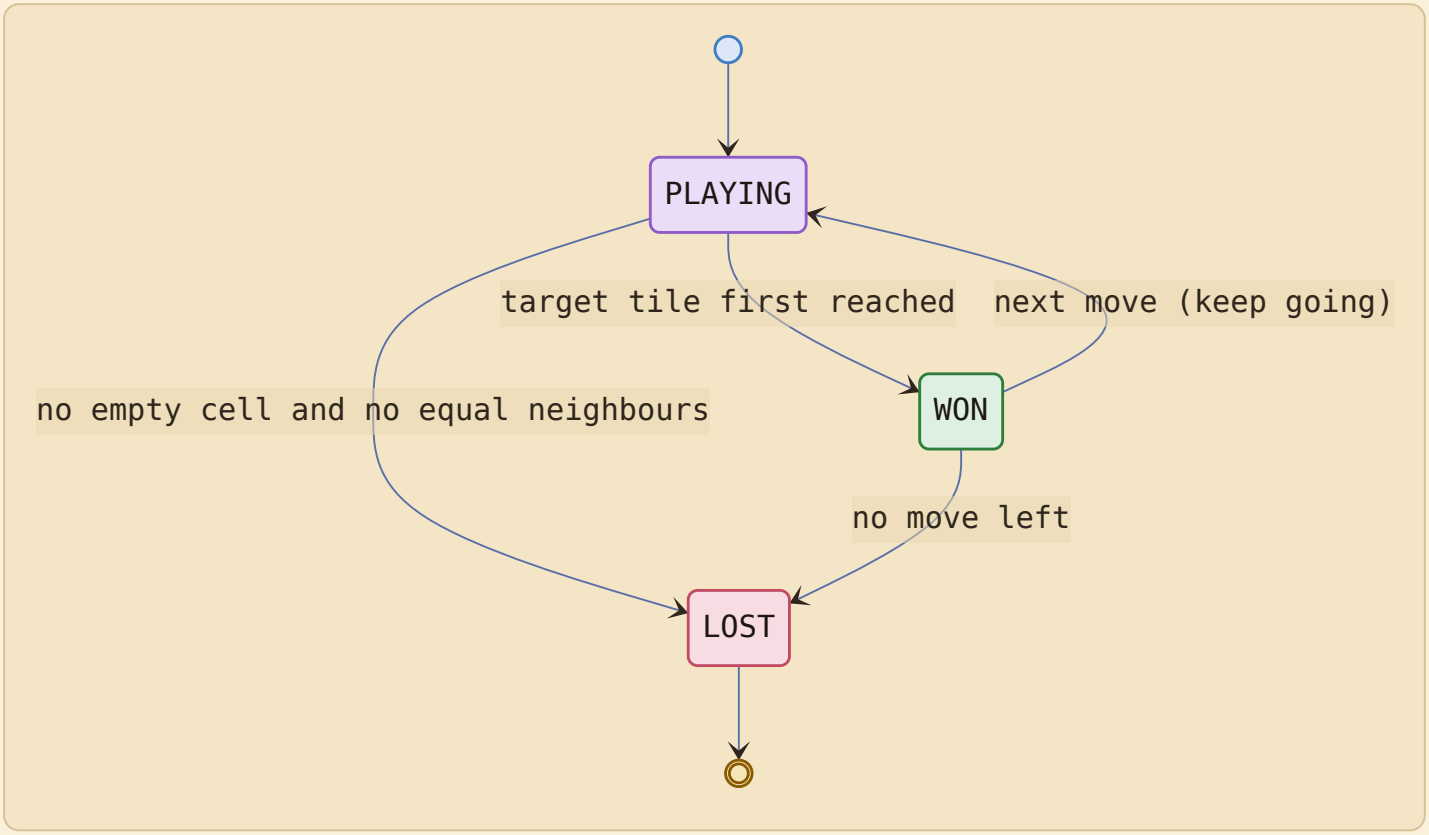
One function for four directions. `Board.coords(direction, i, j)` maps "line `i`, position `j` counted from the wall" to a real cell:

Direction	Line i is	j = 0 is
LEFT	row i	column 0
RIGHT	row i	column N-1
UP	column i	row 0
DOWN	column i	row N-1

Read each line in that order, merge it with `LineMerger`, write it back. No duplicated slide code.

A move





Game-over check. A move is possible if **any cell is empty** or **any two neighbours are equal**. It is enough to compare each cell with its right and lower neighbour (the other two directions are covered by the neighbour's own check). O(N²).

Design decisions & patterns used

Decision	Why	Alternative rejected
One pure `LineMerger` + direction→coordinates mapping.	The tricky rule is written and tested once (DRY — "Don't Repeat Yourself").	Four copies of nested loops (`moveLeft`, `moveRight`...) — classic bug farm.
`LineMerger` is a **pure function** (no state).	Trivial to unit test (test one small piece of code on its own; Part 1 of the output).	Merge logic mixed with board mutation.
Strategy for spawning (`TileSpawner`).	Seeded tests, "evil" spawner, different probabilities for variants.	`Math.random()` inside `Board`.
"No change = no turn" detected by comparing each written cell.	Matches the real game; prevents spawning on a no-op.	Always spawning — makes the game unwinnable/unfair.
`Board.of(int[][])` factory for preset boards.	Game-over and edge-case tests without playing to that position.	Only random starts — untestable edge cases.
Status updated after each move; WON only once.	The player can continue past 2048, as in the original game.	Ending the game at 2048.

Java implementation

Package `lld.ch17.game2048`; full runnable code (one file per class in Core entities, plus `Main`) is in `code/lld/ch17/game2048/`. The essential lines (`/* ... */` and `//` summary comments stand for code left out):

```
/**
 * The heart of 2048, written for ONE line moving toward index 0. Every direction is reduced to this.
 * Rule: slide tiles together, merge equal neighbours once per move, the merge nearest the wall first.
 */
public final class LineMerger {
    public record Result(int[] line, int points) {}          // points = sum of merged tile values

    public static Result merge(int[] in) {
        int[] out = new int[in.length];
        int write = 0, points = 0;
        int pending = 0;                        // tile waiting for a possible merge partner
        for (int v : in) {
            if (v == 0) continue;
            if (pending == 0) {
                pending = v;
            } else if (pending == v) {
                out[write++] = v * 2;           // merge, and the new tile cannot merge again this move
                points += v * 2;
                pending = 0;
            } else {
                out[write++] = pending;
                pending = v;
            }
        }
        if (pending != 0) out[write] = pending;
        return new Result(out, points);
    }
}

/** Strategy for adding a new tile after each successful move. */
public interface TileSpawner {
    void spawn(Board board);                                  // one tile in a random empty cell
    default void spawnStart(Board board) { spawn(board); }    // one of the two starting tiles
}
// RandomTileSpawner: a 2 with probability 0.9, else a 4, in a uniformly random empty cell (seeded);
// spawnStart always places a 2 (the problem statement starts with two 2s).
```

```java
/** N x N grid of tile values (0 = empty). Board.of(int[][]) builds preset boards for tests. */
public final class Board {
    /** Slides every line toward the wall. Returns points earned, or -1 if nothing moved (no turn, no spawn). */
    int slide(Direction d) {
        boolean changed = false;
        int points = 0;
        for (int i = 0; i < n; i++) {
            int[] line = new int[n];
            for (int j = 0; j < n; j++) line[j] = read(d, i, j);       // j = 0 is the cell next to the wall
            LineMerger.Result res = LineMerger.merge(line);
            for (int j = 0; j < n; j++) {
                if (read(d, i, j) != res.line()[j]) changed = true;
                write(d, i, j, res.line()[j]);
            }
            points += res.points();
        }
        return changed ? points : -1;
    }
    /** Maps (line i, position j from the wall) to a real cell for each direction. */
    private int[] coords(Direction d, int i, int j) {
        return switch (d) {
            case LEFT -> new int[]{i, j};
            case RIGHT -> new int[]{i, n - 1 - j};
            case UP -> new int[]{j, i};
            case DOWN -> new int[]{n - 1 - j, i};
        };
    }
    /** A move exists if a cell is empty or two neighbours (right or below) are equal. */
    public boolean hasMove() { /* ... */ }
}

/** One game: board + spawner + score + status. Reaching the target sets WON once; the player may keep going. */
public final class Game {
    public boolean move(Direction d) {
        if (status == GameStatus.LOST) throw new IllegalStateException("game over");
        int points = board.slide(d);
        if (points < 0) return false;          // nothing moved: no spawn, no turn
        score += points;
        moves++;
        spawner.spawn(board);                  // at least one cell is empty, because something moved or merged
        updateStatus();
        return true;
    }
    private void updateStatus() {
        if (!reachedTarget && board.maxTile() >= target) { reachedTarget = true; status = GameStatus.WON; }
        else if (status == GameStatus.WON) status = GameStatus.PLAYING;   // "keep going" after the win screen
        if (!board.hasMove()) status = GameStatus.LOST;
    }
}
```

`Main.java` prints merge results for seven sample lines, shows a no-op move, checks game-over on preset boards, prints the key codes, and plays a seeded game (`new Random(42)`) with the corner strategy LEFT > DOWN > RIGHT > UP and target 128.

Output:

```
== 1. Merge rules for one line moving left ==
  [2, 2, 2, 2] -> [4, 4, 0, 0]  +8
  [2, 2, 2, 0] -> [4, 2, 0, 0]  +4
  [2, 2, 4, 0] -> [4, 4, 0, 0]  +4
  [4, 0, 4, 8] -> [8, 8, 0, 0]  +8
  [2, 0, 0, 2] -> [4, 0, 0, 0]  +4
  [8, 4, 2, 2] -> [8, 4, 4, 0]  +4
  [2, 4, 8, 16] -> [2, 4, 8, 16]  +0

== 2. A move that changes nothing does not count and spawns nothing ==
  move LEFT changed board? false, moves=0
  move DOWN changed board? true, moves=1
       .    .    .    .
       .    .    .    .
       .    2    .    .
       2    4    8   16

== 3. Game-over detection ==
  checkerboard, no empty cell, no equal neighbours -> LOST
  same but last row ends 4,4 -> PLAYING

== 4. Keyboard codes from the problem statement ==
  0=LEFT  1=RIGHT  2=UP  3=DOWN

== 5. Seeded game (seed 42), corner strategy LEFT > DOWN > RIGHT > UP, target 128 ==
  start:
       .    .    .    2
       .    .    .    .
       .    .    .    2
       .    .    .    .
  reached 128 after 85 moves (status WON), score 852
       8    2    .    .
      16    4    4    .
       2    8    4    .
     128    4    .    .
  game over after 232 moves, score 2584, max tile 256
      16    8    4    2
       8   32    8    4
      16    2   16    8
     256   64   32   16
```

Reading the output. Part 1 unit-tests the merge rule (note `[2,2,2,2]` becomes `[4,4,0,0]`, not `[8,0,0,0]`, and `[2,2,2,0]` becomes `[4,2,0,0]`). Part 2 shows that a move changing nothing does not count. Part 3 checks game-over detection on preset boards. Part 4 prints the command-line key mapping. Part 5 plays a whole seeded game that starts from two 2s, using a simple "keep tiles in the bottom-left corner" priority strategy; it reaches the 128 target after 85 moves (status `WON`) and keeps playing until the board locks after 232 moves with a 256 tile.

Concurrency & edge cases

- Single-player and single-threaded by nature. If a UI thread (the one drawing the screen) and an input thread both call `move`, make `move` `synchronized` or post moves to the UI event queue, which the UI thread handles one by one.
- **Key repeat / double input:** a no-op move returns `false` and changes nothing, so extra key presses are harmless.
- **Spawning needs an empty cell:** guaranteed, because a changed board after sliding always has at least one empty cell (tiles only move into gaps or merge, which frees a cell). The spawner still checks and throws if not.
- **Integer overflow** (a value wrapping past its type's maximum): not reachable on 4×4 (the maximum possible tile is 2^17 = 131072); any practical board still fits an `int`.
- **Merge order matters:** `[2,2,2]` moving left is `[4,2,0]` — the pair nearest the wall merges first.

Extensions & follow-up questions

- **"Add undo."** — Memento: push a copy of the board and score before each successful move; undo pops it (the spawned tile disappears too).
- **"Different sizes / targets?"** — Already parameters (`new Game(5, 4096, spawner)`).
- **"An AI player?"** — Expectimax search (max over your moves, average over random spawns) is the well-known approach; plug it in as a move-choosing strategy like the corner strategy in `Main`.
- **"Animations?"** — Have `LineMerger` also return where each tile came from (a list of moves/merges) and publish them to the view (Observer, MVC).
- **"High scores / leaderboard?"** — A separate service; the game only emits "game over with score X".
- **"Why is `[2,2,4]` left `[4,4,0]` and not `[8,0,0]`?"** — Each tile merges at most once per move; the new 4 was created this move.

Real-world example

2048 was created by Gabriele Cirulli in 2014 as open-source JavaScript with the same separation: a grid model, a game manager that handles moves, merges, score and "keep playing" after a win, and a separate view that animates tiles. Its new tiles are 2 with 90% probability and 4 with 10%, the same rule our `RandomTileSpawner` uses.

In the interview

> **30-second answer:** "The core is one pure function that slides and merges a single line toward index 0: skip zeros, merge equal neighbours once, score the merged values. The board maps each direction to lines in the right order, so the function is reused four times. After a move that changes the board, a `TileSpawner` — seeded for tests — adds a 2 (90%) or a 4. The game is won when the target tile appears and over when there's no empty cell and no equal neighbours."

Time plan: 5 min rules (merge once, no-op move) → 5 min entities → 15 min `LineMerger` + direction mapping (write the tests first: `[2,2,2,2]`, `[2,2,4,0]`) → 10 min game loop, spawning, game over → 5 min extensions.

Self-check

1. **Q:** What does `[4,4,8,8]` become moving left, and how many points? **A:** `[8,16,0,0]`, 24 points.
2. **Q:** Why must a no-op move not spawn a tile? **A:** It is not a turn in the real game; spawning would punish harmless key presses and change the game's balance.
3. **Q:** How do you avoid four copies of the slide code? **A:** Map each direction to lines read from the wall, then call one line-merge function.
4. **Q:** When is the game over? **A:** No empty cell and no two equal adjacent tiles.
5. **Q:** How do you make a 2048 test repeatable? **A:** Inject `new Random(seed)` into the spawner, or start from `Board.of(...)`.

TL;DR

- One pure `LineMerger` (merge once per move, from the wall) + a direction→line mapping.
- Spawn only after a real change; 2 (90%) or 4 (10%), seeded spawner for tests.
- WON once at the target (keep playing); LOST when no empty cell and no equal neighbours.

Design a Jigsaw Puzzle

Problem & clarifying questions

A **jigsaw puzzle** is a picture cut into many small pieces that the player puts back together. Each piece is roughly square and has **four sides**. Every side is one of three types:

- **Flat** (also called an *edge* side) — a straight side. It can only lie on the outer frame of the picture.
- **Tab** — a knob that sticks out.
- **Blank** — a hole that a tab slots into.

Two neighbouring pieces **interlock** (fit together) when one has a tab, the other a blank, and the two have the **same cut profile** (the exact wavy outline the cutter made). The puzzle is solved when every cell of the frame holds a piece and every shared side interlocks.

The interview tests whether you put the **matching rule** in one place and the **search algorithm** behind a swappable interface.

Question	Assumed answer
Shape of the puzzle?	A rectangle of `rows × cols` cells, at least 2 × 2. Each piece fills one cell.
How do we know two sides fit?	Each non-flat side carries a **shape code** (an integer standing for its cut profile). A tab fits a blank with the same code. Real systems would get this from image analysis; we take it as given.
May pieces be turned?	Yes, by quarter turns (0°, 90°, 180°, 270°). Pieces are not flipped over — the back has no picture.
Do we match on picture content?	No. Only shapes. (Picture matching is an extension.)
Who places pieces?	A human player (one, or several in a co-op mode), and/or an automatic **solver**.
Can two different pieces have the same shape?	In real puzzles, almost never. Our generator can do both: unique shapes, or a few reused shapes ("look-alike" pieces), which forces the solver to search.

Requirements

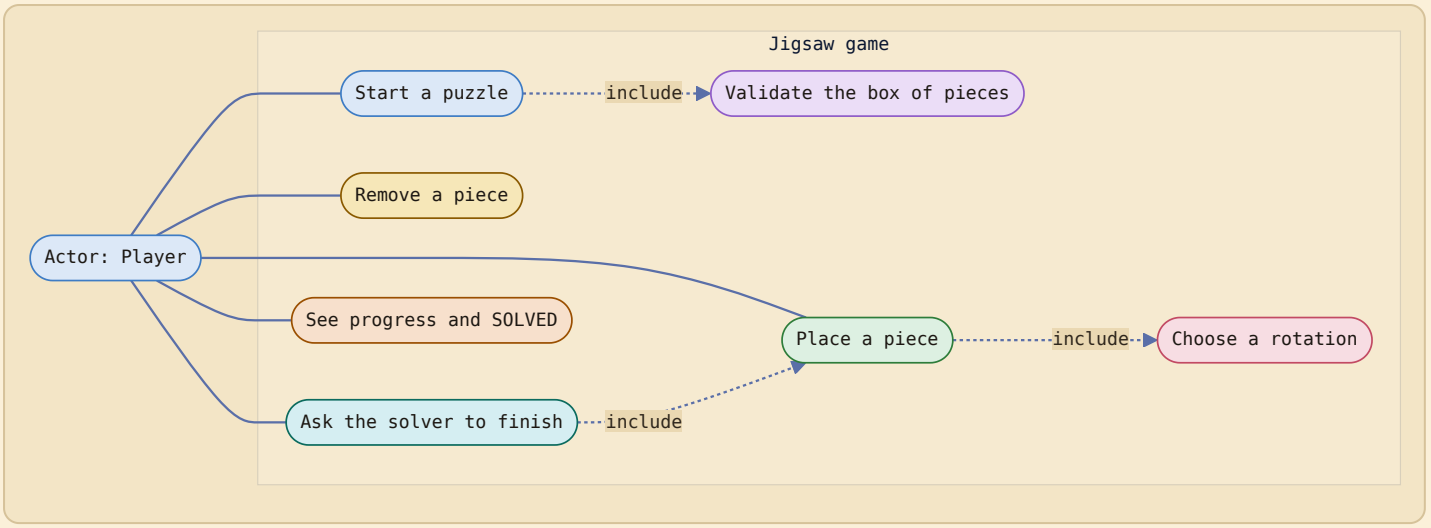
Functional

1. Model a piece with four sides, each FLAT, TAB or BLANK; classify a piece as **corner** (two flat sides), **border** (one flat side) or **interior** (no flat side).
2. Decide whether two sides interlock.
3. Place a piece on a cell with a given rotation; reject the placement, with a reason, if it breaks a rule: off the board, cell taken, piece already used, a non-flat side facing out of the picture, a flat side facing in, or a side that does not interlock with an already-placed neighbour.
4. Remove a piece from a cell.
5. **Validate** a box of pieces before play (right count, 4 corners, right number of border pieces, every tab has a matching blank).
6. **Solve** the puzzle automatically, from empty or from a partly built board, with a **pluggable** solving algorithm.
7. Report progress and detect the SOLVED state.

Non-functional

- One place for the placement rules, so player moves and solver moves can never disagree.
- New solving algorithms can be added without changing `Board`, `Piece` or the game (Open/Closed principle).
- Reproducible: the puzzle generator takes an injected, seeded `Random`.
- Co-op play: several players may place pieces at the same time.

Use cases



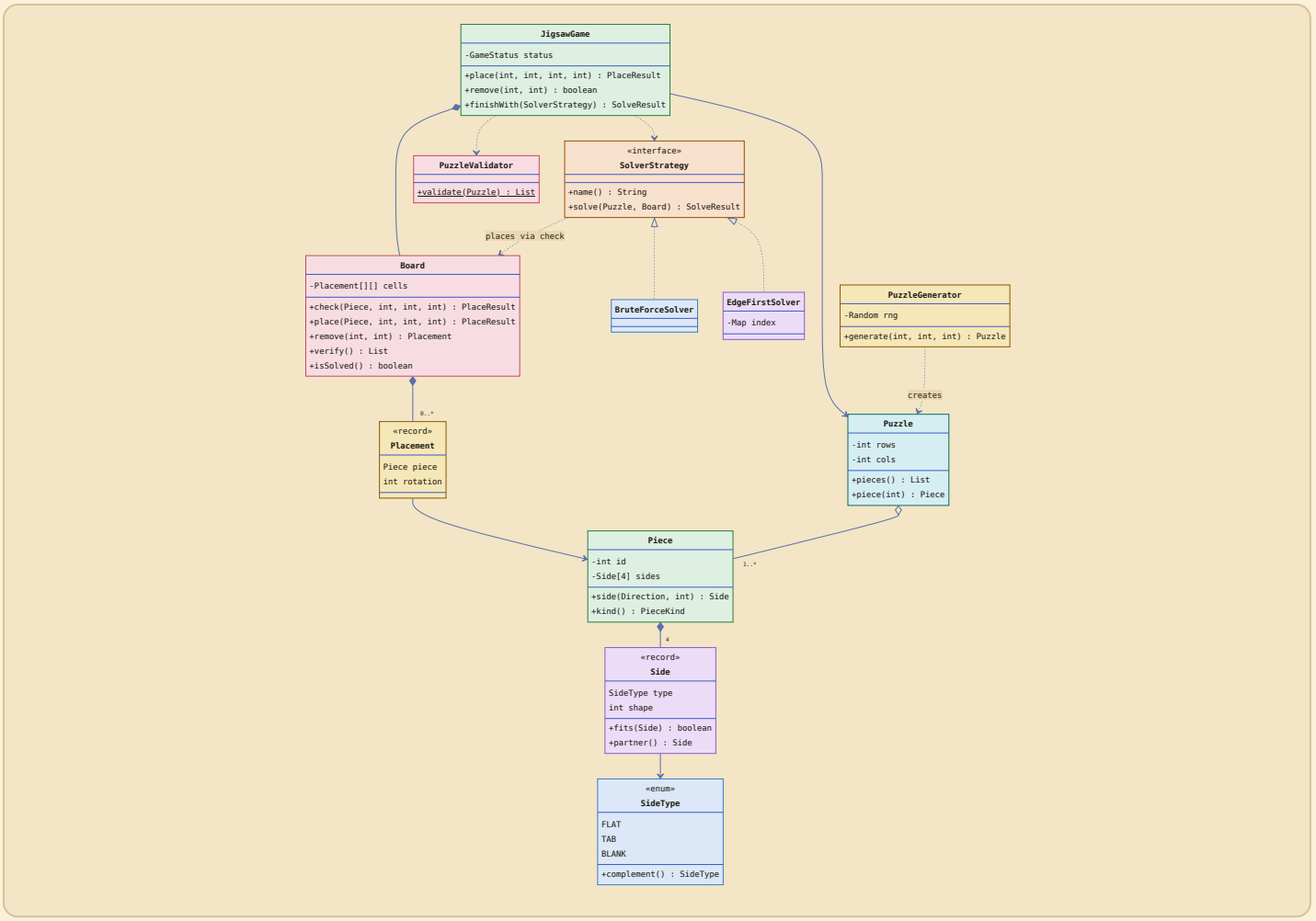
The solver is part of the system, not an actor. "Ask the solver to finish" *includes* "Place a piece": the solver goes through the same placement use case as a human, which is the key design point below. Starting *includes* validation, so a broken box is refused up front.

Core entities

Class	Responsibility
`SideType` (enum)	FLAT, TAB, BLANK; `complement()` (TAB ↔ BLANK).
`Side` (record)	Type + shape code; `fits(other)`, `partner()` (the one side that fits this one).
`Direction` (enum)	TOP, RIGHT, BOTTOM, LEFT in clockwise order; row/column offset to the neighbour; `opposite()`.
`Piece`	Immutable (never changes after creation): id + four sides as found in the box. `side(direction, rotation)`, `kind()`.
`PieceKind` (enum)	CORNER, BORDER, INTERIOR — derived from the number of flat sides.
`Board`	The `rows × cols` grid of `Placement(piece, rotation)`. **Owns every placement rule** in `check(...)`; `place`, `remove`, `verify`, `isSolved`.
`PlaceResult` (enum)	OK or the exact reason a placement was refused.
`Puzzle`	What is in the box: size + list of pieces, lookup by id.
`PuzzleGenerator`	Cuts a picture into pieces, then turns and shuffles them (seeded).
`PuzzleValidator`	Cheap sanity checks on a box before solving.
`SolverStrategy` (interface) + `BruteForceSolver`, `EdgeFirstSolver`	Interchangeable solving algorithms; `SolveResult` reports attempts and backtracks.
`JigsawGame` + `GameStatus`	One puzzle in progress: validates on start, places/removes for players, `finishWith(solver)`, status. Thread-safe.

Why is rotation not a field of `Piece`? How a piece is turned is a fact about *where it lies*, not about the piece. Keeping rotation in `Board.Placement` leaves `Piece` immutable, so a solver can try four rotations with nothing to undo (a mutating `piece.rotate()` would have to be turned back after every failed try).

Class diagram



A `Piece` is **composed** of four `Side`s; a `Placement` only **refers** to a piece. Solvers **depend on** `Board.place`/`remove` and have no rules of their own.

Key flows

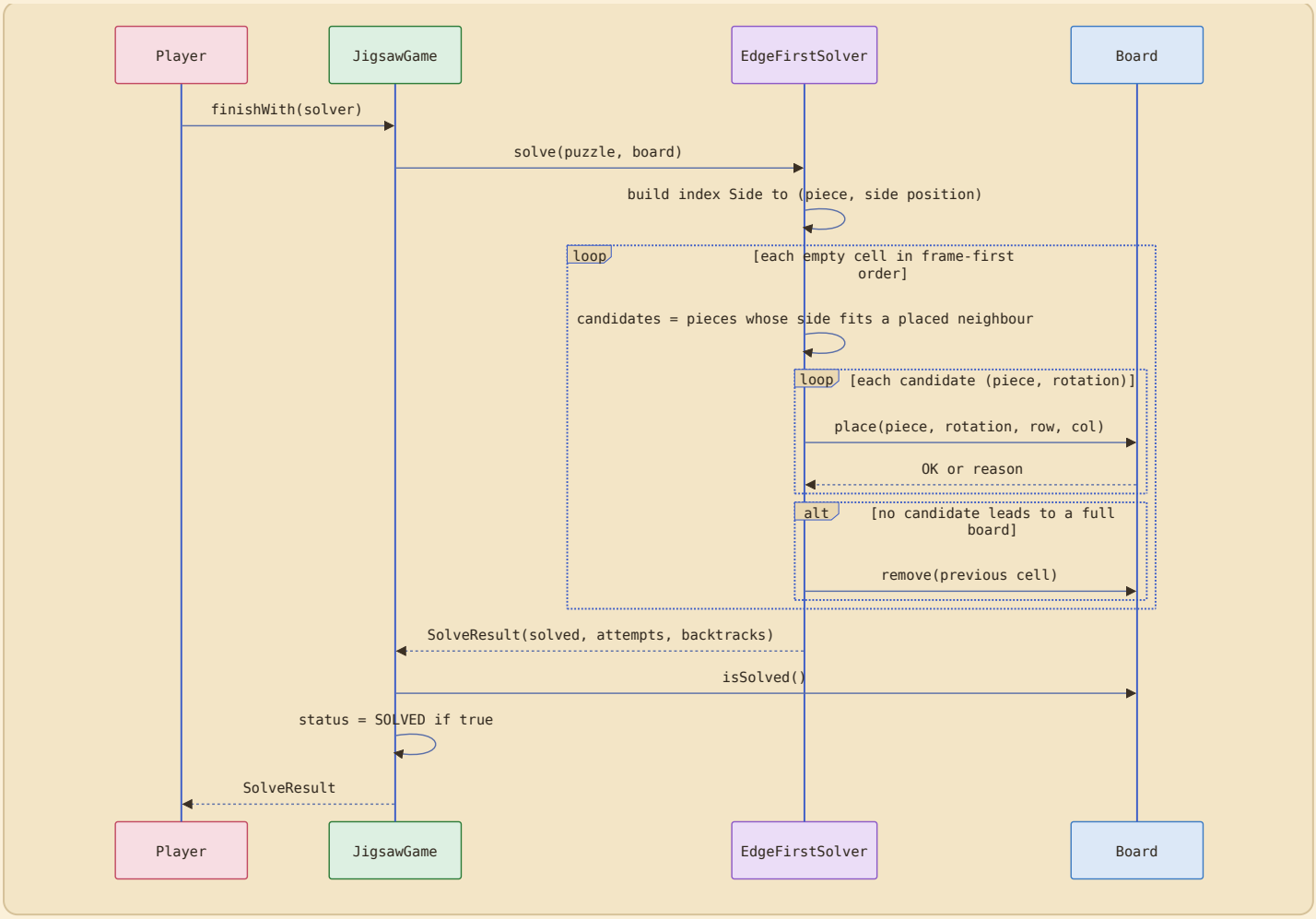
1. Rotation in one line. Sides are stored clockwise as `[TOP, RIGHT, BOTTOM, LEFT]` at rotation 0. Turning a piece clockwise once moves what was on TOP to RIGHT. So after `r` clockwise quarter turns, the side now facing direction `d` is the one stored at index `(d − r) mod 4`. Example from the program's output: the corner `[F, T1, B2, F]` turned once has `TOP=F RIGHT=F BOTTOM=T1 LEFT=B2`.

2. The placement check (`Board.check`) — one method, used by the player and by every solver:

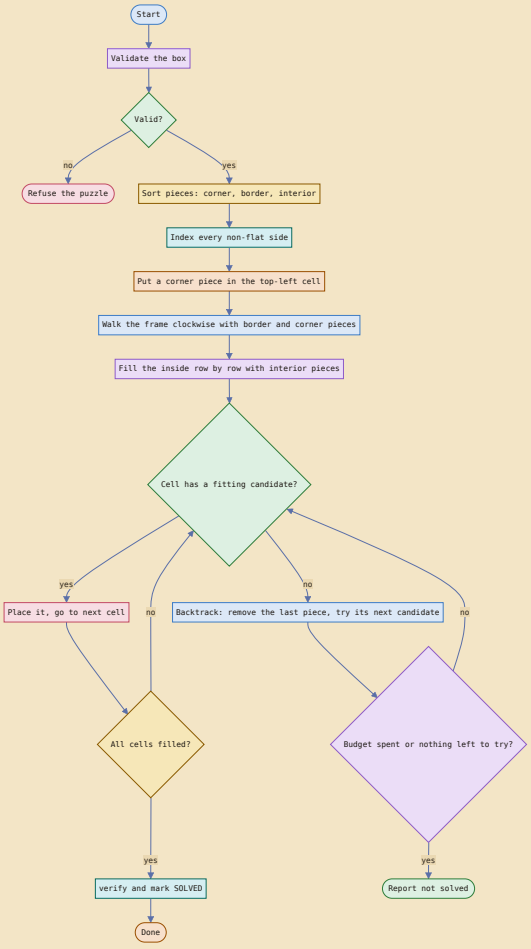
1. Off the board → OUT_OF_BOUNDS. Cell taken → CELL_OCCUPIED. Piece already used elsewhere → ALREADY_PLACED.
2. For each of the four directions, take the side that faces it (after rotation):
   - The neighbour cell is **outside** the picture → the side must be FLAT, else EDGE_MUST_BE_FLAT.
   - The neighbour cell is **inside** → the side must **not** be FLAT (FLAT_FACES_INSIDE); and if a piece is already there, the two sides must interlock (SIDES_DO_NOT_FIT).
3. Otherwise OK. It is O(1): four sides, four neighbours.

Note what this gives for free: a corner piece can only go in a corner cell, and only in the one rotation that turns both flat sides outward. No separate "corner rule" is needed.

3. Asking the solver to finish (sequence diagram):



4. The edge-first solving method (activity diagram) — the way most people do a jigsaw:



Backtracking (used in steps I–M) means: make a choice, carry on, and if you reach a dead end, undo the most recent choice and try the next option. With every shape unique, a correct choice is always the only choice and the solver never backtracks. With look-alike pieces, a wrong piece can fit locally and the mistake is only discovered later, so the solver must undo work.

Why the index makes such a difference. For the next cell, one neighbour is always already placed (that is what the frame-first order guarantees). The cell needs the **partner** of that neighbour's facing side — for example, if the left neighbour shows `T7` on its right, this cell needs `B7` on its left. A hash map from `Side` to the pieces that have it returns those pieces in O(1), and the rotation follows directly: if the needed side sits at index `i` of the piece and must face direction `d`, rotation = `(d − i) mod 4`. With unique shapes there is exactly one candidate per cell, so the whole puzzle is solved in about n attempts for n pieces instead of $n^2 \cdot 4$.

Design decisions & patterns used

Decision	Why	Alternative rejected
All placement rules in `Board.check`.	A human move and a solver move are judged identically; one place to fix a rule.	Each solver re-implementing "does it fit?" — they drift apart.
Strategy for solving — `SolverStrategy` (see §11.3 Strategy).	Brute force, edge-first, or a future picture-aware solver plug in without touching `Board` or `JigsawGame`.	A `switch (algorithm)` inside `JigsawGame`.
`Side` as an immutable **value** (record) with `partner()`.	Equal sides are `equals`, so a side can be a map key — that is what makes the index possible.	Sides as mutable objects compared by identity.
Placement returns a `PlaceResult` **reason**, not `boolean` or an exception.	A UI can show "that side doesn't fit" vs "that's an edge piece"; refusals are normal play, not errors.	Exceptions for routine wrong guesses.
Validate before play (`PuzzleValidator`).	A box with a missing piece is reported in O(n) instead of a solver searching forever.	Discovering the problem by failing to solve.
Solvers carry an **attempt budget**.	Search can explode on look-alike pieces; a bounded solver always returns.	Unbounded recursion on user-supplied data.

Java implementation

Package `lld.ch17.jigsaw` — 16 files in `code/lld/ch17/jigsaw/`; run `lld.ch17.jigsaw.Main` as shown at the top of the chapter. Only the key parts are below; the enums, `Puzzle`, `PuzzleGenerator`, `PuzzleValidator`, `BruteForceSolver`, `JigsawGame` and `Main` are in the folder.

```java
// File: code/lld/ch17/jigsaw/Side.java (excerpt)
package lld.ch17.jigsaw;

/**
 * One side of a piece: its type and a shape code. Two sides interlock when one is a TAB, the other a BLANK,
 * and both have the same shape code (the same cut profile). A FLAT side always has shape 0.
 */
public record Side(SideType type, int shape) {
    public static final Side FLAT = new Side(SideType.FLAT, 0);

    // compact constructor validates: FLAT has shape 0, TAB/BLANK has shape > 0; factories tab(n), blank(n)

    public boolean isFlat() { return type == SideType.FLAT; }

    /** The only side that fits against this one. */
    public Side partner() { return new Side(type.complement(), shape); }

    public boolean fits(Side other) {
        return !isFlat() && !other.isFlat() && other.type == type.complement() && other.shape == shape;
    }
}
```

```java
// File: code/lld/ch17/jigsaw/Piece.java (excerpt)
public final class Piece {
    private final int id;
    private final Side[] sides;   // index = Direction.ordinal() at rotation 0: TOP, RIGHT, BOTTOM, LEFT

    public Piece(int id, Side top, Side right, Side bottom, Side left) {
        this.id = id;
        this.sides = new Side[]{top, right, bottom, left};
        if (flatCount() > 2) throw new IllegalArgumentException("piece " + id + " has more than two flat sides");
        if ((top.isFlat() && bottom.isFlat()) || (left.isFlat() && right.isFlat()))
            throw new IllegalArgumentException("piece " + id + ": two flat sides must be adjacent (a corner)");
    }

    /** Turning clockwise once moves what was on TOP to RIGHT, so the side now facing d came from d - rotation. */
    public Side side(Direction d, int rotation) {
        return sides[Math.floorMod(d.ordinal() - rotation, 4)];
    }

    public PieceKind kind() {
        return switch (flatCount()) { case 2 -> PieceKind.CORNER; case 1 -> PieceKind.BORDER; default ->
PieceKind.INTERIOR; };
    }
    // id(), side(int), flatCount(), toString() ...
}
```

`Math.floorMod` is used instead of `%` because Java's `%` keeps the sign of the left operand: `(0 - 1) % 4` is `-1`, while `floorMod(-1, 4)` is `3`.

```java
// File: code/lld/ch17/jigsaw/Board.java (excerpt)
public final class Board {
    /** A piece lying on a cell, turned clockwise 'rotation' quarter turns. */
    public record Placement(Piece piece, int rotation) {
        public Side side(Direction d) { return piece.side(d, rotation); }
    }

    private final int rows, cols;
    private final Placement[][] cells;
    private final Set<Integer> placedIds = new HashSet<>();

    /** Every rule for one placement, checked against the frame and against neighbours already placed. */
    public PlaceResult check(Piece piece, int rotation, int r, int c) {
        if (rotation < 0 || rotation > 3) throw new IllegalArgumentException("rotation must be 0..3");
        if (!inside(r, c)) return PlaceResult.OUT_OF_BOUNDS;
        if (cells[r][c] != null) return PlaceResult.CELL_OCCUPIED;
        if (placedIds.contains(piece.id())) return PlaceResult.ALREADY_PLACED;
        for (Direction d : Direction.values()) {
            Side mine = piece.side(d, rotation);
            int nr = r + d.dRow, nc = c + d.dCol;
            if (!inside(nr, nc)) {
                if (!mine.isFlat()) return PlaceResult.EDGE_MUST_BE_FLAT;
                continue;
            }
            if (mine.isFlat()) return PlaceResult.FLAT_FACES_INSIDE;
            Placement neighbour = cells[nr][nc];
            if (neighbour != null && !mine.fits(neighbour.side(d.opposite()))) return
PlaceResult.SIDES_DO_NOT_FIT;
        }
        return PlaceResult.OK;
    }

    // place() = check(), then store the Placement if OK; remove(r, c), kindOfCell(r, c), verify() (re-checks
every joint of a finished board),
    // isSolved() = isFull() && verify().isEmpty(), render() ...
}
```

```java
// File: code/lld/ch17/jigsaw/SolverStrategy.java
package lld.ch17.jigsaw;

/**
 * Strategy pattern: one interface, many ways to solve. A solver works on the given board, keeps any
 * pieces already placed, and fills the empty cells using the pieces not yet on the board.
 */
public interface SolverStrategy {
    String name();

    SolveResult solve(Puzzle puzzle, Board board);
}
```

`BruteForceSolver` is the baseline: it visits cells row by row and, at each empty cell, tries **every** unused piece in all four rotations, backtracking on dead ends. `EdgeFirstSolver` is the human method with the side index:

```java
// File: code/lld/ch17/jigsaw/EdgeFirstSolver.java (excerpt)
public final class EdgeFirstSolver implements SolverStrategy {
    private record Hit(Piece piece, int sideIndex) {}

    private final long maxAttempts;
    private long attempts, backtracks;
    private boolean gaveUp;
    private Map<Side, List<Hit>> index;

    @Override public SolveResult solve(Puzzle puzzle, Board board) {
        attempts = backtracks = 0;
        gaveUp = false;
        index = new HashMap<>();
        for (Piece p : puzzle.pieces())
            for (int i = 0; i < 4; i++)
                if (!p.side(i).isFlat()) index.computeIfAbsent(p.side(i), k -> new ArrayList<>()).add(new Hit(p,
i));
        boolean ok = fill(puzzle, board, order(board), 0);
        return new SolveResult(name(), ok, attempts, backtracks);
    }

    /** Frame first, clockwise from the top-left corner; then the inside, row by row. */
    static List<int[]> order(Board b) {
        List<int[]> cells = new ArrayList<>();
        int R = b.rows() - 1, C = b.cols() - 1;
        for (int c = 0; c <= C; c++) cells.add(new int[]{0, c});
        for (int r = 1; r <= R; r++) cells.add(new int[]{r, C});
        for (int c = C - 1; c >= 0; c--) cells.add(new int[]{R, c});
        for (int r = R - 1; r >= 1; r--) cells.add(new int[]{r, 0});
        for (int r = 1; r < R; r++) for (int c = 1; c < C; c++) cells.add(new int[]{r, c});
        return cells;
    }

    private boolean fill(Puzzle puzzle, Board board, List<int[]> order, int k) {
        if (k == order.size()) return true;
        int r = order.get(k)[0], c = order.get(k)[1];
        if (board.at(r, c) != null) return fill(puzzle, board, order, k + 1);   // keep the player's pieces
        for (Board.Placement option : candidates(puzzle, board, r, c)) {
            if (gaveUp || attempts >= maxAttempts) { gaveUp = true; return false; }   // budget spent
            attempts++;
            if (board.place(option.piece(), option.rotation(), r, c) != PlaceResult.OK) continue;
            if (fill(puzzle, board, order, k + 1)) return true;
            board.remove(r, c);                                                  // dead end: undo, try next
            if (!gaveUp) backtracks++;
        }
        return false;
    }

    /** Looked up by the side a placed neighbour needs, else every unused piece of the right kind. */
    private List<Board.Placement> candidates(Puzzle puzzle, Board board, int r, int c) {
        PieceKind wanted = board.kindOfCell(r, c);
        List<Board.Placement> out = new ArrayList<>();
        for (Direction d : Direction.values()) {
            int nr = r + d.dRow, nc = c + d.dCol;
            if (!board.inside(nr, nc) || board.at(nr, nc) == null) continue;
            Side needed = board.at(nr, nc).side(d.opposite()).partner();   // what must face d from this cell
            for (Hit h : index.getOrDefault(needed, List.of()))
                if (!board.isPlaced(h.piece().id()) && h.piece().kind() == wanted)
                    out.add(new Board.Placement(h.piece(), Math.floorMod(d.ordinal() - h.sideIndex(), 4)));
            return out;                                                          // one neighbour narrows enough
        }
        for (Piece p : puzzle.pieces())                                          // no neighbour yet: the first corner
            if (!board.isPlaced(p.id()) && p.kind() == wanted)
                for (int rot = 0; rot < 4; rot++) out.add(new Board.Placement(p, rot));
        return out;
    }
}
```

The solver only *proposes*; `board.place` still runs the full check, so a candidate that fits the left neighbour but not the top one is refused there. `PuzzleValidator.validate` counts pieces, corners (must be

4), border pieces (must be `2(rows−2) + 2(cols−2)`), interior pieces, and, per shape code, checks that the number of tabs equals the number of blanks. These checks are **necessary but not sufficient**: a box that fails can never be solved; a box that passes still might not be.

`JigsawGame` is a thin controller: its constructor runs the validator and refuses a bad box; `place` delegates to `board.place`, counts the move and sets `SOLVED` when `board.isSolved()`; `finishWith(solver)` calls `solver.solve(puzzle, board)` and re-checks the status. All its public methods are `synchronized`.

Output:

```
== 1. Sides and rotation ==
  T3 fits B3? true | T3 fits T3? false | T3 fits B4? false | F fits F? false
  piece #99[F, T1, B2, F] is a CORNER
  rotation 0: TOP=F RIGHT=T1 BOTTOM=B2 LEFT=F
  rotation 1: TOP=F RIGHT=F BOTTOM=T1 LEFT=B2
  rotation 2: TOP=B2 RIGHT=F BOTTOM=F LEFT=T1
  rotation 3: TOP=T1 RIGHT=B2 BOTTOM=F LEFT=F

== 2. A 3 x 4 puzzle out of the box (seed 7) ==
  #1[B8, B16, T9, F] BORDER
  #2[T15, T4, T11, B5] INTERIOR
  #3[B9, T17, F, F] CORNER
  #4[T12, B2, F, T3] BORDER
  #5[F, B13, B3, F] CORNER
  #6[B11, B1, F, T2] BORDER
  #7[T14, T7, F, F] CORNER
  #8[T10, F, F, T1] CORNER
  #9[T6, T13, F, B17] BORDER
  #10[T8, F, B7, B15] BORDER
  #11[B4, B14, F, B10] BORDER
  #12[T16, T5, B12, B6] INTERIOR
  validation problems: []

== 3. A player places pieces; the board enforces every rule ==
  piece 12 (interior) at corner (0,0)    EDGE_MUST_BE_FLAT
  piece 3 at (0,0), wrong turn           FLAT_FACES_INSIDE
  piece 3 at (0,0), right turn           OK
  piece 1 at (0,0) too                   CELL_OCCUPIED
  piece 3 again, at (2,3)                ALREADY_PLACED
  piece 1 at (0,1), wrong turn           EDGE_MUST_BE_FLAT
  piece 1 at (0,1), right turn           OK
  piece 12 at (1,1), right turn          OK
  piece 1 at (5,5)                       OUT_OF_BOUNDS
  progress 3/12, status IN_PROGRESS
      3r1   1r1     .     .
        .  12r0     .     .
        .     .     .     .
  solver finishes the rest: EdgeFirst+Index    solved=true  attempts=        9 backtracks=        0
  progress 12/12, status SOLVED
      3r1   1r1  10r3   7r2
      9r1  12r0   2r0  11r3
      5r3   4r0   6r0   8r0

== 4. Validation catches a bad box ==
  piece 12 lost: [expected 12 pieces, found 11, expected 2 interior pieces, found 1, shape 5: 0 tabs but 1 blanks,
shape 6: 1 tabs but 0 blanks, shape 12: 1 tabs but 0 blanks, shape 16: 0 tabs but 1 blanks]
  one side mis-cut: [shape 8: 2 tabs but 0 blanks]
  new JigsawGame(...) refused: invalid puzzle: [shape 8: 2 tabs but 0 blanks]

== 5. Same puzzles, two strategies ==
  4 x 4, shapeVariety 0 (every joint unique)
    BruteForce         solved=true  attempts=      209 backtracks=        0  verify=true
    EdgeFirst+Index    solved=true  attempts=       16 backtracks=        0  verify=true
  6 x 6, shapeVariety 0 (every joint unique)
    BruteForce         solved=true  attempts=    1,171 backtracks=        0  verify=true
    EdgeFirst+Index    solved=true  attempts=       38 backtracks=        0  verify=true
  6 x 6, shapeVariety 2 (look-alike pieces)
    BruteForce         solved=true  attempts=2,099,470 backtracks=   46,642  verify=true
    EdgeFirst+Index    solved=true  attempts=  501,976 backtracks=   72,683  verify=true
  10 x 10, shapeVariety 0 (every joint unique)
    BruteForce         solved=true  attempts=   11,060 backtracks=        0  verify=true
    EdgeFirst+Index    solved=true  attempts=      101 backtracks=        0  verify=true

== 6. Co-op: four players place pieces of one 6 x 6 puzzle at once ==
  progress 36/36, moves 36, status SOLVED
```

Reading the output. Cell labels such as `3r1` mean "piece 3, turned once". Part 3 hits every refusal reason; an interior piece is refused at a corner and a corner piece in the wrong rotation with no special-case code, and the solver then finishes in 9 attempts. Part 5 runs both strategies on the *same*

seeded puzzles: with unique shapes edge-first needs about one attempt per piece (16, 38, 101) while brute force grows roughly with the square of the piece count. With look-alikes both must search; edge-first makes 4× fewer attempts but backtracks more, because a wrong frame piece is exposed only when the frame closes. Part 6: four threads, one game, exact count.

Concurrency & edge cases

- **Co-op play.** `JigsawGame`'s public methods are `synchronized` (only one thread at a time runs them on a given game), so two players cannot both drop a piece into the same cell or both use the same piece: the second one gets CELL_OCCUPIED or ALREADY_PLACED. A server hosting many puzzles has one lock per game, so games do not block each other. Finer-grained locking (per cell) is possible but must still make "is the piece unused?" and "is the cell empty?" one atomic step (all-or-nothing, never seen half-done).
- **Solver and players at once.** `finishWith` holds the game lock for the whole solve, so players wait; for big puzzles, solve on a *copy* of the board and offer the result as hints.
- **Invalid box** (missing piece, extra piece, three flat sides, opposite flat sides, unbalanced tabs) — rejected by the `Piece` constructor or `PuzzleValidator` before play.
- **Look-alike pieces** — two pieces with identical sides are interchangeable; any arrangement that passes `verify` counts as solved, even if the picture would be scrambled. Only picture matching can tell them apart.
- **Search blow-up** — the attempt budget makes every solver return; `solved=false` leaves the board exactly as it was before the call (every trial placement is undone).
- **1 × N strips** are excluded (a piece there has two *opposite* flat sides); supporting them means relaxing the corner rule in `Piece` and the counts in `PuzzleValidator`.

Extensions & follow-up questions

- **"Match on the picture too."** — Give `Side` an edge colour signature (for example, average colours sampled along the edge) and let a new strategy rank candidates by colour difference. Only a new `SolverStrategy` (and a richer `Side`) is needed; `Board` is unchanged.
- **"Make the solver faster on look-alikes."** — Choose the next cell dynamically: the empty cell with the **fewest** candidates first (the "most constrained variable" heuristic used in constraint solving), and check both placed neighbours when building candidates.
- **"Undo."** — Each successful `place` is a command on a stack; undo = `remove` (Command, §10.2).
- **"Show progress live / multiplayer UI."** — Publish `PiecePlaced` events (Observer, §11.1).
- **"Non-rectangular puzzles or hexagonal pieces."** — Generalise `Direction` to N sides and `Board` to a graph of cells and neighbours; the side-matching rule stays the same.

Real-world example

Computer jigsaw solving is a studied problem: matching only by shape is hard in general (with arbitrary look-alike pieces it is NP-complete — no known fast algorithm guarantees a solution), so practical solvers combine edge-first ordering with **picture-content** scores between neighbouring edges and greedy or best-first placement.

In the interview

30-second answer: "A `Piece` is immutable with four `Side`s stored clockwise; a `Side` is FLAT, TAB or BLANK plus a shape code, and a tab fits a blank with the same code. Rotation lives in the board's `Placement`, and the side facing direction d after r turns is `sides[(d − r) mod 4]`. `Board.check` holds every rule — flat sides out, no flat side in, every placed neighbour interlocks — and both players and solvers use it. Solving is a Strategy: brute-force backtracking, or edge-first with a hash index from side to pieces, which is about one attempt per piece when shapes are unique. A validator rejects broken boxes up front, and `JigsawGame` is synchronized for co-op."

Likely follow-ups with answers

- *"What is the cost of solving?"* — Edge-first with unique shapes: O(n) attempts, each O(1). With look-alikes: exponential in the worst case, which is why there is a budget.
- *"What does validation guarantee?"* — Only that the box *could* be solvable; failing it proves it cannot be.

Self-check

1. **Q:** When do two sides interlock? **A:** One is a TAB, the other a BLANK, with the same shape code. FLAT never interlocks.
2. **Q:** A piece stored as `[F, T1, B2, F]` is turned clockwise twice. What faces TOP? **A:** `sides[(0 − 2) mod 4] = sides[2] = B2`.
3. **Q:** How many border (non-corner) pieces does a 5 × 7 puzzle have? **A:** `2(5−2) + 2(7−2) = 16`.
4. **Q:** Why does the edge-first solver need only about one attempt per piece on a normal puzzle? **A:** Each next cell has a placed neighbour; the index returns the pieces having the partner side, and with unique shapes that is a single piece with a computed rotation.
5. **Q:** Name a box that passes validation but cannot be solved. **A:** A 2 × 2 box in which the only `T1` and the only `B1` are on the **same** piece: counts and tab/blank balance are fine, but a piece cannot interlock with itself, so that joint can never be filled.

TL;DR

- `Side` = FLAT/TAB/BLANK + shape code; fits = TAB↔BLANK with equal codes. `Piece` immutable, sides clockwise; rotation lives in `Placement`; facing side = `sides[(d − r) mod 4]`.
- `Board.check` is the single home of every rule (frame flat, no flat inside, neighbours interlock, cell free, piece unused) and returns a reason.
- Solving is a **Strategy**: brute-force backtracking vs edge-first with a `Side → pieces` index (≈ n attempts with unique shapes). Budget every search.
- Validate the box first (counts, 4 corners, tabs = blanks per shape); `synchronized` game for co-op.

Chapter summary

Game	Core rule object	Pluggable strategies	Randomness for tests	Signature technique
Blackjack	`Hand` scoring + `BlackjackScoring`	`PlayerStrategy`, `DealerPolicy`	Seeded `Shoe`, stacked shoe	Soft/hard totals; generic `Deck<T>`
Chess	`Rules` (legal-move filter)	(bot would be a move strategy)	None needed — deterministic game	Polymorphic pieces; checkmate = in check + no legal move; Command + Memento undo
Tic-Tac-Toe	`Board` counters	`MoveStrategy`	Seeded `RandomStrategy`, scripted players	O(1) win with row/col/diagonal counters
Snake & Ladder	`Board` + `Rules` record	`Dice`, `OverflowRule`, `WinStrategy`	Seeded `RandomDice`, `LoadedDice`	Validated config; queue-based turns
2048	`LineMerger`	`TileSpawner`	Seeded spawner, `Board.of` presets	One merge function reused for 4 directions
Jigsaw	`Board.check` (one placement rule)	`SolverStrategy`	Seeded `PuzzleGenerator`	Side matching + rotation formula; edge-first solver with a side index

Patterns you used in this chapter: Strategy (everywhere), Factory Method (`Deck.standard52`, `Piece.create`, `Board.of`), Template-style shared base (`SlidingPiece`), Command + Memento (chess undo), State (game-status enums; a full State pattern is the suggested upgrade for blackjack rounds), Observer (the event log hooks; suggested for UIs and spectators).

The three habits interviewers look for in game designs:

1. **Put every rule in exactly one place** — and be able to say where.
2. **Inject randomness** — seeded or scripted — so the design is testable.
3. **Model the game's states explicitly** — and reject actions that are invalid in the current state.

Chapter 18 — Case Studies: Platforms & Feeds (Part 1)

> **Where this fits.** Chapters 14–17 designed *things* (a parking lot, a vending machine), *bookings* (movies, hotels, flights) and *games*. This chapter and the next one design **platforms**: websites where many people create content, react to each other's content and build up a history. The objects are no longer machines with buttons; they are people, posts, votes, relationships and feeds. Chapter 19 continues with more platforms.

What you will learn

#	Problem	What makes it interesting	Main patterns and ideas
18.0	What platform problems test	The common toolkit you will reuse in every social or community design	Facade, Observer, Strategy, services for two-sided operations, inverted index
18.1	Design Stack Overflow	Reputation unlocks privileges; one vote per member; moderation; bounties	Shared `Post` base class, Strategy (reputation rules), Observer (badges), privilege table
18.2	Design Cricinfo (+ Udaan "Cricket Match Dashboard")	Modelling a real sport's rules; live scorecards; stats derived from balls	Template Method (formats and stat queries), Observer (live feeds), "derive, don't store"
18.3	Design LinkedIn	Profiles, connections and their degrees, invitations, messaging, jobs, search, feed	Service for two-sided operations, State (invitation), BFS for degrees, Strategy (feed ranking)
18.4	Design Facebook	Friends vs followers, privacy lists, pages, groups, news feed, friend suggestions	One visibility policy for feed + search, sealed `Audience`, pull-model feed, suggestions

How to run every example in this chapter

```
cd chapters/18-case-platforms-1/code
javac --release 21 -d /tmp/lld_18 $(find . -name '*.java')
java -cp /tmp/lld_18 lld.ch18.stackoverflow.Main       # or cricinfo, cricketdashboard, linkedin, facebook
```

`javac` is the Java **compiler** (it turns `.java` source files into `.class` files the Java runtime can execute); `--release 21` says which Java version to target, and `-d` names the output folder. `java -cp` runs a program, where the **classpath** (`-cp`) is the list of folders in which Java looks for compiled classes. The **standard library** is the set of classes that ships with Java itself, so nothing extra has to be downloaded.

All five `Main` classes compile with Java 21 (standard library only) and run. The output printed under each design is the real output of that program; the listings show only the essential lines, and the complete files are in `code/`.

> **Code-style note.** A file sometimes holds several small package-private types (e.g. `Posts.java` holds `Post`, `Question`, `Answer`, `Comment`, `Tag` and `Bounty`). That is legal Java; production code usually gives each public top-level type its own file.

> **Key term — Java vocabulary used in this chapter:** A **package** is a named folder of related classes (`lld.ch18.stackoverflow`); a **package-private** type has no `public`/`private` keyword and can be used only by classes in the same package. A **top-level type** is a class, interface, enum or record not nested inside another one. An **interface** lists methods a class promises to provide, without the code; a class that provides them *implements* it. An **abstract class** can never be created directly — only its subclasses can. An **enum** is a type with a fixed list of named values (`OPEN`, `CLOSED`). A **record** is a

> short way to declare a class that just carries data that never changes. A **sealed** type names exactly which subclasses are allowed, so the compiler knows every possible kind.

18.0 What platform problems test

What it is

A **platform problem** asks you to design a website or app where *users produce the content* — questions and answers, posts and comments, match commentary, job postings. Stack Overflow, Cricinfo, LinkedIn and Facebook are the classic examples. Unlike a vending machine, there is no single "machine" object in the middle. Instead there is a **rich domain model**: many small classes (profile, post, comment, vote, tag, badge, invitation, group, page…) that each hold a little data and a little behaviour.

Everyday analogy: a vending machine is like a toaster — one box with a few buttons. A platform is like a town — many people, many buildings, rules about who may enter where, a notice board (the feed), a post office (notifications) and a library catalogue (search).

The problem it solves

Interviewers pick platform problems to check six skills at once. The table names each one, defines it, and shows where this chapter practises it.

Skill tested	What it means	Where you practise it
Rich domain model	Turning a long list of nouns into classes with clear responsibilities, and knowing which noun is *just data* (a record) and which has *behaviour*	Cricinfo's `Over` knows a wide is not one of its six balls
User roles	Different people may do different things: guest, member, moderator, admin, commentator	Stack Overflow's reputation-based privileges; Facebook's admin who blocks members
Feeds	A **feed** is a personalised, ranked list of recent content from the people and pages you are linked to	LinkedIn and Facebook feed services with a ranking Strategy
Notifications	Telling a user that something happened (new message, friend request, comment) through one or more **channels** (in-app, email, push — a pop-up alert sent to a phone or browser)	Observer + Strategy in every design
Search	Finding things by words. Built with an **inverted index**: a map from each word to the items that contain it	Question search, member search, privacy-aware post search
Reputation / trust	Numbers that summarise past behaviour and unlock abilities (reputation, badges, endorsements, ratings)	Stack Overflow reputation and badges; LinkedIn endorsements; Facebook page ratings

How it works — the reusable toolkit

These ideas repeat in every design in this chapter. Learn them once.

1. **A facade in front.** A **facade** is one class that offers the simple operations the UI (user interface — the screens and buttons people use) needs (`ask`, `vote`, `post`, `invite`) and hides the many classes behind it (Chapter 9). It is also the natural place to check "is this account active?" and "is this person allowed?" before anything changes.
2. **Behaviour shared by content goes in a base class.** Questions, answers and comments are all voted on; posts and comments are all liked. Write that once in a base class (`Post`, `SocialContent`) and let the content-specific parts stay in subclasses.
3. **Store *who*, not *how many*.** Keep likes and votes as a **set of member ids**, not an integer counter. A set blocks double voting, allows "unlike", and still gives the count as `set.size()`.
4. **Derive, don't store.** If a number can be computed from data you already keep (follower count, a batter's runs, a page's rating), compute it. A stored copy will one day disagree with the source

and nobody will know which is right. If it is too slow to compute every time, keep a **cache** (a saved copy of a computed answer, kept so it can be read quickly instead of recomputed) that you can always rebuild from the source. See System Design Fundamentals for caches at scale.

5. **An operation that changes two objects lives in a service.** Accepting a connection or friend request changes *both* members. Put it on neither — put it in a small **service** class (a class whose job is to perform an operation, not to hold data) so there is exactly one place that keeps both sides consistent.
6. **Notifications through Observer + Strategy.** The domain *publishes* events ("answer posted"); **observers** (Chapter 10) react. The **Observer** pattern means objects *subscribe* to another object and are called back automatically whenever it announces an event, so the announcer does not need to know who is listening. The **Strategy** pattern means putting each interchangeable way of doing a job (here: each delivery method) behind one interface, so you can swap or add ways without editing the caller. How a message is *delivered* (email, push, in-app) is a **Strategy** (Chapter 11) so adding a channel means adding a class, not another `if`.
7. **Search is its own class.** An **inverted index** (word → items) belongs in a `SearchIndex`, not on the entity. Rule from the course: every key maps to a *collection*, even when only one item matches, so the second "Sharma" never breaks the index.
8. **Privacy and permissions are asked in one place.** If both the feed and the search must hide private posts, write `canSee(viewer, post)` once and call it from both.

Trade-offs

Choice	Benefit	Cost
Rich model (many small classes)	Each rule has one home; easy to test	More classes to explain in 45 minutes
Sets of voter ids instead of counters	Correct: no double votes, reversible	Memory grows with votes; real systems store votes in a table and cache the count
Derive stats from raw events	Never inconsistent	Slow for large data; needs a cache
Services for two-sided operations	Symmetry guaranteed in one place	One more class; entities become thinner ("anemic") if you overdo it

When not to use it

Do not bring the whole toolkit to a tiny problem. If the interviewer asks only for "a like button", a set on the post is enough — no services, no observers.

Common mistakes

- Putting search as a static method on `Question` ("`Question.search()`"), so the entity must know every other question.
- Storing `likeCount` as a constructor argument — a post can then be created "with 1,000 likes".
- Forgetting that the privacy rule also applies inside search and the feed.
- Writing `acceptInvitation` on the sender (or the receiver) and updating only one side.

Real-world examples

- **Kafka** (a system that stores streams of events and delivers them to any program that wants to read them) was created at LinkedIn to move activity events (page views, posts, connections) between systems — the industrial version of the Observer idea used here.
- **Facebook's TAO** is a graph store (a database built for a **graph** — items joined by links, like people joined by friendships) built to serve the social graph (users, friendships, likes) at very large scale — the production version of our in-memory friend graph.
- **Stack Overflow** publishes its reputation-to-privilege table on its help pages — exactly the data-driven `Privilege` enum used in 18.1.

In the interview

> **30-second answer:** "Platform problems test whether I can model a domain with many small classes, put each rule in one place, and handle roles, feeds, notifications and search. I'll start with requirements and actors, then the core entities, then the tricky flows — voting, accepting requests, privacy — and I'll say which operations need a service because they touch two objects."

Follow-ups:

- *Why a set instead of a counter for likes?* — A bare number does not record *who* liked, so it can neither block a repeat like nor undo one. The set gives the count for free.
- *Where does the notification code go?* — Not inside the domain methods directly; the domain raises an event and a notification service with pluggable channels delivers it.

Self-check

- **Q:** What is an inverted index? **A:** A map from each word to the items containing that word; used to answer "which items contain these words?" quickly.
- **Q:** Why is "accept a friend request" not a method on `Member`? **A:** It changes two members; putting it on one of them means one object edits another's state, and the two halves can drift. A service updates both together.
- **Q:** Name two ways to deliver a notification and the pattern that makes adding a third easy. **A:** Email and push; Strategy (a `NotificationChannel` interface).

TL;DR

Platform problems = rich domain model + roles + feed + notifications + search + reputation. Share behaviour in a content base class, store *who* not *how many*, derive numbers, put two-sided operations in services, notify via Observer + Strategy, keep search and privacy each in one place.

18.1 Design Stack Overflow



Figure — The Stack Overflow logo that opens this case study (course diagram)

Stack Overflow is a huge question-and-answer community for programmers. Members can **upvote** or **downvote** questions and answers, and anyone with enough standing can edit posts, much like a wiki (a website whose pages any permitted reader can edit). Members earn **reputation** (a points score) and **badges** (named awards) for useful contributions. Higher reputation unlocks **privileges** — voting, commenting anywhere, editing other people's posts, voting to close questions. For scale: an upvote on an answer is worth 10 points to its author. The course says a question upvote is worth 5; that was true until November 2019, when Stack Overflow raised it to 10 as well, and this chapter's code uses today's value.

Problem & clarifying questions

Ask these before drawing anything. The answers are the assumptions this design uses.

Question to ask	Answer we assume
Who uses the system?	Guests (not logged in), members, moderators, admins, and the system itself (notifications, badges)
What can a guest do?	Search and read only. Posting or voting requires membership
What can be voted on?	Questions, answers and comments (comments: upvote only, as on the real site)
How much reputation does a vote give?	Upvote +10 on a question or an answer (question upvotes were +5 before November 2019), downvote −2 to the author, −1 to whoever downvotes an answer (downvoting a question is free), accepted answer +15 (and +2 to the asker)
Does reputation unlock actions?	Yes — a privilege table (vote up at 15, comment everywhere at 50, set bounty at 75, vote down at 125, create tag at 1,500, edit at 2,000, close votes at 3,000, delete votes at 10,000)
How are questions closed?	3 close votes from members with the privilege, or one moderator. Moderators can also reopen, delete and undelete
What is a bounty?	Reputation a member (usually the asker) stakes, 50 to 500 points, to attract answers. It runs for 7 days (the real site adds a 24-hour grace period) and goes to the answer the sponsor picks. The staked points are taken at once and are never refunded; if the sponsor picks nobody, the real site may auto-award half to a qualifying new answer
Do we need search?	Yes: by keyword and by tag, plus "most used tags"
Scale?	Read-heavy (many more views than posts); votes must never double-count

Requirements

Functional (the course's thirteen, in our words)

1. Guests can search and view questions; to post or upvote they must become members.
2. Members can post new questions.
3. Members can answer an **open** question.
4. Members can comment on any question or answer.
5. Questions, answers and comments can all be upvoted by members.
6. Members can **flag** any question, answer or comment that has a serious problem or needs a moderator to look at it.
7. A member can attach a **bounty** to a question they asked.
8. Members earn **badges** for being helpful.
9. Members can cast votes to **close** a question; a moderator can close or reopen any question on their own.
10. Members can add **tags** (a word or phrase naming the topic) to questions.
11. Members can vote to **delete** far off-topic or very low-quality questions.
12. Moderators can close questions and restore (undelete) deleted ones.
13. The system can report the **most frequently used tags**.

Plus from the real site (added so the privilege rules have somewhere to live): downvotes, accepting an answer, reputation-gated privileges, blocking accounts.

Non-functional

- **Correctness of counts:** one member, one vote per post; reputation changes must be reversible when a vote changes.
- **Read-heavy performance:** search and "top tags" must not scan every question.
- **Concurrency** (several pieces of work running at the same time — for example two requests handled by two **threads**, the independent paths of execution inside one program)**:** two members voting on the same post at the same instant must both be counted exactly once.

- **Abuse resistance:** you cannot vote on your own post; blocked accounts can do nothing.
- **Extensibility** (how easily new features can be added without rewriting old code)**:** reputation rules, badge rules and notification channels change often — changing them should not touch voting code.

Use cases

The five actors:

- **Guest** — searches and views questions.
- **Member** — everything a guest can do, plus ask, answer, comment, vote, flag, add bounties and tags, and delete or undelete their own posts.
- **Moderator** — everything a member can do, plus close, reopen, delete and undelete *any* question.
- **Admin** — blocks and unblocks members.
- **System** — sends notifications and assigns badges.

Top use cases: search questions; ask a question (with tags and an optional bounty); write or edit answers; add comments; moderators close, delete and undelete questions.

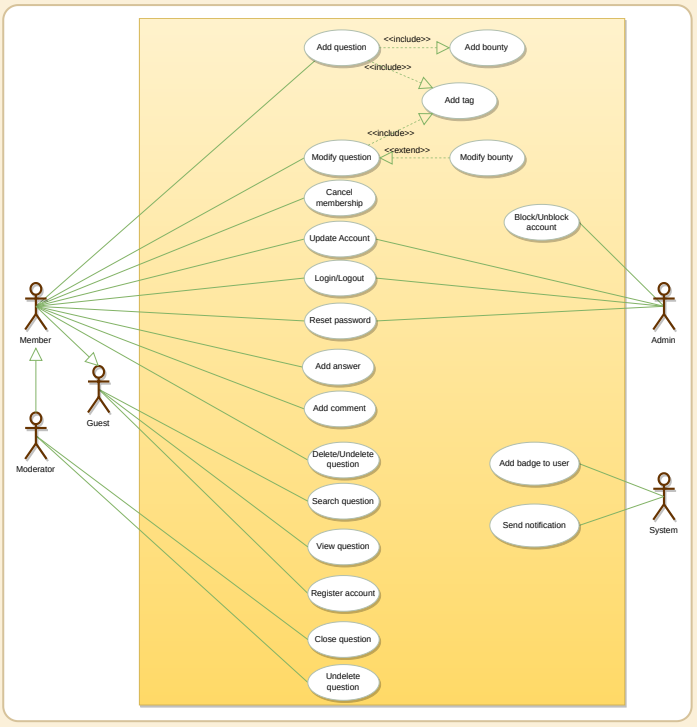


Figure — Use cases per actor: Guest (search, view, register), Member (add/modify question, answer, comment, bounty, tag), Moderator (close, delete/undelete), Admin (block/unblock), System (send notification, add badge) (course diagram)

A **use-case diagram** is a UML picture of the actors (stick figures) and the things each one can do (ovals). In the figure above, notice the `<<include>>` from "Add question" to "Add tag" (adding a tag is part of asking) and the `<<extend>>` for bounties (an optional extra on a question).

An **activity diagram** is a flowchart of the steps and yes/no decisions in one task; a **sequence diagram** shows, top to bottom in time order, the messages objects send to each other.

Activity — post a new question. The member clicks *Ask Question*, types a title and body, and adds tags. For each tag, the system checks whether it already exists; if not, it checks whether the member has **enough reputation to create a tag**. Then it validates the question and either saves and posts it or shows an error.

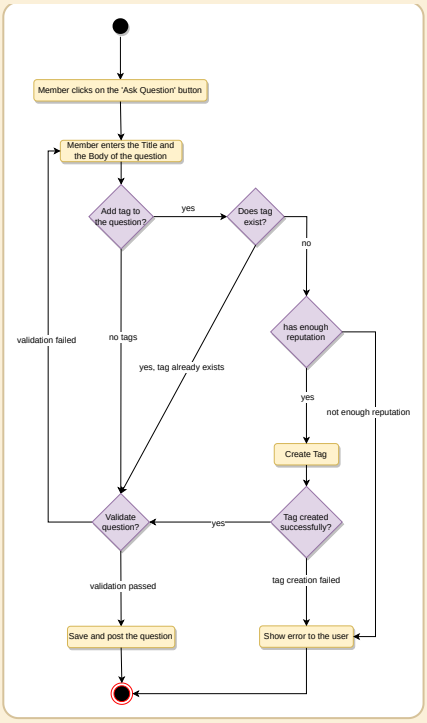


Figure — Activity diagram for posting a question, including the "does the tag exist?" and "has enough reputation?" decisions (course diagram)

Sequence — create a question. The member sets the title and body, looks up tags (creating one if missing), creates the question with a bounty value, the question creates its `Bounty`, and finally the question is published.

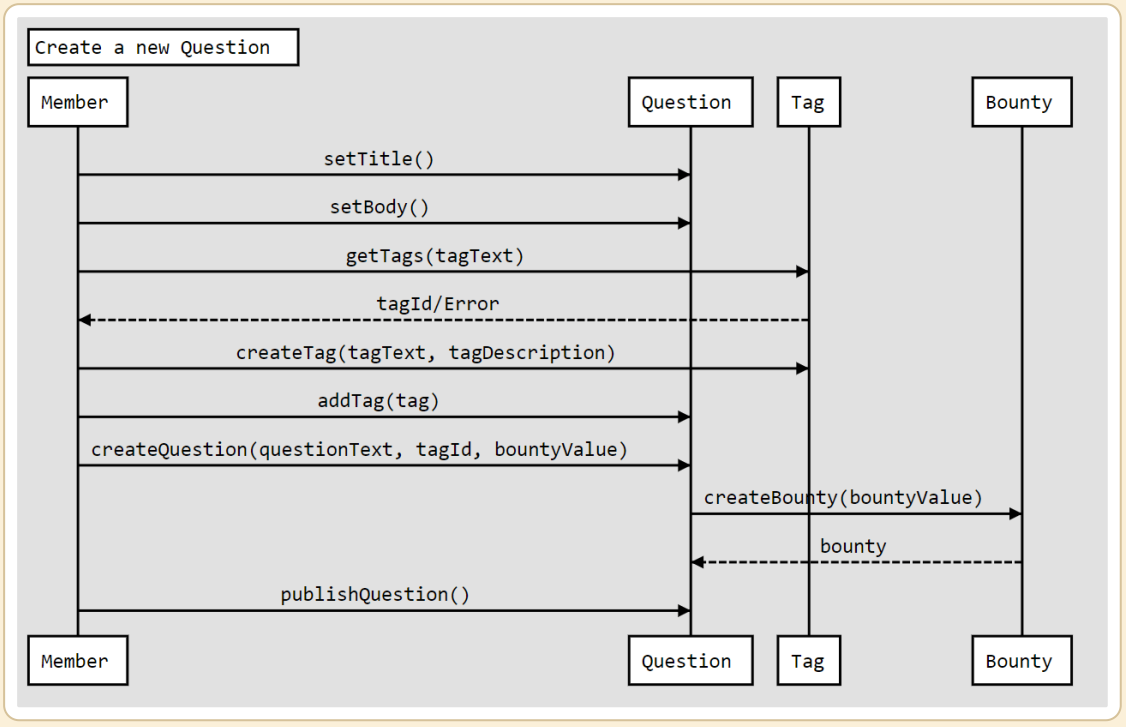


Figure — Sequence diagram: Member → Question (setTitle, setBody), Member → Tag (getTags / createTag), Question → Bounty (createBounty), then publishQuestion (course diagram)

Our Java code follows the same order: `ask()` resolves each tag, creating it only if the asker holds the `CREATE_TAG` privilege, and then builds the question; `offerBounty()` attaches the bounty.

Core entities

Class	Responsibility
`Member` (+ `Moderator`, `Admin`)	Identity, reputation (atomic — safe when updated by many threads at once), badges, account status; `hasPrivilege(p)`
`Privilege` (enum)	The reputation needed for each action — a data table, not scattered `if`s
`Post` (sealed, abstract)	Shared behaviour: author, body, votes per voter, flags, comments
`Question`	Title, tags, answers, status (OPEN / CLOSED / DELETED…), closing remark, close/delete voters, bounty, view count
`Answer`	Belongs to one question; can be *accepted*. (The course also gives answers and comments a view count and vote/flag counts; our code derives the vote and flag counts from the voter and flagger sets)
`Comment`	Belongs to a question or answer
`Tag`	Name + description; counts its own uses (for "top tags")
`Bounty`	Staked reputation, sponsor, expiry date, winner
`Badge` (record)	Name + description
`ReputationPolicy` / `DefaultReputationPolicy`	Strategy: how many points each event is worth
`ReputationListener` / `BadgeAwarder`	Observer: reacts to reputation changes (awards badges)
`Notifier` + `NotificationChannel` / `EmailChannel`	Sends notifications through pluggable channels
`Search` / `QuestionIndex`	Inverted index by word and by tag
`StackOverflow`	Facade: checks status + privilege, delegates, applies reputation

Class diagram

The course builds the class diagram in **five stages**, each adding a few classes for a few requirements:

- **Stage 1 (requirements 8, 10, 13): `Badge`, `Tag`, `Notification`.** `Tag` counts its own uses (daily and weekly counters, `record_use`) when attached to a question, so "most used tags" is a quick read, not a scan. `Notification` is abstract with one subclass per delivery method (`EmailNotification`); an email notification genuinely is a notification, so the inheritance passes the substitution test (the Liskov Substitution Principle: a subclass object must work anywhere its parent is expected).
- **Stage 2 (requirement 7): `Photo` (attachments on questions and answers) and `Bounty`.** The bounty holds the staked amount itself, so the value has one owner until settled; the course's reason is that it can go back to the sponsor if nobody earns it. Careful: on the real site a bounty is *not* refundable, so our code deducts the points when the bounty starts and never returns them.
- **Stage 3 — the heart: abstract `Post` over `Question`, `Answer`, `Comment`.** Upvoting and flagging (requirements 5, 6) work the *same way* on all three, so that behaviour is written once in `Post`; what each contains differs, so they stay three classes rather than one plus a "kind" field. `Post` remembers **who** voted, or one member could vote ten times. Filled diamonds are **composition**: deleting a question deletes its answers, comments, photos and bounty. `Tag` is a plain **association**: many questions share it and it outlives each. `Post` keeps only the author's id, so there is no arrow from a post to a member.
- **Stage 4 (search, requirement 13): the `Search` interface and `QuestionIndex`.** A static (class-level) `Question.search` would force `Question` to know every question. The index *realizes* `Search` and only associates with questions and tags: deleting it deletes no question.
- **Stage 5 (requirement 1): `Account`, `Guest`, `Member`, `Moderator`, `Admin`.** Guest methods sit on `Guest`; `Member` extends it and adds posting and voting, so a guest simply has no `createQuestion` and no check

can be forgotten. `Moderator` and `Admin` extend `Member` (substitution holds). `Member` owns its badges (composition); its link to `Account` is a plain association because the account is created elsewhere.

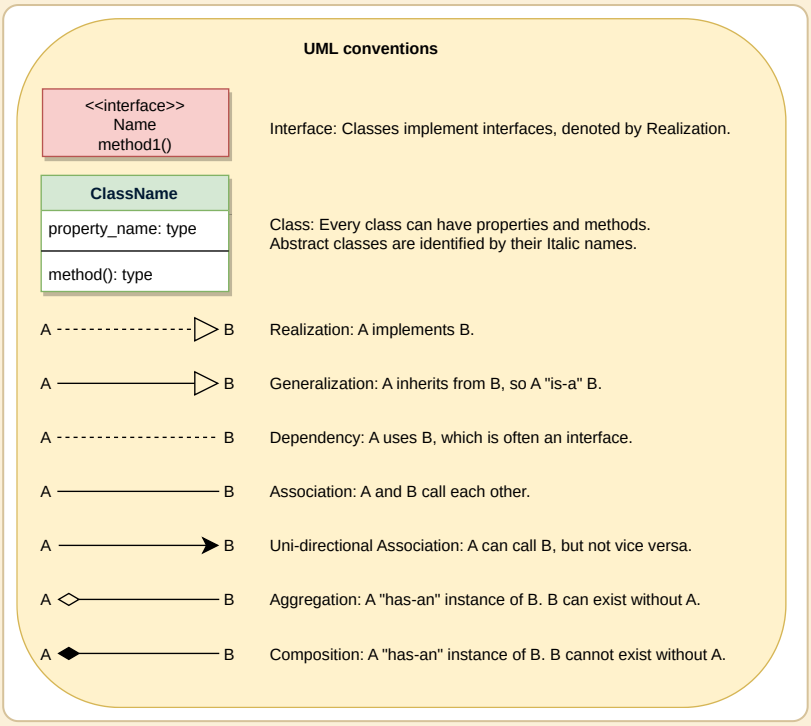


Figure — The UML legend used in these diagrams: class, interface, realization, generalization, dependency, association, aggregation and composition arrows (course diagram)

UML (Unified Modeling Language) is the standard notation for classes and their links: **generalization** is the inheritance arrow ("is a kind of"); **realization** means a class implements an interface; **dependency** means one class merely uses another for a moment; **aggregation** (hollow diamond) means "has, but does not own"; **composition** (filled diamond) means the part dies with the whole; **association** (plain arrow) means one class refers to another.

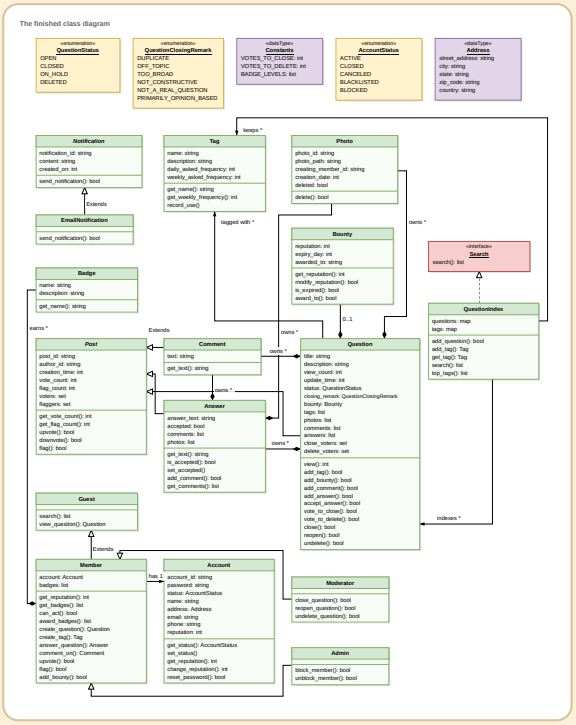
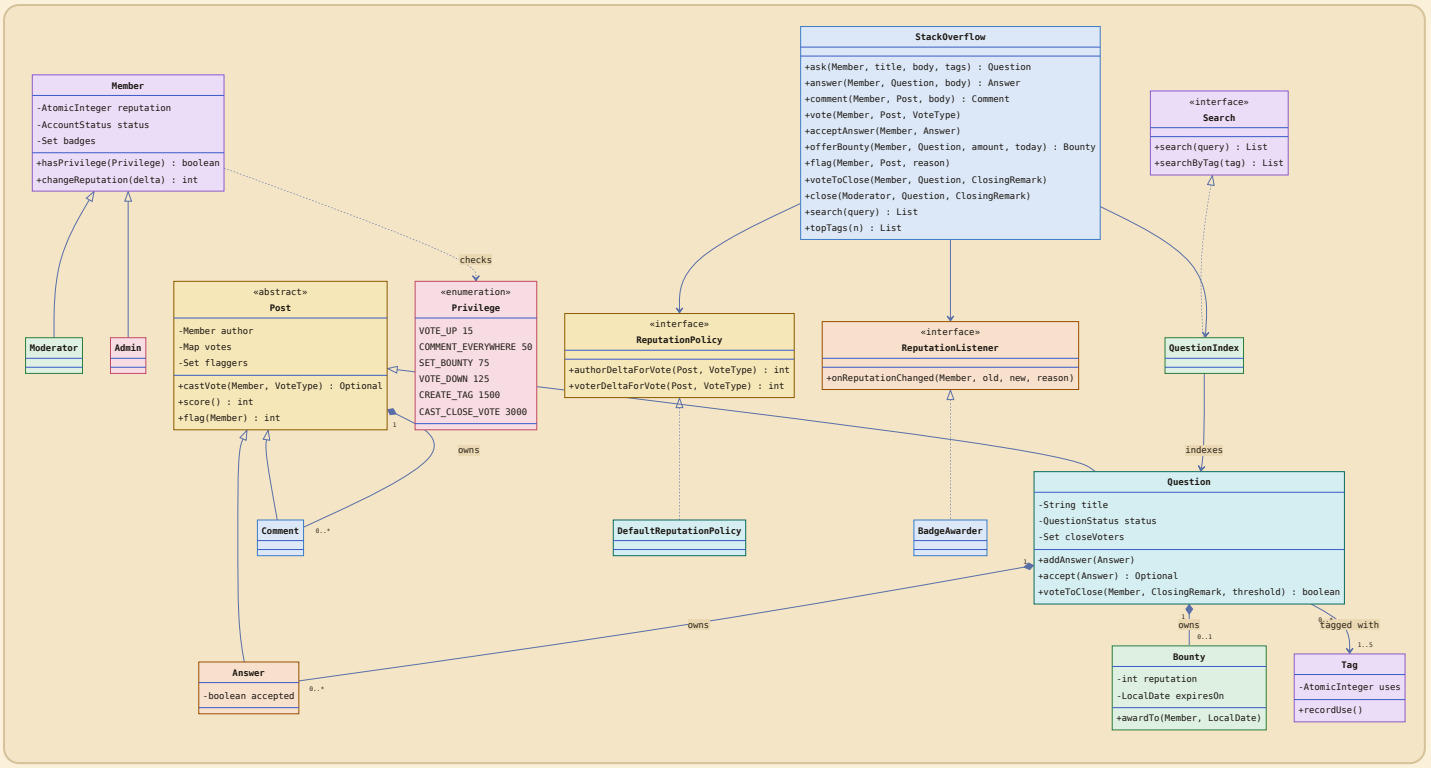


Figure — The finished Stack Overflow class diagram: enums (QuestionStatus, QuestionClosingRemark, AccountStatus), Constants, the posts, tags, badges, bounty, photo, notification, search index and the account hierarchy (course diagram)

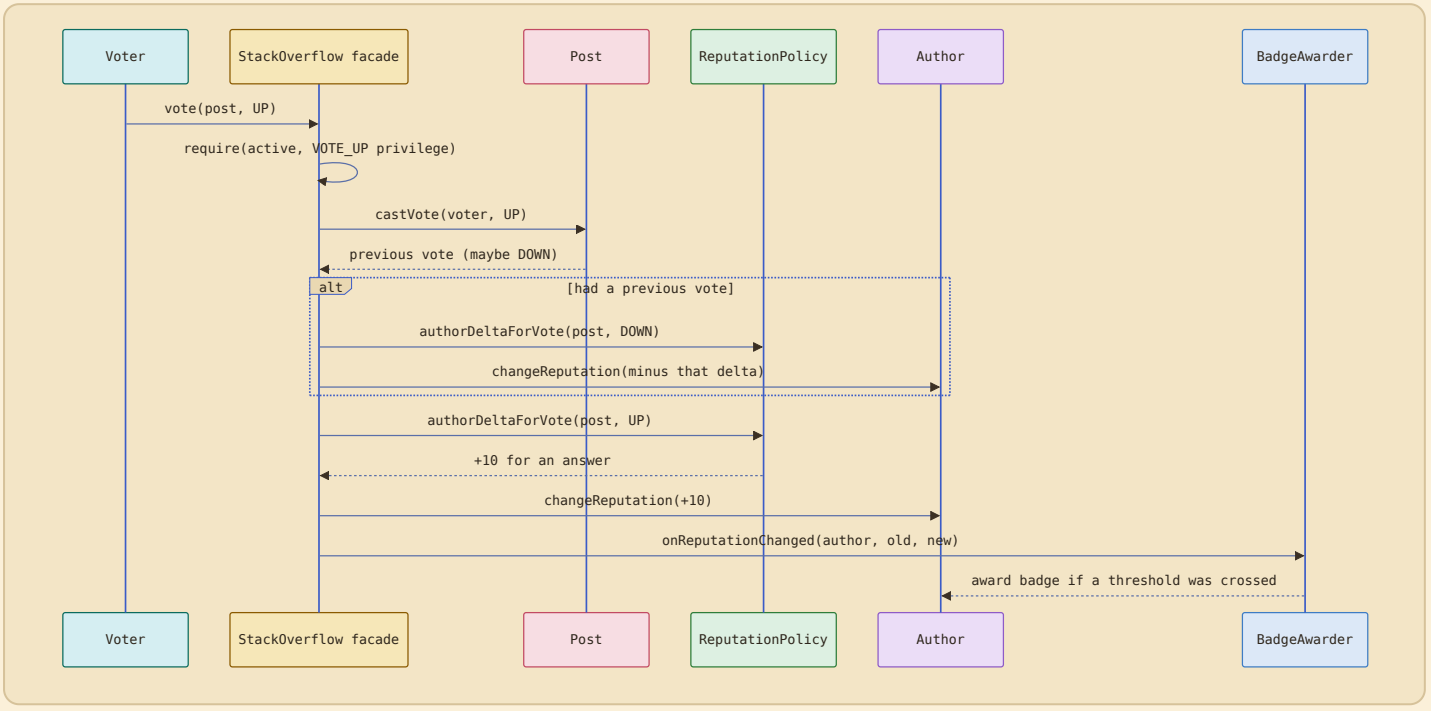
Our Java design. It keeps the course's structure and adds three things the course's "what we left out" section invites: a reputation **Strategy**, a badge **Observer**, and a data-driven **privilege table** (because on

the real site privileges depend on reputation, not only on account type). A guest in our code is simply "no `Member` object": every write operation on the facade requires a `Member` argument, so the type system still enforces requirement 1. The course also lists what it deliberately leaves out: edit history, revision review, linking duplicates, chat, and the reputation a downvoter pays. We add the downvote cost (it is one line in the policy); edit history and duplicate links are covered under Extensions below; chat stays out of scope.

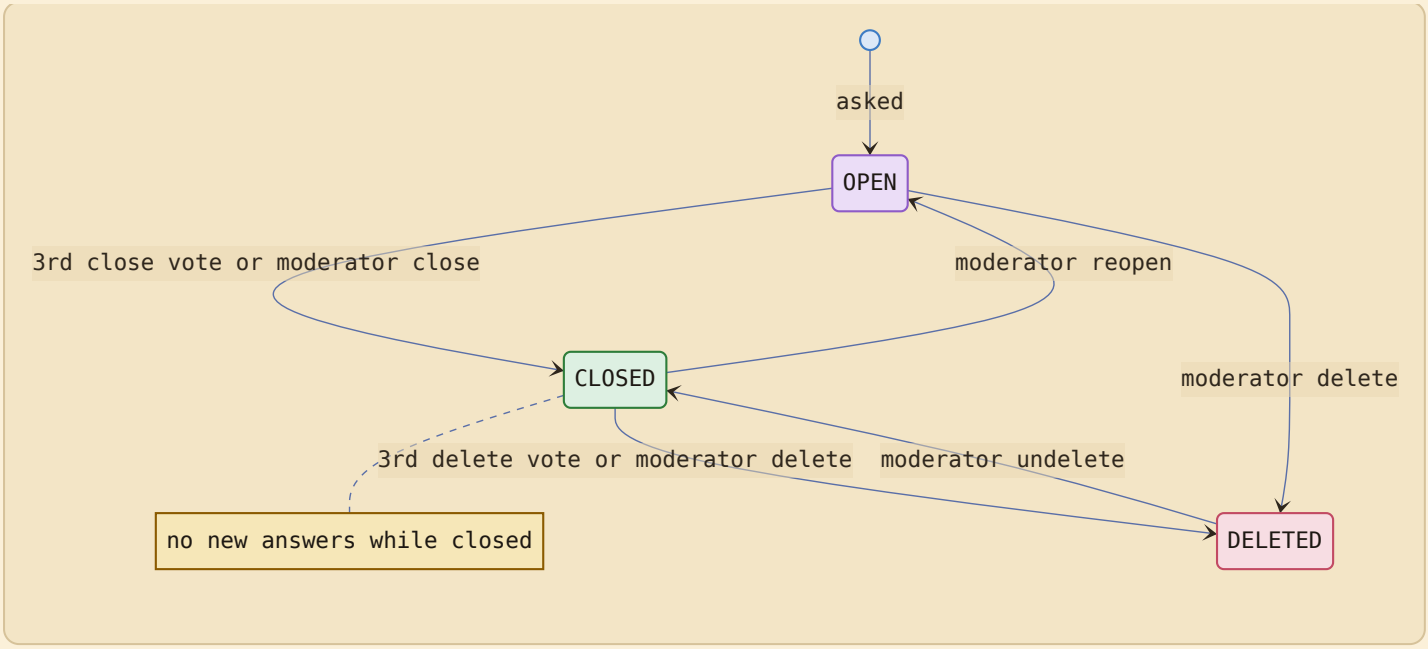


Key flows

Voting (with a changed vote). The facade checks privilege first, then asks the post to record the vote. The post returns the voter's *previous* vote, so the facade can reverse its reputation effect before applying the new one.



Question lifecycle.



Bounty. `offerBounty` checks the `SET_BOUNTY` privilege and the 50–500 range, attaches the bounty to the (open) question, and *immediately* deducts the amount from the sponsor. Later `awardBounty` (sponsor only, not to their own answer, before expiry) credits the winner.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
`Post` base class for question, answer, comment	Inheritance (passes the substitution test), `sealed`	Voting, flagging and commenting are identical on all three	One `Post` class with a `type` field — then every method needs `if (type == …)`
Votes stored as `Map<voterId, VoteType>`	"Store who, not how many"	Blocks double votes; lets a vote be changed and its reputation reversed	An `int voteCount` — cannot tell who voted
`ReputationPolicy` interface	**Strategy**	Scoring rules change (Stack Overflow really did change them); voting code should not	Hard-coded numbers inside `vote()` — the course calls this out as a weakness
`ReputationListener` + `BadgeAwarder`	**Observer**	Badges, notifications, analytics all react to reputation without the facade knowing them	Calling `awardBadges()` directly inside `vote()`
`Privilege` enum with thresholds	Table-driven design	One place holds the rules; `hasPrivilege` is one comparison	`if (rep >= 125)` scattered across methods
Moderator-only methods take a `Moderator` parameter	Type-level permission	You cannot even call `close(…)` without a moderator object	A runtime role check that someone might forget
`QuestionIndex` separate from `Question`	Single Responsibility	Search can later be swapped for Elasticsearch (a popular search-engine server) behind `Search`	Static `Question.search()`
`Tag` counts its own uses	Maintain an aggregate at write time	"Top tags" becomes a sort of a small collection	Scanning all questions on every request
`StackOverflow` facade	**Facade**	One entry point that validates, delegates and logs	Letting the UI call `Question`, `Member`, `Bounty` directly
Pattern matching `switch` over the sealed `Post`	Exhaustive dispatch	The compiler forces us to handle each post kind in the policy	`instanceof` chains that silently miss a new type

Java implementation

Package `lld.ch18.stackoverflow`. Full runnable code is in `code/lld/ch18/stackoverflow/`; read it in this order: `Types.java` (enums and the privilege table), `Accounts.java`, `Posts.java`, `Reputation.java` (Strategy/Observer, notifications), `QuestionIndex.java`, `StackOverflow.java` (the facade) and `Main.java`. The essential lines (`/* ... */` and `//` summary comments stand for code left out):

```
enum QuestionStatus { OPEN, ON_HOLD, CLOSED, DELETED }      // ON_HOLD: the older name for a freshly closed question
enum ClosingRemark { DUPLICATE, OFF_TOPIC, TOO_BROAD, NOT_CONSTRUCTIVE, NOT_A_REAL_QUESTION, PRIMARILY_OPINION_BASED }
enum AccountStatus { ACTIVE, CLOSED, CANCELED, BLACKLISTED, BLOCKED }

/** Reputation needed to unlock an action (stackoverflow.com's public privileges page): data, not scattered ifs. */
enum Privilege {
    ASK_OR_ANSWER(1), VOTE_UP(15), FLAG(15), COMMENT_EVERYWHERE(50), SET_BOUNTY(75), VOTE_DOWN(125),
    CREATE_TAG(1500), EDIT_ANY_POST(2000), CAST_CLOSE_VOTE(3000), CAST_DELETE_VOTE(10000);
    final int minReputation;
    Privilege(int minReputation) { this.minReputation = minReputation; }
}

/** A guest has no Member object: every write on the facade takes a Member, so the signatures enforce requirement 1. */
class Member {
    private final AtomicInteger reputation = new AtomicInteger(1);   // every account starts at 1
    private volatile AccountStatus status = AccountStatus.ACTIVE;
    /** Applies a delta atomically; reputation never drops below 1 (the site's floor). */
    int changeReputation(int delta) { return reputation.updateAndGet(r -> Math.max(1, r + delta)); }
    boolean hasPrivilege(Privilege p) { return reputation() >= p.minReputation; }
}
class Moderator extends Member { @Override boolean hasPrivilege(Privilege p) { return true; } }   // Admin: same
```

```java
/** Votes are stored per voter (not a bare counter): no double votes, and a vote can be changed or reversed. */
abstract sealed class Post permits Question, Answer, Comment {
    private final Map<Long, VoteType> votes = new HashMap<>();
    private final Set<Long> flaggers = new HashSet<>();

    /** Records or changes a vote and returns the voter's previous vote, if any. */
    synchronized Optional<VoteType> castVote(Member voter, VoteType type) {
        if (voter.id() == author.id()) throw new IllegalStateException("you cannot vote on your own post");
        VoteType previous = votes.put(voter.id(), type);
        if (previous == type) throw new IllegalStateException(voter.name() + " already voted " + type + " on this
post");
        return Optional.ofNullable(previous);
    }
    synchronized int score() { return votes.values().stream().mapToInt(v -> v == VoteType.UP ? 1 : -1).sum(); }
    /** Returns the number of distinct flaggers after this flag. */
    synchronized int flag(Member by) { /* flaggers.add or "already flagged" */ }
}

final class Question extends Post {
    Question(Member author, String title, String body, Set<Tag> tags) {
        super(author, body);
        tags.forEach(Tag::recordUse);            // a tag counts its own uses: "top tags" becomes a read
    }
    synchronized void addAnswer(Answer a) {
        if (status != QuestionStatus.OPEN) throw new IllegalStateException("question is " + status + ", no new
answers");
        answers.add(a);
    }
    /** Marks one answer accepted and returns the previously accepted one (only one answer can win). */
    synchronized Optional<Answer> accept(Answer chosen) { /* ... */ }
    /** One close vote per member; returns true when this vote closed the question. */
    synchronized boolean voteToClose(Member voter, ClosingRemark remark, int threshold) {
        if (status != QuestionStatus.OPEN) throw new IllegalStateException("question is already " + status);
        if (!closeVoters.add(voter.id())) throw new IllegalStateException(voter.name() + " already voted to
close");
        if (closeVoters.size() >= threshold) { close(remark); return true; }
        return false;
    }
    // voteToDelete: only CLOSED questions; close/reopen/delete/undelete (undelete -> CLOSED); attachBounty needs
OPEN.
}
// Answer (belongs to one question; accepted flag), Comment (parent post), Tag (shared by many questions and
// outlives each: association, not composition), Bounty (sponsor, staked reputation, expiresOn; awardTo refuses
// after expiry or when already settled).
```

```java
/** Strategy: the scoring rules live behind an interface, so voting code does not know the numbers. */
interface ReputationPolicy {
    int authorDeltaForVote(Post post, VoteType type);
    int voterDeltaForVote(Post post, VoteType type);
    int acceptedAnswerBonus();
    int acceptorBonus();
}
/** Today's rules: +10 for any upvote, -2 to the author per downvote, -1 to whoever downvotes an answer. */
final class DefaultReputationPolicy implements ReputationPolicy {
    @Override public int authorDeltaForVote(Post post, VoteType type) {
        return switch (post) {
            case Question q -> type == VoteType.UP ? 10 : -2;   // +5 before November 2019
            case Answer a -> type == VoteType.UP ? 10 : -2;
            case Comment c -> 0;                                // comment votes earn no reputation
        };
    }
    @Override public int voterDeltaForVote(Post post, VoteType type) { return post instanceof Answer && type ==
VoteType.DOWN ? -1 : 0; }
    @Override public int acceptedAnswerBonus() { return 15; }
    @Override public int acceptorBonus() { return 2; }
}
/** Observer: anything that wants to react to a reputation change. */
interface ReputationListener { void onReputationChanged(Member member, int oldValue, int newValue, String reason);
}
// BadgeAwarder awards Contributor / Helpful / Trusted when reputation crosses 15 / 50 / 125 (course thresholds).
// Notifier (the "System" actor) sends through every NotificationChannel (Strategy: EmailChannel today, push
tomorrow).

/** An inverted index: word -> questions containing it. It refers to questions but does not own them. */
final class QuestionIndex implements Search {
    private final Map<String, Set<Question>> byWord = new ConcurrentHashMap<>();
    private final Map<String, Set<Question>> byTag = new ConcurrentHashMap<>();
    /** Questions containing every word of the query, best score first; deleted questions are hidden. */
    @Override public List<Question> search(String query) { /* intersect byWord sets, then rank */ }
}
```

```java
/** Facade: checks account status + privilege, delegates state changes to the posts, applies reputation, tells listeners. */
public final class StackOverflow {
    static final int VOTES_TO_CLOSE = 3, VOTES_TO_DELETE = 3, FLAGS_FOR_MODERATOR_REVIEW = 3;
    static final int MIN_BOUNTY = 50, MAX_BOUNTY = 500, BOUNTY_DAYS = 7;

    Question ask(Member asker, String title, String body, List<String> tagNames) {
        require(asker, Privilege.ASK_OR_ANSWER);
        Set<Tag> resolved = /* each name: the existing tag, or createTag(asker, ...), which throws without CREATE_TAG */;
        Question q = new Question(asker, title, body, resolved);
        index.add(q);
        return q;
    }

    void vote(Member voter, Post post, VoteType type) {
        require(voter, type == VoteType.UP ? Privilege.VOTE_UP : Privilege.VOTE_DOWN);
        if (post instanceof Comment && type == VoteType.DOWN) throw new IllegalStateException("comments can only be upvoted");
        Optional<VoteType> previous = post.castVote(voter, type);
        previous.ifPresent(p -> {                                   // undo the old vote's effect first
            adjust(post.author(), -policy.authorDeltaForVote(post, p), "vote changed on " + post.label());
            adjust(voter, -policy.voterDeltaForVote(post, p), "vote changed");
        });
        adjust(post.author(), policy.authorDeltaForVote(post, type), type + " on " + post.label());
        adjust(voter, policy.voterDeltaForVote(post, type), "downvoted an answer");
    }

    Bounty offerBounty(Member sponsor, Question q, int amount, LocalDate today) {
        require(sponsor, Privilege.SET_BOUNTY);
        if (amount < MIN_BOUNTY || amount > MAX_BOUNTY) throw new IllegalArgumentException("bounty must be " + MIN_BOUNTY + ".." + MAX_BOUNTY);
        if (sponsor.reputation() - amount < 1) throw new IllegalStateException("not enough reputation to stake " + amount);
        Bounty b = new Bounty(sponsor, amount, today.plusDays(BOUNTY_DAYS));
        q.attachBounty(b);
        adjust(sponsor, -amount, "bounty staked");          // staked reputation leaves the sponsor now
        return b;
    }
    // awardBounty: sponsor only, not to their own answer, before expiry. comment: own thread free, else COMMENT_EVERYWHERE.
    // acceptAnswer: asker only; moves +15 from a previously accepted answer; +2 to the asker the first time.
    // flag: the 3rd flag notifies every moderator. topTags(n): sort tags by their own use counts.

    // Moderator-only operations: the parameter type is the permission check.
    void close(Moderator m, Question q, ClosingRemark remark) { q.close(remark); }
    void block(Admin admin, Member target) { target.setStatus(AccountStatus.BLOCKED); }

    private void adjust(Member m, int delta, String reason) {
        if (delta == 0) return;
        int before = m.reputation();
        int after = m.changeReputation(delta);
        listeners.forEach(l -> l.onReputationChanged(m, before, after, reason));
    }
    private static void require(Member m, Privilege p) {
        requireActive(m);                                   // blocked accounts can do nothing
        if (!m.hasPrivilege(p))
            throw new IllegalStateException(m.name() + " needs " + p.minReputation + " reputation for " + p + " (has " + m.reputation() + ")");
    }
}
```

`Main.java` registers alice (seeded to 200 reputation), bob (3,500), carol (3,200), erin (3,100), dave (1), moderator mia and admin root, then walks through tags and asking, answers and votes, a bounty, badges for a new member, moderation, and guest search.

Output:

```
== 1. Tags and asking ==
  REJECTED: alice needs 1500 reputation for CREATE_TAG (has 200)
alice asked question #101 "How does HashMap handle collisions?" [[java], [collections]]

== 2. Answers, votes, reputation ==
bob posted answer #102 by bob on question #101
    [email to alice@example.com] bob answered "How does HashMap handle collisions?"
carol posted answer #103 by carol on question #101
    [email to alice@example.com] carol answered "How does HashMap handle collisions?"
  REJECTED: dave needs 15 reputation for VOTE_UP (has 1)
  REJECTED: you cannot vote on your own post
bob voted UP on question #101 "How does HashMap handle collisions?" -> score 1
    rep alice  +10 -> 210 (UP on question #101 "How does HashMap handle collisions?")
carol voted UP on answer #102 by bob -> score 1
    rep bob    +10 -> 3510 (UP on answer #102 by bob)
erin voted DOWN on answer #103 by carol -> score -1
    rep carol  -2 -> 3198 (DOWN on answer #103 by carol)
    rep erin   -1 -> 3099 (downvoted an answer)
  REJECTED: erin already voted DOWN on this post
erin voted UP on answer #103 by carol -> score 1
    rep carol  +2 -> 3200 (vote changed on answer #103 by carol)
    rep erin   +1 -> 3100 (vote changed)
    rep carol  +10 -> 3210 (UP on answer #103 by carol)
erin commented on answer #103 by carol: "HashMap uses chaining, not open addressing."
    [email to carol@example.com] erin commented on your answer #103 by carol
  REJECTED: dave needs 50 reputation for COMMENT_EVERYWHERE (has 1)
alice accepted answer #102 by bob
    rep bob    +15 -> 3525 (answer accepted)
    rep alice  +2 -> 212 (accepted an answer)

== 3. Bounty ==
alice asked question #105 "Why is ConcurrentHashMap faster than synchronizedMap?" [[java]]
    rep alice  -50 -> 162 (bounty staked)
alice offered a +50 bounty on question #105 (expires 2026-10-05)
carol posted answer #106 by carol on question #105
    [email to alice@example.com] carol answered "Why is ConcurrentHashMap faster than synchronizedMap?"
    rep carol  +50 -> 3260 (bounty awarded)
alice awarded the bounty to answer #106 by carol

== 4. Badges for a new member ==
dave asked question #107 "Is Java pass-by-value?" [[java]]
bob voted UP on question #107 "Is Java pass-by-value?" -> score 1
    rep dave   +10 -> 11 (UP on question #107 "Is Java pass-by-value?")
carol voted UP on question #107 "Is Java pass-by-value?" -> score 2
    rep dave   +10 -> 21 (UP on question #107 "Is Java pass-by-value?")
    [email to dave@example.com] You earned the 'Contributor' badge
erin voted UP on question #107 "Is Java pass-by-value?" -> score 3
    rep dave   +10 -> 31 (UP on question #107 "Is Java pass-by-value?")
dave's badges: [Contributor]

== 5. Moderation ==
bob voted to close question #107 as DUPLICATE
carol voted to close question #107 as DUPLICATE
erin voted to close question #107 as DUPLICATE -> CLOSED
  REJECTED: question is CLOSED, no new answers
mia (mod) reopened question #107
dave commented on question #107 "Is Java pass-by-value?": "you all are wrong!!!"
bob flagged comment #109 (rude), flags=1
carol flagged comment #109 (rude), flags=2
erin flagged comment #109 (rude), flags=3
    [email to mia@example.com] comment #109 needs moderator attention
mia (mod) deleted question #107
mia (mod) undeleted question #107 -> CLOSED
root blocked dave
  REJECTED: dave is BLOCKED

== 6. Guest search and top tags ==
found: question #101 "How does HashMap handle collisions?" score=1 answers=2 status=OPEN
[java] question #107 "Is Java pass-by-value?"
```

```
[java] question #101 "How does HashMap handle collisions?"
[java] question #105 "Why is ConcurrentHashMap faster than synchronizedMap?"
top tags: [javax3, collectionsx1]
final reputation: [alice(162), bob(3525), carol(3260), erin(3100), dave(31)]
```

Walking through the important parts

- `Post.castVote` is `synchronized` (Java lets only one thread at a time run a synchronized method on the same object; others wait their turn) and uses `votes.put(...)`, which returns the previous vote. That single call both records the vote and tells us whether this is a *change* (so the facade can undo the old reputation effect). Voting the same way twice throws.
- `DefaultReputationPolicy` uses a Java 21 **pattern-matching switch** (a `switch` that picks a branch by the *type* of the value and hands you that typed value) over the sealed `Post`. Because `Post` permits exactly `Question`, `Answer` and `Comment`, the compiler rejects the switch if a case is missing.
- `Member.changeReputation` uses `AtomicInteger.updateAndGet` (an `AtomicInteger` is a number whose read-change-write happens as one indivisible step) so concurrent changes never lose an update, and it enforces the site's floor of 1.
- `BadgeAwarder` fires only when a threshold is *crossed* (`oldValue < level && newValue >= level`), so members who already had high reputation are not showered with badges, and each badge is added once (`Set.add` returns `false` for duplicates).
- `ask()` shows the activity diagram's decision "does the tag exist? has enough reputation?": in the output, Alice (200 rep) cannot create the new tag `hashmap`, so her first attempt is rejected.
- In section 2 of the output, Erin downvotes Carol's answer (Carol −2, Erin −1), then changes her vote to up: the old effect is reversed (+2, +1) and the new one applied (+10).
- `voteToClose` closes the question on the third vote; answering a closed question is then refused; a moderator reopens it.

Concurrency & edge cases

- **Two votes at the same time on one post.** `castVote`, `score` and `flag` are `synchronized` on the post, so the vote map is never corrupted and each voter is counted once. Reputation uses `AtomicInteger`, so two simultaneous reputation changes to the same author both apply.
- **Reputation updates are not in the same lock as the vote.** A reader could briefly see the new vote before the author's reputation changes. That is acceptable (**eventual consistency** for a score: the copies may disagree for a moment but will match shortly — see System Design Fundamentals). In a database you would put both in one **transaction** (a group of changes that either all succeed or all fail together), and enforce "one vote per member per post" with a **unique constraint** (a database rule that refuses a second row with the same values) on `(post_id, voter_id)`.
- **Reputation floor.** Because reputation cannot go below 1, reversing a vote on a member who was at the floor is not perfectly symmetric. The real site has the same quirk; if exact reversal matters, keep a ledger (an append-only list of every change, in order) of reputation events and recompute.
- **Accepting a different answer.** `Question.accept` returns the previously accepted answer so the facade removes its +15; the asker's +2 is given only once.
- **Self-actions.** Voting on your own post and awarding your own bounty are rejected.
- **Closed and deleted questions.** Closed: no new answers. Deleted: hidden from search results.
- **Blocked accounts.** Every write goes through `requireActive`, so a blocked member can do nothing.
- **Bounty expiry.** `awardTo` refuses after the expiry date. On the real site, bounty reputation is never refunded; if the sponsor does not award it, the system may auto-award half to a qualifying answer.

Extensions & follow-up questions

- **"How do you stop one person voting twice?"** — The per-post map of voter ids (the course's key follow-up). In a database: a unique key on `(post_id, voter_id)`.
- **"Add edit history."** — Store each version as an immutable `Revision` (author, time, body) in a list on the post; editing appends a revision. This is close to the **Memento** idea: the past states are kept so they can be shown or rolled back.
- **"Daily reputation cap."** — The real site caps reputation from votes at 200 per day. Wrap the policy: a `CappedReputationPolicy` decorator (the **Decorator** pattern: an object that wraps another object with

the same interface and adds behaviour before or after passing the call on) that tracks today's total per member and trims the delta. No voting code changes.

- **"Mark as duplicate and link."** — Add `duplicateOf: Question` to `Question` and a `ClosingRemark.DUPLICATE`; closing as duplicate sets the link.
- **"Review queues."** — Flags already reach moderators at a threshold; turn that into a `ReviewQueue` that holds flagged posts, with reviewers voting to keep or delete.
- **"Hot questions."** — A ranking Strategy over recent votes, answers and views with time decay (older activity counts for less and less), just like the feed rankers in 18.3 and 18.4.
- **"Tag frequency per day and week."** — The course's `Tag` keeps daily and weekly counters. Keep a counter per time bucket and roll the buckets over with a scheduled job (a task the system runs automatically at set times, e.g. every midnight).

Real-world example

On stackoverflow.com, the numbers in this design are real product rules. The help centre lists privileges by reputation (vote up at 15, comment everywhere at 50, set bounties at 75, vote down at 125, create tags at 1,500, edit at 2,000, cast close and reopen votes at 3,000, moderator tools at 10,000). In November 2019 the company raised question upvotes from +5 to +10 ("rewarding the question askers") — a change that, in our design, touches only the `ReputationPolicy` (one number, or a new policy class) and nothing in the voting code. After a 2019 experiment the number of close votes needed was lowered from 5 to 3, which in our code is one constant.

In the interview

How to spend 30–45 minutes

Minutes	What to do
0–5	Clarify actors and the reputation rules; write the requirement list
5–10	Use cases per actor; say "guest has no posting method"
10–20	Class diagram: `Post` base with three subclasses, `Member` hierarchy, `Tag`, `Bounty`, `QuestionIndex`, facade
20–30	Walk through `vote()` and `acceptAnswer()` including reversal; question state machine
30–40	Code the core: `Post.castVote`, the policy, the facade's `vote`
40–45	Concurrency (per-post lock, atomic reputation, DB unique key) and extensions

30-second answer: "Questions, answers and comments share voting and flagging, so they extend one `Post` that stores votes per voter — that stops double voting and lets me reverse a changed vote. Reputation rules are a Strategy so they can change without touching voting; badges are an Observer on reputation changes. Privileges come from a reputation table; moderator actions take a `Moderator` parameter so a guest or member cannot call them. Search is a separate inverted index, and tags count their own uses for 'top tags'."

Follow-ups with answers

- *Why not store `voteCount` only?* — It cannot prevent duplicates or support vote changes; derive the count from the voter map (or cache it next to a votes table).
- *Where does reputation get awarded — in `upvote`?* — Not inside the post. The facade asks the policy; that keeps posts ignorant of scoring (the course names this as the improvement to make).
- *How would you scale search?* — Keep the `Search` interface and replace `QuestionIndex` with a search engine fed by events when questions are created or edited.

Self-check

- **Q:** Which requirement is enforced by the *shape* of the classes rather than by an `if`? **A:** Guests cannot post or vote: guests have no `Member` object, and every write method needs one (course version: `Guest` has no `createQuestion`).

- **Q:** Why is `Post` abstract and sealed? **A:** Abstract because a bare "post" is never created; sealed so switches over post kinds are checked for completeness.
- **Q:** What happens to reputation when Erin changes a downvote into an upvote on an answer? **A:** The old effect is reversed (author +2, Erin +1) and the new one applied (author +10).
- **Q:** Why does `Tag` count its own uses? **A:** So "most used tags" is a sort of the tags, not a scan of all questions.
- **Q:** What makes a comment different from an answer in voting? **A:** Comments can only be upvoted and earn no reputation (policy returns 0).

> **TL;DR**

Stack Overflow = `Post` base (votes per voter, flags, comments) + `Question`/`Answer`/`Comment` + reputation **Strategy** + badge **Observer** + reputation **privilege table** + moderator-typed methods + inverted-index search + a facade that checks, delegates and scores.

18.2 Design Cricinfo



Figure — A ball hitting the stumps under the ESPNcricinfo logo, the cover image of this case study (course diagram)

Cricinfo (today ESPNcricinfo) is a sports news site only for cricket. It gives **live ball-by-ball commentary** of matches, keeps a database of every historic match, and publishes news and articles.

Before we start, a two-minute cricket primer, because this design is graded on whether you model the *rules*, not just the nouns:

Term	Meaning
Delivery / ball	One throw ("bowl") of the ball by the **bowler** to the **batter** on strike
Over	Six **legal** deliveries by one bowler. A **wide** (too far from the batter) or a **no ball** (illegal delivery, e.g. overstepping the line) gives the batting side a penalty run and must be bowled again, so it does not count toward the six
Striker / non-striker	Two batters are in at once. The striker faces the ball; they change ends whenever they complete an odd number of runs (1, 3, 5), and again after every over
Runs	Off the bat (credited to the batter) or **extras** — wides, no balls, **byes** and **leg byes** (credited to the team, not the batter)
Wicket / dismissal	A batter is out: bowled, caught, LBW (leg before wicket), stumped, hit wicket, run out, obstructing the field. A **run out is not credited to the bowler**
Innings	One team's turn to bat, ending when 10 batters are out ("all out"), the overs run out, or the target is reached
Formats	**T20** (20 overs per side), **ODI** (One Day International, 50 overs per side), **Test** (no over limit, two innings per side, over several days)
Squad / playing eleven	Before a tournament each team names a **squad**; for every match it chooses exactly **eleven** of those players

Problem & clarifying questions

Question to ask	Answer we assume
Which formats?	T20, ODI and Test
Who enters the data?	An **Admin** (scorer) records every ball; a **Commentator** adds text for each ball
Do we follow real cricket rules?	Yes — the international rules, at least: six legal balls per over, extras, who gets credit for a wicket, strike rotation, bowler limits
Live updates?	Yes — the scorecard and commentary update after every ball (push to subscribers, i.e. readers who asked for updates)
Stats?	Per player, match and tournament, plus global questions such as "highest wicket taker of all time"
Tournaments?	Yes, with squads and a points table
Out of scope?	The course leaves these out on purpose: rain rules (Duckworth–Lewis–Stern, a formula that resets the target when rain shortens a match), the follow-on and declarations in a Test, powerplays and fielding restrictions, free hits after a no ball, telling a leg-side wide from an off-side one, partnerships, net run rate (a team's scoring rate minus its opponents', used to break ties in a points table), and the site's news articles. (Our code adds a simple `declare()`; several others appear under Extensions)

Requirements

Functional (the course's eight, in our words, plus the rules they imply)

1. Track every team that plays cricket, and every match it plays.
2. Show live ball-by-ball commentary.
3. Follow international cricket rules.
4. Each team announces a squad for each tournament.
5. For each match, each team announces its playing eleven from the tournament squad.
6. Keep statistics for players, for matches and for tournaments.

7. Answer global statistics questions, such as "which bowler has taken the most wickets ever?" or "which batter has made the most Test hundreds?"
8. Track ODI, Test and T20 matches.

Non-functional

- **Accuracy:** stats must match the ball-by-ball record exactly (a scorer's correction must fix every derived number).
- **Low-latency live updates** (**latency** = the delay between something happening and the reader seeing it)**:** thousands to millions of readers see a new ball within seconds.
- **Consistency of recording:** two scorers must not interleave balls in the same match.
- **Extensibility:** a new stat question, a new format, or a new subscriber (push alerts) should be a new class.

Use cases

Two actors: the **Admin** adds or modifies players, teams, tournaments and matches and records the ball-by-ball details; the **Commentator** writes the text commentary for each ball.

Top use cases: add/modify teams and players; add tournaments and matches; add ball; add stadium, umpire and referee; add/update stats (some generated by the system); add commentary.

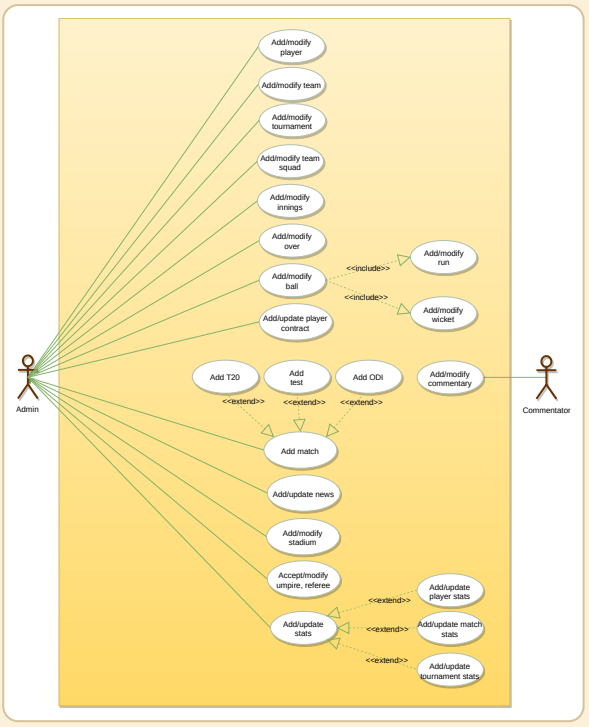


Figure — Cricinfo use cases: the Admin adds tournaments, matches (extended by Test, T20 and ODI), teams, squads, players, stadiums, umpires, innings, overs, balls (which include runs and wickets) and stats; the Commentator adds commentary (course diagram)

In the figure above, "Add match" is *extended* by the three formats, and "Add ball" *includes* adding its runs and wicket — a ball is recorded together with what happened on it.

Activity — record a ball. The scorer selects *Add Ball*; picks the ball type (normal, wide, no ball, or wicket-taking); if a wicket fell, adds wicket details; if runs were scored, adds a run record for each run type; adds commentary; saves the ball.

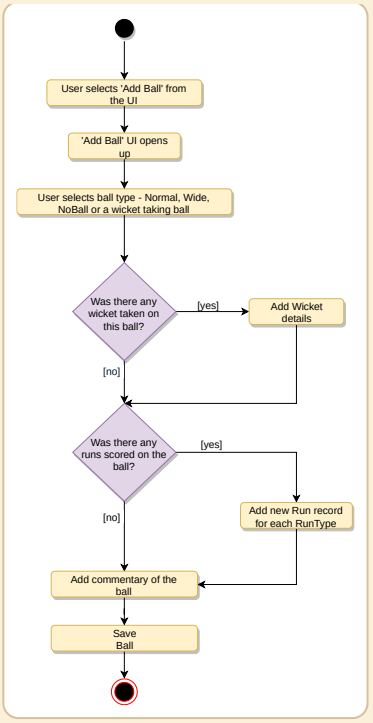


Figure — Activity diagram "record a ball of an over": choose ball type, add wicket details if any, add run records if any, add commentary, save (course diagram)

Core entities

Class	Responsibility
`Person` (record)	A human: name, country. Roles *hold* a person
`Player`	A role holding a `Person` (the course's `Player` also keeps a basic profile and contract details)
`MatchOfficial` (abstract) → `Umpire`, `Referee`, `Commentator`	Shared "assign to a match" behaviour; each says its role
`Team`	Every player the team has ever picked
`TournamentSquad`	Who a team may use in one tournament (must be team members)
`Playing11`	Exactly eleven, all from the squad, in batting order
`Tournament`	Owns squads, lists matches, keeps the points table
`BallOutcome` (record)	What happened: runs off the bat, extra type and runs, boundary?, wicket type, which batter was out, commentary text. Validates the laws (no bowled off a no ball)
`Ball` (record)	A recorded delivery: over, delivery number, bowler, striker, outcome, wicket
`Wicket` (record) + `WicketType`	Who was out, how, off whose bowling; `creditedToBowler()`
`Over`	One bowler's six **legal** deliveries; runs conceded; maiden?
`Innings`	Owns overs; decides striker, end of over, new batter, bowler limits, and when the innings ends
`Match` (abstract) → `T20`, `ODI`, `Test`	Holds everything; formats say only overs per innings and innings per side
`Result` (sealed) → `Win`, `Tie`, `Draw`	The outcome
`MatchObserver` → `CommentaryFeed`, `LiveScorecard`	Subscribers to ball / over / innings / match events
`PlayerStats`, `StatsService`	Numbers derived from balls
`StatQuery` (abstract) → `MostRuns`, `MostHundreds`, `MostWickets`, `MostSixes`	Template Method: one question per class

Class diagram

The course builds this design in **six stages**.

Stage 1 — the people (`Person`, `Player`, `MatchOfficial` → `Umpire`/`Referee`/`Commentator`, `Admin`). A player *holds* a person instead of extending one: one real human can play today and commentate after retiring, and composition lets them take both roles without copying. Umpires, referees and commentators all do one thing in common — get assigned to matches — so that lives once in `MatchOfficial`.

Stage 2 — teams, squads and the eleven. Requirements 4 and 5 look like one idea but are three nested sets: the team (everyone ever picked) ⊇ the squad (one tournament) ⊇ the eleven (one match). `Playing11` keeps a reference to its squad, which lets it reject anyone outside the squad, and it also rejects a twelfth pick. Requirement 5 thus lives in code rather than in a comment. Only the tournament owns anything (its squads).

Stage 3 — the delivery (enums `WicketType`, `BallType`, `RunType`; classes `Run`, `Wicket`, `Commentary`, `Ball`, `Over`). This stage decides whether the design understands cricket. Three rules hide in it: (1) an over is six *legal* deliveries, so `Over.isComplete` counts only legal balls, and an over with two wides takes eight deliveries; (2) a run out is credited to the fielding side, *not the bowler* — get this wrong and "highest wicket taker" is silently wrong for years; (3) byes and leg byes belong to the team, not

the batter, so each run carries a type that says who gets the credit. `Ball` owns its runs, wicket and commentary (composition); it only *refers* to the bowler and batter.

Stage 4 — the innings. Because the innings owns the overs, it alone decides when an over is finished and a new one starts; if that rule lived anywhere else, it would soon exist in two slightly different copies. Its `score` uses scorecard notation: runs/wickets, then full overs and the legal balls of the over in progress.

Stage 5 — the match and the formats (`Stadium`, abstract `Match` owning its innings, `ODI`, `T20`, `Test`). Before creating three classes, check what really changes between the formats. Starting an innings, recording a ball and deciding the winner are the same. Only two numbers differ: overs per innings and innings per match. So `Match` is abstract and holds everything; each format overrides two short methods. A Test returns zero overs (meaning "no limit") and four innings in total.

Stage 6 — stats (`PlayerStats`, `StatsService`, the `StatQuery` hierarchy). **Template Method** (§11.4): a base class fixes an algorithm's outline and subclasses fill in one step — here `StatQuery.answer()` loops over players keeping the best, and each subclass supplies only `valueOf()` ("runs", "wickets"…). Stats are **worked out from the balls**, not stored beside them. Keep a separate counter and it will drift from the ball record as soon as a scorer fixes a mistake, and after that nobody can say which number is correct. Fifties and hundreds are per-innings milestones, so they are counted after each innings (`countInnings`) next to the per-ball counting (`countBall`). Each global question is one small `StatQuery` subclass.

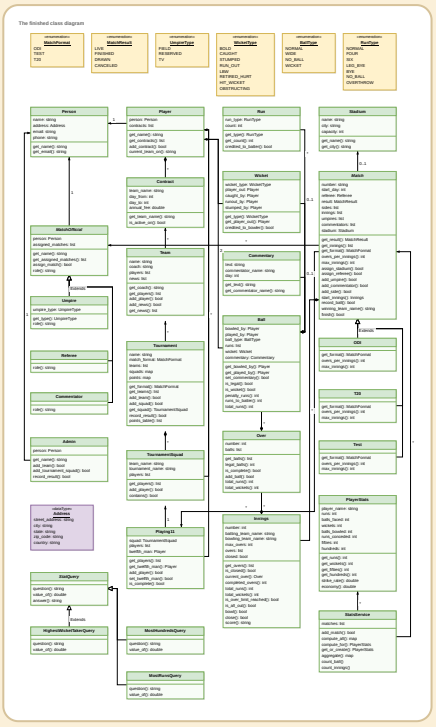
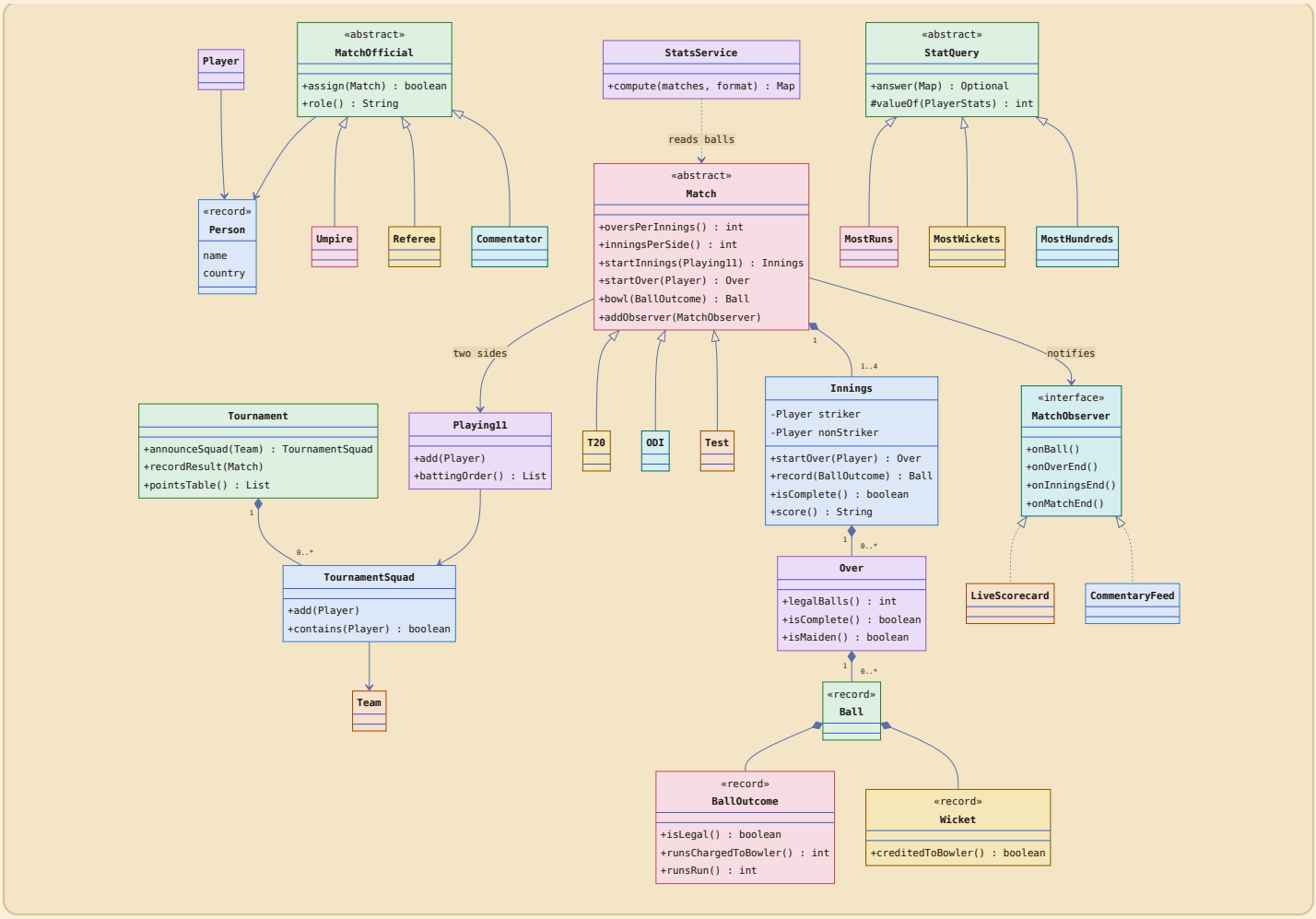


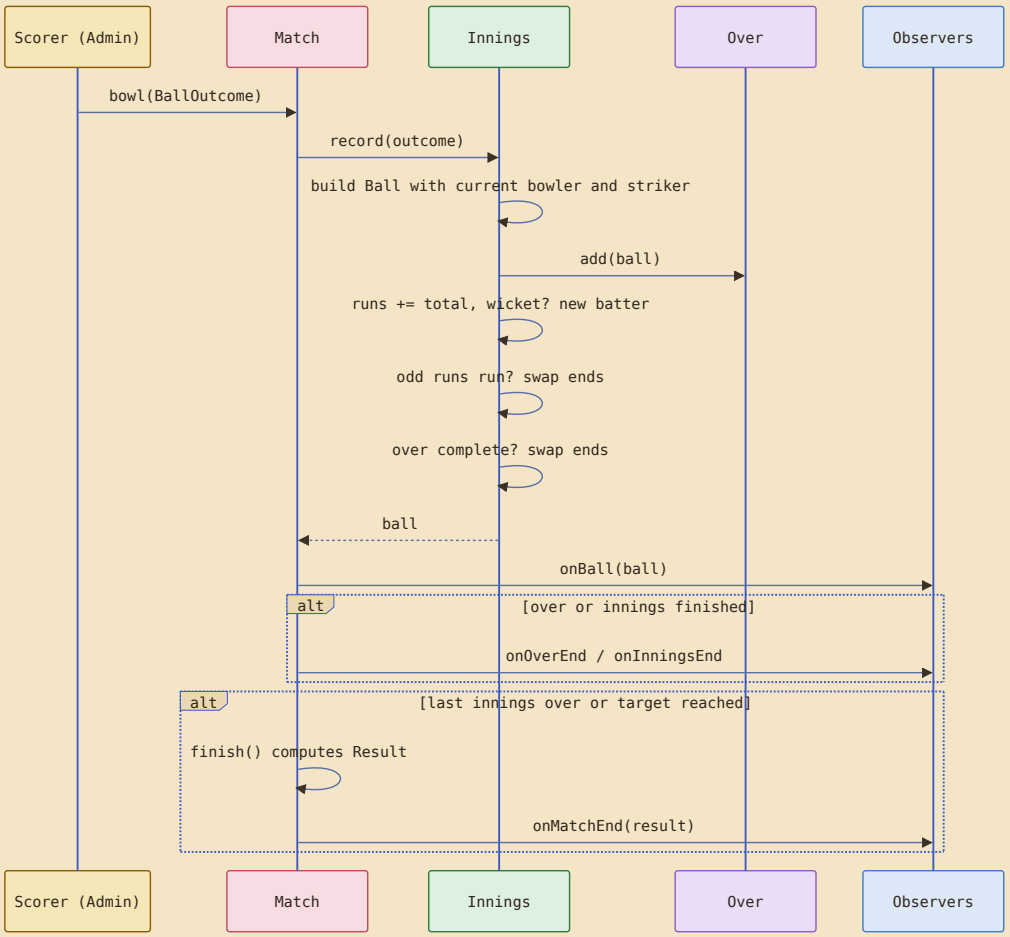
Figure — The finished Cricinfo class diagram, with every class from the six stages in full colour (course diagram)

Our Java design keeps all six stages and adds what the course flags as missing: **strike rotation** (who is on strike), bowler rules (not two overs in a row; 4 overs per bowler in a T20, 10 in an ODI), the laws about which dismissals are possible off a wide or no ball, a result calculation, and **Observers** for live updates. Instead of separate `Run` objects per run type, a `BallOutcome` record carries bat runs, extra type and extra runs — the same information in one immutable value (an object that can never be changed after it is created).

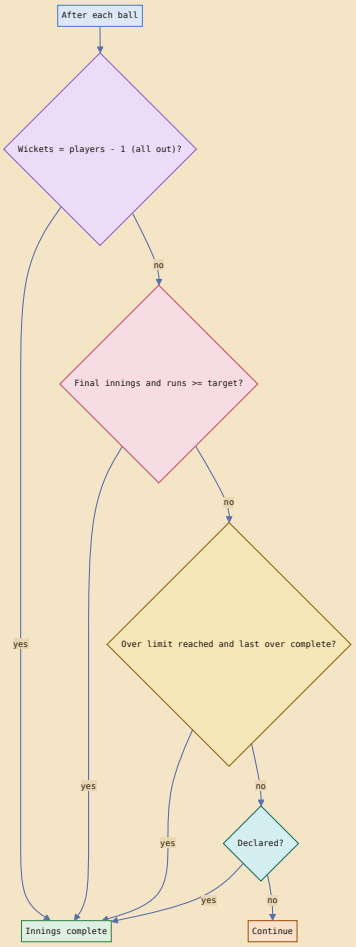


Key flows

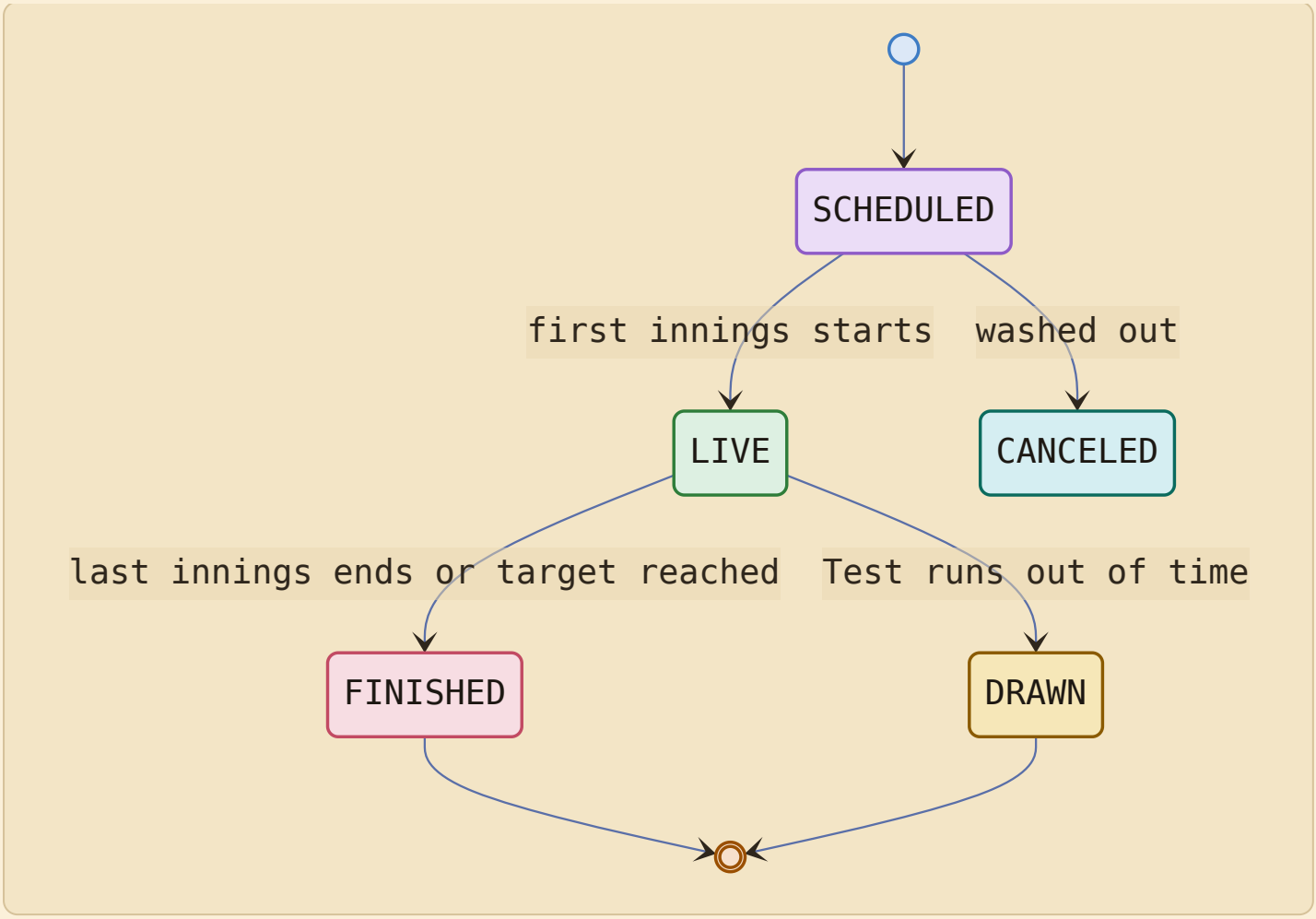
Recording one ball (the scorer's "Add Ball"):



Innings end conditions (checked by `Innings.isComplete`):



Match status:


SCHEDULED
first innings starts
washed out
LIVE
CANCELED
last innings ends or target reached
Test runs out of time
FINISHED
DRAWN

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
`Player` and `MatchOfficial` hold a `Person`	Composition over inheritance	One human, many roles over time	`Player extends Person` — a player who becomes a commentator must be copied
`Match` abstract; `T20` / `ODI` / `Test` override two numbers	Template-style base class (the course notes it is not a named pattern here)	Only what differs is overridden	Three full `Match` copies
`StatQuery.answer()` final, `valueOf()` abstract	**Template Method**	A new statistic is one tiny subclass; the search loop is written once (Open/Closed: open to new features, closed to editing old code)	A `switch (questionType)` inside the service
Stats derived from balls	Single source of truth	Corrections fix every number automatically	Stored counters that drift; a cache is fine if it is rebuildable
`Innings` owns the over rule and strike	Information Expert (GRASP — guidelines for deciding which class gets a job)	The class with the data makes the decision	Scorer UI deciding when an over ends
`BallOutcome` validates the laws in its constructor	Fail fast, immutable value	Impossible deliveries (bowled off a no ball) never enter the record	Checking later in the stats code
`MatchObserver` with default methods	**Observer**	Commentary feed, live scorecard, push alerts subscribe without the match knowing them	The match printing to the console itself
`Result` is a sealed interface of records	Algebraic data type (a value that is exactly one of a closed set of shapes)	`switch` over results is exhaustive (see `Tournament.recordResult`)	Strings like "Team A won"
`Match.bowl` is `synchronized`	Monitor lock (Java's built-in per-object lock) per match	Two scorers cannot interleave balls	Unsynchronised updates that corrupt the over

Java implementation

Package `lld.ch18.cricinfo`. The demo builds two teams, a tournament, squads and elevens (showing both refusals), then plays a scripted first over (a wide and a no ball make it **eight deliveries**; a bowled that counts for the bowler; a run out that does not), and simulates the rest with a fixed random **seed** (the same seed always gives the same "random" sequence) so the output is repeatable. An ODI is simulated too, so the global stat queries have data from two formats.

Full runnable code is in `code/lld/ch18/cricinfo/`: `People.java` (`Person`, `Player`, `MatchOfficial` → `Umpire` / `Referee` / `Commentator`), `Teams.java` (`Team`, `TournamentSquad`, `Playing11`, `Tournament`), `Delivery.java`, `Innings.java`, `Match.java`, `Observers.java` (`CommentaryFeed`, `LiveScorecard`, `Scorecard`), `Stats.java` and `Main.java`. The essential lines (`/* ... */` and `//` summary comments stand for code left out):

```java
enum WicketType {
    BOWLED, CAUGHT, LBW, STUMPED, HIT_WICKET, RUN_OUT, OBSTRUCTING_THE_FIELD;
    /** A run out (or obstructing the field) is a wicket for the fielding side, NOT for the bowler. */
    boolean creditedToBowler() { return this != RUN_OUT && this != OBSTRUCTING_THE_FIELD; }
}

/** What happened on one delivery, as typed by the scorer. Validates the laws in its constructor. */
record BallOutcome(int batRuns, Extra extra, int extraRuns, boolean boundary,
                   WicketType wicket, boolean nonStrikerOut, String commentary) {   // Extra: NONE, WIDE, NO_BALL,
BYE, LEG_BYE
    BallOutcome {
        if ((extra == Extra.WIDE || extra == Extra.NO_BALL) && extraRuns < 1)
            throw new IllegalArgumentException(extra + " carries at least one penalty run");
        // Laws of cricket: off a no ball the bowler cannot take a wicket; off a wide only stumped/hit wicket/run
out.
        if (wicket != null && extra == Extra.NO_BALL && wicket.creditedToBowler())
            throw new IllegalArgumentException(wicket + " is not possible off a no ball");
        /* ... same check for a wide ... */
    }
    /** A wide or a no ball is not one of the six balls of an over. */
    boolean isLegal() { return extra != Extra.WIDE && extra != Extra.NO_BALL; }
    /** Byes and leg byes are not the bowler's fault; wides and no balls are. */
    int runsChargedToBowler() { return batRuns + (isLegal() ? 0 : extraRuns); }
    /** Runs physically run between the wickets: an odd number means the batters swapped ends (0 for a boundary).
*/
    int runsRun() { /* ... */ }
}

/** An over is SIX LEGAL deliveries, not six balls. */
final class Over {
    static final int LEGAL_BALLS = 6;
    int legalBalls() { return (int) balls.stream().filter(b -> b.outcome().isLegal()).count(); }
    boolean isComplete() { return legalBalls() == LEGAL_BALLS; }
    boolean isMaiden() { return isComplete() && runsConceded() == 0; }
}
```

```java
/** One side's turn to bat. It owns its overs, so it is the ONE place that decides over end, strike and innings
end. */
final class Innings {
    Over startOver(Player bowler) {
        // ... innings not complete, previous over finished, bowler in the fielding eleven
        if (current != null && current.bowler().equals(bowler))
            throw new IllegalStateException(bowler + " cannot bowl two overs in a row");
        if (maxOversPerBowler > 0 && bowled >= maxOversPerBowler)
            throw new IllegalStateException(bowler + " has used all " + maxOversPerBowler + " overs");
        /* ... add the new Over ... */
    }

    Ball record(BallOutcome o) {
        Over over = currentOver();
        int deliveryNo = over.legalBalls() + 1;          // a wide or no ball is re-bowled, so it shares the next
number
        Wicket wicket = null;
        if (o.wicket() != null) wicket = new Wicket(o.wicket(), o.nonStrikerOut() ? nonStriker : striker,
over.bowler());
        Ball ball = new Ball(over.number(), deliveryNo, over.bowler(), striker, o, wicket);
        over.add(ball);
        balls.add(ball);
        runs += o.totalRuns();
        if (o.runsRun() % 2 == 1) swapEnds();              // odd runs: batters crossed
        if (wicket != null) { wickets++; /* next batter in the order replaces whoever was out */ }
        if (over.isComplete()) swapEnds();                  // ends change after every over
        return ball;
    }

    boolean isComplete() {
        if (closed || allOut()) return true;               // closed = declared (a Test captain may declare)
        if (target != null && runs >= target) return true;
        Over current = currentOver();
        return maxOvers > 0 && overs.size() == maxOvers && current != null && current.isComplete();
    }
}

/** Everything the same in every format lives here; formats answer only two questions. */
abstract class Match {
    abstract int oversPerInnings();       // 0 means no limit
    abstract int inningsPerSide();
    /** ODIs allow 10 overs per bowler and T20s 4, i.e. one fifth of the innings. */
    int maxOversPerBowler() { return oversPerInnings() / 5; }

    /** Records one delivery and fires the events; synchronized so two scorers cannot interleave balls. */
    synchronized Ball bowl(BallOutcome outcome) {
        Innings in = requireCurrent();
        Ball ball = in.record(outcome);
        observers.forEach(o -> o.onBall(this, in, ball));
        // onOverEnd when the over or innings completes; onInningsEnd, then finish() -> Result.Win/Tie and
onMatchEnd
        return ball;
    }
}
final class T20 extends Match { @Override int oversPerInnings() { return 20; } @Override int inningsPerSide() {
return 1; } }
final class ODI extends Match { @Override int oversPerInnings() { return 50; } @Override int inningsPerSide() {
return 1; } }
final class Test extends Match { @Override int oversPerInnings() { return 0; }  @Override int inningsPerSide() {
return 2; } }

sealed interface Result { record Win(Team winner, String margin) implements Result {} record Tie() implements
Result {} record Draw() implements Result {} }
interface MatchObserver {   // default no-op methods: subscribers override only what they need
    default void onBall(Match m, Innings in, Ball b) {}
    default void onOverEnd(Match m, Innings in, Over o) {}
    default void onInningsEnd(Match m, Innings in) {}
    default void onMatchEnd(Match m, Result r) {}
}
// Playing11.add rejects anyone outside the squad and a twelfth player; TournamentSquad.add rejects non-team
```

```
players.
// Tournament.recordResult switches exhaustively over Result: 2 points for a win, 1 each for a tie.
```

```
/** Recomputes stats from the recorded balls of the given matches, optionally for one format. */
final class StatsService {
    Map<Player, PlayerStats> compute(Collection<Match> matches, MatchFormat formatOrNull) {
        /* for every ball: striker.countBall(b, true), bowler.countBall(b, false);
           after each innings: countInnings(runs) for the 50/100 milestones */
    }
}
// PlayerStats.countBall credits a wicket to the bowler only if wicket.creditedToBowler(); byes are not his runs.

/** Template Method: answer() is the fixed algorithm (walk all players, keep the best); valueOf() is the step. */
abstract class StatQuery {
    record Leader(Player player, int value) {}
    abstract String question();
    abstract int valueOf(PlayerStats s);
    final Optional<Leader> answer(Map<Player, PlayerStats> stats) { /* max of valueOf, ties broken by name */ }
}
final class MostWickets extends StatQuery {
    @Override String question() { return "Highest wicket taker"; }
    @Override int valueOf(PlayerStats s) { return s.wickets; }
}
// MostRuns, MostHundreds, MostSixes: one line each.
```

Output:

```
== Picking the elevens ==
  REJECTED: Manav is not in the Lions squad
  REJECTED: a playing eleven cannot have a twelfth player (Lokesh)

== T20 Lions v Tigers at Riverside Oval ==
  0.1  Tarun to Arun, FOUR  [Ravi: driven through cover]
  0.2  Tarun to Arun, wide  [Ravi: sprayed down leg]
  0.2  Tarun to Arun, 1 run(s)  [Ravi: pushed to mid-on]
  0.3  Tarun to Bala, no ball  [Ravi: overstepped, free hit next]
  0.3  Tarun to Bala, OUT BOWLED (Bala)  [Ravi: through the gate!]
  0.4  Tarun to Chetan, OUT RUN_OUT (Chetan) - not the bowler's wicket  [Ravi: a second run was never there]
  0.5  Tarun to Arun, no run  [Ravi: defended]
  0.6  Tarun to Arun, SIX  [Ravi: launched over long-on]
  REJECTED: Tarun cannot bowl two overs in a row
  [live] Lions 29/2 (5 ov)
  [live] Lions 76/2 (10 ov)
  [live] Lions 117/3 (15 ov)
  [live] Lions 151/5 (20 ov)
  [innings 1 over] Lions 151/5 (20 ov)
  0.1  Gopal to Nabil, wide
  0.1  Gopal to Nabil, 1 run(s)
  0.2  Gopal to Omar, 1 run(s)
  0.3  Gopal to Nabil, 2 run(s)
  0.4  Gopal to Nabil, no run
  0.5  Gopal to Nabil, wide
  0.5  Gopal to Nabil, no run
  0.6  Gopal to Nabil, no ball
  0.6  Gopal to Nabil, no run
  [live] Tigers 40/0 (5 ov)  need 112 to win
  [live] Tigers 76/0 (10 ov)  need 76 to win
  [live] Tigers 91/2 (15 ov)  need 61 to win
  [live] Tigers 128/4 (20 ov)  need 24 to win
  [innings 2 over] Tigers 128/4 (20 ov)
  [result] Lions won by 23 runs

== Scorecards ==
Batting - Lions
  Arun       b Uday                 45 (32)  4s:4 6s:1
  Bala       b Tarun                 0 (2)  4s:0 6s:0
  Chetan     run out                 1 (1)  4s:0 6s:0
  Dev        caught b Yash          51 (51)  4s:4 6s:2
  Eshan      caught b Wasim         42 (31)  4s:5 6s:1
  Farhan     not out                 5 (3)  4s:0 6s:0
  Gopal      not out                 1 (1)  4s:0 6s:0
  Extras 6   Total 151/5 (20 ov)
Bowling - Tigers
  Tarun      4.0-0-41-1
  Uday       4.0-0-27-1
  Varun      4.0-0-29-0
  Wasim      4.0-0-28-1
  Yash       4.0-0-24-1
Batting - Tigers
  Nabil      lbw b Kiran            55 (56)  4s:3 6s:2
  Omar       lbw b Jai              44 (41)  4s:5 6s:0
  Pranav     run out                 1 (4)  4s:0 6s:0
  Qasim      not out                20 (19)  4s:1 6s:1
  Rahul      caught b Kiran          0 (1)  4s:0 6s:0
  Sameer     not out                 0 (0)  4s:0 6s:0
  Extras 8   Total 128/4 (20 ov)
Bowling - Lions
  Gopal      4.0-0-21-0
  Hari       4.0-0-22-0
  Imran      4.0-0-38-0
  Jai        4.0-0-19-1
  Kiran      4.0-0-27-2
Points table: [Lions=2, Tigers=0]

== ODI Tigers v Lions at Riverside Oval ==
  [live] Tigers 181/3 (25 ov)
  [live] Tigers 348/9 (50 ov)
```

```
  [innings 1 over] Tigers 348/9 (50 ov)
  [live] Lions 159/1 (25 ov)  need 190 to win
  [live] Lions 349/5 (49.4 ov)
  [innings 2 over] Lions 349/5 (49.4 ov)
  [result] Lions won by 5 wickets

== Global stat queries (all formats) ==
  Most runs: Chetan (131)
  Most hundreds: Chetan (1)
  Highest wicket taker: Kiran (5)
  Most sixes: Nabil (5)
  Most runs in ODIs: Chetan (130)
  Ravi is assigned to 1 match(es)
```

Walking through the important parts

- Ball numbers follow the scorecard convention "completed overs . ball", so the first ball of the match is `0.1`. A wide or no ball carries the number of the ball that must be re-bowled, which is why `0.2` and `0.3` each appear twice.
- In over 1 of the output, there are eight lines but the over ends at `0.6`: the wide and the no ball did not count toward the six (`Over.isComplete` counts legal balls).
- Chetan is **run out** after completing one run: `Innings.record` first swaps ends for the completed single, then replaces the dismissed batter, so the new batter (Dev) comes in at the correct end and Arun faces the next ball. In the scorecard, Chetan's line says "run out" and Tarun's bowling figures show only **1** wicket (the bowled), because `WicketType.RUN_OUT.creditedToBowler()` is `false`.
- Trying to give Tarun the second over is rejected ("cannot bowl two overs in a row"). The simulation rotates five bowlers, so each bowls exactly 4 overs in the T20 (`maxOversPerBowler() = 20 / 5`).
- `LiveScorecard` prints every 5 overs and the chase line ("need 112 to win") comes from the target set automatically for the final innings (`first-innings total + 1`). The random ball generator is tuned to about 6.5 runs an over, so totals look like real matches (151 and 128 in the T20; 348 and 349 in the ODI, which the Lions win by 5 wickets with two balls to spare).
- The batting card lists every batter who faced a ball or was out, plus the two batters at the crease (Sameer is "not out 0 (0)": he walked in but never faced).
- The global queries (`MostRuns`, `MostHundreds`, `MostWickets`, `MostSixes`) are each a one-line subclass; "Most runs in ODIs" reuses the same service with a format filter.

Machine-coding variant: Cricket Match Dashboard (Udaan)

A **machine-coding round** asks you to write a *working, runnable* program in 60–90 minutes, judged on correctness, readability and extensibility. This problem was set by Udaan (an Indian B2B — business-to-business — e-commerce company) and appears in the kumaransg/LLD collection.

Problem statement (in our words)

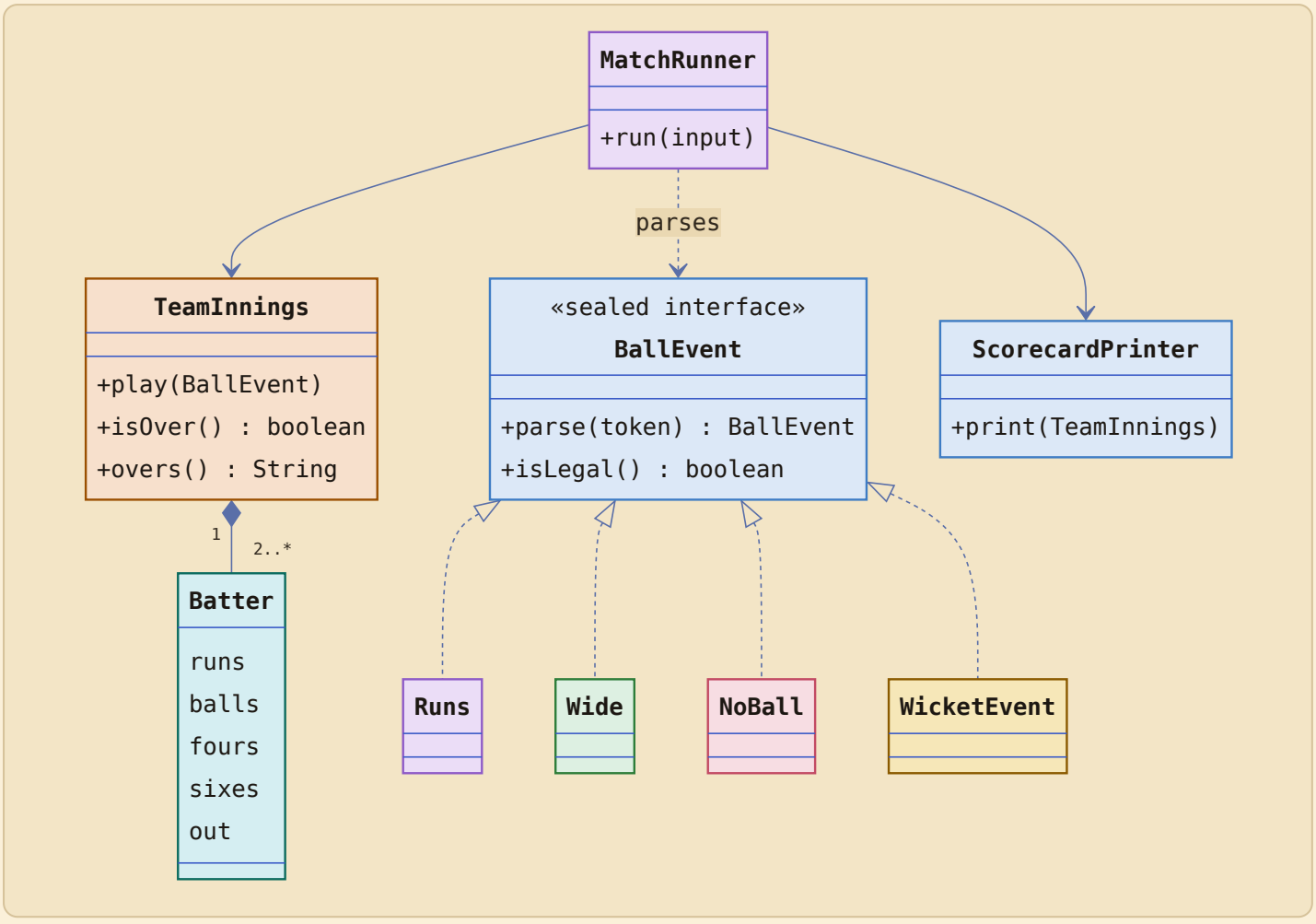
Build a cricket **scorecard** for a two-team, limited-overs match.

- **Input:** number of players per team, number of overs, then for each team its batting order, followed by the balls of each over, one per line: a number of runs (`0`–`6`), `W` (wicket), `Wd` (wide) or `Nb` (no ball).
- **Output after every over:** for each batter of the batting side — runs, 4s, 6s, balls faced — with a `*` on the two batters currently at the crease; then team total/wickets and overs.
- **Rules to handle:** strike changes on 1s and 3s and at the end of every over; wides and no balls add one run and are re-bowled (they do not count toward the six); the innings ends when the team is all out, the overs run out, or (second innings) the target is passed.
- **At the end:** print the winner and the margin (by runs or by wickets).
- **Good to have:** bowler figures (overs, runs, wickets, maidens, dot balls, economy), team extras, batter strike rates.

How the full Cricinfo design shrinks to a 90-minute solution

Cricinfo class	Machine-coding version	Why it is enough here
`BallOutcome` with extras, wickets, validation	`BallEvent` sealed interface: `Runs`, `Wide`, `NoBall`, `Wicket` with a `parse(token)` factory (a method that builds the right object from one input word)	Input has only four kinds of token
`Innings` (strike, over rule, end conditions)	`TeamInnings` — same responsibilities, ints instead of objects	Same rules, no bowler data in the input
`Scorecard` derived from balls	`Batter` counters + `ScorecardPrinter`	Only batting card needed; printer separated so another view could replace it
`Match` + `Result`	`MatchRunner` (reads input, plays two innings, prints result)	One format, two innings

This is the lesson of machine coding: keep the *same responsibilities in the same places* (the innings owns strike and the over rule; parsing is separate from scoring; printing is separate from both), but use the simplest data that satisfies the input.



(In the code the wicket record is called `BallEvent.Wicket`; it is shown as `WicketEvent` above only to keep the diagram label unique.)

Code

Package `lld.ch18.cricketdashboard`; full runnable code is in `code/lld/ch18/cricketdashboard/` (`Model.java`: `BallEvent`, `Batter`, `TeamInnings`; `Main.java`: `ScorecardPrinter`, `MatchRunner`, `Main`). `Main` feeds the problem's sample input as a text block: 5 players, 2 overs, batting orders P1–P5 and P6–P10, team 1's balls `1 1 1 1 1 2` then `W 4 4 Wd W 1 6`, team 2's `4 6 W W 1 1` then `6 1 W W`. The essential lines:

```
/** One input token, parsed once. Factory method: BallEvent.parse("Wd") etc. */
sealed interface BallEvent {
    record Runs(int runs) implements BallEvent {}
    record Wide() implements BallEvent {}
    record NoBall(int batRuns) implements BallEvent {}
    record Wicket() implements BallEvent {}

    static BallEvent parse(String token) {
        String t = token.trim();
        if (t.equalsIgnoreCase("W")) return new Wicket();
        if (t.equalsIgnoreCase("Wd")) return new Wide();
        if (t.regionMatches(true, 0, "Nb", 0, 2)) return new NoBall(t.length() > 2 ?
Integer.parseInt(t.substring(2)) : 0);
        int runs = Integer.parseInt(t);
        if (runs < 0 || runs > 6) throw new IllegalArgumentException("bad runs: " + t);
        return new Runs(runs);
    }
    default boolean isLegal() { return !(this instanceof Wide || this instanceof NoBall); }
}

/** One team's batting innings: owns strike rotation, extras, and the "is it over?" rule. */
final class TeamInnings {
    void play(BallEvent e) {
        if (isOver()) throw new IllegalStateException("innings already finished");
        Batter s = order.get(striker);
        switch (e) {
            case BallEvent.Wide w -> { total += 1; extras += 1; }
            case BallEvent.NoBall nb -> {
                total += 1 + nb.batRuns(); extras += 1;
                s.faced(nb.batRuns());              // a no ball counts as a ball faced by the batter
                if (nb.batRuns() % 2 == 1) swap();
            }
            case BallEvent.Wicket w -> {
                legalBalls++;
                s.balls++;
                s.out = true;
                wickets++;
                if (!allOut()) striker = next++;    // new batter takes strike
            }
            case BallEvent.Runs r -> {
                legalBalls++;
                total += r.runs();
                s.faced(r.runs());
                if (r.runs() % 2 == 1) swap();      // 1s and 3s change strike
            }
        }
        if (e.isLegal() && legalBalls % 6 == 0 && !isOver()) swap();  // end of over
    }
    boolean isOver() { return allOut() || legalBalls == maxOvers * 6 || (target != null && total >= target); }
}
// Batter: runs, balls, fours, sixes, out, strikeRate(). ScorecardPrinter is separate (Single Responsibility)
// so a web or JSON view could replace it.

// MatchRunner.playInnings: for each "Over N:" line, read tokens until 6 LEGAL balls or the innings ends,
// then print the card; skip input left over after the innings ended. run() declares the winner by runs or
wickets.
while (legalThisOver < 6 && !in.isOver()) {         // an over = 6 LEGAL balls
    BallEvent e = BallEvent.parse(lines.poll());
    in.play(e);
    if (e.isLegal()) legalThisOver++;
}
```

Output:

```
Batting order for Team 1: [P1, P2, P3, P4, P5]
Over 1:
Scorecard for Team 1:
Player   Score  4s  6s Balls     SR
P1*          3   0   0     3  100.0
P2*          4   0   0     3  133.3
P3           0   0   0     0      -
P4           0   0   0     0      -
P5           0   0   0     0      -
Total: 7/0   Extras: 0
Overs: 1
Over 2:
Scorecard for Team 1:
Player   Score  4s  6s Balls     SR
P1           3   0   0     4   75.0
P2*         10   0   1     4  250.0
P3           8   2   0     3  266.7
P4*          1   0   0     1  100.0
P5           0   0   0     0      -
Total: 23/2   Extras: 1
Overs: 2
Batting order for Team 2: [P6, P7, P8, P9, P10]
Over 1:
Scorecard for Team 2:
Player   Score  4s  6s Balls     SR
P6          10   1   1     3  333.3
P7*          1   0   0     1  100.0
P8           0   0   0     1    0.0
P9*          1   0   0     1  100.0
P10          0   0   0     0      -
Total: 12/2   Extras: 0
Overs: 1
Over 2:
Scorecard for Team 2:
Player   Score  4s  6s Balls     SR
P6          10   1   1     3  333.3
P7*          8   0   1     3  266.7
P8           0   0   0     1    0.0
P9           1   0   0     2   50.0
P10          0   0   0     1    0.0
Total: 19/4   Extras: 0
Overs: 1.4
Result: Team 1 won the match by 4 runs
```

Notes on the sample and on correctness

- Leaving aside the strike-rate and extras columns we add, our output matches the published sample for team 1 exactly, and for team 2 in every line except two places where the published sample itself is inconsistent: after team 2's first over it prints `12/1`, but the over contains **two** `W` tokens (P6 and P8 were both out), so the correct total is `12/2`; and in the final card it shows P6 with 2 balls after 3 in the previous card. Following the stated rules gives `12/2` and 3 balls, which is what our program prints. Pointing out such inconsistencies calmly is a plus in a machine-coding round.
- Team 2's second over stops after four balls because the fourth wicket leaves only one batter (5 players → all out at 4 wickets), so "Overs: 1.4".
- The loop that reads an over counts **legal** balls (`legalThisOver < 6`), so a wide in team 1's second over makes that over seven tokens long.
- Extras and strike rate (two of the "good to have" items) are printed; bowler figures would need bowler names in the input — add a `Bowler` class with the same counters and a line per over naming the bowler.

Concurrency & edge cases

- **Two scorers, one match.** `Match.bowl`, `startOver` and `startInnings` are `synchronized` on the match, so balls are applied one at a time in order. Different matches never block each other.

- **Many readers.** Observers are held in a `CopyOnWriteArrayList` (a list that makes a fresh copy on every change, so readers never see it half-updated), which is safe to iterate while another thread subscribes. At web scale, the match would publish each ball to a **message broker** (a middleman service that receives messages and passes them on to every subscriber) and readers would receive it over **WebSockets** (a connection the browser keeps open so the server can send data at any time) or **server-sent events** (a one-way stream of updates from server to browser); the scorecard would be a cached view rebuilt from balls.
- **Illegal sequences rejected:** starting an over while one is in progress; the same bowler twice in a row; a bowler over his quota; bowling after the innings ended; a bowler from the batting side; a twelfth player; a player outside the squad.
- **Impossible dismissals rejected:** bowled, caught, LBW, stumped or hit wicket off a no ball; bowled, caught or LBW off a wide.
- **Wicket on the last ball of an over:** the new batter comes in, then ends change for the new over — both handled in `Innings.record`.
- **Tie:** equal totals give `Result.Tie` (a super over would be an extension).
- **Test matches:** `oversPerInnings() = 0` means no limit; the demo's `declare()` lets a captain close an innings; `declareDraw()` ends a Test that runs out of time.

Extensions & follow-up questions

- **"Who is on strike?"** — The course leaves this out; our `Innings` tracks it (odd runs and end of over swap ends).
- **"Stats are slow to recompute from every ball."** — Keep a running total per player as a cache, updated by an Observer on each ball, and rebuild it from balls whenever a scorer corrects a ball. The balls stay the truth; the total is only a cache.
- **"Add Duckworth–Lewis–Stern."** — A `TargetCalculator` Strategy on the match: the default returns "opponent total + 1"; a DLS implementation recomputes the target from overs lost.
- **"Add a free hit after a no ball."** — The innings remembers "next legal ball is a free hit"; `BallOutcome` validation then allows only run-out-type dismissals on that ball.
- **"Add partnerships and fall of wickets."** — Derive from balls: split the ball list at each wicket and sum.
- **"Add net run rate to the points table."** — Derive from innings totals and overs faced/bowled; still no stored numbers.
- **"Push notifications for wickets and milestones."** — A new `MatchObserver` that sends a push when `ball.wicket() != null` or a batter crosses 50/100.

Real-world example

ESPNcricinfo (founded in 1993 and bought by ESPN in 2007) is the reference site for cricket statistics; its **Statsguru** tool answers questions like the ones in requirement 7 across every recorded international match. Live pages update ball by ball with commentary lines very much like our `CommentaryFeed`, and millions follow big matches on phones — which is why the push-based Observer design (and, at scale, message brokers and caches) matters.

In the interview

Minutes	What to do
0–5	Clarify formats, actors, and that the real rules matter; say early: **"an over ends after six *legal* balls, however many deliveries that takes"**
5–10	Team ⊇ squad ⊇ playing eleven; people hold a `Person`
10–20	Class diagram: match (abstract) → innings → over → ball; formats differ in two numbers
20–30	Ball-recording flow, strike rotation, **the bowler gets no credit for a run out**, byes are not the batter's runs
30–40	Stats derived from balls; Template Method for stat queries; Observers for live feed
40–45	Concurrency (one scorer lock per match, broadcast to readers) and extensions

30-second answer: "I model cricket's rules, not just its nouns. An over is six legal balls; a wide or no ball is re-bowled. Because the overs belong to the innings, only the innings decides when an over ends, who is on strike and when the innings is over. Formats differ only in overs per innings and innings per match, so they override two methods. Stats are derived from the balls so corrections propagate, and each stat question is a Template Method subclass. Live pages subscribe as observers."

Follow-ups with answers

- *Why not store `wickets` on `Player`?* — It would drift from the ball record and cannot be filtered by format or tournament; derive it.
- *How do you add "most catches"?* — Add the fielder to `Wicket`, count it in `PlayerStats`, and write `MostCatches extends StatQuery` — no change to the service's loop.
- *Is Template Method the right name for `Match`/formats?* — The course says only `StatQuery` is a named pattern here; the formats are plain abstract-class specialisation. Be precise about that.

Self-check

- **Q:** A bowler bowls a wide and a no ball in an over. How many deliveries does the over have? **A:** Eight — six legal plus the two illegal ones.
- **Q:** Who is credited with a run-out wicket? **A:** The fielding side, not the bowler; it counts in the team's wickets but not in the bowler's figures.
- **Q:** Are leg byes added to the batter's score? **A:** No — they are extras for the team, and they are not charged to the bowler either.
- **Q:** Why does `Playing11` hold its `TournamentSquad`? **A:** So it can refuse a player not in the squad (requirement 5) — the rule lives in code.
- **Q:** What two methods distinguish `T20`, `ODI` and `Test`? **A:** `oversPerInnings()` (20, 50, 0 = unlimited) and `inningsPerSide()` (1, 1, 2).
- **Q:** In the Udaan variant, what is the correct team 2 score after over 1 of the sample? **A:** 12/2 — the over contains two wickets.

TL;DR

Cricinfo = people hold a `Person`; team ⊇ squad ⊇ eleven; `Match` (abstract, formats override two numbers) → `Innings` (owns overs, strike, end rules) → `Over` (six **legal** balls) → `Ball` (outcome + wicket). Run outs are not the bowler's; byes are not the batter's. Stats are derived from balls; stat questions are **Template Method** subclasses; live pages are **Observers**. The Udaan dashboard is the same responsibilities in a 90-minute, input-driven form.

18.3 Design LinkedIn



Figure — A tablet showing the LinkedIn logo surrounded by a network of professionals, the cover image of this case study (course diagram)

LinkedIn is a networking site for people's **working lives**. Members link up with colleagues and contacts they trust, and they use the site to find their next career move. A profile lists a member's skills, jobs and schooling; the home page shows a **feed** of updates from the member's network, arranged in sections (modules) the member can reorder. It works much like Facebook, with features specialised for careers: **connections** (mutual, both sides agree), **follows** (one-sided), **endorsements**, **recommendations**, **company pages** and **job postings**.

Problem & clarifying questions

Question to ask	Answer we assume
Is a connection mutual?	Yes: an **invitation** is sent, and the receiver accepts or rejects it. Following is one-sided and needs no approval
Do we show degrees of connection?	Yes: 1st (direct), 2nd (connection of a connection), 3rd; beyond that "out of network"
Who can message whom?	A member can message their connections (messaging most non-connections needs LinkedIn's paid *InMail*, out of scope)
Who can endorse a skill?	Only 1st-degree connections, once per skill
Recommendations?	A member *requests* one from a connection; only that person can write it
Profile stats?	Profile views, number of connections, search appearances
Companies and jobs?	Members create company pages; page admins post jobs; members apply once per job
Feed?	Posts from connections and followed members, ranked (the course leaves the feed out; we add a simple one)

Requirements

Functional (the course's twelve, in our words)

1. A member fills in a profile: basic details, jobs held, schooling, skills and accomplishments.
2. Any user can search for members or companies by name (and jobs by title).
3. Members send, accept (or reject) connection requests.

4. Members can request a recommendation from other members.
5. The system shows profile stats: profile views, total connections, search appearances.
6. Members create posts to share with their connections.
7. Members can comment on posts; they can like or share both posts **and comments**.
8. Members send messages to other members.
9. A member gets a notification for each new message, each connection invitation, and each comment left on one of their posts.
10. Members create company pages and add job postings.
11. Members create groups and join any group.
12. Members follow other members or companies.

Non-functional

- **Symmetry:** a connection must always appear on *both* members, never only one.
- **Fast reads:** feed, search and degree lookups are far more frequent than writes.
- **Privacy/abuse:** blocked members' content disappears; you cannot spam strangers with messages.
- **Extensibility:** new notification channels and feed-ranking rules plug in without edits.

Use cases

Three actors: **Member** (search, connect, post, message, follow…), **Admin** (block and unblock members) and **System** (sends notifications for new messages, invitations and so on).

Top use cases: add/update profile; search members, companies or jobs and send connection requests; follow or unfollow a member or company; send a message to connections; create a post, like and comment; send notifications.

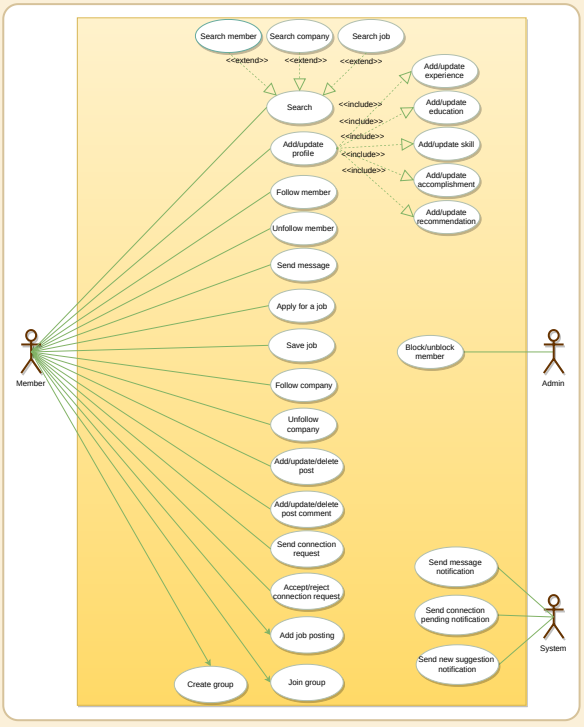


Figure — LinkedIn use cases for Member, Admin and System, including follow/unfollow member, with `<<include>>` *and* `<<extend>>` *relationships (course diagram)*

Activity — add experience to a profile. The member opens *Add Experience*, enters title, company, location, dates and description, optionally uploads a document or photo with a description, and presses save. The system verifies the data; if anything is missing it notifies the member, otherwise it saves the experience.

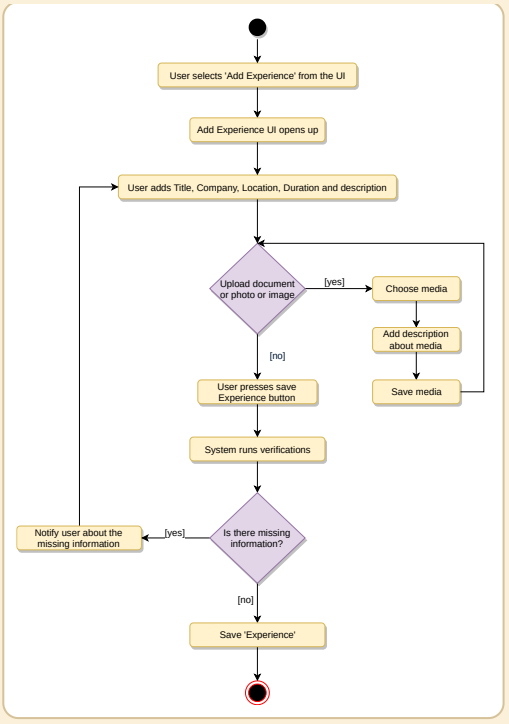


Figure — Activity diagram "add experience to profile" with optional media upload and a verification loop (course diagram)

Activity — send a message. The member composes a message, picks one or more recipients, adds a body and optional media, and presses send. After verification the message goes to every recipient, and each gets an email and a push notification.

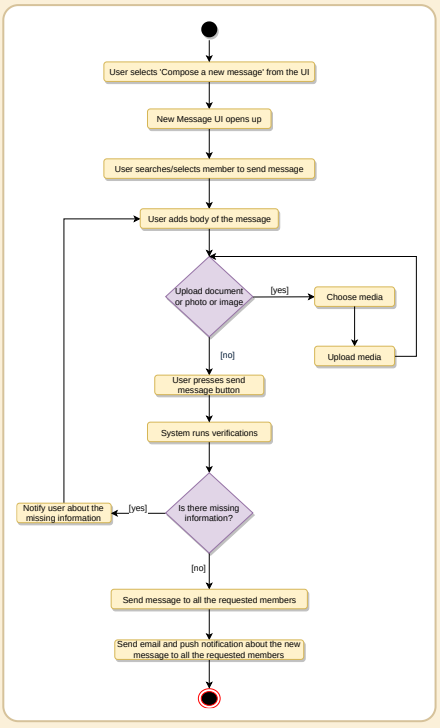


Figure — Activity diagram "send message": choose recipients, add body and media, verify, deliver to all recipients, notify them by email and push (course diagram)

Core entities

Class	Responsibility
`Member`, `Admin`	Identity, profile, connection set (changed only by the service), follows, posts, inbox, notifications
`Profile`	Owns `Experience`, `Education`, `Skill`, `Accomplishment`, `Recommendation`, `ProfileStats`
`Skill`	Name + set of endorsers (no double endorsement)
`Recommendation`	Created empty on request; only the named author writes it (REQUESTED → WRITTEN)
`ProfileStats`	View and search-appearance counters pushed in from where the events happen
`SocialContent` (abstract) → `Post`, `Comment`	Author, text, time, likes as a set; posts own comments and can be shares of another post
`Message` (record)	Sender, recipients, text — not "social content" (nobody likes a message). The course allows media on messages and posts too; our code keeps text only
`ConnectionInvitation` + `InvitationStatus`	State over time: PENDING → ACCEPTED / REJECTED / CANCELED
`ConnectionService`	Invite, accept, reject, withdraw; degree (BFS); mutual connections; people-you-may-know
`Company`, `JobPosting`, `JobApplication`, `Group`	Pages, jobs (apply once), groups (owner cannot leave)
`Notification`, `NotificationService`, channels	One notification class with a *type* field; delivery Strategy
`SearchIndex`	Word → set of members / companies / jobs
`FeedService` + `FeedRanker`	Pull-model feed with pluggable ranking
`LinkedIn`	Facade

Class diagram

The course builds LinkedIn in **six stages**.

Stage 1 — the profile (`Profile` owning `Experience`, `Education`, `Skill`, `Accomplishment`, `Recommendation`, `ProfileStats`). Each of requirement 1's five parts is a small class, not a line of text, because other code uses them (search looks up the current employer; endorsements are counted on the skill). Two details matter. You cannot pass text when creating a `Recommendation`: requirement 4 is about *asking* for one, so the object is born empty and the author fills it in later. And the profile never updates `ProfileStats` itself: whoever witnesses the event bumps the counter — the code that shows a profile, the code that accepts an invitation, the code that returns search results — because an object cannot count events it never sees. All arrows out of `Profile` are composition: delete the profile and its parts go with it.

Stage 2 — posts, comments and messages. Requirement 7 hides a detail: *comments get likes as well.* Put likes only on `Post` and part of the requirement has no home. So both classes extend one parent, `SocialContent`, which holds the author, the text and the likes. The likes are kept as the **ids of the members who liked**, not as a number, which makes a second like by the same person impossible and lets a like be taken back. `Message` is not part of this family: it is delivered to people rather than published, and no one likes a message.

Stage 3 — invitations and notifications (`ConnectionInvitation` with a status, `Notification` with a type). An invitation to connect starts pending and later becomes accepted or rejected. Because it **changes state as time passes**, it deserves its own object (this is the idea behind the **State** pattern: an object's allowed actions depend on which state it is in, and only certain moves between states are legal) rather than a true/false field on a member, and that object can still record, afterwards, that

the answer was no. Requirement 9 lists three notification events, but *delivering* them is identical, so the kind is a **field**, not three subclasses. The general rule: **subclass when behaviour differs; use a field when only data differs.**

Stage 4 — companies, jobs and groups. A company page owns its job postings (a posting means nothing once the page is gone) and keeps followers as a set of ids. A group records who created it, and that creator is never allowed to leave or be removed.

Stage 5 — the actors, and an operation that fits neither of them (`Account`, `Person`, `Member`, `Admin`, `ConnectionService`). Nearly every requirement becomes a method on `Member`. **Accepting an invitation does not**: it edits two members together. If the sender's class did it, the receiver's connection list would be changed by a different object; if each side did half, a bug would one day update only one half, leaving a one-way link. So the operation goes into a small `ConnectionService`, and that single class is responsible for keeping connections **symmetric** (A lists B exactly when B lists A). Following is different: it is one-sided and needs no agreement, so it is just a set on the follower, no service needed.

Stage 6 — search (`Search` interface; `SearchIndex` with three maps from key to list: members, companies, job postings). The course shows a real bug from an earlier version: on the first occurrence of a name the index saved *one* member object, and on the next occurrence it tried to "add" to that object — fine for the first Sharma, broken for the second. The cure is a rule: **an index value is always a collection, even when it holds one item.** Keys are single lowercase words, so first and last names both work.

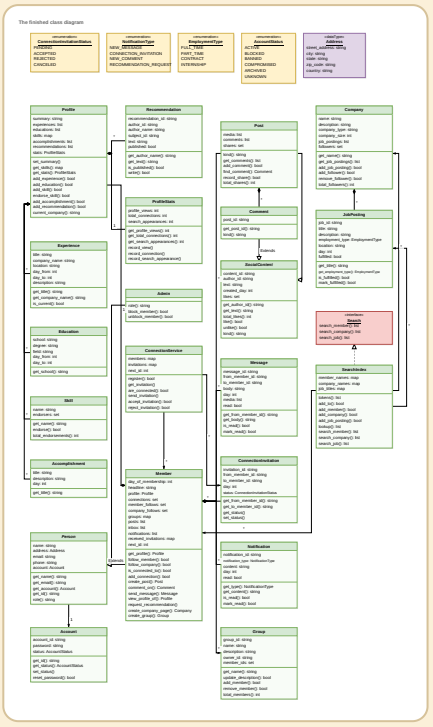
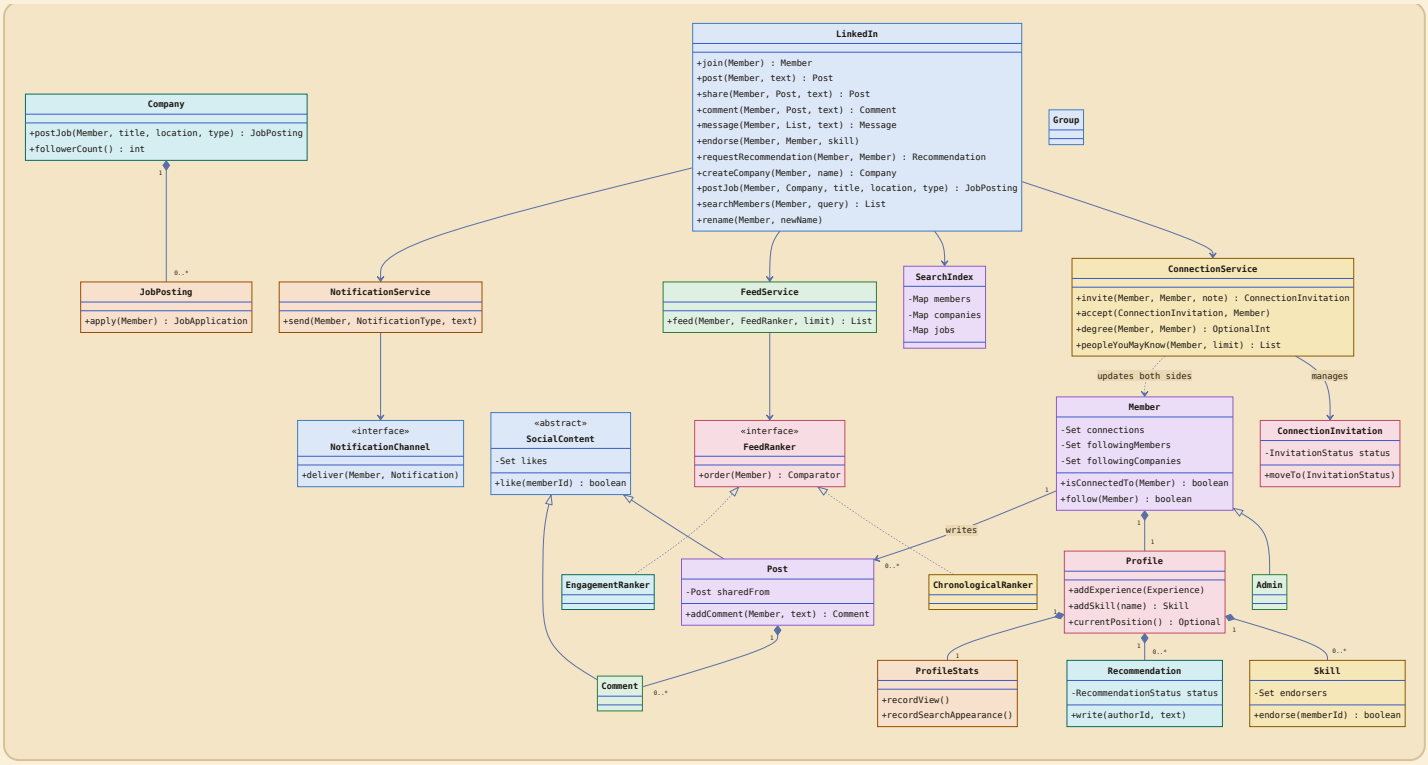


Figure — The finished LinkedIn class diagram, with all six stages in full colour (course diagram)

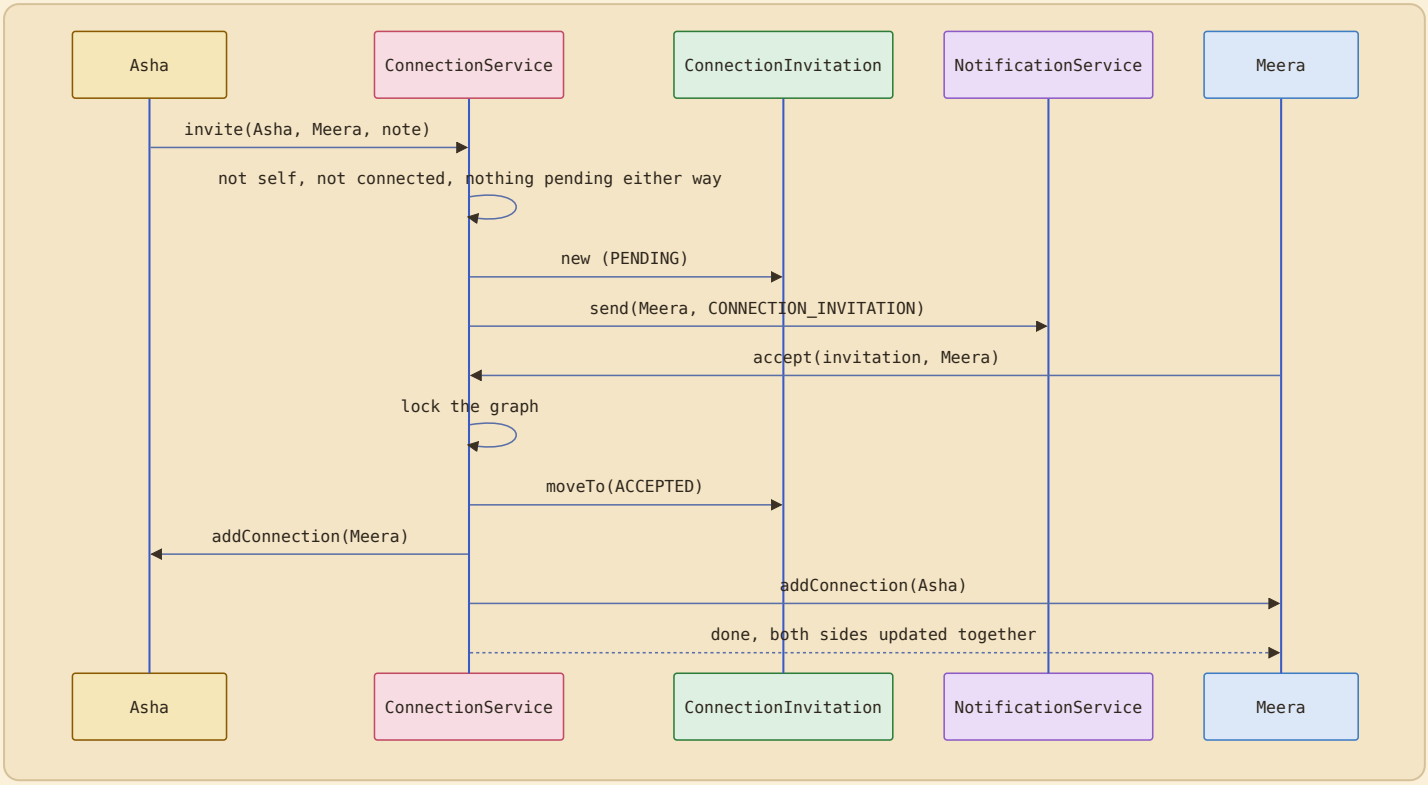
The course deliberately omits several features: the feed and how it is ranked, suggested connections, degrees of separation, applying for jobs and tracking applicants, privacy controls, and one member blocking another. It also admits two weak spots a reviewer will probe. First, there is no feed; building one means either gathering the connections' posts at the moment the page loads, or copying each new post into all the connections' feeds when it is written, and which is better depends on how many connections a typical member has. Second, the search index goes stale (out of date) when a member changes their name, because nothing informs it.

Our Java design follows the six stages and fills the gaps the course names: a **feed** (pull model — the feed is gathered when the member opens it — with a ranking Strategy), **degrees of connection** (breadth-first search, described under Key flows), **people you may know**, a **rename that updates the index**, job **applications**, and connection-count derived from the set instead of a stored counter.

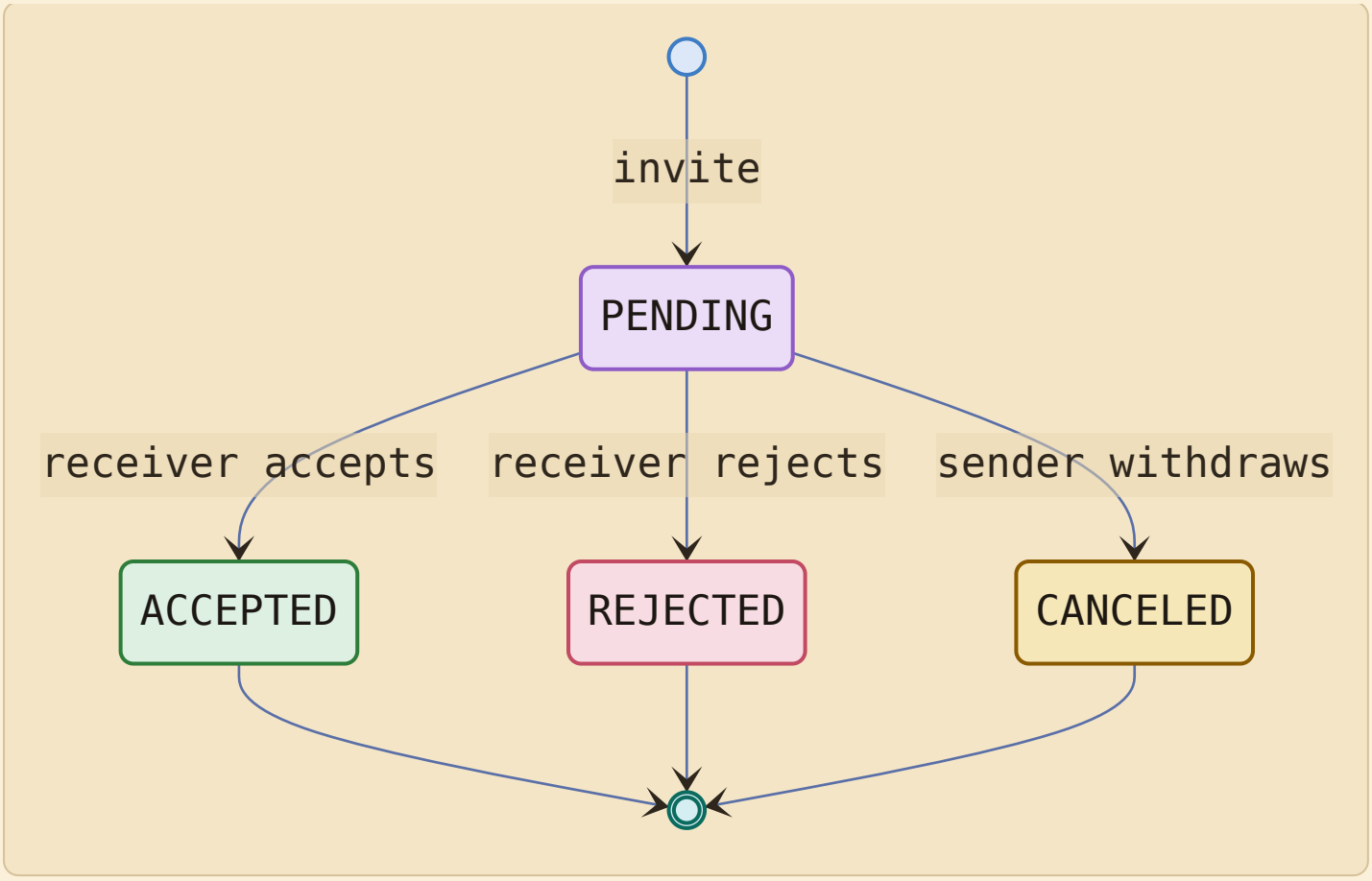


Key flows

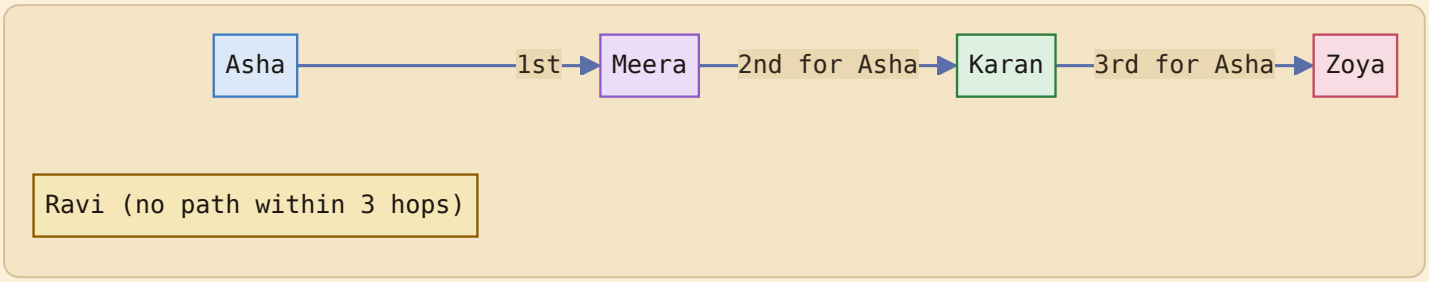
Invite and accept (the two-sided operation):



Invitation states:



Degree of connection — breadth-first search (**BFS**: explore all 1-hop neighbours, then all 2-hop, and so on) from the viewer, stopping at depth 3:



Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
`ConnectionService.accept` updates both members under one lock	Service for a two-object operation; Information Expert at the right level	Symmetry has one guardian	`member.accept(inv)` editing the other member
Following is a set on the follower	Simplicity	One-sided relationship needs no agreement or service	Reusing invitations for follows
`ConnectionInvitation` with explicit states	**State** (as an enum with allowed transitions)	Accept-twice and accept-after-reject are rejected	A boolean `accepted` flag
`SocialContent` base for `Post` and `Comment`	Inheritance for shared behaviour	Requirement 7: comments are liked too	Likes only on `Post`
One `Notification` class with a `type` field	"Field when only data differs"	Delivery is identical	A subclass per event
`NotificationChannel` interface	**Strategy**	In-app, email, push are interchangeable	`if (email) … else if (push) …`
`FeedRanker` interface	**Strategy**	"Recent" vs "Top" ranking swap at run time	Sorting code inside the feed service
Index maps key → set	Robust inverted index	Two people named Sharma both found	Key → single member (the course's bug)
`LinkedIn.rename` tells the index	Keep derived structures in sync at the write point	Search is never stale	Rebuilding the index nightly
Connection count = `connections().size()`	Derive, don't store	Cannot disagree with the set	A separate counter bumped on accept

Java implementation

Package `lld.ch18.linkedin`. Full runnable code is in `code/lld/ch18/linkedin/`: `Profile.java` (`Tick` — a deterministic logical clock so output repeats; a real system uses `Instant.now()` — `Experience`, `Education`, `Accomplishment`, `Skill`, `Recommendation`, `ProfileStats`, `Profile`), `Content.java` (`SocialContent`, `Post`, `Comment`, `Message`), `Network.java` (`Member`, `Admin`, `ConnectionInvitation`, `ConnectionService`), `Pages.java` (`Company`, `JobPosting`, `JobApplication`, `Group`), `Services.java` (notifications, `SearchIndex`, feed ranking and `FeedService`), `LinkedIn.java` (the facade) and `Main.java`. The essential lines (`/* ... */` and `//` summary comments stand for code left out):

```java
/** A skill keeps the set of members who endorsed it, so nobody endorses twice. */
final class Skill { synchronized boolean endorse(long memberId) { return endorsers.add(memberId); } }

/** Created EMPTY when requested; only the named author may fill it in, once. */
final class Recommendation {
    synchronized void write(long byMemberId, String text) {
        if (byMemberId != authorId) throw new IllegalStateException("only " + authorName + " can write this
recommendation");
        if (status != RecommendationStatus.REQUESTED) throw new IllegalStateException("recommendation is " +
status);
        this.text = text;
        this.status = RecommendationStatus.WRITTEN;
    }
}
// ProfileStats: views and search appearances are pushed in by whoever sees the event (a profile cannot observe
// its own views); the connection count is NOT stored — it is the size of the connection set.
// Profile owns all its parts (composition) and rejects an experience that ends before it starts.

/** Posts AND comments can be liked, so likes live in a shared base class, as a set of member ids. */
abstract class SocialContent {
    private final Set<Long> likes = new LinkedHashSet<>();
    synchronized boolean like(long memberId) { return likes.add(memberId); }
    synchronized boolean unlike(long memberId) { return likes.remove(memberId); }
    synchronized int likeCount() { return likes.size(); }
}
// Post adds comments, sharedFrom (null for an original) and a share counter.
/** Sent to people, never posted; nobody likes a message, so it is not SocialContent. */
record Message(long id, Member sender, List<Member> recipients, String text, long sentAt) {}
```

```java
class Member {
    // Connections change only through ConnectionService, which keeps both sides symmetric.
    private final Set<Long> connections = ConcurrentHashMap.newKeySet();
    // Following is one-sided: nobody has to agree, so it is just a set on the follower.
    private final Set<Long> followingMembers = ConcurrentHashMap.newKeySet();
    void addConnection(long otherId) { connections.add(otherId); }         // package-private: only the service
calls it
}

enum InvitationStatus {
    PENDING, ACCEPTED, REJECTED, CANCELED;
    boolean canMoveTo(InvitationStatus next) { return this == PENDING && next != PENDING; }
}

/** Accepting an invitation changes TWO members, so it belongs to neither: it lives here. */
final class ConnectionService {
    synchronized ConnectionInvitation invite(Member from, Member to, String note) {
        if (from.id() == to.id()) throw new IllegalArgumentException("cannot connect to yourself");
        if (from.isConnectedTo(to)) throw new IllegalStateException(from + " and " + to + " are already
connected");
        if (pendingBetween(from, to)) throw new IllegalStateException("an invitation between " + from + " and " +
to + " is already pending");
        /* ... store it, notify the invitee ... */
    }

    /** One lock for the whole graph change: both sides are updated together or not at all. */
    synchronized void accept(ConnectionInvitation inv, Member by) {
        if (by.id() != inv.to().id()) throw new IllegalStateException("only " + inv.to() + " can accept");
        inv.moveTo(InvitationStatus.ACCEPTED);
        inv.from().addConnection(inv.to().id());
        inv.to().addConnection(inv.from().id());
    }
    // reject (invitee only), withdraw (sender only -> CANCELED), removeConnection (both sides)

    /** Degree of connection (1st, 2nd, 3rd) by breadth-first search; empty means "out of network". */
    OptionalInt degree(Member from, Member to) {
        if (from.id() == to.id()) return OptionalInt.of(0);
        Set<Long> seen = new HashSet<>(Set.of(from.id()));
        Deque<long[]> queue = new ArrayDeque<>();                // {memberId, depth}
        queue.add(new long[]{from.id(), 0});
        while (!queue.isEmpty()) {
            long[] cur = queue.poll();
            if (cur[1] == 3) continue;                             // LinkedIn stops at 3rd degree
            for (long next : lookup.apply(cur[0]).connections()) {
                if (next == to.id()) return OptionalInt.of((int) cur[1] + 1);
                if (seen.add(next)) queue.add(new long[]{next, cur[1] + 1});
            }
        }
        return OptionalInt.empty();
    }
    /** "People you may know": 2nd-degree members ranked by mutual connections, then name. */
    List<Member> peopleYouMayKnow(Member me, int limit) { /* count mutuals via friends of friends; skip pending */
}
}
```

```
/** Inverted indexes. RULE: every key maps to a collection, even for one match, so the second "Sharma" never
breaks it. */
final class SearchIndex implements Search {
    private final Map<String, Set<Member>> members = new ConcurrentHashMap<>();   // also companies, jobs (open
only)
    /** Items matching ALL words of the query. */
    private static <T> Stream<T> match(Map<String, Set<T>> index, String query) { /* ... */ }
}

/** Strategy for ordering the feed: ChronologicalRanker ("latest") or EngagementRanker ("top"). */
interface FeedRanker { String name(); Comparator<Post> order(Member viewer); }

/** Engagement score with a simple age decay; 1st-degree connections get a boost over follows. */
final class EngagementRanker implements FeedRanker {
    private static double score(Post p, Member viewer) {
        double engagement = p.likeCount() + 2.0 * p.comments().size() + 3.0 * p.shares();
        double affinity = viewer.isConnectedTo(p.author()) ? 1.5 : 1.0;
        double age = Tick.now() - p.createdAt();
        return (1 + engagement) * affinity / (1 + age / 20.0);
    }
}

/** Pull model ("fan-out on read"): build the feed when it is opened. */
final class FeedService {
    List<Post> feed(Member viewer, FeedRanker ranker, int limit) {
        Set<Long> sources = new java.util.HashSet<>(viewer.connections());
        sources.addAll(viewer.followingMembers());
        return sources.stream().map(lookup::apply)
                .filter(m -> m.status() == AccountStatus.ACTIVE)
                .flatMap(m -> m.posts().stream())
                .sorted(ranker.order(viewer)).limit(limit).toList();
    }
}
// NotificationService sends each Notification(type, text, at) through every NotificationChannel (InAppChannel,
EmailChannel).
```

```
/** Facade that wires the services and checks account status before every action. */
final class LinkedIn {
    /** The code that changes a name must tell the index, or search goes stale. */
    void rename(Member m, String newName) {
        String old = m.name();
        search.removeMember(m, old);
        m.rename(newName);
        search.addMember(m);
    }
    void endorse(Member by, Member owner, String skillName) {
        active(by);
        if (!by.isConnectedTo(owner)) throw new IllegalStateException("only 1st-degree connections can endorse " +
owner);
        /* ... the skill must exist; Skill.endorse refuses a second endorsement ... */
    }
    Message message(Member from, List<Member> to, String text) {
        active(from);
        for (Member r : to)
            if (!from.isConnectedTo(r)) throw new IllegalStateException(r + " is not a connection (that would need
InMail)");
        /* ... deliver to every recipient's inbox and notify ... */
    }
    // also: post, share, comment (notifies the author), requestRecommendation (connections only), searchMembers
    // (records search appearances), viewProfile, createCompany, postJob (company admin only), followCompany,
block(Admin, ...)
}
// Company owns its job postings (gone with the page) and counts followers from a set; JobPosting.apply refuses a
// closed job or a second application; a Group owner cannot leave their own group.
```

Output:

```
== 1. Profile ==
  REJECTED: experience ends before it starts
Asha Sharma: Backend engineer | Java | Distributed systems, now at Acme Cloud

== 2. Connections ==
    [email to meera@example.com] CONNECTION_INVITATION: Asha Sharma wants to connect: "We met at the Java meetup"
  REJECTED: invitation is already ACCEPTED
    [email to karan@example.com] CONNECTION_INVITATION: Meera Iyer wants to connect: "Ex-colleague"
    [email to zoya@example.com] CONNECTION_INVITATION: Karan Mehta wants to connect: "Hi"
    [email to ravi@example.com] CONNECTION_INVITATION: Asha Sharma wants to connect: "Same surname!"
  REJECTED: an invitation between Ravi Sharma and Asha Sharma is already pending
  REJECTED: only Ravi Sharma can accept
degree asha->meera=1, asha->karan=2, asha->zoya=3, asha->ravi=out of network
people asha may know: [Karan Mehta]

== 3. Endorsements and recommendations ==
  REJECTED: Meera Iyer already endorsed Java
  REJECTED: only 1st-degree connections can endorse Asha Sharma
    [email to meera@example.com] RECOMMENDATION_REQUEST: Asha Sharma asked you for a recommendation
  REJECTED: only Meera Iyer can write this recommendation
Java endorsements=1, recommendation from Meera Iyer: Asha owns problems end to end. [WRITTEN]

== 4. Posts, comments, likes, feed ==
    [email to meera@example.com] NEW_COMMENT: Asha Sharma commented: "Sharing with my network"
double like accepted? false
feed(asha, latest): ["We are hiring Java engineers at Acme Cloud!" by Meera Iyer]
feed(karan, top):   ["Tip: prefer composition over inheritance." by Asha Sharma, "We are hiring Java engineers at
Acme Cloud!" by Meera Iyer]

== 5. Messaging ==
    [email to meera@example.com] NEW_MESSAGE: from Asha Sharma: Coffee on Friday?
  REJECTED: Karan Mehta is not a connection (that would need InMail)

== 6. Companies, jobs, groups ==
  REJECTED: Asha Sharma is not an admin of Acme Cloud
search jobs 'java engineer': [Senior Java Engineer @ Acme Cloud (Bengaluru, FULL_TIME)]
  REJECTED: Asha Sharma already applied to Senior Java Engineer
  REJECTED: the owner cannot leave their own group
Acme Cloud followers=1, applicants=1, group members=2

== 7. Search and profile stats ==
search 'sharma' (by meera): [Asha Sharma, Ravi Sharma]
search 'khan' after rename: [], search 'ali': [Zoya Ali]
asha stats: views=2, connections=1, search appearances=1
meera's in-app notifications: [CONNECTION_INVITATION, RECOMMENDATION_REQUEST, NEW_COMMENT, NEW_MESSAGE]

== 8. Admin ==
  REJECTED: Karan Mehta is BLOCKED
feed(meera) hides blocked Karan's share: ["Tip: prefer composition over inheritance." by Asha Sharma]
```

Walking through the important parts

- `ConnectionService.invite` refuses a request when one is already pending *in either direction* — in the output Ravi's counter-invite to Asha is rejected.
- `accept` checks that the caller is the receiver, moves the invitation to ACCEPTED (a second accept fails because only PENDING can move), and adds each member to the other's set inside one `synchronized` block.
- `degree` is a BFS over connection sets that stops at depth 3: Asha → Meera is 1st, Karan 2nd, Zoya 3rd, and Ravi (only a pending invitation) is "out of network".
- `peopleYouMayKnow` counts mutual connections for every 2nd-degree member, skips anyone with a pending invitation, and sorts by count then name, so the list is stable.
- `endorse` requires a 1st-degree connection and uses the skill's endorser set to reject a second endorsement.
- `Recommendation.write` accepts only the named author and only once.

- `SearchIndex` maps each lowercase word to a *set*, so "sharma" returns both Asha and Ravi; `rename` removes the old words before adding the new ones, so "khan" finds nobody after Zoya's rename.
- `FeedService` is a **pull** feed: it gathers posts from connections and followed members when asked, drops blocked authors, then sorts with the chosen `FeedRanker`. Karan sees Asha's tip because he *follows* her, even though they are not connected.

Concurrency & edge cases

- **Symmetric connections:** all graph changes (`invite`, `accept`, `reject`, `withdraw`, `removeConnection`) are `synchronized` on the single service, so two concurrent accepts or an accept racing a withdraw cannot leave a one-way edge. In a distributed system (one spread across many machines that talk over a network), write both edges (an **edge** is one link in a graph) in one database transaction (or one row keyed by the ordered pair of ids).
- **Double invitations** in either direction are refused while one is pending.
- **Likes, endorsements, applications:** sets and maps keyed by member id make repeats harmless or rejected; their mutators are `synchronized`.
- **Collections on `Member`** are concurrent sets or `CopyOnWriteArrayList`, and getters return copies, so callers cannot mutate internal state.
- **Stale search:** handled by `rename`; the general rule is "whatever changes the data tells the index" (in production, via an event).
- **Blocked members:** writes are refused and their posts vanish from feeds.
- **Validation:** an experience that ends before it starts is rejected; blank posts are rejected.

Extensions & follow-up questions

- **"Build the feed for 900 million members."** — Two classic strategies. **Fan-out on write (push):** when someone posts, append the post id to each connection's precomputed feed list — fast reads, expensive for members with huge followings. **Fan-out on read (pull):** assemble the feed when it is opened, as our `FeedService` does — cheap writes, slower reads. Large networks use a **hybrid**: push for ordinary members, pull for very popular accounts, merged at read time.
- **"People you may know at scale."** — Compute suggestions offline (in a background batch job, not while the user waits) on a schedule (friends-of-friends plus signals such as shared companies and schools) and store them; the live page reads the stored list.
- **"Connection limit."** — LinkedIn caps members at 30,000 connections; enforce in `accept`.
- **"InMail to non-connections."** — A premium credit balance on `Member`; `message` spends a credit instead of rejecting.
- **"Who viewed my profile."** — Store view events (viewer, time) instead of a bare counter, then derive the count.
- **"Job recommendations."** — A Strategy that matches a member's skills and current title against open postings, ranked by overlap.

Real-world example

LinkedIn shows 1st, 2nd and 3rd-degree badges next to names in search results — the exact BFS depth our `degree` computes — and its "People You May Know" panel is driven by shared connections and other signals. Messaging someone who is not a connection usually requires paid **InMail** (some members open their profile to free messages). And the event stream that feeds notifications, analytics and the feed at LinkedIn gave rise to **Apache Kafka**, which LinkedIn open-sourced (published free, with its source code, for anyone to use) in 2011.

In the interview

Minutes	What to do
0–5	Clarify connect vs follow, degrees, who can message and endorse
5–15	Profile composition, `SocialContent` (comments can be liked), invitation as an object with state
15–25	**Say it out loud:** accepting changes two objects, so it belongs to neither — `ConnectionService`; following is one-sided, so it needs no service
25–35	Search index (key → list), degrees by BFS, people you may know
35–45	Feed: pull vs push vs hybrid; notifications via Strategy channels; concurrency

30-second answer: "Members own profiles made of small parts. Posts and comments share a base class because both are liked, with likes as a set. A connection request is an object with states. Accepting changes both members, so it lives in a `ConnectionService` that updates both sides under one lock; following is one-sided, just a set. Search is an inverted index mapping each word to a list. Degrees come from a BFS to depth 3. The feed is a pull model with a pluggable ranking Strategy, and I'd move to a push/pull hybrid at scale."

Follow-ups with answers

- *Why not store `connectionCount` on the profile?* — It is `connections.size()`; a stored copy can drift.
- *How do you compute a 2nd-degree badge quickly for every search result?* — Precompute or cache each member's 1st-degree set; a result is 2nd-degree if its connection set intersects yours.
- *What if the same name appears twice?* — The index maps words to lists (sets), never to a single member.

Self-check

- **Q:** Why is accepting an invitation not a method on `Member`? **A:** It updates two members; a service keeps both sides consistent in one place.
- **Q:** What is the difference between connecting and following? **A:** Connecting is mutual and needs acceptance; following is one-sided and immediate.
- **Q:** Why does `Recommendation` start without text? **A:** Because it is *requested* first; the author fills it in later.
- **Q:** What is the rule for search index values? **A:** Always a collection, even for one match.
- **Q:** Pull vs push feed — which does `FeedService` implement, and what is its cost? **A:** Pull; cheap writes, but reading scans every connection's posts each time.

TL;DR

LinkedIn = profile composition + `SocialContent` (likes as sets) + invitation **State** + `ConnectionService` for the two-sided accept (following is just a set) + BFS degrees + people-you-may-know + inverted-index search (key → list) + notification and feed-ranking **Strategies** + a facade.

18.4 Design Facebook



Figure — A browser showing the Facebook home page, the cover image of this case study (course diagram)

Facebook is an online social network where users connect with **friends** (a mutual relationship), **follow** people and **pages**, join **groups**, and read and write posts through a website or mobile apps. Compared with LinkedIn, the new challenges are **privacy** (who may see each post), a real **news feed**, **pages** with ratings, **groups** (public and private), and **friend suggestions**.

Problem & clarifying questions

Question to ask	Answer we assume
Friend vs follow?	Friendship is mutual (request + accept); following is one-sided
Who can see a post?	The author chooses an **audience**: Public, Friends, a custom **privacy list**, or Only me
What is a privacy list?	A named subset of the author's friends (e.g. "Close friends"); a post linked to it is visible only to those friends
Pages?	Created by members; others follow them and add **recommendations** (a 1–5 rating and text)
Groups?	Public (join at once) or private (owner approves); posts inside a private group are visible only to members
Search?	Members, groups and pages by name; posts by a word — respecting privacy
News feed?	Posts from friends, followed members, followed pages and joined groups, filtered by privacy, ranked
Extended requirement?	Suggest people you may know (friends of friends)

Requirements

Functional (the course's twelve plus the extended one, in our words)

1. A member fills in a profile: basic details, jobs, schooling and places lived.
2. Any user can search members, groups or pages by name.
3. Members send friend requests and accept or reject them.
4. Members can follow other members without becoming friends.
5. Members create groups and pages, join groups, and follow pages.

6. Members create posts to share with their friends.
7. Members can comment on posts; posts **and comments** can both be liked or shared.
8. Members create privacy lists of friends; a post linked to a list is visible only to that list.
9. Members send messages to other members.
10. Members add a recommendation to any page.
11. A member is notified about each new message, each friend request, and each comment on one of their posts.
12. Members can search posts for a word.
13. **Extended:** write a function that suggests connections for a member.

Non-functional

- **Privacy is never leaked** — through the feed, search, comments, likes or shares.
- **Feed latency** — the feed must load fast even for members with hundreds of friends.
- **Consistency of the friend graph** — friendship is symmetric.
- **Stable output** — suggestions and feeds must not reshuffle randomly between page loads.

Use cases

Three actors: **Member**, **Admin** (blocks/unblocks members) and **System** (sends notifications).

Top use cases: add/update profile; search members, groups, pages and posts; send friend requests; follow/unfollow a member or page; send messages to friends; create posts, like and comment on visible posts; send notifications; create a page; recommend a page.

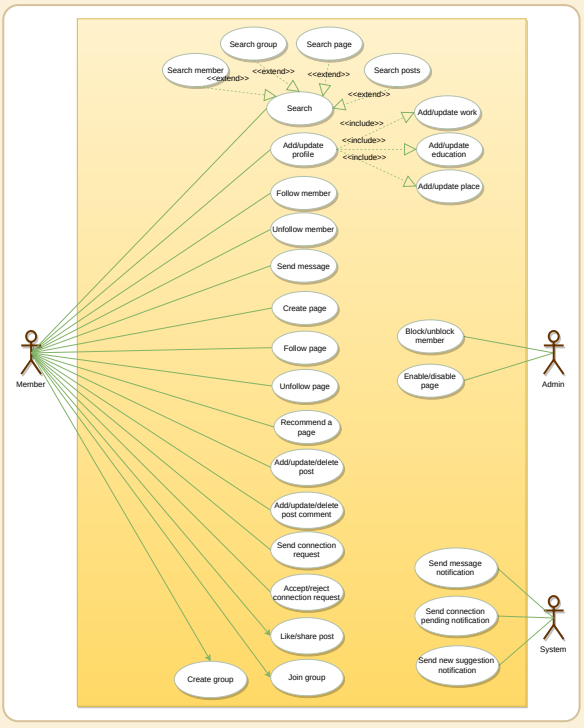


Figure — Facebook use cases for Member, Admin and System, including "Create page", "Recommend a page" and "Unfollow member" (course diagram)

Activity — add work experience. The member opens *Add Work*, enters position, company, location, dates and description, and presses save; the system verifies, asks for missing information if needed, then saves.

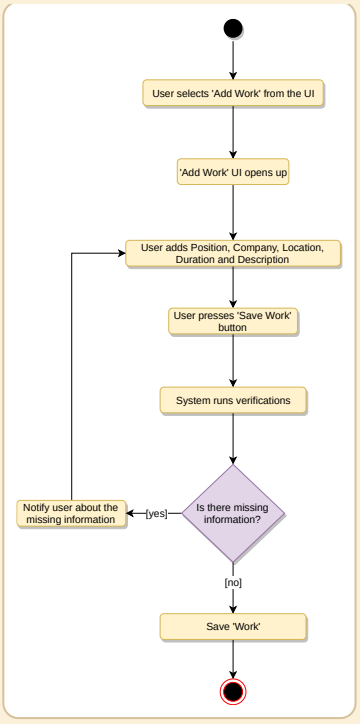


Figure — Activity diagram "add work to profile" with a verification loop (course diagram)

Activity — create a post. The member opens *New Post*, writes the text, optionally uploads a photo or video, **chooses the privacy list** that may see it, and presses publish. After verification the post is published to the chosen audience, who receive an email or push notification.

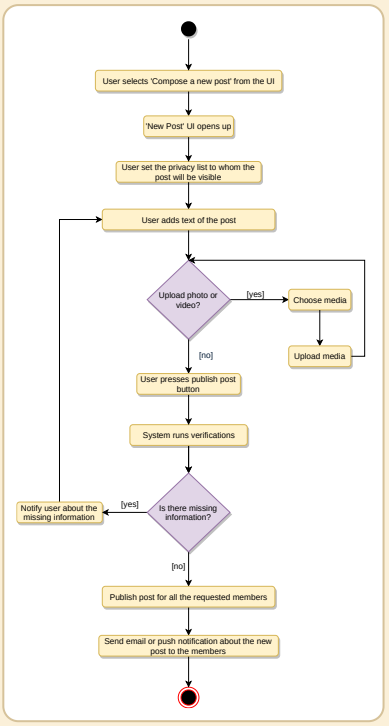


Figure — Activity diagram "create a new post": add text, optional media, set the privacy list, verify, publish, notify (course diagram)

Core entities

Class	Responsibility
`Member`, `Admin`	Identity, profile, follows (members and pages), groups, posts, privacy lists, inbox, notifications (the course's messages may carry photos and videos; ours are text only)
`Profile` (+ `Work`, `Education`, `Place`)	Defaults for every field, lists of parts
`PrivacyList`	Owner + named set of friend ids
`Audience` (sealed) → `Public`, `Friends`, `Custom(list)`, `OnlyMe`	Who may see a post
`SocialContent` → `Post`, `Comment`	Likes as a set; posts own comments; a post may belong to a group or page or be a share
`FriendRequest` + `FriendService`	Requests with both ends, the friend graph, `areFriends`, `pendingBetween` (both ways)
`VisibilityService`	The single `canSee(viewer, post)` rule, combining the post's privacy list and the friend graph
`Page` (+ `PageType`, `PageRecommendation`)	Followers (counted), one recommendation per member, average rating, page posts
`Group` (+ `GroupPrivacy`)	Members, join requests, owner approval, group posts
`NotificationService` + `PushChannel` / `EmailChannel`	Routes each notification type to channels
`SearchIndex`	Names of members/groups/pages; post words — `searchPosts` takes the viewer
`NewsFeedService` + `FeedRanker`	Gathers candidates, filters by `canSee`, ranks
`SuggestionService` + `Suggestion`	Friends-of-friends by mutual count
`Facebook`	Facade

Class diagram

The course builds Facebook in **five stages plus the extended requirement**.

Stage 1 — the profile (`Profile` holding lists of `Work`, `Education`, `Place`). Work, education and places are small classes held in lists (composition). Every profile field starts with a **default** value, so a brand-new account already has a working profile, and no other code needs to make up a placeholder picture for someone who never uploaded one.

Stage 2 — posts, privacy, messages and notifications. Three choices here repair bugs in older versions. (1) **Store the likers, not a like count** — an old `Post` received its like count through the constructor, so a fresh post could start with a thousand likes, and there was no way to know whether you had liked it already. (2) **Comments get likes too**, so the likes set lives in the shared `SocialContent` parent. (3) **Privacy is checked by the post itself** — `isAllowedByPrivacy` is a method on the post, not something each screen must remember. Yet it can answer only part of the question, because a post has no idea who is friends with whom; stage 5 supplies the rest. The link from `Post` to `PrivacyList` is marked `0..1`: a post might have no list (it is public), and one list is reused by many posts and survives them. `Notification` is abstract, with a subclass for each delivery method.

Stage 3 — pages and groups (`Page` owning its `Recommendation`s, followers as ids; `Group`). Look at what these classes do *not* have: a member-count field. An earlier version accepted the count in its constructor *and* also stored the members, so the two numbers could be different from day one. The rule:

if a number can be computed by counting a collection, do not keep it as a separate field; keep the collection and count it when asked.

Stage 4 — the actors and the friend graph (`Account`, `Person`, `Member`, `Admin`, `ConnectionInvitation`, `FriendService`). An earlier invitation class remembered only the person invited, not the person inviting, so there was no way to accept it properly. Since acceptance updates two members, it belongs in `FriendService`; that service therefore holds the **friend graph** (who is friends with whom) and is the obvious class to ask "are these two friends?" and "is a request already waiting between them?". Following (requirement 4) is deliberately different: one-sided, a set on the follower, no service.

Stage 5 — search (`Search`, `SearchIndex` holding a `FriendService`). The most important detail is small: **`searchPost` needs the viewer as a parameter.** The privacy rule (requirement 8) still applies when someone searches; a keyword search that skips it would let anyone read posts that were never shared with them. Deciding visibility therefore needs two sources — the post's own privacy list, and the friendship data that only the friend service holds — and neither source can answer for the other. The index follows the same rule as LinkedIn's: each key maps to a collection.

Extended requirement — friend suggestions (`SuggestionService` reads the friend graph and produces `Suggestion` objects). Friends of friends, **scored by mutual friends** (someone reached through three of your friends beats someone reached through one). Never suggest yourself, an existing friend, or anyone with a request pending **in either direction** (the one candidates forget — showing someone you invited yesterday is a visible bug). Sort by mutual count, then by id, so equal scores do not reshuffle between page loads. Notice that the feature arrives as two new classes with no edits to existing ones; if a design must be modified every time a new question is asked, it will keep being modified.

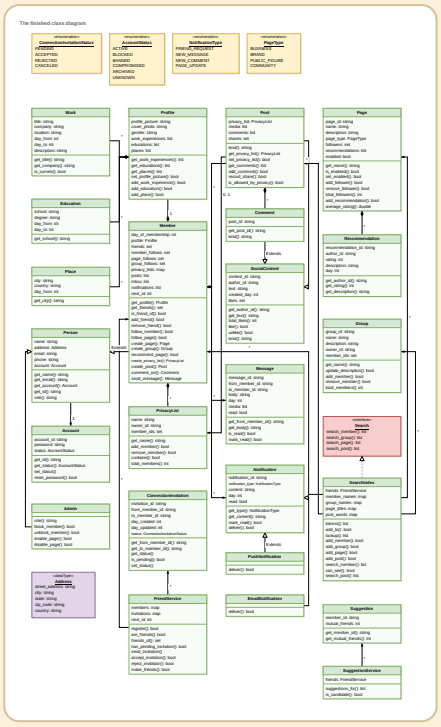
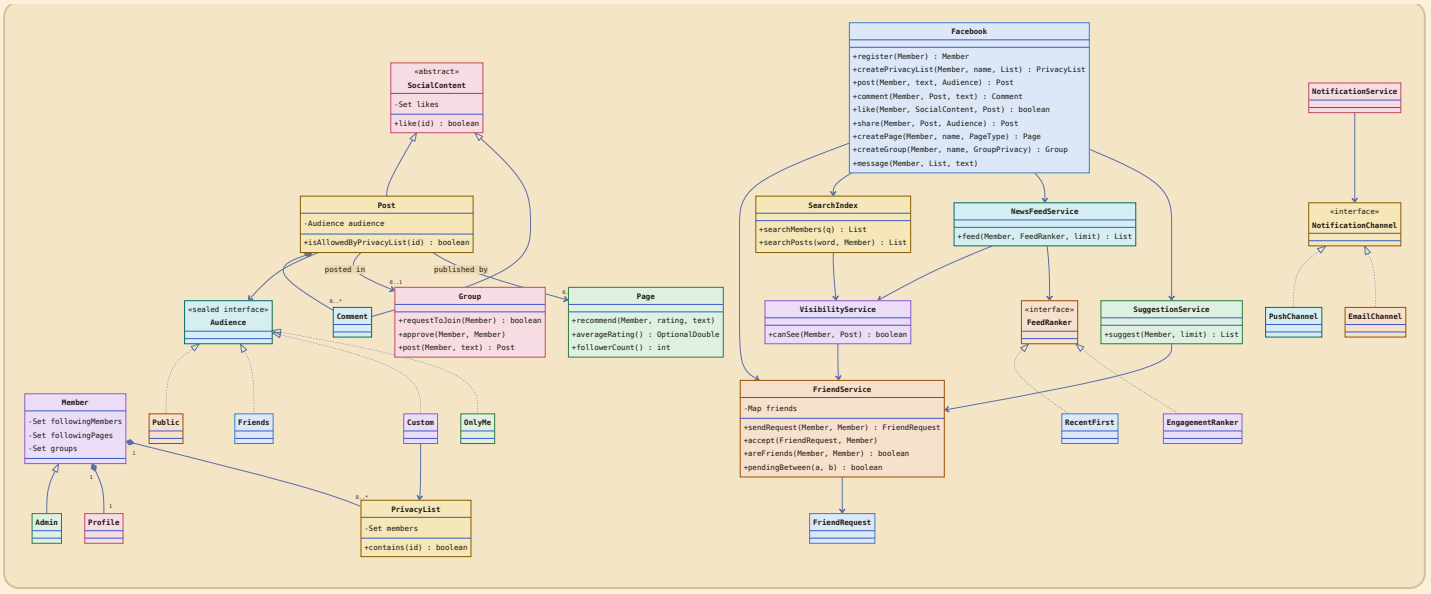


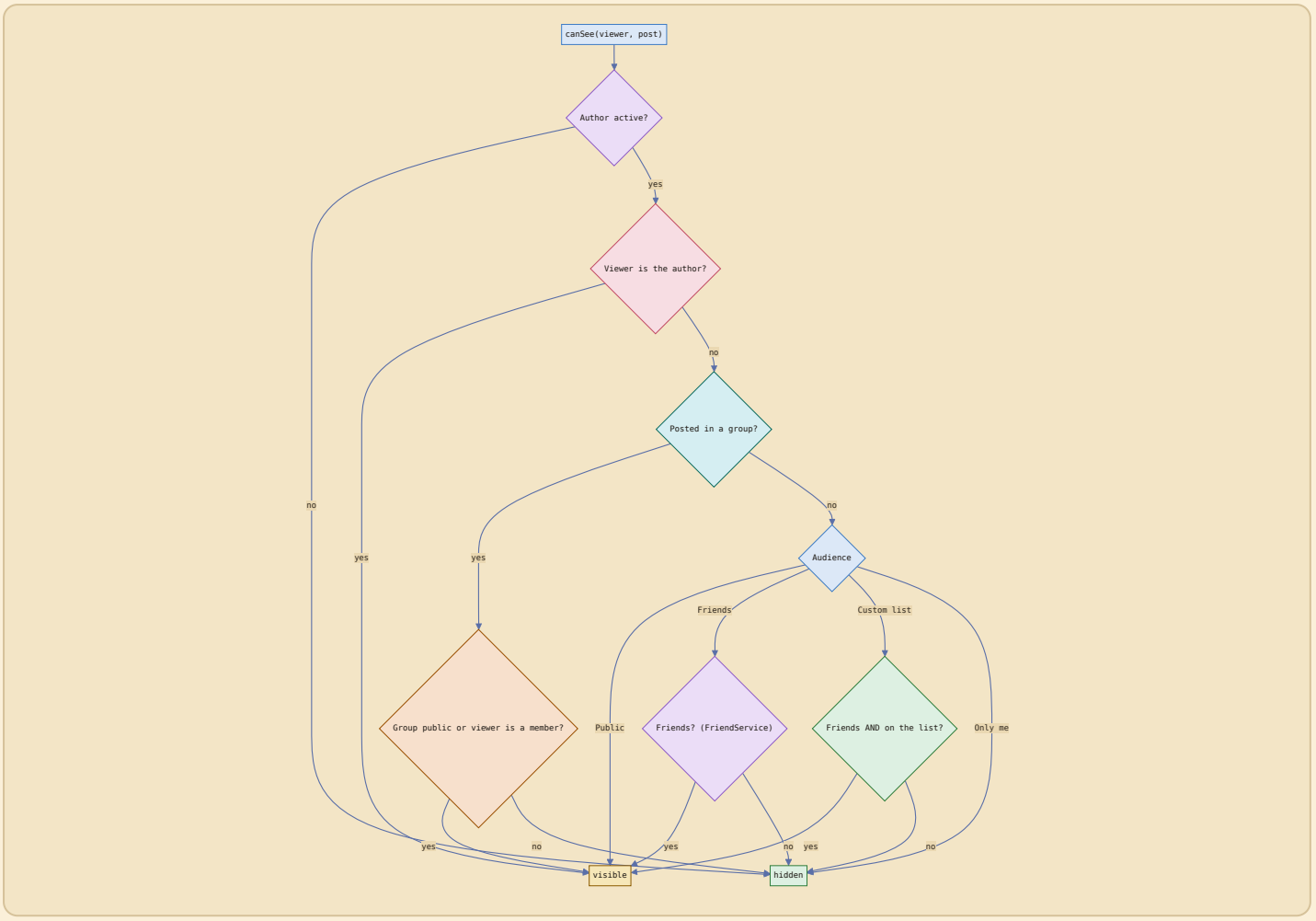
Figure — The finished Facebook class diagram with every class in full colour (course diagram)

Our Java design keeps the course's structure and makes three things explicit: a sealed `Audience` type (Public / Friends / Custom list / Only me), a single **`VisibilityService`** used by the feed, search, comments, likes and shares, and a **`NewsFeedService`** (the course leaves the feed out) with ranking Strategies. Private groups with owner approval and page posts in the feed are added too. The course lists what it leaves out on purpose: the news feed and its ranking, one member blocking another, reactions other than a plain like, tagging people, events, marketplace, stories, and photo albums that carry separate privacy settings. We add the feed; the rest are sketched under Extensions.

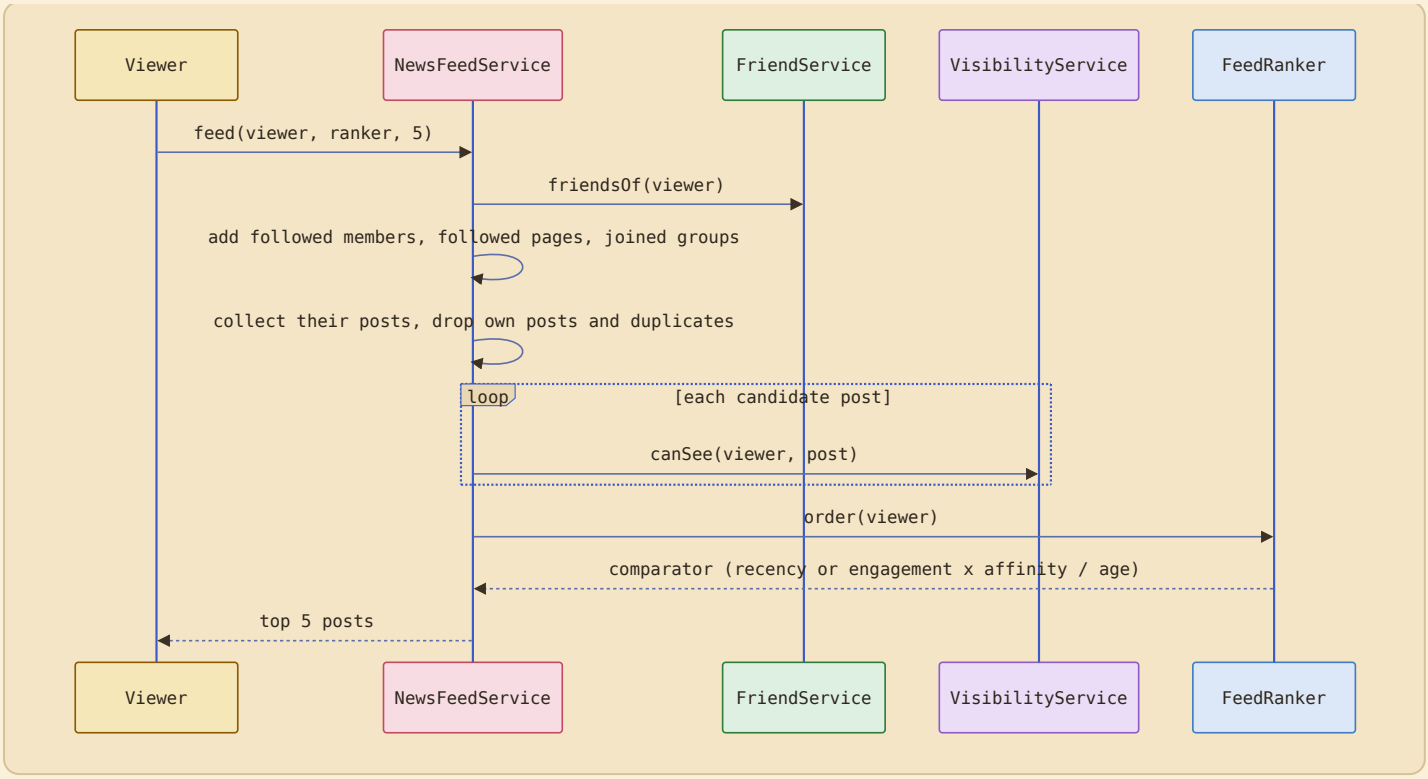


Key flows

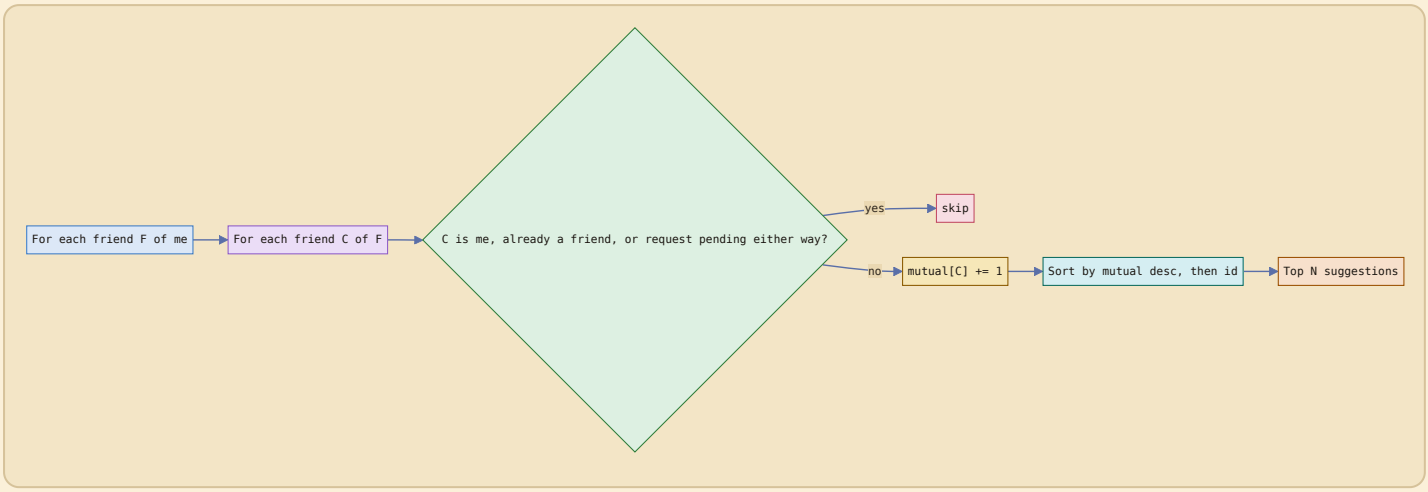
Can this viewer see this post? (used by feed, search, comment, like and share):



Building a news feed (pull model):



Friend suggestions:



Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
`VisibilityService.canSee` used everywhere	Single point of policy (a **Specification**-like rule object — one object that answers a yes/no business question)	The privacy rule is written once; search and feed cannot leak	Checking privacy in each screen
`Audience` sealed interface of records	Algebraic data type + exhaustive `switch`	Adding an audience forces every decision point to handle it	Integer or string codes
`Post.isAllowedByPrivacyList` + `FriendService.areFriends`	Each object answers what it knows (Information Expert)	Neither half pretends to know the other's data	Post holding a reference to the whole friend graph
`FriendService` owns the graph	Service for two-sided operations	Symmetric friendships; one place for "pending either way"	`member.acceptFriend()`
Likes as a set	"Store who, not how many"	No fake like counts, no double likes	`likeCount` field
No stored member/follower counts	Derive, don't store	Counts cannot disagree with lists	Constructor-supplied counts (the course's bug)
`NotificationService` routes types to channels	**Strategy** + configuration map	Friend requests go by push *and* email, comments by push only	`if/else` on type inside each domain method
`FeedRanker`	**Strategy**	"Most recent" vs ranked feed swap at run time	Ranking code inside the feed
`SuggestionService` added without editing others	Open/Closed Principle	The extended requirement is two new classes	Adding suggestion fields to `Member`
`Facebook` facade	**Facade**	One entry point, account-status checks	UI talking to every service

Java implementation

Package `lld.ch18.facebook`. Full runnable code is in `code/lld/ch18/facebook/`: `Model.java` (`Tick`, `Work`, `Education`, `Place`, `Profile` — every field has a default, so a profile is usable the moment the account exists — `Member`, `Admin`, `PrivacyList`, `Audience`, `SocialContent`, `Post`, `Comment`, `Message`), `PagesAndGroups.java`, `Services.java` (friends, privacy, notifications, search, news feed, suggestions), `Facebook.java` (the facade) and `Main.java`. The essential lines (`/* ... */` and `//` summary comments stand for code left out):

```java
/** A named subset of the owner's friends, e.g. "Close friends". Outlives any single post. */
final class PrivacyList { boolean contains(long memberId) { return members.contains(memberId); } }

/** Who may see a post. A sealed type makes the visibility switch exhaustive. */
sealed interface Audience {
    record Public() implements Audience {}
    record Friends() implements Audience {}
    record Custom(PrivacyList list) implements Audience {}
    record OnlyMe() implements Audience {}
}

final class Post extends SocialContent {          // SocialContent: likes as a set — no double likes, can be undone
    private final Audience audience;
    private final Group group;          // null unless posted in a group
    private final Page page;            // null unless published by a page
    /** The half of the privacy rule the post CAN answer on its own. */
    boolean isAllowedByPrivacyList(long viewerId) {
        return !(audience instanceof Audience.Custom c) || c.list().contains(viewerId);
    }
}
```

```java
/** Records BOTH ends; an invitation that only knows the invitee cannot be accepted. */
final class FriendRequest { /* from, to, PENDING -> ACCEPTED / REJECTED / CANCELED, only from PENDING */ }

/** Owns the friend graph. Accepting changes two members, so it lives here, not on either member. */
final class FriendService {
    private final Map<Long, Set<Long>> friends = new ConcurrentHashMap<>();
    synchronized void accept(FriendRequest r, Member by) {
        if (by.id() != r.to().id()) throw new IllegalStateException("only " + r.to() + " can accept");
        r.moveTo(RequestStatus.ACCEPTED);
        friends.computeIfAbsent(r.from().id(), k -> ConcurrentHashMap.newKeySet()).add(r.to().id());
        friends.computeIfAbsent(r.to().id(), k -> ConcurrentHashMap.newKeySet()).add(r.from().id());
    }
    /** Looks BOTH ways: a request from a to b or from b to a. */
    boolean pendingBetween(long a, long b) { /* ... */ }
    // sendRequest rejects yourself, existing friends and a pending request either way; reject; unfriend (both
sides)
}

/** The ONE place that decides "can this viewer see this post?". Feed, search and profile pages all call it. */
final class VisibilityService {
    boolean canSee(Member viewer, Post post) {
        Member author = post.author();
        if (author.status() != AccountStatus.ACTIVE) return false;
        if (viewer.id() == author.id()) return true;
        if (post.group().isPresent()) {
            Group g = post.group().get();
            return g.isPublic() || g.isMember(viewer.id());
        }
        return switch (post.audience()) {
            case Audience.Public p -> true;
            case Audience.Friends f -> friends.areFriends(viewer, author);
            case Audience.Custom c -> friends.areFriends(viewer, author) &&
post.isAllowedByPrivacyList(viewer.id());
            case Audience.OnlyMe o -> false;
        };
    }
}

/** Delivery Strategy; NotificationService routes each NotificationType to its own list of channels. */
interface NotificationChannel { void deliver(Member to, Notification n); }   // PushChannel, EmailChannel

final class SearchIndex {
    /** Takes the viewer: privacy does not stop being true inside a search. */
    List<Post> searchPosts(String word, Member viewer) {
        return find(posts, word).filter(p -> visibility.canSee(viewer, p))
                .sorted(Comparator.comparingLong(Post::createdAt).reversed()).toList();
    }
}
```

```
/** Pull model: gather candidates, drop what the viewer may not see, rank (FeedRanker Strategy), cut to size. */
final class NewsFeedService {
    List<Post> feed(Member viewer, FeedRanker ranker, int limit) {
        Set<Long> people = new HashSet<>(friends.friendsOf(viewer.id()));
        people.addAll(viewer.followingMembers());
        Stream<Post> fromPeople = people.stream().map(members::apply).flatMap(m -> m.posts().stream());
        Stream<Post> fromPages = viewer.followingPages().stream().map(pages::apply).flatMap(pg ->
pg.posts().stream());
        Stream<Post> fromGroups = viewer.groups().stream().map(groups::apply).flatMap(g -> g.posts().stream());
        return Stream.of(fromPeople, fromPages, fromGroups).flatMap(s -> s)
                .distinct()
                .filter(p -> p.author().id() != viewer.id())
                .filter(p -> visibility.canSee(viewer, p))
                .sorted(ranker.order(viewer))
                .limit(limit)
                .toList();
    }
}
// RecentFirst; EngagementRanker = (1 + likes + 2 x comments) x (2.0 if friends else 1.0) / (1 + age / 10).

/** Friends of friends, scored by mutual friends; never yourself, a friend, or anyone with a pending request. */
final class SuggestionService { List<Suggestion> suggest(Member me, int limit) { /* ties broken by member id */ }
}
```

```
/** One recommendation per member per page, with a 1-5 rating. */
record PageRecommendation(long memberId, int rating, String text) { /* rejects a rating outside 1..5 */ }
final class Page {
    int followerCount() { return followers.size(); }            // never store what you can count
    OptionalDouble averageRating() { return
recommendations.values().stream().mapToInt(PageRecommendation::rating).average(); }
    Post publish(Member by, String text) { /* owner only; posts are Public */ }
}
final class Group {
    /** Public groups: join at once. Private groups: the request waits for the owner. Returns true if joined now.
*/
    boolean requestToJoin(Member m) { /* ... */ }
    // approve (owner only), post (members only)
}

/** Facade: wires the services and checks account status on every write. */
final class Facebook {
    PrivacyList createPrivacyList(Member owner, String name, List<Member> people) { /* every person must be a
friend */ }
    Post post(Member author, String text, Audience audience) {
        active(author);
        if (audience instanceof Audience.Custom c && c.list().owner() != author)
            throw new IllegalArgumentException("you can only use your own privacy lists");
        /* ... add to author's posts and the search index ... */
    }
    // share, comment and like first check visibility.canSee(by, post); block(Admin, ...); pages and groups are
indexed.
}
```

Output:

```
== 1. Profile and search by name ==
Alice's picture before upload: default-avatar.png
search 'lee': [Frank Lee, Grace Lee]

== 2. Friend requests ==
    [push -> Bob Das] Alice Rao sent you a friend request
    [email -> bob@example.com] Alice Rao sent you a friend request
  REJECTED: request is already ACCEPTED
    [push -> Carol Nair] Alice Rao sent you a friend request
    [email -> carol@example.com] Alice Rao sent you a friend request
    [push -> Erin Shah] Alice Rao sent you a friend request
    [email -> erin@example.com] Alice Rao sent you a friend request
    [push -> Dave Paul] Bob Das sent you a friend request
    [email -> dave@example.com] Bob Das sent you a friend request
    [push -> Dave Paul] Carol Nair sent you a friend request
    [email -> dave@example.com] Carol Nair sent you a friend request
    [push -> Grace Lee] Erin Shah sent you a friend request
    [email -> grace@example.com] Erin Shah sent you a friend request

== 3. Friend suggestions (extended requirement) ==
before: [Dave Paul(2 mutual), Grace Lee(1 mutual)]
    [push -> Grace Lee] Alice Rao sent you a friend request
    [email -> grace@example.com] Alice Rao sent you a friend request
after inviting Grace: [Dave Paul(2 mutual)]
  REJECTED: a request between Grace Lee and Alice Rao is pending

== 4. Posts and privacy ==
  REJECTED: Frank Lee is not a friend of Alice Rao
  Alice Rao  public=true  friends=true  closeFriends=true
  Bob Das    public=true  friends=true  closeFriends=true
  Erin Shah  public=true  friends=true  closeFriends=false
  Frank Lee  public=true  friends=false closeFriends=false
search posts 'party' as Bob:  ["Planning a surprise party for Erin" by Alice Rao]
search posts 'party' as Erin: []

== 5. Comments, likes, shares ==
    [push -> Alice Rao] Bob Das commented on your post
Bob likes: true / again: false
  REJECTED: Erin Shah cannot see that post

== 6. Pages and groups ==
  REJECTED: Alice Rao already recommended Joe's Pizza
Joe's Pizza: followers=1, rating=4.5 from 2 recommendations
Frank joined at once? false
  REJECTED: Alice Rao is not a member of Chess Club
search posts 'chess' as Alice: [], as Dave: ["Chess tournament on Sunday" by Frank Lee [group Chess Club]]

== 7. News feeds ==
Alice (recent): ["Two-for-one pizza on Friday" by Carol Nair [page Joe's Pizza], "shared: New portfolio is live!"
by Erin Shah]
Bob   (ranked): ["Planning a surprise party for Erin" by Alice Rao, "Moving to Pune next month" by Alice Rao, "New
portfolio is live!" by Alice Rao]
Frank (recent): ["New portfolio is live!" by Alice Rao]

== 8. Messages and admin ==
Dave before Frank is blocked: ["Chess tournament on Sunday" by Frank Lee [group Chess Club]]
    [push -> Alice Rao] new message from Bob Das
    [push -> Carol Nair] new message from Bob Das
Dave after Frank is blocked: []
Alice's notifications: [NEW_COMMENT, NEW_MESSAGE]
```

Walking through the important parts

- **The visibility table in section 4** is the course's key scenario, with four viewers. Alice is the author, so she sees all three of her posts. Bob is a friend *and* on the "Close friends" list, so he sees all three posts. Erin is a friend but *not* on the list, so she sees the public and friends posts but not the close-friends one. Frank is not Alice's friend, so only the public post is visible to him. Two rules, both applied, neither written twice.

- **Search respects privacy:** searching "party" returns Alice's close-friends post for Bob and nothing for Erin, because `searchPosts` takes the viewer and calls `canSee`.
- `createPrivacyList` refuses Frank because a privacy list may contain only friends.
- **Comments, likes and shares go through `canSee` too:** Erin cannot comment on the close-friends post. Without this, "comment" would be a side door into private content.
- **Suggestions:** before inviting Grace, Alice sees Dave (2 mutual: Bob and Carol) and Grace (1 mutual: Erin). After Alice invites Grace, Grace disappears (pending), and Grace cannot send a request back while one is pending.
- **Pages:** one recommendation per member (`putIfAbsent`), follower count and average rating computed from the stored data.
- **Private group:** Frank's join waits for Dave's approval; Alice (not a member) cannot post there and cannot find the group's post by search, while Dave can.
- **Feeds:** Alice's feed includes the page she follows and Erin's share; Bob's ranked feed puts the most-engaged, most recent friend posts first; Frank (who only follows Alice) sees only her public post.
- **Blocking:** once Frank is blocked, `canSee` hides his posts, so his group post disappears from Dave's feed.
- **Notification routing:** friend requests go to push *and* email; comments and messages go to push only — configured with a `Map<NotificationType, List<NotificationChannel>>`.

Concurrency & edge cases

- **Friend graph:** `sendRequest`, `accept`, `reject` and `unfriend` are `synchronized` on the service, and adjacency sets (each member's set of directly linked members) are concurrent, so an accept cannot leave a one-way friendship. In a database, a friendship is one row keyed by the ordered pair `(min(id), max(id))`, which makes symmetry automatic.
- **Pending in both directions:** checked by `pendingBetween(a, b)` for both requests and suggestions.
- **Privacy changes after posting:** because visibility is computed at read time, removing someone from a list or unfriending them hides the post immediately. (A push-model feed that copied posts into feeds would need to re-check `canSee` at read time anyway.)
- **Shares:** sharing requires seeing the original; a share of a friends-only post should not expose it to the sharer's audience — a production rule would cap the share's audience at the original's.
- **Deterministic ordering** (the same input always produces the same order)**:** suggestions tie-break by id; feeds tie-break by creation time; our logical `Tick` clock (a counter that goes up by one per event and stands in for real time) makes demo output repeatable.
- **Blocked or deleted authors:** `canSee` returns false for inactive authors.

Extensions & follow-up questions

- **"Scale the news feed."** — **Fan-out on write**: when a member posts, push the post id into each friend's feed cache (fast reads; costly for accounts with millions of followers). **Fan-out on read**: assemble at open time (our design). Real systems mix them: push for normal users, pull for celebrities, merge at read, and always re-check privacy.
- **"Rank the feed better."** — Add signals (affinity = how often the viewer interacts with the author, content type, recency decay) behind the same `FeedRanker` interface; that is the idea behind Facebook's early **EdgeRank** (affinity × weight × time decay).
- **"Suggestions at scale."** — Precompute friends-of-friends offline on a schedule and store them (the course notes the original diagram had a `memberSuggestions` field for exactly this), adding signals like shared groups, schools and workplaces.
- **"Blocking one member from another."** — Add a per-member block set; `canSee` and `sendRequest` check it both ways.
- **"Reactions beyond like."** — Change the like set to `Map<memberId, Reaction>`, exactly like Stack Overflow's vote map.
- **"Tagging people in posts."** — Add tagged ids to `Post`, notify them, and (a privacy decision) optionally widen the audience to their friends.

- **"Events, stories, marketplace, albums with their own privacy."** — Each is a new content type extending `SocialContent` with its own `Audience`.

Real-world example

Facebook launched **News Feed** in 2006, turning profiles you had to visit into a stream pushed to you — the feature this design adds. Posts carry an audience selector (Public, Friends, specific friends, Only me, custom lists), and a personal profile is capped at **5,000 friends** (beyond that people use followers or pages). The social graph itself is served by **TAO**, a distributed store Facebook built for objects (users, posts) and associations (friendship, likes) — the production-scale version of our `FriendService` plus `SocialContent` likes.

In the interview

Minutes	What to do
0–5	Clarify friend vs follow, the audience options, pages, groups, feed
5–15	Profile, `SocialContent` (likes as set, comments liked too), `PrivacyList`, `Audience`
15–25	`FriendService` owns the graph (accept is two-sided); following is a set
25–35	**The strongest point:** the post holds its list and the friend service holds friendships, so `canSee` must consult both — and search, feed, comments and shares all call it
35–45	Feed (pull, push, hybrid), suggestions (mutual count, exclude pending both ways, stable sort), concurrency

30-second answer: "Posts and comments share a likes set. A post has an audience — public, friends, a custom privacy list, or only me. The post can check its own list, but only the friend service knows friendships, so one `VisibilityService.canSee` asks both, and every read path — feed, search, comment, like, share — calls it. The friend service owns the graph because accepting changes two members; following is one-sided. The feed gathers from friends, follows, pages and groups, filters by `canSee`, and ranks with a Strategy. Suggestions are friends of friends by mutual count, excluding pending requests either way, with a stable tie-break."

Follow-ups with answers

- *How could a private post leak?* — Through any read path that forgets privacy: search, feed, comment threads, shares, notifications. That is why `canSee` is one method used by all of them.
- *Why exclude pending requests in suggestions?* — Re-suggesting someone you already invited is a visible bug; and the check must look both ways.
- *Why the second sort key?* — Without it, people with the same mutual count can swap places between page loads, and a list that reshuffles itself looks broken.

Self-check

- **Q:** Erin is Alice's friend but not on "Close friends". Can she see a close-friends post? **A:** No — she must be a friend *and* on the list.
- **Q:** Why does `searchPosts` take a viewer? **A:** Because privacy applies inside search; otherwise search leaks private posts.
- **Q:** Where are follower counts stored? **A:** Nowhere — they are the size of the follower set.
- **Q:** Which three people are excluded from suggestions? **A:** Yourself, existing friends, and anyone with a pending request in either direction.
- **Q:** Name the two feed-building strategies and one weakness of each. **A:** Push (fan-out on write): expensive for accounts with huge audiences. Pull (fan-out on read): slower reads because it gathers at open time.

TL;DR

Facebook = LinkedIn's structure + **privacy**: a sealed `Audience`, privacy lists of friends, and one `VisibilityService.canSee` (post's list AND friend graph) called by feed, search, comments, likes and shares. `FriendService` owns the symmetric graph; following is a set. Pages and groups store collections, not counts. The news feed gathers, filters, and ranks with a Strategy; suggestions are friends-of-friends by mutual count, excluding pending requests both ways, with a stable tie-break.

18.5 Chapter summary

The four designs side by side

Aspect	Stack Overflow	Cricinfo	LinkedIn	Facebook
Core content	Question / answer / comment	Ball / over / innings / match	Post / comment / message	Post / comment / message / page post / group post
Relationship between users	None (votes and reputation)	Team membership, officials	Connection (mutual) + follow	Friendship (mutual) + follow
Two-sided operation → service	—	—	`ConnectionService.accept`	`FriendService.accept`
Role / permission model	Reputation privileges + Moderator/Admin types	Admin scorer, Commentator	Member / Admin	Member / Admin + audiences
"Derive, don't store"	Score from votes, tag counts	All stats from balls	Connection count, follower count	Member, follower counts, ratings
Strategy	Reputation policy, channels	(Target calculator as extension)	Feed ranker, channels	Feed ranker, channels
Observer	Reputation listeners (badges)	Match observers (live feed)	Notifications	Notifications
Template Method	—	`StatQuery`	—	—
Search	Words and tags	Stat queries	Members, companies, jobs	Members, groups, pages, privacy-aware posts

Common mistakes across all four

- Counters instead of sets (votes, likes, endorsements) — cannot stop duplicates.
- Stored counts next to the collections they summarise — they drift.
- Two-sided operations on one entity — asymmetric graphs.
- Privacy or permission checks in one read path but not another.
- Search index values that are a single object instead of a list.
- Modelling the nouns but not the rules (six balls vs six *legal* balls; run outs; byes).

Chapter TL;DR

Platforms are rich domain models. Share behaviour in a content base class; keep votes and likes as sets of ids; derive every number you can; put operations that change two objects in a service; notify through Observer with Strategy channels; rank feeds with a Strategy; keep search in an inverted index

that maps words to lists; and write each permission or privacy rule exactly once, then call it from every read path.

Chapter 19 — Case Studies: Commerce & Marketplaces (Part 2 of Platforms)

> **Where this fits.** Chapter 18 designed platforms where people *talk* (Q&A sites, social networks, sports scores). This chapter designs platforms where people *buy, sell and trade*. Money moves, stock runs out, prices change every second, and two people often want the same last thing at the same moment. Those three pressures — **money must be exact, state must follow rules, and shared things must not be double-sold** — shape every design here.

What you will learn

#	Problem	The one idea that makes it work	Main patterns
19.1	Amazon — online shopping	An order is an immutable copy of the cart, moved through a **state machine**	State machine (enum), Strategy (payment), Observer (notifications), Facade
19.2	Online stock brokerage + stock-exchange matching engine	Four order types differ in only **two questions**; a book matches by **price, then time**	Template Method, Observer, Strategy (price policy)
19.3	Online auction	"Read highest bid, compare, replace" must be **one atomic step**	Monitor lock, Observer, Specification
19.4	Music streaming (Spotify)	Play order is a **pluggable strategy**; plan rules live **in the plan**	Strategy, Observer, enum-with-behaviour
19.5	Food delivery (Swiggy) + FoodKart	Capacity and riders are **claimed atomically**; selection is a **strategy**	Strategy, Observer, State machine, CAS
19.6	Grocery store (POS & checkout)	A **pure pricing engine**: best offer per product, then a coupon spread over items so tax and refunds stay exact	Strategy (offers, tax, payment), Facade, Observer, enum state machine

How to run every example in this chapter

```
cd chapters/19-case-platforms-2/code
javac --release 21 -d /tmp/lld_19 $(find . -name '*.java')
java -cp /tmp/lld_19 lld.ch19.amazon.Main          # or brokerage, stockexchange, auction, music, fooddelivery, foodkart, grocery
```

`javac` is the Java compiler (it turns `.java` files into runnable `.class` files); `-d` names the output folder, and `java -cp` runs a class, where the **classpath** (`-cp`) is the list of folders Java searches for compiled classes.

All eight `Main` classes compile with Java 21 (standard library only — only the classes that ship with Java) and run. The output under each implementation is the real output of that program. The stock-exchange `Main` also runs a small test suite (a set of automatic checks that confirm the code behaves as expected; no JUnit — the usual Java testing library — needed).

> **Code-style note.** To keep each case readable, one file sometimes holds several small package-private types (for example `Payments.java` holds `Payment`, its two subclasses and `Shipment`). That is legal Java. In production you would usually give each top-level type its own file.

> **Key term — Java vocabulary used in this chapter:** A **package-private** type has no access keyword and is visible only inside its own package (folder of classes). A **top-level type** is a class, interface, enum or record not nested in another. An **interface** lists methods a class promises to provide. An **enum** is a type with a fixed list of named values. A **record** is a compact class that

just carries data that never changes. A **sealed** type lists exactly which subclasses are allowed (the `permits` clause), so the compiler knows every possible kind. `synchronized` means only one thread at a time may run that code on the same object.

Key term — the design patterns named in this chapter: Strategy puts each interchangeable way of doing a job (each payment method, each play order) behind one interface so it can be swapped or added without editing the caller. **Observer** lets objects subscribe to events from another object and be called back automatically, so the publisher does not know who listens. **Facade** is one simple class that hides many classes behind a few easy methods. **Template Method** is a base-class method that fixes the outline of an algorithm and lets subclasses fill in one or two steps. **Specification** turns each business rule or search condition into a small yes/no object that can be combined with others.

Four rules that apply to every design in this chapter

1. **Money is `BigDecimal`, never `double`.** `double` is Java's floating-point number type: it stores numbers in binary, so most decimal fractions are only approximated. `BigDecimal` is Java's exact decimal-number class. A `double` cannot store 0.1 exactly, so `0.1 + 0.2` prints `0.30000000000000004`. With `BigDecimal` you choose the scale (number of decimal places) and the rounding mode yourself. Two traps: build it from a `String` (`new BigDecimal("0.10")`), not from a `double`; and compare with `compareTo`, not `equals` — `new BigDecimal("2.0").equals(new BigDecimal("2.00"))` is `false` because the scales differ.
2. **Status changes follow a state machine.** A **state machine** is a fixed list of states plus the allowed moves between them. We put the allowed moves *inside the enum*, so an illegal move (for example "cancel a delivered order") throws immediately instead of silently corrupting data.
3. **Check-then-act on shared stock must be atomic.** "Is there one left? Then take it" is two steps. If two threads (independent paths of execution running at the same time inside one program) run step 1 together, both see "yes" and both take it — a **race condition**, where the result depends on which thread happens to get there first. The fix is to do both steps under one lock (a guard that only one thread can hold at a time; the others wait), or with one atomic compare-and-set (CAS: "set to B only if it is still A").
4. **Copy prices into orders.** An order remembers the price that was paid, and is **immutable** (it cannot be changed after it is created). If the menu or catalog price changes later, old orders must not change.

Design Amazon — Online Shopping System

Problem & clarifying questions

Amazon started as an online bookshop and now sells almost every kind of consumer product and digital media. We design the **online retail part**: people list products, other people find them, put them in a cart, pay, and track the parcel.

Questions worth asking before you draw anything (with the answers we assume):

Question	Assumed answer
Can someone buy without an account?	No. Browsing and filling a cart are open to everyone; placing an order needs a **registered member** account.
Who sells?	Members can list products for sale (a marketplace, not only first-party — goods sold by the platform itself).
How do people pay?	Credit card or electronic bank transfer. Cards clear at once, transfers clear later.
Can an order be cancelled?	Yes, until it has shipped.
Do we notify people?	Yes, on every order or shipping status change.
Returns, wish lists, seller payouts, recommendations, stock spread over several warehouses?	Out of scope — a deliberate choice, and you should say so out loud.

Requirements

Functional

1. Members can add new products to sell.
2. Anyone can search products by **name** or **category**.
3. Anyone can search and view products; only registered members can buy.
4. Users can add, remove and change the quantity of items in their shopping cart.
5. Users can check out and buy everything in the cart.
6. Members can rate and review a product.
7. Members give a shipping address for delivery.
8. A member can cancel an order that has not shipped.
9. Members are notified each time the status of their order or of its shipment changes.
10. Members pay with a credit card or with an electronic bank transfer.
11. Members can track the shipment and see the current order state.

Non-functional

- **Correctness of stock:** two buyers can never both get the last unit.
- **Exact money:** totals are computed with `BigDecimal`.
- **Auditability** (being able to prove later exactly what happened and when)**:** every order and shipment keeps a history of its states, not just the latest one.
- **Extensibility** (how easily new features can be added without rewriting old code)**:** adding a payment method or a notification channel must not change the checkout code.

Use cases

Four actors:

- **Admin** — manages accounts (block / unblock) and adds or changes product categories.
- **Guest** — searches the catalog, adds and removes cart items, and can register to become a member.
- **Member** — everything a guest can do, plus placing orders and listing products to sell.
- **System** — sends order and shipment notifications.

Top use cases: add or update products (each change also updates the catalog), search products (by name, or by category), put items into the cart or take them out, check out, pay, add a product category, and send shipment notifications.

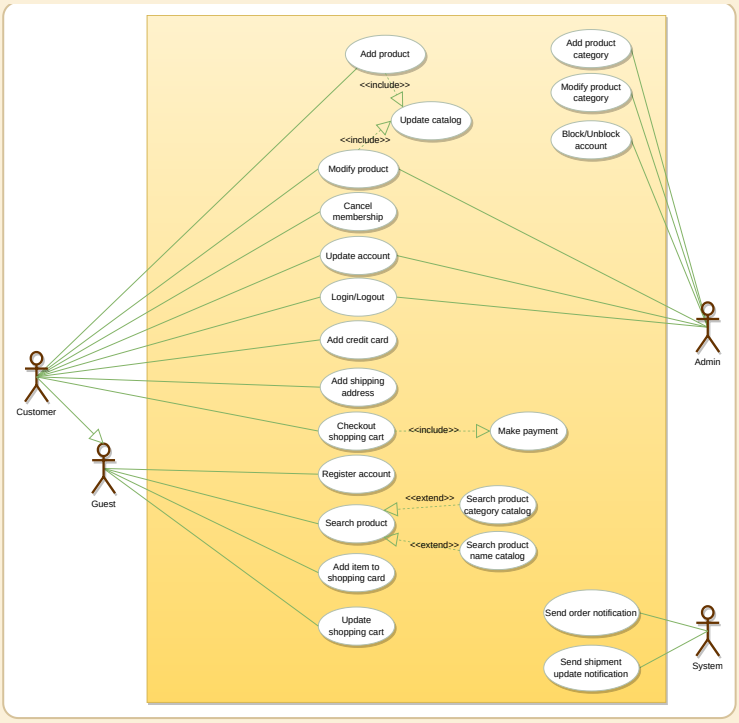


Figure — Use-case diagram for the online shopping system: Admin, Guest, Member and System and what each may do (course diagram)

In the figure above, notice that "Checkout" *includes* "Make payment" and "Add shipping address": you cannot check out without them. Search by name and search by category *extend* the general "Search product" use case.

Core entities

Class	Responsibility
`ProductCategory`	A named group such as Books or Electronics. Exists whether or not any product uses it.
`Product`	The thing sold: price, seller, stock count. **Owns** its reviews and guards its own stock (`reserve` / `release`).
`ProductReview`	Rating 1–5 plus text. Records the reviewer **by id**, so `Product` never depends on `Member`.
`Search` / `Catalog`	Interface with two search methods. The catalog keeps one index (a lookup map built in advance) per search (name words → products, category → products), so a lookup reads a map instead of scanning every product, and `addProduct` updates both indexes together so they cannot drift apart. The catalog only *points to* products; it never owns them.
`ShoppingCart`	Mutable (changeable) working area. **Owns** its items; clearing the cart destroys them.
`Item`	One line: product id, name, quantity, unit price (an immutable `record`). A pen is a *product*; ten pens in stock are ten product *items*.
`Order`	Immutable **copy** of the cart lines + address + a status that moves only along legal edges.
`OrderLog`	One entry per status change (unshipped, pending, complete, cancelled...).
`Payment` (`CreditCardPayment`, `BankTransferPayment`)	Takes the money. A card clears immediately; a bank transfer comes back `PENDING` from `process()` and clears later.
`Shipment` / `ShipmentLog`	A parcel and its tracking history (pending, shipped, delivered, on hold). The last log entry is "where is it now".
`Notification` channel (`EmailChannel`, `SmsChannel`)	One way of delivering a message. (The course draws an abstract `Notification` class with one subclass per channel; we use an interface — same idea.)
`Account`	Login data, addresses, status (active, blocked, banned...).
`Customer` → `Guest`, `Member`	`Customer` has only what *anyone* may do; ordering lives on `Member`.
`Admin`	Adds categories, blocks and unblocks members.
`OrderService`	Application service (a class that coordinates domain objects to carry out one user action) that turns a cart into an order and drives its state machine.

Class diagram

The course builds the model in five stages (products → catalog → payment/shipment/notification → cart/order → accounts). Here is the finished picture:

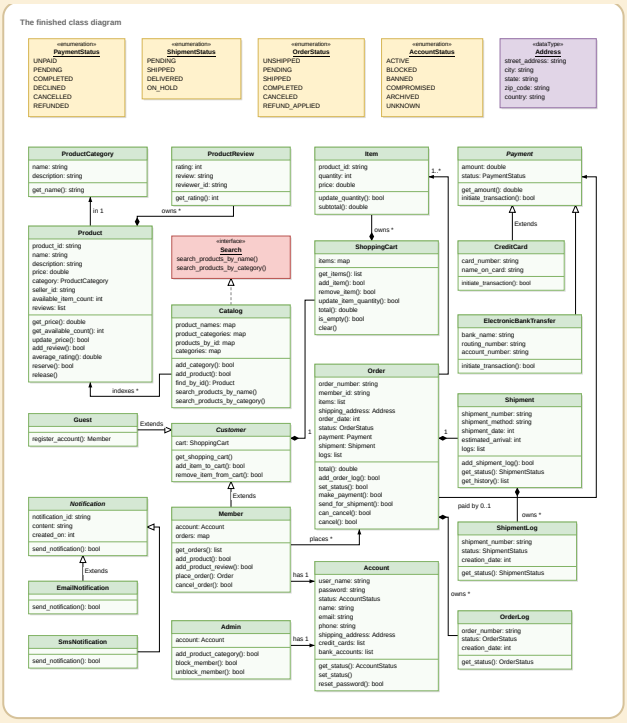


Figure — The finished class diagram of the online shopping system (course diagram)

In the figure above, read the arrows this way: a **filled diamond** means *composition* (the part belongs to the whole and is destroyed along with it — `Product` owns its reviews, `ShoppingCart` owns its items, `Order` owns its log and shipment). A **plain arrow** is *association* (a link without ownership — a product points at its category; an order only *refers to* the payment it was handed). The hollow triangle is inheritance (`Guest` and `Member` extend `Customer`).

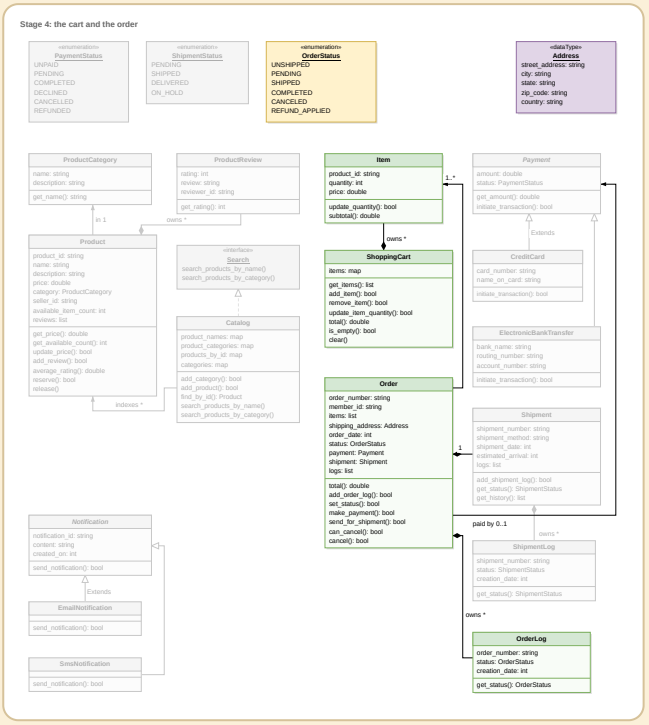
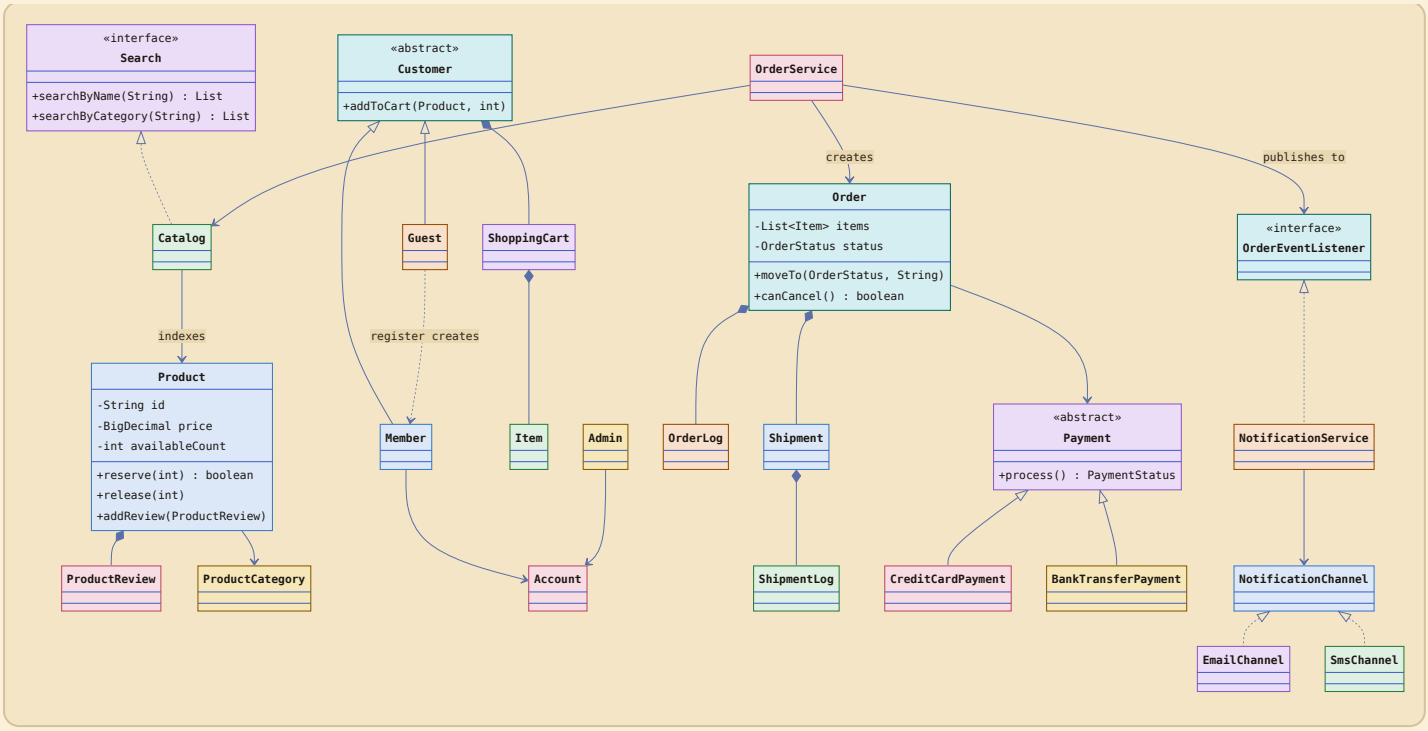


Figure — Stage 4: the cart owns its items, while the order holds its own copy of what was bought (course diagram)

Our Java design follows the same shape:



Key flows

Shopping activity. The course's activity diagram shows the loop a shopper goes round: search, view, add to cart, update the cart, "more shopping?", and finally checkout.

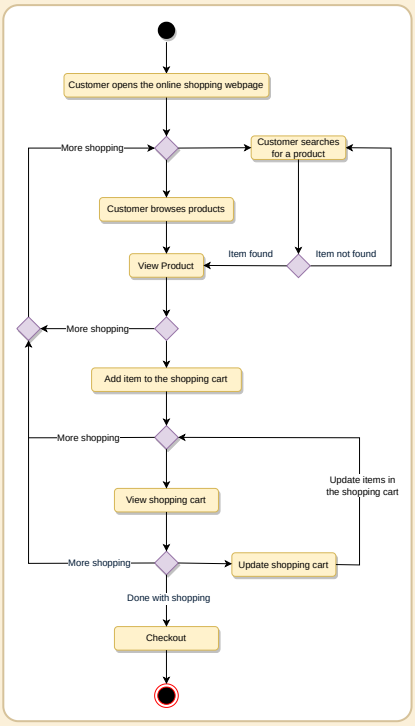


Figure — Activity diagram of a customer shopping online, from opening the site to checkout (course diagram)

Searching the catalog, and adding to the cart. The first two course sequence diagrams show the simple read path (customer → catalog → matching products) and the cart update path (customer → cart → item).

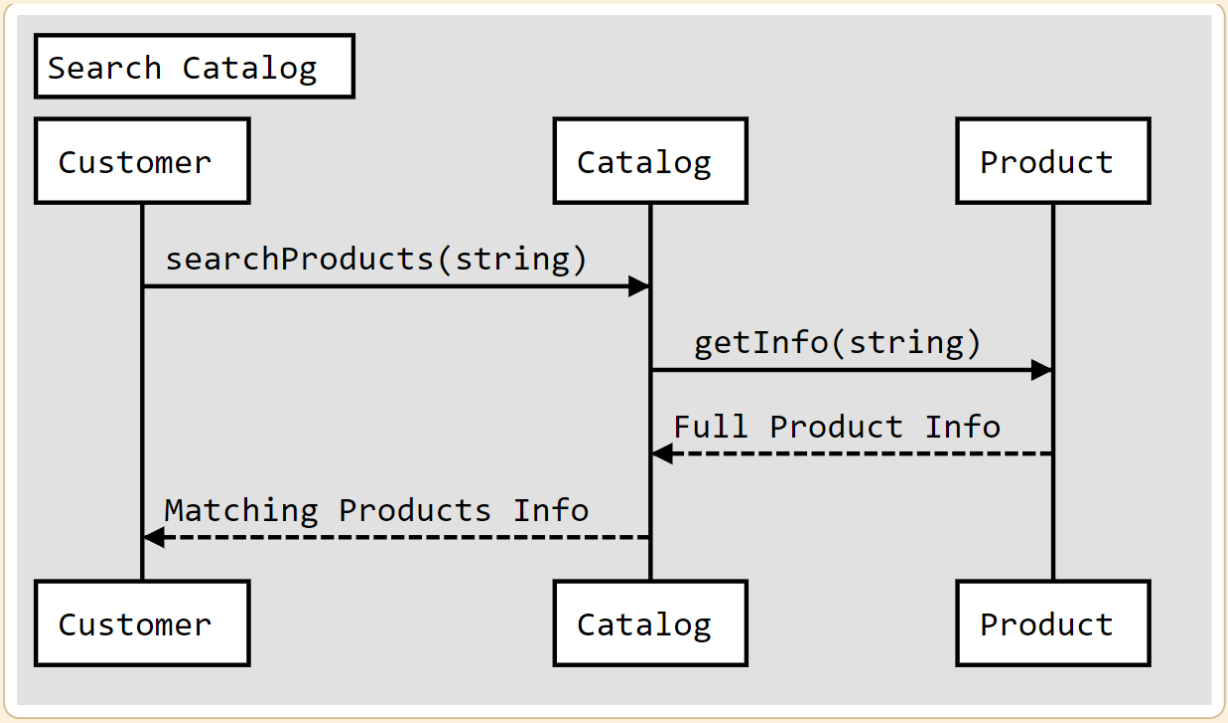


Figure — Sequence diagram for searching the catalog (course diagram)

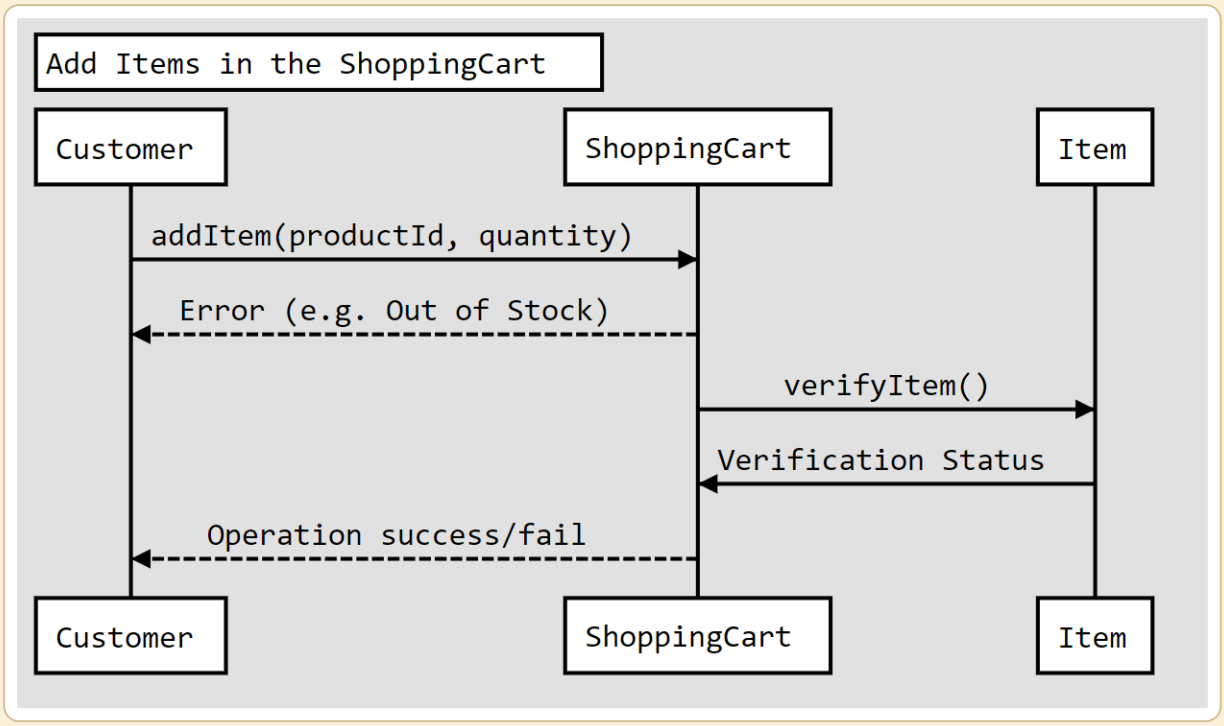


Figure — Sequence diagram for adding an item to the shopping cart (course diagram)

Checkout. The third course sequence diagram shows the important one: get the cart items, get billing and shipping info, create the order, verify every item, process the payment, create the shipment, and return the order status — with an error path after item verification and after payment.

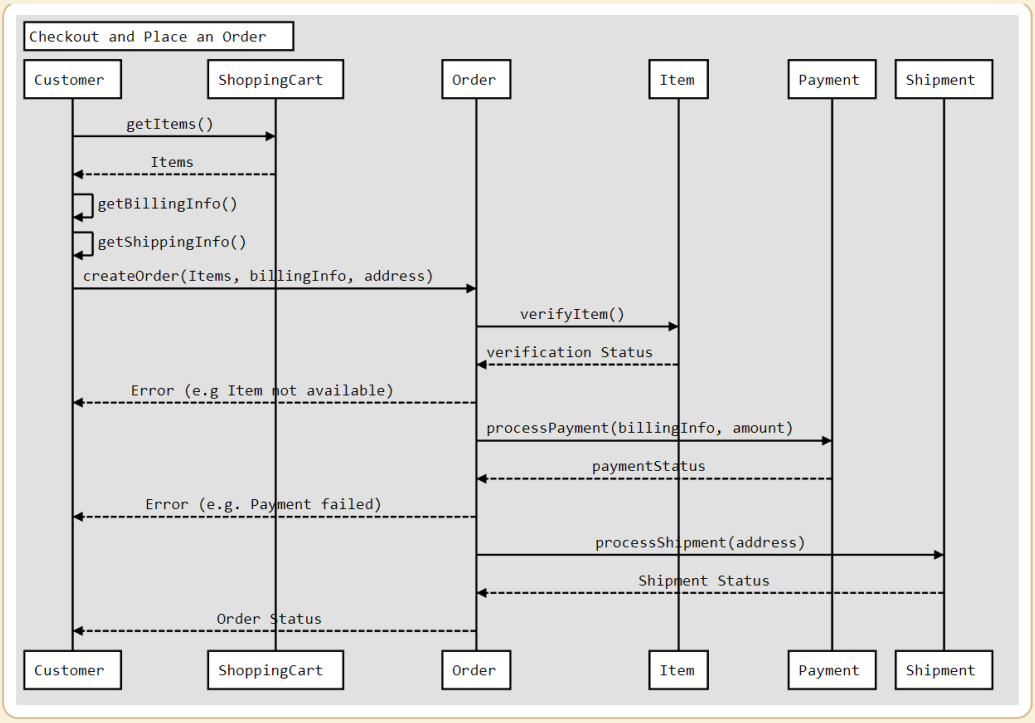
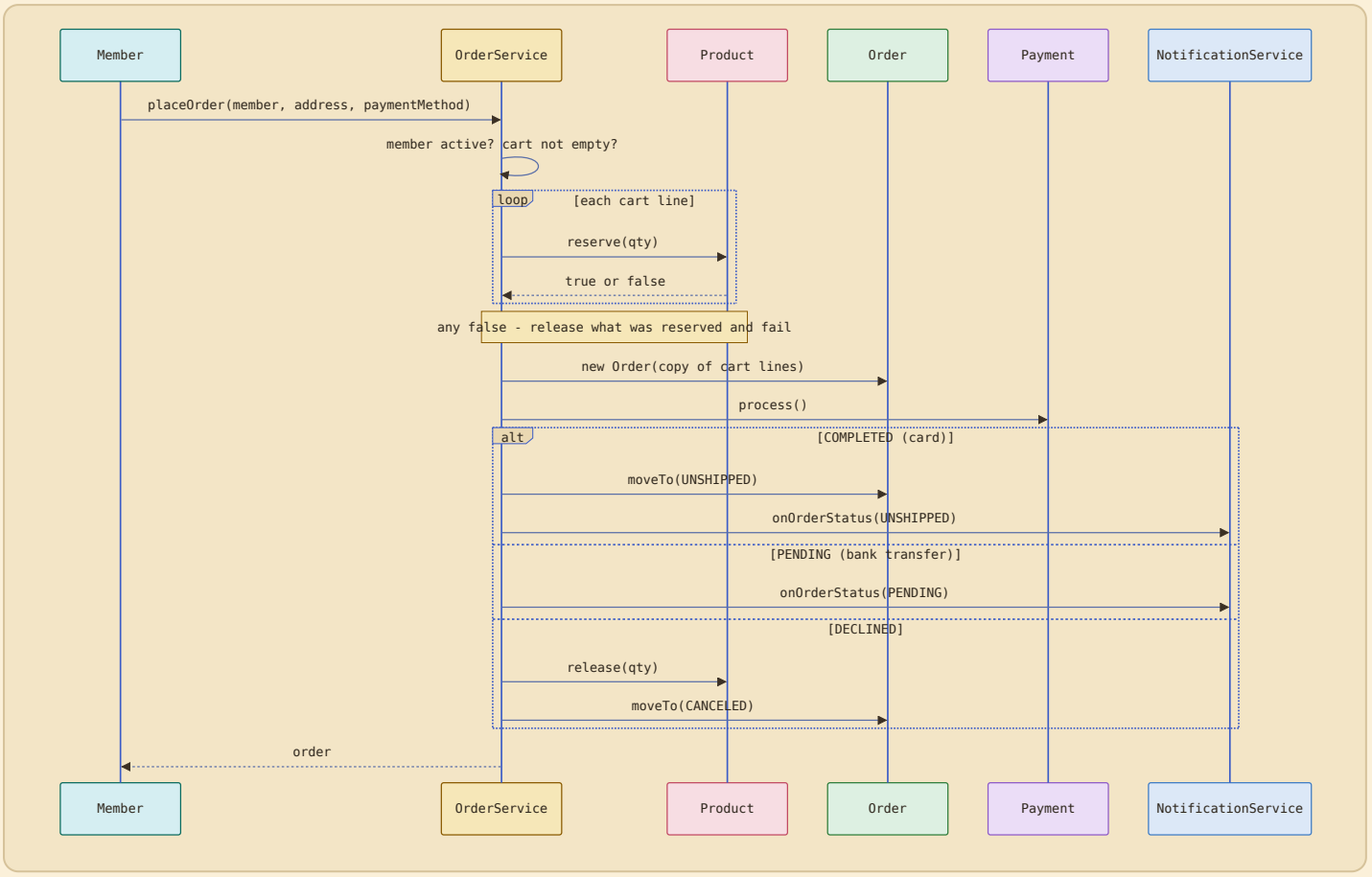
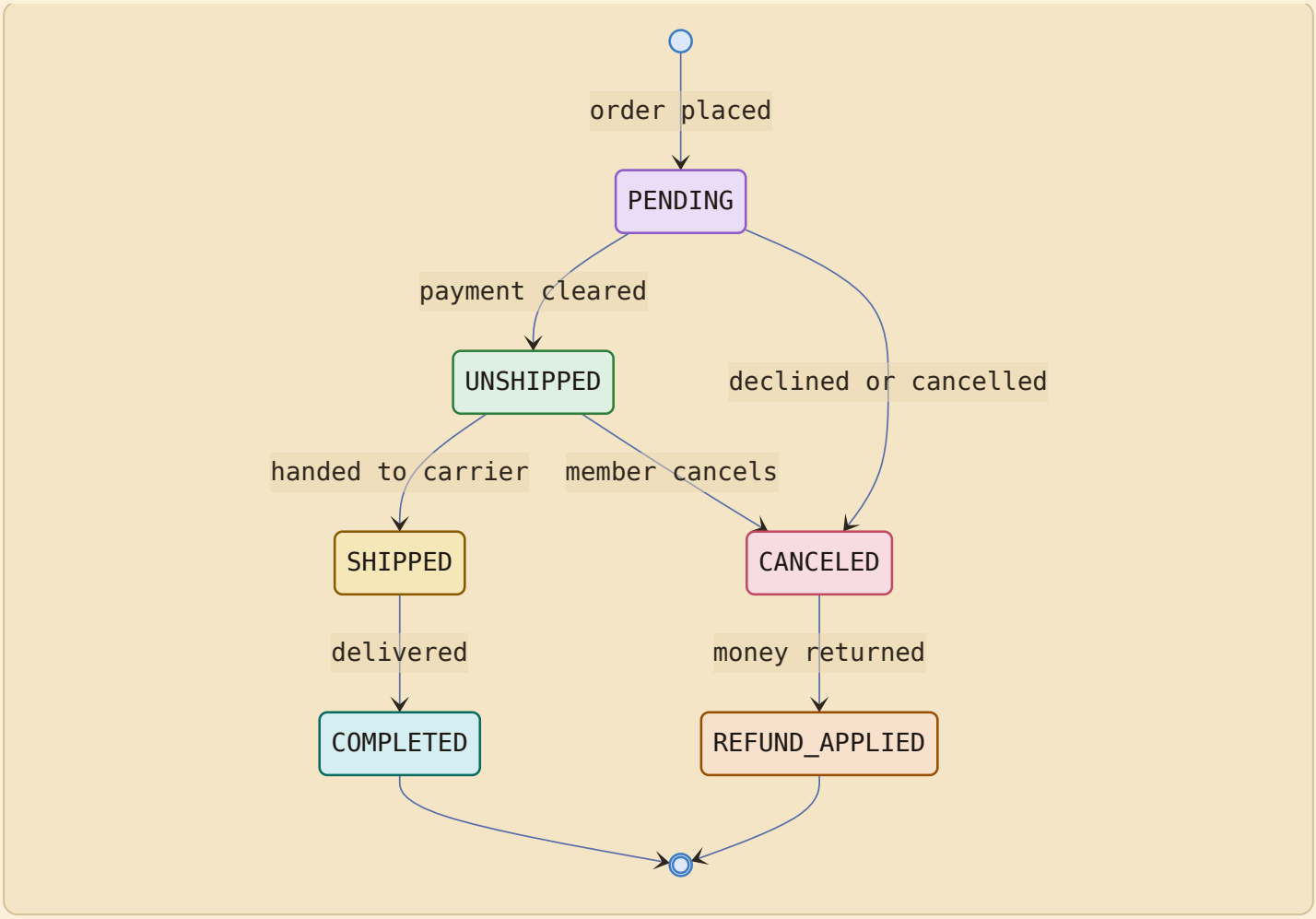


Figure — Sequence diagram for checkout: verify items, take payment, create shipment (course diagram)

Our `OrderService.placeOrder` follows that order, with one addition: stock is **reserved** during verification (not merely checked), so the check cannot go stale before payment.



The order state machine. Every legal move is listed in `OrderStatus.next()`. Anything else throws.



Requirement 8 ("cancel only if not shipped") is now one line: `canCancel()` returns `status.canMoveTo(CANCELED)`. There is no `SHIPPED → CANCELED` edge, so it is impossible by construction.

Design decisions & patterns used

Decision	Why	Alternative rejected
Ordering lives on `Member`, not `Customer`	A `Guest` *is a* `Customer`. If `Customer` had `placeOrder`, every guest would inherit an operation it has to reject — breaking the **Liskov Substitution Principle** (a subclass must be usable wherever its parent is, without surprises).	`Customer.placeOrder()` that throws for guests (the classic LSP smell).
`Guest.register()` returns a new `Member` and carries the cart across	Registration is a real-world moment where the cart must survive.	Mutating a guest into a member (Java objects cannot change class).
`Order` copies the cart lines	An order documents a *finished purchase*; it must stay fixed even if the cart is edited later.	Sharing `Item` objects — cheaper, but editing the cart would silently rewrite a finished order.
Stock reserved at order time, not at payment time	Prevents two shoppers from both getting the final unit. Cost: an abandoned order holds stock until cancelled; real systems release it on a timer.	Check stock, then pay, then decrement (a race window between check and decrement).
State machine inside `OrderStatus`	One place owns the rules. `moveTo` rejects illegal edges.	`if` checks scattered over `cancel`, `ship`, `deliver`.
`Payment` is an abstract (sealed) class, one subclass per method — a **Strategy**	The two methods *behave* differently (card clears now, transfer is pending). An **abstract class** cannot be instantiated itself — only its subclasses can — so every payment is a real card or transfer. Each subclass passes the substitution test (it can genuinely stand in for `Payment`: both really take money), and the same holds for the notification channels. A new method = a new subclass; checkout does not change (Open/Closed Principle: code should be open to extension but closed to modification). The course calls this out: it is why `Payment` is not an enum.	`enum PaymentType` + `switch` in checkout.
Payment passed in as `Function<BigDecimal, Payment>` (a function object: give it an amount, get a `Payment` back)	Checkout knows the total only after building the order, so it asks the caller's factory (a piece of code whose job is to create objects) for a payment of that amount.	Creating the payment before the total is known.
Observer for notifications	`OrderService` publishes events; `NotificationService` decides who to tell and how. Adding push notifications = a new `NotificationChannel`.	Calling email and SMS code directly inside `OrderService`.
`Shipment` keeps a log	Requirement 11 = read the last entry; the whole history stays for support and disputes.	A single `status` field (history lost).
Reviews store `reviewerId`	Avoids a two-way dependency `Product ↔ Member`; each class can be understood alone.	`ProductReview` holding a `Member` object.
`OrderService` as a Facade	One simple entry point hides stock, payment, shipment and notification steps.	Callers orchestrating five objects themselves.

Java implementation

Files: `Money.java` (BigDecimal helpers), `Catalog.java` (category, product, review, search, catalog), `Order.java` (address, item, cart, order status machine, order), `Payments.java` (payments, shipment), `Accounts.java` (account, customer, guest, member, admin), `OrderService.java` (listeners, notifications, service), `Main.java` (demo).

The full runnable source is in `code/lld/ch19/amazon/`; below are the lines that carry the design (getters, constructors and `Main` omitted).

```java
// File: lld/ch19/amazon/Money.java - money is always BigDecimal at scale 2, never double
static BigDecimal of(String amount) { return new BigDecimal(amount).setScale(2, RoundingMode.HALF_EVEN); }

// File: lld/ch19/amazon/Catalog.java
final class Product {                                  // owns its reviews, guards its own stock
    private int availableCount;                        // guarded by this
    /** Take stock out of the pool. False means "not enough" - nothing changes. */
    synchronized boolean reserve(int quantity) {
        if (quantity <= 0 || quantity > availableCount) return false;
        availableCount -= quantity;
        return true;
    }
    synchronized void release(int quantity) { availableCount += quantity; }
}

final class Catalog implements Search {                // points to products, never owns them
    private final Map<String, Set<Product>> byNameWord = new ConcurrentHashMap<>();
    private final Map<String, List<Product>> byCategory = new ConcurrentHashMap<>();
    void addProduct(Product p) {                        // every index updated in one place
        byId.put(p.id(), p);
        for (String word : words(p.name()))
            byNameWord.computeIfAbsent(word, k -> ConcurrentHashMap.newKeySet()).add(p);
        byCategory.computeIfAbsent(key(p.category().name()), k -> new CopyOnWriteArrayList<>()).add(p);
    }
    // searchByName: intersect the sets of every query word; searchByCategory: one map read
}

// File: lld/ch19/amazon/Accounts.java
/** Ordering is deliberately NOT here: a Guest is a Customer and must not inherit it (LSP). */
abstract sealed class Customer permits Guest, Member {
    private final ShoppingCart cart = new ShoppingCart();
    void addToCart(Product p, int qty) { cart.addItem(p, qty); }
}
final class Guest extends Customer {
    Member register(String memberId, Account account) {   // the cart survives registration
        Member m = new Member(memberId, account);
        m.cart().mergeFrom(cart());
        cart().clear();
        return m;
    }
}
final class Member extends Customer {
    boolean canBuy() { return account.status() == AccountStatus.ACTIVE; }
}

// File: lld/ch19/amazon/Order.java
record Item(String productId, String name, int quantity, BigDecimal unitPrice) { }  // price copied at add time

enum OrderStatus {
    PENDING, UNSHIPPED, SHIPPED, COMPLETED, CANCELED, REFUND_APPLIED;
    private Set<OrderStatus> next() {
        return switch (this) {
            case PENDING -> EnumSet.of(UNSHIPPED, CANCELED);      // waiting for payment
            case UNSHIPPED -> EnumSet.of(SHIPPED, CANCELED);      // paid, not yet handed to carrier
            case SHIPPED -> EnumSet.of(COMPLETED);                // too late to cancel
            case CANCELED -> EnumSet.of(REFUND_APPLIED);          // money goes back if it was taken
            case COMPLETED, REFUND_APPLIED -> EnumSet.noneOf(OrderStatus.class);
        };
    }
    boolean canMoveTo(OrderStatus target) { return next().contains(target); }
}

final class Order {
    private final List<Item> items;                   // List.copyOf(cart lines), never shared
    private final List<OrderLog> log = new ArrayList<>();
    synchronized boolean canCancel() { return status.canMoveTo(OrderStatus.CANCELED); }
    synchronized void moveTo(OrderStatus target, String note) {
        if (!status.canMoveTo(target))
            throw new IllegalStateException(orderNumber + ": " + status + " -> " + target + " is not allowed");
        status = target;
```

```
        log.add(new OrderLog(target, note));
    }
}

// File: lld/ch19/amazon/Payments.java - Strategy: a card clears now, a transfer is PENDING
abstract sealed class Payment permits CreditCardPayment, BankTransferPayment {
    abstract PaymentStatus process();
    synchronized PaymentStatus refund() {
        if (status == PaymentStatus.COMPLETED) status = PaymentStatus.REFUNDED;
        else if (status == PaymentStatus.PENDING || status == PaymentStatus.UNPAID) status =
PaymentStatus.CANCELLED;
        return status;
    }
}
// Shipment keeps a List<ShipmentLog>; current() = last entry, history() = all of them

// File: lld/ch19/amazon/OrderService.java - Facade; publishes to OrderEventListener (Observer)
Order placeOrder(Member member, Address address, Function<BigDecimal, Payment> paymentMethod) {
    if (!member.canBuy()) throw new IllegalStateException("account " + member.account().status());
    List<Item> items = member.cart().snapshot();
    if (items.isEmpty()) throw new IllegalStateException("cart is empty");

    List<Item> reserved = new ArrayList<>();          // all-or-nothing reservation
    for (Item i : items) {
        Product p = catalog.find(i.productId()).orElseThrow();
        if (!p.reserve(i.quantity())) {
            reserved.forEach(r -> catalog.find(r.productId()).orElseThrow().release(r.quantity()));
            throw new IllegalStateException("out of stock: " + i.name());
        }
        reserved.add(i);
    }
    Order order = new Order("ORD-" + orderSeq.incrementAndGet(), member.id(), items, address);
    orders.put(order.orderNumber(), order);

    Payment payment = paymentMethod.apply(order.total());
    order.attachPayment(payment);
    switch (payment.process()) {
        case COMPLETED -> { transition(order, OrderStatus.UNSHIPPED, "paid by " + payment.describe());
member.cart().clear(); }
        case PENDING -> { publish(order, OrderStatus.PENDING); member.cart().clear(); }
        default -> {                                  // declined: give the stock back, keep the cart
            releaseStock(order);
            transition(order, OrderStatus.CANCELED, "payment declined");
        }
    }
    return order;
}

boolean cancel(String orderNumber) {
    Order o = get(orderNumber);
    synchronized (o) {                     // check-then-act must be atomic against ship()
        if (!o.canCancel()) return false;
        releaseStock(o);
        transition(o, OrderStatus.CANCELED, "cancelled by member");
    }
    if (o.payment().refund() == PaymentStatus.REFUNDED)
        transition(o, OrderStatus.REFUND_APPLIED, "refunded " + Money.fmt(o.total()));
    return true;
}
// ship() locks the order around transition(SHIPPED); transition() = moveTo + notify every listener
```

`NotificationService implements OrderEventListener` and sends each message through every registered `NotificationChannel` (`EmailChannel`, `SmsChannel`).

Output:

```
== 1. Search ==
name 'code'      -> [Clean Code ($32.50)]
name 'usb cable' -> [USB-C Cable ($9.99)]
category 'books' -> [Clean Code ($32.50), Refactoring ($41.00), Design Patterns (last copy) ($55.00)]

== 2. Guest fills a cart, then registers (cart carries over) ==
  cart: 1 x Clean Code = $32.50
  cart: 2 x USB-C Cable = $19.98

== 3. Checkout with a credit card ==
    [email -> alice@mail.test] Order ORD-1001 is now UNSHIPPED
    [sms   -> +1-555-0101] Order ORD-1001 is now UNSHIPPED
  ORD-1001 total $52.48, status UNSHIPPED, stock of Clean Code now 4

== 4. Ship, track, deliver ==
    [email -> alice@mail.test] Order ORD-1001 is now SHIPPED
    [sms   -> +1-555-0101] Order ORD-1001 is now SHIPPED
    [email -> alice@mail.test] Shipment for ORD-1001: SHIPPED @ sort centre
    [email -> alice@mail.test] Shipment for ORD-1001: SHIPPED @ Portland hub
    [email -> alice@mail.test] Shipment for ORD-1001: DELIVERED @ front door
    [email -> alice@mail.test] Order ORD-1001 is now COMPLETED
    [sms   -> +1-555-0101] Order ORD-1001 is now COMPLETED
  tracking now: ShipmentLog[status=DELIVERED, location=front door], order COMPLETED
  cancel after delivery? false

== 5. Bank transfer (pending) then cancel before shipping -> refund ==
    [email -> alice@mail.test] Order ORD-1002 is now PENDING
    [sms   -> +1-555-0101] Order ORD-1002 is now PENDING
  ORD-1002 status PENDING, payment PENDING
    [email -> alice@mail.test] Order ORD-1002 is now UNSHIPPED
    [sms   -> +1-555-0101] Order ORD-1002 is now UNSHIPPED
    [email -> alice@mail.test] Order ORD-1002 is now CANCELED
    [sms   -> +1-555-0101] Order ORD-1002 is now CANCELED
    [email -> alice@mail.test] Order ORD-1002 is now REFUND_APPLIED
    [sms   -> +1-555-0101] Order ORD-1002 is now REFUND_APPLIED
  cancel before shipping? true -> REFUND_APPLIED, payment REFUNDED, Refactoring stock back to 2
  order log: [PENDING, UNSHIPPED, CANCELED, REFUND_APPLIED]

== 6. Illegal transition is rejected by the state machine ==
  ORD-1002: REFUND_APPLIED -> SHIPPED is not allowed

== 7. Review ==
  Clean Code average rating = 4.5

== 8. Blocked member cannot buy ==
  rejected: account BLOCKED

== 9. Ten members race for the last copy ==
  winners=1 losers=9 stock left=0
```

Reading the output. Step 1 answers both searches from indexes (a name search matches every query word, so `usb cable` finds "USB-C Cable"). Step 2 shows the guest's cart surviving registration. Step 3 prints two notifications (email + SMS) because both channels are registered. Step 5 walks the full cancel path `PENDING → UNSHIPPED → CANCELED → REFUND_APPLIED`, and the stock returns to 2. Step 6 proves the state machine refuses `REFUND_APPLIED → SHIPPED`. Step 9 starts ten threads at the same instant (a `CountDownLatch` — a gate that keeps threads waiting until a counter reaches zero — releases them together) and exactly one wins the last copy.

Concurrency & edge cases

Risk	What could go wrong	How the code prevents it
Last unit, two buyers	Both see `availableCount == 1`, both decrement → stock −1	`Product.reserve` is `synchronized`: check and decrement are one step. Step 9 proves it.
Multi-line cart, one line out of stock	Lines 1–2 reserved, line 3 fails → stock leaked	`placeOrder` remembers what it reserved and releases it all on failure (all-or-nothing).
Cancel and ship at the same time	Order shipped *and* stock released	`cancel` and `ship` both lock the `Order` object around check-then-move.
Payment declined	Stock stays reserved forever	Declined path releases stock and moves the order to `CANCELED`; the cart is kept so the member can retry.
Bank transfer never clears	Stock held forever	Mention a timeout job that cancels `PENDING` orders after N hours.
Price changes after adding to cart	Customer charged a price they never saw	`Item` stores the unit price at add time; production systems re-validate prices at checkout and show changes.
Blocked member	Places orders anyway	`Member.canBuy()` checks `AccountStatus.ACTIVE` first (step 8).
Deadlock (two threads each waiting forever for a lock the other holds)	Two locks taken in different orders	Each lock (product, order, cart) is held briefly and never while waiting for another object's lock in the opposite order.

Real-world example

On Amazon's "Your Orders" page, the *Cancel items* button is offered only while an order has not yet entered the shipping process — the same edge our state machine removes after `SHIPPED`. Amazon also treated the shopping cart as something that must *always* accept writes: its 2007 Dynamo paper (the key-value store — a database that simply maps a key to a value — behind services such as the cart) chose **availability** (staying usable even when some machines fail; see System Design Fundamentals) so that "add to cart" works even during failures, and merges conflicting cart versions later. Our in-memory design shows the object model; at Amazon's scale the cart, catalog and orders live in separate services.

Extensions & follow-up questions

- **"Add a new payment type, e.g. gift cards or UPI (India's instant bank-to-bank mobile payment system)."** Add a `GiftCardPayment extends Payment` (and list it in `permits`). `OrderService` is untouched, because it only calls `process()`.
- **"How do you prevent two shoppers from both buying the final unit?"** Reserve at order time inside a lock (`Product.reserve`). In a distributed system (one spread over many machines): a conditional database update (`UPDATE ... SET stock = stock - 1 WHERE id = ? AND stock >= 1`) or **optimistic locking** with a version column (no lock is held; each row carries a version number, and an update succeeds only if the version is still the one you read — otherwise you retry).
- **"Abandoned orders hold stock."** Give each reservation an expiry. A scheduled job cancels `PENDING` orders older than, say, 30 minutes and releases stock.
- **"Returns after delivery."** Add `COMPLETED → RETURN_REQUESTED → RETURNED → REFUND_APPLIED` edges; the enum change is local.
- **"Multiple warehouses."** Replace `Product.availableCount` with an `Inventory` per warehouse; checkout picks a warehouse per line (a Strategy: nearest, cheapest shipping).
- **"Split shipments."** One `Order` owns many `Shipment`s, each with its own item subset and log.
- **"Search at scale."** The in-memory `Catalog` becomes a search index (for example Elasticsearch/OpenSearch, popular search-engine servers) fed by product change events; the `Search` interface stays.

- **"Seller payouts, recommendations, wish lists."** Separate modules listening to order events (Observer again).

In the interview

30–45 minute plan

1. (3 min) Clarify: guest vs member, payment types, cancel rule, notifications, what is out of scope.
2. (5 min) Actors and use cases; name the four actors.
3. (10 min) Class diagram. **Say the LSP sentence out loud**: "Ordering is on `Member`, not `Customer`, otherwise every guest inherits an ordering method it is not allowed to use."
4. (5 min) The order state machine; show that "cancel after ship" has no edge.
5. (10 min) Code `placeOrder` (reserve all-or-nothing, copy items, pay, notify) and `cancel`.
6. (5 min) Concurrency: last-item race; abandoned-order timeout; extensions.

> **30-second answer:** "Catalog with name and category indexes; a cart per customer; ordering only on `Member`; checkout reserves stock for every line atomically, copies the lines into an immutable order, then takes payment through a `Payment` strategy. The order moves through a state machine that forbids cancelling after shipping, and every status change is published to a notification observer."

Follow-ups with answers

- *Why is `Payment` abstract rather than an enum?* Because the methods behave differently (clears now vs pending). New methods are new classes — Open/Closed.
- *Why does `Order` copy items?* So later cart edits or price changes cannot rewrite history.
- *Where would you put tax and discounts?* A `PricingRule` chain (Chain of Responsibility — a pattern where a request is passed along a line of handlers, each doing its part — or a list of Strategies) applied when building the order; the order stores the final line prices.
- *How do you make checkout idempotent (safe to repeat: doing it twice has the same effect as doing it once) if the user double-clicks?* Accept a client-generated idempotency key (a unique id the browser sends with the request); if an order with that key exists, return it instead of creating a second one.

Self-check

1. **Q:** Why can't `placeOrder` live on `Customer`? **A:** `Guest extends Customer` and guests must not order; inheriting a method you must refuse violates LSP.
2. **Q:** What does the filled diamond between `Order` and `OrderLog` mean? **A:** Composition — the order owns its log entries; they are created and destroyed with it.
3. **Q:** Name two things wrong with "check stock, take payment, then decrement stock". **A:** A race window lets two buyers pass the check; and a payment can succeed for goods that are no longer there.
4. **Q:** Which status edges allow a cancel? **A:** `PENDING → CANCELED` and `UNSHIPPED → CANCELED`.
5. **Q:** Why `compareTo` and not `equals` on `BigDecimal`? **A:** `equals` also compares scale, so `2.0` and `2.00` are "not equal".
6. **Q:** What pattern lets you add push notifications without touching `OrderService`? **A:** Observer (listener) plus a new `NotificationChannel` implementation.

TL;DR

- Guests browse and fill carts; **only members order** (LSP).
- Checkout = **reserve all lines atomically → copy into an immutable order → pay (Strategy) → publish (Observer)**.
- `OrderStatus` is a **state machine**; "cancel after ship" is impossible because the edge does not exist.
- Money in `BigDecimal`; histories in logs; stock guarded by a lock.

Design an Online Stock Brokerage System

This problem has two halves, and interviews ask both:

- **Part A — the brokerage** (what the course designs): members, money in and out, watchlists, the four order types, lots and positions, statements.
- **Part B — the exchange's matching engine** (a machine-coding problem — write a working program in about 90 minutes — originally from Geektrust, an Indian coding-assessment platform, kept in kumaransg/LLD under a project named "Navi"; we call it "the Navi statement" below): an **order book** (the list of all waiting buy and sell orders for one stock) that matches buy and sell orders by **price-time priority**, implemented and tested.

A **broker** is the company you trade *through* (it holds your cash and shares and forwards your orders). An **exchange** is the market where buy and sell orders from all brokers *meet* and are matched. Keep the two apart in your head; the design keeps them apart in code.

Problem & clarifying questions

An online brokerage lets people buy and sell stocks without a human middleman. Members track and carry out their transactions, see charts of how the stocks they hold are performing, and get an alert when a stock moves by an amount they chose. The platform must keep those transactions secure. Because computers replace the phone call to a broker, each trade is quicker and cheaper, and members get fast access to real-time (updated the moment they change) prices, market trends and stock reports.

Question	Assumed answer
Which order types?	Market, limit, stop-loss, stop-limit (explained in the table under Key flows).
Can one order fill at several prices?	Yes — an order can be filled in parts.
Do we track separate purchases of the same stock?	Yes — each purchase is a **lot**, because tax depends on which lot you sell.
How does money come in and out?	Check, wire (a direct bank-to-bank transfer), or electronic bank transfer, each with a per-transfer limit.
Reports?	Quarterly statements and a yearly tax statement.
Notifications?	When an order executes.
Margin, short selling, options, dividends and stock splits, real-time streaming quotes, the settlement delay, the wash-sale tax rule?	Out of scope — a deliberate decision, say so.

For reference, those out-of-scope terms: **margin** is trading with money borrowed from the broker; **short selling** is selling borrowed shares, hoping to buy them back cheaper; **options** are contracts giving the right to buy or sell later at a fixed price; **dividends** are profits a company pays to its shareholders; a **stock split** turns each share into several cheaper ones; **settlement** is the actual swap of cash and shares, which happens a day or so after the trade; the **wash-sale rule** is a US tax rule that blocks claiming a loss if you rebuy the same stock within 30 days.

Requirements

Functional (brokerage)

1. Any member can buy and sell stocks.
2. A member can have **multiple watchlists** (named lists of stocks to keep an eye on), each with multiple stock quotes (a quote is a stock's latest price).
3. Orders can be **market, limit, stop-loss, stop-limit**.
4. A member can hold **multiple lots** of the same stock; the system keeps them apart.
5. The system generates **quarterly** updates and **yearly tax** statements.

6. Money goes in and out of the account in three ways: **check, wire, or electronic bank transfer**.
7. A notification is sent whenever an order executes.

Functional (matching engine, Navi variant)

8. Orders arrive as `<id> <time> <symbol> <buy/sell> <price> <qty>`, e.g. `#1 09:45 BAC sell 240.12 100`. They are read from a text file whose path is given as the program's argument; trades are printed to the console.
9. On every new order, match against the opposite side. Lower sell prices and higher buy prices go first; at the same price, **earlier orders go first** (price-time priority).
10. A trade happens when buy price ≥ sell price. In the Navi statement the trade is recorded at the **sell order's price**.
11. Print each trade as `<buy_id> <price> <qty> <sell_id>`; unmatched quantity stays in the book.

Non-functional

- Money is never over-committed: ten $1,000 orders cannot all pass a $1,000 balance check.
- Matching is deterministic (no randomness or timing luck): same input, same trades.
- The engine is safe under concurrent submissions and conserves quantity (shares bought = shares sold).

Use cases

Three actors:

- **Admin** — administrative jobs such as blocking or unblocking members.
- **Member** — searches the stock inventory, buys and sells, manages watchlists.
- **System** — sends order notifications and periodically fetches quotes from the exchange.

Top use cases: register / cancel membership; add, remove or edit a watchlist; search stocks by symbol; place an order; cancel an order; deposit or withdraw money.

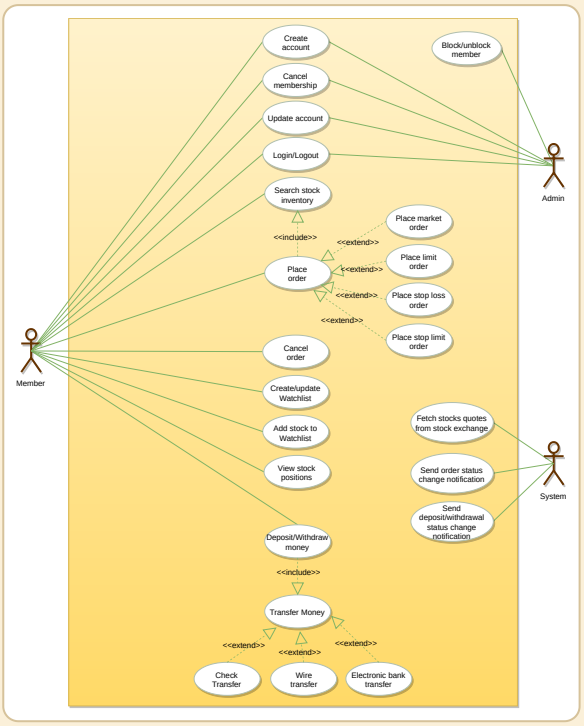


Figure — Use-case diagram for the stock brokerage: Admin, Member and System (course diagram)

Core entities

Class	Responsibility
`MoneyTransfer` → `CheckTransfer`, `WireTransfer`, `ElectronicBankTransfer`	Move money in or out. They differ only in name and clearing days (5 / 1 / 3 in our code); the per-transfer limit (100,000) is a single rule, so it lives only in the base class.
`Notifier` / `EmailNotifier`	Delivers "your order filled" messages. The course draws one base with two delivery ways (email and SMS); an `SmsNotifier` would be a second implementation.
`Order` (abstract)	Shared by all order types: member, symbol, side (buy or sell), quantity, time-in-force (how long the order stays live), status, list of fills. Declares the **two questions**.
`MarketOrder`, `LimitOrder`, `StopLossOrder`, `StopLimitOrder`	Each answers "may I trade now?" and "what is my worst price?". `StopLimitOrder extends StopLossOrder`.
`OrderPart`	One fill: quantity, price, day. A 100-share order might be filled as 70 @ 10.00 plus 30 @ 10.05.
`Stock`, `StockInventory`	Symbol + latest price; the inventory is the **single source of truth** (the one place whose value counts if copies disagree) for prices.
`StockExchange`	Holds open orders; on each new order or price tick tries to fill them; calls back through `OrderListener`.
`OrderListener`	Interface the exchange calls (`onFill`, `onCancelled`). The exchange never learns what a member is.
`StockLot`	One purchase at one price.
`StockPosition`	All lots of one symbol; sells from the oldest lot first (FIFO — first in, first out) unless a lot is named. Returns a `LotSale` per lot touched.
`LotSale`	Shares taken from one lot, buy price, sell price → realised gain (profit actually locked in by selling).
`Watchlist`	A named set of **symbols** (not `Stock` objects, so it never goes stale).
`Trade`	One fill as the member sees it (sells carry their `LotSale`s).
`Statement` → `QuarterlyStatement`, `TaxStatement`	Base class filters trades by day range; subclasses write their own lines.
`Member`	Cash, **reserved funds**, positions, watchlists, trades. Implements `OrderListener`. (The course's `Account` also stores name, address, e-mail and phone; we keep only what the demo uses.)
`Admin`	Blocks / unblocks members.
(engine) `OrderBook`, `MatchingEngine`, `Trade`, `ExecutionPricePolicy`	Price levels with FIFO queues per symbol; routing and locking; who sets the trade price.

Class diagram

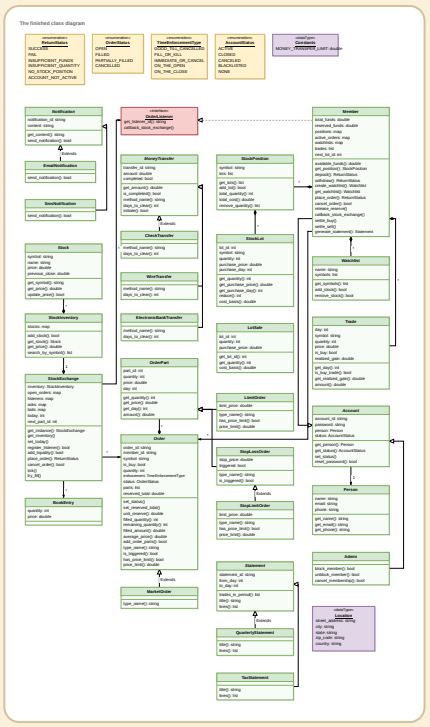


Figure — The finished class diagram of the online stock brokerage (course diagram)

The course builds this model in six stages: money transfers and notifications → the order hierarchy → stock, inventory and exchange → lots and positions → watchlists, trades and statements → members and admins. In the figure above, `Member` extends `Account` and implements `OrderListener` (dashed arrow): the exchange reports fills through that interface and never needs the `Member` type. The filled diamonds from `Member` mean each position, watchlist and trade is owned by exactly one member; the plain arrow to `Order` says the exchange shares those orders.

The course singles out stage 2 — the order hierarchy — as the part that earns the most credit in an interview:

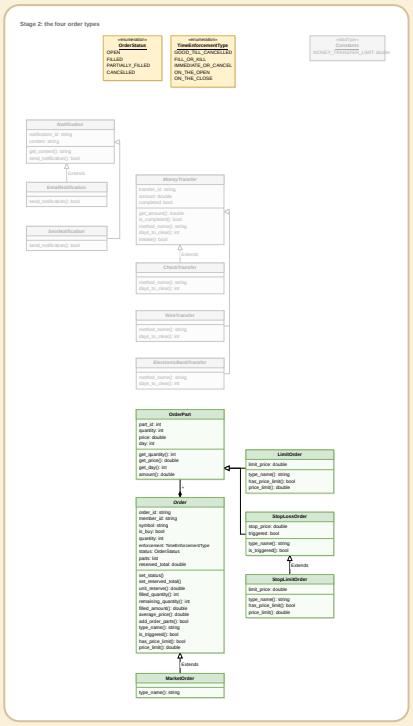
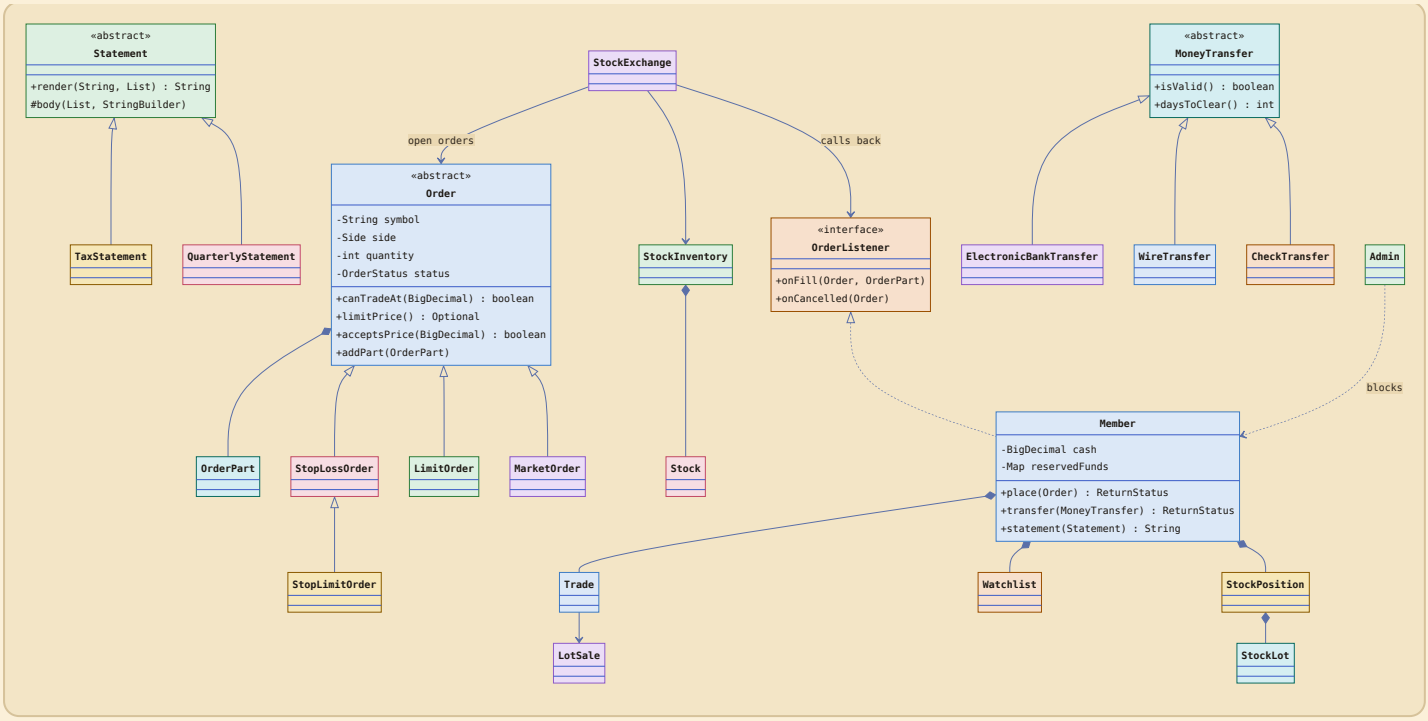
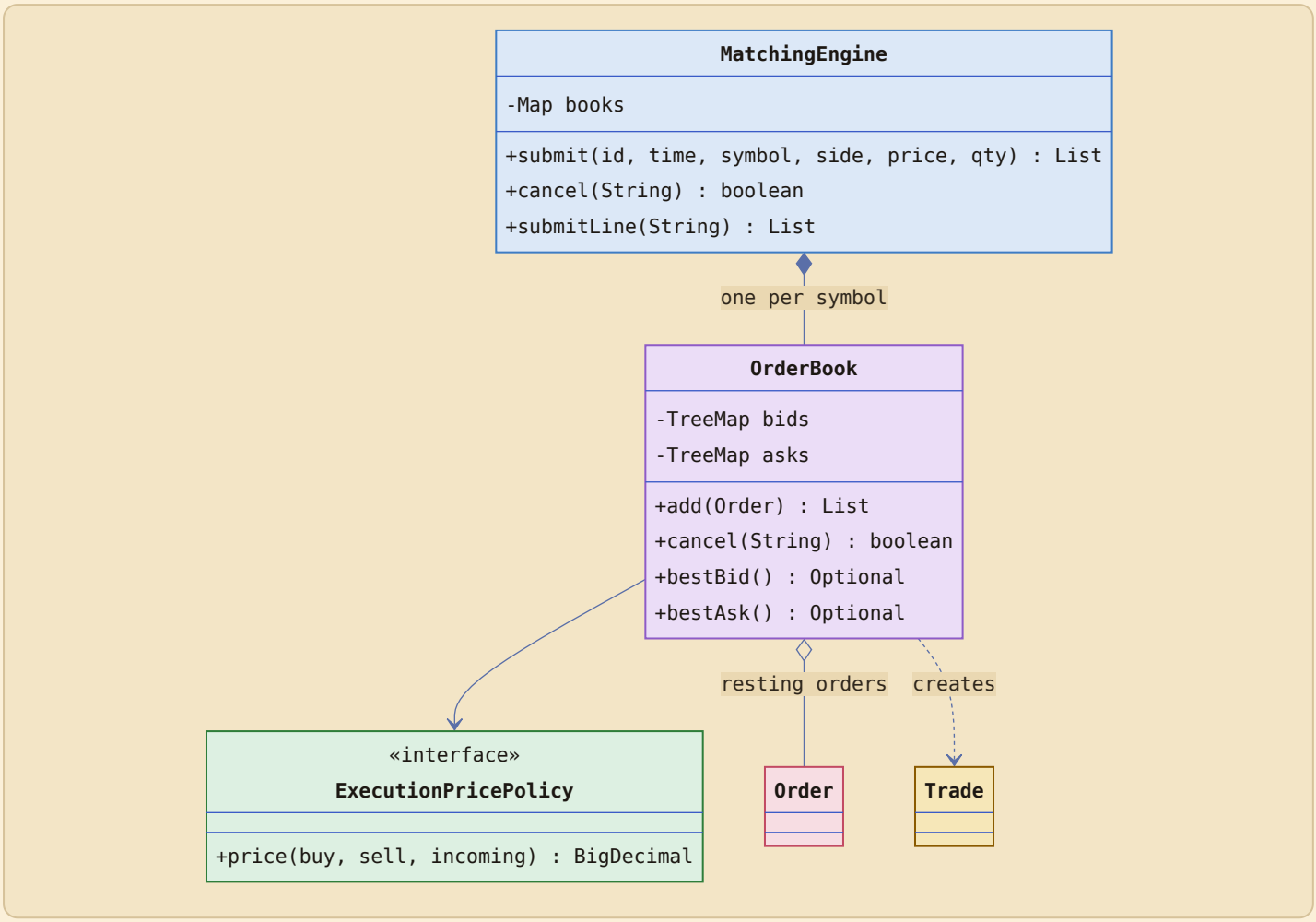


Figure — Stage 2: one abstract Order and four small subclasses that differ only in two answers (course diagram)

Our brokerage code:



And the matching engine (Part B):



Key flows

Placing a buy order. The course's activity diagram: pick a stock, pick the order type (market, limit, stop-loss, stop-limit), enter quantity, price limit and time enforcement; the system checks there is enough money, then deducts it (we *reserve* it) and sends the order to the exchange; when the exchange acknowledges, the member sees "order placed". The member can then update the order.

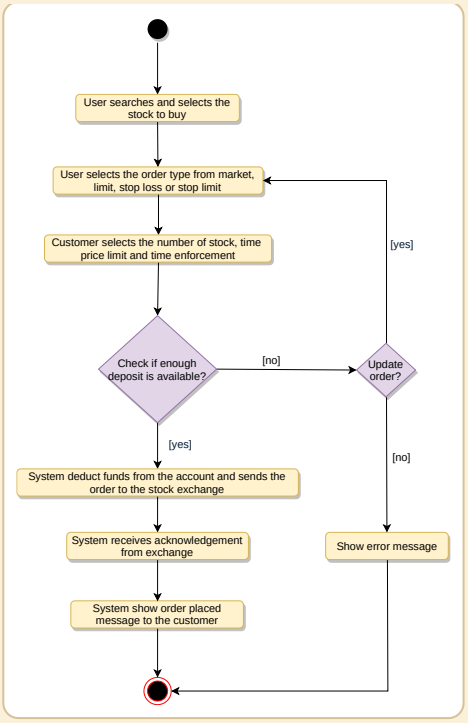


Figure — Activity diagram for placing a buy order (course diagram)

Placing a sell order is the mirror image, with one extra step: the member picks **which lots** to sell, and the check is "do I own enough shares?" rather than "do I have enough money?".

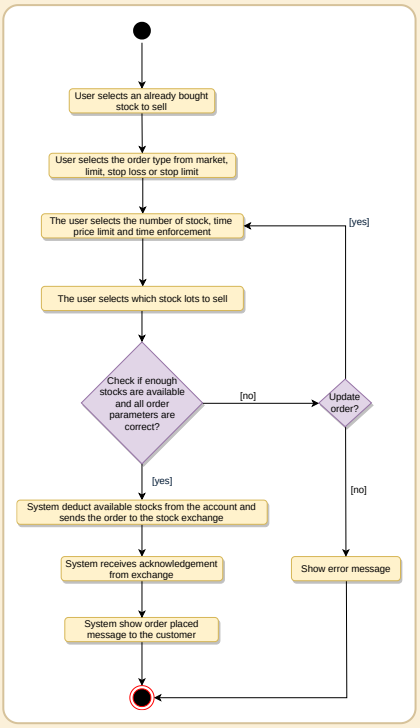


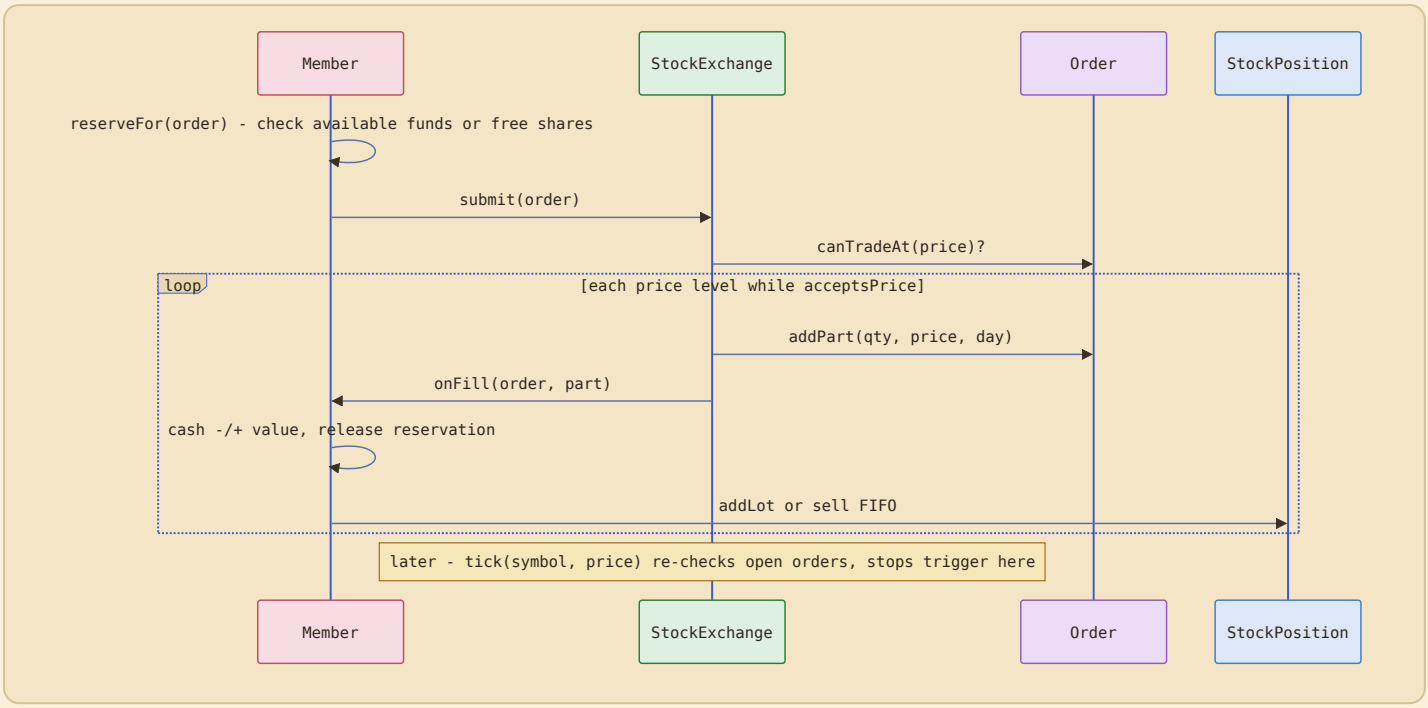
Figure — Activity diagram for placing a sell order, including choosing lots (course diagram)

The four order types in one table. They share everything except two answers:

Type	Q1: may it trade at this market price?	Q2: worst acceptable price	Plain words
Market	Always	None	"Buy/sell now at whatever the market gives."
Limit	Always	The limit	"Buy at 9.50 or less / sell at 10.90 or more."
Stop-loss (stop)	Only after the price **reaches the stop**, then forever	None (becomes a market order)	"If it falls to 10.00, get me out."
Stop-limit	Only after the stop is reached	The limit	"If it rises to 305, buy — but pay no more than 306."

A SELL stop triggers when price ≤ stop; a BUY stop triggers when price ≥ stop. Once triggered it stays triggered (the code "latches" a boolean).

Fill → money and lots move (Observer callback).

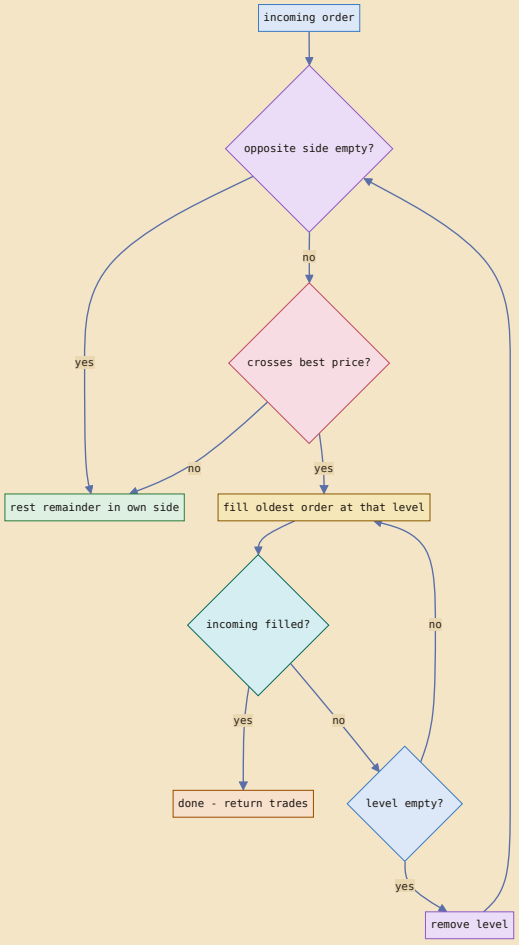


Matching engine: price-time priority. Each symbol has an `OrderBook` with two sides. Each side is a `TreeMap` (a map that keeps its keys sorted) from price to a FIFO queue (`ArrayDeque`, Java's fast double-ended queue). **Bids** (buy orders) are sorted highest-first, **asks** (sell orders) lowest-first. For an incoming order:

1. Look at the best price on the opposite side.
2. If it does not **cross** (buy price < best ask, or sell price > best bid), stop.
3. Otherwise fill against that level's queue, **oldest order first**, for `min(incoming remaining, resting remaining)`.
4. Remove fully filled resting orders; drop empty price levels; repeat from step 1.
5. If the incoming order still has quantity, add it to the back of its own price level's queue.

Walk-through of the sample input:

Order	Book before (asks / bids)	What happens
#1 sell 240.12 × 100	empty	rests on asks
#2 sell 237.45 × 90	asks 240.12×100	rests; best ask now 237.45
#3 buy 238.10 × 110	asks 237.45×90, 240.12×100	crosses 237.45 → trade `#3 237.45 90 #2`; 20 left; 240.12 > 238.10 so stop; rests 20 @ 238.10
#4 buy 237.80 × 10	best ask 240.12	no cross; rests
#5 buy 237.80 × 40	same	no cross; rests **behind #4** at 237.80
#6 sell 236.00 × 50	bids 238.10×20, 237.80×(10, 40)	fills #3 for 20, then #4 for 10, then #5 for 20 — all at 236.00 (the sell price)



Design decisions & patterns used

Decision	Why	Alternative rejected
One abstract `Order` + two abstract questions	The four types differ only in "may I trade?" and "worst price?". Each subclass is a few lines. The course stresses this is just a type hierarchy (plain polymorphism — calling the same method on different subclasses and each running its own version), not a named pattern: every subclass answers for itself whether the current price lets it trade, and no order ever turns into another kind — so it is not the State pattern either. (The only named patterns the course lists are Observer and Singleton — a class that allows exactly one instance; the Template Method and Strategy labels in this table are our additions.)	Four unrelated classes with copied fields; or one class with a `type` field and `switch`es.
`StopLimitOrder extends StopLossOrder`	A stop-limit is a stop that becomes a *limit* instead of a *market* order: it inherits the trigger, overrides only `limitPrice()`.	Duplicating the trigger logic.
`OrderPart` per fill	One order can fill at several prices/days; each fill needs its own price and quantity (and becomes its own lot).	Storing only an average price (loses tax information).
Observer: `OrderListener`	The exchange tells whoever placed the order; it does not import `Member`. Exchange and account code change independently.	The exchange updating member balances directly.
Reserved funds / reserved shares	Without it, a member with $1,000 could place ten $1,000 orders and all would pass the check. Money is reserved at placement and released as fills or cancellations arrive. Money and shares actually move only in `onFill`, which the **exchange** calls (the course's `callbackStockExchange`) — so the moment an order is *placed* and the moment it is *paid for* are different.	Checking the balance only at fill time (over-commitment).
Lots, FIFO by default	Tax depends on *which* shares you sold; each `LotSale` records it.	Merging all purchases into one average-cost position.
Template Method: `Statement`	Filtering by day range is shared; only the lines differ. `render` is `final`; subclasses fill in `body`.	Two statement classes each re-implementing the filter.
`MoneyTransfer` base class holds the limit rule	One rule, three delivery methods.	Copying the limit check into each subclass.
Singleton?	The course makes `StockExchange` a Singleton (one gateway to the real exchange). We pass one instance into `Member` instead (**dependency injection**: an object is handed the things it needs from outside rather than creating or looking them up itself) — same "only one" in practice, but testable with a fake exchange.	A global `getInstance()` that tests cannot replace.
Engine: `TreeMap<price, ArrayDeque>` per side	Best price in O(log n) (Big-O notation: how the work grows with n items — here, very slowly); FIFO inside a level gives time priority for free.	One `PriorityQueue` (a structure that always hands back the smallest item first) sorted by (price, time) — works, but cancel is O(n) (time grows in step with the number of orders) and levels are not visible.

Decision	Why	Alternative rejected
Engine: lock per symbol	Orders for different symbols match in parallel; same-symbol orders are strictly serialised (handled one at a time, in order), so "time priority" is well defined.	One global lock (slower) or no lock (corrupt book).
Engine: `ExecutionPricePolicy` (Strategy)	The Navi statement prices trades at the **sell** order's price; real continuous-trading exchanges use the **resting** (already-in-book) order's price. Both are one-line policies.	Hard-coding one rule.

Java implementation

Part A — Brokerage (`lld.ch19.brokerage`)

Files: `Orders.java` (enums, money helper, `OrderPart`, the order hierarchy), `Market.java` (stock, inventory, listener, simulated exchange), `Holdings.java` (lots, positions, watchlists, trades, statements), `Accounts.java` (transfers, notifier, member, admin), `Main.java`.

The simulated exchange offers a fixed depth of shares per price level (70 shares at each level, 5 cents apart, 3 levels) so fills come back in parts, like a real book. Part B builds a real matching engine.

The full runnable source is in `code/lld/ch19/brokerage/`; below are the lines that carry the design.

```java
// File: lld/ch19/brokerage/Orders.java
/** The four order types differ in exactly two answers. */
abstract class Order {
    /** Question 1. Market and limit orders always say yes; stop orders wait for the trigger. */
    abstract boolean canTradeAt(BigDecimal marketPrice);
    /** Question 2. Empty means "any price" (market behaviour). */
    abstract Optional<BigDecimal> limitPrice();

    /** Is a fill at this price acceptable? BUY: price <= limit. SELL: price >= limit. */
    final boolean acceptsPrice(BigDecimal price) {
        return limitPrice().map(l -> side == Side.BUY ? price.compareTo(l) <= 0 : price.compareTo(l) >= 0)
                           .orElse(true);
    }
    final synchronized void addPart(OrderPart p) {
        if (p.quantity() > remaining()) throw new IllegalStateException("overfill");
        parts.add(p);
        status = remaining() == 0 ? OrderStatus.FILLED : OrderStatus.PARTIALLY_FILLED;
    }
}

final class MarketOrder extends Order {
    @Override boolean canTradeAt(BigDecimal p) { return true; }
    @Override Optional<BigDecimal> limitPrice() { return Optional.empty(); }
}
final class LimitOrder extends Order {
    @Override boolean canTradeAt(BigDecimal p) { return true; }
    @Override Optional<BigDecimal> limitPrice() { return Optional.of(limit); }
}
class StopLossOrder extends Order {
    @Override final synchronized boolean canTradeAt(BigDecimal p) {
        if (!triggered)                                    // latches: once true, stays true
            triggered = side() == Side.SELL ? p.compareTo(stopPrice) <= 0 : p.compareTo(stopPrice) >= 0;
        return triggered;
    }
    @Override Optional<BigDecimal> limitPrice() { return Optional.empty(); }
}
final class StopLimitOrder extends StopLossOrder {          // inherits the trigger, adds a limit
    @Override Optional<BigDecimal> limitPrice() { return Optional.of(limit); }
}

// File: lld/ch19/brokerage/Market.java - simulated exchange: 70 shares per level, 5 cents apart, 3 levels
synchronized void tick(String symbol, BigDecimal newPrice) {      // stops trigger here
    inventory.find(symbol).orElseThrow().setPrice(newPrice);
    for (Order o : List.copyOf(openOrders.getOrDefault(symbol, List.of()))) tryFill(o, newPrice);
    prune(symbol);
}
private void tryFill(Order o, BigDecimal market) {
    if (!o.isOpen() || !o.canTradeAt(market)) return;
    for (int k = 0; k < levels && o.remaining() > 0; k++) {
        BigDecimal step = tickSize.multiply(BigDecimal.valueOf(k));
        BigDecimal levelPrice = o.side() == Side.BUY ? market.add(step) : market.subtract(step);
        if (!o.acceptsPrice(levelPrice)) break;          // limit reached: rest stays open
        OrderPart part = new OrderPart(Math.min(depthPerLevel, o.remaining()), levelPrice, day);
        o.addPart(part);
        listeners.get(o.memberId()).onFill(o, part);     // Observer callback
    }
}

// File: lld/ch19/brokerage/Holdings.java
List<LotSale> sell(int qty, BigDecimal price, Optional<String> preferredLot) {   // StockPosition
    // named lot first, then oldest first (FIFO); one LotSale per lot touched
    for (StockLot lot : order) {
        if (left == 0) break;
        int take = Math.min(left, lot.quantity());
        if (take == 0) continue;
        lot.take(take);
        out.add(new LotSale(lot.lotId(), take, lot.buyPrice(), price));
        left -= take;
    }
}
```

```
abstract class Statement {                                  // Template Method
    final String render(String memberName, List<Trade> all) {
        List<Trade> inRange = all.stream().filter(t -> t.day() >= fromDay && t.day() <= toDay).toList();
        StringBuilder sb = new StringBuilder(title() + " for " + memberName + " (days " + fromDay + "-" + toDay +
")\n");
        body(inRange, sb);                                  // QuarterlyStatement / TaxStatement fill this in
        return sb.toString();
    }
}

// File: lld/ch19/brokerage/Accounts.java
abstract class MoneyTransfer {                              // Check (5 days), Wire (1), ElectronicBank (3)
    static final BigDecimal TRANSFER_LIMIT = Money.of("100000");
    boolean isValid() { return amount.signum() > 0 && amount.compareTo(TRANSFER_LIMIT) <= 0; }
    abstract int daysToClear();
}

final class Member implements OrderListener {
    ReturnStatus place(Order o) {
        ReturnStatus check = reserveFor(o);                // under our lock
        if (check == ReturnStatus.SUCCESS) exchange.submit(o);   // outside our lock (lock order)
        return check;
    }
    private synchronized ReturnStatus reserveFor(Order o) {
        if (status != AccountStatus.ACTIVE) return ReturnStatus.ACCOUNT_NOT_ACTIVE;
        if (o.side() == Side.BUY) {
            BigDecimal perShare = o.limitPrice().orElse(inventory.priceOf(o.symbol()));
            BigDecimal need = Money.times(perShare, o.quantity());
            if (need.compareTo(availableFunds()) > 0) return ReturnStatus.INSUFFICIENT_FUNDS;
            reservedFunds.put(o.id(), need);           // released part by part as fills arrive
            return ReturnStatus.SUCCESS;
        }
        // SELL: shares owned minus shares already promised to other open sells
        if (o.quantity() > pos.quantity() - promised) return ReturnStatus.INSUFFICIENT_QUANTITY;
        pendingSells.put(o.id(), o);
        return ReturnStatus.SUCCESS;
    }
    /** Called by the exchange: this is where money and shares actually move. */
    @Override public synchronized void onFill(Order o, OrderPart part) {
        // BUY:  cash -= value, shrink the reservation, add a new StockLot for this part
        // SELL: cash += value, StockPosition.sell(...) FIFO -> LotSales for the tax statement
        trades.add(new Trade(part.day(), o.symbol(), o.side(), part.quantity(), part.price(), sales));
    }
}
```

Output:

```
== 1. Fund the account ==
  wire 5,000        -> SUCCESS
  check 200,000     -> FAIL (over the 100,000 limit)
  cash = $5000.00

== 2. Watchlists and search ==
  tech: AAPL=10.00 MSFT=300.00
  retail: AMZN=120.00
  search 'A' -> [AAPL, AMZN]

== 3. Market buy 100 AAPL fills in two parts (two lots) ==
    [email alice@mail.test] filled 70 AAPL @ 10.00 (O1 PARTIALLY_FILLED)
    [email alice@mail.test] filled 30 AAPL @ 10.05 (O1 FILLED)
  place -> SUCCESS
  O1 MARKET BUY 100 AAPL [FILLED] parts=[OrderPart[quantity=70, price=10.00, day=1], OrderPart[quantity=30,
price=10.05, day=1]]

== 4. Next day: limit buy 50 AAPL @ 9.50 waits, then fills when price drops ==
  place -> SUCCESS, O2 LIMIT@9.50 BUY 50 AAPL [OPEN], reserved $475.00
    [email alice@mail.test] filled 50 AAPL @ 9.40 (O2 FILLED)
  after tick 9.40: O2 LIMIT@9.50 BUY 50 AAPL [FILLED], reserved $0.00
  holdings {AAPL=[L1:70@10.00, L2:30@10.05, L3:50@9.40]}

== 5. Day 3: price rises, sell 120 AAPL (FIFO across lots) ==
    [email alice@mail.test] filled 70 AAPL @ 11.00 (O3 PARTIALLY_FILLED)
    [email alice@mail.test] filled 50 AAPL @ 10.95 (O3 FILLED)
  place -> SUCCESS, O3 LIMIT@10.90 SELL 120 AAPL [FILLED]
  holdings {AAPL=[L3:30@9.40]}

== 6. Protect the rest with a stop-loss @ 10.00, then the price falls ==
  place -> SUCCESS, O4 STOP@10.00 SELL 30 AAPL [OPEN]
  tick 10.50: O4 STOP@10.00 SELL 30 AAPL [OPEN]
    [email alice@mail.test] filled 30 AAPL @ 9.95 (O4 FILLED)
  tick 9.95: O4 STOP@10.00 SELL 30 AAPL [FILLED] parts=[OrderPart[quantity=30, price=9.95, day=4]]

== 7. Stop-limit buy triggers but its limit is not met ==
  place -> SUCCESS, O5 STOP@305.00/LIMIT@306.00 BUY 5 MSFT [OPEN]
  tick 310 (stop hit, above limit): O5 STOP@305.00/LIMIT@306.00 BUY 5 MSFT [OPEN]
    [email alice@mail.test] filled 5 MSFT @ 305.50 (O5 FILLED)
  tick 305.50 (within limit): O5 STOP@305.00/LIMIT@306.00 BUY 5 MSFT [FILLED]

== 8. Reserved funds block over-spending, cancel releases them ==
  cash $3617.00, available $3617.00
  O6 buy 20 AMZN @ 100 -> SUCCESS, available $1617.00
  O7 buy 20 AMZN @ 100 -> INSUFFICIENT_FUNDS
    [email alice@mail.test] O6 cancelled, 20 shares unfilled
  cancel O6 -> SUCCESS, available $3617.00
  sell 10 AMZN (none owned) -> NO_STOCK_POSITION

== 9. Blocked account cannot trade ==
  -> ACCOUNT_NOT_ACTIVE

== 10. Reports ==
Quarterly statement for Alice (days 1-90)
  day 1 BUY  AAPL   70 @ 10.00
  day 1 BUY  AAPL   30 @ 10.05
  day 2 BUY  AAPL   50 @ 9.40
  day 3 SELL AAPL   70 @ 11.00
  day 3 SELL AAPL   50 @ 10.95
  day 4 SELL AAPL   30 @ 9.95
  day 4 BUY  MSFT    5 @ 305.50
Tax statement for Alice (days 1-365)
  AAPL L1: 70 sh bought @ 10.00 sold @ 11.00 gain $70.00
  AAPL L2: 30 sh bought @ 10.05 sold @ 10.95 gain $27.00
  AAPL L3: 20 sh bought @ 9.40 sold @ 10.95 gain $31.00
  AAPL L3: 30 sh bought @ 9.40 sold @ 9.95 gain $16.50
  realised gain: $144.50
```

Reading the output. Step 1: the 200,000 check fails the 100,000 transfer limit. Step 3 reproduces the course's example: a market order for 100 fills as 70 @ 10.00 and 30 @ 10.05 and creates **two lots**.

Notice the course's warning in action: the order reserved $1,000.00 (100 × last price 10.00) but cost $1,001.50 — a market order can spend more than was reserved. Step 4: a limit buy waits until the price tick to 9.40. Step 5 sells 120 shares FIFO across three lots. Step 6: the stop does nothing at 10.50 and fires at 9.95. Step 7: the stop-limit triggers at 310 but its 306 limit blocks the fill until 305.50. Step 8: the reserve blocks a second $2,000 order; cancelling returns the money. Step 10: the tax statement lists the gain **per lot**.

Part B — Matching engine (`lld.ch19.stockexchange`)

Files: `OrderBook.java` (side, order, trade, price policy, book), `MatchingEngine.java` (routing, per-symbol locking, line parser), `MatchingEngineTests.java` (nine tests, standard library only), `Main.java` (runs the sample — or a file given as the first argument — then the tests).

The full runnable source (including all nine tests) is in `code/lld/ch19/stockexchange/`; below is the heart of it.

```java
// File: lld/ch19/stockexchange/OrderBook.java
/** Who sets the trade price. The Navi problem uses the SELL order's price; real exchanges use the resting
order's. */
interface ExecutionPricePolicy {
    BigDecimal price(Order buy, Order sell, Order incoming);
    ExecutionPricePolicy SELL_PRICE = (buy, sell, incoming) -> sell.price();
    ExecutionPricePolicy RESTING_PRICE = (buy, sell, incoming) -> incoming == buy ? sell.price() : buy.price();
}

/** The book for ONE symbol: price level -> FIFO queue. Bids highest first, asks lowest first. */
final class OrderBook {
    private final NavigableMap<BigDecimal, Deque<Order>> bids = new TreeMap<>(Collections.reverseOrder());
    private final NavigableMap<BigDecimal, Deque<Order>> asks = new TreeMap<>();
    private final Map<String, Order> resting = new HashMap<>();      // id -> order, for cancel

    /** Match the incoming order against the other side, then rest whatever is left. */
    List<Trade> add(Order incoming) {
        List<Trade> trades = new ArrayList<>();
        NavigableMap<BigDecimal, Deque<Order>> opposite = incoming.side() == Side.BUY ? asks : bids;
        while (!incoming.isDone() && !opposite.isEmpty()) {
            Map.Entry<BigDecimal, Deque<Order>> best = opposite.firstEntry();
            if (!crosses(incoming, best.getKey())) break;          // best price does not meet ours: stop
            Deque<Order> queue = best.getValue();
            Iterator<Order> it = queue.iterator();
            while (!incoming.isDone() && it.hasNext()) {
                Order restingOrder = it.next();                    // oldest first = time priority
                int qty = Math.min(incoming.remainingQty(), restingOrder.remainingQty());
                Order buy = incoming.side() == Side.BUY ? incoming : restingOrder;
                Order sell = incoming.side() == Side.SELL ? incoming : restingOrder;
                trades.add(new Trade(buy.id(), pricePolicy.price(buy, sell, incoming), qty, sell.id()));
                incoming.fill(qty);
                restingOrder.fill(qty);
                if (restingOrder.isDone()) { it.remove(); resting.remove(restingOrder.id()); }
            }
            if (queue.isEmpty()) opposite.pollFirstEntry();
        }
        if (!incoming.isDone()) rest(incoming);                    // addLast to its own price level
        return trades;
    }

    private static boolean crosses(Order incoming, BigDecimal bestOpposite) {
        return incoming.side() == Side.BUY
                ? incoming.price().compareTo(bestOpposite) >= 0     // buy meets or beats the ask
                : incoming.price().compareTo(bestOpposite) <= 0;    // sell meets or undercuts the bid
    }
}

// File: lld/ch19/stockexchange/MatchingEngine.java - one lock per symbol
List<Trade> submit(String id, LocalTime time, String symbol, Side side, BigDecimal price, int qty) {
    if (qty <= 0 || price.signum() <= 0) throw new IllegalArgumentException("bad quantity/price: " + id);
    if (symbolOfOrder.putIfAbsent(id, symbol) != null) throw new IllegalArgumentException("duplicate order id " +
id);
    OrderBook book = books.computeIfAbsent(symbol, s -> new OrderBook(s, policy));
    List<Trade> trades;
    synchronized (book) {
        // the sequence is taken inside the lock, so arrival order == matching order
        Order o = new Order(id, time, symbol, side, price, qty, sequence.incrementAndGet());
        trades = book.add(o);
    }
    trades.forEach(tradeSink);        // publish outside the lock
    return trades;
}
// cancel(orderId) takes the same per-symbol lock; submitLine parses "#1 09:45 BAC sell 240.12 100"
```

`Main` runs the six sample lines (or a file given as the first argument) twice — once with `SELL_PRICE`, once with `RESTING_PRICE` — then `MatchingEngineTests.runAll()`.

Output:

```
== Trades (execution price = sell order's price) ==
#3 237.45 90 #2
#3 236.00 20 #6
#4 236.00 10 #6
#5 236.00 20 #6
book after: BAC bids=[237.80x20] asks=[240.12x100]

== Same input, real-exchange rule (price of the resting order) ==
#3 237.45 90 #2
#3 238.10 20 #6
#4 237.80 10 #6
#5 237.80 20 #6

== Tests ==
  PASS  sample input gives the expected four trades
  PASS  better price wins over earlier time
  PASS  same price: earlier order fills first
  PASS  no trade when prices do not cross
  PASS  partial fill leaves the remainder resting
  PASS  cancelled order is never matched
  PASS  symbols never match each other
  PASS  duplicate id and bad quantity rejected
  PASS  concurrent submits conserve quantity
  9/9 passed
```

Reading the output. The first block matches the expected output in the problem statement exactly. The second block runs the same orders with the real-exchange rule: #6 (the incoming sell) trades at each *resting* buyer's price (238.10, 237.80), which is how a continuous limit order book normally prices trades. The tests cover price priority, time priority, no-cross, partial fills, cancel, symbol isolation, validation, and a concurrency test (8 threads × 500 orders) that checks quantity is conserved and the book is never left crossed (best bid must stay below best ask).

Concurrency & edge cases

Risk	Where	Prevention
Over-spending with many open orders	Brokerage	`reserveFor` (synchronized on the member) subtracts all reservations from cash before accepting a new buy.
Selling the same shares twice	Brokerage	Open sell orders are tracked; free shares = position − remaining quantity of open sells *of that symbol*.
Deadlock between member and exchange	Brokerage	Lock order is fixed: the member never calls the exchange while holding its own lock (`place` reserves under the lock, then submits outside it). The exchange may call the member (exchange → member), never the reverse.
Market order costs more than reserved	Brokerage	Known gap (shown in step 3). Fix: reserve a buffer (e.g. +5%) or reject prices too far above the last price ("price band").
Time in force not enforced	Brokerage	The enum has five values (GTC, fill-or-kill, immediate-or-cancel, on-the-open, on-the-close) and no code reads them — every order behaves as good-till-cancelled within the demo. A GTC order should survive into the next trading day; FOK must be cancelled if it cannot be filled in full; IOC cancels whatever is left after one pass. All of this belongs in the exchange, next to the fill loop.
Two orders at the same price arrive "together"	Engine	Whichever thread gets the book's lock first is appended to the level's queue first, so lock-acquisition order = time priority. The `sequence` number is taken inside the same lock, so it records exactly that order (useful for logs and audits).
Cancel racing a match	Engine	Cancel takes the same per-symbol lock; an order is either matched or cancelled, never both.
Duplicate id / zero quantity / negative price	Engine	Validated before the id is registered (test `validation`).
Trade publishing slowing matching	Engine	Trades are collected under the lock and published after releasing it.
Floating-point prices	Both	`BigDecimal` everywhere; `TreeMap` keys compare with `compareTo`, so `237.8` and `237.80` land on the same level.

Real-world example

Indian brokers such as Zerodha offer the same stop order types to retail users (individual investors): **SL** (stop-loss *limit*) and **SL-M** (stop-loss *market*) — our `StopLimitOrder` and `StopLossOrder`. (Which segments allow SL-M has changed over time, so check the broker's current rules.) Their orders go to exchanges such as NSE (National Stock Exchange of India), whose continuous-trading order book matches on price-time priority, like our `OrderBook`. At the extreme end of performance, the LMAX exchange (described in Martin Fowler's article "The LMAX Architecture") runs its business logic, including matching, on a **single thread** that reads input events from a ring buffer (a fixed-size array used as a circular queue; LMAX's version is called the Disruptor) — the same idea as our "one lock per symbol, strictly serialised", taken to its logical end. On the tax side, US rules treat shares as sold first-in-first-out unless you identify specific lots, and FIFO is a common broker default — the reason `StockPosition.sell` takes an optional lot id.

Extensions & follow-up questions

- **"Add a trailing-stop order."** New subclass: it tracks the best price seen and moves its stop by a fixed distance. `canTradeAt` changes; nothing else does.
- **"Implement IOC and FOK."** IOC: after the fill loop, cancel any remainder. FOK: before filling, sum the quantity available at acceptable prices; if short, cancel without filling.
- **"Price alerts (the 'notify when a stock moves by X%' feature)."** A `PriceAlert` observer registered on `StockInventory`/`tick`; fires once when the move crosses the threshold.

- **"Modify an order."** On many exchanges, reducing quantity keeps time priority, but changing price (or increasing quantity) loses it — implement as cancel + new order.
- **"Make the engine faster."** Single writer thread per symbol fed by a queue (no locks on the hot path — the code that runs for every single order), pre-allocated objects, prices as `long` ticks instead of `BigDecimal`.
- **"Market orders in the engine."** Treat as a limit at +∞ (buy) or 0 (sell); never rest the remainder — cancel it.
- **"Settlement."** Trades settle later (for example T+1: one business day after the trade); add `unsettledCash` to the member so sale proceeds are not withdrawable immediately.

In the interview

30–45 minute plan (brokerage)

1. Clarify order types, lots, transfers, reports (3 min).
2. Actors and use cases (3 min).
3. Say the key sentence: *"The four order types differ in exactly two things: whether they may trade yet, and the worst price they accept."* Draw `Order` + four tiny subclasses (8 min).
4. Explain **lots** and why they exist: tax depends on which shares you sold (4 min).
5. Explain **reserved funds** (3 min).
6. Code `canTradeAt` / `limitPrice` for the four types and `StockPosition.sell` (10 min).
7. Observer callback, statements, extensions (5 min).

Machine-coding (Navi) plan, 90 minutes: parse lines → `Order`; `OrderBook` with two `TreeMap`s of `ArrayDeque`; `add()` matching loop; print trades; then tests for the sample, price priority, time priority, partial fill and cancel. Get the sample passing first, then refactor (restructure the code without changing what it does).

Follow-ups with answers

- *Why not four unrelated order classes?* They share member, symbol, quantity, side, status and fills; duplicating that invites bugs. The hierarchy isolates the two real differences.
- *Why is `StockInventory` separate from `StockExchange`?* One place to read prices, so no two components disagree about a price.
- *What data structure for the book, and the complexity?* `TreeMap` of price levels (a red-black tree — a self-balancing sorted tree): finding the best level with `firstEntry` and inserting a new level are both O(log L) for L price levels; FIFO queue per level: append O(1), match O(1) per fill. Cancel: O(log L) to find the level + O(k) in that level (use a linked structure with a handle for O(1)).
- *At what price does a trade execute?* In the Navi statement: the sell order's price. In real continuous trading: the resting order's price — the incoming order may get a better price than it asked for ("price improvement").

Self-check

1. **Q:** A SELL stop-loss at 10.00; the price goes 10.50 → 9.95 → 10.20. When does it trade? **A:** At the 9.95 tick. It stays triggered even when the price goes back up.
2. **Q:** What does a stop-limit add to a stop-loss? **A:** A worst acceptable price after triggering — it becomes a limit order, not a market order.
3. **Q:** Why does one market order create two lots in step 3? **A:** It filled at two prices, and a lot is defined by its purchase price.
4. **Q:** Two bids at 50.00 arrive, b1 then b2; a sell for 7 at 49 arrives. Trades? **A:** b1 gets 5, b2 gets 2 — time priority within the level (test `timePriority`).
5. **Q:** What does "the book is crossed" mean and why is it a bug? **A:** Best bid ≥ best ask; those two orders should already have traded.
6. **Q:** Which pattern produces the tax and quarterly statements? **A:** Template Method — shared filtering in `Statement.render`, specific lines in `body`.

TL;DR

- **Brokerage:** one abstract `Order`, four tiny subclasses answering *may I trade now?* and *worst price?*; fills are `OrderPart`s; purchases are **lots**; money is **reserved** at placement; the exchange calls back via **Observer**; statements are a **Template Method**.
- **Matching engine:** per symbol, two `TreeMap<price, FIFO queue>`; match best price first, oldest first; rest the remainder; one lock per symbol; trade price is a pluggable policy (Navi: sell price; real exchanges: resting price).

Design an Online Auction System

Problem & clarifying questions

An online auction (think eBay auctions) lets a **seller** list an item with a starting price and an end time. **Bidders** offer higher and higher amounts. When time runs out, the highest bidder wins — if the seller's hidden minimum (the **reserve price**) was reached.

Question	Assumed answer
What does a listing contain?	Item name, description, category, starting price, minimum bid increment, optional reserve price, duration.
What is a valid bid?	The first bid must be at least the starting price; later bids must beat the current highest by at least the increment.
Can the seller bid? Can the leader outbid themselves?	No and no.
When does an auction end?	At its end time. A late bid is refused even if the closing job has not run yet.
Who is told what?	The outbid bidder immediately; the winner and seller at close; the seller if unsold.
Proxy (automatic) bidding, anti-sniping (stopping last-second bids from stealing a win), payments?	Extensions — mention them.

Requirements

Functional

1. Users register and log in. (We model registration; login itself is the usual authentication (checking who a user is) service and is not coded here.)
2. Users create listings with item name, description, starting price and duration.
3. Users browse and search listings by name, category and price range.
4. Users place bids on **active** listings.
5. The system keeps the **current highest bid** up to date and notifies bidders.
6. When the duration ends, the auction closes and the **highest bidder wins** (if the reserve is met).

Non-functional

7. Many users bid on the same listing at the same moment; the system must stay consistent (one true highest bid).
8. Extensible: new bid rules, notification channels or search criteria without rewriting the core.

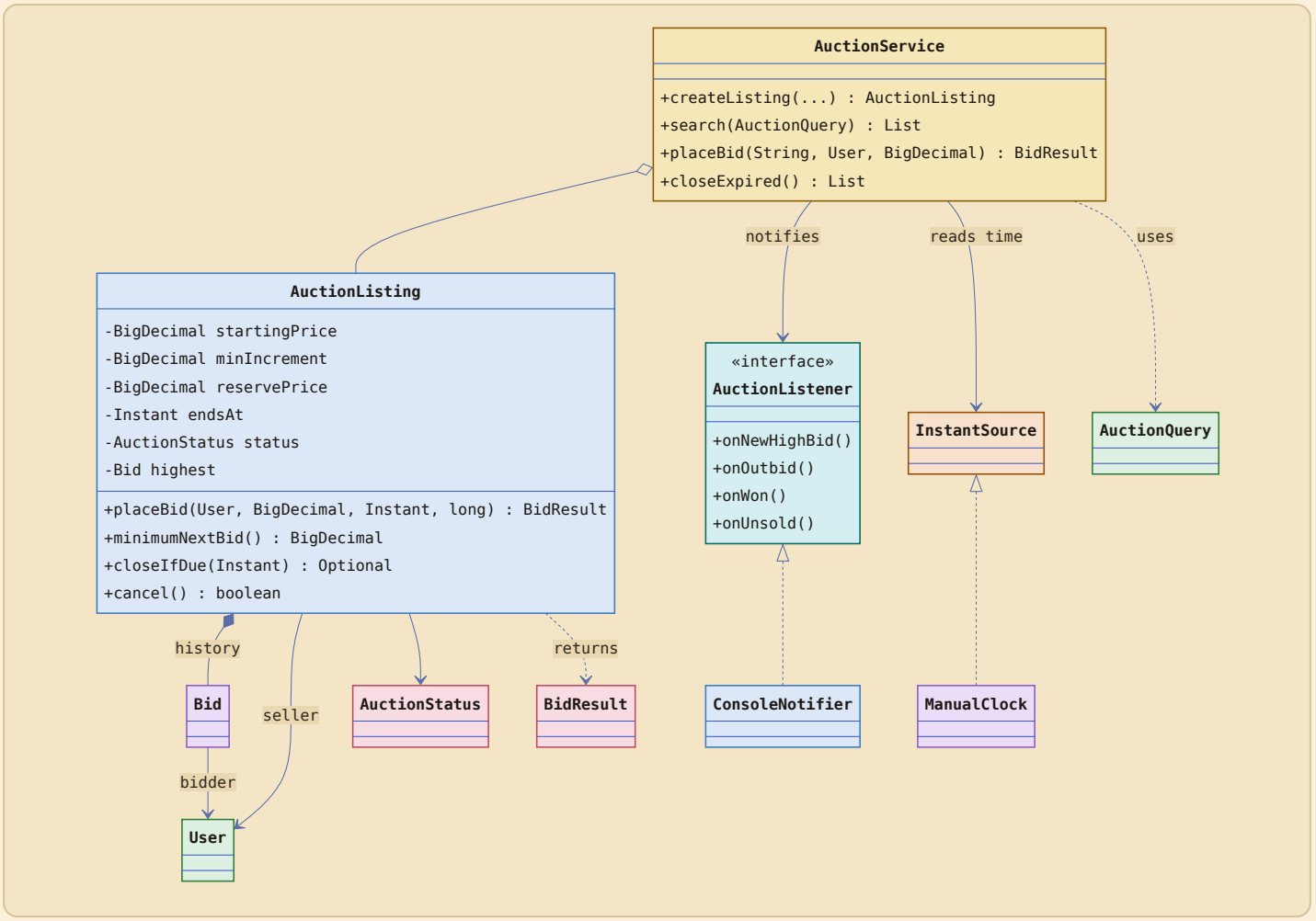
Use cases

Actor	Use cases
Seller	Create listing, cancel listing (only while nobody has bid), see result
Bidder	Search, view listing and bid history, place bid, be told when outbid or when won
System (scheduler)	Activate scheduled auctions, close expired ones, declare winner, send notifications

Core entities

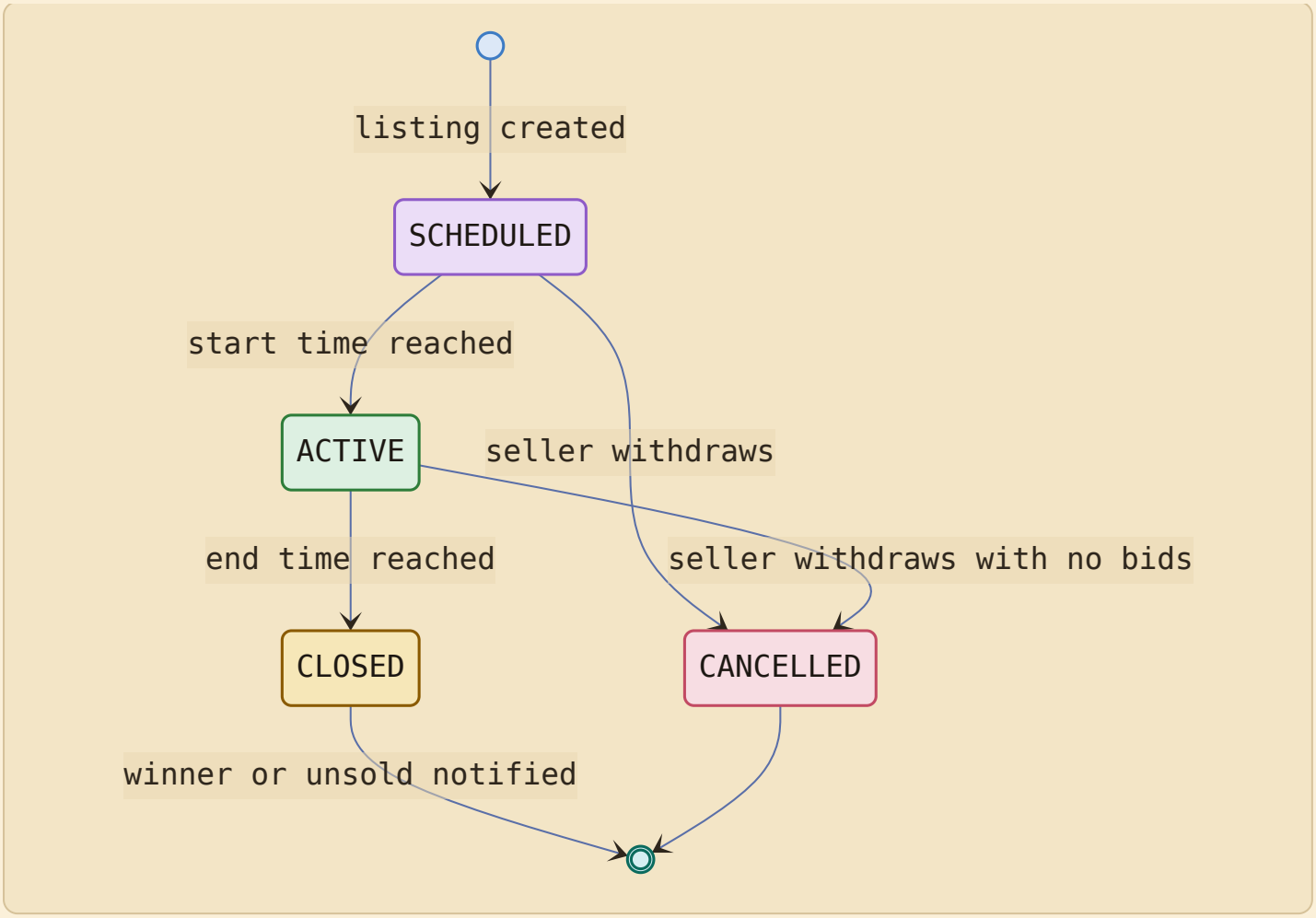
Class	Responsibility
`User`	id, name, email (a `record`).
`Bid`	bidder, amount (`BigDecimal`), time, sequence number. Immutable.
`AuctionStatus`	`SCHEDULED → ACTIVE → CLOSED`, or `CANCELLED`.
`AuctionListing`	Item details, seller, prices, times, bid history, highest bid. **Owns the bidding rules** and guards them with its own lock.
`BidOutcome` / `BidResult`	Why a bid was accepted or refused, the next minimum bid, the accepted bid, who was outbid.
`CloseResult`	Listing, winning bid (if any), whether the reserve was met.
`AuctionQuery`	Fluent (calls chained one after another) search criteria; each criterion is a `Predicate` (a function that answers true/false; Specification pattern).
`AuctionListener` / `ConsoleNotifier`	Observer: new high bid, outbid, won, unsold.
`AuctionService`	Facade: register, create, search, bid, close expired. Calls listeners **outside** listing locks.
`ManualClock`	An `InstantSource` (Java's "what time is it now?" interface) we can move forward in demos and tests.

Class diagram



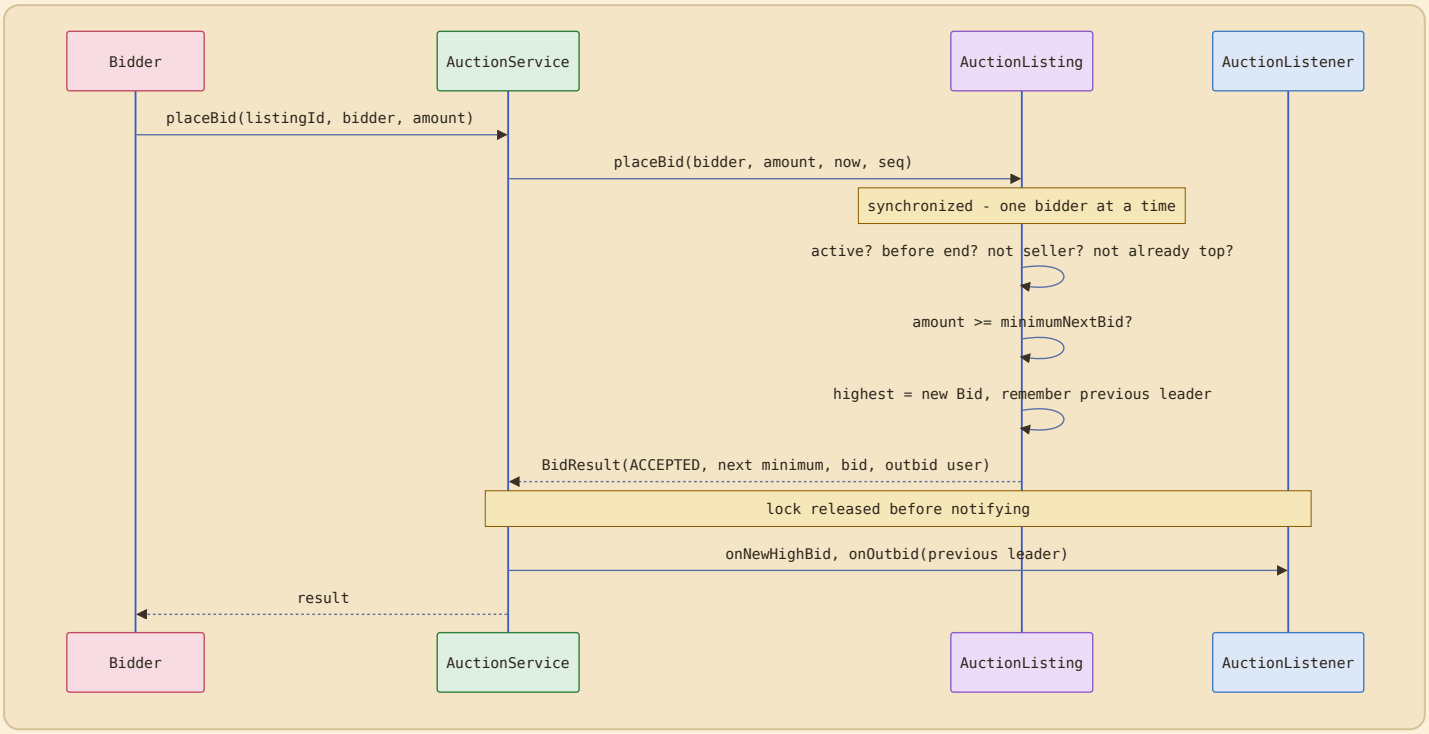
Key flows

Auction lifecycle



`SCHEDULED → ACTIVE` happens **lazily** (only when someone next touches the listing, not at the exact moment): every method first calls `refresh(now)`. No background thread is needed just to flip the state.

Placing a bid (the critical section) — a **critical section** is a piece of code that only one thread may run at a time because it reads and changes shared data.



Closing. A scheduler (in production a `ScheduledExecutorService` — Java's built-in timer that runs a task repeatedly — running every few seconds) calls `closeExpired()`. Each listing's `closeIfDue(now)` atomically

moves `ACTIVE → CLOSED` and returns the winner only if the reserve is met. Notifications go out after all locks are released.

Design decisions & patterns used

Decision	Why	Alternative rejected
Bid rules live inside `AuctionListing` and run under its lock (a *monitor*: an object whose methods are all `synchronized`)	"Read highest, compare, replace" must be one atomic step. The lock is per listing, so bids on different auctions never wait for each other.	A global lock in the service (all auctions serialised); or `AtomicReference<Bid>` (a reference that can be swapped atomically) + CAS loop (retry compare-and-set until it succeeds) — valid, but the rule set (seller, leader, end time, increment) is easier to get right under one lock.
Refusals are returned (`BidOutcome`), not thrown	A too-low bid is normal business, not an exceptional error (an **exception** is Java's way of aborting a method to signal that something went wrong). The caller also gets the next minimum to show in the UI.	Throwing an exception for every low bid.
Observer for notifications, called outside the lock	Slow email or push calls must not block other bidders. `BidResult` carries a **snapshot** (a copy frozen at one moment) of the accepted bid taken inside the lock, so listeners never read a newer, unrelated value.	Notifying inside `synchronized` (slow, and risks deadlock if a listener calls back).
Minimum increment	Stops 1-cent "wars" and makes the next minimum predictable.	Any higher amount accepted.
Reserve price	Lets a seller refuse to sell too cheaply; checked only at close.	Rejecting bids below the reserve (would reveal it).
`InstantSource` injected	Time-dependent code becomes testable: the demo moves a `ManualClock` forward instead of sleeping.	Calling `Instant.now()` everywhere.
Specification pattern for search	Each criterion is a `Predicate<AuctionListing>`; they combine with `and`. New criteria = one method.	One `search(name, category, min, max, ...)` method whose parameter list keeps growing.
The service is not a Singleton	awesome-low-level-design's version uses a Singleton `AuctionSystem`; we pass one instance around instead so tests can create fresh ones.	Global state shared by all tests.

Java implementation

Files: `Auction.java` (user, bid, status, outcome records, listing, manual clock), `AuctionService.java` (listener, notifier, query, service), `Main.java`.

The full runnable source is in `code/lld/ch19/auction/`; below are the lines that carry the design.

```java
// File: lld/ch19/auction/Auction.java
/** Why a bid was refused - returned, not thrown, because refusals are normal business. */
enum BidOutcome { ACCEPTED, AUCTION_NOT_ACTIVE, AUCTION_ENDED, SELLER_CANNOT_BID, TOO_LOW, ALREADY_HIGHEST }

/** One auction. Every state change is synchronized, so "read highest, compare, replace" is atomic. */
final class AuctionListing {
    private final BigDecimal reservePrice;          // hidden minimum; ZERO = no reserve
    private Bid highest;

    private void refresh(Instant now) {             // lazy SCHEDULED -> ACTIVE
        if (status == AuctionStatus.SCHEDULED && !now.isBefore(startsAt)) status = AuctionStatus.ACTIVE;
    }

    synchronized BidResult placeBid(User bidder, BigDecimal rawAmount, Instant now, long seq) {
        refresh(now);
        BigDecimal amount = money(rawAmount);
        if (status != AuctionStatus.ACTIVE) return refuse(BidOutcome.AUCTION_NOT_ACTIVE);
        if (!now.isBefore(endsAt)) return refuse(BidOutcome.AUCTION_ENDED);  // closing job may not have run yet
        if (bidder.id().equals(seller.id())) return refuse(BidOutcome.SELLER_CANNOT_BID);
        if (highest != null && highest.bidder().id().equals(bidder.id())) return
refuse(BidOutcome.ALREADY_HIGHEST);
        if (amount.compareTo(minimumNextBid()) < 0) return refuse(BidOutcome.TOO_LOW);

        Optional<User> outbid = Optional.ofNullable(highest).map(Bid::bidder);
        highest = new Bid(bidder, amount, now, seq);
        bids.add(highest);
        return new BidResult(BidOutcome.ACCEPTED, minimumNextBid(), Optional.of(highest), outbid);
    }

    /** First bid must reach the starting price; later bids must beat the leader by the increment. */
    synchronized BigDecimal minimumNextBid() {
        return highest == null ? startingPrice : highest.amount().add(minIncrement);
    }

    synchronized Optional<CloseResult> closeIfDue(Instant now) {
        refresh(now);
        if (status != AuctionStatus.ACTIVE || now.isBefore(endsAt)) return Optional.empty();
        status = AuctionStatus.CLOSED;
        boolean reserveMet = highest != null && highest.amount().compareTo(reservePrice) >= 0;
        return Optional.of(new CloseResult(this, reserveMet ? Optional.of(highest) : Optional.empty(),
reserveMet));
    }

    /** A seller may withdraw only while nobody has bid. */
    synchronized boolean cancel() {
        if (status == AuctionStatus.CLOSED || status == AuctionStatus.CANCELLED || highest != null) return false;
        status = AuctionStatus.CANCELLED;
        return true;
    }
}

// File: lld/ch19/auction/AuctionService.java
final class AuctionQuery {                          // Specification: each criterion is a Predicate
    AuctionQuery category(String c) { parts.add(a -> a.category().equalsIgnoreCase(c)); return this; }
    Predicate<AuctionListing> toPredicate() { return parts.stream().reduce(x -> true, Predicate::and); }
}

BidResult placeBid(String listingId, User bidder, BigDecimal amount) {    // AuctionService (facade)
    AuctionListing a = listings.get(listingId);
    BidResult r = a.placeBid(bidder, amount, clock.instant(), bidSeq.incrementAndGet());
    r.accepted().ifPresent(top -> {                 // snapshot taken inside the lock; notify outside it
        listeners.forEach(l -> l.onNewHighBid(a, top));
        r.outbid().ifPresent(u -> listeners.forEach(l -> l.onOutbid(a, u, top.amount())));
    });
    return r;
}
// closeExpired(): closeIfDue on every listing, then onWon / onUnsold after all locks are released
```

`Main` drives a `ManualClock` (an `InstantSource` it moves forward) instead of sleeping.

Output:

```
== 1. Create listings ==
  A1 'Vintage Guitar' now 100.00, A2 'Pocket Watch' now 50.00

== 2. Search (name 'watch', price 0-100) ==
  [A2 'Pocket Watch' now 50.00]

== 3. Bidding rules ==
  Ana bids 90 (below start) -> TOO_LOW (next minimum 100.00)
  Ana bids 100 -> ACCEPTED (next minimum 110.00)
  Ana bids 120 (already top) -> ALREADY_HIGHEST (next minimum 110.00)
  Bob bids 105 (increment is 10) -> TOO_LOW (next minimum 110.00)
    [notify Ana] you were outbid on Vintage Guitar, now 110.00
  Bob bids 110 -> ACCEPTED (next minimum 120.00)
  Sam bids on own item -> SELLER_CANNOT_BID (next minimum 120.00)
    [notify Bob] you were outbid on Vintage Guitar, now 150.00
  Ana bids 150 -> ACCEPTED (next minimum 160.00)
  Bob bids 60 on the watch -> ACCEPTED (next minimum 65.00)

== 4. One hour later: the watch closes, reserve (500) not met ==
    [notify Sam] Pocket Watch unsold (reserve not met)
  closed A2 reserveMet=false

== 5. 50 threads bid on the guitar at once ==
  accepted=28 (varies run to run), highest=650.00 by bidder49, history strictly increasing by >= increment: true

== 6. Time is up: the guitar closes and the winner is told ==
  late bid -> AUCTION_ENDED (next minimum 660.00)
    [notify bidder49] you WON Vintage Guitar for 650.00
    [notify Sam] Vintage Guitar sold to bidder49
  status CLOSED, final price 650.00
```

Reading the output. Step 3 exercises every rule: below start, leader re-bidding, below increment, seller bidding. The outbid notices print *before* the bidder's own result line because the service notifies before returning. Step 4 closes the watch as unsold because 60 < the 500 reserve. Step 5 fires 50 concurrent bids: how many are accepted depends on thread timing (a bid that arrives after a higher one is refused as too low), but the final highest is always the maximum, 650.00, and the history is always strictly increasing by at least the increment — the invariant (a rule that must always be true) the lock protects. Step 6 refuses a bid after the end time even though the closing job has not yet run.

Concurrency & edge cases

Case	Handling
Two bidders submit the same amount at the same time	The first one through the lock wins; the second is now below `minimumNextBid()` and gets `TOO_LOW`.
Bid arrives exactly at the end time	`!now.isBefore(endsAt)` → `AUCTION_ENDED`. The end instant is exclusive.
Close job runs late	Bids after `endsAt` are refused anyway, so a late job cannot change the winner.
Close and bid race	Both are `synchronized` on the listing; the bid either lands before the close (and counts) or after (and is refused).
Notification listener is slow or throws	Called outside the lock; in production, publish to a queue so failures are retried without affecting bidding.
Seller cancels after bids	Refused (`cancel()` returns false) — protects bidders.
Money precision	Amounts normalised to scale 2 with `BigDecimal`; compared with `compareTo`.
Very hot auction (thousands of bids per second)	One lock per listing is fine for one JVM (Java Virtual Machine — one running Java process); distributed: a single-writer per auction (partition bids by auction id — split them into separate streams so all bids for one auction go to the same place, e.g. a Kafka partition, Kafka being a widely used event-streaming system), or a conditional update `UPDATE ... SET high = ? WHERE id = ? AND high < ?`.

Real-world example

eBay auctions use **proxy bidding**: you enter the most you are willing to pay, and eBay bids on your behalf only as much as needed to stay ahead, by the current increment. eBay also supports hidden **reserve prices**, and its auctions end at a fixed time, which is why "sniping" (bidding in the last seconds) is common there. Some other auction sites use a **soft close** — a late bid extends the end time — precisely to defeat sniping.

Extensions & follow-up questions

- **"Add proxy (automatic) bidding."** Each bidder may register a `maxAmount`. After any accepted bid, inside the same lock, let the strongest proxy respond with `min(its max, competitor + increment)`. Ties go to the earlier proxy (use `Bid.seq`).
- **"Stop sniping."** Soft close: in `placeBid`, if an accepted bid arrives within the last N minutes, set `endsAt = endsAt + N` (make `endsAt` non-final, change under the same lock).
- **"Buy It Now."** A fixed price that closes the auction immediately if paid before the first bid.
- **"Dutch / sealed-bid / Vickrey auctions."** (Dutch: the price starts high and drops until someone accepts; sealed-bid: bidders cannot see each other's bids.) Extract an `AuctionRules` strategy: `validate(bid)`, `winner(bids)`, `price(bids)`. Vickrey (second-price sealed bid): highest bidder wins but pays the second-highest bid.
- **"Payment after winning."** Winner has 48 hours to pay (new state `AWAITING_PAYMENT`); if not, offer to the second-highest bidder.
- **"Scale the close job."** Keep a priority queue (or a sorted index) by `endsAt` so the scheduler only touches auctions that are due.

In the interview

30–45 minute plan

1. Clarify the bid rules (start price, increment, reserve, seller/leader restrictions) — 4 min.
2. Entities and the lifecycle state machine — 8 min.
3. Code `placeBid` under the listing lock and **say why the lock is per listing** — 12 min.

4. Closing + notifications outside the lock — 6 min.
5. Concurrency discussion and extensions (proxy bidding, soft close) — 10 min.

> **30-second answer:** "Each listing owns its bid rules and its highest bid, and `placeBid` is synchronized on the listing, so compare-and-replace is atomic and different auctions never contend. Refusals return a reason and the next minimum. The service notifies outbid users and, at close, the winner and seller, always after releasing the lock. Time comes from an injected clock, so closing and late bids are testable."

Follow-ups with answers

- *Why not `AtomicReference` + CAS?* Possible for the amount alone, but the rule also checks seller, leader and time. A tiny critical section under one lock is simpler and still fast because the lock is per listing.
- *How do you close auctions on time across servers?* A scheduler that owns a time-ordered index of end times; the close itself is idempotent (`closeIfDue` returns empty the second time), so running it twice is harmless.
- *How do you show a live price to thousands of viewers?* Push `onNewHighBid` events to a pub/sub channel (publish/subscribe: senders post messages to a topic and every subscriber receives them) and WebSockets (a connection the browser keeps open so the server can push data at any time); viewers read a cache (a fast saved copy of data — see System Design Fundamentals), not the lock-holding object.

Self-check

1. **Q:** What two conditions make a bid `TOO_LOW`? **A:** Below the starting price when there is no bid, or below highest + increment otherwise.
2. **Q:** Why are listeners called after `placeBid` returns from the listing? **A:** To keep the lock short and avoid deadlock or slowdown from notification code.
3. **Q:** A bid arrives at 12:00:00.000 and the auction ends at 12:00:00.000. Accepted? **A:** No — the end time is exclusive.
4. **Q:** Where does the reserve price matter? **A:** Only at close: if the highest bid is below it, the item is unsold.
5. **Q:** What is the Specification pattern here? **A:** Search criteria as composable `Predicate`s combined with `and`.

TL;DR

> - One **lock per listing**; all bid rules inside it; refusals are values, not exceptions.
> - Lifecycle `SCHEDULED → ACTIVE → CLOSED | CANCELLED`; activation is lazy, closing is idempotent.
> - **Observer** notifications after the lock; **injected clock** for testable time; **Specification** for search.

Design a Music Streaming Service (Spotify)

Problem & clarifying questions

A music streaming service lets users search a huge catalog, build playlists, and play songs with the usual controls (play, pause, skip, previous, seek, shuffle, repeat). Free and paid plans behave differently. The service suggests new music.

Question	Assumed answer
What can users search?	Songs, albums, artists.
Playlists?	Users create, reorder and edit their own playlists.
Playback controls?	Play, pause, resume, skip, previous, seek; shuffle on/off; repeat off / all / one.
Plans?	FREE (ads, limited skips per hour, no downloads) and PREMIUM (no ads, unlimited skips, offline).
Recommendations?	A stub (a simple placeholder) based on listening history; real ones are an ML (machine learning — software that learns patterns from data) system.
Actual audio streaming, DRM (copy protection), CDNs (content delivery networks — servers near users that hold copies of files)?	Out of scope for LLD (low-level design: classes and their code) — the player models *what* plays, not the bytes.

Requirements

Functional

1. Browse and search songs, albums and artists.
2. Create and manage playlists.
3. User accounts and authentication (we model users and plans; login itself is the usual auth service).
4. Playback controls: play, pause, skip, seek (plus previous, shuffle, repeat).
5. Recommendations based on preferences and listening history.
6. Subscriptions: FREE vs PREMIUM with different limits.

Non-functional

7. Many users stream at once; the shared catalog is read-mostly and thread-safe (works correctly when many threads use it at once).
8. Scales to a large catalog and user base (indexes, not scans, in production).
9. Extensible: social sharing, offline mode, new play orders, new recommenders.

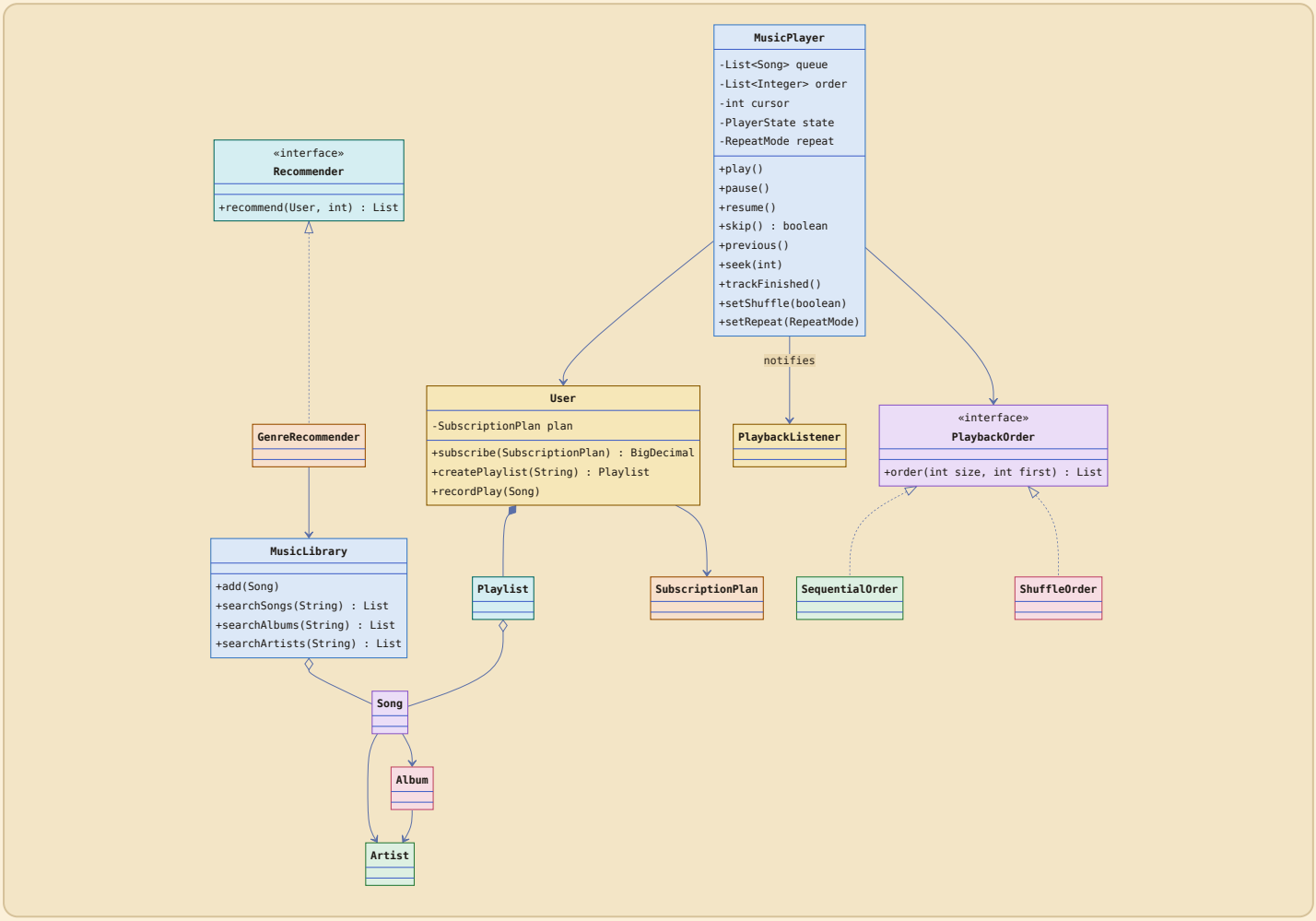
Use cases

Actor	Use cases
Listener	Search, play/pause/skip/seek, shuffle/repeat, create and edit playlists, upgrade plan
Artist / label (content admin)	Add songs and albums to the catalog
System	Record plays (history, royalties — money owed to artists per play), insert ads for free users, recommend songs

Core entities

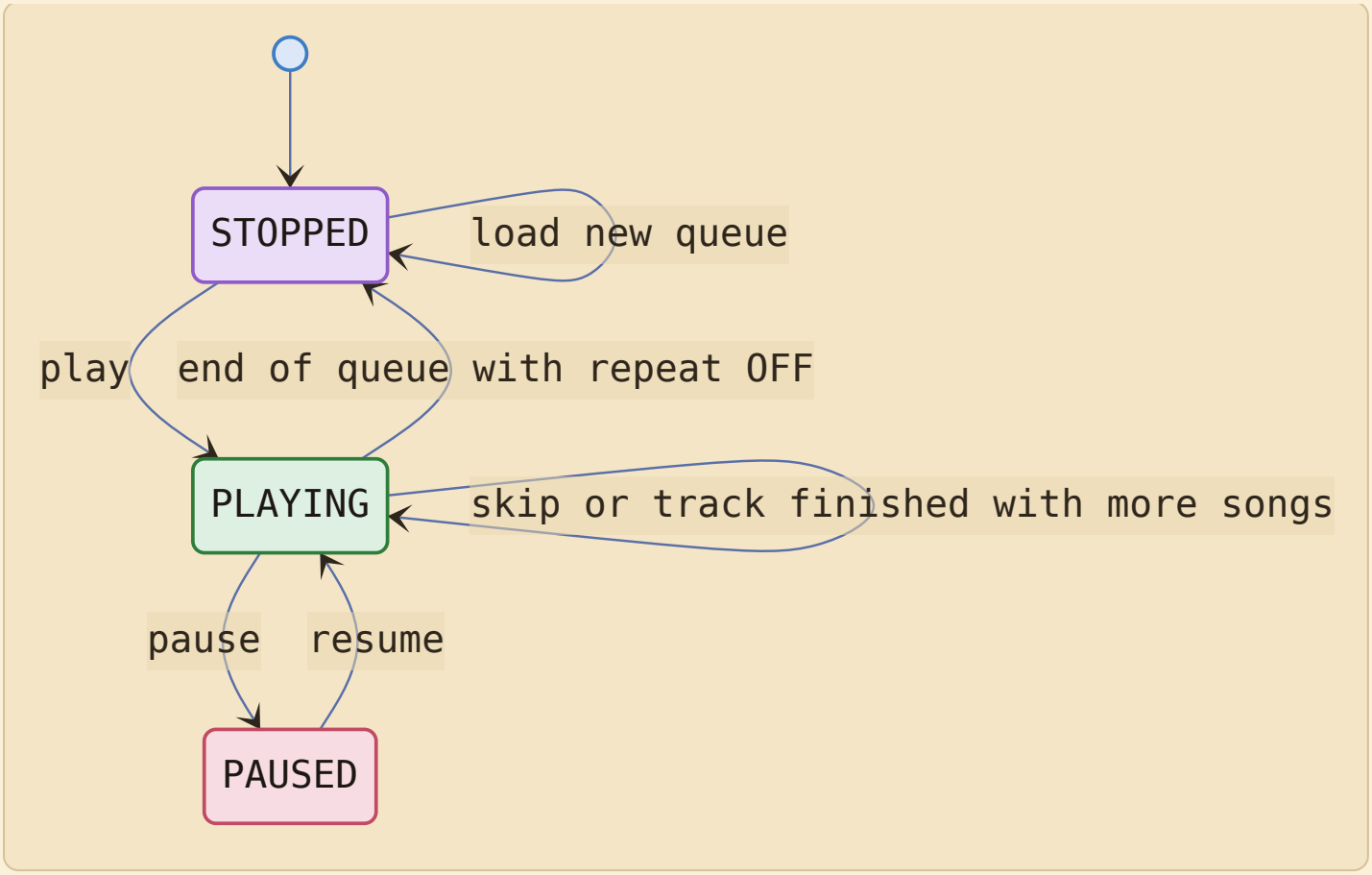
Class	Responsibility
`Artist`, `Album`, `Song`, `Genre`	Immutable catalog records.
`MusicLibrary`	Thread-safe catalog with search by song title, album title, artist name.
`SubscriptionPlan`	Enum that **carries the rules**: monthly price, skips per hour, ads, offline downloads.
`User`	Name, current plan, playlists, listening history.
`Playlist`	Ordered, user-owned song list: add, remove, move.
`PlaybackOrder` → `SequentialOrder`, `ShuffleOrder`	Strategy: produces the play order (a permutation — every song exactly once, in some order) starting from the current song.
`RepeatMode`, `PlayerState`	`OFF / ALL / ONE`; `STOPPED / PLAYING / PAUSED`.
`MusicPlayer`	One user's session: queue, play order, cursor (where we are in the play order), position, state, skip window, ad counter.
`PlaybackListener`	Observer on "song started" (history, royalties, recommendations).
`Recommender` → `GenreRecommender`	Strategy: suggest unplayed songs from the user's favourite genres.

Class diagram

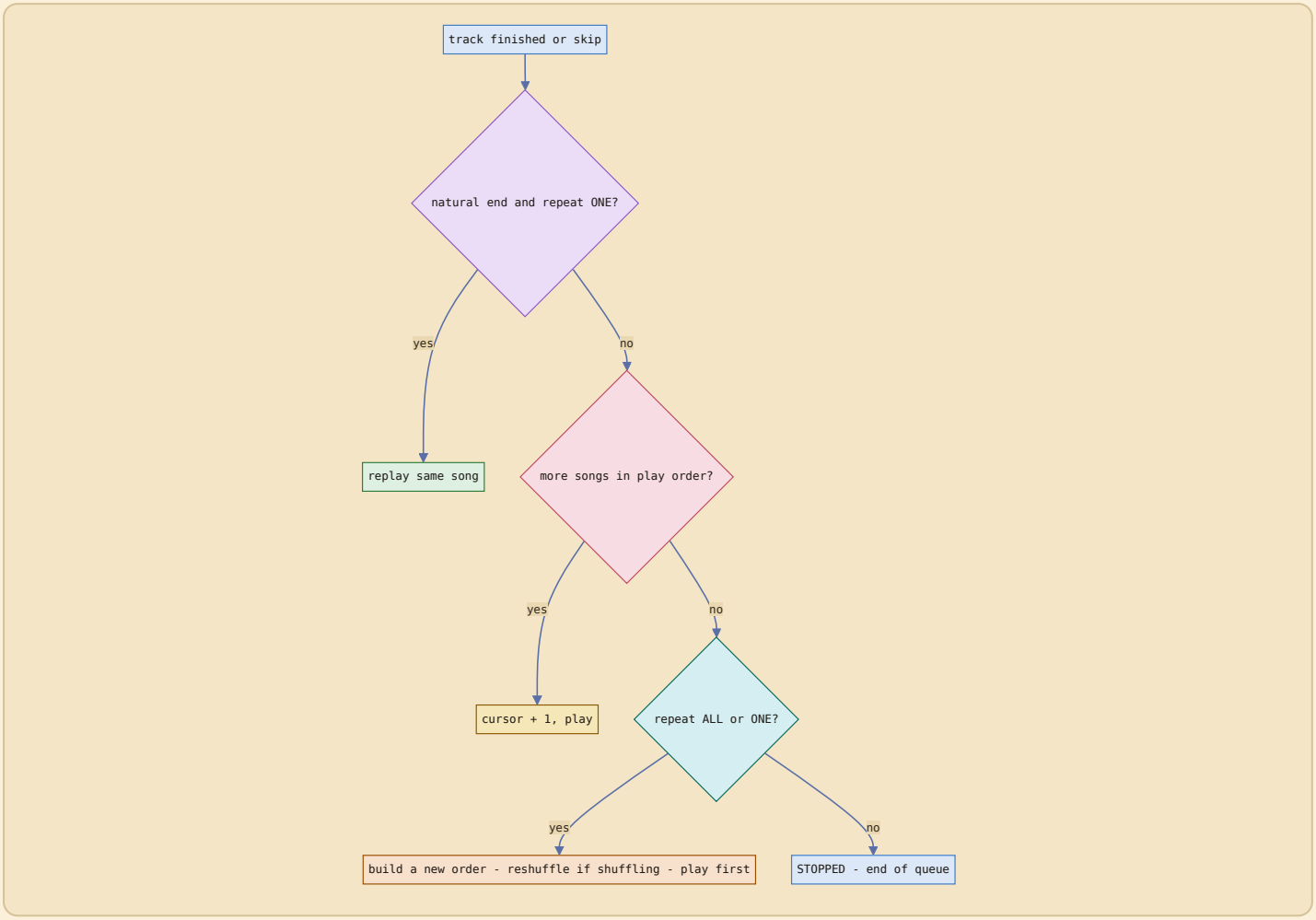


Key flows

Player states



What happens when a song ends or the user presses Next



Skip first checks the plan's rolling (always the last 60 minutes, not a fixed clock hour) one-hour limit (a deque — a queue you can add to or remove from at both ends — of skip timestamps; drop those older than an hour, compare the size with `skipsPerHour`). Pressing Next on repeat-ONE moves on — like real players, repeat-one only affects songs that end naturally.

Shuffle keeps the current song. Turning shuffle on builds a new permutation whose first element is the song playing now, and shuffles the rest with `Collections.shuffle(list, random)`. Turning it off rebuilds the sequential order starting from the current song. The queue itself never changes — only the **order** of indexes into it — so switching back and forth loses nothing.

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for play order (`PlaybackOrder`)	Shuffle, sequential, "smart shuffle" (spread the same artist apart) are interchangeable algorithms. The player just asks for an order.	`if (shuffle) ... else ...` inside `skip`, `previous`, `wrap`.
Order as a list of indexes into an unchanged queue	Turning shuffle off/on never loses the original order; `upNext` is a simple walk.	Physically shuffling the queue list (original order lost).
Seeded `Random`	Repeatable demos and tests. Production uses an unseeded (or per-session) seed.	`Math.random()` (not testable).
Plan rules inside `SubscriptionPlan` enum	`user.plan().skipsPerHour` and `.ads` — no `if (plan == FREE)` scattered around; a new plan is one enum constant (or a class if rules get complex).	Boolean flags on `User`.
Observer for plays (`PlaybackListener`)	History, royalty counting and recommendation features all react to "song started" without the player knowing them.	Player calling each subsystem directly.
Strategy for recommendations	`Recommender` can be genre-based today, collaborative filtering ("people who liked what you like also liked…") tomorrow.	Hard-coding one algorithm in `User`.
One `MusicPlayer` per user session, all methods `synchronized`	Commands can arrive from two devices (phone + desktop "connect"). The shared `MusicLibrary` uses concurrent maps.	A single shared player object.
State as an enum, not the State pattern	Three states with trivial behaviour; the GoF ("Gang of Four", the authors of the classic 1994 *Design Patterns* book) State pattern (one class per state) pays off only when each state has substantial, different behaviour.	Three state classes for four one-line methods.
Singletons?	awesome-low-level-design makes `MusicLibrary`, `UserManager` and `MusicRecommender` Singletons. We inject single instances instead — same sharing, easier tests.	Global `getInstance()`.

Java implementation

Files: `Catalog.java` (genre, artist, album, song, library, plans, user, playlist), `Player.java` (play-order strategies, repeat/state enums, listener, player, recommender), `Main.java`.

The full runnable source is in `code/lld/ch19/music/`; below are the lines that carry the design.

```java
// File: lld/ch19/music/Catalog.java - the plan carries its own rules
enum SubscriptionPlan {
    FREE(new BigDecimal("0.00"), 6, true, false),
    PREMIUM(new BigDecimal("10.99"), Integer.MAX_VALUE, false, true);
    final BigDecimal monthlyPrice;
    final int skipsPerHour;
    final boolean ads;
    final boolean offlineDownloads;
}

// File: lld/ch19/music/Player.java
interface PlaybackOrder {                       // Strategy: a permutation of 0..size-1 starting with first
    List<Integer> order(int size, int first);
}
final class SequentialOrder implements PlaybackOrder {
    @Override public List<Integer> order(int size, int first) {
        List<Integer> out = new ArrayList<>(IntStream.range(0, size).boxed().toList());
        Collections.rotate(out, -first);        // start at the current track, keep the rest in order
        return out;
    }
}
final class ShuffleOrder implements PlaybackOrder {
    ShuffleOrder(long seed) { this.random = new Random(seed); }   // seeded: repeatable demo and tests
    @Override public List<Integer> order(int size, int first) {
        List<Integer> rest = new ArrayList<>(IntStream.range(0, size).filter(i -> i != first).boxed().toList());
        Collections.shuffle(rest, random);
        rest.add(0, first);                     // the song you are hearing keeps playing
        return rest;
    }
}

final class MusicPlayer {                       // one per user session; every public method synchronized
    private List<Song> queue = List.of();
    private List<Integer> order = List.of();    // indexes into the unchanged queue
    private int cursor;                         // index into order

    /** User pressed "next". Free users have a skip limit per rolling hour. */
    synchronized boolean skip() {
        if (state == PlayerState.STOPPED) return false;
        Instant now = clock.instant();
        while (!recentSkips.isEmpty() && recentSkips.peekFirst().isBefore(now.minus(Duration.ofHours(1))))
recentSkips.pollFirst();
        if (recentSkips.size() >= user.plan().skipsPerHour) {
            System.out.println("  [player] skip limit reached on " + user.plan() + " plan");
            return false;
        }
        recentSkips.addLast(now);
        advance(false);
        return true;
    }

    synchronized void trackFinished() { advance(true); }   // repeat-one honoured only on a natural end

    synchronized void setShuffle(boolean on) {
        int now = order.isEmpty() ? 0 : order.get(cursor);
        playbackOrder = on ? new ShuffleOrder(42) : new SequentialOrder();
        order = playbackOrder.order(queue.size(), now);
        cursor = 0;
    }

    private void advance(boolean natural) {
        if (natural && repeat == RepeatMode.ONE) { start(); return; }
        if (cursor + 1 < order.size()) { cursor++; start(); return; }
        if (repeat == RepeatMode.ALL || repeat == RepeatMode.ONE) {   // wrap around (reshuffle if shuffling)
            order = playbackOrder.order(queue.size(), order.get(0));
            cursor = 0;
            start();
            return;
        }
        state = PlayerState.STOPPED;
```

```
            System.out.println("  [player] end of queue");
        }

        private void start() {                      // FREE plan: an ad every few songs; then notify PlaybackListeners
            if (user.plan().ads && ++songsSinceAd > 3) {
                System.out.println("  [player] (30-second ad for FREE plan)");
                songsSinceAd = 1;
            }
            positionSec = 0;
            state = PlayerState.PLAYING;
            Song s = current();
            System.out.println("  [player] > " + s);
            listeners.forEach(l -> l.onStart(user, s));
        }
    }
    // previous(): restarts the song if more than 3 s in (or at the first song), else goes back one.
    // GenreRecommender: count the user's history per Genre, suggest unheard songs from the top genres.
```

Output:

```
== 1. Search ==
  songs 'blue'    -> [All Blues - Miles Davis, Blue in Green - Miles Davis]
  albums 'opera'  -> [A Night at the Opera]
  artists 'da'    -> [Miles Davis]

== 2. Playlist ==
  Road trip: [Bohemian Rhapsody - Queen, Blue in Green - Miles Davis, So What - Miles Davis, Hello - Adele, Love
of My Life - Queen]

== 3. Play in order, pause, seek, repeat-one ==
  [player] > Bohemian Rhapsody - Queen
  state PAUSED at 120s
  [player] > Bohemian Rhapsody - Queen
  [player] > Blue in Green - Miles Davis

== 4. Shuffle keeps the current song, reorders the rest ==
  now Blue in Green - Miles Davis, up next [Love of My Life - Queen, So What - Miles Davis, Bohemian Rhapsody -
Queen, Hello - Adele]

== 5. FREE plan: skip limit (6/hour) and ads, repeat-all wraps the shuffled queue ==
  [player] (30-second ad for FREE plan)
  [player] > Love of My Life - Queen
  [player] > So What - Miles Davis
  [player] > Bohemian Rhapsody - Queen
  [player] (30-second ad for FREE plan)
  [player] > Hello - Adele
  [player] > Blue in Green - Miles Davis
  [player] > Hello - Adele
  [player] skip limit reached on FREE plan

== 6. Upgrade to PREMIUM: no ads, unlimited skips ==
  charged $10.99
  [player] > Death on Two Legs - Queen
  [player] > Freddie Freeloader - Miles Davis
  [player] > Death on Two Legs - Queen
  [player] > Freddie Freeloader - Miles Davis
  [player] end of queue

== 7. Recommendations from listening history ==
  plays so far: 13
  recommended: [You're My Best Friend - Queen, All Blues - Miles Davis, When We Were Young - Adele]
```

Reading the output. Step 3: repeat-ONE replays "Bohemian Rhapsody" when it ends naturally. Step 4: shuffle keeps "Blue in Green" playing and reorders the rest. Step 5: on the FREE plan an ad plays after every three songs, repeat-ALL wraps and reshuffles the queue, and the seventh skip within the hour is refused. Step 6: PREMIUM has no ads; with repeat-ALL the second skip wraps to the start; with repeat OFF the queue ends. Step 7: the recommender suggests songs the user has **not** heard, from the genres they played most.

Concurrency & edge cases

Case	Handling
Two devices send "next" at the same instant	Player methods are `synchronized`; the two skips apply one after the other.
Catalog updated while users search	`ConcurrentHashMap` (a thread-safe hash map) for songs/albums/artists; search iterates a weakly consistent view (it may or may not show changes made during the loop, but never breaks).
Playlist edited while it is playing	`load` copies the playlist (`List.copyOf`), so the running queue is a snapshot; a real app would sync edits into "up next".
Seek past the end or before 0	Clamped to `[0, duration]`.
Previous after 3 seconds	Restarts the current song (common player behaviour); within 3 seconds goes back one song.
Skip when stopped	Refused (and does not count against the limit).
Plan changes mid-session	`plan` is `volatile` (a Java keyword that makes every thread see the latest written value); the next skip or ad check sees it.
Empty queue	`play()` throws `IllegalStateException`.

Real-world example

Spotify's free tier has historically limited skips (for example, six per hour on mobile) and inserts ads between songs, while Premium removes both and allows downloads — the rules our `SubscriptionPlan` enum carries. Spotify engineers also wrote (2014) that users complained a truly random shuffle "felt" non-random when the same artist came up twice in a row, so they changed the algorithm to spread songs by the same artist across the list. That is exactly the kind of change the `PlaybackOrder` strategy absorbs: a new `ArtistSpreadShuffle` class, no player changes. Personalised playlists such as Discover Weekly are the industrial version of our `Recommender` stub.

Extensions & follow-up questions

- **"Add 'play next' and 'add to queue'."** Keep a separate *user queue* that is consumed before the context (playlist/album) order.
- **"Offline mode."** `SubscriptionPlan.offlineDownloads`; a `DownloadManager` stores encrypted files; the player chooses a local or remote `AudioSource` (Strategy).
- **"Social sharing / collaborative playlists."** Playlist gets a collaborator set and an edit log; listeners (Observer) push changes to other users.
- **"Crossfade, gapless playback."** Player behaviour around `trackFinished`; a `TransitionStrategy`.
- **"Royalties."** A `PlaybackListener` that records a play once the user has listened for a threshold time.
- **"Search at scale."** Inverted index (a map from each word to the items containing it) / search service with prefix (typeahead — suggestions as you type) and fuzzy matching (tolerating typos); the `MusicLibrary` interface stays.
- **"Smart shuffle."** New `PlaybackOrder` that spaces out songs by the same artist.

In the interview

30–45 minute plan

1. Clarify controls, plans and scope (skip audio bytes) — 4 min.
2. Catalog entities + library search — 6 min.
3. User, playlist, subscription plan — 5 min.
4. **Player**: queue + order + cursor; state machine; shuffle/repeat as strategy/enum — 15 min.
5. Observer for history; recommender strategy; concurrency; extensions — 10 min.

30-second answer: "Immutable catalog records in a thread-safe library. Each user has a player session holding the queue, a play order produced by a `PlaybackOrder` strategy, a cursor and a repeat mode. Shuffle rebuilds the order around the current song. Plan limits live in the plan enum. A playback listener records history, which feeds a pluggable recommender."

Follow-ups with answers

- *How does shuffle avoid repeating a song before all have played?* It is a permutation (each index once), not "pick a random next song".
- *Where do ads fit?* The player asks the plan whether ads apply and inserts one every N songs; in production an ad service picks which ad.
- *How would you implement "previous" in shuffle mode?* Walk back along the same order list (the cursor), so previous is the song you actually heard before.

Self-check

1. **Q:** Why store the play order as indexes rather than reordering the queue? **A:** So switching shuffle off restores the original order and nothing is lost.
2. **Q:** What does repeat-ONE do when the user presses Next? **A:** Moves to the next song; it only replays on natural end.
3. **Q:** Which pattern lets you add a smarter shuffle without touching the player? **A:** Strategy (`PlaybackOrder`).
4. **Q:** How is "6 skips per hour" enforced? **A:** A deque of skip times; drop entries older than one hour; refuse if the size has reached the limit.
5. **Q:** Why is the player's state an enum rather than the State pattern? **A:** The states have almost no distinct behaviour; the pattern would add classes without benefit.

TL;DR

- Catalog = immutable records in a concurrent library.
- Player = **queue + order (Strategy) + cursor + repeat mode + state**; shuffle keeps the current song.
- Plan rules in the **enum**; plays published via **Observer**; recommendations via **Strategy**.

Design an Online Food Delivery Service (Swiggy) + FoodKart

This problem also comes in two shapes:

- **Part A — Swiggy-style delivery platform** (the awesome-low-level-design problem): restaurants with menus, a cart, orders with tracking states, payments, and **assigning a delivery rider** by a pluggable strategy. We add **kitchen capacity** (a restaurant can only cook so many items at once), the key twist in a commonly reported Flipkart food-ordering variant (it is not part of the FoodKart statement below).
- **Part B — FoodKart** (Flipkart machine-coding round, from kumaransg/LLD): a smaller, command-driven problem — users and pincodes (Indian postal codes), one-dish restaurants, ratings, `show_restaurant` sorted by rating or price, `place_order` — plus a **restaurant selection strategy** (highest rating / lowest price) filtered by remaining capacity.

Problem & clarifying questions

Question	Assumed answer
Can one cart mix restaurants?	No — one restaurant per cart (as in Swiggy/Zomato).
Who confirms an order?	The restaurant; it may refuse if its kitchen is full.
When is capacity freed?	When the food is ready (handed over), not when it is delivered.
How is a rider chosen?	By a strategy: nearest to the restaurant, or fair share (fewest deliveries today).
What if no rider is free?	The order waits in a queue; the next freed rider takes the oldest waiting order.
Can a customer cancel?	Only before the kitchen starts cooking (PENDING or CONFIRMED).
Payment types?	UPI, card, cash on delivery (COD: nothing to refund).
Live GPS, ETA (estimated time of arrival) prediction, batching two orders per rider, surge fees (higher fees at busy times)?	Extensions.

Requirements

Functional (Part A)

1. Customers browse restaurants, view menus and place orders.
2. Restaurants manage menus, prices and availability.
3. Delivery agents accept and fulfil orders.
4. Customers track order status and get notified of changes.
5. Several payment methods.
6. Real-time notifications to customers, restaurants and delivery agents.

Functional (Part B — FoodKart)

7. `register_user(name, gender, phone, pincode)` — phone is unique.
8. `login_user(phone)` — sets the context; logging in someone else logs out the previous user.
9. `register_restaurant(name, pincodes "A/B", dish, price, quantity)` — in the context of the logged-in user (the owner). A restaurant serves **one** dish to **many** pincodes.
10. `update_quantity(name, add)`, `update_location(name, pincodes)` — owner only.
11. `rate_restaurant(name, 1..5, optional comment)`; rating = average of all ratings.
12. `show_restaurant(rating|price)` — **serviceable** restaurants (deliver to my pincode **and** quantity > 0), sorted **descending** by the chosen field.
13. `place_order(name, qty)`; bonus: order history.
14. *(Variant)* place an order without naming a restaurant: pick one by a **selection strategy**, among those that deliver here **and** have capacity for the whole quantity.

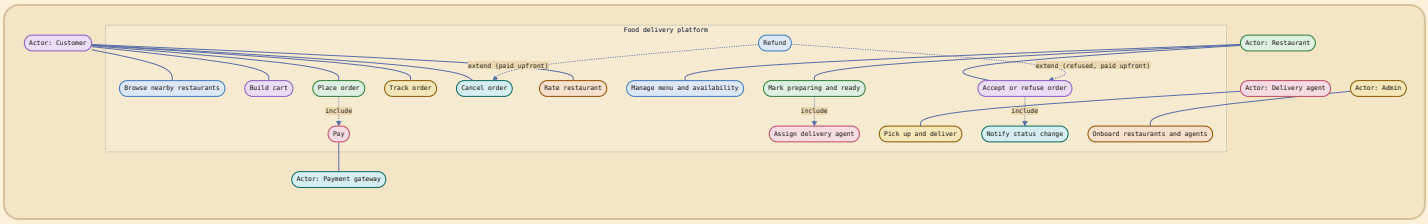
Non-functional

- Handle many concurrent orders without double-booking kitchen capacity or riders.
- Keep data consistent (no negative stock, no rider on two orders).
- In-memory only for the machine-coding round; demoable driver (a small `Main` program that feeds in commands and prints results); graceful failures.

Use cases

Notation (system box, include/extend arrows) as in §3.10. The **Admin** onboards restaurants and delivery agents (`addRestaurant`, `addAgent`); the **Payment gateway** is the outside service that charges UPI and

cards.

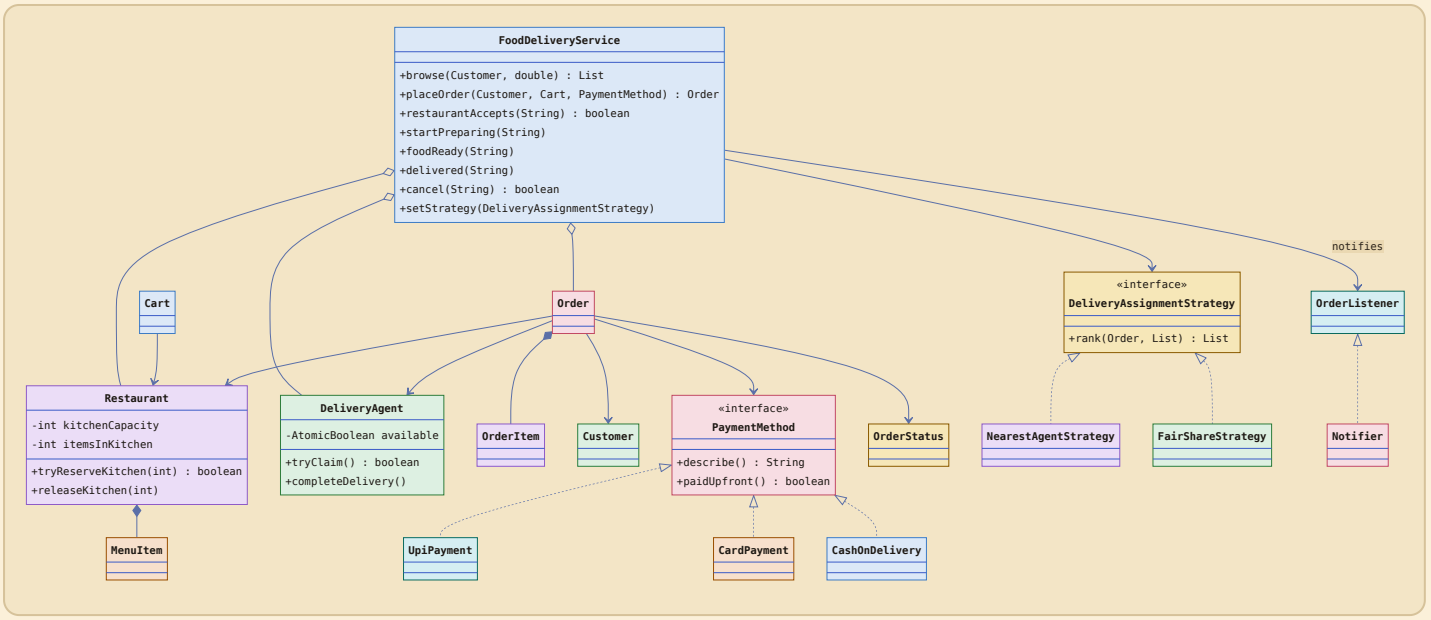


"Assign delivery agent" has no human actor: the platform runs it when food is ready or a rider becomes free. Every status change also includes "Notify status change"; only one such arrow is drawn to keep the picture readable.

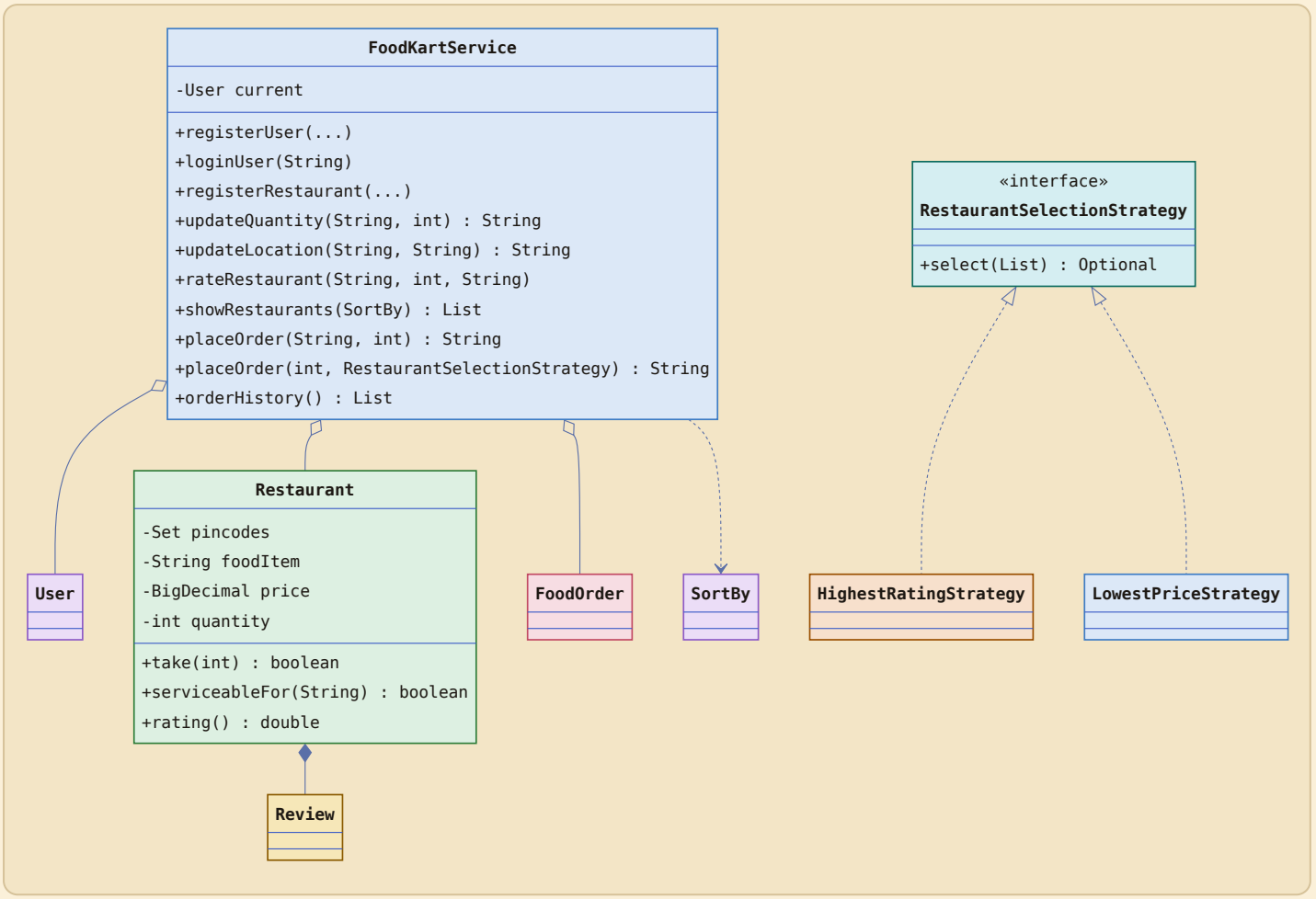
Core entities

Class	Responsibility
`Location`	A point; `distanceTo`.
`Customer`	id, name, phone, address.
`MenuItem`	Name, price, available flag (both changeable by the restaurant).
`Restaurant`	Menu, open flag, **kitchen capacity** (max items cooking at once), `tryReserveKitchen` / `releaseKitchen`.
`Cart`	One restaurant + item quantities; refuses items from a second restaurant.
`OrderItem`	Snapshot of item name, **unit price at order time**, quantity.
`OrderStatus`	State machine: PENDING → CONFIRMED → PREPARING → READY_FOR_PICKUP → OUT_FOR_DELIVERY → DELIVERED, or CANCELLED.
`Order`	Customer, restaurant, items, delivery fee, payment, status history, assigned agent.
`PaymentMethod` → `UpiPayment`, `CardPayment`, `CashOnDelivery`	Sealed interface; COD is not paid upfront.
`DeliveryAgent`	Location, `AtomicBoolean available` (a true/false flag that can be checked-and-changed atomically), deliveries today; `tryClaim()` is a CAS.
`DeliveryAssignmentStrategy` → `NearestAgentStrategy`, `FairShareStrategy`	Ranks free riders for an order.
`OrderListener` / `Notifier`	Observer for status changes.
`FoodDeliveryService`	Facade: browse, place, accept, prepare, ready, deliver, cancel; the waiting-for-rider queue.
(FoodKart) `User`, `Restaurant`, `Review`, `FoodOrder`, `SortBy`, `RestaurantSelectionStrategy`, `FoodKartService`	The machine-coding model; `Restaurant.take(q)` is the atomic capacity check.

Class diagram

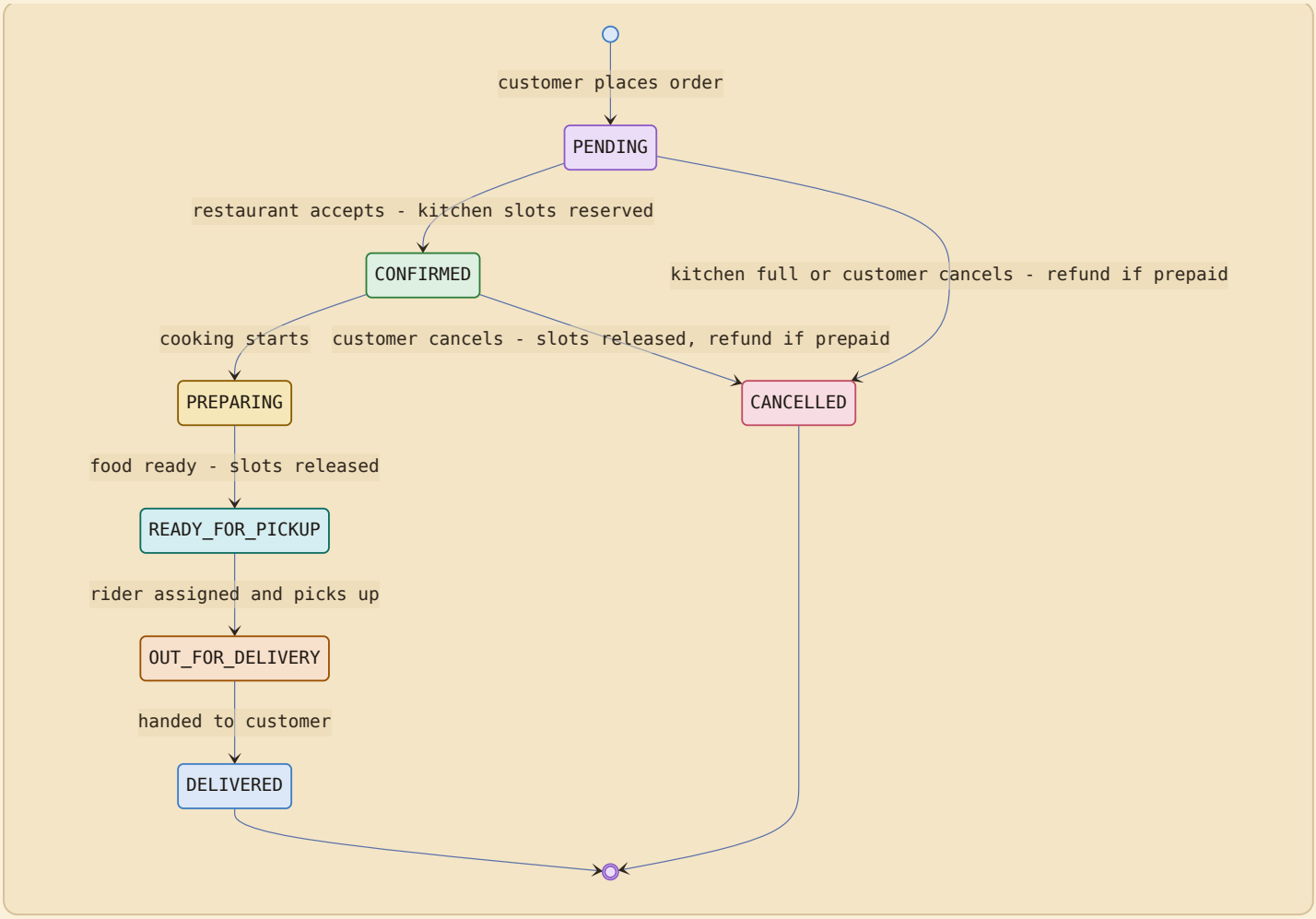


FoodKart (Part B):



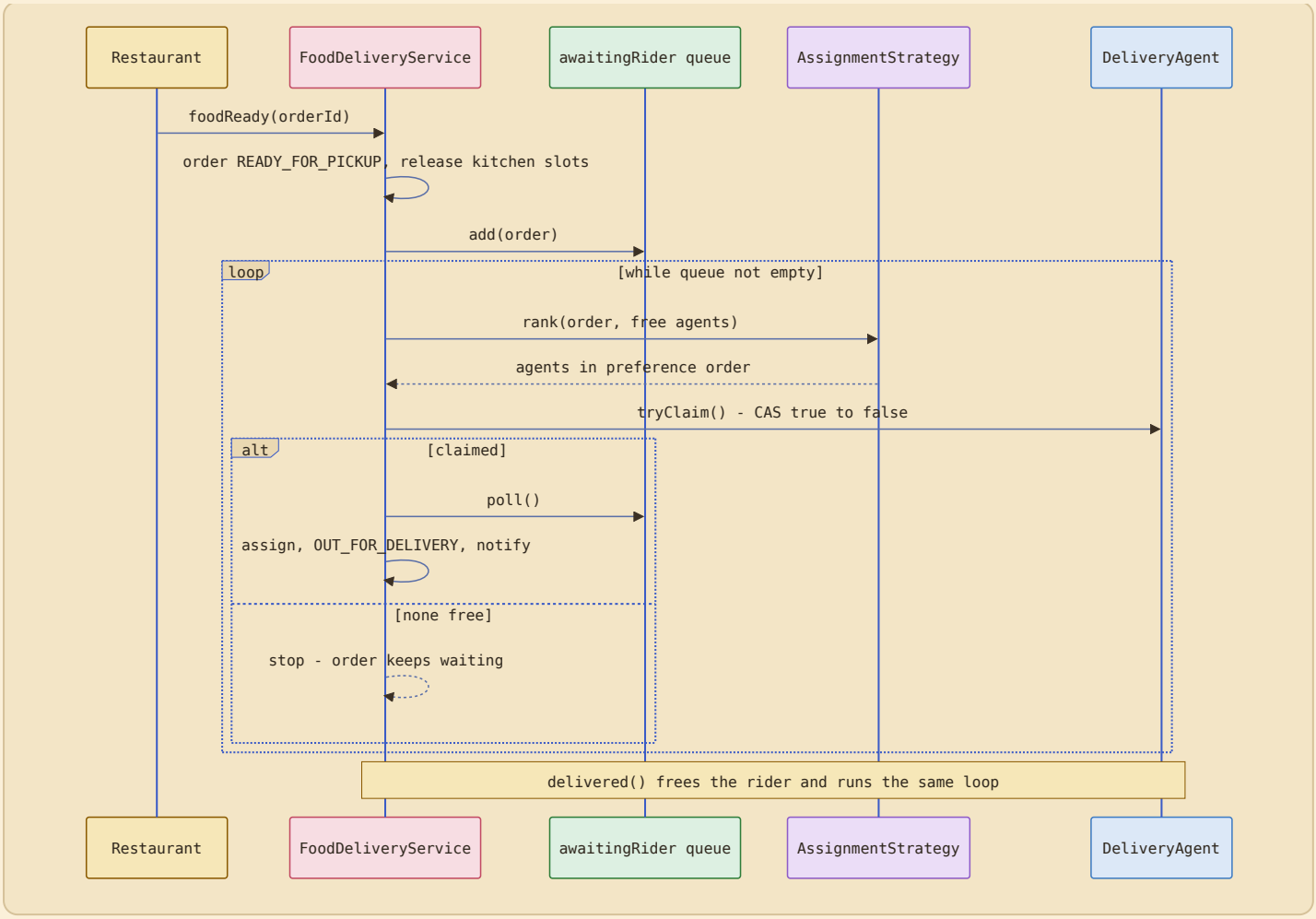
Key flows

Order tracking state machine

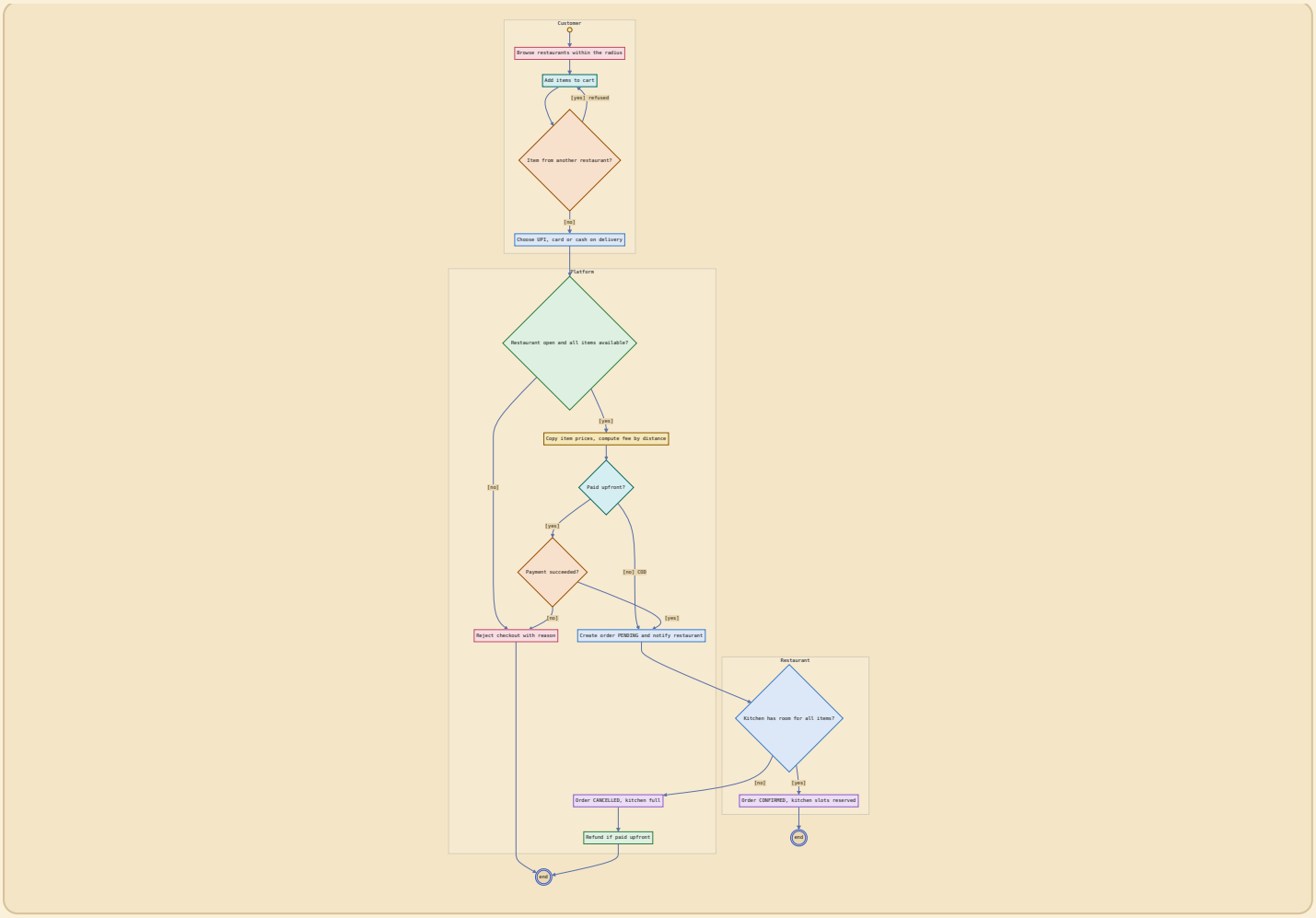


(Real apps often split `READY_FOR_PICKUP → OUT_FOR_DELIVERY` into "rider assigned", "arrived at restaurant", "picked up". Adding them is a local enum change.)

Food ready → rider assignment

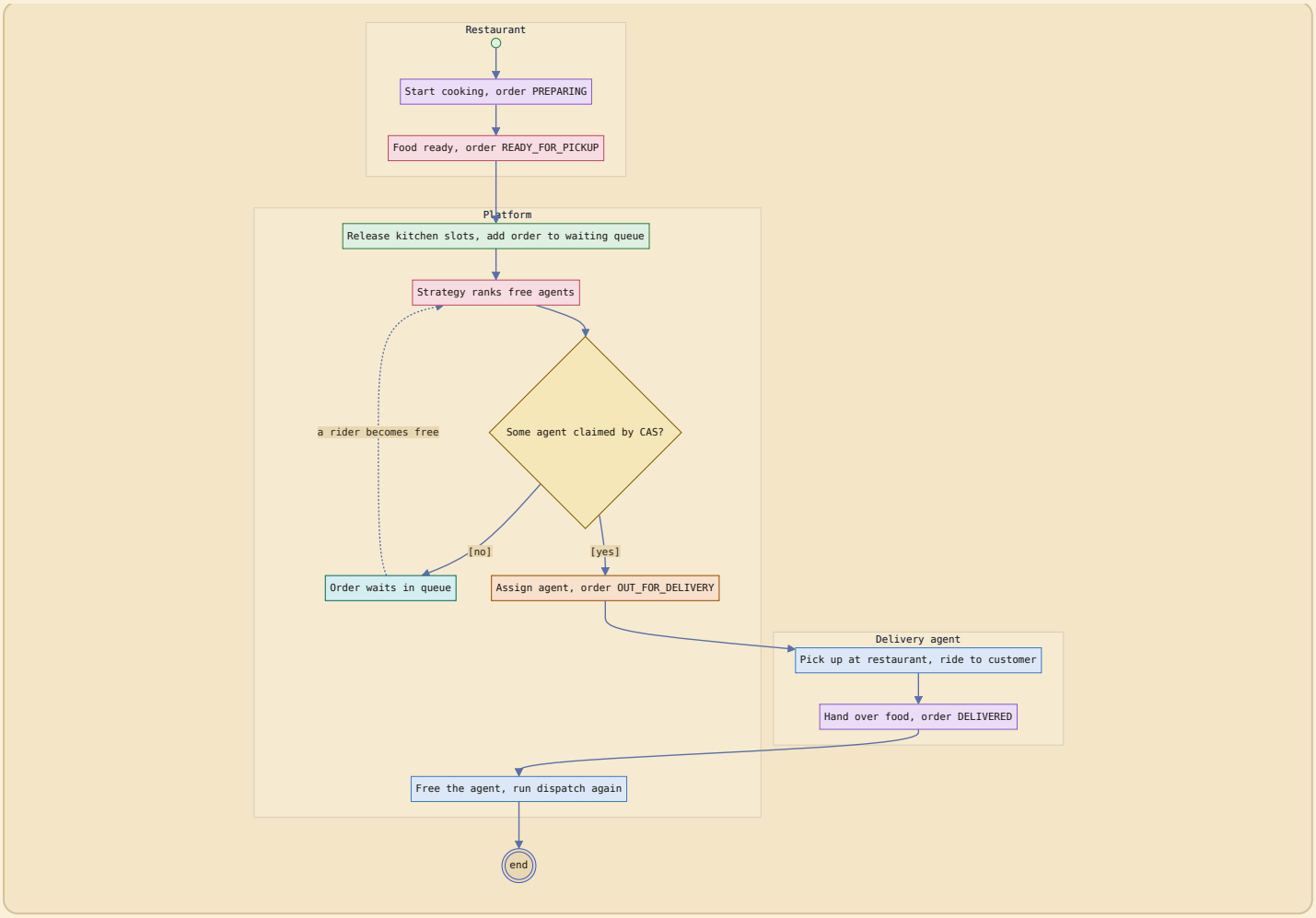


Activity — place an order (with the restaurant's accept or refuse; one swimlane per actor, notation as in §3.13).



The payment-gateway call is a stub in the demo (every payment succeeds); the branch shows where a real gateway's failure goes.

Activity — cook, assign a delivery agent and deliver.



Cancelling is possible only before cooking starts (PENDING or CONFIRMED), so this second flow has no cancel branch. The sequence diagram above shows the same dispatch loop as calls between objects.

FoodKart `place_order(name, qty)`: user logged in? → restaurant exists? → delivers to the user's pincode? → `take(qty)` atomically (quantity ≥ qty, then subtract) → record in history → "Order Placed Successfully." Otherwise "Cannot place order".

FoodKart selection variant: filter restaurants that deliver to the user's pincode and have `quantity ≥ qty` → ask the strategy to pick one → `take(qty)`; if another user took the capacity in between, drop that restaurant and ask again (bounded retries).

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for rider assignment (`rank` returns a preference list)	Nearest vs fair-share vs rating-based are swappable at run time (`setStrategy`, step 7). Returning a *ranked list* lets the service try the next rider if the first was just taken (the free list is read before claiming, so it can be stale).	Returning one rider (fails if that rider was claimed a moment earlier).
CAS on the rider (`AtomicBoolean.compareAndSet(true,false)`)	The rider guards itself: whoever calls `tryClaim()` — this dispatcher, a second dispatcher instance, or a future "rider accepts" button — exactly one `compareAndSet` succeeds. In this code `dispatchWaiting` is also `synchronized` (to drain the queue in FIFO order), so the CAS is the safety net that keeps working even when claims come from outside that lock.	Check `isAvailable()` then set (a race as soon as two callers exist).
Kitchen capacity reserved on accept, released on ready	Models the rule "a restaurant cannot accept more items while its kitchen is full" from the capacity-based food-ordering variant.	Unlimited orders per restaurant.
Waiting queue for riders	No free rider is normal at peak; FIFO keeps it fair; freeing a rider re-runs dispatch.	Failing the order.
State machine in `OrderStatus`	Tracking states and legal moves in one place; cancel after cooking starts is impossible.	Scattered `if`s.
`OrderItem` copies the price	A later menu price change (step 3) cannot alter a placed order.	Referencing `MenuItem` directly.
Sealed `PaymentMethod` + records	A closed, known set; COD overrides `paidUpfront()` so refunds skip it.	`String paymentType`.
Observer (`OrderListener`)	Customer app, restaurant tablet and	The service printing/sending messages itself.

Decision	Why	Alternative rejected
	rider app all react to the same events.	
One restaurant per cart	Matches real apps; a single kitchen, single pickup, single fee.	Multi-restaurant carts (needs split orders).
FoodKart: `SortBy` enum holds its comparator (an object that decides which of two items comes first)	`show_restaurant(rating	price)`becomes`sorted(sortBy.comparator)`; both descending as the statement says; name as tie-breaker for stable output.
FoodKart: owner recorded at registration	The statement says restaurants are registered "in the context of the logged-in user"; only that owner may update quantity or location.	Anyone updating any restaurant.
FoodKart: `Restaurant.take(q)` synchronized	"Check quantity then subtract" is atomic — two users cannot both take the last plates.	Separate `getQuantity()` + `setQuantity()`.

Java implementation

Part A — Food delivery platform (`lld.ch19.fooddelivery`)

Files: `Model.java` (location, customer, menu item, restaurant, cart, order item, status machine, payments, order, delivery agent), `FoodDeliveryService.java` (strategies, listener, notifier, service), `Main.java`.

The full runnable source is in `code/lld/ch19/fooddelivery/`; below are the lines that carry the design.

```
// File: lld/ch19/fooddelivery/Model.java
synchronized boolean tryReserveKitchen(int items) {          // Restaurant: check-and-take kitchen slots
    if (itemsInKitchen + items > kitchenCapacity) return false;
    itemsInKitchen += items;
    return true;
}
synchronized void add(Restaurant r, String itemId, int qty) { // Cart: one restaurant per cart
    if (restaurant != null && !restaurant.id().equals(r.id()))
        throw new IllegalStateException("cart has items from " + restaurant.name() + " - clear it first");
    restaurant = r;
    quantities.merge(itemId, qty, Integer::sum);
}
enum OrderStatus {   // next(): PENDING -> CONFIRMED|CANCELLED, CONFIRMED -> PREPARING|CANCELLED,
                     // PREPARING -> READY_FOR_PICKUP -> OUT_FOR_DELIVERY -> DELIVERED
    boolean canMoveTo(OrderStatus t) { return next().contains(t); }
}
sealed interface PaymentMethod permits UpiPayment, CardPayment, CashOnDelivery {
    default boolean paidUpfront() { return true; }           // CashOnDelivery overrides: false
}
final class DeliveryAgent {                                   // available is an AtomicBoolean
    /** Compare-and-set: exactly one caller wins. */
    boolean tryClaim() { return available.compareAndSet(true, false); }
    void completeDelivery() { deliveriesToday.incrementAndGet(); available.set(true); }
}

// File: lld/ch19/fooddelivery/FoodDeliveryService.java
interface DeliveryAssignmentStrategy {                        // NearestAgentStrategy, FairShareStrategy
    List<DeliveryAgent> rank(Order order, List<DeliveryAgent> free);
}

Order placeOrder(Customer c, Cart cart, PaymentMethod pay) {
    // ... closed restaurant / sold-out item -> IllegalStateException
    cart.quantities().forEach((itemId, qty) -> {
        MenuItem m = r.item(itemId).orElseThrow();
        items.add(new OrderItem(itemId, m.name(), m.price(), qty));   // price copied at order time
    });
    BigDecimal km = BigDecimal.valueOf(r.location().distanceTo(c.address())).setScale(1, RoundingMode.HALF_UP);
    BigDecimal fee = BASE_FEE.add(PER_KM.multiply(km)).setScale(2, RoundingMode.HALF_EVEN);   // 20 + 8/km
    // ... new Order(...), cart.clear(), publish PENDING
}

/** The restaurant accepts only if its kitchen has room; otherwise the order is auto-cancelled and refunded. */
boolean restaurantAccepts(String orderId) {
    Order o = orders.get(orderId);
    synchronized (o) {                                // same lock as cancel(): capacity can never leak
        if (o.status() != OrderStatus.PENDING) return false;
        if (o.restaurant().tryReserveKitchen(o.itemCount())) {
            transition(o, OrderStatus.CONFIRMED);
            return true;
        }
        System.out.println("    (" + o.restaurant().name() + " kitchen full: needs " + o.itemCount()
                + " item slots, free " + o.restaurant().freeCapacity() + ")");
        transition(o, OrderStatus.CANCELLED);
    }
    refundIfPrepaid(o);                               // like cancel(): a prepaid order gets its money back
    return false;
}

/** Food is ready: kitchen capacity comes back, and we look for a rider. */
void foodReady(String orderId) {
    Order o = orders.get(orderId);
    transition(o, OrderStatus.READY_FOR_PICKUP);
    o.restaurant().releaseKitchen(o.itemCount());
    awaitingRider.add(o);
    dispatchWaiting();
}
// delivered(): DELIVERED, agent.completeDelivery(), then dispatchWaiting() so a freed rider takes the oldest
waiting order

boolean cancel(String orderId) {
```

```
    Order o = orders.get(orderId);
    synchronized (o) {
        if (!o.status().canMoveTo(OrderStatus.CANCELLED)) return false;
        boolean kitchenHeld = o.status() == OrderStatus.CONFIRMED;
        transition(o, OrderStatus.CANCELLED);
        if (kitchenHeld) o.restaurant().releaseKitchen(o.itemCount());
    }
    refundIfPrepaid(o);
    return true;
}

/** Runs after the order lock is released; a real gateway call is slow and must not block the order. */
private void refundIfPrepaid(Order o) {
    if (o.payment().paidUpfront()) System.out.println("    refund " + o.total() + " to " +
o.payment().describe());
}

/** Hand waiting orders (oldest first) to free riders. CAS on the rider prevents double booking. */
private synchronized void dispatchWaiting() {
    Order o;
    while ((o = awaitingRider.peek()) != null) {
        List<DeliveryAgent> free = agents.stream().filter(DeliveryAgent::isAvailable).toList();
        Optional<DeliveryAgent> got = Optional.empty();
        for (DeliveryAgent a : strategy.rank(o, free)) if (a.tryClaim()) { got = Optional.of(a); break; }
        if (got.isEmpty()) {
            System.out.println("    (" + o.id() + " waiting for a free rider)");
            return;
        }
        awaitingRider.poll();
        o.assign(got.get());
        transition(o, OrderStatus.OUT_FOR_DELIVERY);
    }
}
// transition(o, s) = o.moveTo(s) (throws on an illegal edge) + notify every OrderListener
```

Output:

```
== 1. Browse within 10 km ==
  Dosa Corner
  Pizza Hub

== 2. One restaurant per cart ==
  cart has items from Dosa Corner - clear it first

== 3. Place, accept, prepare, ready, deliver ==
    [OD101 PENDING] -> restaurant Dosa Corner: new order
  subtotal 280.00 + delivery 31.20 = 311.20
  price changed after ordering; order still 311.20
    [OD101 CONFIRMED] -> customer Asha
    [OD101 PREPARING] -> customer Asha
    [OD101 READY_FOR_PICKUP] -> customer Asha
    [OD101 OUT_FOR_DELIVERY] -> customer Asha: Ravi is on the way
  rider: Ravi (nearest to Dosa Corner)

== 4. Kitchen capacity: 4 items max at Dosa Corner ==
    [OD102 PENDING] -> restaurant Dosa Corner: new order
    [OD102 CONFIRMED] -> customer Vikram
    [OD103 PENDING] -> restaurant Dosa Corner: new order
    (Dosa Corner kitchen full: needs 2 item slots, free 1)
    [OD103 CANCELLED] -> customer Asha
    refund 111.20 to card ****4242

== 5. Only one rider free: second ready order waits ==
    [OD102 PREPARING] -> customer Vikram
    [OD102 READY_FOR_PICKUP] -> customer Vikram
    [OD102 OUT_FOR_DELIVERY] -> customer Vikram: Meena is on the way
    [OD104 PENDING] -> restaurant Pizza Hub: new order
    [OD104 CONFIRMED] -> customer Vikram
    [OD104 PREPARING] -> customer Vikram
    [OD104 READY_FOR_PICKUP] -> customer Vikram
    (OD104 waiting for a free rider)
    [OD101 DELIVERED] -> customer Asha
    [OD104 OUT_FOR_DELIVERY] -> customer Vikram: Ravi is on the way
  o4 rider: Ravi

== 6. Cancel rules ==
  cancel o2 (out for delivery)? false
    [OD105 PENDING] -> restaurant Pizza Hub: new order
    [OD105 CONFIRMED] -> customer Asha
    [OD105 CANCELLED] -> customer Asha
    refund 651.60 to card ****4242
  cancel o5 (confirmed)? true

== 7. Swap strategy at run time: fair share ==
    [OD102 DELIVERED] -> customer Vikram
    [OD104 DELIVERED] -> customer Vikram
  deliveries today: Ravi=2, Meena=1 (both now at Vikram's address)
    [OD106 PENDING] -> restaurant Dosa Corner: new order
    [OD106 CONFIRMED] -> customer Asha
    [OD106 PREPARING] -> customer Asha
    [OD106 READY_FOR_PICKUP] -> customer Asha
    [OD106 OUT_FOR_DELIVERY] -> customer Asha: Meena is on the way
  o6 rider: Meena (fewer deliveries today)
  o1 history: [PENDING, CONFIRMED, PREPARING, READY_FOR_PICKUP, OUT_FOR_DELIVERY, DELIVERED]
```

Reading the output. Step 1 hides the far-away grill (outside 10 km). Step 2 refuses a second restaurant in one cart. Step 3: the delivery fee is 20 + 8 × 1.4 km = 31.20; changing the dosa price afterwards does not change the order. Step 4: Dosa Corner holds 4 item slots; an order for 3 fits, the next card order for 2 is auto-cancelled and refunded (the same refund path as a customer cancel). Step 5: Meena takes OD102; OD104 waits because Ravi is still out; as soon as OD101 is delivered, Ravi is claimed for OD104. Step 6: an order out for delivery cannot be cancelled; a confirmed card order can, and is refunded. Step 7: with both riders at the same spot, the fair-share strategy picks Meena (1 delivery) over Ravi (2).

Part B — FoodKart (`lld.ch19.foodkart`)

Files: `FoodKart.java` (gender, user, review, order, restaurant, sort, selection strategies, service), `Main.java` (driver replaying the problem's sample test case, then extras).

The full runnable source (including the `Main` driver) is in `code/lld/ch19/foodkart/`; below are the lines that carry the design.

```
// File: lld/ch19/foodkart/FoodKart.java
final class Restaurant {                    // ONE dish, MANY pincodes; quantity = remaining capacity
    /** Check-and-take in one step, so two users cannot both get the last plates. */
    synchronized boolean take(int q) {
        if (q <= 0 || q > quantity) return false;
        quantity -= q;
        return true;
    }
    /** Serviceable = delivers to this pincode AND has non-zero quantity (from the problem statement). */
    boolean serviceableFor(String pincode) { return pincodes.contains(pincode) && quantity() > 0; }
}

/** show_restaurant(rating|price): both sorted in DESCENDING order, as the problem states. */
enum SortBy {
    RATING(Comparator.comparingDouble(Restaurant::rating).reversed()),
    PRICE(Comparator.comparing(Restaurant::price).reversed());
    final Comparator<Restaurant> comparator;
    SortBy(Comparator<Restaurant> c) { comparator = c.thenComparing(Restaurant::name); }
}

/** Strategy: when the user does not name a restaurant, which one should we pick? */
interface RestaurantSelectionStrategy { Optional<Restaurant> select(List<Restaurant> canFulfil); }
// HighestRatingStrategy (tie: most capacity left), LowestPriceStrategy (tie: higher rating)

final class FoodKartService {
    private volatile User current;                              // one logged-in user at a time

    String placeOrder(String name, int qty) {
        User u = requireLogin();
        Restaurant r = restaurants.get(name);
        if (r == null || !r.pincodes().contains(u.pincode()) || !r.take(qty)) return "Cannot place order";
        record(u, r, qty);
        return "Order Placed Successfully.";
    }

    /** Variant: the system picks the restaurant. Only those that deliver here AND have capacity for qty qualify.
*/
    String placeOrder(int qty, RestaurantSelectionStrategy strategy) {
        User u = requireLogin();
        List<Restaurant> canFulfil = restaurants.values().stream()
                .filter(r -> r.pincodes().contains(u.pincode()) && r.quantity() >= qty).toList();
        for (int attempt = 0; attempt < 3; attempt++) {          // retry if a racer took the capacity
            Optional<Restaurant> pick = strategy.select(canFulfil);
            if (pick.isEmpty()) return "Cannot place order";
            if (pick.get().take(qty)) {
                record(u, pick.get(), qty);
                return "Order Placed Successfully from " + pick.get().name() + ".";
            }
            canFulfil = canFulfil.stream().filter(r -> r != pick.get()).toList();
        }
        return "Cannot place order";
    }
    // updateQuantity / updateLocation go through ownedByCurrentUser(name): only the owner may change them
}
```

Output:

```
== Sample test case ==
show_restaurant(price):
  Food Court-2, Burger
  Food Court-1, NI Thali
place_order(Food Court-1, 2): Order Placed Successfully.
place_order(Food Court-2, 7): Cannot place order
show_restaurant(rating):
  Food Court-1, NI Thali
  Food Court-2, Burger
update_quantity(Food Court-2, 5): Food Court-2, BTM, Burger - 8
update_location(Food Court-2, BTM/HSR): Food Court-2, BTM/HSR, Burger - 8

== Extras ==
non-owner update: only the owner can update Food Court-1
rating 6: rating must be 1..5
order from a restaurant that does not deliver to BTM: Cannot place order

== Selection strategy variant (user in BTM, wants 4 plates) ==
highest rating: Order Placed Successfully from Food Court-2.
lowest price  : Order Placed Successfully from Food Court-4.
lowest price  : Order Placed Successfully from Food Court-2.
lowest price  : Cannot place order  (no one has 4 left)

== Order history of phoneNumber-3 ==
  FoodOrder[id=1, userPhone=phoneNumber-3, restaurant=Food Court-1, item=NI Thali, quantity=2, total=200]
  FoodOrder[id=2, userPhone=phoneNumber-3, restaurant=Food Court-2, item=Burger, quantity=4, total=480]
  FoodOrder[id=3, userPhone=phoneNumber-3, restaurant=Food Court-4, item=Dosa, quantity=4, total=320]
  FoodOrder[id=4, userPhone=phoneNumber-3, restaurant=Food Court-2, item=Burger, quantity=4, total=480]
```

Reading the output. The "Sample test case" block reproduces every expected line from the problem statement: price-sorted list (Burger 120 before Thali 100), the successful and failed orders, the rating-sorted list, and the owner's quantity and location updates (`Burger - 8`). The extras show owner-only updates, rating validation, a rating without comment, and a restaurant that does not deliver to the user's pincode. The variant shows the two strategies choosing **different** restaurants (highest rating → Food Court-2; lowest price → Food Court-4), capacity filtering (Food Court-1 has only 3 plates, so it is never picked for 4), and a clean refusal once no one has 4 plates left.

Concurrency & edge cases

Case	Handling
Two claims for the same rider	`tryClaim()` CAS: only one succeeds; the loser moves to the next rider in its ranked list. (Inside one service the synchronized dispatcher already serialises claims; the CAS also covers claims made outside it.)
Two dispatch loops run at once	`dispatchWaiting` is `synchronized`, so the waiting queue is drained by one thread at a time (FIFO fairness preserved).
Kitchen over-booked	`tryReserveKitchen` is `synchronized` (check + add in one step).
Cancel races with accept	Both lock the `Order`; accept re-checks `PENDING`, cancel releases slots only if they were reserved (`CONFIRMED`). No leak, no double release.
Item sold out after it was added to the cart	`placeOrder` re-checks `available` and rejects.
Restaurant closes	`browse` hides it; `placeOrder` refuses.
COD cancelled	No refund (nothing was paid).
FoodKart: no user logged in	`requireLogin()` throws a clear `IllegalStateException`.
FoodKart: duplicate phone / restaurant name	Rejected with `putIfAbsent`.
FoodKart: rating outside 1–5	The `Review` record's compact constructor (a record's short validation block that runs on creation) rejects it.
FoodKart: strategy picks a restaurant whose capacity was just taken	`take()` fails; that restaurant is dropped and the strategy asked again (bounded retries).

Real-world example

In Swiggy and Zomato the order screen walks through the same states — order placed, restaurant confirmed, food being prepared, rider assigned / picked up, delivered — and if you add a dish from a second restaurant, the app asks whether to **reset your cart**, which is our one-restaurant `Cart` rule. The rider side is a dispatch problem: these platforms assign riders using distance, rider load and fairness signals, and may batch two orders on one trip — all variations of `DeliveryAssignmentStrategy`. FoodKart itself is a Flipkart machine-coding question: a 90-minute, in-memory, driver-class exercise that rewards clean separation (service, model, strategy) over clever algorithms.

Extensions & follow-up questions

- **"Batch two orders for one rider."** A strategy that accepts a rider already carrying one order if the second pickup is on the way; the agent then has a list of orders and a capacity (e.g. 2).
- **"ETA."** An `EtaEstimator` strategy: prep time (per restaurant load) + travel time (distance / speed, or a maps API — an online service other programs call).
- **"Surge / distance-based fees."** Extract `DeliveryFeePolicy` (Strategy): base + per-km today, add surge when waiting orders exceed free riders.
- **"Rider rejects an assignment."** Put the rider back to available, add the order back to the head of the queue, and exclude that rider for this order.
- **"Multiple dishes per restaurant in FoodKart."** Replace `foodItem/price/quantity` with a `Map<String, MenuItem>`; `take` becomes per item and all-or-nothing for multi-item orders.
- **"Order split across restaurants (Flipkart variant)."** The selection strategy returns a *set* of restaurants that together cover all items; reserve capacity in each, roll back all on any failure.
- **"Live tracking."** The rider app pushes location updates; an Observer forwards them to the customer's app (WebSocket).

In the interview

Design round (45 min): clarify (5) → entities + one-restaurant cart (8) → **order state machine** (7) → **assignment strategy + CAS claim + waiting queue** (12) → payments/notifications (5) → concurrency and extensions (8).

Machine-coding round (FoodKart, 90 min): model classes first (User, Restaurant, Review, Order) → service with one method per command → the driver replaying the sample → sort enum → owner checks and validation → selection strategy → order history. Keep it runnable at every step; print exactly the expected lines.

> **30-second answer:** "Restaurants own their menus and a kitchen capacity reserved on accept and released when food is ready. Orders copy item prices and move through a state machine from PENDING to DELIVERED. When food is ready the order joins a FIFO queue; a pluggable assignment strategy ranks free riders and we claim one with a compare-and-set, so a rider can never get two orders. Every status change is published to observers for the customer, restaurant and rider apps."

Follow-ups with answers

- *Why release kitchen capacity on "ready" and not on "delivered"?* The kitchen is free once the food leaves it; delivery time is the rider's constraint, not the kitchen's.
- *Why does the strategy return a list?* Because between ranking and claiming, the top rider may be claimed by someone else (another dispatcher or the rider's own app); the list lets us fall through without re-ranking.
- *How would you scale assignment across cities?* Partition by city/zone; each zone has its own dispatcher (single writer), so there is no cross-zone contention (threads or servers fighting over the same lock or data).

Self-check

1. **Q:** Which states allow a customer cancel, and why not later? **A:** PENDING and CONFIRMED; once cooking starts the food cost is already spent.
2. **Q:** What stops two orders getting the same rider? **A:** `AtomicBoolean.compareAndSet(true, false)` in `tryClaim()`.
3. **Q:** In FoodKart, what makes a restaurant "serviceable"? **A:** It delivers to the user's pincode and has quantity > 0.
4. **Q:** Why does `show_restaurant(price)` list Burger (120) before NI Thali (100)? **A:** The statement asks for descending order.
5. **Q:** How does the selection variant avoid choosing a restaurant that cannot fulfil the order? **A:** It filters by `quantity ≥ qty` before the strategy chooses, and `take()` re-checks atomically.
6. **Q:** Which patterns appear? **A:** Strategy (assignment, selection, payment), Observer (notifications), state machine (order status), Facade (service).

TL;DR

- One-restaurant cart; order copies prices; **state machine** PENDING → … → DELIVERED.
- **Kitchen capacity** reserved on accept, released on ready; **riders** claimed with **CAS**, chosen by a swappable **strategy**, with a FIFO queue when none are free.
- FoodKart: pincode-based serviceability, one dish per restaurant, average rating, `SortBy` enum, atomic `take`, and a **selection strategy** filtered by capacity.

Design a Grocery Store (POS & Checkout)

A **point-of-sale (POS) system** is the software behind a supermarket till: the cashier (or a self-checkout kiosk) scans items, the screen shows a running total, offers are applied, the customer pays in one or

more ways, a receipt is printed, and stock goes down. Amazon (§19.1) was about an online cart that becomes an order; a grocery till is different in three ways that make it a good interview question:

- **Items are weighed as well as counted.** Loose produce is priced per kilogram; the scale sends a weight and the price is computed at the till.
- **Offers interact.** "Buy 2 get 1 free", "3 for ₹80", "10% off produce" and a basket coupon can all apply to one basket — and the rules for combining them decide what the customer pays, what tax is due, and how much a return refunds.
- **The till must never block a physical sale.** If the system says milk stock is 0 but a customer is holding a carton, the *count* is wrong, not the customer.

Money in this section is Indian rupees (₹; written `Rs` in the program output) and uses `BigDecimal`, following the chapter's four rules.

Problem & clarifying questions

Question	Assumed answer
Who uses it?	Cashiers at several registers (lanes) in one store; a supervisor for voids and refunds. Self-checkout uses the same register API.
How are items identified?	Packaged goods by **barcode** (EAN-13, the standard 13-digit product barcode); loose produce by a 4-digit **PLU** (price look-up) code plus the scale's weight.
Are shelf prices tax-inclusive?	Yes (as in India, the UK and the EU): the receipt shows how much tax the price *contains*. A US-style store adds sales tax at the till — the design must support both.
Which offers?	Buy X get Y free, multi-buy ("3 for ₹80"), percent off a department, and a basket coupon ("₹50 off when you spend ₹500").
Can offers stack?	Each product gets at most **one** item offer — the best for the customer. A basket coupon applies on top, to what is still owed.
How do people pay?	Cash (with change), card, UPI — and **split tender** (part UPI, part cash).
Can the customer change their mind?	Void a line while scanning; go back to scanning if nothing is paid yet; cancel the whole sale (card/UPI payments reversed).
Returns?	Yes, with the receipt: refund what the customer **actually paid** for that item, never more units than bought.
Stock?	Decrease on sale, increase on return; alert when low; never refuse a sale because of the stock count.

Requirements

Functional

1. Catalog lookup by barcode or PLU; weighed items priced by weight.
2. A **sale** collects line items; lines can be voided (kept on the receipt, marked VOID).
3. Item offers and basket coupons, with clear stacking rules.
4. Totals with tax per tax category (inclusive or exclusive policy).
5. Split payment: cash (with change), card, UPI; a declined card changes nothing.
6. Immutable **receipt** with lines, discounts, tax breakdown, tenders and change.
7. Returns against a receipt, refunding the net price paid, at most the quantity bought.
8. Stock updates and **low-stock / negative-stock alerts**.

Non-functional

- **Exact money**: `BigDecimal`, 2 decimal places, `HALF_UP` rounding (a 5 or more rounds up), and parts (per-product discounts, refunds) that always add up to the total.
- **Several registers at once** share the catalog, the stock and the receipts safely.
- **Deterministic pricing**: the same basket always prices the same way (a **pure function**: its result depends only on its inputs, and it changes nothing outside itself) — needed for audits and for returns.

- **Auditable**: voids stay on the receipt; prices are copied into lines at scan time.

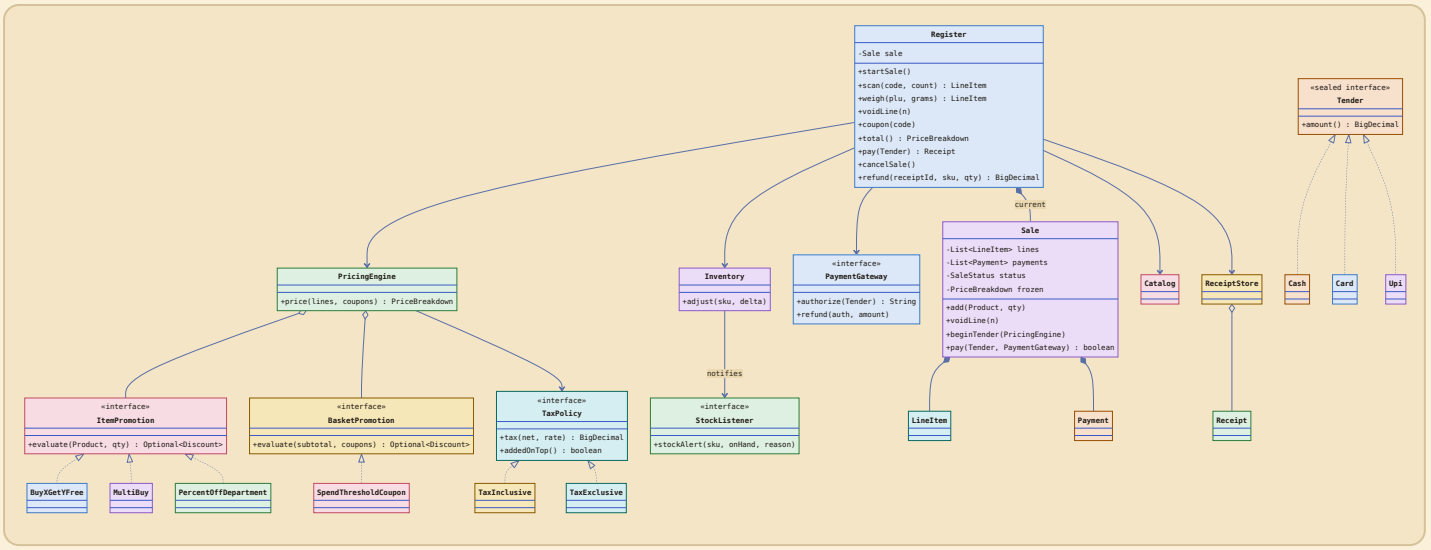
Use cases

Actor	Use cases
Cashier / self-checkout	start sale, scan, weigh, void line, add coupon, total, take payment, cancel sale
Customer	pay by cash / card / UPI, return items with the receipt
Supervisor	approve voids and returns (not modelled), reprice products
Payment network (system)	authorise and reverse card / UPI payments
Purchasing / store manager (subscriber)	receive low-stock and negative-stock alerts

Core entities

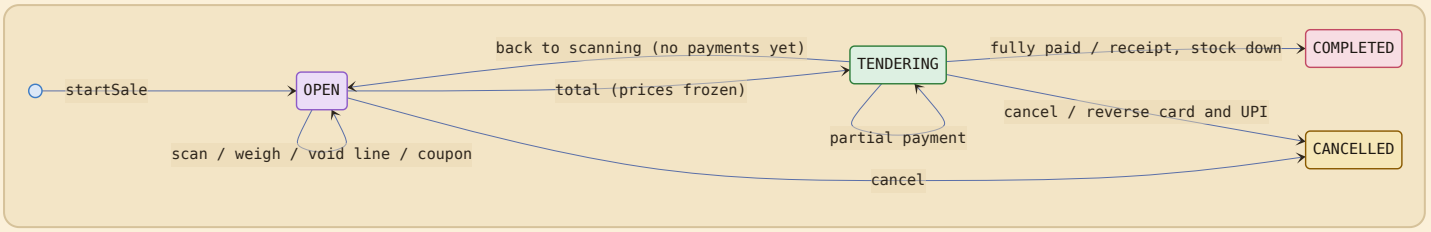
Class	Kind	Responsibility
`Product`	record	SKU (stock-keeping unit — the store's id for one distinct product), barcode/PLU, name, department, `PricingUnit` (EACH or PER_KG), unit price, `TaxCategory`.
`Catalog`	class	Lookup by code; `reprice` replaces the record (old sales keep old prices).
`LineItem`	record	One scan: product copy, quantity (count or kg), gross amount, voided flag.
`Sale` + `SaleStatus`	class + enum	OPEN → TENDERING → COMPLETED, or → CANCELLED; lines, coupons, payments, frozen price breakdown.
`ItemPromotion` + `BuyXGetYFree`, `MultiBuy`, `PercentOffDepartment`	interface + records	**Strategy** for item offers.
`BasketPromotion` + `SpendThresholdCoupon`	interface + record	**Strategy** for basket offers.
`TaxPolicy` + `TaxInclusive`, `TaxExclusive`	interface + classes	**Strategy** for how tax relates to the shelf price.
`PricingEngine` → `PriceBreakdown`	class → record	Pure function: lines + coupons → discounts, net per SKU, tax per category, total.
`Tender` (sealed) `Cash`, `Card`, `Upi`; `Payment`	records	How the customer pays; the part applied to the bill.
`PaymentGateway`	interface	Port (an interface through which the core code talks to an outside system) to the card / UPI networks (authorise, refund).
`Receipt`, `ReceiptStore`	record, class	Immutable proof of purchase; returns ledger per receipt and SKU.
`Inventory`, `StockListener`	class, interface	Shared stock; **Observer** for edge-triggered alerts (fired once when a line is crossed, not on every sale below it).
`Register`	class	**Facade** for one lane: the only API the till UI calls.

Class diagram

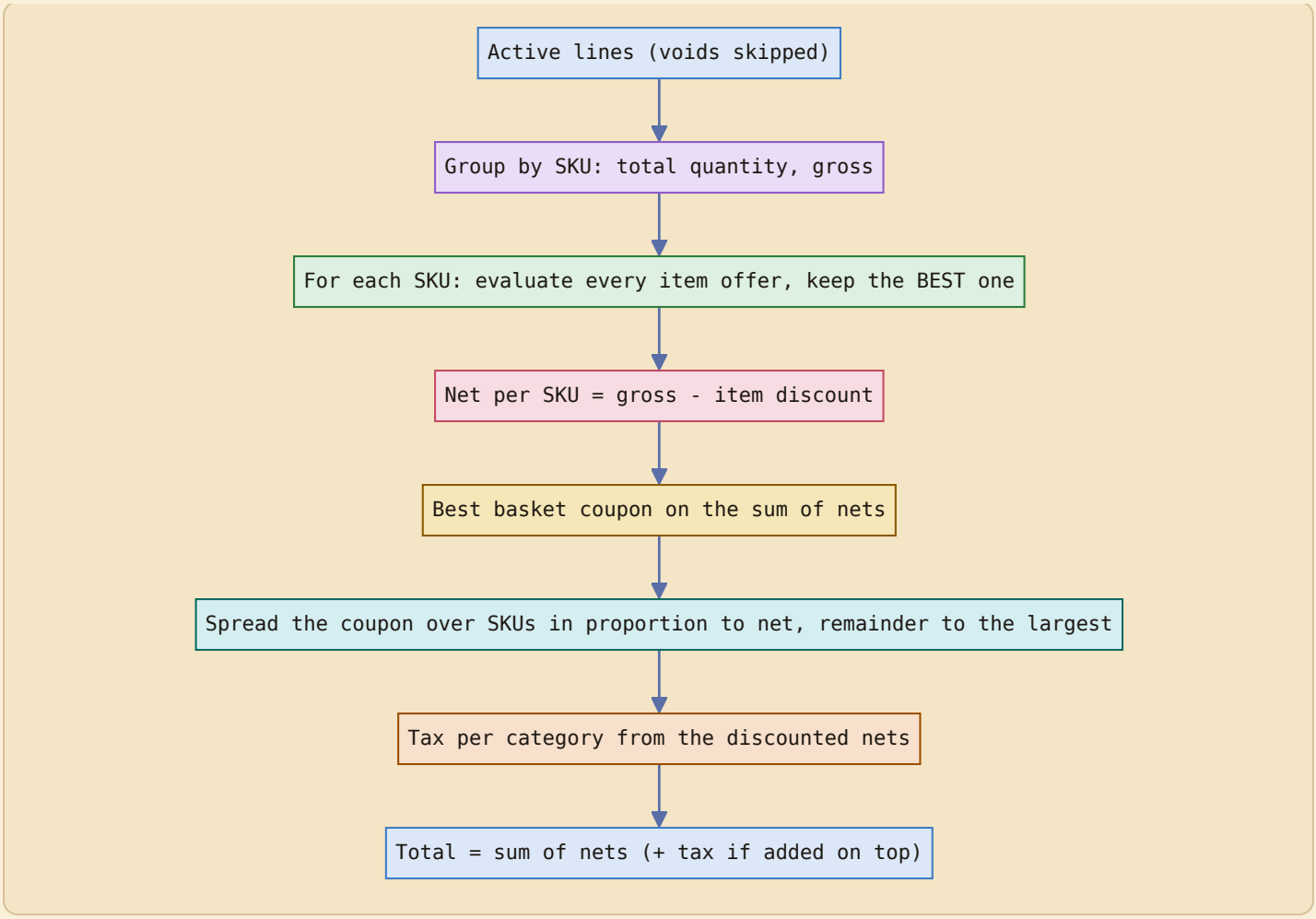


Key flows

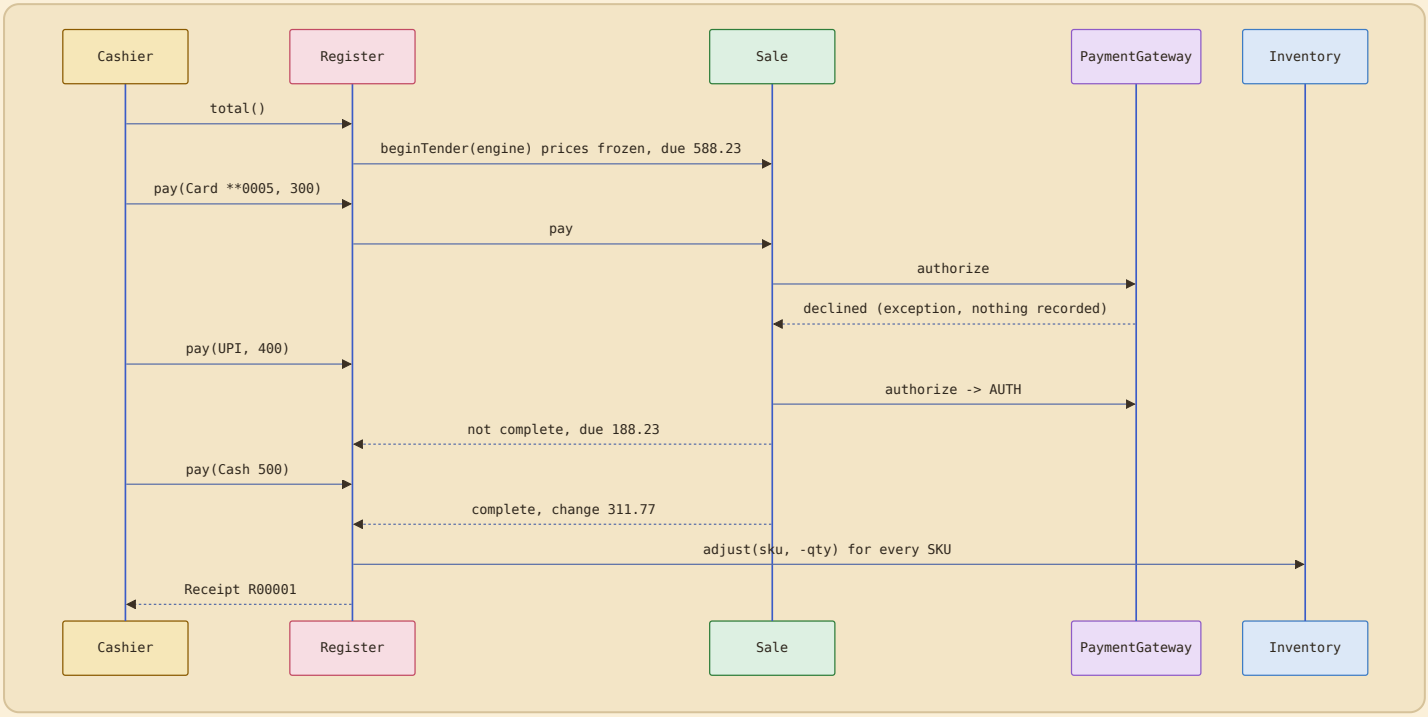
Sale lifecycle.



How one basket is priced (the hard method, `PricingEngine.price`):



Split payment.



Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Each offer is a small class behind `ItemPromotion` / `BasketPromotion`	**Strategy**	Marketing adds offers weekly; a new offer is one new class, the engine does not change	A `switch` over offer types inside the pricing code
Best offer per SKU wins; basket coupon after item offers	Explicit stacking policy	Stacking rules are business decisions that must be written down once; "best for the customer" is simple to explain at the till	Applying every offer in list order (results depend on the order; can go below zero)
`PricingEngine` is a **pure function** of lines and coupons	Stateless service	Same basket → same price; one engine shared by all registers (stateless: it keeps no data between calls); easy to test	Mutating discounts into the line items as they are scanned
Prices **frozen** when payment starts	Snapshot	A price change or an offer expiring mid-payment must not change the amount due	Re-pricing on every payment
Coupon **spread over SKUs**, remainder to the largest	Allocation with exact rounding	Tax is per category and returns refund per item, so every rupee of discount must belong to some item — and the parts must add up exactly	Keeping the coupon as one unallocated number
Tax as a `TaxPolicy`	**Strategy**	Inclusive (India/EU) vs exclusive (US) is a store-level choice	Two versions of the engine
Tenders as a **sealed** interface	Sealed types	The receipt printer's `switch` is checked by the compiler	`String paymentType`
`Register` as the only API the UI calls	**Facade**	The UI does not need to know about engines, stores and gateways	UI code calling each part directly
Sale status in an enum with `canMoveTo`	Enum state machine	"Scan after payment started" and "pay a cancelled sale" fail loudly	Boolean flags (`isPaying`, `isDone`)
Stock may go **negative**, alerts are **edge-triggered**	**Observer**	The shelf is the truth; negative stock is a signal for a recount. Each crossing alerts once	Refusing the sale; alerting on every sale below the line
Voided lines are kept and printed	Audit trail (a permanent record of every action)	Voids are a common way to hide theft; supervisors review them	Deleting the line

Java implementation

Package `lld.ch19.grocery`. Files: `Catalog.java` (products, EAN-13 check, catalog), `Promotions.java` (offer strategies), `Pricing.java` (tax policies, pricing engine), `Sale.java` (sale state machine, tenders, gateway port), `Register.java` (inventory, receipts, register facade), `Main.java`. The full runnable source is in `code/lld/ch19/grocery/`; below are the lines that carry the design.

```java
// File: lld/ch19/grocery/Promotions.java - one offer = one small Strategy class
record BuyXGetYFree(String sku, int buy, int free) implements ItemPromotion {
    @Override public Optional<Discount> evaluate(Product p, BigDecimal qty) {
        if (!p.sku().equals(sku)) return Optional.empty();
        int groups = qty.intValue() / (buy + free);
        if (groups == 0) return Optional.empty();
        BigDecimal off = p.unitPrice().multiply(BigDecimal.valueOf((long) groups * free));
        return Optional.of(new Discount("Buy " + buy + " get " + free + " free: " + p.name(), sku,
Money.round(off)));
    }
}
// MultiBuy ("3 for Rs 50"), PercentOffDepartment; BasketPromotion: SpendThresholdCoupon

// File: lld/ch19/grocery/Pricing.java
final class TaxInclusive implements TaxPolicy {     /** The price already contains the tax: tax = price x r / (100
+ r). */
    @Override public BigDecimal tax(BigDecimal net, BigDecimal r) {
        return net.multiply(r).divide(BigDecimal.valueOf(100).add(r), 2, RoundingMode.HALF_UP);
    }
    @Override public boolean addedOnTop() { return false; }      // TaxExclusive: net x r / 100, true
}

PriceBreakdown price(List<LineItem> lines, Set<String> coupons) {  // PricingEngine: a pure function
    // group non-voided lines by SKU -> bySku (total quantity, gross)
    for (SkuTotal t : bySku.values()) {
        Optional<Discount> best = itemOffers.stream()                // rule 1: ONE best item offer per SKU
                .map(p -> p.evaluate(t.product(), t.quantity()))
                .flatMap(Optional::stream)
                .max(Comparator.comparing(Discount::amount));
        BigDecimal off = best.map(d -> d.amount().min(t.gross())).orElse(Money.ZERO);
        best.ifPresent(d -> discounts.add(new Discount(d.description(), d.sku(), off)));
        net.put(t.product().sku(), t.gross().subtract(off));
    }
    BigDecimal subtotal = sum(net.values());
    Optional<Discount> basket = basketOffers.stream()                // rule 2: best basket coupon on what is
owed
            .map(b -> b.evaluate(subtotal, coupons))
            .flatMap(Optional::stream)
            .max(Comparator.comparing(Discount::amount));
    basket.ifPresent(d -> { discounts.add(d); spread(d.amount(), net); });   // rule 3
    // rule 4: tax per TaxCategory on the discounted nets; total = nets (+ tax if addedOnTop)
}

/** Proportional allocation; the rounding remainder goes to the largest SKU so the parts add up exactly. */
private static void spread(BigDecimal amount, Map<String, BigDecimal> net) {
    BigDecimal base = sum(net.values());
    if (base.signum() == 0) return;
    BigDecimal given = Money.ZERO;
    String largest = null;
    for (var e : net.entrySet()) {
        BigDecimal share = amount.multiply(e.getValue()).divide(base, 2, RoundingMode.HALF_UP);
        e.setValue(e.getValue().subtract(share));
        given = given.add(share);
        if (largest == null || e.getValue().compareTo(net.get(largest)) > 0) largest = e.getKey();
    }
    net.merge(largest, given.subtract(amount), BigDecimal::add);
}

// File: lld/ch19/grocery/Sale.java
boolean canMoveTo(SaleStatus next) {                          // SaleStatus
    return switch (this) {
        case OPEN -> next == TENDERING || next == CANCELLED;
        case TENDERING -> next == COMPLETED || next == OPEN || next == CANCELLED;
        case COMPLETED, CANCELLED -> false;
    };
}
/** Returns true when the bill is fully paid and the sale is complete. */
boolean pay(Tender t, PaymentGateway gateway) {
    require(SaleStatus.TENDERING, "pay");                     // beginTender() froze the breakdown
    BigDecimal due = due();
```

```
    if (t.amount().signum() <= 0) throw new PosException("Tender must be positive");
    String auth = "CASH";
    BigDecimal applied = t.amount();
    if (t instanceof Cash) {
        if (t.amount().compareTo(due) > 0) { change = t.amount().subtract(due); applied = due; }
    } else {
        if (t.amount().compareTo(due) > 0) throw new PosException("Card/UPI cannot be more than the amount due " +
Money.fmt(due));
        auth = gateway.authorize(t);                // throws if declined: nothing recorded
    }
    payments.add(new Payment(t, applied, auth));
    if (due().signum() == 0) { move(SaleStatus.COMPLETED); return true; }
    return false;
}
// cancel(gateway): reverses every non-cash payment, then CANCELLED

// File: lld/ch19/grocery/Register.java
/** delta < 0 for a sale, > 0 for a return. merge() is atomic per key, so registers never lose updates. */
void adjust(String sku, BigDecimal delta) {                    // Inventory
    BigDecimal after = onHand.merge(sku, delta, BigDecimal::add);
    BigDecimal before = after.subtract(delta);
    BigDecimal point = reorderPoint.getOrDefault(sku, BigDecimal.ZERO);
    // edge-triggered: alert only when THIS change crosses the line, so each crossing alerts exactly once
    if (before.compareTo(point) > 0 && after.compareTo(point) <= 0)
        listeners.forEach(l -> l.stockAlert(sku, after, "LOW - reorder"));
    if (before.signum() >= 0 && after.signum() < 0)
        listeners.forEach(l -> l.stockAlert(sku, after, "NEGATIVE - recount the shelf"));
}

/** Refund what the customer actually paid per unit (after offers and coupons), not the shelf price. */
BigDecimal refund(String receiptId, String sku, int qty) {     // Register
    // ... find the SKU on the receipt -> bought
    BigDecimal q = BigDecimal.valueOf(qty);
    BigDecimal after = receipts.claimReturn(r, sku, q, bought.quantity());   // ConcurrentHashMap.compute: atomic
    // refund = share(after) - share(before): the parts always add up to exactly what was paid
    BigDecimal net = r.breakdown().netBySku().get(sku);
    BigDecimal refund = share(net, after, bought.quantity()).subtract(share(net, after.subtract(q),
bought.quantity()));
    inventory.adjust(sku, q);
    return refund;
}
```

How the code works, part by part

- **Scanning.** `Register.scan` checks the EAN-13 check digit (a last digit computed from the others, so a misread code fails the sum) first (a misread barcode is caught before lookup), then refuses weighed products; `weigh` converts grams to kilograms with `movePointLeft(3)` so 1245 g becomes exactly `1.245`. `Sale.add` copies the product — and therefore its price — into the new `LineItem`.
- **Pricing.** `PricingEngine.price` skips voided lines, groups by SKU, asks every item offer for a discount and keeps the largest (capped at the gross), applies the best basket coupon to what is left, spreads that coupon across SKUs, and computes tax per category on the discounted amounts. With `TaxInclusive`, tax = amount × r ÷ (100 + r); with `TaxExclusive`, tax = amount × r ÷ 100 and is added to the total.
- **Paying.** `beginTender` freezes the breakdown. `pay` treats cash specially: if it is more than the amount due, the extra becomes change. Card and UPI may not exceed the amount due and must be authorised; a declined authorisation throws before anything is recorded. When the amount due reaches zero, the sale completes and the register writes the receipt and reduces stock.
- **Returns.** `ReceiptStore.claimReturn` uses `ConcurrentHashMap.compute` so "check how many were already returned, then add" is atomic. The refund for a return is `share(after) − share(before)`, where `share(k)` is the net paid for the first *k* units; that way three separate one-item returns add up to exactly what was paid.
- **Stock.** `Inventory.adjust` uses `merge`, which is atomic per key; it compares the value before and after *this* change, so each crossing of the reorder point — the stock level at which more should be ordered — (or of zero) alerts exactly once, however many registers sell at the same time.

Output:

```
== 1. Scan a basket ==
   error: Unknown code 8901000000095: call a supervisor
   error: Bad barcode 8901000000018: check digit wrong, rescan
   error: Bananas is sold by weight: put it on the scale

== 2. Total, then split payment: declined card, UPI, cash with change ==
   amount due Rs 588.23
   error: Cannot scan when the sale is TENDERING
   error: Card **0005 declined by issuer
   error: Card/UPI cannot be more than the amount due Rs 588.23
   UPI 400 -> receipt? null
  ---------------- RECEIPT R00001 (LANE-1) ----------------
   1 Bananas            1.245 kg @ Rs 60.00/kg       Rs 74.70
   2 Tomatoes           0.500 kg @ Rs 40.00/kg       Rs 20.00
   3 Milk 1L            1 x Rs 68.00                 Rs 68.00
   4 Milk 1L            1 x Rs 68.00                 Rs 68.00  VOID
   5 Paneer 200g        1 x Rs 90.00                 Rs 90.00
   6 Chips 50g          3 x Rs 20.00                 Rs 60.00
   7 Biscuits           4 x Rs 30.00                Rs 120.00
   8 Detergent 1kg      1 x Rs 245.00               Rs 245.00
  Gross                                            Rs 677.70
    10% off PRODUCE: Bananas                        -Rs 7.47
    10% off PRODUCE: Tomatoes                       -Rs 2.00
    Buy 2 get 1 free: Chips 50g                    -Rs 20.00
    3 for Rs 80.00: Biscuits                       -Rs 10.00
    Coupon SAVE50                                  -Rs 50.00
  TOTAL                                            Rs 588.23
    incl. tax EXEMPT @0%                            Rs 0.00
    incl. tax REDUCED @5%                           Rs 8.78
    incl. tax STANDARD @18%                        Rs 40.07
  Paid UPI asha@okbank                             Rs 400.00
  Paid cash                                        Rs 500.00
  Change                                           Rs 311.77

== 3. Returns refund what was really paid ==
   return 1 of 3 chips -> refund Rs 12.29
   return 2 more chips -> refund Rs 24.58  (all three add up to the Rs 36.87 paid)
   error: Only 0 of CHP left to return on R00001
   error: Only 1 of PNR left to return on R00001

== 4. Price change during a sale; going back to scanning; cancelling ==
   2 milk scanned around a reprice: gross Rs 140.00 = Rs 68 (old price kept) + Rs 72 (new)
   after adding paneer, due Rs 230.00
   error: Payments already taken; cancel them first
   gateway: reversed Rs 100.00 on AUTH1002
   error: Cannot pay when the sale is CANCELLED

== 5. Same basket, US-style tax added at the till ==
   inclusive total Rs 588.23 vs exclusive total Rs 644.73 (tax {EXEMPT=0.00, REDUCED=9.22, STANDARD=47.28})

== 6. Four registers sell 30 paneer each at once; stock was 100 ==
   paneer on hand: -21  (sold 120 + 1 earlier; the till never refuses a real item)
   alert: PNR LOW - reorder (on hand 20)
   alert: PNR NEGATIVE - recount the shelf (on hand -1)
```

Reading the output: the misread barcode (wrong check digit), an unknown code and scanning bananas without the scale are all refused. On the receipt, chips qualify for both "buy 2 get 1 free" (saves ₹20) and "3 for ₹50" (saves ₹10); only the better one is applied. The ₹50 coupon applied because the basket was still over ₹500 after item offers. The total ₹588.23 contains ₹8.78 + ₹40.07 of tax; the same basket under a US-style policy costs ₹644.73. The declined card and the over-large UPI amount changed nothing; UPI ₹400 plus cash ₹500 gave ₹311.77 change. Returning the chips one, then two, refunded ₹12.29 + ₹24.58 = ₹36.87 — exactly the net paid after the offer and the coupon share, not the shelf price of ₹60. The re-priced milk shows that a line keeps the price it was scanned at. Four registers selling 30 paneer each drove stock from 99 to −21 without refusing anyone, and the low and negative alerts each fired once.

Concurrency & edge cases

Case	Handling
Four registers sell the same product at once	`Inventory.adjust` uses `ConcurrentHashMap.merge` (atomic per key); no lost updates
Stock count says zero but the item is in hand	Sale goes through; stock goes negative; one "recount" alert
Alert spam when stock is low	Edge-triggered: alert only on the change that crosses the line
Two service desks refund the same receipt line at once	`ReceiptStore.claimReturn` uses `compute`: check-and-add is atomic; the second over-claim is refused
Price changes during a sale	Lines keep the price copied at scan; the frozen breakdown fixes the amount due
Offer changes during payment	Breakdown frozen at `total()`; going back to scanning re-prices
Card declined	Exception from the gateway before anything is recorded
Cancel after a card payment	`cancel` reverses every non-cash payment through the gateway
Rounding pennies in coupon allocation and refunds	Remainder to the largest SKU; refunds as differences of cumulative shares
One register used by one cashier	`Register` and `Sale` are single-threaded by design; shared objects (`Catalog`, `Inventory`, `ReceiptStore`) are the thread-safe ones

Real-world example

Supermarket tills identify loose produce by **PLU codes** — 4011 is the standard code for yellow bananas, and tomatoes have their own codes in the 4000 range — and read the weight from an integrated scale, exactly like `weigh(plu, grams)`. Packaged goods carry EAN-13 barcodes whose last digit is a check digit, which is why a smudged barcode is rejected rather than misread as another product. Receipts in tax-inclusive countries (for example GST — Goods and Services Tax — invoices in India or VAT — value-added tax — receipts in the UK) print how much tax the total *contains*, while US receipts add sales tax below the subtotal — our two `TaxPolicy` strategies. Retailers also run periodic "cycle counts" to fix the kind of negative stock our `Inventory` reports.

Extensions & follow-up questions

- **"Loyalty points and member prices."** A `CustomerContext` passed to the engine; member-only offers check it. Points are earned on the *net* paid (after discounts), from the receipt.
- **"Weighed items with a pre-packed label."** In-store printed barcodes that encode the price or weight in the digits (a variable-measure barcode); the scanner decodes them into a `LineItem` directly.
- **"Offers that span products (buy pasta + sauce, save ₹20)."** A `BundlePromotion` that looks at several SKUs; the "one offer per unit" rule then needs a small optimiser (try the combinations, keep the best) instead of best-per-SKU.
- **"Refund to the original payment method."** Store tender splits on the receipt; refund card/UPI parts through the gateway first, then cash.
- **"The network is down."** Offline mode: queue card authorisations (with a floor limit — a maximum amount accepted without online approval), keep selling, reconcile (match up the records) later; the catalog and offers are cached locally in each register.
- **"End-of-day."** A `Shift` per cashier: opening float (the cash put in the drawer at the start), cash counted at close, expected cash = float + cash sales − change − cash refunds.
- **"Self-checkout."** The same `Register` facade with a weight-check step at the bagging area (weight mismatch → assistance needed).

In the interview

Plan for 45 minutes: clarify offers, tax, tenders, returns (5) → entities and the Register facade (8) → sale state machine (5) → **pricing engine** with the stacking rules (12) → split payment and receipt (7) →

returns, stock and concurrency (8).

> **30-second answer:** "A Register facade drives one Sale through OPEN, TENDERING, COMPLETED or CANCELLED. Lines copy the price at scan time; produce is priced by weight from the scale. A pure pricing engine groups lines by SKU, gives each SKU its single best offer, applies a basket coupon, spreads it over SKUs so tax and refunds are exact, and computes tax through a pluggable inclusive or exclusive policy. The price is frozen at total; tenders are a sealed type, cash can give change, card and UPI go through a gateway. On completion an immutable receipt is stored and shared stock is reduced atomically, with edge-triggered alerts — and the till never refuses a real item."

Follow-ups with answers

- *Why not apply all offers that match?* Stacking can give away more than intended or even a negative price; the business has to choose a rule, and "best single offer per product, then one basket coupon" is simple and explainable.
- *Why spread the coupon over items?* Tax differs per category and returns must refund the price actually paid for that item.
- *Why is the pricing engine stateless?* So every register can share it and the same basket always gives the same answer — also what makes returns reproducible.

Self-check

1. **Q:** Three chips qualify for "buy 2 get 1 free" and "3 for ₹50". What is applied? **A:** Only the larger discount (₹20), because each SKU gets its single best item offer.
2. **Q:** Why does a return of one chip refund ₹12.29, not ₹20? **A:** The refund is the net price paid — after the offer and the item's share of the coupon.
3. **Q:** What stops two service desks refunding the same unit? **A:** `ConcurrentHashMap.compute` makes "check already returned, add" one atomic step.
4. **Q:** Why is stock allowed to go negative? **A:** The physical item is in the customer's hand; the count is wrong, so we sell and raise a recount alert.
5. **Q:** When are prices frozen, and why? **A:** At `total()`, so price or offer changes during payment cannot change the amount due.
6. **Q:** Which patterns appear? **A:** Strategy (offers, tax, payment gateway), Observer (stock alerts), Facade (Register), enum state machine (sale), sealed types (tenders).

TL;DR

- **Register facade** → one **Sale** state machine; lines copy prices; produce priced by **weight** (PLU + scale).
- **Pure pricing engine**: best item offer per SKU → basket coupon → spread over SKUs → tax per category via a **TaxPolicy** strategy.
- **Freeze** prices at total; **split tender** with a sealed `Tender`; declined cards change nothing; cancel reverses non-cash payments.
- Returns refund the **net paid**, claimed atomically; stock updates are atomic, may go negative, alerts are **edge-triggered**.

Chapter summary

Problem	Hard part	The move that solves it
Amazon	Guests vs members; last-item races; cancel rules	Ordering only on `Member` (LSP); atomic all-or-nothing stock reservation; order state machine
Brokerage	Four order types; tax lots; over-spending	Two abstract questions on `Order`; `StockLot` + FIFO `LotSale`; reserved funds
Stock exchange	Fair matching	Per-symbol `TreeMap<price, FIFO queue>`; price then time; one lock per symbol
Auction	Concurrent bids	All bid rules in one per-listing critical section; notify after unlocking
Music streaming	Shuffle/repeat without losing order	Play order as a strategy-produced permutation of indexes; plan rules in the enum
Food delivery / FoodKart	Capacity and riders under concurrency	Reserve kitchen slots atomically; CAS-claim riders; strategies for assignment and selection
Grocery POS	Weighed items, stacking offers, split tender, returns	Pure pricing engine (best offer per SKU, coupon spread, tax per category); frozen price; atomic return claims; edge-triggered stock alerts

Patterns that kept coming back: Strategy (payments, play order, rider assignment, restaurant selection, trade-price policy, grocery offers and tax), Observer (notifications, fills, plays), state machines in enums (orders, auctions, player), Template Method (statements), Facade (every `*Service`), and one concurrency rule — **make check-then-act atomic** (lock, or compare-and-set).

Sources combined in this chapter: the DesignGurus *Grokking the Object-Oriented Design Interview* lessons on Amazon and the stock brokerage (requirements, actors, staged class design and diagrams); the awesome-low-level-design problem statements for the auction system, music streaming service and food delivery service; and the kumaransg/LLD machine-coding statements for the Navi stock exchange and Flipkart's FoodKart. All Java code is original. The grocery point-of-sale case is designed from first principles, using public retail conventions (GS1 EAN-13 check digits, IFPS PLU codes for produce, tax-inclusive vs sales-tax receipts).

Chapter 20 — Case Studies: Pattern-Driven Components

> **Goal.** The problems in this chapter are not whole products. They are **building blocks** that almost every real system contains: a ledger that splits money, a service that sends notifications, a cache, a matcher that pairs riders with drivers, a logger, a message broker and a rate limiter. Each one is small enough to finish in an interview, and each one is **anchored on one or two design patterns**. If you can name the pattern and say what change it absorbs, you have already answered half of the question.

Why "components" deserve their own chapter. A component is a part that other parts of a system *use*. It has a small public surface (a few methods), and a lot of hidden rules behind it. That makes it the perfect interview question: the interviewer can keep asking "what if this rule changes?" and a good design answers with "one new class, nothing else touched".

#	Component	Anchor pattern(s)	The change the pattern absorbs
20.1	Splitwise (expense sharing)	**Strategy** (split rules)	A new way to split a bill
20.2	Notification service	**Template Method** (retry loop), **Strategy** (send rules), **Observer** (outcomes), Simple Factory (channels)	A new channel, a new reason not to send, a new thing that watches results
20.3	In-memory cache (+ LRU from scratch)	**Strategy** (eviction policy), **Observer** (removal listener), Builder	A new eviction rule
20.4	Ride sharing (Uber)	**Strategy** (matching, pricing, payment), **Decorator** (surge), State machine (trip)	A new matching rule, surge on/off, a new payment method
20.5	Logging framework	**Chain of Responsibility** (logger hierarchy), **Singleton** (LogManager), Strategy (formatter), Decorator (async appender)	A new sink, a new format, async writing
20.6	Pub-Sub system	**Observer / Publish-Subscribe**, append-only log with offsets	New subscribers, replay, slow consumers
20.7	Rate limiter	**Strategy** (algorithm), Simple Factory, Composite	A new algorithm, a new tier, combined limits

Key terms — the patterns named above. A **design pattern** is a named, reusable shape for solving a design problem that keeps coming back. In one line each: **Strategy** puts each interchangeable rule in its own class behind one interface (a list of methods with no code, which several classes implement in their own way); **Template Method** writes the fixed steps of an algorithm once in a parent class and lets subclasses fill in one or two steps; **Observer** lets objects sign up to be told when something happens; **Simple Factory** is one place that decides which class to create; **Builder** assembles an object with many optional settings step by step; **Decorator** wraps an object in another object of the same type to add behaviour; a **state machine** is a fixed set of states plus the allowed moves between them; **Chain of Responsibility** passes a request along a line of handlers; **Singleton** guarantees exactly one instance of a class; **Composite** lets a group of objects be used as if it were one; **Publish-Subscribe** sends messages through a middleman so senders and receivers never know each other. An **append-only log with offsets** is a list that is only ever added to at the end, where each entry is found by its position number (its offset).

How every case is laid out (the case-study template used in Chapters 14–21): *Problem & clarifying questions → Requirements → Use cases → Core entities → Class diagram → Key flows → Design decisions & patterns used → Java implementation (+ real output) → Concurrency & edge cases → Extensions & follow-up questions → Real-world example → In the interview → Self-check → TL;DR.* (*Concurrency* means several threads running at the same time, which is where most subtle bugs hide.)

Running the code. All code is Java 21, standard library only (only the classes that ship with Java itself, no extra downloads), under `code/lld/ch20/<name>/`. Every package (Java's folder-like grouping of related classes) has a `Main` (a class with the `main` method the program starts from). From `chapters/20-case-components/code`:

```
javac --release 21 -d /tmp/lld_20 $(find . -name '*.java')
java -cp /tmp/lld_20 lld.ch20.splitwise.Main      # also: notification, cache, lrucache,
                                                  # ridesharing, carpool, logging, pubsub, ratelimiter
```

Money is always `BigDecimal`, never `double` (binary floating point: `0.1 + 0.2` prints `0.30000000000000004`); compare with `compareTo`, not `equals` (which also compares the scale, so `2.0` ≠ `2.00`) — see Chapter 19's four rules. One more rule: always say how to round when you divide (`divide(x, 2, RoundingMode.HALF_EVEN)`), or Java throws an `ArithmeticException` for results like 1/3. (An **exception** is Java's way of signalling an error: normal execution stops and the error travels up to whoever catches it.)

Reading the course diagrams. The figures marked "(course diagram)" are UML diagrams (UML, the Unified Modeling Language, is the standard box-and-arrow notation for drawing classes and how they relate) and use one arrow legend throughout:

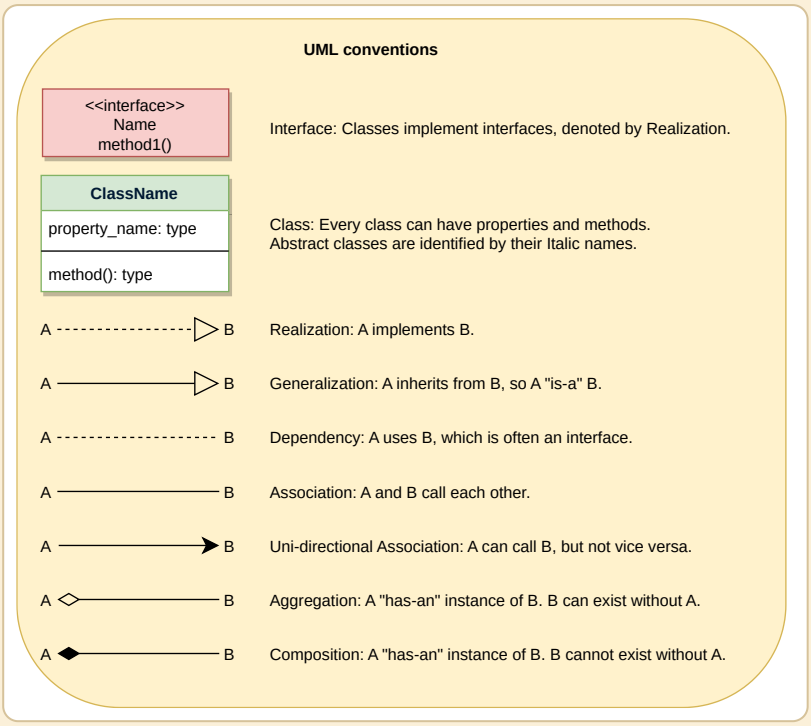


Figure — UML conventions used in the course class diagrams: interface, class, realization (implements), generalization (inherits), dependency, association, one-way association, aggregation and composition (course diagram)

The mermaid diagrams (Mermaid is a text format that a renderer turns into pictures) are drawn from **this chapter's Java code**, so they match the code line by line.

20.1 Design Splitwise

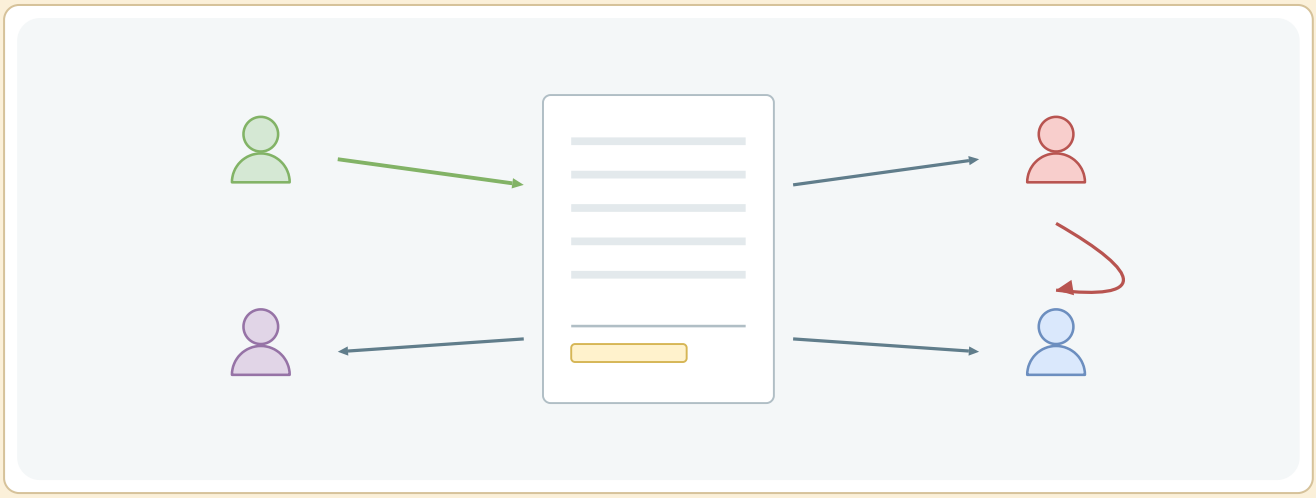


Figure — Splitwise in one picture: people send expenses into one shared record, and the record tells each person what they owe (course diagram)

Problem & clarifying questions

What it is. Splitwise is an app that remembers who paid for what when friends share costs (Alice's card pays the hotel, Bob covers dinner, Carol the cab), and tells everyone who owes whom — a shared notebook that does the maths.

The hard part is **not** the UI (user interface: the screens people tap). The real decision is *which class owns the money arithmetic*, so the books still balance after every single operation.

Questions to ask the interviewer (with the answers we assume):

Question	Assumed answer
Which split types?	Equal, exact amounts, percentage. More may come later.
Must shares add up to the total exactly?	Yes. If they don't, the whole expense is rejected and no balance changes.
Rounding?	Amounts are money with 2 decimal places. No cent may be lost.
Groups, or only one-to-one?	Both. A group is just a name plus a member list (a trip, a shared flat).
Can a person pay for only themselves?	No. That is not shared spending: at least two people (the payer counts) must be involved.
Settle up?	Yes: record a real payment that reduces a debt.
Simplify debts?	Yes: on request, replace many small debts with a shorter list of transfers.
Multiple currencies, receipts, comments, undo?	Out of scope (say so explicitly).

Requirements

Functional

1. Every user is identified by a name and an email.
2. Users form groups; a group has a name and members.
3. An expense records a description, total, who paid and who shares it.
4. Split types: equal, exact, percent.
5. Shares must add up to the total; otherwise the expense is refused and **nothing changes**.
6. At least two distinct people (the payer plus the sharers) must be involved in an expense.

7. Balances between every pair of people are tracked continuously.
8. A user can query their balance with one specific person, and their overall net balance.
9. Settle up: a real payment reduces a debt.
10. A group can request a simplified settlement plan with fewer transfers.
11. Rounding never loses or creates a cent.
12. Expenses are stored permanently, so a group's history can be listed.

Non-functional

- Correctness of money above everything (exact decimals, no drift).
- Adding a new split type must not change existing classes (Open/Closed Principle).
- Thread-safe: two people adding expenses at the same time must not corrupt the ledger. (A **thread** is one independent line of execution inside a running program; code is **thread-safe** when several threads can use it at the same moment without corrupting its data.)

Use cases

Actors:

- **Member** — adds expenses, settles up, asks what they owe.
- **Group creator** — sets up a group and invites its members.
- **System** — works out each share, checks shares add up, updates the ledger, simplifies debts.

Main use cases: *Add an expense* (which includes *Work out each share*, *Check the shares add up*, *Update the ledger*), *Settle up*, *See what I owe*, *Create a group / add people*, *Simplify the debts*.

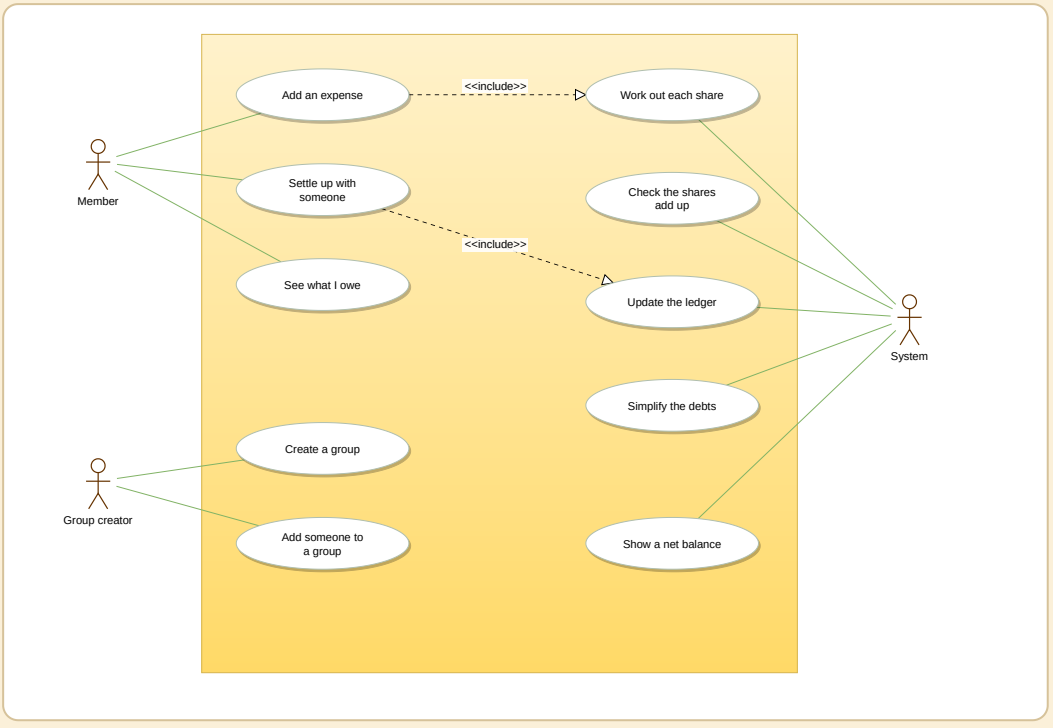


Figure — Splitwise use cases: members add expenses and settle up; the system computes shares, validates them, updates the ledger and simplifies debts (course diagram)

In the figure above, "Add an expense" *includes* the three system steps. "Include" in a use-case diagram means the step always happens as part of the bigger use case.

Core entities

Class	Responsibility
`User` (record)	Identity: id, name, email. No money.
`Group`	A named set of member ids + the list of its expense ids. **Holds no balances.**
`SplitStrategy` (interface)	Turns a total into one `Share` per person.
`EqualSplit`, `ExactSplit`, `PercentSplit`	The three rules.
`Share` (record)	A (person, amount) pair for a single expense (a value object).
`Expense`	Description, total, payer, shares. Validates itself once, when created.
`Ledger`	Who owes whom: exactly **one signed number per pair** of people. One mutator: `adjust`.
`DebtSimplifier`	Greedy minimum-cash-flow: turns net positions into a short list of payments.
`Payment` (record)	One suggested transfer (a suggestion, not a record of money moving).
`Splitwise`	Facade: the one class the app talks to. Orders the steps, contains no arithmetic.
`Money`	Tiny helper: 2-decimal `BigDecimal` values.

A few words from the table: a **record** is Java's short syntax for a class that only carries data, whose fields cannot change after creation; a **value object** is a small object defined only by its values (two shares with the same person and amount are interchangeable); a **mutator** is a method that changes an object's state; a **Facade** is one simple front class that hides several classes behind it; and a **greedy** algorithm makes the best-looking choice at each step without reconsidering earlier choices.

Why the group holds no money. If each group kept its own balances, a person in two groups would have "what Bob owes Alice" stored in two places. Sooner or later the two copies disagree. One ledger, one number per pair, is the single source of truth.

Why `Group → User` is association, not composition. Association = one object refers to another that lives independently. Composition = the part is owned and dies with the whole. Deleting a group must not delete its people, so it is association.

Class diagram

The course builds the model in five stages, and the order is worth copying in an interview because every stage leaves a slice that already works (run out of time and you still have something that runs, not half a diagram):

Stage	Adds	Works afterwards
1	`User`, `Group`	You can say who shares costs
2	`SplitStrategy` + three rules, `Share`	A total becomes one amount per person
3	`Expense` (self-checking)	A bill that does not add up is rejected
4	`Ledger`	Pairwise balances are tracked
5	`Splitwise` facade + `DebtSimplifier`	A whole trip can be recorded, settled and simplified

Here is the finished course diagram:

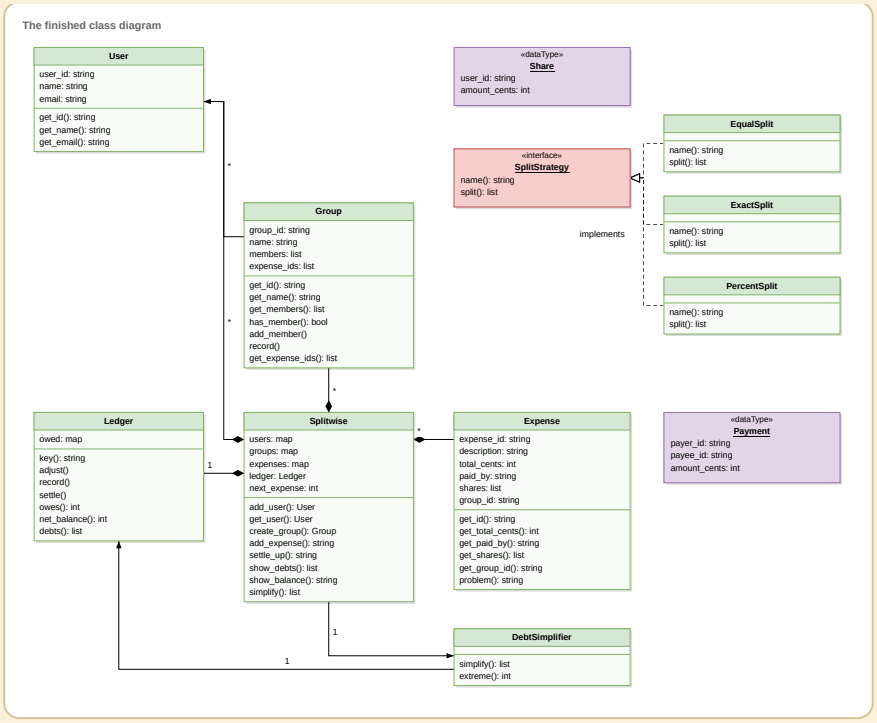
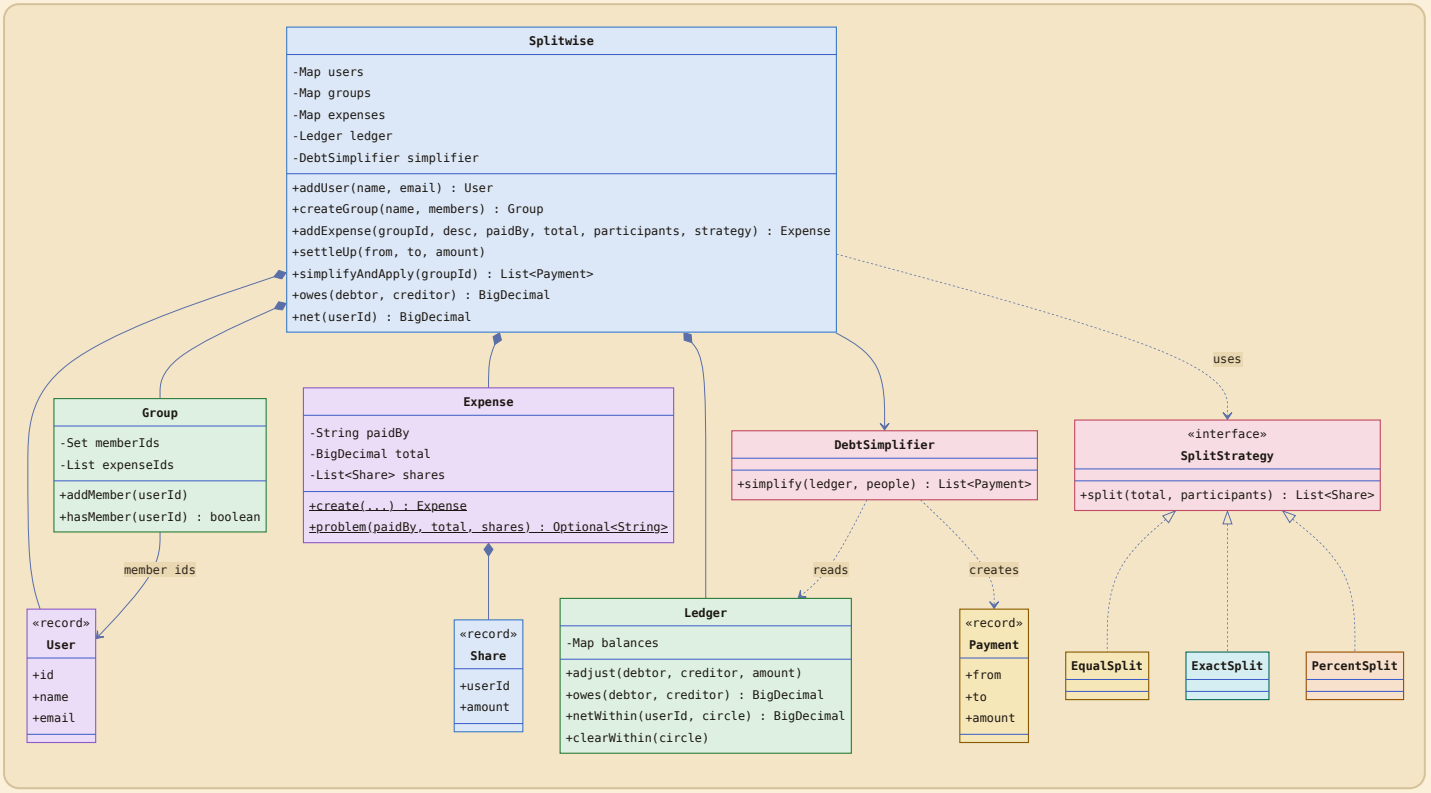


Figure — The finished Splitwise class diagram: service owning users, groups, expenses and the ledger, with the split strategies and the simplifier (course diagram)

In the figure above, the three split rules meet in one hollow triangle: they all realize the same interface. The service's filled diamonds are composition: it owns users, groups, expenses and the ledger.

Our Java code, as a class diagram:



Key flows

Adding an expense. Nothing touches the ledger until both checks pass.

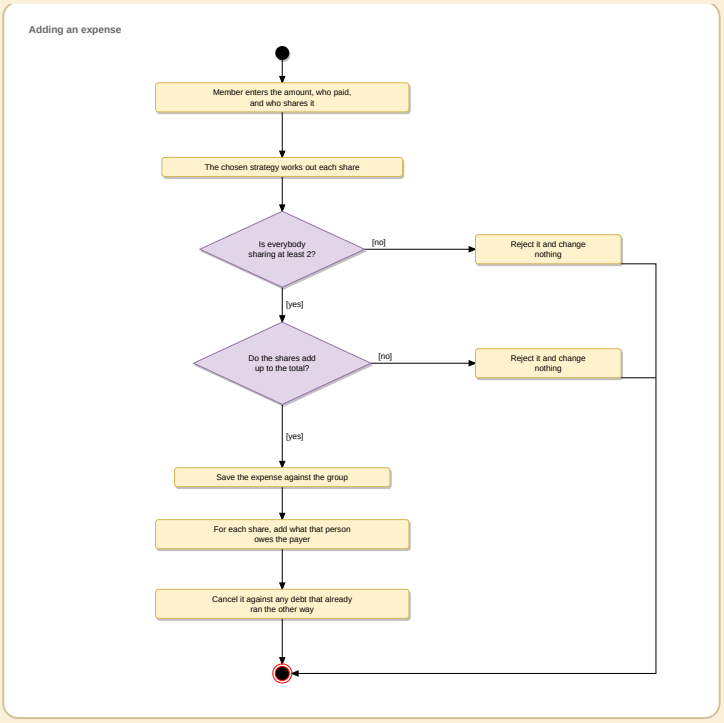
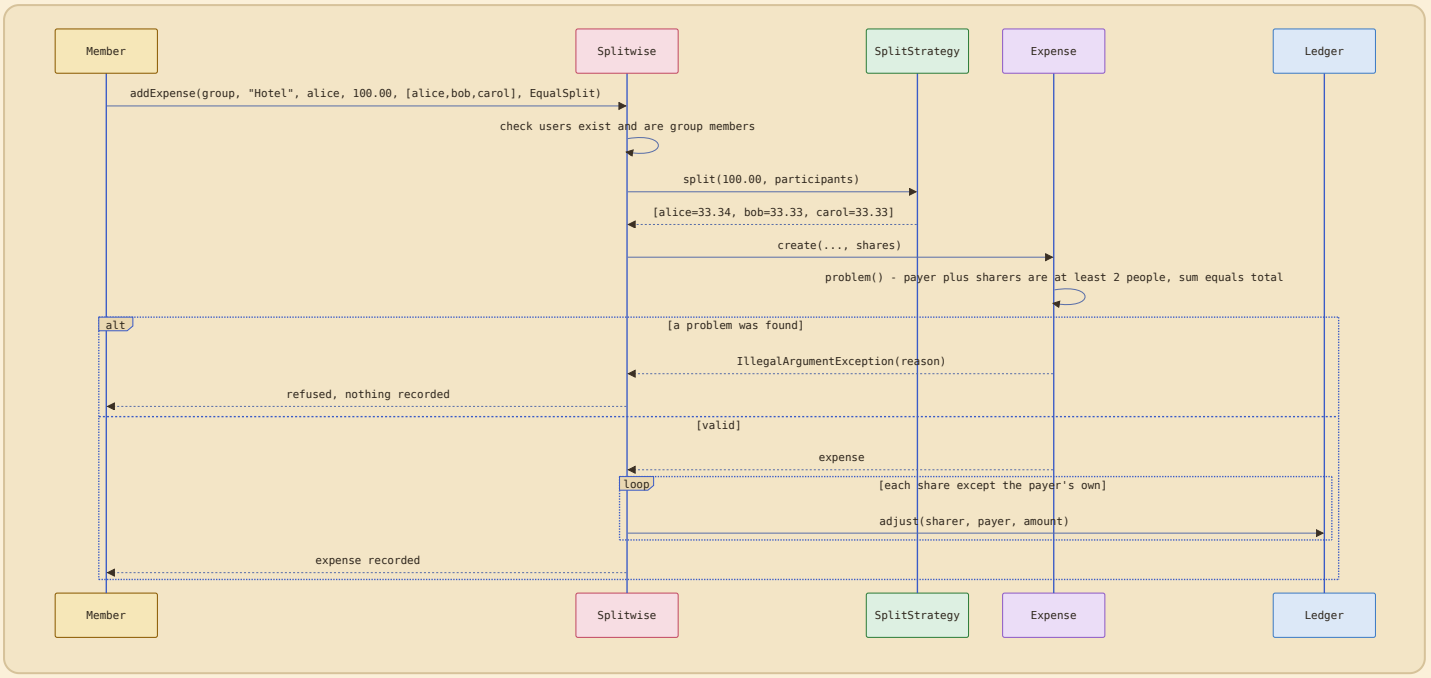


Figure — Adding an expense: the strategy computes shares, the expense checks itself, and only then is the ledger updated (course diagram)



Simplifying a group's debts (minimum cash flow, greedy).

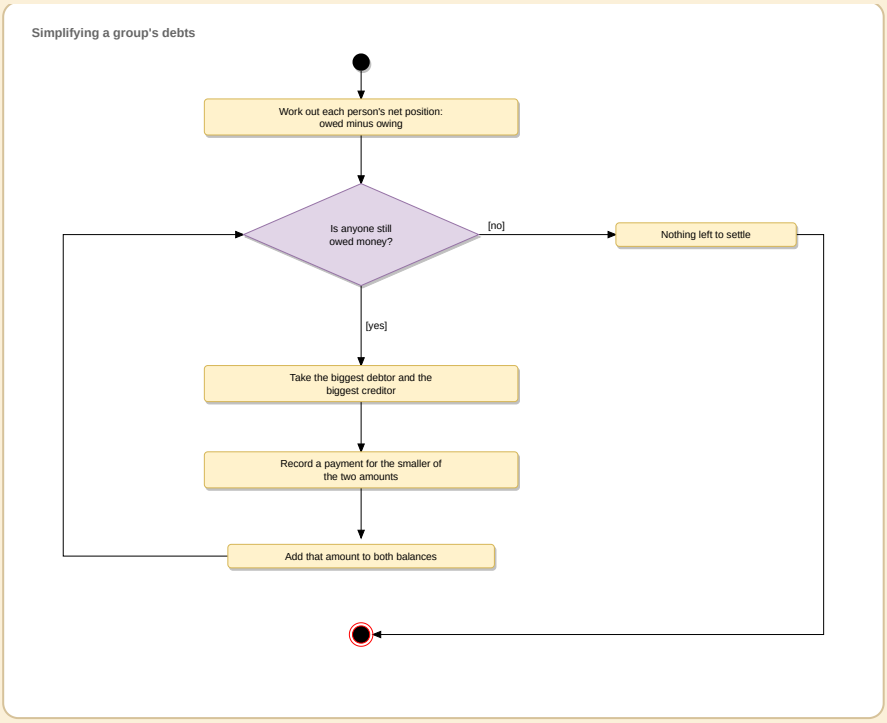


Figure — Simplifying debts: compute net positions, then the largest debtor repeatedly pays the largest creditor until every balance is zero (course diagram)

How it works, step by step:

1. For each member, compute the **net position** = (what others owe them) − (what they owe others). Positive = creditor, negative = debtor. The nets always sum to zero.
2. Put creditors in one max-heap and debtors in another (by amount; ties by name, so the output is deterministic). A **max-heap** is a priority queue that always hands back its largest item first; **deterministic** means the same input always gives the same output.
3. Pop the biggest creditor C and biggest debtor D. Let x = min(C, D). Suggest "D pays C x".
4. Push back whichever side still has a remainder.
5. Stop when a heap is empty (everyone is at zero).

Each round brings at least one person to zero, and the final round brings two to zero, so there are **at most n − 1 payments** for n people. It is **not always the true minimum**: finding the minimum number of transfers is NP-hard (no fast algorithm for it is known: the work needed explodes as the number of people grows; it is related to splitting people into as many zero-sum subgroups as possible). Say that before you are asked.

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for split rules (`SplitStrategy`)	The rule arrives from outside and is interchangeable. A fourth rule (by shares, by adjustment) is a new class. Every rule returns the same `List<Share>`, so nothing downstream knows which rule ran.	`if (type == EQUAL) ... else if (type == EXACT)` in the service: every new type edits old, tested code.
Expense validates itself in a **static factory** (a static method that builds the object instead of `new`; here `Expense.create`)	Checked once, when created; after that every class can trust its shares. An invalid expense can never exist.	Checking in the ledger: re-checks on every read and still lets bad objects exist.
`problem()` returns the **reason**, not a boolean	The user sees *why* ("shares add up to 15.00 but the total is 30.00").	`boolean isValid()`: loses the reason.
Ledger stores **one signed number per unordered pair**	A debt is never stored twice, so it can never disagree with itself. Money moving the other way cancels automatically.	Two maps "A owes B" and "B owes A": two places for the truth.
Ledger has **one mutator** (`adjust`)	Expenses, settle-ups and simplification all go through it, so the cancel rule exists in exactly one place.	Separate `addDebt`, `settle`, `reduce` methods each with their own arithmetic.
`DebtSimplifier` separate from `Ledger`	Single Responsibility (each class has one reason to change): *keeping* balances and *planning* settlements change for different reasons.	Putting the algorithm inside the ledger.
`BigDecimal` with explicit rounding; spare cents handed out one at a time	100.00 / 3 = 33.33 × 3 = 99.99 would lose a cent. Equal split gives the extra cent to the first people; percent split lets the last person absorb the remainder.	`double`: 0.1 + 0.2 ≠ 0.3. (The course keeps whole cents in a `long`; that is equally correct. We use `BigDecimal` as this book's standard.)
`Splitwise` is a **Facade**	One entry point; it orders the steps and decides nothing.	UI code calling ledger and strategies directly.

Only one named GoF pattern here (Strategy). (GoF, the "Gang of Four", are the four authors of the 1994 book that catalogued 23 classic design patterns.) That is fine. Good designs do not collect patterns; they use the one that absorbs the change that is likely.

Java implementation

Files: `Model.java` (money helper, user, group, share, payment), `SplitStrategy.java`, `Expense.java`, `Ledger.java`, `DebtSimplifier.java`, `Splitwise.java` (the facade), `Main.java`. The full runnable source is in `code/lld/ch20/splitwise/`; below are the lines that carry the design.

```java
// File: code/lld/ch20/splitwise/Model.java
return new BigDecimal(amount).setScale(2, RoundingMode.UNNECESSARY); // Money.of: refuses 10.005

// File: code/lld/ch20/splitwise/SplitStrategy.java
/** Strategy: turns one total into one Share per participant. */
interface SplitStrategy {
    List<Share> split(BigDecimal total, List<String> participants);
}

/** Equal split. Spare cents are handed out one at a time, in participant order. */
final class EqualSplit implements SplitStrategy {
    @Override
    public List<Share> split(BigDecimal total, List<String> participants) {
        BigDecimal n = BigDecimal.valueOf(participants.size());
        BigDecimal base = total.divide(n, 2, RoundingMode.DOWN);           // 100.00/3 -> 33.33
        BigDecimal leftover = total.subtract(base.multiply(n));           // 0.01
        int spareCents = leftover.divide(Money.CENT).intValueExact();     // 1
        List<Share> shares = new ArrayList<>();
        for (int i = 0; i < participants.size(); i++) {
            BigDecimal amount = i < spareCents ? base.add(Money.CENT) : base;
            shares.add(new Share(participants.get(i), amount));
        }
        return shares;
    }
}
// ExactSplit: the caller states each amount. PercentSplit: percentages must total 100,
// each share rounded HALF_EVEN, and the last person gets total - given (absorbs the remainder).

// File: code/lld/ch20/splitwise/Ledger.java
/** Exactly ONE signed number per unordered pair; key (lo, hi), positive = "hi owes lo". */
private record Pair(String lo, String hi) {
    static Pair of(String a, String b) { return a.compareTo(b) < 0 ? new Pair(a, b) : new Pair(b, a); }
}

/** The ONLY mutator: 'debtor' now owes 'creditor' 'amount' more. Opposite debts cancel. */
void adjust(String debtor, String creditor, BigDecimal amount) {
    if (debtor.equals(creditor) || amount.signum() == 0) return;
    Pair key = Pair.of(debtor, creditor);
    BigDecimal delta = creditor.equals(key.lo()) ? amount : amount.negate();
    BigDecimal updated = balances.getOrDefault(key, Money.ZERO).add(delta);
    if (updated.signum() == 0) balances.remove(key); else balances.put(key, updated);
}

// File: code/lld/ch20/splitwise/DebtSimplifier.java
/** Greedy "minimum cash flow": largest debtor pays largest creditor, repeat. At most (people - 1) payments. */
while (!creditors.isEmpty() && !debtors.isEmpty()) {     // two max-heaps of net positions
    Position c = creditors.poll();
    Position d = debtors.poll();
    BigDecimal x = c.amount().min(d.amount());
    payments.add(new Payment(d.userId(), c.userId(), x));
    if (c.amount().compareTo(x) > 0) creditors.add(new Position(c.userId(), c.amount().subtract(x)));
    if (d.amount().compareTo(x) > 0) debtors.add(new Position(d.userId(), d.amount().subtract(x)));
}

// File: code/lld/ch20/splitwise/Splitwise.java
synchronized Expense addExpense(String groupId, String description, String paidBy, BigDecimal total,
                                List<String> participants, SplitStrategy strategy) {
    // ... every participant (and the payer) must exist and belong to the group
    List<Share> shares = strategy.split(total, participants);       // may throw
    Expense e = Expense.create("e" + ids.incrementAndGet(), description, paidBy, total, shares, groupId); //
validates
    for (Share s : e.shares()) ledger.adjust(s.userId(), paidBy, s.amount()); // ledger touched only after
validation
    expenses.put(e.id(), e);
    if (group != null) group.recordExpense(e.id());
    return e;
}
```

Walking through the code.

- `Money.of("10.005")` throws, because `RoundingMode.UNNECESSARY` refuses to silently round. Inputs with more than 2 decimals are a bug, not something to hide.
- `EqualSplit` divides with `RoundingMode.DOWN` (33.33), computes the leftover (0.01), and gives one extra cent to the first participant(s). The shares always add up exactly.
- `PercentSplit` rejects percentages that do not total 100, rounds each share with `HALF_EVEN` (banker's rounding: halves go to the even neighbour, which avoids a systematic upward bias), and lets the **last** person absorb whatever remains.
- `Ledger.Pair.of(a, b)` sorts the two ids so (alice, bob) and (bob, alice) are the same key. The sign of the stored value says who owes whom.
- `Splitwise.addExpense` builds shares, then the expense (which validates), and **only then** updates the ledger. That order is requirement 5.
- `settleUp` refuses to pay more than is owed, then calls `adjust(to, from, amount)`: a debt in the reverse direction that cancels the existing one.
- `simplifyAndApply` replaces all debts between the group's members with the simplified plan. Every net position stays the same (the output proves it).

Output:

```
== 1. Equal split with a spare cent
Hotel shares: [alice=33.34, bob=33.33, carol=33.33]
== 2. Exact split
== 3. Percent split
Taxi shares: [alice=16.66, bob=16.66, dave=16.68]
== 4. Invalid expenses are refused, ledger untouched
  refused: shares add up to 15.00 but the total is 30.00
  refused: an expense needs at least two people
  refused: percentages add up to 90, not 100
== 5. Who owes whom
  bob owes alice 23.33
  carol owes alice 16.67
  carol owes bob 13.34
  dave owes carol 16.68
  net alice = 40.00
  net bob = -9.99
  net carol = -13.33
  net dave = -16.68
  bob owes alice: 23.33
== 6. Simplify the group's debts (min cash flow, greedy)
  suggested: dave pays alice 16.68
  suggested: carol pays alice 13.33
  suggested: bob pays alice 9.99
  bob owes alice 9.99
  carol owes alice 13.33
  dave owes alice 16.68
  net alice = 40.00
  net bob = -9.99
  net carol = -13.33
  net dave = -16.68
== 7. Settle up
  refused: dave owes alice only 16.68, cannot pay 999.00
  after a partial payment of 5.00, bob owes alice: 4.99
  (everyone is settled up)
== 8. Group history
  e2 Hotel 100.00 paid by alice [alice=33.34, bob=33.33, carol=33.33]
  e3 Dinner 60.00 paid by bob [alice=10.00, bob=20.00, carol=30.00]
  e4 Taxi 50.00 paid by carol [alice=16.66, bob=16.66, dave=16.68]
```

Look at step 5 and step 6: four pairwise debts became three payments, and every net (alice +40.00, bob −9.99, carol −13.33, dave −16.68) is identical before and after.

Related machine-coding problem: The Ledger Co. (Navi)

Splitwise is not the only "ledger" question you will meet. The Geektrust / Navi machine-coding round (an interview round where you write a complete, runnable program in one or two hours) **"Ledger Co."** also keeps money records, but for **bank loans** (simple interest — interest charged only on the original amount

borrowed; a fixed monthly instalment; lump-sum prepayments — extra one-off payments made early) rather than bills shared between friends. It is fully designed, coded and run in **Chapter 21, §21.6**, so it is not repeated here. The comparison is the useful part: both keep **one authoritative record** and compute every answer from it (Splitwise: one signed balance per pair of people; Ledger Co.: one loan per bank and borrower, holding its lump-sum payments), and both use `BigDecimal` with an explicitly named rounding mode. What differs is the rule set: Splitwise's rules are about *who owes whom*, Ledger Co.'s are about *how much has been repaid by a given month*.

Concurrency & edge cases

- **Two expenses at once.** Every public method on `Splitwise` is `synchronized` (a Java keyword that lets only one thread at a time run such methods on the same object), so "build → validate → adjust ledger" is atomic (it happens as one indivisible step that no other thread can see half-done). Simple and correct for one process. At scale you would lock per group or per pair of users (a **lock** is an object only one thread can hold at a time; others wait for it), or store balances in a database and update them in one transaction (a group of database changes that either all succeed or all fail together).
- **Rounding.** Covered by spare-cent distribution and "last person absorbs". Test: 100.00 / 3 must give 33.34 + 33.33 + 33.33.
- **Payer not among the sharers** (Alice pays for Bob and Carol only, or for Bob alone): allowed. Every share becomes a debt to Alice. `Expense.problem` counts the payer as one of the "at least two people", so Alice buying Bob's lunch is a valid expense, while Dave paying only for Dave is refused.
- **Payer's own share:** `adjust(alice, alice, …)` is a no-op (does nothing) by design.
- **Overpaying on settle-up:** refused, so a debt can never flip direction by accident.
- **Unknown user / not a group member:** refused before any computation.
- **Duplicate person in the shares list, negative share, zero total:** refused by `Expense.problem`.
- **Simplify and other groups.** Because the ledger has one number per *pair*, simplifying a group rewrites all debts *between its members*, even ones that came from another group with the same two people. That is the price of the single source of truth. If per-group simplification must not touch other groups, keep the ledger per group *and* a global view derived from them.

Extensions & follow-up questions

- **"Add a split by shares (e.g. 2:1:1)."** A new `ShareRatioSplit implements SplitStrategy`. Nothing else changes.
- **"Delete or edit an expense."** Our ledger stores totals, not history, so undo means re-adding the inverse (`adjust(payer, sharer, amount)` for each share). Cleaner: make the ledger an **append-only list of entries** (like accounting journals / event sourcing — storing every change as an event and rebuilding the current state from them) and compute balances on demand or in a cache (a saved copy of computed results kept for quick reuse; see 20.3). Then delete = append a reversing entry.
- **"Multiple currencies."** Add a `Currency` to every amount (a `Money(amount, currency)` value object) and keep one ledger per currency, or convert at the time of the expense and store the rate used.
- **"Is the greedy always optimal?"** No. Counter-example: nets A −6, B −5, C +2, D +4, E +5. Greedy: A pays E 5, B pays D 4, then two more small payments to C = **4 payments**. Optimal: B pays E 5 (they cancel exactly), A pays D 4 and C 2 = **3 payments**. The optimum groups people into as many zero-sum subsets as possible, which needs an exponential search; greedy is the accepted interview answer.
- **"Isn't `Splitwise` doing too much?"** A fair criticism: it holds users, groups, expenses *and* the ledger. At larger scale, give each of the four its own repository (store) and let `Splitwise` shrink to the part that orders the steps. Say this before the interviewer does.
- **"Show a user's activity feed."** Keep expenses and settle-ups as events; the feed is a query over them.
- **"Notify people when an expense is added."** Publish an `ExpenseAdded` event to an Observer / the notification service in 20.2.

Real-world example

Splitwise really does offer a "simplify debts" option for groups: when it is on, the app shows each member a small number of payments to make instead of every pairwise debt, exactly the net-position idea above. Payment apps with "split the bill" features face the same rounding problem and solve it the same way: one person absorbs the leftover cent.

In the interview

30-second answer: "Users and groups hold no money. Each split rule is a `SplitStrategy` that returns shares. An `Expense` validates itself once: at least two people, shares add to the total. The `Ledger` keeps one signed number per pair of people and has a single `adjust` method, so debts cancel automatically. Settling up is just an adjust in the other direction. Simplification uses net positions and a greedy biggest-debtor-pays-biggest-creditor loop: at most n − 1 payments, not always the true minimum."

Tip: reach the split rules early and say the phrase "one class per split rule" while you draw them; that is the part of the question with a pattern name. Then invest your time in the ledger, because the one-number-per-pair decision is what keeps everything else short.

Suggested 40-minute plan: clarify (5) → entities + the "no money in groups" decision (5) → Strategy for splits and the cents problem (8) → Expense validation (5) → Ledger with one number per pair (7) → simplify algorithm (5) → concurrency and extensions (5).

Likely follow-ups (with answers):

- *"What if 100 does not divide by 3?"* → Hand out spare cents one by one; the total is always exact.
- *"How do you simplify debts?"* → Net positions + greedy; mention n − 1 bound and that optimal is NP-hard.
- *"Why not store balances in the group?"* → A person in two groups would have two copies of the truth.

Self-check

1. Why does `Group` hold no balances? → So each debt lives in exactly one place (the ledger); two copies would eventually disagree.
2. Split 10.00 equally among 3 people. → 3.34, 3.33, 3.33.
3. Why does `Expense.problem` return a `String` instead of a `boolean`? → So the caller can show the reason it was refused.
4. What is the maximum number of payments the greedy simplifier suggests for n people? → n − 1.
5. Why compare `BigDecimal` with `compareTo` rather than `equals`? → `equals` also compares scale, so `2.0` does not equal `2.00`.
6. Alice pays 20.00 for Bob's lunch and Bob owes the whole 20.00. Is that a valid expense? → Yes. Two people are involved (payer and sharer), even though only one of them has a share.

TL;DR

- **Strategy** for split rules; every rule returns shares.
- **Expense validates itself** once; invalid expenses never exist.
- **Ledger = one signed number per pair**, one mutator; settle-up is a reverse adjust.
- **Simplify** = net positions + greedy; ≤ n − 1 payments; optimal is NP-hard.
- **BigDecimal**, explicit rounding, no lost cents.

20.2 Design a Notification Service

Problem & clarifying questions

What it is. A notification service is the component of a product responsible for emails, text messages (SMS) and push alerts ("Your order has shipped", "Your login code is 482913"). Everyday analogy: a post office counter. Many departments hand it letters; the counter picks the right delivery method, respects "do not disturb" notes, retries if the courier fails, and keeps a receipt for every letter.

Why it is a design question. Every product grows one. It starts as one `sendEmail()` function. Then someone adds SMS. Then "don't wake me at 3 a.m.". Then marketing wants "max 2 promos per hour". Six months later one method has fourteen `if`s and nobody dares touch it. The whole exercise is: **where should each of those additions have gone instead?**

Question	Assumed answer
Channels?	Email, SMS, push. More later (WhatsApp, Slack).
Who chooses the channel?	The user, **per kind** of notification (SMS for deliveries, email for newsletters), in order of preference.
Wording?	Templates with placeholders, e.g. `Hi {name}, it arrives {eta}.`
Quiet hours?	Yes, per user; they can wrap past midnight (22:00–07:00). Urgent messages (login codes) ignore them.
Rate limits?	At most N notifications of one kind per user per hour.
Failures?	Retry a channel up to a limit, then fall back to the next accepted channel; if all fail, record FAILED.
Audit?	Every attempt recorded: SENT, FAILED or SUPPRESSED, and why.
Sync or async? (synchronous = caller waits; asynchronous = caller moves on)	The caller should not wait for a slow SMS gateway (the provider service that actually sends texts).
Bulk sends / translations / open tracking / unsubscribe links?	Out of scope.

Requirements

Functional

1. A user has a name, email, phone number and device token (the address a push service uses for one phone).
2. Channels: email, SMS, push.
3. Each user chooses accepted channels **per kind** of notification, in order.
4. The text for each kind comes from a template with placeholders ("holes") filled in per request.
5. Users can set quiet hours; nothing normal is sent during them.
6. Urgent notifications ignore quiet hours.
7. A user receives no more than N notifications of a given kind in any hour.
8. A channel that fails is retried, but only a bounded number of times.
9. When a channel runs out of retries, the service falls back to the user's next accepted channel.
10. When no channel succeeds, the outcome is stored as FAILED.
11. Every attempt is recorded with its outcome and reason.
12. Adding a channel, **or** a new reason to hold a message back, requires no edit to the service class.

Non-functional: the caller is not blocked by slow providers (async option); thread-safe under concurrent sends; testable without waiting for real time (injected clock); a broken listener must never break delivery. An **injected** clock is one handed to the class from outside (instead of the class reading

the system time itself), so a test can pass in a fake clock it moves by hand; a **listener** is an object that has asked to be told when something happens.

Use cases

Actors: **Product code** (asks for a notification), **Person** (chooses channels, sets quiet hours), **Operator** (adds channels, watches outcomes), **Service** (renders, checks rules, tries channels, retries, records).

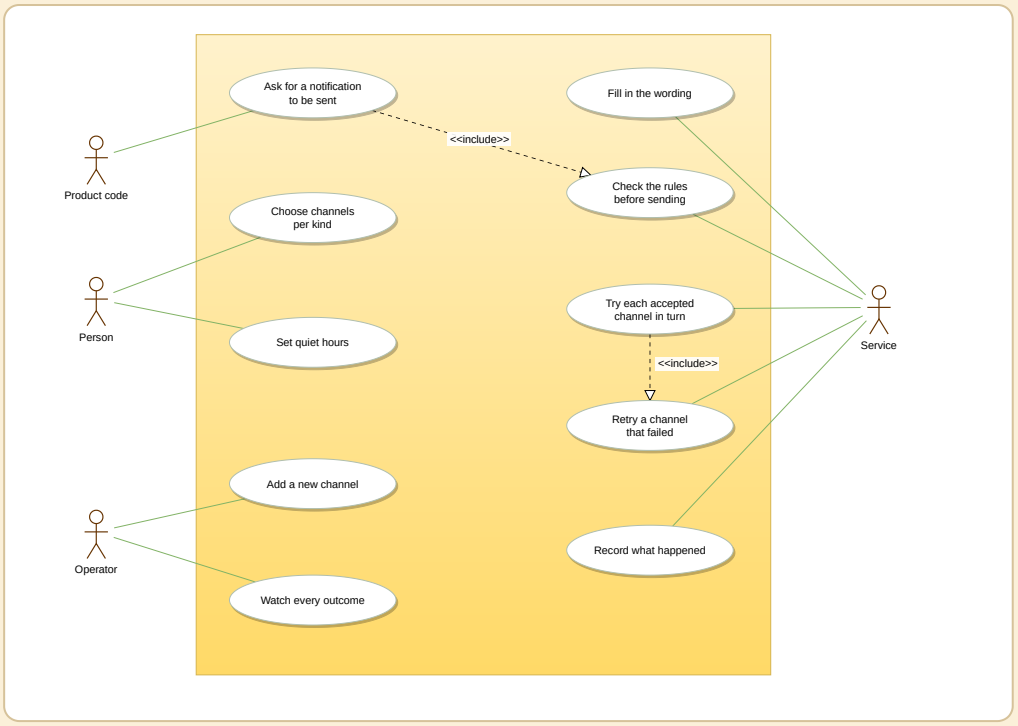


Figure — Notification service use cases: product code asks for a notification; the service checks rules, fills in the wording, tries each channel with retries and records the outcome (course diagram)

Core entities

Class	Responsibility
`User` (record)	Who someone is: id, name, email, phone, device token.
`Preferences`	What they accept: ordered channels per kind; quiet hours (with midnight wrap logic in **one** place).
`Template` (record)	Subject + body with `{holes}`; `render(values)` → `Message`.
`Message` (record)	The finished subject and body, identical on every channel.
`NotificationRequest` (record)	One ask: kind, user, values, priority (NORMAL / URGENT).
`Gateway` (interface)	The external provider (SMTP server, SMS gateway, push service). Swappable and fakeable.
`NotificationChannel` (abstract)	**Template Method**: the final `deliver()` holds the retry-with-backoff loop; subclasses supply only `addressOf(user)`.
`EmailChannel`, `SmsChannel`, `PushChannel`	Three tiny subclasses.
`ChannelFactory`	**Simple Factory**: channel type → channel object.
`RetryPolicy` (record)	Max attempts + initial backoff (doubles each time).
`SendRule` (interface)	**Strategy**: one reason not to send. Returns the reason text, or empty.
`QuietHoursRule`, `RateLimitRule`	Two rules. Quiet hours knows urgent bypasses it; rate limit counts only *successful* sends.
`DeliveryRecord` (record)	What happened: status, channel, attempts, reason.
`DeliveryListener` (interface)	**Observer**: told every outcome. `LoggingListener`, `MetricsListener`.
`TimeSource`	Where "now" comes from (a manual clock in tests).
`NotificationService`	Thin coordinator: rules → preferences → render → channels in order → `finish()`. Decides nothing itself.

Terms from the table: **SMTP** is the protocol mail servers use to pass email along; a **fake** is a pretend version of a real dependency used in tests; an **abstract** class cannot be created on its own and exists to be extended; a `final` method cannot be overridden by subclasses; **backoff** means waiting a little longer before each new retry, so a struggling provider gets time to recover.

Why `User` and `Preferences` are separate classes. Who you are rarely changes; what you are willing to receive changes often. Separate reasons to change → separate classes (Single Responsibility Principle).

Class diagram

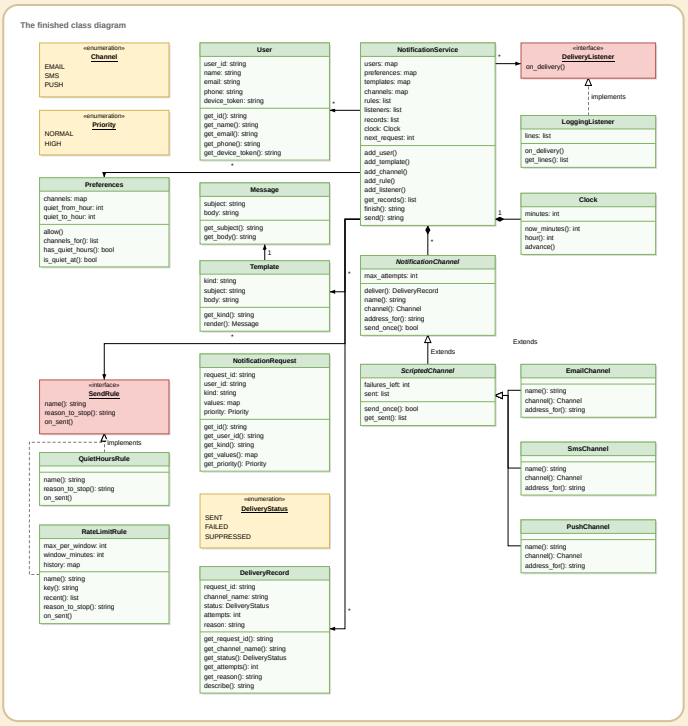
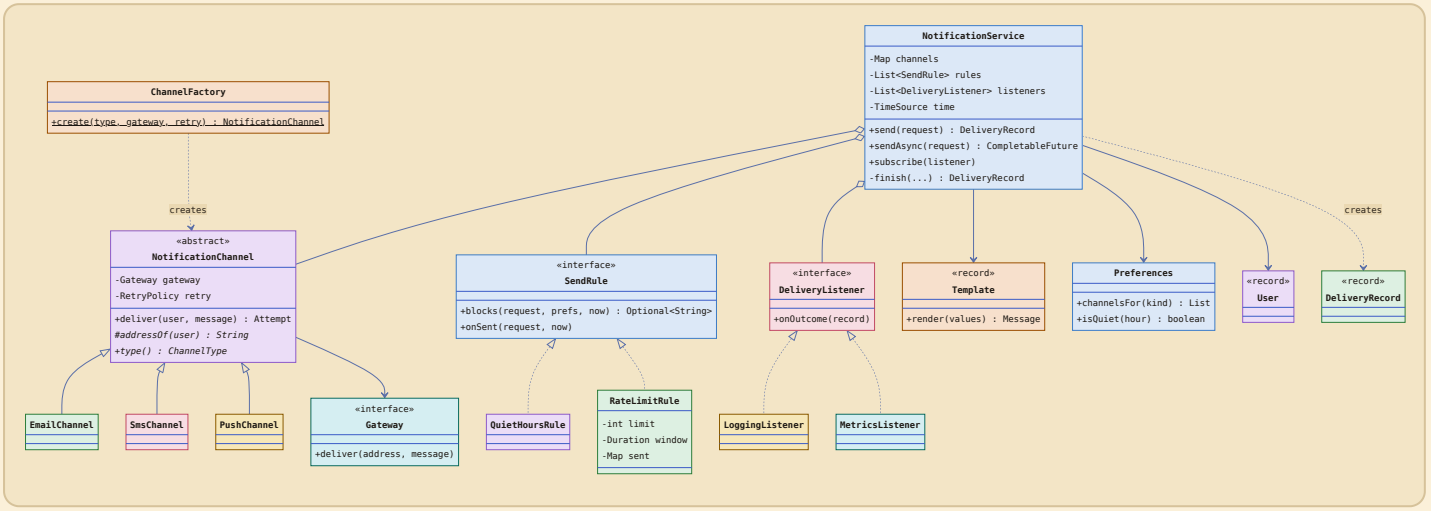


Figure — The finished notification service class diagram: the service in the middle, the channel hierarchy with the retry in the parent, the send rules, templates, preferences and the delivery listeners (course diagram)

In the figure above, notice that the three channel classes are tiny: all the behaviour that makes sending reliable lives in the abstract parent.

The course reaches this diagram in five build stages, each leaving something that already works: (1) `User` + `Preferences` (you can ask what a person accepts, and when); (2) `Template` + `Message` + `NotificationRequest` (wording filled in once); (3) the channel hierarchy (delivery with retries and an outcome); (4) the `SendRule`s (a notification can be stopped before sending); (5) `NotificationService` + listeners (end-to-end sending, observable by any listener). Reuse this order when you interview.



Key flows

Sending one notification. A rule (or a missing channel preference) may stop it before any work is done; otherwise channels are tried in the user's order until one succeeds.

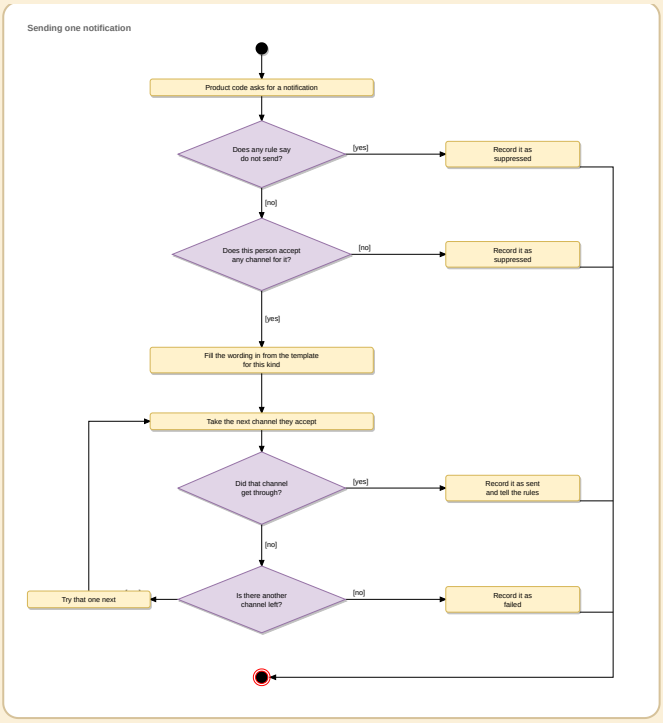


Figure — Sending one notification: ask every rule, render once, try each accepted channel in turn, and record sent / failed / suppressed (course diagram)

Inside one channel — the retry loop, which exists exactly once, in the abstract parent:

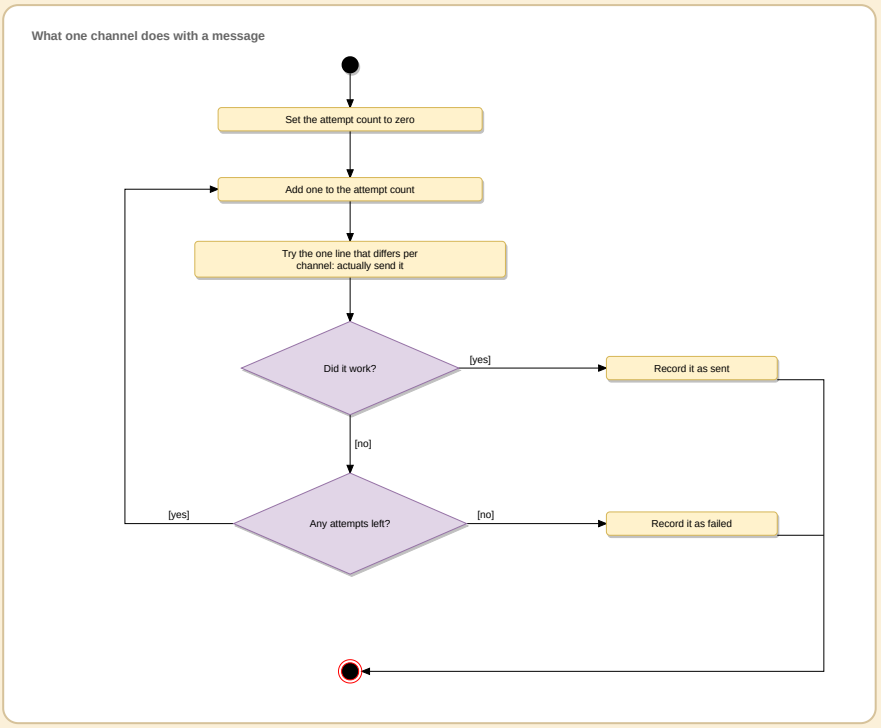
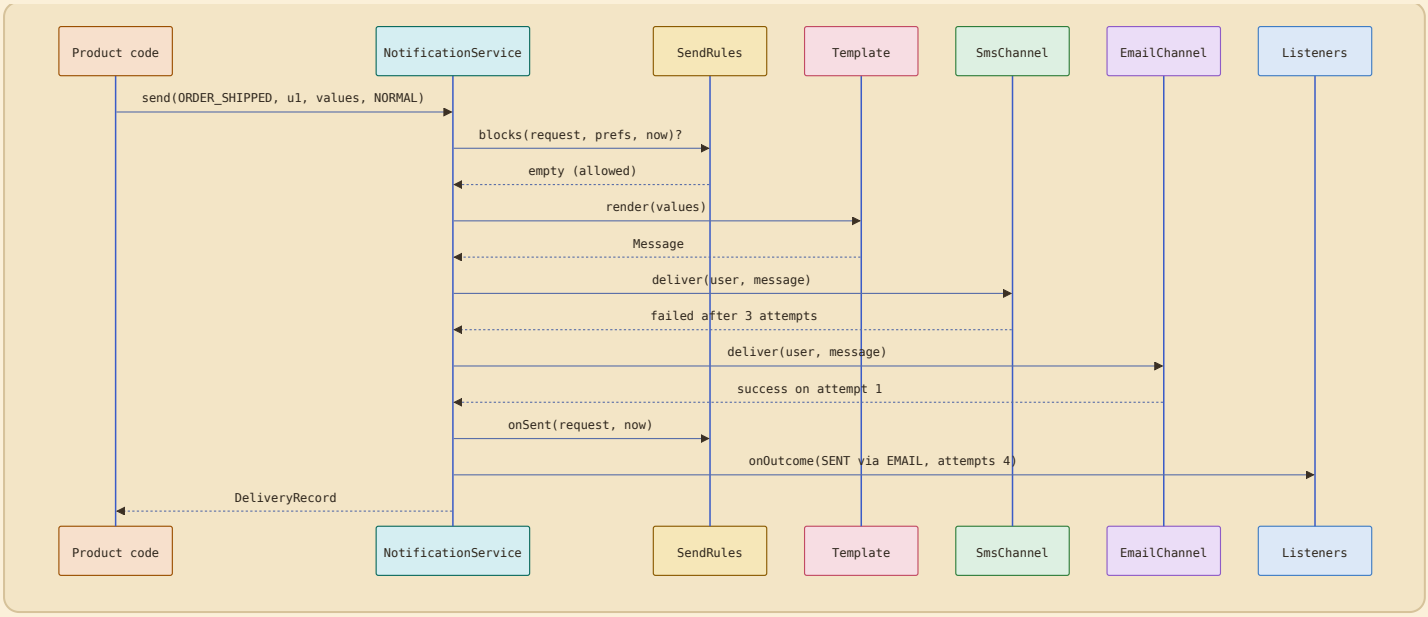


Figure — One channel's delivery loop: attempt, and on failure retry while attempts remain; then report success or give up (course diagram)


Product code
NotificationService
SendRules
Template
SmsChannel
EmailChannel
Listeners
send(ORDER_SHIPPED, u1, values, NORMAL)
blocks(request, prefs, now)?
empty (allowed)
render(values)
Message
deliver(user, message)
failed after 3 attempts
deliver(user, message)
success on attempt 1
onSent(request, now)
onOutcome(SENT via EMAIL, attempts 4)
DeliveryRecord
Product code
NotificationService
SendRules
Template
SmsChannel
EmailChannel
Listeners

Design decisions & patterns used

Pattern / decision	Where	Why	Alternative rejected
Template Method	`NotificationChannel.deliver()` is `final`; subclasses fill `addressOf()`	The retry loop is identical for all channels. Write it once; new channels inherit reliability. Only a *step* varies, not the whole algorithm.	Copying the loop into every channel (they drift apart), or Strategy (would replace the whole method, not one step).
Strategy	`SendRule`	Every "reason not to send" is its own class. A new rule (e.g. "user unsubscribed", "duplicate within 5 minutes") is a new class plus one line in the wiring code.	`if (quiet) … if (tooMany) …` inside `send`.
Observer	`DeliveryListener`	Logs, dashboards, alerts all want outcomes. The service announces; it doesn't know who listens.	Hard-coding logging/metrics in the service; it grows forever.
Simple Factory	`ChannelFactory`	One place maps `ChannelType` → class; config can list channel names.	`new SmsChannel(...)` scattered through wiring code.
Adapter (target interface; an adapter converts one API into the shape your code expects)	`Gateway`	Each real provider SDK (the vendor's client library) has its own API (the set of calls a library or service offers to programs); an adapter per provider implements `Gateway`. Also lets tests use a fake.	Channels calling vendor SDKs directly.
Rule returns **reason text**, not boolean	`SendRule.blocks`	The reason is recorded (requirement 11).	`boolean allowed()` loses why.
Urgency handled **inside** the rule	`QuietHoursRule`	The service doesn't need to know which rules urgent messages ignore.	A special case in the service.
Rate limit counts only **successful** sends	`onSent` hook	A suppressed/failed message shouldn't use up the user's hourly allowance.	Counting at check time.
Single exit `finish()`	Service	Every path stores a record and notifies listeners. A second exit that forgot would leave invisible gaps.	Returning from many places.
Template rendered **once**	Service	Every channel gets the same text; a wording change is one edit.	Wording inside each channel.

Using three patterns here is not decoration: each one soaks up a *separate* axis of change — another channel (Template Method + Factory), another reason to hold a message back (Strategy), another party interested in outcomes (Observer).

Java implementation

Files: `Model.java` (channel types, user, preferences with quiet hours, template, request, record, time source), `Channels.java` (gateway port, retry policy, channel template method, three channels, factory),

`Rules.java` (send rules, listeners), `NotificationService.java`, `Main.java`. The full runnable source is in `code/lld/ch20/notification/`; below are the lines that carry the design.

```java
// File: code/lld/ch20/notification/Channels.java
/**
 * Template Method: deliver() is the fixed algorithm (try, back off, retry) and is final.
 * Subclasses fill in only the steps that differ: which address to use.
 */
abstract class NotificationChannel {
    abstract ChannelType type();
    /** Hook: where this channel reaches the user, or null if the user has no such address. */
    protected abstract String addressOf(User user);

    final Attempt deliver(User user, Message message) {
        String address = addressOf(user);
        if (address == null || address.isBlank()) return new Attempt(false, 0, "no " + type() + " address");
        String lastError = null;
        long backoff = retry.initialBackoff().toMillis();
        for (int attempt = 1; attempt <= retry.maxAttempts(); attempt++) {
            try {
                gateway.deliver(address, message);           // Gateway: the outside provider (adapter target)
                return new Attempt(true, attempt, null);
            } catch (Exception e) {
                lastError = e.getMessage();
                if (attempt < retry.maxAttempts()) sleep(backoff);
                backoff *= 2;                                // exponential backoff
            }
        }
        return new Attempt(false, retry.maxAttempts(), lastError);
    }
}
// ChannelFactory.create(type, gateway, retry): switch -> EmailChannel | SmsChannel | PushChannel

// File: code/lld/ch20/notification/Rules.java
/** Strategy: one reason not to send. Returns the reason, or empty to let it through. */
interface SendRule {
    Optional<String> blocks(NotificationRequest request, Preferences prefs, LocalDateTime now);
    /** Told after a successful send (so suppressed messages do not count). */
    default void onSent(NotificationRequest request, LocalDateTime now) {}
}
// QuietHoursRule: URGENT ignores it, otherwise prefs.isQuiet(hour).
// RateLimitRule: a Deque of send times per (user, kind), pruned on every check; blocks at 'limit'.

// File: code/lld/ch20/notification/NotificationService.java
/** Synchronous send: rules -> render -> try each accepted channel in the user's order. */
DeliveryRecord send(NotificationRequest req) {
    // ... unknown user -> FAILED
    for (SendRule rule : rules) {
        Optional<String> reason = rule.blocks(req, prefs, now);
        if (reason.isPresent()) return finish(req, DeliveryStatus.SUPPRESSED, null, 0, reason.get());
    }
    List<ChannelType> accepted = prefs.channelsFor(req.kind());
    // ... no channel -> SUPPRESSED; no template -> FAILED
    Message message;
    try {
        message = template.render(req.values()); // rendered ONCE, same text on every channel
    } catch (IllegalArgumentException e) {        // e.g. a {hole} with no value: still leave via finish()
        return finish(req, DeliveryStatus.FAILED, null, 0, "template: " + e.getMessage());
    }
    int totalAttempts = 0;
    String lastError = null;
    for (ChannelType type : accepted) {           // fallback: next channel if this one fails
        NotificationChannel channel = channels.get(type);
        if (channel == null) continue;
        Attempt a = channel.deliver(user, message);
        totalAttempts += a.attempts();
        if (a.success()) {
            rules.forEach(r -> r.onSent(req, now));
            return finish(req, DeliveryStatus.SENT, type, totalAttempts, null);
        }
        lastError = type + ": " + a.error();
    }
    return finish(req, DeliveryStatus.FAILED, null, totalAttempts, "all channels failed, last " + lastError);
```

```
}
// finish(): the single exit - store the DeliveryRecord, then tell every DeliveryListener (a throwing listener is caught)
```

Walking through the code.

- `Preferences.isQuiet` handles the midnight wrap: if start ≤ end (09–17) it is a simple range; otherwise (22–07) it is "after start **or** before end".
- `Template.render` uses a regex (regular expression: a small pattern language for finding text) `\{(\w+)}` and `Matcher.appendReplacement` with `Matcher.quoteReplacement`, so a value containing `$` or `\` is inserted literally. A missing value is an error, not a silent `{name}` in a customer's inbox.
- `NotificationChannel.deliver` is `final`: subclasses cannot break the retry loop. Backoff doubles after each failure (exponential backoff). The course's version retries immediately and names that as a weakness (three back-to-back tries against a server that is down are three quick failures); the waiting belongs in this one parent loop, which is exactly where we put it.
- `RateLimitRule` keeps a deque (double-ended queue: items can be added or removed at either end) of send times per (user, kind) and **prunes** old ones on every check, so one key's deque never holds more than `limit` entries (the course's version never discarded old timestamps and flagged that as a known leak — a **memory leak** is memory a program keeps holding although it no longer needs it). The *map of keys* still grows with every (user, kind) pair ever seen; a production version adds a periodic sweep that drops empty or stale deques, or uses a fixed-size window counter per key.
- `NotificationService.send` has one exit, `finish()`. Even a template with a missing value is caught and recorded as FAILED (step 7 of the output) instead of escaping as an exception that no listener would ever hear about. A listener that throws is caught so it cannot break delivery.
- `NotificationRequest` copies its values into a `TreeMap` (a map that keeps its keys sorted), so the values are always in the same order and the same request always renders the same text: handy when comparing test output.
- `sendAsync` runs `send` on a worker pool (a fixed set of threads that pick up tasks as they arrive) and returns a `CompletableFuture` (a handle to a result that will be ready later); in production a durable queue (Kafka, SQS) would sit here instead.

> **Key term — message queue:** a queue is a waiting line for work. Producers put messages in at one end; workers take them out at the other, at their own pace. A **durable** queue stores its messages on disk, so they survive a crash or restart. **Kafka** (an open-source log-based messaging system) and **Amazon SQS** (a managed queue service in AWS, Amazon's cloud) are two widely used examples. See System Design Fundamentals for queues in larger systems.

Output:

```
== 1. Normal send over the first accepted channel (SMS)
    SMS-GW -> +91-98450 : Order A-17 shipped | Hi Asha, it arrives Tuesday.
  [log] SENT       ORDER_SHIPPED user=u1 channel=SMS attempts=1
== 2. SMS fails 3 times -> retried, then falls back to EMAIL
    SMTP -> asha@mail.com : Order A-17 shipped | Hi Asha, it arrives Tuesday.
  [log] SENT       ORDER_SHIPPED user=u1 channel=EMAIL attempts=4
== 3. Quiet hours at 23:00: normal is suppressed, urgent goes through
  [log] SUPPRESSED PROMO         user=u1 channel=- attempts=0 reason=quiet hours
    FCM -> tok-asha : Your code | 482913 is your login code.
  [log] SENT       OTP           user=u1 channel=PUSH attempts=1
== 4. Rate limit: 2 PROMO per hour, the 3rd is suppressed
    SMTP -> asha@mail.com : Sale! | Asha, 10% off today.
  [log] SENT       PROMO         user=u1 channel=EMAIL attempts=1
    SMTP -> asha@mail.com : Sale! | Asha, 11% off today.
  [log] SENT       PROMO         user=u1 channel=EMAIL attempts=1
  [log] SUPPRESSED PROMO         user=u1 channel=- attempts=0 reason=rate limit 2 per 60 min
  ...61 minutes later the window has moved:
    SMTP -> asha@mail.com : Sale! | Asha, 50% off today.
  [log] SENT       PROMO         user=u1 channel=EMAIL attempts=1
== 5. Every channel gives up -> FAILED
  [log] FAILED     ORDER_SHIPPED user=u2 channel=- attempts=3 reason=all channels failed, last PUSH: FCM timeout
== 6. No accepted channel for this kind -> SUPPRESSED
  [log] SUPPRESSED PROMO         user=u2 channel=- attempts=0 reason=user accepts no channel
== 7. Template value missing -> FAILED, still recorded through finish()
  [log] FAILED     OTP           user=u1 channel=- attempts=0 reason=template: missing value for {code}
== 8. Async send (caller not blocked)
    FCM -> tok-ben : Order B-3 shipped | Hi Ben, it arrives Monday.
  [log] SENT       ORDER_SHIPPED user=u2 channel=PUSH attempts=1
  future completed with SENT via PUSH
== Metrics: {SENT=7, FAILED=2, SUPPRESSED=3}, history size = 12
```

Step 2 shows Template Method + fallback: 3 failed SMS attempts + 1 successful email = 4 attempts. Step 4 shows that the suppressed third promo did not count: an hour later the window has moved and sending works again.

Concurrency & edge cases

- **Shared state is thread-safe:** users/preferences/templates in `ConcurrentHashMap` (a map built for many threads to read and write at once); listeners and history in `CopyOnWriteArrayList` (a list that copies itself on every write, so reads need no lock: cheap reads, rare writes); the rate-limit deque is locked per key.
- **Check-then-act gap in rate limiting.** (Check-then-act: code first checks a condition, then acts on it, and another thread can change things in between.) Two concurrent (running at the same time) sends for the same user and kind can both pass `blocks()` before either calls `onSent()`, so the limit can be exceeded by the number of concurrent sends. Fixes: reserve a slot atomically at check time and release it if the send fails, or serialize sends per user (run them strictly one after another, e.g. a per-user queue, which also keeps their order).
- **Retries block a worker thread.** With `Thread.sleep` backoff, a slow provider ties up threads. Real systems re-enqueue the message with a delay instead (a "retry topic" — a topic is a named channel that messages are published to — or delayed queue, which hands a message out only after a set delay).
- **Idempotency.** A retry after a timeout (giving up on waiting for a reply after a set time) may deliver twice (the provider may have sent it but the reply was lost). Pass an idempotency key / message id to providers that support it, and make receivers tolerate duplicates.

> **Key term — idempotency:** an operation is **idempotent** when doing it twice has the same effect as doing it once ("set the balance to 10" is; "add 10 to the balance" is not). Retries are only safe on idempotent operations. An **idempotency key** is a unique id sent with a request, so the receiver can recognise a repeat and skip it. More in System Design Fundamentals.

- **Missing address** (user has no phone): the channel returns a failed attempt with 0 tries and the next channel is used.

- **Unknown kind / no template:** FAILED with a reason; **no accepted channel:** SUPPRESSED.
- **Time zones:** quiet hours should use the *user's* time zone; store a `ZoneId` (Java's type for a time zone) in preferences and convert `Instant` (an exact moment on the global timeline) → local hour.

Extensions & follow-up questions

- **"Add WhatsApp."** One `WhatsAppChannel extends NotificationChannel` (3 small methods), one enum value (an enum is a type with a fixed list of named values), one `Gateway` adapter. Nothing else changes.
- **"Someone is getting too many messages."** A new `SendRule` (e.g. a global daily cap, or "dedupe identical messages within 10 minutes"), not a new `if`.
- **"Send to a million users (a campaign)."** Split into a fan-out stage (fan-out: one input turned into many copies or tasks) that expands the audience into individual requests on a queue, and many workers that each run this `send`. Rate-limit per provider too (SMS gateways have throughput limits — **throughput** is how much work gets done per unit of time, e.g. messages per second).
- **"Priorities."** Separate queues per priority (OTP — one-time password, the login-code kind — queue drained first), or a `PriorityBlockingQueue` (a thread-safe queue that always hands out the highest-priority item next).
- **"Localization."** (Localization: adapting text to a user's language and region.) Key templates by (kind, locale — a language-plus-region code such as `fr-CA`) and store the user's locale in preferences.
- **"Delivery receipts / opened tracking."** Providers call back via webhooks (an HTTP request — HTTP being the protocol web clients and servers talk — that the provider sends to a URL you registered, when something happens on their side); record them as further events on the `DeliveryRecord`.
- **"Circuit breaker."** If a provider fails repeatedly, stop calling it for a while (Resilience4j-style; Resilience4j is a Java library of such fault-tolerance tools) and go straight to the next channel. Like an electrical breaker, it "trips" after too many failures and lets calls through again after a pause.

Real-world example

Push notifications on phones go through platform services: **Firebase Cloud Messaging (FCM)** for Android and **Apple Push Notification service (APNs)** for iOS; the app's server sends to a *device token*, exactly the `deviceToken` field above. Email is usually sent through providers such as Amazon SES or SendGrid, and SMS through gateways such as Twilio; each has its own API, which is why the `Gateway` adapter exists. **AWS SNS** is a managed (run for you by the cloud provider) pub-sub that fans one message out to email, SMS, mobile push and queues. LinkedIn's engineering blog describes **Air Traffic Controller (ATC)**, a central service that almost all LinkedIn notifications now pass through and that decides whether, when and how to notify each member, to stop excessive, repetitive notifications: the `SendRule` idea at company scale. And phone "Focus" / "Do Not Disturb" modes are quiet hours with an urgent-bypass list.

In the interview

> **30-second answer:** "Channels sit behind one abstract class whose final `deliver` method holds the retry-with-backoff loop (Template Method); each channel only supplies its address. Each "do not send" condition is its own `SendRule` (Strategy) returning a reason string, so we can record why. The user's preferences give an ordered channel list per kind; we render the template once and try channels in order, falling back on failure. Every outcome goes through one `finish` method that stores a record and notifies listeners (Observer). For scale, a queue sits in front and workers call `send`."

Plan (35–45 min): clarify (5) → entities, User vs Preferences (5) → Template + Message (3) → channel hierarchy + retry (8) → rules (7) → service flow + single exit + listeners (7) → async/queue, idempotency, follow-ups (5).

Follow-ups with answers:

- *"Add a new channel."* → One subclass + gateway adapter.
- *"What about duplicate messages on retry?"* → Idempotency keys; at-least-once delivery (every message arrives, but some may arrive more than once) means receivers must tolerate duplicates.

- *"Why not Strategy for channels?"* → Channels share an algorithm and vary in one step; that is Template Method. (Composition-based alternative: inject a `RetryPolicy` object and a `Sender` Strategy; also valid, mention it.)

Self-check

1. Which pattern puts the retry loop in one place? → Template Method (`final deliver()` in the abstract channel).
2. Why does `SendRule.blocks` return `Optional<String>` (Java's box that holds either one value or nothing)? → So the reason for suppression is recorded.
3. Why should a suppressed notification not count against the rate limit? → The user never received it; counting it would block legitimate later messages.
4. Quiet hours 22–07: is 06:00 quiet? Is 12:00? → Yes; no.
5. Where would you put a queue to avoid blocking checkout? → Between product code and `NotificationService.send` (our `sendAsync` is the in-process version).
6. What must every exit path of `send` do? → Go through `finish()`: store the record and notify listeners.

TL;DR

- **Template Method** for channels: retry/backoff once in the parent; subclasses give the address.
- **Strategy** for send rules (quiet hours, rate limit); rules return the *reason*.
- **Observer** for outcomes; **Simple Factory** for channels; **Adapter** for providers.
- Render once, try channels in the user's order, fall back, single `finish()` exit.
- Scale with a queue in front; watch for idempotency and the rate-limit check-then-act gap.

20.3 Design an In-Memory Cache (and an LRU Cache from scratch)

Problem & clarifying questions

What it is. A cache is a small, fast store that keeps recently used values close at hand, so you don't have to fetch them again from a slow place (a database, another service, the disk). (For where caches sit in bigger systems, see System Design Fundamentals.) Everyday analogy: the few books on your desk. The library (database) has everything, but the desk (cache) holds the ones you are using. When the desk is full and you need a new book, one must go back to the shelf. **Which one goes back** is the eviction policy, and it is the rule most likely to change.

Because the question is about **one component**, vague answers are exposed quickly. The interviewer will ask: *which data structure gives O(1) get and put?* (O(1), "constant time", means the time does not grow with the number of entries.)

Question	Assumed answer
Capacity measured in?	Number of entries (not bytes).
Eviction rule?	LRU (least recently used) by default; must be swappable for FIFO or LFU without changing the cache.
Time limits?	Optional TTL (time to live) per entry. Reading an expired entry is a miss and removes it.
Stats?	Hits, misses, evictions, expirations.
Notifications?	Interested code can subscribe to removals and learn the cause.
Thread-safe?	Yes (the course version left it out; we add it).
Load from DB (database) on miss / write-back (save changes to the DB later)?	Out of scope; discussed in extensions.

Requirements

Functional

1. Holds key → value pairs, never more than a fixed number.
2. `get` returns the value or reports a miss.
3. `put` inserts a new key or overwrites the value of an existing one.
4. Inserting a new key into a full cache first evicts one entry.
5. Default rule: **LRU** (least recently used — the entry whose last access is the oldest).
6. Swapping in another rule (FIFO — first in, first out; LFU — least frequently used) needs no change to the cache class.
7. `get` and `put` are O(1).
8. An entry may carry a TTL; once it has passed, a read returns a miss and deletes the entry.
9. Counts hits, misses, evictions, expirations.
10. Registered listeners hear about every removal together with its cause.
11. Callers can invalidate a single key or empty the whole cache.
12. Every entry leaves through **one** code path, whatever the reason.

Non-functional: O(1) operations; thread-safe; testable TTL (injected clock); listeners must not slow down or deadlock the cache (a **deadlock** is when two threads each wait forever for something the other one holds).

Use cases

Actors: **Application** (reads, stores, removes keys), **Operator** (chooses the eviction rule, reads stats), **Cache** (evicts, expires, counts, notifies).

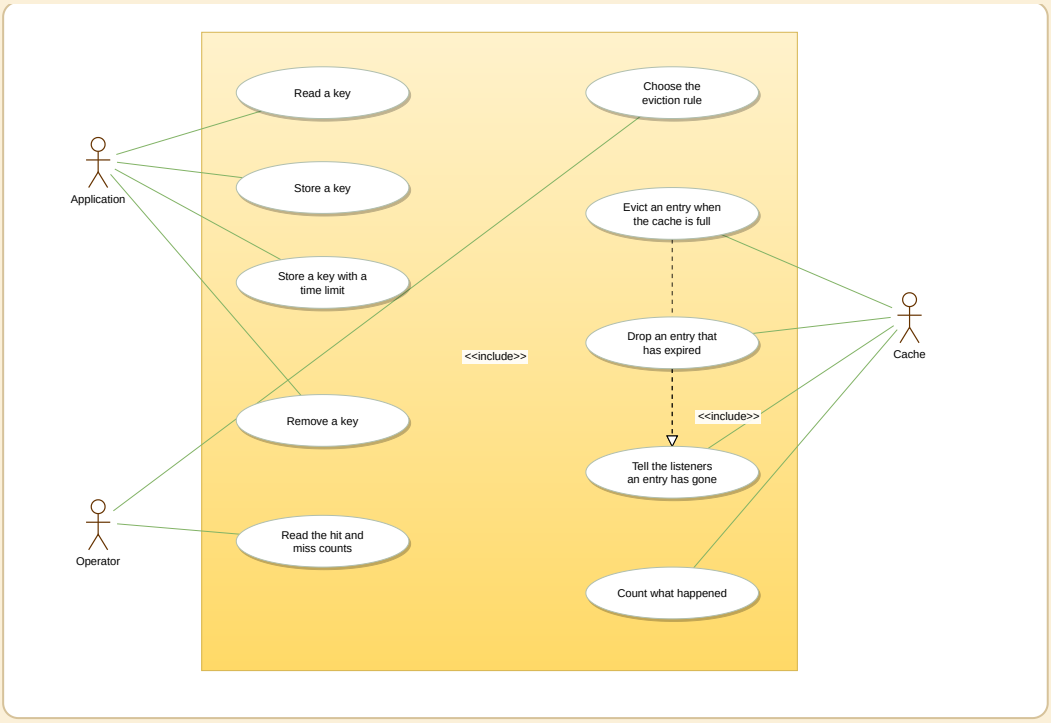


Figure — Cache use cases: read, store (with optional time limit), remove; the cache evicts, drops expired entries and tells listeners (course diagram)

Core entities

Class	Responsibility
`Node<K,V>`	One entry: key, value, `expiresAtMillis`, `frequency`, and its own `prev`/`next` links.
`DoublyLinkedList<K,V>`	Order of entries, newest at the front, with two **sentinel** (dummy) nodes so add/remove never need null checks.
`EvictionPolicy<K,V>` (interface)	**Strategy**: three events (`onInsert`, `onAccess`, `onRemove`) and one question (`victim()`). Owns the *order*.
`LruPolicy`, `FifoPolicy`, `LfuPolicy`	The three rules. LFU is O(1) using a chain of frequency buckets.
`EvictionPolicies` + `PolicyType`	**Simple Factory**: enum → policy.
`Cache<K,V>`	The map + policy + clock + stats + listeners. One lock. **One removal method.**
`Cache.Builder`	**Builder**: capacity, policy, clock with defaults, validated once.
`CacheStats`	Counters (`LongAdder`, a thread-safe counter).
`RemovalReason`	EVICTED, EXPIRED, REMOVED, CLEARED.
`EvictionListener<K,V>`	**Observer**: told key, value and reason.
`ShardedCache<K,V>`	N independent caches picked by key hash, to reduce lock contention.

Key terms — hash map, linked list, hashing, contention. A **hash map** (`HashMap`) stores key → value pairs; it turns each key into a number (its **hash**) that says which slot to look in, so finding a key takes about the same time however many entries there are. A **linked list** is a chain of nodes where each node points to the next one (a **doubly** linked list also points back to the previous one); inserting or unlinking a node you already hold is just a few pointer changes. **Lock**

> **contention** is threads queueing for the same lock; splitting data into **shards** (independent pieces, here chosen by key hash), each with its own lock, reduces it.

The core idea: two structures, each answering one question.

Question	Structure	Cost
"Where is key k?"	`HashMap<K, Node>`	O(1)
"Who should leave next?"	Doubly linked list ordered by recency	O(1) at the ends
"Move this entry to the front"	Node has its own `prev` / `next`, and the map hands us the node	O(1) — no search

A map alone cannot tell you the oldest entry. A list alone cannot find a key without walking it. Together they give O(1) for everything. A **doubly** linked list is needed because removing a node from the middle requires its predecessor, and a singly linked list would have to walk to find it.

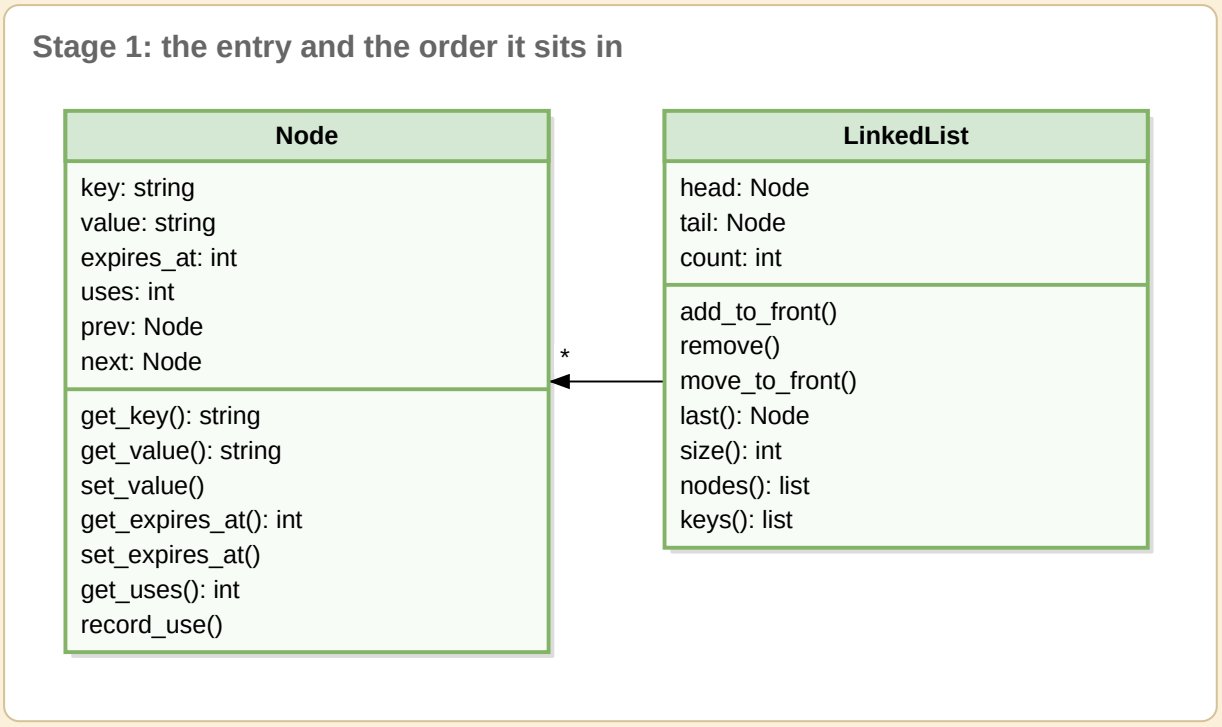


Figure — Stage 1 of the course build: the Node (key, value, expiry, use count, prev and next links) and the linked list that orders the nodes (course diagram)

Class diagram

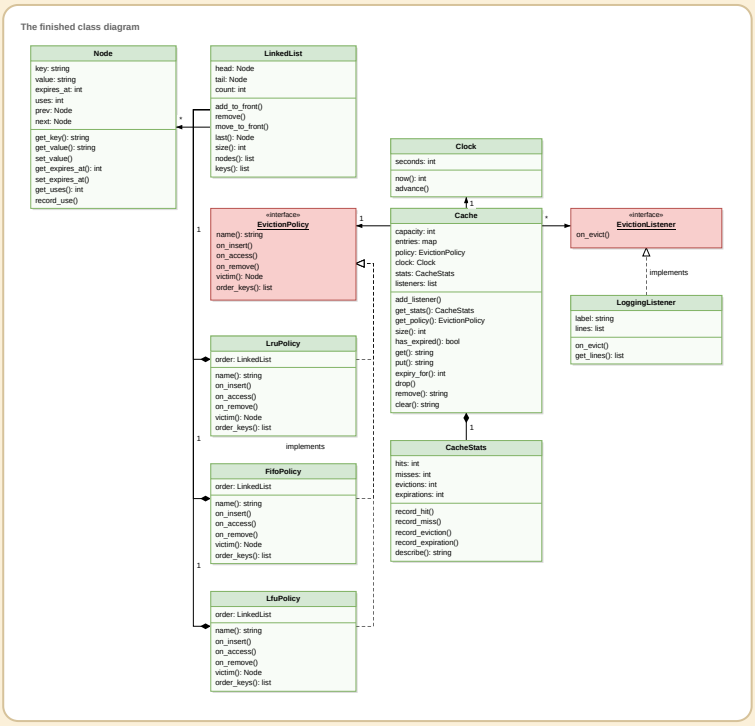
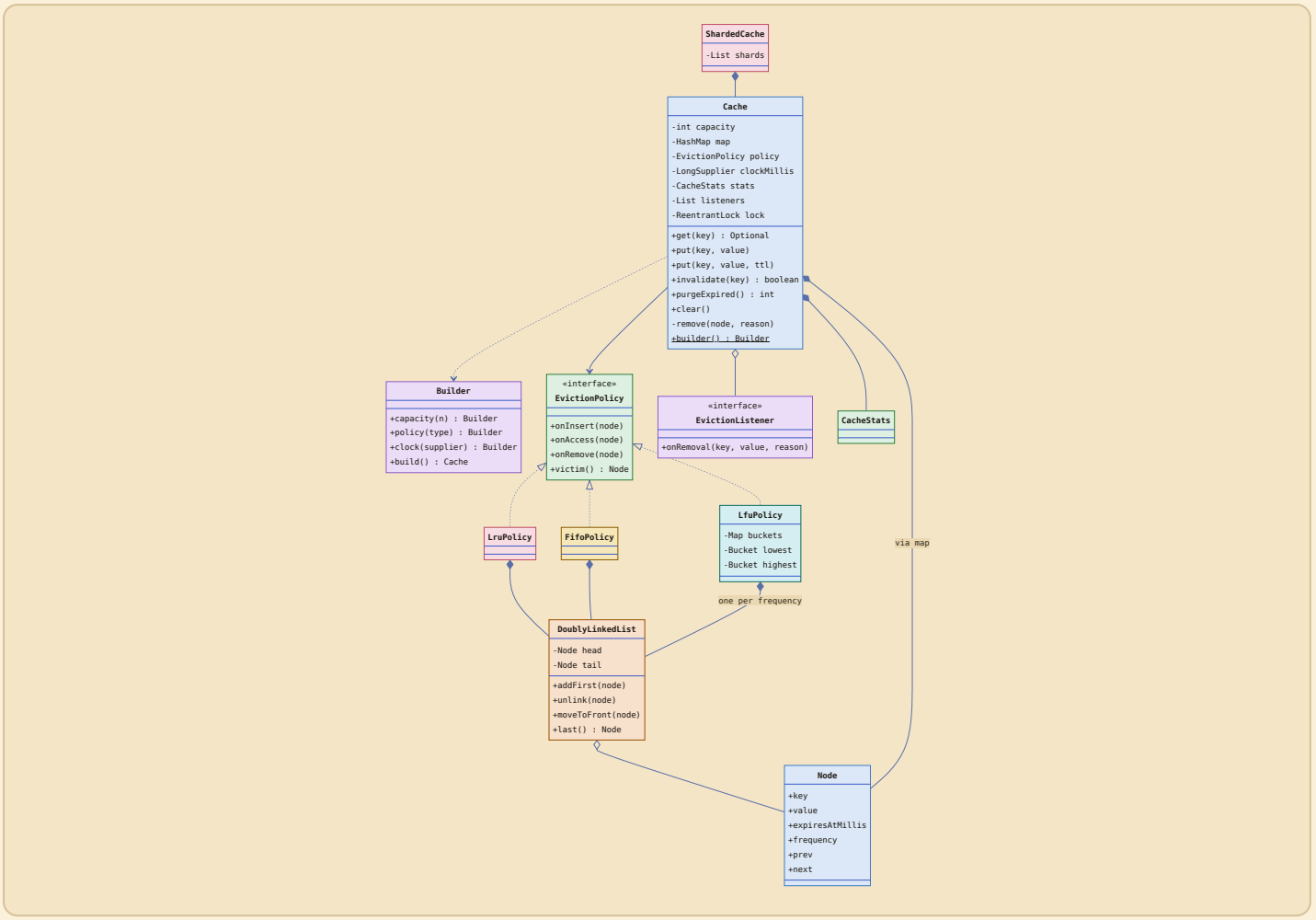


Figure — The finished cache class diagram: Cache with its map, policy, clock, stats and listeners; the three eviction policies behind one interface (course diagram)



Build order (from the course). (1) `Node` + `DoublyLinkedList`: entries can be ordered and moved to the front; (2) `EvictionPolicy` + `LruPolicy`: you can ask who should leave; (3) `FifoPolicy` + `LfuPolicy`: the same question gets three answers with no change to the cache; (4) `Cache` with clock, stats and TTL:

reads and writes work; (5) `EvictionListener`: other code can react when an entry goes. Each stage leaves something runnable; reuse this order when you interview.

Who owns what. The cache owns the map (and so the nodes). The policy owns the *order* of those nodes. Neither reaches into the other's structure. That split is exactly what lets us add FIFO and LFU without touching `Cache`.

Key flows

Reading a key — three ways out, only one returns a value:

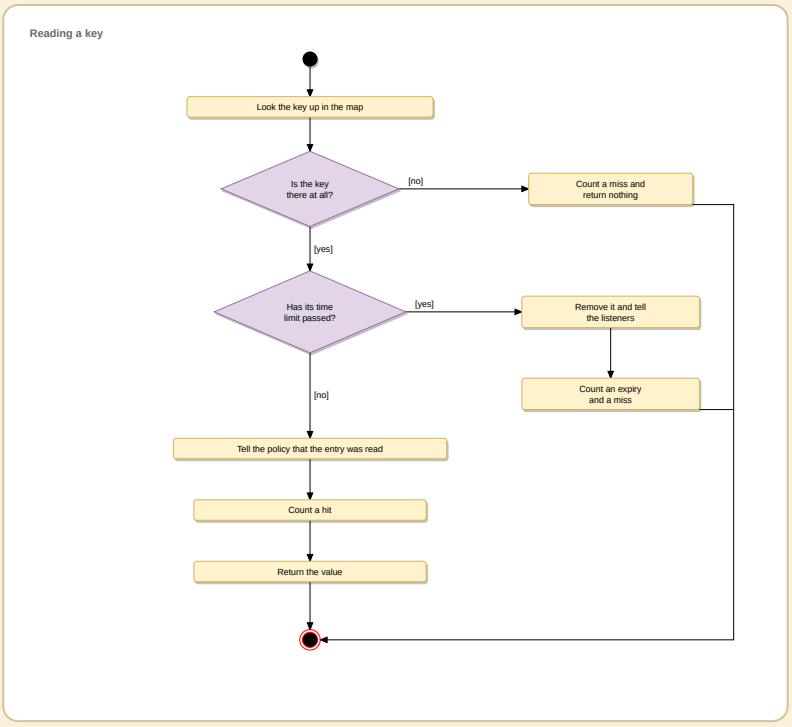


Figure — Reading a key: not present means a miss; present but expired means a miss plus an expiry removal; otherwise a hit, the policy is told, and the value is returned (course diagram)

Storing a key — eviction work happens only when the cache is already full, so a cache with free room pays nothing extra:

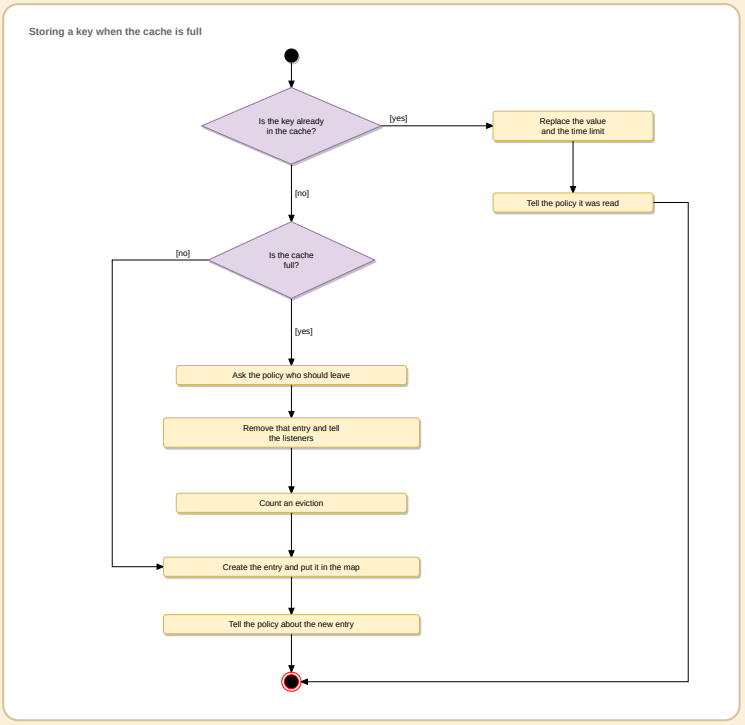
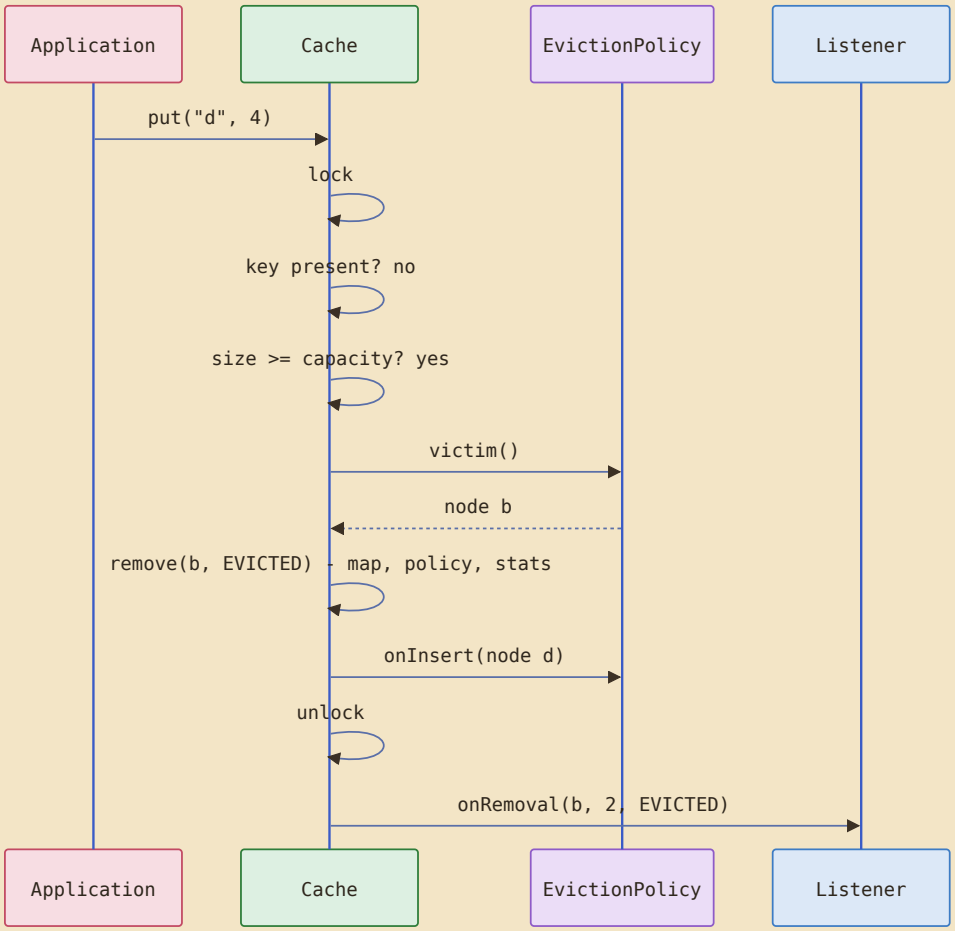


Figure — Storing a key: replace if present; otherwise, if full, ask the policy for a victim and remove it through the single removal path, then insert (course diagram)



How the three policies differ (the only difference is what they do on each event):

Event	LRU	FIFO	LFU (O(1) buckets)
`onInsert`	add to front	add to front	frequency = 1, add to bucket 1 (always first in the chain)
`onAccess`	move to front	**nothing**	move from bucket f to bucket f+1 (created right after f if missing); drop bucket f from the chain if it emptied
`onRemove`	unlink	unlink	unlink; drop the bucket if it emptied
`victim()`	back of the list	back of the list	back of the **first** bucket in the chain (least recently used among the least frequent)

The FIFO policy is LRU with one empty method, which is good evidence that the interface boundaries are well chosen. The course's LFU scanned all entries (O(n): the time grows in step with the number of entries) and named a structure grouping entries by count as the fix; our `LfuPolicy` implements that fix. The buckets are themselves kept in a **doubly linked chain in increasing frequency order**, and a new bucket f+1 is always created right after bucket f, so the chain never needs sorting and the lowest bucket is always the first one. That makes `onInsert`, `onAccess`, `onRemove` and `victim()` all O(1), even after a manual removal or an expiry empties the lowest bucket (this is the O(1) LFU scheme of Shah, Mitra and Matani, 2010). Ties inside the lowest bucket are broken by recency (the back of the bucket), so two runs of the same test always evict the same key.

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for eviction	"Who leaves" is the rule most likely to change.	A `switch (policyType)` inside `get`/`put`.
Observer for removals	Logging, metrics, deleting a temp file on eviction — none belong inside the cache. The reason is passed so one listener can handle all cases.	Callbacks (methods handed over to be called later) hard-coded per reason.
One removal method (`remove(node, reason)`)	Eviction, expiry, manual removal and clear all update map, policy, stats and listeners the same way; nothing can be forgotten on one path.	Four copies of the clean-up code.
Sentinel nodes	Add and unlink are always the same four pointer writes (a **pointer** here is a reference from one node to another); no special cases for empty list or ends.	Null checks everywhere (a classic source of bugs).
Injected clock (`LongSupplier`, a function that returns a `long`)	TTL tests run instantly by moving a fake clock forward.	`System.currentTimeMillis()` inside the cache: tests must `sleep`.
Lazy expiry + optional sweeper	Lazy = checked only when needed: expired entries are dropped when read; `purgeExpired()` lets a background task reclaim memory.	Only lazy (memory held by dead entries) or a timer per entry (expensive).
One `ReentrantLock` (Java's explicit lock object), listeners called **after unlock**	In LRU even `get` writes (moves a node), so a read-write lock (one that lets many readers in together but a writer only alone) would not help. Calling listeners outside the lock means a slow or re-entrant listener cannot block or deadlock the cache.	`synchronized` around listener calls.
Builder	Optional settings with defaults; validation once in `build()`.	Telescoping constructors (many constructors, each taking one more parameter).
Sharding (splitting data into independent pieces; `ShardedCache`)	N locks instead of one: threads touching different shards never wait for each other.	One global lock (every reader waits for every writer).

Java implementation

Files: `Structures.java` (`Node`, sentinel `DoublyLinkedList`), `EvictionPolicy.java` (LRU, FIFO, O(1) LFU, factory), `Cache.java` (removal reasons, listener, stats, cache + builder, `ShardedCache`), `Main.java`. The full runnable source is in `code/lld/ch20/cache/`; the doubly linked list itself is shown in the LRU-from-scratch listing below.

```java
// File: code/lld/ch20/cache/EvictionPolicy.java
/** Strategy: owns the ORDER of entries and answers "who leaves?". The cache owns the map. */
interface EvictionPolicy<K, V> {
    void onInsert(Node<K, V> node);
    void onAccess(Node<K, V> node);
    void onRemove(Node<K, V> node);
    Node<K, V> victim();
}

/** Least recently used: a read moves the entry to the front; the back is the victim. */
final class LruPolicy<K, V> implements EvictionPolicy<K, V> {
    private final DoublyLinkedList<K, V> order = new DoublyLinkedList<>();
    public void onInsert(Node<K, V> n) { order.addFirst(n); }
    public void onAccess(Node<K, V> n) { order.moveToFront(n); }
    public void onRemove(Node<K, V> n) { order.unlink(n); }
    public Node<K, V> victim() { return order.last(); }
}
// FifoPolicy: identical except onAccess does nothing.
// LfuPolicy (O(1)): one list per frequency ("bucket"), buckets chained in increasing frequency;
// the victim is the back (least recent) of the lowest bucket.

// File: code/lld/ch20/cache/Cache.java
Optional<V> get(K key) {
    List<Removal<K, V>> removed = new ArrayList<>(1);
    lock.lock();                                    // one ReentrantLock: even an LRU get writes
    try {
        Node<K, V> n = map.get(key);
        if (n == null) {
            stats.misses.increment();
            return Optional.empty();
        }
        if (n.isExpired(clockMillis.getAsLong())) { // a miss, an expiry and a removal at once
            stats.misses.increment();
            removed.add(remove(n, RemovalReason.EXPIRED));
            return Optional.empty();
        }
        stats.hits.increment();
        policy.onAccess(n);
        return Optional.of(n.value);
    } finally {
        lock.unlock();
        notifyListeners(removed); // outside the lock: a slow listener cannot block the cache
    }
}

// put(key, value, ttl), inside the lock:
if (map.size() >= capacity) removed.add(remove(policy.victim(), RemovalReason.EVICTED));

/** THE single exit path for every entry, whatever the reason. Caller holds the lock. */
private Removal<K, V> remove(Node<K, V> n, RemovalReason reason) {
    map.remove(n.key);
    policy.onRemove(n);
    if (reason == RemovalReason.EVICTED) stats.evictions.increment();
    if (reason == RemovalReason.EXPIRED) stats.expirations.increment();
    return new Removal<>(n.key, n.value, reason);
}
// Builder: capacity (default 100), policy (default LRU), clock; build() validates capacity > 0.
// ShardedCache: N independent caches chosen by key hash.
```

Walking through the code.

- `DoublyLinkedList` never touches `null` at the ends because `head` and `tail` are dummy nodes that are never removed.
- In `Cache.get`, an expired entry counts as a miss **and** an expiration, and is removed through `remove()` — all three in one place, so the counters always add up.
- Every public method collects `Removal` events while holding the lock and calls `notifyListeners` in `finally` (a block that always runs, even when an exception is thrown) **after** `unlock()`.

- `put` on an existing key replaces value and TTL and counts as an access (in LRU the key moves to the front).
- `CacheStats` uses `LongAdder`, which is faster than `AtomicLong` (a `long` that threads can update safely without a lock) when many threads increment.

Output:

```
== LRU (capacity 3): put a,b,c; get b,b,b,c,a,a; put d
  listener: b=2 left, EVICTED
  still cached: [a, c, d]
== FIFO (capacity 3): put a,b,c; get b,b,b,c,a,a; put d
  listener: a=1 left, EVICTED
  still cached: [b, c, d]
== LFU (capacity 3): put a,b,c; get b,b,b,c,a,a; put d
  listener: c=3 left, EVICTED
  still cached: [a, b, d]
== TTL with a manual clock
  t=0s  get session:42 -> alice
  listener: session:42 left, EXPIRED
  t=6s  get session:42 -> MISS
  listener: session:43 left, EXPIRED
  sweeper purged 1 more expired entry
  config (no TTL) -> v1
  listener: config left, REMOVED
  stats: hits=2 misses=1 evictions=0 expirations=2 hitRate=67%
== Sharded cache under 8 threads
  finished: true, size <= 8000: true
```

With the same access pattern (b read 3 times, then c once, then a twice), the three policies evict three **different** keys: LRU evicts b (least recently touched), FIFO evicts a (inserted first), LFU evicts c (read only once).

LRU Cache from scratch (the classic interview question)

Very often the interviewer skips the pluggable design and asks for exactly this: **"Implement an LRU cache with O(1) `get` and `put`"** (LeetCode 146; LeetCode is a popular website of coding interview problems). The answer is the same two structures, written inline:

1. A `HashMap<K, Node>` for lookup.
2. A doubly linked list with `head`/`tail` sentinels; most recent right after `head`, least recent right before `tail`.
3. `get(k)`: look up the node; if found, unlink it and add it at the front; return the value.
4. `put(k, v)`: if present, update and move to front. Otherwise, if full, remove `tail.prev` from the list **and** from the map (that is why the node stores its key); then add the new node at the front and to the map.

Java also has a shortcut: `LinkedHashMap` with `accessOrder = true` keeps entries in access order, and overriding `removeEldestEntry` evicts automatically. Mention it, but write the manual version when asked "from scratch" — that is what is being tested.

```java
// File: code/lld/ch20/lrucache/Main.java (toString and the main() demo that prints the output below are omitted)
/** The classic interview answer: HashMap + hand-written doubly linked list. get/put are O(1). */
static final class LRUCache<K, V> {
    private final class Node {
        K key;
        V value;
        Node prev, next;
        Node(K key, V value) { this.key = key; this.value = value; }
    }

    private final int capacity;
    private final Map<K, Node> index = new HashMap<>();
    private final Node head = new Node(null, null); // sentinel: most recent side
    private final Node tail = new Node(null, null); // sentinel: least recent side

    LRUCache(int capacity) {
        if (capacity <= 0) throw new IllegalArgumentException("capacity must be positive");
        this.capacity = capacity;
        head.next = tail;
        tail.prev = head;
    }

    synchronized V get(K key) {
        Node n = index.get(key);
        if (n == null) return null;
        unlink(n);
        addFront(n);                // now the most recently used
        return n.value;
    }

    synchronized void put(K key, V value) {
        Node n = index.get(key);
        if (n != null) {            // update + mark as recently used
            n.value = value;
            unlink(n);
            addFront(n);
            return;
        }
        if (index.size() == capacity) {
            Node lru = tail.prev;   // least recently used lives just before the tail sentinel
            unlink(lru);
            index.remove(lru.key);
        }
        n = new Node(key, value);
        addFront(n);
        index.put(key, n);
    }

    private void addFront(Node n) {
        n.prev = head;
        n.next = head.next;
        head.next.prev = n;
        head.next = n;
    }

    private void unlink(Node n) {
        n.prev.next = n.next;
        n.next.prev = n.prev;
    }
}

/** The JDK shortcut: LinkedHashMap in access order + removeEldestEntry. Not thread-safe. */
static <K, V> Map<K, V> jdkLru(int capacity) {
    return new LinkedHashMap<>(16, 0.75f, true) {
        @Override
        protected boolean removeEldestEntry(Map.Entry<K, V> eldest) { return size() > capacity; }
    };
}
```

Output:

```
== Hand-written LRU (LeetCode 146 sequence, capacity 2)
get(1) = 1   order [1, 2] (most -> least recent)
put(3) evicts 2        order [3, 1] (most -> least recent)
get(2) = null
put(4) evicts 1        order [4, 3] (most -> least recent)
get(1) = null, get(3) = 3, get(4) = 4
== Same sequence with LinkedHashMap(accessOrder=true)
keys (least -> most recent) = [3, 4]
```

The printed values match LeetCode 146's expected answers for this sequence (1, miss, miss, 3, 4); LeetCode's `int` version returns `-1` for a miss, while this generic version returns `null`.

Approach	get / put	Thread-safe	When to use
HashMap + own doubly linked list	O(1) / O(1)	Only with a lock (`synchronized` here)	Interviews; when you need custom behaviour
`LinkedHashMap(accessOrder=true)` + `removeEldestEntry`	O(1) / O(1)	No (wrap with a lock)	Quick, small, single-threaded caches
Caffeine / Guava cache (widely used Java caching libraries)	amortized O(1) (O(1) on average over many calls)	Yes, highly concurrent	Production Java

Concurrency & edge cases

- **Why a plain read lock is not enough:** an LRU `get` moves the node (several pointer writes). Two threads doing that at once corrupt the list. So `get` needs the exclusive lock.
- **Contention:** one lock serializes everything. `ShardedCache` splits keys over 8 caches (8 locks); the demo hammers it from 8 threads and the size never exceeds the total capacity. Trade-off: eviction is per shard, so the global LRU order is approximate.
- **Listeners:** called outside the lock; a listener that calls back into the cache cannot deadlock.
- **Cache stampede** (many threads miss the same key at once and all hit the database): use a "loading" cache where the first thread computes and the others wait for its result (`ConcurrentHashMap.compute`/`computeIfAbsent` — `computeIfAbsent` creates and stores a value only if the key is missing, as one atomic step — or a `CompletableFuture` stored per key). Chapter 13 (§13.6) builds a TTL cache whose `getOrLoad` does this with `ConcurrentHashMap.compute`.
- **TTL semantics, precisely.** An entry expires *at* its deadline (`now >= expiresAt`), so a 5-second TTL written at t=0 is already a miss at t=5.000 s. Expiry is **lazy**: a dead entry still occupies a slot until it is read, swept by `purgeExpired()`, or picked as the eviction victim (in which case it is reported as `EVICTED`, not `EXPIRED`, because it left to make room). `put(key, value)` without a TTL on an existing key **clears** the old TTL (the entry becomes permanent); pass the TTL again to keep one. A production cache would also let the sweeper run on a `ScheduledExecutorService` (Java's tool for running tasks on a timer).
- **Capacity 0 or negative:** refused by the builder.
- **Null keys/values:** not supported here; say so (Caffeine forbids nulls too).
- **Clock going backwards:** use a monotonic clock (one that only ever moves forward, unlike wall-clock time that can be adjusted; `System.nanoTime`) for TTL in production.

Extensions & follow-up questions

- **"Make it LFU."** One new policy class (done). Volunteer that the naive LFU (scan for the lowest count) is O(n) and the chained-bucket version is O(1).
- **"Size in bytes, not entries."** Add a `Weigher<K,V>` strategy; evict until `totalWeight <= maxWeight`.
- **"Load from the database on a miss."** *Read-through*: `get(key, loader)` calls the loader on a miss and stores the result. *Write-through*: writes go to cache and DB together. *Write-back (write-behind)*: writes go to cache and are flushed to DB later (faster, risk of loss). *Cache-aside*: the application does "check cache → on miss read DB → put in cache" itself.

- **"Distributed cache."** Consistent hashing (a way to assign keys to servers so that adding or removing a server moves only a small share of the keys) to spread keys over nodes (Redis Cluster, Memcached clients — Redis and Memcached are in-memory key-value stores that run as separate servers); that is high-level design (HLD: designing services, databases and networks, as opposed to the classes inside one program).
- **"Which policy is best?"** Depends on access patterns. Modern caches (Caffeine's *Window TinyLFU*) combine recency and frequency and beat pure LRU on many workloads (patterns of real requests).

Real-world example

`java.util.LinkedHashMap` with access order is the JDK's (Java Development Kit: Java's compiler plus standard library) built-in LRU. **Guava's** `CacheBuilder` and **Caffeine** expose exactly this chapter's knobs: `maximumSize` (capacity), `expireAfterWrite`/`expireAfterAccess` (TTL), `removalListener` (Observer, with a cause such as `EXPIRED`, `SIZE`, `EXPLICIT`, `REPLACED`), and `recordStats()` (hit rate: the share of reads answered from the cache). **Redis** supports eviction policies selected by configuration (`maxmemory-policy`: `allkeys-lru`, `allkeys-lfu`, `volatile-ttl`, `noeviction` and others) — a Strategy chosen by config — and approximates LRU by sampling a few keys rather than keeping a full list, to save memory. Browsers, CDNs (content delivery networks: servers spread around the world that keep copies of files close to users) and operating-system page caches (recently read disk data kept in memory) all use variants of LRU.

In the interview

30-second answer: "A HashMap from key to node for O(1) lookup, and a doubly linked list with sentinels for O(1) ordering; the node carries its own links so the map hands us something we can unlink directly. The eviction rule goes behind an `EvictionPolicy` interface — LRU moves on access, FIFO doesn't, LFU uses frequency buckets for O(1). Every exit goes through one `remove` method that updates stats and notifies listeners after the lock is released. For concurrency, one lock is correct; sharding by key hash reduces contention."

Plan: sketch the map and the list together straight away and explain what each one lacks on its own → write LRU get/put → pull the rule behind an interface unprompted → TTL + stats + listeners → thread safety and sharding → follow-ups.

Follow-ups with answers:

- *"Why doubly and not singly linked?"* → Removing a node needs its predecessor; with `prev` we have it in O(1).
- *"Why does the node store the key?"* → When evicting the tail we must also delete it from the map.
- *"Make it thread-safe."* → Lock around each operation; better, shard by key; production: Caffeine.

Self-check

1. Which two structures give O(1) LRU? → HashMap (key → node) + doubly linked list (recency order).
2. What do sentinel nodes buy you? → No null checks; add/remove are always the same pointer writes.
3. How does FIFO differ from LRU in code? → `onAccess` does nothing.
4. Why call listeners after releasing the lock? → A slow or re-entrant listener cannot block or deadlock the cache.
5. Why can't a read-write lock speed up an LRU cache's `get`? → `get` modifies the list, so it is a write.
6. What is a cache stampede and one fix? → Many concurrent misses for one key hitting the DB; let one thread load while others wait on the same future.

TL;DR

- **HashMap + doubly linked list (with sentinels)** = O(1) get/put.
- **Strategy** for eviction (LRU / FIFO / O(1) LFU); **Observer** for removals; **Builder** for config.
- **One removal path** keeps stats and listeners correct; TTL is lazy + optional sweeper; clock injected.

- Thread safety: one lock (gets are writes), listeners outside the lock, **shard** to scale.
- Know `LinkedHashMap(accessOrder=true)` + `removeEldestEntry`, and Caffeine/Guava/Redis policies.

20.4 Design a Ride Sharing Service (Uber)

Problem & clarifying questions

What it is. A ride sharing service connects a rider who wants a car with a nearby driver. Everyday analogy: a taxi dispatcher with a radio. A call comes in, the dispatcher looks at the map, radios the nearest free cab; if that driver says no, the next one. At the end the meter shows the fare.

Two decisions are the whole question: **which driver is asked** (matching) and **what the trip costs** (pricing). The rest is bookkeeping. A good design lets either rule change with no edits anywhere else.

Question	Assumed answer
Vehicle types?	Economy, premium, van. Rider asks for one type.
Who can get a request?	Drivers who are online, not on a trip, with the right car type, within range.
Offer to one driver or broadcast?	One at a time, nearest first; decline → next; nobody left → cancel.
Trip states?	Requested → accepted → started → ended; canceled from requested/accepted only. Illegal actions do nothing.
Fare?	Base + per km + per minute, never below a minimum; per vehicle type.
Surge?	A multiplier when the city is busy, applied to the same fare.
Ratings?	Rider and driver rate each other once after a finished trip; accounts show the average.
Payments?	Charge the rider's payment method (cash, wallet, card) at the end.
Map?	A grid; distance measured along streets (Manhattan distance), which keeps all distances integral (whole numbers). Production systems query a routing engine (a service that works out the road route between two points) that gives back road distance plus an estimated duration, and pricing would take both from it.
Pool/shared rides, scheduled rides, live tracking, payouts?	Out of scope.

Requirements

Functional

1. Riders and drivers have a name and phone number.
2. A driver has one vehicle: plate, seats, type.
3. Drivers go online/offline; only online, free drivers get requests.
4. A rider requests a trip with pickup, drop and car type.
5. Free drivers with a matching car type are ordered by distance to the pickup, and the closest one is offered the trip first.
6. A driver accepts or declines; a decline goes to the next driver; if none are left, the request is canceled.
7. A trip moves through fixed states; an action that doesn't fit the current state does nothing.
8. Fare = base + per-km + per-minute, never below a minimum.
9. During peak demand, a surge multiplier scales that same fare.
10. Riders may cancel until the trip starts; after that, the only way out is ending it.
11. After a finished trip, rider and driver rate each other; accounts show the average.
12. Riders and drivers can see their finished trips.

13. (Added) The rider is charged through their payment method.

Non-functional: a driver can never be assigned to two trips at once (even with concurrent requests); rules swappable without editing the service; deterministic behaviour for tests (tie-breaks).

Use cases

Actors: **Rider** (request, cancel, rate), **Driver** (go online, accept/decline, start/end, rate), **System** (find nearby drivers, offer, price, surge, history).

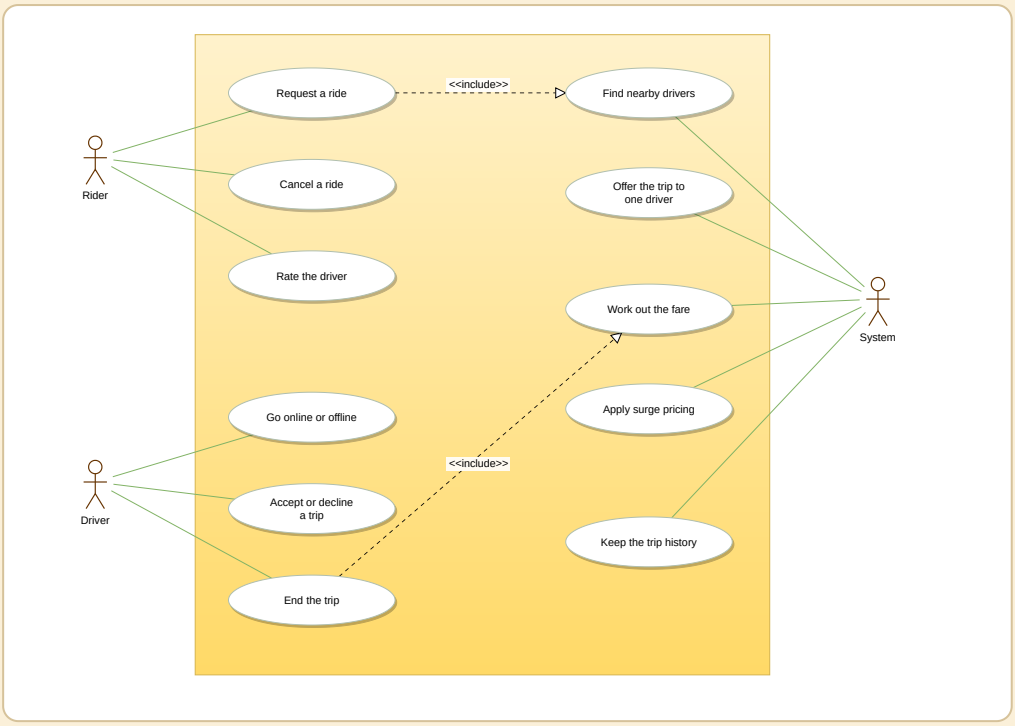


Figure — Ride sharing use cases: rider requests, cancels and rates; driver goes online, accepts or declines and ends the trip; the system matches, offers, prices and applies surge (course diagram)

Core entities

Class	Responsibility
`Location` (record)	Grid point; `distanceTo` = Manhattan distance in km.
`VehicleType` (enum), `Vehicle` (record)	Car type and details.
`Account` (abstract)	What riders and drivers share: id, name, phone, **rating as running total + count**, finished trips.
`Rider`	+ payment method.
`Driver`	+ vehicle, location, online flag, **atomic busy flag** (`tryClaim`).
`TripStatus` (enum)	The state machine: legal next states per state.
`Trip`	One journey: rider, pickup, drop, type, **ranked candidate list + cursor** (a marker for the current position in the list), status, driver, minutes, fare. Every action checks state first.
`PricingStrategy`	**Strategy**: fare for (type, km, minutes).
`StandardPricing` + `RateCard`	Base + per km + per minute, with a floor (a minimum fare), per vehicle type.
`SurgePricing`	**Decorator**: wraps any `PricingStrategy` and multiplies its answer.
`MatchingStrategy`	**Strategy**: ranks drivers. `NearestDriverMatcher`, `BestRatedMatcher`.
`PaymentMethod`	**Strategy**: `CashPayment`, `WalletPayment`.
`TripListener`	**Observer**: apps are told about trip events.
`RideSharingService`	Facade: owns accounts and trips; holds one pricing and one matching rule; knows only the order of steps.

Why the rating is a total and a count. Only the average is ever displayed, so a sum and a count suffice, and recording a new rating is O(1) whether the account has 10 trips or 10,000.

Why `Rider` and `Driver` extend `Account`. They pass the "is-a" substitution test (Liskov Substitution Principle): wherever the code reads a name or records a rating, a `Rider` or a `Driver` can be passed.

Class diagram

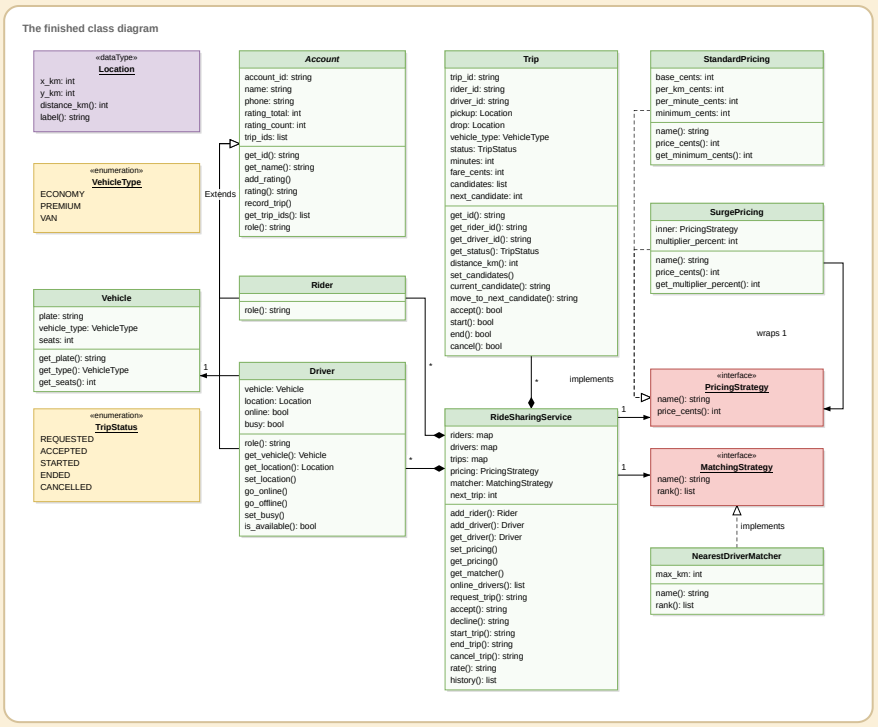
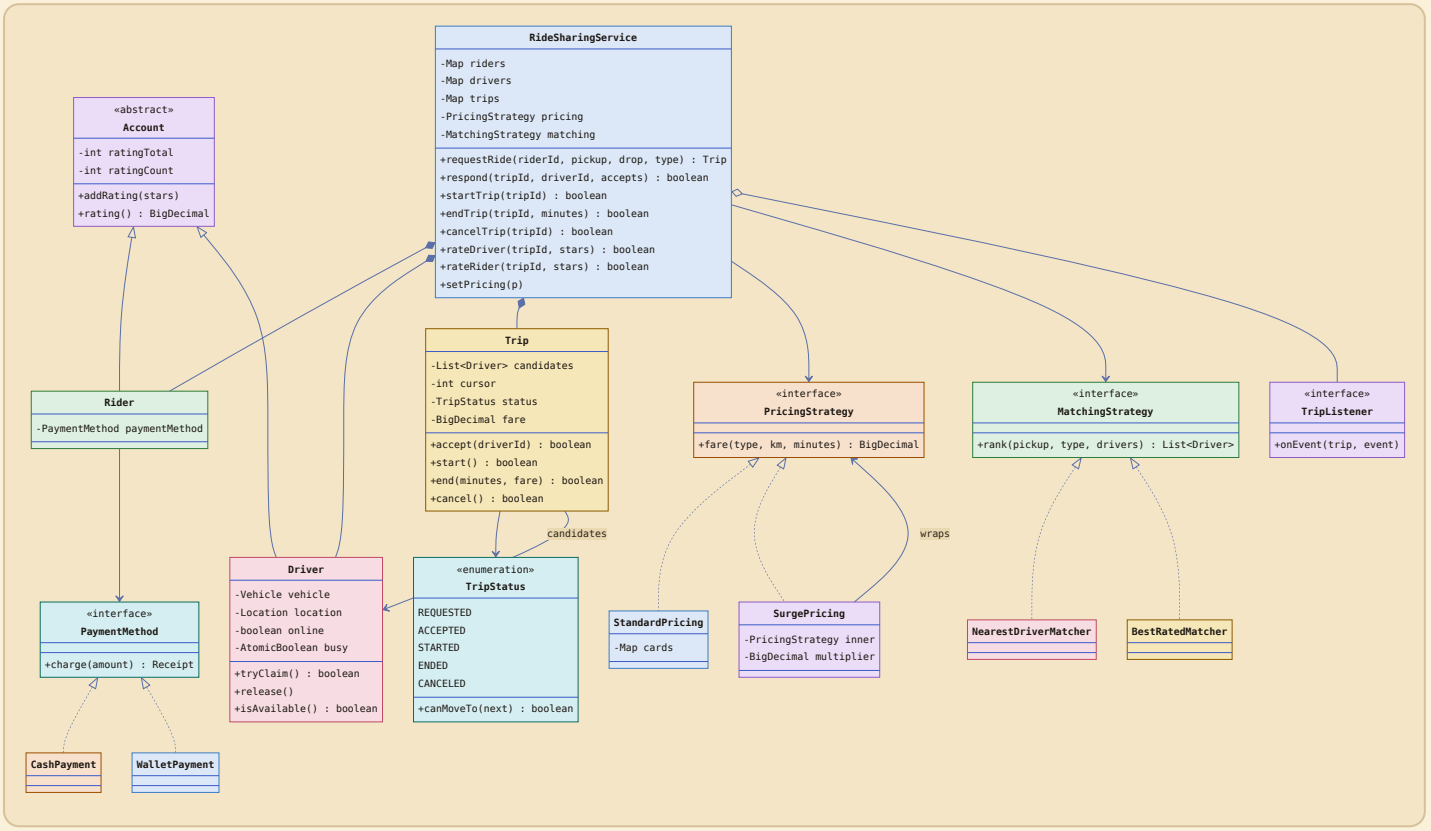


Figure — The finished ride sharing class diagram: the service owning riders, drivers and trips; PricingStrategy with StandardPricing and the SurgePricing wrapper; MatchingStrategy with the nearest-driver matcher (course diagram)

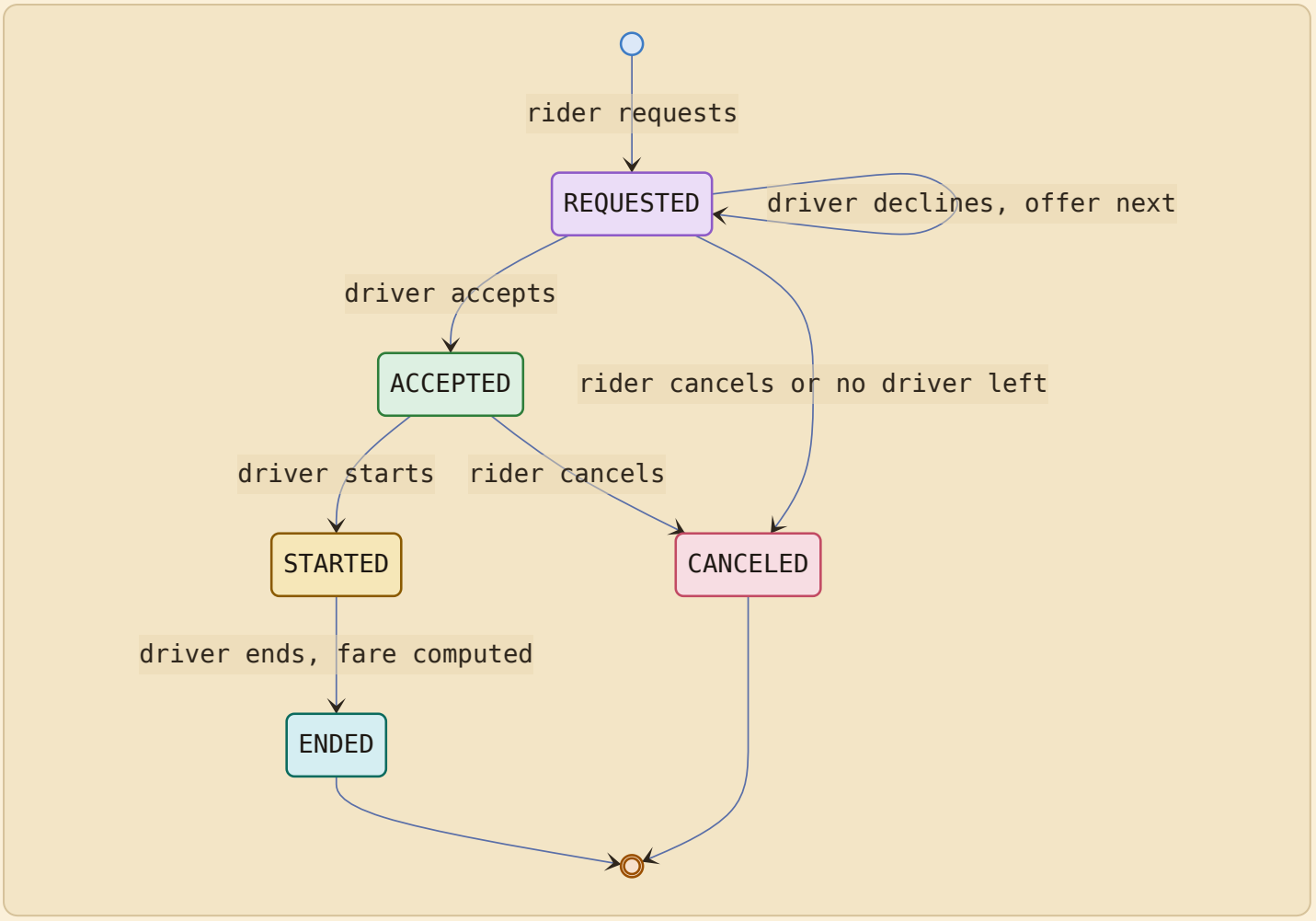
The course builds it in five stages, each leaving something runnable: (1) `Account` / `Rider` / `Driver`, `Vehicle`, `Location` (who is free and where); (2) `Trip` + `TripStatus` (the lifecycle, with illegal moves refused); (3) pricing with the surge wrapper (a finished trip has a fare); (4) matching (drivers who could take a request can be ranked); (5) `RideSharingService` (the complete flow, request through rating).

In the figure above, look for the extra arrow from `SurgePricing` back to the `PricingStrategy` interface: that "has-a of my own type" arrow is what turns it into a Decorator instead of one more Strategy.



Key flows

Trip lifecycle (state machine). A *state machine* is a fixed set of states plus the only moves allowed between them.



Finding a driver — including the case where every candidate declines:

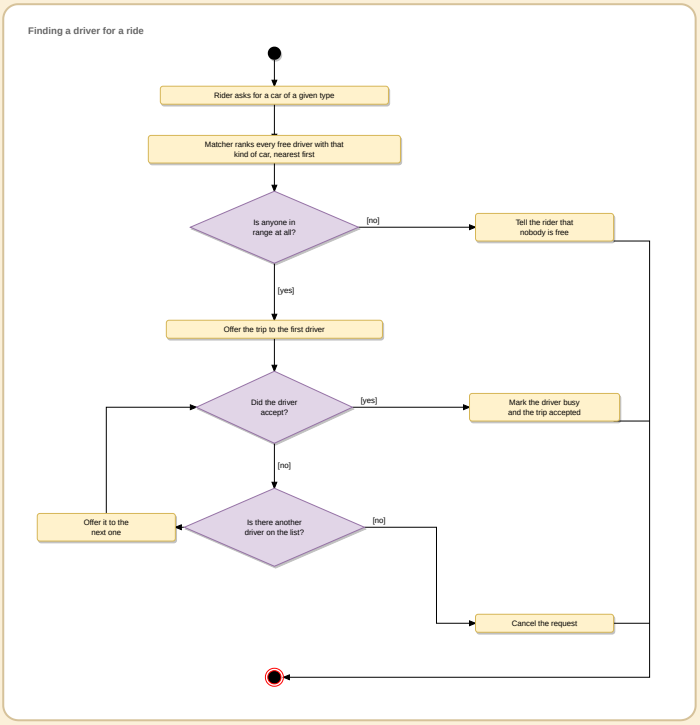


Figure — Finding a driver: rank free drivers of the right type nearest first; offer to the first; on decline move to the next; cancel when nobody is left (course diagram)

Ending and charging a trip — pricing is a single call no matter how many decorators wrap the rule:

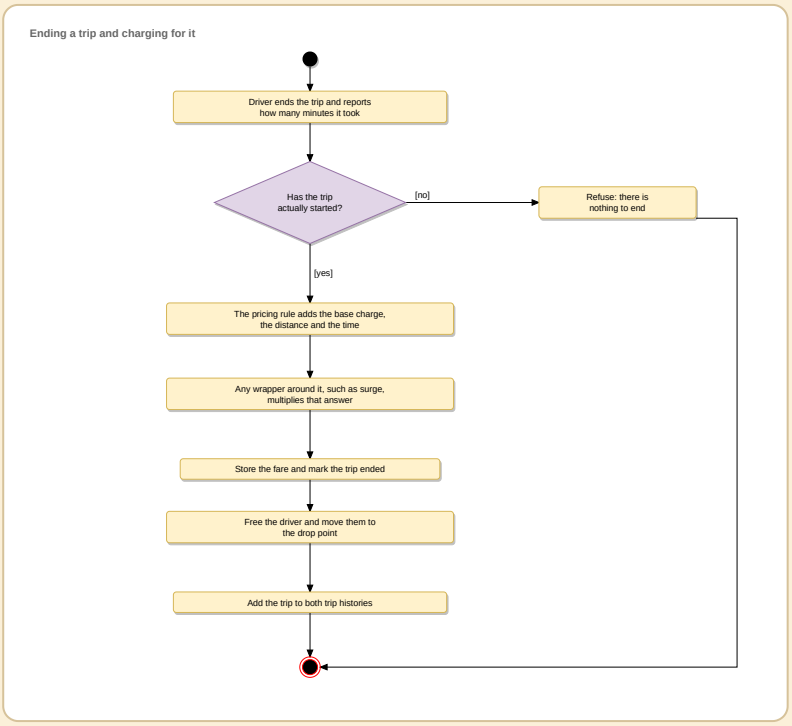
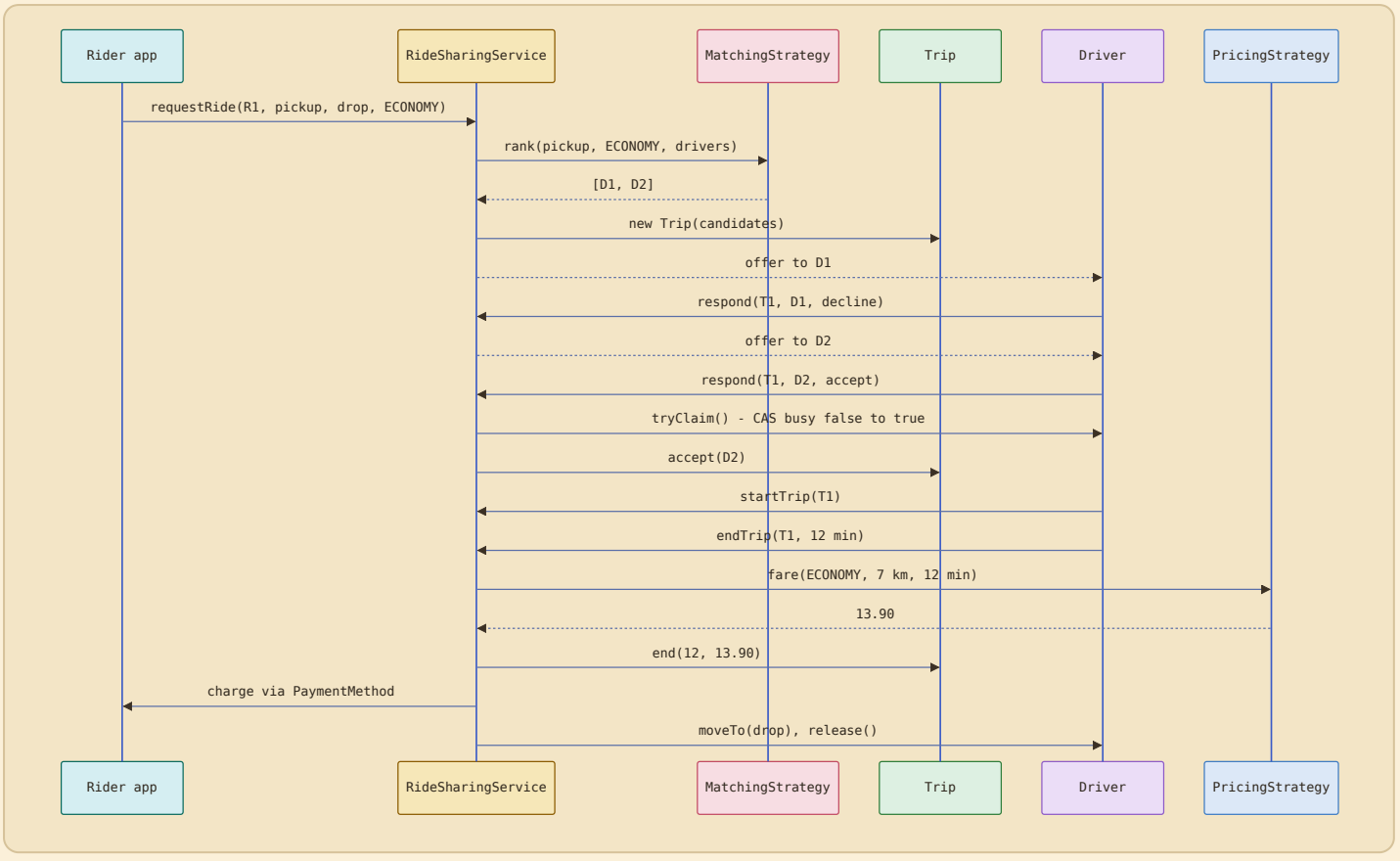


Figure — Ending a trip: refuse if not started; otherwise the pricing rule computes base plus distance plus time, any surge wrapper multiplies it, and the trip is recorded (course diagram)



Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for matching	Cities rank differently (nearest, best-rated, least-earned-today). One class each.	Sorting logic inside the service.
Matcher returns a **ranked list**, stored on the trip	A decline just moves the cursor; no new city-wide search, and the system remembers who refused.	Returning one driver: every decline re-searches from scratch.
Strategy for pricing, **Decorator** for surge	Surge wraps *any* pricing rule (even future ones). Turning it off = removing the wrapper, not editing an `if`. Decorators stack (surge + airport fee + discount).	A `surgeMultiplier` field and `if (surge)` in the pricing code.
State machine via `TripStatus.canMoveTo`	One table says what is legal; every method checks it and returns `false` on an illegal move. No exceptions for normal refusals.	Scattered `if (status == …)` checks; or the full State pattern (one class per state). The State pattern is worth it when behaviour per state is large; here a transition table is enough.
Atomic claim of a driver (`AtomicBoolean.compareAndSet`: "set to true only if it is still false", in one indivisible step)	Two trips racing for the same driver: exactly one wins, the other treats it as a decline.	Checking `isAvailable()` then setting busy: a check-then-act race (a race condition: the result depends on which thread happens to run first).
Driver **moved to the drop point** at trip end	Otherwise the next match uses where they were this morning.	Forgetting (a real bug class).
Observer for trip events	Rider/driver apps, push notifications, analytics (code that collects usage data for reports) listen; the service doesn't know them.	Service calling the notification code directly.
Deterministic tie-breaks (by driver id)	Same input → same ranking → reproducible tests.	Unordered ties: flaky tests (tests that sometimes pass and sometimes fail with no code change).
Money in `BigDecimal`, rounded `HALF_UP` (halves round up: 2.345 → 2.35) to 2 places	Exact fares. (The course keeps money as whole cents and the surge multiplier as a whole-number percentage, 150 = ×1.5, so no floating point (binary fractional numbers such as `double`) appears; `BigDecimal` is this book's equivalent.)	`double`.

Java implementation

Files: `Model.java` (location, vehicle, rider, driver with `tryClaim`), `Trip.java` (status table, trip), `Strategies.java` (pricing + surge decorator, matching, payment), `RideSharingService.java`, `Main.java`. The full runnable source is in `code/lld/ch20/ridesharing/`; below are the lines that carry the design.

```java
// File: code/lld/ch20/ridesharing/Trip.java
/** The trip lifecycle as a table of legal moves (a compact State machine). */
enum TripStatus {
    REQUESTED, ACCEPTED, STARTED, ENDED, CANCELED;

    private static final Map<TripStatus, Set<TripStatus>> NEXT = Map.of(
            REQUESTED, EnumSet.of(ACCEPTED, CANCELED),
            ACCEPTED, EnumSet.of(STARTED, CANCELED),
            STARTED, EnumSet.of(ENDED),
            ENDED, EnumSet.noneOf(TripStatus.class),
            CANCELED, EnumSet.noneOf(TripStatus.class));

    boolean canMoveTo(TripStatus next) { return NEXT.get(this).contains(next); }
}
// Trip: candidates ranked once + a cursor; every action is synchronized and calls move(next),
// which returns false (and changes nothing) for an illegal move.

// File: code/lld/ch20/ridesharing/Model.java
boolean tryClaim() { return online && busy.compareAndSet(false, true); }   // Driver
void release() { busy.set(false); }

// File: code/lld/ch20/ridesharing/Strategies.java
/** Decorator: wraps ANY pricing rule and multiplies its answer. Remove the wrapper to switch surge off. */
final class SurgePricing implements PricingStrategy {
    private final PricingStrategy inner;
    private final BigDecimal multiplier;
    @Override
    public BigDecimal fare(VehicleType type, int km, int minutes) {
        return inner.fare(type, km, minutes).multiply(multiplier).setScale(2, RoundingMode.HALF_UP);
    }
}
// StandardPricing: base + perKm x km + perMinute x minutes, at least the minimum (one RateCard per VehicleType).

/** Returns a RANKED LIST (not one driver) so a decline just moves to the next name. */
interface MatchingStrategy {
    List<Driver> rank(Location pickup, VehicleType type, Collection<Driver> drivers);
}
// NearestDriverMatcher: available, right vehicle type, within maxRadiusKm, nearest first (tie: id).
// BestRatedMatcher: same filter, best rating first. PaymentMethod: CashPayment, WalletPayment.

// File: code/lld/ch20/ridesharing/RideSharingService.java
/** Driver answers the offer. A lost race for the driver counts as a decline. */
boolean respond(String tripId, String driverId, boolean accepts) {
    Trip trip = trips.get(tripId);
    Driver current = trip.currentCandidate();
    if (current == null || !current.id().equals(driverId) || trip.status() != TripStatus.REQUESTED) return false;
    if (accepts && current.tryClaim()) {
        if (trip.accept(driverId)) {
            publish(trip, "accepted by " + driverId);
            return true;
        }
        current.release(); // trip was canceled meanwhile
        return false;
    }
    publish(trip, (accepts ? "could not claim " : "declined by ") + driverId);
    offerOrCancel(trip, trip.nextCandidate());
    return false;
}

boolean endTrip(String tripId, int minutes) {
    Trip trip = trips.get(tripId);
    BigDecimal fare = pricing.fare(trip.type(), trip.distanceKm(), minutes);
    if (!trip.end(minutes, fare)) return false;               // (publishes "end refused")
    Receipt receipt = riders.get(trip.riderId()).paymentMethod().charge(fare);
    Driver d = drivers.get(trip.driverId());
    d.moveTo(trip.drop());       // easy to forget: the car is now at the drop point
    d.release();
    // ... record the trip in both histories, publish "ended"
    return true;
}
```

Walking through the code.

- `TripStatus.NEXT` is the whole lifecycle in five lines; `Trip.move` refuses any move not in the table.
- `Trip` methods are `synchronized`: rider cancel and driver accept on the same trip cannot interleave (have their steps mixed together).
- `RideSharingService.respond`: only the *current* candidate may answer. Accept first **claims the driver atomically**; if the claim fails (another trip got them), it's treated as a decline and the next candidate is offered.
- `endTrip` works in a fixed order: price, store the fare on the trip, charge the rider, relocate the driver to the drop-off point, and only then release them (so no matcher can see a free driver at the old position) — then it records history for both sides. The course lists the same steps; relocating the driver is the step most often skipped, and skipping it makes the next match use the driver's morning position.
- Ratings are allowed once per side and only for ENDED trips (`markDriverRated` / `markRiderRated`).

Output:

```
== 1. Request, first driver declines, second accepts, full ride
  [T1] requested by R1, candidates [D1, D2]
  [T1] offered to D1
  [T1] declined by D1
  [T1] offered to D2
  [T1] accepted by D2
  D2 available while on trip? false
  [T1] started
  [T1] start refused in state STARTED
  [T1] ended: 7 km, 12 min, fare 13.90 (wallet debited 13.90, left 36.10)
  D2 now at (5,3), available? true
  rate driver twice allowed? false; D2 rating 5.00, R1 rating 4.00
== 2. Surge: wrap the same rule in a decorator (x1.5)
  [T2] requested by R2, candidates [D1, D2]
  [T2] offered to D1
  [T2] accepted by D1
  [T2] started
  [T2] ended: 1 km, 3 min, fare 7.50 (collect 7.50 in cash)
== 3. Rider cancels after accept; cancel after start is refused
  [T3] requested by R1, candidates [D3]
  [T3] offered to D3
  [T3] accepted by D3
  [T3] canceled by rider
  D3 released? true
  [T4] requested by R1, candidates [D3]
  [T4] offered to D3
  [T4] accepted by D3
  [T4] started
  [T4] cancel refused in state STARTED
  [T4] ended: 4 km, 10 min, fare 17.00 (wallet debited 17.00, left 19.10)
== 4. Nobody available for a VAN
  [T5] requested by R2, candidates []
  [T5] no driver left - canceled
== 5. Race: 40 riders, 10 drivers, all accept at once
  trips accepted = 10 (never more than 10 drivers)
```

Check the fares: trip 1 is 7 km and 12 min → 2.50 + 7 × 1.20 + 12 × 0.25 = **13.90**. Trip 2 is 1 km, 3 min → 4.45, below the 5.00 minimum → 5.00, then the surge decorator × 1.5 → **7.50**. The race test shows 40 concurrent requests against 10 drivers produce exactly 10 accepted trips — no driver is ever double-booked.

Machine-coding variant: carpool / "offer and select a ride"

A different ride-sharing problem is popular in machine-coding rounds (it appears in the kumaransg/LLD collection — a public GitHub repository of low-level design (LLD: designing the classes inside one program) problems — as a Flipkart-style round). Here there is **no dispatcher**: users *offer* spare seats in their own vehicle on a route, and other users *select* a ride using a **selection strategy**.

Requirements (from the problem statement, in brief):

- `add_user`, `add_vehicle` (a user may own several vehicles).
- `offer_ride(user, origin, seats, vehicle, destination)` — **fails if that vehicle already has an active offered ride**.
- `select_ride(user, origin, destination, seats, strategy)` — seats is 1 or 2; strategy is **Most Vacant** (most free seats) or **Preferred Vehicle** (e.g. "Activa", a scooter model).
- `end_ride(ride)` and `print_ride_stats()` — rides *taken* and *offered* per user, counting only **ended** rides (an offered ride counts even if nobody took it).

Design: `Ride` with an atomic `book(user, seats)`; `RideSelectionStrategy` (Strategy) with `MostVacant` and `PreferredVehicle`; one `RideService` that filters active rides by route and seats, excludes the user's own offers, and lets the strategy choose. The demo replays the problem's sample test case and prints the expected answers (2(c), 2(b), none, 2(d), none; then the stats).

The whole program is one file, `code/lld/ch20/carpool/Main.java`; its core:

```
// File: code/lld/ch20/carpool/Main.java
/** Atomic check-and-book, so two users cannot take the last seat. */
synchronized boolean book(User u, int seats) {                    // Ride
    if (status != RideStatus.ACTIVE || availableSeats < seats) return false;
    availableSeats -= seats;
    riders.add(u);
    return true;
}

/** Strategy: pick one of the matching rides. */
interface RideSelectionStrategy { Optional<Ride> select(List<Ride> candidates); }

static final class MostVacant implements RideSelectionStrategy {
    public Optional<Ride> select(List<Ride> c) { return
c.stream().max(Comparator.comparingInt(Ride::availableSeats)); }
}
record PreferredVehicle(String model) implements RideSelectionStrategy {
    public Optional<Ride> select(List<Ride> c) {
        return c.stream().filter(r -> r.vehicle.model().equalsIgnoreCase(model)).findFirst();
    }
}

/** Fails if this vehicle already has an ACTIVE offered ride. */
synchronized Optional<Ride> offerRide(String name, String origin, int seats, String plate, String destination) {
    Vehicle v = vehicles.get(plate);
    if (v == null || !v.owner().equals(name)) return Optional.empty();
    boolean busy = rides.values().stream()
            .anyMatch(r -> r.vehicle.plate().equals(plate) && r.status == RideStatus.ACTIVE);
    if (busy) return Optional.empty();
    // ... create and store the Ride
}

synchronized Optional<Ride> selectRide(String name, String origin, String destination, int seats,
                                       RideSelectionStrategy strategy) {
    if (seats < 1 || seats > 2) throw new IllegalArgumentException("a user may request 1 or 2 seats");
    List<Ride> matching = rides.values().stream()
            .filter(r -> r.status == RideStatus.ACTIVE)
            .filter(r -> r.origin.equals(origin) && r.destination.equals(destination))
            .filter(r -> !r.offeredBy.name().equals(name))          // never your own offer
            .filter(r -> r.availableSeats() >= seats)
            .toList();
    Optional<Ride> chosen = strategy.select(matching);
    chosen.ifPresent(r -> r.book(users.get(name), seats));
    return chosen;
}
// endRide(): ACTIVE -> ENDED and only then counts the ride in print_ride_stats (offered, and taken per rider)
```

Output:

```
== offer_ride
  2a Rohan Swift Hyderabad->Bangalore seats=1
  2b Shipra Activa Bangalore->Mysore seats=1
  2c Shipra Polo Bangalore->Mysore seats=2
  2d Shashank Baleno Hyderabad->Bangalore seats=2
  2e Rahul XUV Hyderabad->Bangalore seats=5
  No rides found / request failed
== select_ride
  2c Shipra Polo Bangalore->Mysore seats=1
  2b Shipra Activa Bangalore->Mysore seats=0
  No rides found / request failed
  2d Shashank Baleno Hyderabad->Bangalore seats=1
  No rides found / request failed
== end_ride 2a..2d, then print_ride_stats
  Rohan: 1 Taken, 1 Offered
  Shashank: 0 Taken, 1 Offered
  Nandini: 1 Taken, 0 Offered
  Shipra: 0 Taken, 2 Offered
  Gaurav: 1 Taken, 0 Offered
  Rahul: 0 Taken, 0 Offered
```

	Uber-style (20.4 main design)	Carpool variant
Who supplies the ride	Professional drivers, online/offline	Any user, per offered ride
Who chooses	The system (matching strategy) offers to drivers	The rider picks with a selection strategy
Key pattern	Strategy + Decorator + state machine	Strategy (selection)
Key invariant	A driver is on at most one trip	A vehicle has at most one active offer; seats never go negative

Concurrency & edge cases

- **Double booking a driver:** prevented by `Driver.tryClaim()` (compare-and-set). Proven by the 40-riders/10-drivers race in the output.
- **Cancel vs accept race:** both go through `synchronized` `Trip` methods; if the rider cancels after the claim but before `accept`, `accept` fails and the driver is released.
- **Stale candidate list:** the list is computed once. A driver may go offline or be claimed later; `tryClaim` fails and we move on. A nearer driver who becomes free later is not noticed (real systems re-match periodically).
- **Offer timeouts:** a driver who never answers should be treated as a decline after N seconds (a `ScheduledExecutorService` task per offer).
- **No drivers at all:** trip canceled immediately (see trip T5).
- **Illegal transitions** (start twice, cancel after start, end before start): return `false`; state unchanged.
- **Wallet with insufficient balance:** `Receipt(paid=false)`; a real system would retry another method or record a debt on the rider's account.
- **Rating abuse:** one rating per side per trip; stars validated 1–5.

Extensions & follow-up questions

- **"Rank by rating instead."** Swap `MatchingStrategy` (`BestRatedMatcher` is included) — one line at startup.
- **"Airport fee, promo code, tolls."** More decorators around the pricing rule: `new PromoDiscount(new AirportFee(new SurgePricing(standard, 1.5)))`.
- **"How is surge computed?"** From demand (open requests) vs supply (free drivers) per area and time window; the result feeds the multiplier. The pricing design doesn't change.
- **"Find nearby drivers fast in a real city."** A geospatial index (a lookup structure for finding things by location): split the map into cells (geohash, Google S2 or Uber's H3 hexagons — three

standard ways of dividing the globe into numbered cells) and keep a map cell → drivers; search the pickup cell and its neighbours instead of all drivers.

- **"Shared rides (pool)."** Honest answer: this design assumes one rider per trip. The change starts at `Trip` (many riders, many stops) and a matcher that considers detours.
- **"Live tracking."** Drivers send location updates every few seconds; store the latest per driver and publish to the rider (pub-sub, see 20.6).
- **"Scheduled rides."** A request with a future time; a scheduler (a component that runs jobs at chosen times) starts matching shortly before.

Real-world example

Uber really does show riders a surge ("dynamic pricing") multiplier when demand exceeds supply in an area, and it sends a request to a nearby driver who has a short time to accept before it goes to someone else — the ranked-list-with-cursor idea. Uber open-sourced (published free for anyone to use) **H3**, a hierarchical (cells nest inside bigger cells) hexagonal grid that its engineering blog describes as built for optimizing ride pricing and dispatch (for example, bucketing supply and demand per hexagon); the same cell idea answers "which drivers are near this point" without scanning the whole city. Ola, Lyft and Grab follow the same request → match → accept → trip → rate lifecycle.

In the interview

> **30-second answer:** "Two rules behind interfaces: `MatchingStrategy` returns a ranked list of free drivers with the right car, and `PricingStrategy` computes base + distance + time with a floor. Surge is a Decorator that wraps any pricing rule and multiplies it. The trip is a small state machine; every action checks the transition table and returns false if illegal. A decline moves a cursor along the stored ranking. Drivers are claimed with a compare-and-set so one driver can never get two trips."

Tip: name the two rules (matching and pricing) in your first minutes and say both go behind interfaces; that one sentence pre-answers most follow-ups. Then spend real time on surge: *wrapping* the pricing rule instead of adding a flag to it is what separates strong answers from weak ones.

Plan (40 min): clarify (5) → accounts, vehicle, location (5) → trip + state machine (8) → pricing + surge decorator (7) → matching list + decline flow (7) → concurrency (claim) (4) → extensions (4).

Follow-ups with answers:

- *"What happens when a driver declines?"* → Next driver in the stored ranked list; none left → cancel.
- *"Strategy vs Decorator here?"* → Strategy replaces the rule; Decorator wraps it and remains in the call path (it holds an `inner` of the same interface).
- *"Two riders get the same driver?"* → Impossible: atomic claim; the loser moves to its next candidate.

Self-check

1. Why does the matcher return a list instead of one driver? → So a decline just moves to the next candidate without re-searching and remembers who refused.
2. What makes `SurgePricing` a Decorator? → It is a `PricingStrategy` that also wraps a second `PricingStrategy` and adjusts that one's result.
3. Can a rider cancel a STARTED trip? → No; `STARTED` can only move to `ENDED`.
4. Why update the driver's location to the drop-off when the trip ends? → Later matches must use their real position.
5. Which Java class makes the driver claim atomic? → `AtomicBoolean` (`compareAndSet(false, true)`).
6. In the carpool variant, when does `offer_ride` fail? → When the same vehicle already has an active offered ride.

TL;DR

- **Strategy** ×3 (matching, pricing, payment) + **Decorator** (surge) + **state machine** (trip).

- Matcher returns a **ranked list**; the trip keeps a cursor; decline = next.
- **Atomic claim** of drivers prevents double booking; relocate the driver to the drop-off at trip end.
- Ratings = running total + count; `BigDecimal` fares with a minimum.
- Carpool variant: users offer seats; riders select with Most Vacant / Preferred Vehicle strategies.

20.5 Design a Logging Framework

Problem & clarifying questions

What it is. A logging framework lets code write messages like "payment 991 authorised" with a **level** (how serious it is) and sends them to one or more **destinations** (console, file, a log server), in a chosen **format**. Everyday analogy: a company's internal mail room. Every department drops notes in; each note has an urgency stamp; the mail room decides, by department and urgency, which notes get copied to which inboxes and in what envelope.

Think Log4j 2, Logback or `java.util.logging` (the most widely used Java logging libraries): this question asks you to design a small version of them.

Question	Assumed answer
Levels?	TRACE < DEBUG < INFO < WARN < ERROR < FATAL (and OFF).
Per-module configuration?	Yes: loggers are named `app.payment`, `app.orders`… and form a **hierarchy** by dots. A child inherits its level from the nearest ancestor that sets one.
Destinations?	Console, file, in-memory (tests); pluggable. Each can have its own minimum level.
Formats?	Plain pattern and JSON (a text format of nested key-value pairs); pluggable.
Propagation?	An event goes to the logger's own destinations and then **up** to its ancestors' destinations, unless that logger turns propagation (additivity) off.
Async?	Yes, optional: the application thread must not wait for disk I/O (input/output: reading or writing the disk).
Thread-safe?	Yes: many threads log at once.
Config files, rolling files (a log file is closed and a new one started at a size or date limit), remote shipping (sending logs to another server)?	Out of scope (extensions).

Requirements

Functional

1. Log a message at a level; messages below the logger's effective level are dropped cheaply.
2. Loggers are obtained by name; the same name returns the same logger.
3. Loggers form a dot-separated hierarchy under a root; level is inherited when not set.
4. A logger has zero or more appenders (destinations). Each appender has a minimum level and a formatter.
5. Events propagate up to parent appenders unless additivity is off.
6. Formatters are pluggable (pattern, JSON).
7. Asynchronous appending is available without changing the appenders.

8. Exceptions can be attached to an event.
9. Expensive messages can be built lazily (only if the level is enabled).

Non-functional: very cheap when disabled; thread-safe; logging must never crash the application; testable (inject a clock).

Use cases

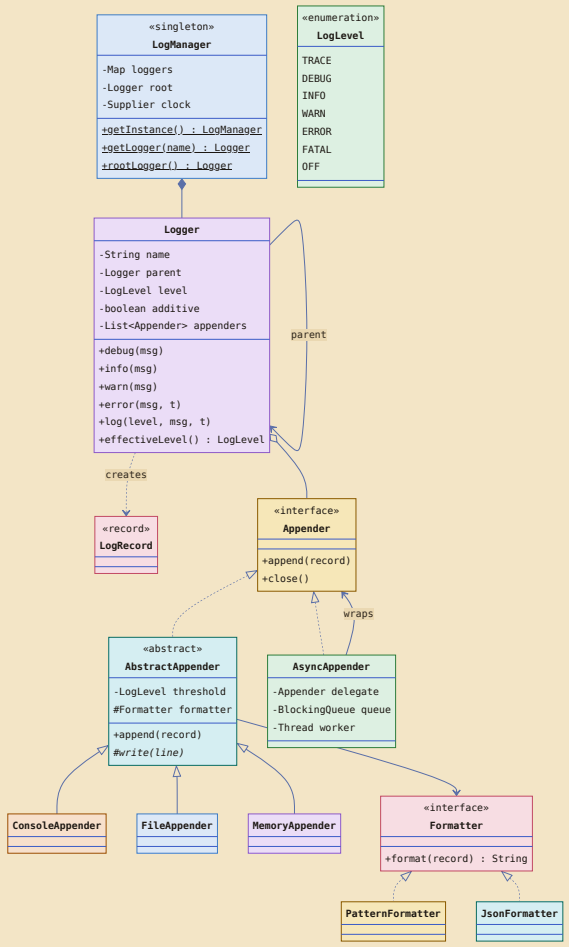
Actors: **Developer code** (logs messages), **Operator** (configures levels, appenders, formats at startup or runtime — while the program is running), **Framework** (filters, formats, routes, writes).

Main use cases: *Log a message*, *Get a logger by name*, *Set a level for a subtree*, *Add a destination*, *Choose a format*, *Switch to async writing*, *Stop propagation for a subtree*.

Core entities

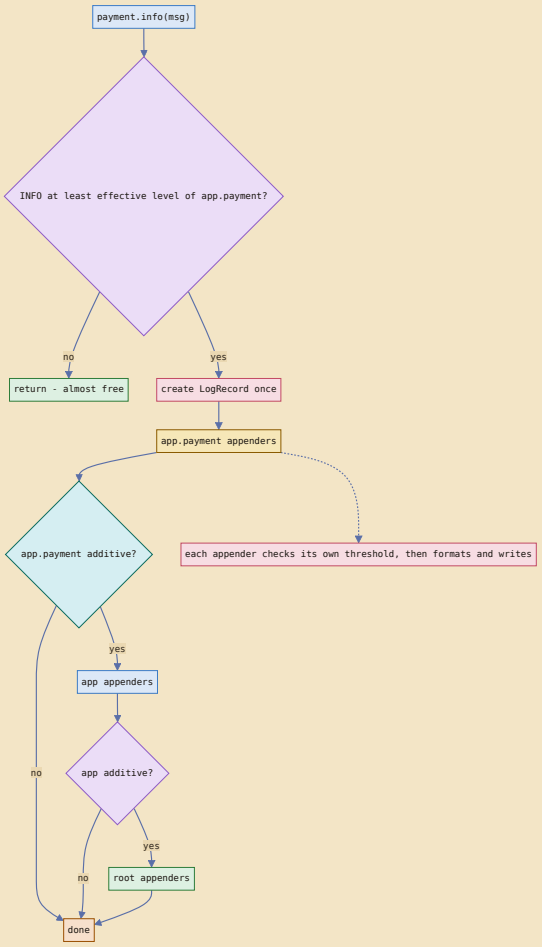
Class	Responsibility
`LogLevel` (enum)	Ordered severities; `isAtLeast(threshold)`.
`LogRecord` (record)	One immutable (cannot change after creation) event: time, level, logger name, thread, message, error.
`Logger`	Named node in the hierarchy: parent, optional level, appenders, additivity; `log()` walks the chain upward.
`LogManager`	**Singleton** registry; `getLogger(name)` creates parents first and returns one instance per name; holds the clock.
`Appender` (interface)	A destination (sink).
`AbstractAppender`	**Template Method**: `append()` checks the appender's threshold and formats; subclasses only `write(line)`.
`ConsoleAppender`, `FileAppender`, `MemoryAppender`	Concrete sinks.
`AsyncAppender`	**Decorator**: wraps any appender with a bounded queue + one writer thread.
`Formatter` (interface)	**Strategy**: record → text. `PatternFormatter`, `JsonFormatter`.

Class diagram



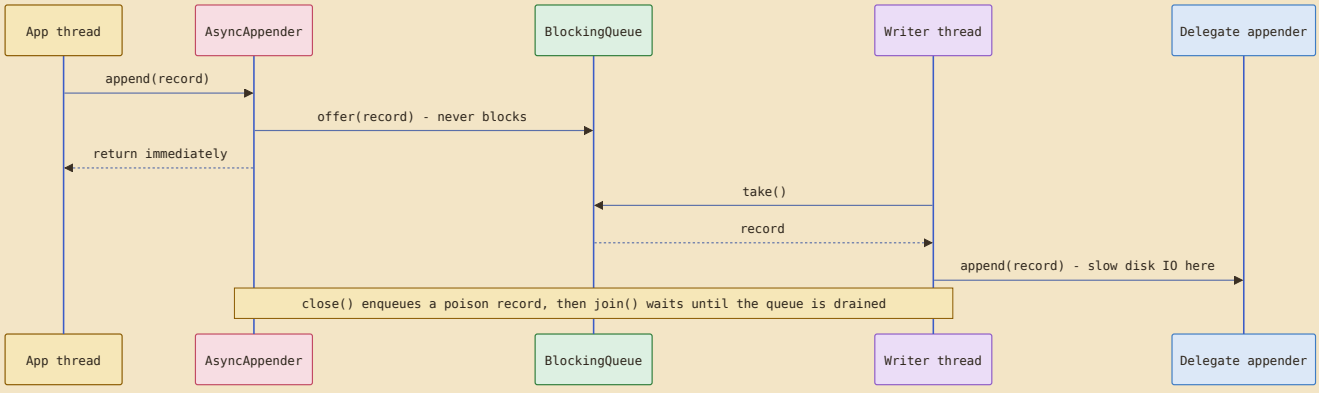
Key flows

Logging one event (Chain of Responsibility up the hierarchy). *Chain of Responsibility* = a request is passed along a chain of handlers; each may handle it and/or pass it on. Here every logger in the chain hands the record to its own appenders and passes it to its parent, unless additivity is off.



Note the Log4j/Logback rule we copy: the **level check happens once, at the logger where the call was made**. Parents' *levels* are not re-checked during propagation; only each **appender's own threshold** filters. That is why, in the demo, a DEBUG event from `app.payment` (set to DEBUG) still reaches the root console appender even though root is at INFO.

Async appending.



Design decisions & patterns used

Pattern / decision	Where	Why	Alternative rejected
Chain of Responsibility	`Logger.log` walks `this → parent → … → root`, stopping when `additive == false`	Configure destinations once near the root; subtrees can add or stop.	Every logger listing all its appenders explicitly.
Singleton	`LogManager` (initialization-on-demand holder)	One registry per process, so `getLogger("x")` always returns the same object. Holder idiom (a standard coding recipe) = lazy + thread-safe with no locking (the JVM — Java Virtual Machine, the program that runs Java code — initializes the nested class once).	Public static mutable (changeable) map; double-checked locking (check, lock, check again; easy to get wrong).
Static factory method	`LogManager.getLogger(name)`	Hides creation, caches instances, creates parents first.	`new Logger(...)` by users (breaks the hierarchy).
Strategy	`Formatter`	Format is independent of destination: any sink × any format.	`JsonFileAppender`, `PatternFileAppender`, … (class explosion).
Template Method	`AbstractAppender.append` (final) → `write`	Threshold + formatting done once; sinks only write text.	Each sink re-implementing filtering.
Decorator	`AsyncAppender` wraps any `Appender`	Async becomes an add-on, not a rewrite of every sink.	`AsyncFileAppender`, `AsyncConsoleAppender`, …
Level check first; **lazy** `Supplier<String>` (a function that builds the string only when called)	`isEnabled`, `debug(Supplier)`	Disabled logging costs one comparison; expensive strings are never built.	Always building the message (a real performance bug).
Appender failures caught	`Logger.log`	Logging must never crash the app.	Letting exceptions escape.
Parent resolved **outside** `computeIfAbsent`	`LogManager.loggerFor`	A `ConcurrentHashMap` mapping function must not modify the same map (the JDK may throw `IllegalStateException: Recursive update`).	Recursive `computeIfAbsent`.

A note on another common "CoR logger" answer. Many LLD write-ups model Chain of Responsibility as a chain of *level handlers* (`ErrorHandler → WarnHandler → InfoHandler → DebugHandler`), each writing messages of "its" level and passing others along. It works for a toy, but real frameworks put the chain on the **logger hierarchy** (as here) because that is what operators actually configure. If an interviewer expects the level-handler version, explain both and why the hierarchy is the one Log4j/Logback use.

Java implementation

Files: `Core.java` (level, record, formatters), `Appenders.java` (appender interface, template base, console/file/memory appenders, async decorator), `Logger.java` (logger chain, `LogManager` singleton), `Main.java`. The full runnable source is in `code/lld/ch20/logging/`; below are the lines that carry the design.

```java
// File: code/lld/ch20/logging/Core.java
/** Severity, lowest to highest. A logger set to WARN drops TRACE, DEBUG and INFO. */
enum LogLevel {
    TRACE, DEBUG, INFO, WARN, ERROR, FATAL, OFF;
    boolean isAtLeast(LogLevel threshold) { return this.ordinal() >= threshold.ordinal(); }
}
/** One immutable log event. Created once, passed along the chain, formatted by each appender. */
record LogRecord(Instant time, LogLevel level, String loggerName, String thread, String message, Throwable error)
{}
interface Formatter { String format(LogRecord r); }      // Strategy: PatternFormatter, JsonFormatter

// File: code/lld/ch20/logging/Logger.java
LogLevel effectiveLevel() {
    for (Logger l = this; l != null; l = l.parent) if (l.level != null) return l.level;
    return LogLevel.INFO;
}

/** Lazy message: the (possibly expensive) string is only built if the level is enabled. */
void debug(Supplier<String> msg) { if (isEnabled(LogLevel.DEBUG)) log(LogLevel.DEBUG, msg.get(), null); }

void log(LogLevel lvl, String msg, Throwable t) {
    if (!isEnabled(lvl)) return;                    // cheap check first
    LogRecord r = new LogRecord(manager.now(), lvl, name, Thread.currentThread().getName(), msg, t);
    for (Logger l = this; l != null; l = l.parent) { // the chain: "app.payment" -> "app" -> root
        for (Appender a : l.appenders) {
            try {
                a.append(r);
            } catch (RuntimeException e) {
                System.err.println("appender failed: " + e); // logging must never crash the app
            }
        }
        if (!l.additive) break;
    }
}

/** Singleton registry of loggers (initialization-on-demand holder idiom: lazy and thread-safe). */
final class LogManager {
    private LogManager() { root.setLevel(LogLevel.INFO); }
    private static final class Holder { static final LogManager INSTANCE = new LogManager(); }
    static LogManager getInstance() { return Holder.INSTANCE; }
    /** Factory method: same name -> same Logger instance; parents are created first. */
    static Logger getLogger(String name) { return getInstance().loggerFor(name); }
}

// File: code/lld/ch20/logging/Appenders.java
abstract class AbstractAppender implements Appender {     // threshold + formatter handled once
    @Override
    public final void append(LogRecord r) {
        if (r.level().isAtLeast(threshold)) write(formatter.format(r));
    }
    protected abstract void write(String line);           // Console/File/Memory: synchronized write
}

/** Decorator: makes ANY appender asynchronous. Callers only enqueue; one background thread writes. */
final class AsyncAppender implements Appender {
    private static final LogRecord POISON = new LogRecord(null, LogLevel.OFF, "", "", "", null);

    @Override
    public void append(LogRecord r) {
        if (closed || !queue.offer(r)) dropped.incrementAndGet(); // full or closed: count it, never block
    }

    private void drain() {                                  // the background writer thread
        try {
            for (LogRecord r = queue.take(); r != POISON; r = queue.take()) {
                try {
                    delegate.append(r);
                } catch (RuntimeException e) {  // one bad write must not kill the writer thread
                    System.err.println("async appender: delegate failed: " + e);
                }
```

```
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
    // close(): closed = true; queue.put(POISON); worker.join(); delegate.close() - flushes what was queued
}
```

Walking through the code.

- `Logger.effectiveLevel` walks up until a logger has a level; root always has one.
- `Logger.log` creates the `LogRecord` **once** and hands the same immutable object to every appender in the chain (safe to share across threads because it is a record with final fields).
- `AbstractAppender.append` is `final`; `ConsoleAppender` and `FileAppender` synchronize `write` so lines from different threads never interleave mid-line.
- `AsyncAppender` uses a bounded `ArrayBlockingQueue` (a fixed-size, thread-safe queue) and `offer` (non-blocking: returns at once instead of waiting): when full, it drops and counts instead of stalling the application. `close()` puts a "poison pill" record (a special marker that means "stop after this") so the writer drains everything before stopping. The no-loss promise is precise: every record appended **before** `close()` is either written or counted in `dropped` (queue full); a record appended after `close()` is counted as dropped too, so `written + dropped` always equals the number of `append` calls. A delegate that throws is caught inside the writer loop, so one bad write cannot kill the writer thread (which would otherwise leave `close()` waiting forever on a full queue).
- `LogManager` uses the holder idiom: `Holder.INSTANCE` is created the first time `getInstance()` runs, and the JVM guarantees that happens once.

Output:

```
== 1. Same name -> same instance: true
== 2. Levels: 'app.payment' inherits INFO from root
2026-09-28 10:15:30.123 INFO  [main] app.payment - payment 991 authorised
2026-09-28 10:15:30.123 WARN  [main] app.payment - retrying gateway call
== 3. Raise one subtree to DEBUG; extra JSON sink only for payments
2026-09-28 10:15:30.123 DEBUG [main] app.payment - card token=tok_42 (built lazily)
  audit sink got: [{"ts":"2026-09-28T10:15:30.123Z","level":"DEBUG","logger":"app.payment","msg":"card
token=tok_42 (built lazily)"}]
== 4. Additivity off: 'app.payment' events stop at its own appenders
  (nothing printed on console above) audit sink now has 2 lines
== 5. File sink with its own threshold (WARN+)
2026-09-28 10:15:30.123 INFO  [main] app - info goes to console only
2026-09-28 10:15:30.123 WARN  [main] app - disk 91% full
  file contains: [2026-09-28 10:15:30.123 WARN  [main] app - disk 91% full]
== 6. Async appender: 4 threads x 1000 events, then close() flushes
  written=4000 dropped=0 total=4000
```

Step 4: with additivity off, the ERROR went only to the JSON audit sink (2 lines now) and not to the console. Step 6: 4 threads × 1000 events all arrived (4000 written, 0 dropped), and `close()` flushed the queue.

Concurrency & edge cases

- **Many threads logging:** appenders list is `CopyOnWriteArrayList` (reads without locking, rare writes); level and additivity are `volatile` (a Java keyword that makes every thread see the latest written value) so a runtime change is seen by all threads.
- **Interleaved output:** each sink's `write` is synchronized.
- **Async queue full:** drop-and-count (never block). Alternatives: block (back-pressure — slowing the producer down when the consumer cannot keep up — can stall the app) or drop only low levels. Logback's `AsyncAppender`, by default, starts discarding TRACE/DEBUG/INFO events when the queue is 80% full.
- **Shutdown:** without `close()`, queued events are lost; register a JVM shutdown hook (code the JVM runs just before the program exits) that closes appenders.
- **Recursive logging** (an appender that logs while appending) can loop forever; guard with a thread-local (each thread gets its own separate copy) "already inside" flag.

- **Security:** never let log messages trigger lookups or code; Log4Shell (CVE-2021-44228, December 2021; a CVE is a public id for a known security flaw) came from Log4j 2 interpreting `${jndi:...}` (a lookup that could load code from a remote server) inside logged strings. Treat messages as plain data.
- **Sensitive data:** mask card numbers, tokens and passwords before formatting.

Extensions & follow-up questions

- **"Rolling files."** A `RollingFileAppender` with a `RollingPolicy` (size- or time-based) Strategy: when triggered, close, rename to `app-2026-09-28.log`, open a new file.
- **"Config file."** Parse XML/YAML/properties (common configuration-file text formats) into calls on `LogManager` (set levels, attach appenders). Watching the file lets levels change at runtime.
- **"Filters."** A `Filter` chain per appender (e.g. only `app.payment`, only messages containing "refund") — another Chain of Responsibility.
- **"MDC / request id."** A thread-local map (Mapped Diagnostic Context) copied into each `LogRecord`, so every line of one request carries the same id.
- **"Ship logs to a server."** A network appender behind `AsyncAppender`, batching records.
- **"Structured logging."** Key-value fields in `LogRecord` rendered by `JsonFormatter` for tools like Elasticsearch/Kibana (a search engine and its dashboard, often used to store and browse logs).

Real-world example

SLF4J (Simple Logging Facade for Java) is a facade (the API your code calls); **Logback** and **Log4j 2** are implementations with exactly this model: dot-named loggers in a hierarchy, inherited levels, appenders with layouts/encoders, and *additivity*. Log4j 2's async loggers use the LMAX Disruptor (a lock-free ring buffer: a fixed-size circular array that threads share without taking locks) to hand events to a background thread. `java.util.logging` has the same shape with different names: `Logger`, `Handler` (appender), `Formatter`, `Level`, and `setUseParentHandlers(false)` (additivity off).

In the interview

> **30-second answer:** "Loggers are named and form a dot hierarchy under a root managed by a singleton `LogManager`. A log call first checks the effective level (inherited), builds one immutable record, and passes it up the chain of loggers to their appenders — Chain of Responsibility — until additivity stops it. Each appender has its own threshold and a pluggable formatter (Strategy). `AsyncAppender` is a Decorator with a bounded queue and a writer thread, so the app never waits for disk."

Plan (35 min): clarify (5) → levels + record (4) → logger hierarchy + effective level (6) → appenders + formatters (6) → propagation/additivity (5) → singleton + thread safety (4) → async (5).

Follow-ups with answers:

- *"Why is a disabled debug call cheap?"* → One level comparison before any string is built; use `Supplier` for expensive messages.
- *"How do you make it thread-safe?"* → Immutable records, `CopyOnWriteArrayList` appenders, synchronized sinks, `ConcurrentHashMap` registry, volatile config.
- *"What happens to queued logs on crash?"* → They're lost unless flushed; close on shutdown, or accept the trade-off for speed.

Self-check

1. `app.payment` has no level; `app` is WARN; root is INFO. What is `app.payment`'s effective level? → WARN (nearest ancestor that sets one).
2. What does additivity=false do? → Stops the event from reaching ancestor loggers' appenders.
3. Which pattern lets any appender become async? → Decorator (`AsyncAppender`).
4. Why is `Formatter` separate from `Appender`? → So formats and destinations combine freely (Strategy) without a class per combination.

5. Why resolve the parent before `computeIfAbsent`? → Modifying the same `ConcurrentHashMap` inside its mapping function is not allowed.
6. How does `close()` guarantee nothing queued is lost? → It enqueues a poison pill and joins the writer thread, which drains every record queued before the pill. (Records refused because the queue was full, or appended after `close()`, are not lost silently: they are counted as dropped.)

TL;DR

- Named **logger hierarchy**; effective level inherited; one cheap level check up front.
- **Chain of Responsibility** up to the root, stopped by additivity; **Singleton** `LogManager`.
- **Strategy** formatters, **Template Method** appender base, **Decorator** async appender.
- Immutable `LogRecord`, thread-safe sinks, never crash the app, never block on a full queue.

20.6 Design a Pub-Sub System

Problem & clarifying questions

`What it is.` Publish-subscribe (pub-sub) is a way for parts of a system to talk **without knowing each other**. A *publisher* sends a message to a named *topic* ("orders"); every *subscriber* of that topic receives it. Everyday analogy: a newspaper. The newspaper prints once; every subscriber gets a copy; the newspaper doesn't know or care who reads it, and new readers can sign up any time.

It is the **Observer pattern** turned into infrastructure: in Observer the subject calls its observers directly; in pub-sub a **broker** in the middle decouples them further (makes each side depend less on the other: by topic, by time, and often by machine). We design a simplified, Kafka-like, in-process (living inside one running program) broker.

Question	Assumed answer
Delivery to how many?	Every subscription gets every message (fan-out).
Order?	Messages are delivered in publish order per subscription.
Guarantee?	**At-least-once**: a message is only acknowledged after the subscriber processes it; failures are retried; duplicates are possible.
Slow subscriber?	Must not slow down the others or the publisher.
Keep messages after delivery?	Yes: an append-only log with **offsets**, so late subscribers can start from the beginning and anyone can replay.
Poison message (always fails)?	After N attempts, send to a dead-letter topic (a side topic that holds messages that could not be processed) and move on.
Push or pull?	Push to a callback, driven by one dispatcher (the loop that hands out messages) per subscription.
Partitions, consumer groups sharing work, persistence to disk, retention?	Out of scope (extensions; each is explained there).

Requirements

Functional

1. Create topics; publish a message (key, payload) to a topic; the broker assigns an **offset** (its position in the topic's log).
2. Subscribe to a topic with a subscription id and a callback, starting from EARLIEST (offset 0) or LATEST (only new messages).
3. Each subscription receives every message in order.
4. A message is acknowledged (offset committed) only after the callback returns normally.

5. Failures are retried with backoff; after `maxAttempts` the message goes to a dead-letter topic.
6. A subscription can **seek** to an offset to replay.
7. Unsubscribe; shut down cleanly.

Non-functional: thread-safe publish from many threads; subscribers isolated from each other; no busy-waiting (a thread spinning in a loop checking "anything yet?" and burning CPU (processor) time; here threads sleep until data arrives); ordering preserved.

Use cases

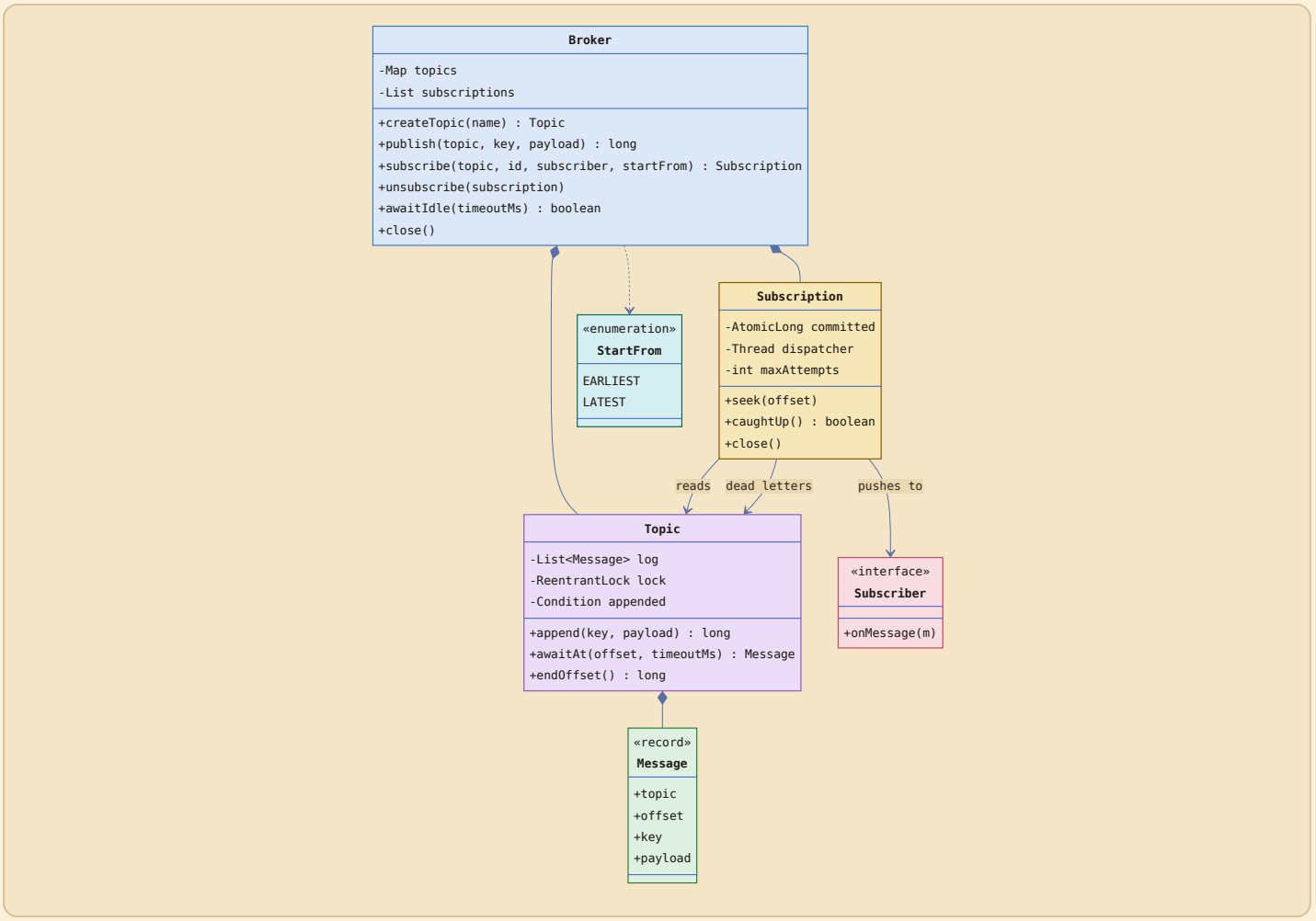
Actors: **Publisher** (publishes), **Subscriber** (subscribes, processes, fails sometimes), **Operator** (creates topics, replays, inspects dead letters), **Broker** (stores, dispatches, retries).

Core entities

Class	Responsibility
`Message` (record)	topic, offset, key, payload. Immutable.
`Topic`	Append-only log (`List<Message>`), guarded by a `ReentrantLock` with a `Condition` (an object a thread can wait on until another thread signals it) "appended" to wake waiting dispatchers.
`Subscriber` (interface)	Callback `onMessage(Message)`; throwing = negative acknowledgement ("I could not process this").
`Subscription`	One subscriber on one topic: committed offset (`AtomicLong`), its **own dispatcher thread**, retry and dead-letter logic, `seek`.
`StartFrom` (enum)	EARLIEST / LATEST.
`Broker`	Topic registry, `publish`, `subscribe`, `unsubscribe`, dead-letter topic, `awaitIdle` (tests), `close`.

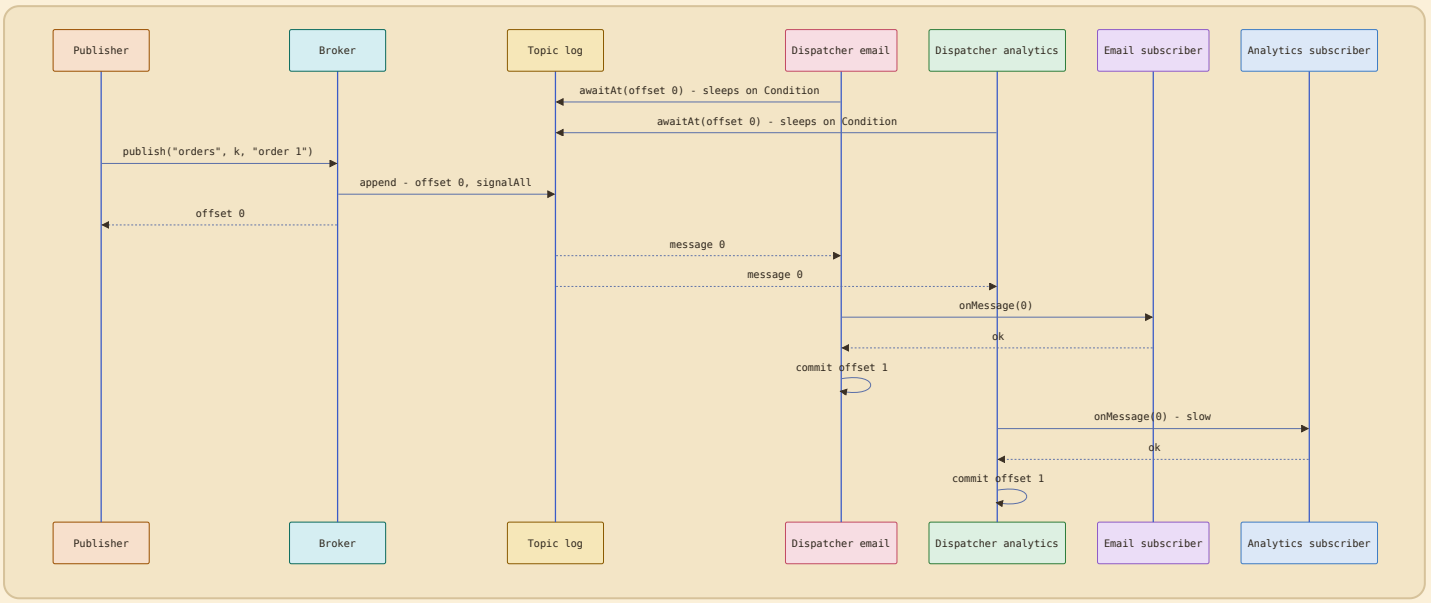
The key idea: messages are not removed when consumed. Each subscription only remembers *how far it has read* (its offset). That gives, almost for free: late subscribers from the beginning, replay (`seek`), and independent progress per subscriber. This is Kafka's model. (Classic queues such as RabbitMQ, a popular open-source message broker, instead delete a message once it is acknowledged.)

Class diagram

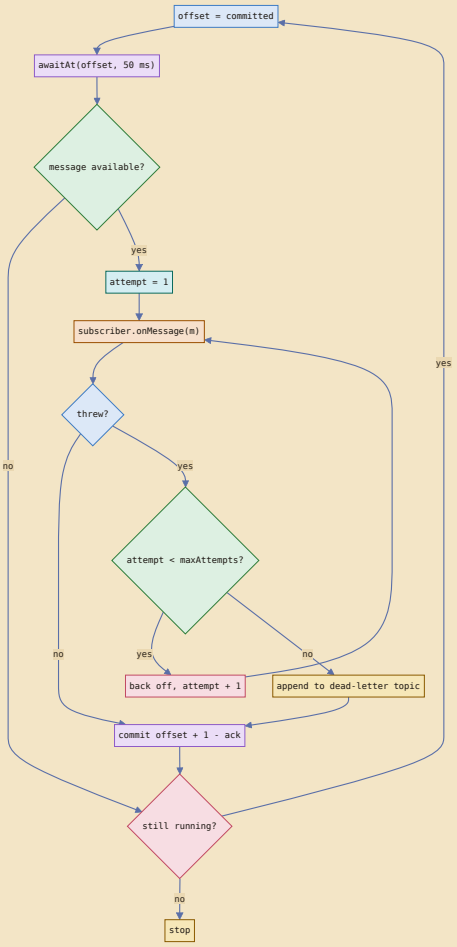


Key flows

Publish and fan-out.



One subscription's dispatcher loop (at-least-once with dead letters).


offset = committed
awaitAt(offset, 50 ms)
message available?
yes
attempt = 1
subscriber.onMessage(m)
threw?
yes
attempt < maxAttempts?
yes
back off, attempt + 1
no
append to dead-letter topic
commit offset + 1 - ack
still running?
yes
no
stop
no
yes
no
yes
no

Design decisions & patterns used

Decision	Why	Alternative rejected
Publish-Subscribe (Observer via a broker)	Publishers and subscribers never reference each other; add a subscriber with zero publisher changes.	Publisher calling each consumer directly (tight coupling).
Append-only log + offsets	Replay, late joiners and independent progress. Ordering is the log order.	Deleting on delivery: no replay; per-subscriber copies: wasteful.
One dispatcher thread per subscription (virtual threads: Java 21's very cheap threads managed by the JVM rather than the operating system)	A slow subscriber only delays itself (see "analytics" in the output). Order within a subscription is kept because only one thread delivers for it. Virtual threads make one-thread-per-subscription cheap.	One shared thread delivering to all subscribers in turn (the slowest blocks everyone); a thread pool (a fixed set of reusable threads) per message (loses ordering).
`Lock` + `Condition` wait, with timeout	No busy-waiting; the timeout lets the loop notice `close()` and `seek()`. `while` loop guards against spurious wake-ups (a waiting thread may wake up although nobody signalled it).	`Thread.sleep` polling (repeatedly checking; wastes CPU and adds latency — the delay before something is handled).
Ack after processing (commit after `onMessage`)	At-least-once: a crash mid-processing re-delivers.	Commit before processing = at-most-once (each message is delivered once or not at all; can lose messages).
Dead-letter topic	One poison message cannot block the subscription forever; operators can inspect and re-publish.	Infinite retries (the subscription is stuck).
`compareAndSet` when committing	If `seek()` ran during delivery, the seek wins instead of being overwritten.	Plain `set`.

Delivery guarantees in one table:

Guarantee	How	Risk
At-most-once	Commit offset, then process	A crash loses the message
At-least-once (ours)	Process, then commit	A crash or retry duplicates the message → make subscribers **idempotent**
Exactly-once	At-least-once + deduplication (spotting and dropping repeats) / transactions that commit output and offset together	Complex; Kafka offers it within its own transactions

Java implementation

Files: `Topic.java` (message, append-only topic log), `Broker.java` (subscriber, subscription with its dispatcher, broker), `Main.java`. The full runnable source is in `code/lld/ch20/pubsub/`; below are the lines that carry the design.

```java
// File: code/lld/ch20/pubsub/Topic.java
/** An append-only log (like one Kafka partition). Consumption never removes; each subscription keeps an offset.
*/
long append(String key, String payload) {
    lock.lock();
    try {
        long offset = log.size();
        log.add(new Message(name, offset, key, payload));
        appended.signalAll();                  // wake every waiting dispatcher
        return offset;
    } finally {
        lock.unlock();
    }
}

/** Returns the message at 'offset', waiting up to the timeout for it to be published. */
Message awaitAt(long offset, long timeoutMs) throws InterruptedException {
    lock.lock();
    try {
        long nanos = TimeUnit.MILLISECONDS.toNanos(timeoutMs);
        while (offset >= log.size()) {         // loop: guards against spurious wake-ups
            if (nanos <= 0) return null;
            nanos = appended.awaitNanos(nanos);
        }
        return log.get((int) offset);
    } finally {
        lock.unlock();
    }
}

// File: code/lld/ch20/pubsub/Broker.java
/** One subscriber's view of one topic: committed offset + its own dispatcher (virtual) thread. At-least-once, in
order. */
final class Subscription {
    private void loop() {
        try {
            while (running) {
                long offset = committed.get();
                Message m = topic.awaitAt(offset, 50);
                if (m == null) continue;              // timed out: re-check 'running' and any seek()
                deliver(m);
                committed.compareAndSet(offset, offset + 1); // ack; a concurrent seek() wins
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    private void deliver(Message m) throws InterruptedException {
        for (int attempt = 1; attempt <= maxAttempts; attempt++) {
            try {
                subscriber.onMessage(m);
                return;
            } catch (Exception e) {
                if (attempt == maxAttempts) {
                    deadLetters.append(id, m.topic() + "@" + m.offset() + " " + m.payload() + " -> " +
e.getMessage());
                    return;                           // skip it so one poison message cannot block the rest
                }
                Thread.sleep(5L * attempt);           // simple backoff
            }
        }
    }

    /** Replay (or skip) by moving the offset - possible because the log keeps every message. */
    void seek(long offset) { committed.set(offset); }
}

// Broker: subscribe(topic, id, subscriber, StartFrom.EARLIEST | LATEST) starts at offset 0 or at endOffset();
// maxAttempts = 3, failures go to the "__dead_letters" topic; publishers never see subscribers.
```

Walking through the code.

- `Topic.append` adds to the list and calls `signalAll()` under the lock, waking every dispatcher waiting for new data.
- `Topic.awaitAt` waits on the `Condition` in a `while` loop with `awaitNanos` (correct handling of spurious wake-ups and timeouts), then returns the message at that offset.
- `Subscription.loop` delivers one message at a time and only then commits `offset + 1` with `compareAndSet`, so a concurrent `seek()` wins.
- `deliver` retries with a small backoff and, after `maxAttempts`, writes a description to the `__dead_letters` topic and moves on.
- `Broker.awaitIdle` is a test helper: it waits until every subscription's committed offset has reached its topic's end.

Output:

```
== 1. Fan-out: every subscriber gets every message, in order
  all acknowledged: true
  email     [0:order#1, 1:order#2, 2:BAD-JSON, 3:order#3]
  analytics [0:order#1, 1:order#2, 2:BAD-JSON, 3:order#3]  (slow, but did not slow the others)
  inventory [0:order#1, 1:order#2, 3:order#3]  (offset 1 retried, offset 2 dead-lettered)
  dead letter: orders@2 BAD-JSON -> cannot parse
== 2. A late subscriber chooses where to start
  audit (EARLIEST) [0:order#1, 1:order#2, 2:BAD-JSON, 3:order#3, 4:order#4]
  live  (LATEST)   [4:order#4]
== 3. Replay: seek the 'live' subscription back to offset 3
  live after replay [4:order#4, 3:order#3, 4:order#4] (order#4 seen twice: at-least-once)
== 4. 4 publisher threads x 250 messages on a new topic
  received 1000 messages, offsets strictly in order: true
```

Points to notice: the slow "analytics" subscriber still got everything, and the other subscribers were not delayed; "inventory" retried offset 1 (a transient — temporary — failure) and dead-lettered offset 2 (the poison message) while still receiving offset 3; after `seek(3)`, "live" saw order#4 twice — that is what at-least-once means. The 1000 messages from 4 concurrent publishers got unique offsets and arrived strictly in order.

Concurrency & edge cases

- **Concurrent publishers:** `append` is under a lock, so offsets are unique and gap-free.
- **Readers vs writers:** dispatchers only read under the same lock; messages are immutable records, safe to share.
- **Unbounded memory:** the log grows forever; real brokers apply **retention** (delete segments — the chunks the log is stored in — older than N days or beyond N bytes).
- **Slow subscriber falling far behind:** it is fine here (it just reads older offsets); with retention it may fall off the end — detect "offset out of range" and reset.
- **Subscriber that blocks forever:** add a processing timeout. It stalls only its own subscription, because every subscription has its own dispatcher thread. (One Java 21 caveat: a virtual thread that blocks while inside a `synchronized` block *pins* its carrier platform thread (the real operating-system thread it runs on); many such subscribers could exhaust the carriers. Prefer `ReentrantLock` in subscriber code, or use platform threads.)
- **Duplicates:** at-least-once means subscribers must be idempotent (e.g. remember processed order ids).
- **Shutdown:** `close()` flips `running` and joins each dispatcher (waits for its thread to finish); the 50 ms wait timeout bounds how long that takes.

Extensions & follow-up questions

- **"Consumer groups (share the work)."** Split a topic into **partitions** (several logs); messages with the same key go to the same partition (`hash(key) % n`), keeping per-key order. A **consumer group** is a set of consumers that split one topic's work between them; a group's members each own some partitions; the committed offset is per (group, partition). That is how Kafka scales consumers.
- **"Persist to disk."** Append to segment files; keep an index (a lookup table) from offset to file position; store committed offsets in a special topic (Kafka uses `__consumer_offsets`).

- **"Pull instead of push."** Expose `poll(subscriptionId, maxMessages)` returning messages from the committed offset and a `commit(offset)` call; the client controls its pace (back-pressure for free).
- **"Message filtering."** Subscribe with a predicate (a yes/no test, e.g. only `region = EU`); dispatcher skips non-matching messages but still commits them.
- **"Wildcard topics."** Subscribe to `orders.*` (`*` is a wildcard that matches any name part); the broker matches topic names on creation.
- **"Delayed messages / retry topics."** Re-publish failed messages to `orders.retry.5s` consumed after a delay, instead of blocking the dispatcher with sleeps.

Real-world example

Apache Kafka is exactly this model at scale: topics split into partitions, each an append-only log; consumers track offsets; consumer groups share partitions; ordering is per partition; messages are retained by time/size rather than deleted on read, so services can replay history. **Google Cloud Pub/Sub** and **AWS SNS + SQS** (Amazon's notification and queue services) offer managed fan-out with acknowledgements and **dead-letter** topics/queues. In Java, Spring Kafka's (Spring is the most popular Java application framework) `DeadLetterPublishingRecoverer` publishes a record that keeps failing to a dead-letter topic after the retries are used up — the same idea as our `__dead_letters`. Inside one JVM, Guava's `EventBus` and Spring's `ApplicationEventPublisher` are simple in-process pub-sub.

In the interview

> **30-second answer:** "Each topic is an append-only log; publishing appends and returns an offset. Each subscription keeps a committed offset and has its own dispatcher thread that waits on a condition for new data, calls the subscriber, and commits only after success — at-least-once and in order. Failures retry with backoff, then go to a dead-letter topic. Because messages aren't deleted on read, late subscribers can start at the beginning and anyone can replay by seeking. To scale, split topics into partitions by key and let consumer groups share partitions."

Plan (35–40 min): clarify guarantees (5) → topic log + offsets (6) → subscription + dispatcher + ack (8) → retries + DLQ (dead-letter queue) (5) → thread safety and ordering (6) → partitions / consumer groups / persistence (5).

Follow-ups with answers:

- *"How do you keep a slow consumer from slowing others?"* → One dispatcher per subscription; the log is shared, progress is not.
- *"Exactly-once?"* → Hard; at-least-once + idempotent consumers is the practical answer.
- *"How is order guaranteed?"* → One writer (the lock) assigns offsets; one dispatcher per subscription delivers in offset order. With partitions, order is per partition (per key).

Self-check

1. What is an offset? → A message's position in a topic's log; subscribers remember the next one to read.
2. Commit before or after processing for at-least-once? → After.
3. Why a dead-letter topic? → So one poison message cannot block a subscription forever.
4. Why does replay work here but not in a delete-on-ack queue? → Messages stay in the log; seeking just moves the offset.
5. Why `while` around `Condition.await`? → Spurious wake-ups; re-check the condition every time.
6. How does Kafka let several consumers share one topic's work while keeping order per key? → Partitions by key hash; each partition consumed by one group member.

TL;DR

- **Pub-sub = Observer through a broker**; publishers and subscribers are decoupled.
- **Append-only log + per-subscription offsets** → fan-out, late joiners, replay.
- **One dispatcher per subscription**: isolation + order; **ack after processing** = at-least-once.

- Retries with backoff, then **dead-letter**; subscribers must be **idempotent**.
- Scale: partitions by key, consumer groups, retention, persisted offsets (Kafka).

20.7 Design a Rate Limiter

Problem & clarifying questions

What it is. A rate limiter decides whether a request is allowed right now, so one client can't overwhelm a service (by accident or on purpose). Everyday analogy: a turnstile that lets through at most 10 people per minute per ticket type. Over the limit, the answer is "come back in 6 seconds" — in HTTP, status **429 Too Many Requests** with a `Retry-After` header (a header is a named line of metadata sent with an HTTP response).

Chapter 13 (§13.11) already built the concurrency core: a thread-safe token bucket (a bucket of tokens refilled at a steady rate; each request spends one, and an empty bucket means "wait") with `synchronized` and a lock-free CAS version (CAS = compare-and-set: update a value only if it still holds what you last read, as one hardware-level step, so no lock is needed), and compared the algorithms. This section is about the **object-oriented design around it**: pluggable algorithms, rules per API and per user tier (a plan level such as free or paid), a factory, combined limits, and a clean decision object.

Question	Assumed answer
Limit by what?	Per (user, API). Could also be IP (the caller's network address) or API key (a secret string that identifies the calling app) — same design, different key.
Different limits?	Yes: per API and per tier (FREE vs PREMIUM); a default for everything else.
Algorithms?	Token bucket, fixed window, sliding window log, sliding window counter (all compared in the table under Key flows) — swappable per rule.
Several limits at once?	Yes, e.g. "2 per 10 s **and** 3 per minute" on `/login`.
Response?	Allowed / denied + remaining + retry-after.
Single process or distributed (spread over several machines)?	Single process here; distributed in extensions.

Requirements

Functional

1. `check(userId, api)` → `Decision(allowed, remaining, retryAfter)`.
2. Rules configured per API and tier; fallback default rule.
3. Each (user, api) gets its **own** limiter instance, created on first use.
4. Algorithms pluggable per rule.
5. Several rules can apply together (all must allow).

Non-functional: O(1) (or close) per check; thread-safe (exactly N allowed under any concurrency); injectable clock for tests; memory bounded (discussed).

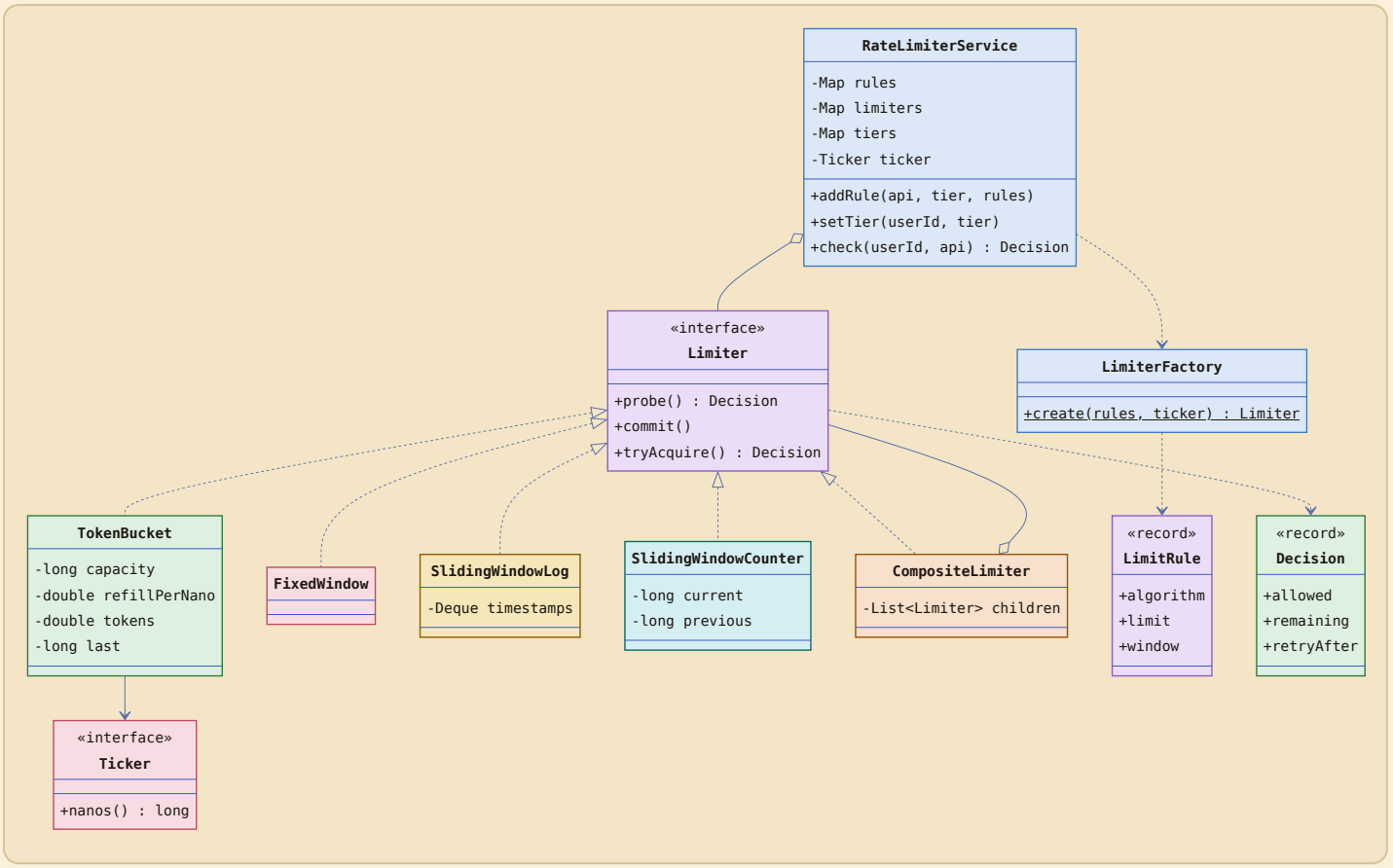
Use cases

Actors: **Client** (sends requests), **API gateway / filter** (the front door that every request passes through; calls `check` before the handler — the code that does the real work), **Operator** (configures rules and tiers).

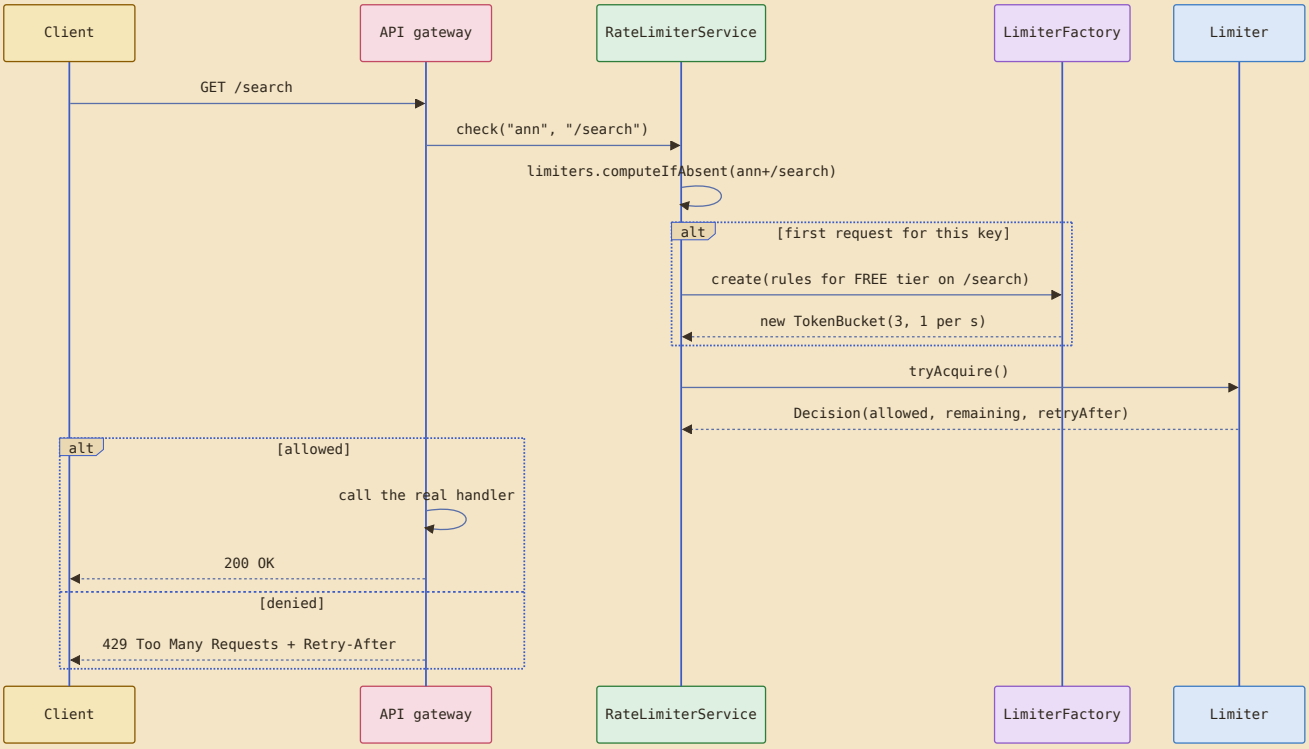
Core entities

Class	Responsibility
`Limiter` (interface)	**Strategy**: `probe()` (would one more request pass? consumes nothing), `commit()` (record it), and `tryAcquire()` = both in one atomic step → `Decision`. One instance per key.
`TokenBucket`, `FixedWindow`, `SlidingWindowLog`, `SlidingWindowCounter`	The algorithms (each `synchronized`).
`CompositeLimiter`	**Composite**: allowed only if every child allows; probes all children, then commits to all, under one lock, so a denied request consumes nothing.
`Decision` (record)	allowed, remaining, retryAfter.
`LimitRule` (record)	algorithm, limit, window.
`Algorithm`, `Tier` (enums)	Configuration values.
`LimiterFactory`	**Simple Factory**: rule(s) → `Limiter` (single or composite).
`RateLimiterService`	**Facade**: resolves the user's tier and the API's rules, keeps one limiter per (user, api) in a `ConcurrentHashMap`.
`Ticker`	Injected time source (nanoseconds).

Class diagram



Key flows



The four algorithms side by side (limit N per window W):

Algorithm	State per key	Bursts	Accuracy	Notes
Token bucket	tokens, last refill time	Up to capacity	Good	Most common for APIs; smooth refill
Fixed window counter	window start, count	**Up to 2N across a boundary**	Poor at edges	Simplest
Sliding window log	one timestamp per allowed request	None beyond N	Exact	Memory O(N) per key
Sliding window counter	2 counters	Approximate	Good	Estimate = previous × overlap + current
(Leaky bucket)	a queue	Smooths output to a constant rate	—	Traffic shaping (evening out the flow); not implemented here

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy (`Limiter`)	Algorithms differ in accuracy/memory; pick per rule. New algorithm = new class.	`switch` on algorithm inside `check`.
Simple Factory (`LimiterFactory`)	One place turns config into objects; service never names concrete classes.	`new TokenBucket(...)` in the service.
Composite (`CompositeLimiter`)	"10/s and 1000/day" is just a limiter made of limiters; callers don't know. **Check all, then commit all**, so a request denied by one child never uses up a slot in another.	Special-casing multiple rules in the service; or calling each child's `tryAcquire` in turn (a later denial would leave earlier children charged for a request that never ran).
Facade (`RateLimiterService`)	Gateway code asks one question.	Gateway juggling tiers, rules and limiters.
One limiter per key via `computeIfAbsent`	Exactly one limiter per (user, api), even when the first two requests race.	`get` then `put` (two limiters could be created and one's counts lost).
`Decision` with `remaining` and `retryAfter`	Maps directly to HTTP headers (`X-RateLimit-Remaining`, `Retry-After`).	`boolean`.
Injected `Ticker`	Tests move time instantly (the demo never sleeps).	`System.nanoTime()` inside algorithms.
Each algorithm `synchronized`; `tryAcquire` = `probe` + `commit` under the limiter's lock	Refill + check + take must be one atomic step (see §13.11 for the lock-free version).	Unsynchronized fields (two threads take the last token).

Java implementation

Files: `Limiters.java` (ticker, decision, `Limiter` interface, four algorithms, composite), `RateLimiterService.java` (tiers, rules, factory, service), `Main.java`. The full runnable source is in `code/lld/ch20/ratelimiter/`; below are the lines that carry the design.

```java
// File: code/lld/ch20/ratelimiter/Limiters.java
/** Allowed or not, what is left, and when to retry (HTTP 429 Retry-After). */
record Decision(boolean allowed, long remaining, Duration retryAfter) { }

/** Strategy, one instance per key. probe() answers without using anything up; commit() records. */
interface Limiter {
    Decision probe();
    void commit();
    /** Check and take in ONE atomic step (probe + commit under this limiter's lock). */
    default Decision tryAcquire() {
        synchronized (this) {
            Decision d = probe();
            if (d.allowed()) commit();
            return d;
        }
    }
}

/** Token bucket: bursts up to 'capacity', refills at 'perSecond'. Lazy refill, no background thread. */
@Override
public synchronized Decision probe() {
    long now = ticker.nanos();
    tokens = Math.min(capacity, tokens + (now - last) * refillPerNano); // refilling is safe to do early
    last = now;
    if (tokens >= 1) return Decision.allow((long) (tokens - 1));
    long waitNanos = refillPerNano == 0 ? Long.MAX_VALUE : (long) Math.ceil((1 - tokens) / refillPerNano);
    return Decision.deny(Duration.ofNanos(waitNanos));
}
@Override
public synchronized void commit() { tokens -= 1; }

/** Sliding window counter: weights the previous window by how much of it still overlaps. */
@Override
public synchronized Decision probe() {
    long now = ticker.nanos();
    long start = now - Math.floorMod(now, windowNanos);
    if (start != currentStart) {
        previous = (start - currentStart == windowNanos) ? current : 0; // older than one window: forget it
        current = 0;
        currentStart = start;
    }
    double overlap = 1.0 - (double) (now - currentStart) / windowNanos;
    double estimate = previous * overlap + current;
    if (estimate + 1 <= limit) return Decision.allow((long) (limit - estimate - 1));
    return Decision.deny(Duration.ofNanos(currentStart + windowNanos - now));
}
// FixedWindow: count per window start (now - now mod W). SlidingWindowLog: a Deque of timestamps.

/** Composite: allowed only if EVERY child allows; a denied request uses up NOTHING. */
final class CompositeLimiter implements Limiter {
    @Override
    public synchronized Decision probe() {
        long remaining = Long.MAX_VALUE;
        for (Limiter l : children) {
            Decision d;
            synchronized (l) { d = l.probe(); }
            if (!d.allowed()) return d;           // first denial wins; nothing has been consumed
            remaining = Math.min(remaining, d.remaining());
        }
        return Decision.allow(remaining);
    }
    @Override
    public synchronized void commit() {
        for (Limiter l : children) synchronized (l) { l.commit(); }
    }
}

// File: code/lld/ch20/ratelimiter/RateLimiterService.java
// LimiterFactory.create(rules): one rule -> its algorithm; several -> a CompositeLimiter of fresh children.
Decision check(String userId, String api) {
```

```
    // computeIfAbsent creates exactly ONE limiter per (user, api), even under a race
    Limiter limiter = limiters.computeIfAbsent(new Key(userId, api), k -> {
        Tier tier = tiers.getOrDefault(userId, Tier.FREE);
        List<LimitRule> r = rules.getOrDefault(api, Map.of()).getOrDefault(tier, defaultRules);
        return LimiterFactory.create(r, ticker);
    });
    return limiter.tryAcquire();
}
```

Walking through the code.

- `TokenBucket` refills **lazily**: on each call it adds `elapsed × rate` tokens (capped at capacity). No background thread. When empty, `retryAfter` is the time until one whole token exists.
- `FixedWindow` computes the current window start with `now − (now mod W)` and resets the count when the window changes.
- `SlidingWindowLog` drops timestamps older than `now − W` and allows if fewer than N remain.
- `SlidingWindowCounter` estimates the count in the sliding window as `previous × (portion of previous window still inside) + current`.
- `Limiter` is split into `probe()` (answer only) and `commit()` (record); the default `tryAcquire()` runs both inside `synchronized (this)`, and every algorithm's own methods are `synchronized` on the same object, so check-and-take stays one atomic step.
- `CompositeLimiter` probes **every** child first and returns the first denial without committing anything; only if all allow does it commit to all of them. Its children are created for it alone by the factory and never shared, so the composite's lock makes the whole probe-then-commit atomic. `remaining` is the smallest of the children's.
- `RateLimiterService.check` builds the limiter inside `computeIfAbsent`, so the tier and rule are resolved once per key.

Output:

```
== 1. Per-user, per-API token bucket (FREE: burst 3, 1 token/s)
  t=0.0 ann /search #1 -> 200 OK (remaining 2)
  t=0.0 ann /search #2 -> 200 OK (remaining 1)
  t=0.0 ann /search #3 -> 200 OK (remaining 0)
  t=0.0 ann /search #4 -> 429 Too Many Requests (retry after 1000 ms)
  t=0.0 ann /search #5 -> 429 Too Many Requests (retry after 1000 ms)
  t=1.0 ann /search    -> 200 OK (remaining 0)
  t=1.0 bob /search    -> 200 OK (remaining 2)  (own bucket)
  t=1.0 pat (PREMIUM) 12 calls -> 10 allowed
  t=1.0 ann /orders (default rule) -> 200 OK (remaining 99)
== 2. Composite rule on /login: 2 per 10 s AND 3 per minute
  t=+0s ann /login -> 200 OK (remaining 1)
  t=+1s ann /login -> 200 OK (remaining 0)
  t=+2s ann /login -> 429 Too Many Requests (retry after 8000 ms)
  t=+11s ann /login -> 200 OK (remaining 0)
  t=+12s ann /login -> 429 Too Many Requests (retry after 48000 ms)
  t=+61s ann /login -> 200 OK (remaining 1)
== 3. A denied request uses up nothing: 3 per minute AND 2 per 10 s (minute rule first)
  t=+0s -> 200 OK (remaining 1)
  t=+1s -> 200 OK (remaining 0)
  t=+2s -> 429 Too Many Requests (retry after 8000 ms)
  t=+11s -> 200 OK (remaining 0)
== 4. Boundary burst: limit 5 per 10 s, 5 calls at t=9.9 and 5 at t=10.1
  FIXED_WINDOW     allowed 10 of 10 within 0.2 s
  SLIDING_LOG      allowed 5 of 10 within 0.2 s
  SLIDING_COUNTER  allowed 5 of 10 within 0.2 s
== 5. 64 threads x 10 calls on one bucket (capacity 50, no refill)
  allowed = 50 (exactly 50, never more)
```

Step 1: ann's burst of 3 is allowed, then 429 with "retry after 1000 ms"; a second later exactly one more is allowed; bob and the PREMIUM user have their own buckets. Step 2: the /login composite denies at +2s (10-second rule) and at +12s (1-minute rule, retry when the +0s call leaves the minute window). Step 3: the denied +2s call consumed nothing, so the +11s call passes both rules. Step 4 is the classic interview point: **fixed window lets 10 requests through in 0.2 seconds** across the boundary, while both

sliding algorithms hold the line at 5. Step 5 proves thread safety: 640 concurrent attempts, exactly 50 allowed.

Concurrency & edge cases

- **Atomicity:** each limiter's `tryAcquire` is `synchronized`; the service map is a `ConcurrentHashMap`. Contention is per key, so different users never block each other.
- **Composite and denied requests.** The naive composite calls each child's `tryAcquire()` in turn. If child 1 allows (and consumes) and child 2 then denies, the request never runs but child 1 has been charged for it. Step 3 of the demo is built to expose that: with "3 per minute" first, the denied call at +2s would have used up the third per-minute slot, and the legitimate call at +11s would be refused for about 49 s. Our composite probes all children and commits only if all allow, so +11s is correctly allowed. (An alternative is *acquire, and refund on denial*, which needs a `release()` in every algorithm and is easy to get wrong when a window rolls over in between.)
- **Memory growth:** one limiter per (user, api) forever. Evict idle limiters (e.g. store them in a cache with expire-after-access, like 20.3). Set the idle timeout to at least the longest window (or the full-refill time of a bucket): after that much idle time the old limiter would allow a full burst anyway, so replacing it with a fresh one changes nothing.
- **Clock:** use a monotonic clock (`System.nanoTime`) — wall-clock time (the calendar time a person reads) can jump backwards.
- **Rule changes at runtime:** existing limiters keep old settings; clear the map (or version the rules) when config changes.

Extensions & follow-up questions

- **"Distributed (many gateway servers)."** Keep counters in a shared store: Redis with an atomic Lua script (Redis can run small Lua programs as one uninterrupted step) (or `INCR` + `EXPIRE` for fixed windows: Redis commands for "add one" and "delete after N seconds"). Trade-off: a network hop (one extra round trip to another machine) per request; mitigate with local pre-allocation of tokens. Or use sticky routing (the load balancer — the component that spreads requests over servers — always sends one user to the same server) so each user always hits the same node.
- **"Leaky bucket."** A bounded queue drained at a constant rate by a worker: smooths bursts into a steady output (good for protecting a fragile downstream — a service that this one calls).
- **"Different cost per request."** `tryAcquire(permits)`: an expensive export costs 10 tokens.
- **"Soft limits / warnings."** Return `remaining` and warn the client at 80%.
- **"Rate limit by IP when not logged in."** The key is just a string; the rule resolver decides which key to use.
- **"Where does the limiter run?"** In an API gateway or a servlet filter / Spring `HandlerInterceptor` (Java web hooks that run before every request) before the controller (the class that handles a given URL).

Real-world example

Stripe has written publicly about using several limiters together ("Scaling your API with rate limiters": a request-rate limiter based on the token bucket, a concurrent-requests limiter, and load shedders — components that deliberately drop low-priority work when overloaded — implemented with Redis) — several independent checks on one request, the composite idea. **GitHub's API** returns `X-RateLimit-Limit`, `X-RateLimit-Remaining` and `X-RateLimit-Reset` headers, which is our `Decision`. HTTP 429 is defined in RFC 6585 (RFCs are the official documents that define internet standards). **NGINX** (a widely used web server and reverse proxy) `limit_req` uses the leaky-bucket method. In Java, **Guava's** `RateLimiter`, **Resilience4j's** `RateLimiter` and **Bucket4j** (token bucket) are common libraries. Cloudflare has described using the sliding-window-counter approximation (weighting the previous window) to rate limit at the edge (its servers closest to users) cheaply.

In the interview

> **30-second answer:** "A `Limiter` interface with interchangeable algorithms — token bucket by default, sliding window when exactness matters. A factory builds limiters from rules; rules are chosen by

API and user tier; a composite limiter combines rules like '10 per second and 1000 per day'. The service keeps one limiter per (user, api) in a `ConcurrentHashMap` via `computeIfAbsent`, and returns a decision with remaining and retry-after for the 429 response. Each limiter makes refill-check-take atomic; for multiple servers, move the state into Redis with an atomic script."

Plan (30–40 min): clarify key/limits (4) → algorithms table and choice (8) → OO design: interface, factory, service, composite (10) → thread safety (5, point to §13.11) → distributed + memory (5).

Follow-ups with answers:

- *"Why is fixed window risky?"* → Up to 2N requests across a window boundary (shown in the output).
- *"Token bucket vs sliding log?"* → Bucket allows bursts, O(1) memory; log is exact but stores every timestamp.
- *"How do you avoid creating two limiters for the same user?"* → `ConcurrentHashMap.computeIfAbsent`.

Self-check

1. Which algorithm allows a burst up to its capacity and then a steady rate? → Token bucket.
2. How many requests can a fixed window of limit 5 let through in 0.2 s across a boundary? → Up to 10.
3. Which pattern lets "2 per 10 s AND 3 per minute" look like one limiter? → Composite.
4. What HTTP status and header go with a denial? → 429 and `Retry-After`.
5. Why must refill + check + take be atomic? → Otherwise two threads can both see the last token and both take it.
6. How do you keep memory bounded with millions of users? → Evict limiters idle for at least the longest window (expire-after-access cache); by then a fresh limiter behaves exactly like the old one.
7. Why does `CompositeLimiter` probe every child before committing to any? → So a request denied by one rule does not use up a slot in another rule.

TL;DR

- **Strategy** for algorithms, **Simple Factory** from rules, **Composite** for combined limits (probe all, then commit all), **Facade** service.
- One limiter per (user, api) via `computeIfAbsent`; `Decision` → 200 (HTTP "OK") or 429 + Retry-After.
- Token bucket (bursty, O(1)), fixed window (2× edge burst), sliding log (exact, O(N)), sliding counter (approximate, O(1)).
- Atomic refill-check-take (§13.11); Redis + Lua for distributed limits.

20.8 Chapter summary

Component	Anchor pattern(s)	The one sentence to remember
Splitwise	Strategy	Split rules return shares; the ledger stores one signed number per pair; simplify with net positions + greedy (≤ n − 1 payments).
Ledger Co. (full case in Ch 21 §21.6)	—	A loan ledger, not a bill-splitting ledger: same "one authoritative record + `BigDecimal`" idea, different rules.
Notification service	Template Method + Strategy + Observer	Retry once in the parent, one class per reason not to send, one exit that tells the listeners.
In-memory cache	Strategy + Observer	HashMap + doubly linked list; eviction policy owns the order; one removal path.
LRU cache	— (data structure)	Map to node, move to front on use, evict before tail; or `LinkedHashMap(accessOrder=true)`.
Ride sharing	Strategy + Decorator + state machine	Ranked list for declines, surge wraps pricing, atomic driver claim.
Carpool variant	Strategy	Offer seats per vehicle; select with Most Vacant / Preferred Vehicle.
Logging framework	Chain of Responsibility + Singleton + Decorator	Hierarchy with inherited levels; propagate up until additivity stops; async is a wrapper.
Pub-sub	Observer via broker	Append-only log + per-subscription offsets; ack after processing; dead letters.
Rate limiter	Strategy + Factory + Composite	One limiter per key; algorithms swappable; 429 with Retry-After.

The common thread. In every design, the part most likely to change sits behind an interface, and the "service" class only orders the steps. When the interviewer says "now add X", the best answer is a new class and a one-line change where objects are wired together.

20.9 Chapter self-check

1. Which two patterns appear in more than half of this chapter's designs? → Strategy and Observer.
2. Strategy vs Decorator in one line each? → Strategy *replaces* a rule; Decorator *wraps* one and adds to its result.
3. Strategy vs Template Method? → Strategy swaps the whole algorithm (composition); Template Method fixes the algorithm and lets subclasses fill steps (inheritance).
4. Which designs used a "single exit" method, and why? → Notification (`finish`) and cache (`remove`): so records, stats and listeners can never be skipped on one path.
5. Where did we call listeners outside a lock, and why? → The cache: a slow or re-entrant listener must not block or deadlock it.
6. What does at-least-once delivery require from consumers? → Idempotency.
7. Name three places this chapter used compare-and-set or `computeIfAbsent` for correctness. → Driver claim (`AtomicBoolean`), subscription commit (`AtomicLong.compareAndSet`), limiter creation (`computeIfAbsent`).

20.10 Chapter TL;DR

- Components are small, so interviews probe the **rule most likely to change** — put it behind an interface.

- **Strategy** everywhere (splits, eviction, matching, pricing, send rules, formatters, rate-limit algorithms); **Observer** for "tell someone when it happens"; **Decorator** to add behaviour without editing (surge, async); **Template Method** for a shared algorithm with one varying step; **Chain of Responsibility** for the logger hierarchy; **Composite** for combined limits.
- Money in `BigDecimal`; time from an injected clock; thread safety with the smallest correct tool (one lock, CAS, `computeIfAbsent`, per-key locks, sharding).
- Always volunteer the honest limits: greedy is not optimal, LRU needs a write lock, at-least-once duplicates, fixed windows burst at edges.